參考用書：Introduction to Algorithms, 3rd Edition
Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein

演算法

完整版講義

A4 直式 · 上下兩張 · Lin Cing

目錄

一、演算法分析與正確性

演算法分析與正確性 3

二、遞迴、遞迴式與分治

遞迴、遞迴式與分治 29

三、排序、選擇與隨機分析

排序、選擇與隨機分析 60

四、資料結構與攤銷分析

資料結構與攤銷分析 137

五、動態規劃

動態規劃 252

六、貪心方法

16 貪婪演算法 291

七、圖的走訪與連通

22 基礎圖演算法 303

八、最小生成樹與最短路徑

不相交集合：只合併，不拆分 318

23 最小生成樹 338

24 單源最短路徑 352

25 全點對最短路徑 367

九、網路流

26 最大流 380

十、計算幾何

33 計算幾何 404

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

演算法

完整版講義

A4 直式 · 上下兩張 · Lin Cing

目錄

十一、計算複雜度與近似

34 NP 完備性 419

35 近似演算法 444

十二、字串比對

32 字串匹配 462

十三、矩陣運算與 FFT

28 矩陣運算 476

30 多項式與快速傅立葉轉換 489

十四、數論演算法

31 數論演算法 500

十五、線性規劃

29 線性規劃 523

十六、平行演算法

27 多執行緒演算法 542

查用附錄

A 求和 566

B 集合、關係、函數、圖與樹 576

C 計數與機率 590

D 矩陣 607

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

演算法分析與正確性

把程式跑完，只回答了「這次發生什麼」。分析演算法還要回答：**所有合法輸入都會得到正確結果嗎？輸入變大時，成本如何增加？**

本章依序建立規格與證明、逐步計數、漸近比較、級數與迴圈分析。插入排序會作為共同案例，從每次搬移一路走到一般成本公式。

先備知識：陣列、迴圈、基本代數與對數；面積與極限推導另用到基礎微積分。

參考用書：Cormen 等，*Introduction to Algorithms*, 3rd Edition，Chapters 1–3；求和參閱 Appendix A。

閱讀路線

1. 規格與終止性 → 線性搜尋的不變量 → 插入排序的完整分析。
2. 漸近符號與量詞 → 比較成長 → 用級數分析迴圈。
3. 整數邊界與空間 → 成本式的限制 → 數學延伸。

先走完主線，再讀延伸；不熟悉的符號可回查本章定義。

先寫規格，再看程式

問題由合法輸入集合 D 與輸出關係 $R(x)$ 描述。輸入 $x \in D$ 可以有不只一個合法答案；演算法則是產生答案的明確程序。

排序的輸入是一列資料，每筆有比較用的鍵值與固定身分。例如 $(3, a), (1, b), (3, c)$ 中，a, b, c 用來辨識原資料，不是額外排序依據。

輸出要求	檢查方式
鍵值非遞減	相鄰鍵值皆滿足 $A[i].key \le A[i+1].key$
不遺失、不複製資料	輸出是輸入資料的排列，重複次數也相同
若要求穩定排序	相等鍵值維持原來的先後

只有「有序」仍不夠：刪光資料後的空序列雖有序，卻不是原輸入的排序結果。

重複值要靠身分辨認

輸入 [3a, 1b, 3c] → 鍵值排序；字母表示資料身分

有序、保存、穩定	有序、保存	有序，但遺失資料
1b 3a 3c	1b 3c 3a	1b 3a
三項要求全部成立	等鍵 a、c 的先後反轉	少了 3c，不是排序結果

穩定性是額外要求。若題目只要求有序且保存全部資料，交換兩筆相等鍵值不會使答案錯誤；若要求穩定，就必須保留 a 在 c 前面。

空輸入與單筆輸入都已排序；演算法仍須合法處理，不能讀取不存在的第一筆。

正確結果與一定停下，是兩件事

部分正確性（partial correctness）：只要演算法終止，輸出就符合規格。完全正確性（total correctness）還要求每個合法輸入都會終止。

```
while true
    skip
```

這段程式從不交出錯誤答案，也從不交出答案。若只證「終止後正確」，還不能說它完成任務。

證明迴圈會停，常找一個非負整數量：每次迭代嚴格下降，且不會降到負數。有限行程式、有限測試案例，都不能代替這項論證。

線性搜尋：先固定返回哪一個

給陣列 $A[1..n]$ 與目標 x，返回**第一個**符合 $A[i] = x$ 的索引；若沒有則返回 NIL。允許 $n = 0$，比較與讀取一筆資料各為常數成本。

```
FIRST-MATCH(A, x)
    i = 1
    while i <= n
        if A[i] == x: return i
        i = i + 1
    return NIL
```

本章陣列索引從 1 開始；$A[a..b]$ 在 $a > b$ 時表示空區間，不會讀取元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

陣列參照與資料身分，要分開理解

本章偽碼以參照存取陣列。把陣列傳給程序後，修改 $A[i]$ 會改到呼叫端看見的資料；只把程序內的變數重新綁到另一個陣列，則不會改綁呼叫端的變數。

操作	結果
$A = [3, 1]$；令 $B = A$；再令 $B[1] = 8$	兩個名稱指向同一陣列，A 也變成 $[8, 1]$
$A = [3, 1]$；令 $B = copy(A)$；再令 $B[1] = 8$	陣列儲存體不同，A 仍是 $[3, 1]$

陣列複製若只複製物件參照，兩個陣列仍可能指向同一批物件；「陣列獨立」不表示「每個物件都深層複製」。排序中標記的身分 a, b, c 則用來辨識資料，不能直接當作其記憶體位址。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋的不變量，記錄已排除的範圍

每次判斷 `i <= n` 前，維持：$1 \le i \le n+1$，而 $A[1..i-1]$ 都不等於 x。

階段	理由
初始化	$i = 1$，已排除範圍為空，條件成立
維護	若本筆不等於 x，令 $i \leftarrow i + 1$；剛檢查的位置加入排除範圍
找到時	$A[i] = x$，且之前沒有匹配，所以返回的是第一個
掃完時	$i = n + 1$，所有位置都已排除，因此返回 NIL

未返回的每次迭代都讓非負整數 $n - i + 1$ 減 1，因此必停。$n = 0$ 時直接返回 NIL，不讀取 $A[1]$。

i 之前已排除，i 之後還不能下結論

在 $A = [6, 2, 6, 5]$ 搜尋 5。到 $i = 3$ 前，只能斷言前兩筆不匹配；即使看見第 3 筆是 6，也要實際比較後才能擴大排除範圍。

到 $i = 4$ 時找到 5，而前段已證明沒有 5，所以返回最早匹配。圖中的「尚未檢查」不等於「不匹配」。

同一段程式，三種完整執行

輸入與目標	依序比較的位置	返回值	鍵值比較次數
$[4, 2, 4]$，找 4	1	1	1
$[4, 2, 4]$，找 7	1、2、3	NIL	3
空陣列，找 4	無	NIL	0

三例分別檢查最早匹配、搜尋失敗與空輸入；一般正確性仍由不變量證明。

若成功位置均勻分布在 $1..n$，平均比較數為 $(n+1)/2$；若輸入分布不同，不能沿用這個數字。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

計時之前，先說明在數什麼

本章先採 word-RAM 模型：索引與資料可放入固定數量的機器字；讀寫、比較與模型允許的基本算術各計常數成本。這是分析的約定，不是所有實際運算都一樣快。

觀察對象	常見成本單位
陣列搜尋	鍵值比較次數、讀取次數
排序	比較次數、搬移次數、額外空間
大整數演算法	位元數與多字運算成本
外部儲存	區塊傳輸次數

若整數含 b 個位元，任意精度乘法不能不加說明就視為一次常數操作。後面的結論都要連同成本模型閱讀。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

輸入大小不一定是數值本身

正整數 N 的二進位長度是 $b = \lfloor \lg N \rfloor + 1$，其中 $\lg = \log_2$。以 N 為次數的迴圈，對輸入長度 b 可能是指數成本。

例如 $N = 13$ 寫成 `1101`，只需 4 個位元。若逐一試到 N，迭代數為 $\Theta(N)$；對 b 位數輸入，$2^{b-1} \le N < 2^b$，所以是 $\Theta(2^b)$ 次。

陣列通常以元素數 n 為規模；圖常同時用頂點數 V、邊數 E。把二參數成本寫成 $O(V + E)$，比默默把所有東西叫作 n 更清楚。

最好、最壞、平均：先選成本函數

對長度為 n 的輸入集合 D_n，令 $C(x)$ 為該輸入的成本。

$$B(n) = \min_{x \in D_n} C(x), \qquad W(n) = \max_{x \in D_n} C(x).$$

平均成本還需要指定機率分布 P_n：

$$A(n) = \mathbb{E}_{x \sim P_n}[C(x)] = \sum_{x \in D_n} P_n(x) C(x).$$

Big-O 並不等於最壞情況。 B, W, A 都是函數，都能各自求上界或下界。隨機演算法還要區分輸入的隨機性與演算法自己擲出的隨機位元。

插入排序：把下一筆放進已排序區

輸入 $[(7,a),(2,b),(5,c),(2,d),(1,e)]$，簡記為 $[7a, 2b, 5c, 2d, 1e]$。每輪把一筆資料暫存為 `key`，只把左邊較大的資料往右搬。

```
INSERTION-SORT(A)
    for j = 2 to n
        key = A[j]
        i = j - 1
        while i >= 1 and A[i].key > key.key
            A[i+1] = A[i]
            i = i - 1
        A[i+1] = key
```

`and` 由左至右短路求值；$i = 0$ 時不讀取 $A[i]$。$n \le 1$ 時外層不執行。比較採嚴格大於，讓舊的相等鍵留在新資料前面。

每一輪都留下可以核對的結果

j／暫存資料	由右向左的鍵值比較	右移數 h_j	填入後的陣列
2／$2b$	$7 > 2$；到邊界	1	$2b, 7a, 5c, 2d, 1e$
3／$5c$	$7 > 5$、$2 > 5$ 為假	1	$2b, 5c, 7a, 2d, 1e$
4／$2d$	$7 > 2, 5 > 2$、$2 > 2$ 為假	2	$2b, 2d, 5c, 7a, 1e$
5／$1e$	$7, 5, 2d, 2b$ 都大於 1	4	$1e, 2b, 2d, 5c, 7a$

最後 $2b$ 仍在 $2d$ 前面。共右移 8 次，但鍵值比較是 10 次：第 3、4 輪各多一次失敗比較；到 $i = 0$ 的第 2、5 輪不再比較鍵值。

搬移時的重複值，用「洞」來理解

空格是**邏輯洞**，不是寫入的空值。每步由「洞外位置＋暫存 key」保存原資料；第一次搬移後，實體陣列有兩份 $7a$，所以不能把整段實體陣列當成原輸入的排列。

外層證明：有序、保存、穩定一起維持

令 S 為開始排序前的輸入副本。每輪開始時：$A[1..j-1]$ 是 $S[1..j-1]$ 的穩定排序，且 $A[j..n] = S[j..n]$。

階段	不變量如何成立
初始化 $j = 2$	單筆前段已排序，後段未改動
維護	右移所有比新鍵大的尾段，再填入新資料；等鍵不跨越，後段不變
終止 $j = n + 1$	前段就是全部輸入，故全部已穩定排序

內層的 i 每次減 1，最多搬 $j - 1$ 筆；外層最多 $n - 1$ 輪，所以必定終止。$n \le 1$ 另外由不執行迴圈成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

內層證明：洞的位置也要寫進條件

固定一輪 j，令 $B = A[1..j]$ 為該輪開始的副本，$B[1..j-1]$ 已排序。暫存 $key = B[j]$，令洞在 $i + 1$。

每次內層判斷前：

1. 左段 $A[1..i] = B[1..i]$。
2. 右段 $A[i+2..j] = B[i+1..j-1]$，而這些鍵都大於 key。
3. 不計洞內舊值，再加上 key，恰好保存 B 的每筆資料。

初始化右段為空；搬移 $A[i]$ 到洞再令 $i \leftarrow i - 1$，三項仍成立。停止時左段皆不大於 key，右段皆大於它，填洞便得到有序且保存資料的前段。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把比較、搬移與迴圈判斷分開計數

對 $n \geq 2$，第 j 輪右移 h_j 次；若停在某個 $i \geq 1$ 的失敗鍵值比較，令 $q_j = 1$，否則令 $q_j = 0$。

計數對象	第 j 輪	全程
右移寫入	h_j	$\sum_{j=2}^{n} h_j$
最後填入 key 的寫入	1	$n-1$
內層完整 guard 判斷	$h_j + 1$	$\sum h_j + n - 1$
鍵值比較	$h_j + q_j$	$\sum h_j + \sum q_j$

上例 $\sum h_j = 8, \sum q_j = 2$，所以比較 10 次、guard 判斷 12 次、陣列寫入 12 次。暫存 key 的讀取不算在「陣列寫入」裡。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼右移數恰好等於逆序對數？

逆序對（inversion）是原輸入中滿足 $p < q$ 且 $S[p].key > S[q].key$ 的位置對。令總數為 $I(S)$。

插入原來第 j 筆時，左側是前 $j-1$ 筆的排序；被右移的恰是其中鍵值大於 $S[j]$ 的資料。因此本輪 h_j 等於以 j 為右端點的逆序對數。

每個逆序對有唯一右端點，所以

$$\sum_{j=2}^{n} h_j = I(S), \qquad T(S) = \Theta(n + I(S)).$$

等鍵不算逆序。上例依右端點分別有 $1, 1, 2, 4$ 對，共 8 對，與右移表一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一演算法的最好、最壞與平均

輸入條件	逆序對數	時間成本
已非遞減排序	0	$\Theta(n)$
相異鍵嚴格遞減	$n(n-1)/2$	$\Theta(n^2)$
相異鍵的所有排列等機率	期望 $n(n-1)/4$	期望 $\Theta(n^2)$

平均情況的理由：對每對位置 $p < q$，令指示變數 X_{pq} 表示是否形成逆序。在均勻排列模型下，$\mathbb{E}[X_{pq}] = 1/2$；線性期望給

$$\mathbb{E}[I] = \sum_{p<q} \mathbb{E}[X_{pq}] = \binom{n}{2}\frac{1}{2}.$$

不需要假設各 X_{pq} 獨立。含重複鍵或非均勻輸入時，必須重新檢查分布。

漸近分析：忽略什麼，保留什麼？

我們關心 $n \to \infty$ 時的成長階，允許忽略固定常數倍率與有限多個小輸入。這不代表小輸入的正確性可以忽略。

以下 n 取正整數，f 最終非負，g 最終嚴格正。$O(g), \Omega(g), \Theta(g)$ 都是**函數集合**；$f = O(g)$ 是常用寫法，意思是 $f \in O(g)$。

例如 $n \in O(n^2)$ 正確，只是上界不緊。若要表示已找到上下同階的界，應寫 Θ，不能把所有 Big-O 都讀成「恰好」。

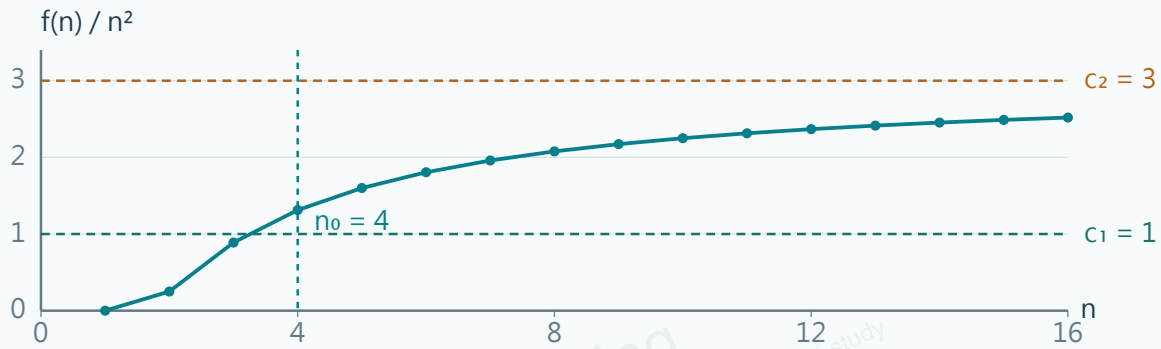
上界、下界與緊界的量詞

記號	正式條件：對所有整數 $n \geq n_0$
$f \in O(g)$	存在 $c > 0, n_0$，使 $0 \leq f(n) \leq cg(n)$
$f \in \Omega(g)$	存在 $c > 0, n_0$，使 $0 \leq cg(n) \leq f(n)$
$f \in \Theta(g)$	存在 $c_1, c_2 > 0, n_0$，使 $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$

常數可以依固定的 f, g 而定，**不能依輸入大小 n 改變**。門檻不必最小；只要整段尾端都滿足條件即可。

要否定 $f \in O(g)$，必須對任意候選 c, n_0，都找得到 $n \geq n_0$ 使 $f(n) > cg(n)$；打敗一組猜測常數還不夠。

把常數與門檻畫在同一張圖



n ≥ 4 時整段比值落在 1 與 3 之間；不要求 n₀ 以前也成立。

取 $f(n) = 3n^2 - 8n + 5$，$n \geq 4$ 時，$f(n)/n^2 = 3 - 8/n + 5/n^2 \geq 1$；$n \geq 1$ 時 $f(n) \leq 3n^2$。

因此 $c_1 = 1, c_2 = 3, n_0 = 4$ 可證 $f \in \Theta(n^2)$。圖顯示比值的位置；涵蓋所有 $n \geq 4$ 的理由是上述不等式，不是有限採樣點。

同一個函數，怎麼證明不是線性上界？

已知 $f(n) = 3n^2 - 8n + 5 \geq n^2$ 對 $n \geq 4$ 成立。假設它也在 $O(n)$，則存在固定 c, n_0，使 $f(n) \leq cn$ 對所有 $n \geq n_0$ 成立。

選整數 $n > \max\{4, n_0, c\}$，便得到

$$n^2 \leq f(n) \leq cn \quad \Longrightarrow \quad n \leq c,$$

與 $n > c$ 矛盾。因此 $f \notin O(n)$。

這種證明的重點是：先讓對方任選常數，再選足夠大的輸入。不是先固定一個 n，再觀察某組常數不適用。

兩側的門檻不同，取較大的那一個

若 $f \in O(g)$ 的證據是 (c_U, N_U)，$f \in \Omega(g)$ 的證據是 (c_L, N_L)，取 $n_0 = \max(N_U, N_L)$，兩側就同時成立。因此

$$\Theta(g) = O(g) \cap \Omega(g).$$

其他常用推論也要保留條件：

性質	理由或條件
O 的遞移性	兩個倍率相乘、兩個門檻取最大
$f \in O(g) \Leftrightarrow g \in \Omega(f)$	把同一不等式反向解讀，倍率取倒數
$f + g = \Theta(\max(f, g))$	非負時 $\max(f,g) \le f+g \le 2\max(f,g)$

若允許正負相消，最後一項不能直接套用，例如 $n + (-n) = 0$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

little-o：每個倍率都能壓過它

$$f \in o(g) \iff \forall c > 0\ \exists n_0(c)\ \forall n \ge n_0(c):\ 0 \le f(n) < cg(n).$$

Big-O 只要求找到一個倍率；little-o 要求**任何正倍率**最終都成立。例如 $7n \in o(n^2)$：給任意 $c > 0$，取整數 $n_0 > 7/c$ 即可。

同理，$f \in \omega(g)$ 要求對每個 $c > 0$，最終都有 $0 \le cg(n) < f(n)$。因此 $n^2 \in \omega(7n)$。

在最終為正的條件下，$f \in o(g)$ 等價於 $f/g \to 0$，$f \in \omega(g)$ 等價於 $f/g \to \infty$。$7n^2$ 與 n^2 同階，不是 little-o。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

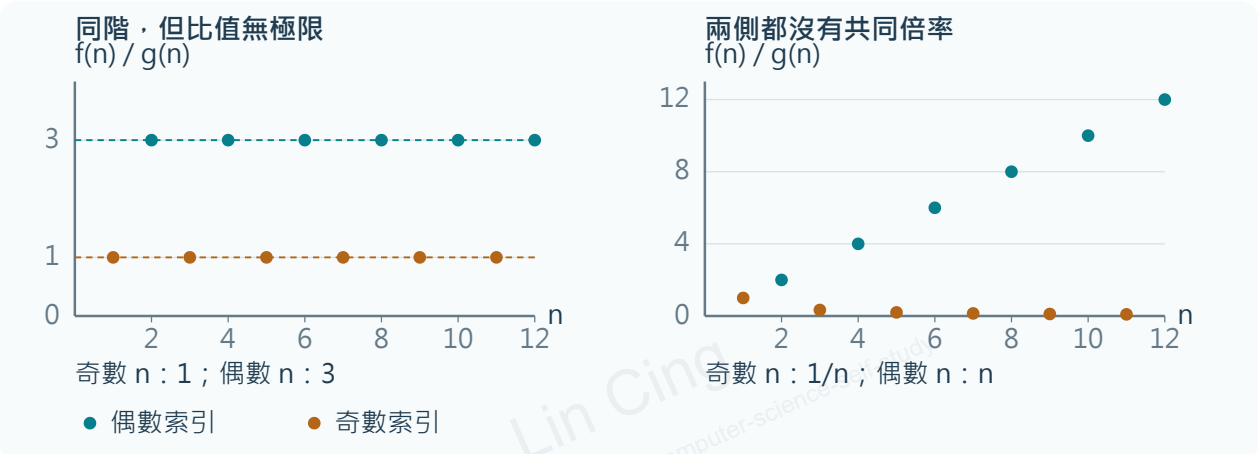
比值極限是工具，不是唯一判準

$\lim f(n)/g(n)$	可以得到的結論
0	$f \in o(g)$
有限正數 L	$f \in \Theta(g)$
$+\infty$	$f \in \omega(g)$
不存在	必須繼續分析，不能直接判定不可比較

有限正 L 的理由：取 $\varepsilon = L/2$，最終有 $L/2 < f/g < 3L/2$，所以兩側都能用固定倍率夾住。

只有比值存在時才使用前三列。對數法也需檢查定義域；$\lg f / \lg g \to 1$ 本身**不保證** $f = \Theta(g)$。

沒有極限，也可能同階



左圖 $f = n(2 + (-1)^n), g = n$：$n \le f \le 3n$，比值雖無極限仍同階。右圖偶數取 $f = n^2$、奇數取 $f = 1$，$g = n$：偶數比值無界，否定 $O(g)$；奇數比值趨 0，否定 $\Omega(g)$。

函數界與輸入界，不要交換量詞

令 $C(x)$ 為輸入成本，$W(n) = \max_{x \in D_n} C(x)$。若 $W(n) \in O(n^2)$，表示存在**共同**常數 c, n_0，對所有夠大的 n、所有 $x \in D_n$，都有 $C(x) \le cn^2$。

若只說「對每個輸入都找得到一個常數」，而常數可以隨輸入增大，便不能得到共同上界。

插入排序的 $W(n) = \Theta(n^2)$，只保證最壞輸入達平方階；已排序輸入仍是 $\Theta(n)$。因此不能推成「每個輸入都至少需要平方時間」。

函數比較的三種實際做法

方法	適合的情況	要避免的跳步
定義夾界	多項式、振盪、要給常數	只列數值或只證一側
比值極限	正函數且極限可求	把極限不存在當成無法比較
取對數比較	正的乘積、冪次、指數	把對數同階誤當原函數同階

例如 $f = n \lg n, g = n$，雖然 $\lg f / \lg g = 1 + \lg\lg n / \lg n \to 1$，但 $f/g = \lg n \to \infty$，所以 $f \in \omega(g)$。

對數能協助比較大小；是否達到常數倍率的同階，仍須回到原函數或更精確的對數差。

成長階梯上的常數，都必須固定

對固定 $a > 1$、$b > 0$、$\varepsilon > 0$、$k > 0$，有

$$1 \prec \lg\lg n \prec (\lg n)^b \prec n^\varepsilon \prec a^n \prec n! \prec n^n,$$

其中 $f \prec g$ 表示 $f = o(g)$；只比較充分大的正整數 n。

取對數令 $x = \ln n$，則 $(\lg n)^b / n^\varepsilon$ 等於固定常數乘 $x^b / e^{\varepsilon x}$，趨近 0。又 $\ln(n^k / a^n) = k \ln n - n \ln a \to -\infty$，故 $n^k = o(a^n)$。

這裡使用基本極限 $\ln n / n \to 0$，下個版面補上理由。若底數或指數依 n 變動，不能套用「固定常數」的結論。

為什麼對數會被正次方壓過？

對實數 $x \ge 4$，$\sqrt{x} - \ln x$ 的導數為 $(\sqrt{x} - 2)/(2x) \ge 0$，且在 $x = 4$ 時為正。因此 $0 \le \ln x / x \le 1/\sqrt{x} \to 0$。

現在固定 $b, \varepsilon > 0$，令 $t = \ln n$，取自然對數比較：

$$\ln \frac{(\ln n)^b}{n^\varepsilon} = b \ln t - \varepsilon t = t \left(b \frac{\ln t}{t} - \varepsilon \right).$$

因 $\ln t / t \to 0$，括號最終不超過 $-\varepsilon/2$，整式趨向負無限，原比值便趨 0。這就證出 $(\ln n)^b = o(n^\varepsilon)$；換成固定底數只差常數。

對數、指數與底數轉換

式子	正確解讀
$\log_a n = \lg n / \lg a$	固定 $a > 1$ 時，不同底數只差常數
$\lg(n^k) = k \lg n$	先做冪次再取對數
$\lg^k n = (\lg n)^k$	先取對數再做冪次
$2^{\lg n} = n$	底數與對數配對
$2^{2 \lg n} = n^2$	指數裡的常數不能丟掉

$\lg n$ 與 $2 \lg n$ 同階，卻不能推出 $2^{\lg n}$ 與 $2^{2 \lg n}$ 同階。對數底數若隨輸入改變，例如 $\log_n n = 1$，也不是固定底數的換底情況。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從迴圈到級數：每層的範圍要重看

```
count = 0
for i = 1 to n
    for j = 1 to i
        count = count + 1
```

第 i 輪執行 i 次，因此總次數為

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2).$$

不能把內層「平均大約一半」當證明；精確求和已給一般公式。另一種夾界：總數不超過 n^2；後半至少 $n/2$ 輪，每輪至少 $n/2$ 次，所以至少 $n^2/4$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三角形不是少一個成長階

上界：不超過 6 × 6 = 36 格。下界：第 4–6 輪，各取前 3 格。

每格代表一次內層執行。$n = 6$ 共 $1 + 2 + 3 + 4 + 5 + 6 = 21$ 次；第 4–6 輪各取前三次，已有 $3 \times 3 = 9$ 次。這是前面一般平方下界的具體子集合。

倍增迴圈：數的是乘了幾次 2

```
i = 1
while i <= n
    visit(i)
    i = 2*i
```

若採無溢位的整數算術且 $n \geq 1$，依次造訪 $1, 2, 4, \ldots, 2^{\lfloor \lg n \rfloor}$，共 $\lfloor \lg n \rfloor + 1$ 次；$n = 13$ 時是 $1, 2, 4, 8$，共 4 次。

固定寬度實作必須避免 `2*i` 溢位。例如本輪造訪後，若 $i > \lfloor n/2 \rfloor$ 就停止，否則才倍增。數學偽碼的終止證明不能直接涵蓋溢位後的機器行為。

幾何級數：最末一層已決定同階

若外層 $i = 1, 2, 4, \ldots, 2^k \leq n$，內層各做 i 次，則總成本

$$1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1.$$

因 $2^k \leq n < 2^{k+1}$，有 $n/2 < 2^k \leq n$，所以總成本是 $\Theta(n)$，不是 $\Theta(n \lg n)$。

$n = 13$ 時四輪各做 $1, 2, 4, 8$ 次，共 15 次。雖有 $\Theta(\lg n)$ 輪，但不能把每輪都當成 n 次後，誤認為已得到緊界；那樣只證出寬鬆上界。

倍增的是每輪工作量，不是輪數乘最後一輪

左圖每格是一份工作，四輪各做 $1, 2, 4, 8$ 份；右圖的累積量是 $1, 3, 7, 15$。最後一輪的 8 份已占總量一半以上。

對一般最後一輪 2^k，$2^k \le \sum_{r=0}^{k} 2^r < 2 \cdot 2^k$。所以總量與最後一輪同階，而不是再多乘一個輪數。

調和級數：用面積取得兩側界

令 $H_n = \sum_{i=1}^{n} 1/i$。$1/x$ 在正半軸遞減，因此

$$\int_{1}^{n+1} \frac{dx}{x} \le H_n \le 1 + \int_{1}^{n} \frac{dx}{x}.$$

亦即 $\ln(n+1) \le H_n \le 1 + \ln n$。當 $n \ge 3$，$1 + \ln n \le 2 \ln n$，故 $H_n = \Theta(\ln n) = \Theta(\lg n)$。

上界把第一項 1 分開，再用每段左側的函數值蓋住第 2 項以後；下界則在每段用左端矩形蓋住積分。下個版面把兩種矩形分開畫。

冪次求和：不用展開也能找緊界

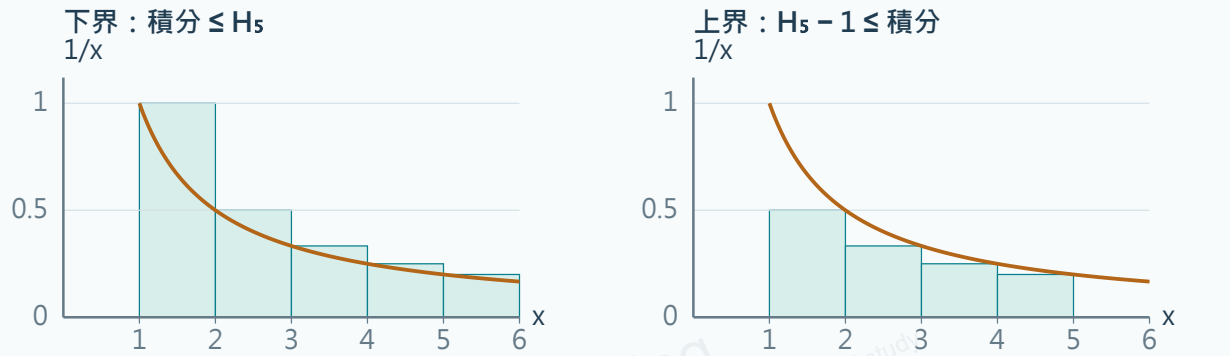
對固定實數 $p \ge 0$，考慮 $S_p(n) = \sum_{i=1}^{n} i^p$。上界直接用 n 項、每項不超過 n^p：$S_p(n) \le n^{p+1}$。

下界只取 $i = \lceil n/2 \rceil .. n$，至少 $n/2$ 項，每項至少 $(n/2)^p$，所以

$$S_p(n) \ge \frac{n}{2} \left(\frac{n}{2}\right)^p = \frac{n^{p+1}}{2^{p+1}}.$$

因此 $S_p(n) = \Theta(n^{p+1})$。這個證法保留了固定倍率，也說清楚為什麼 p 必須固定；$p = -1$ 是調和級數，不能套這個結論。

同一條曲線，兩種矩形方向



左圖 $n = 5$：每段 $[i, i+1]$ 的矩形高 $1/i$，總面積 H_5 大於 $\int_1^6 dx/x$。右圖第 $i = 2..5$ 項的矩形放在 $[i-1, i]$，位於曲線下，所以 $H_5 - 1 \le \int_1^5 dx/x$。

$H_5 = 137/60 \approx 2.2833$，介於 $\ln 6 \approx 1.7918$ 與 $1 + \ln 5 \approx 2.6094$ 之間。兩圖的座標與面積單位相同。

相依的巢狀迴圈，可能是 n log n

```
for i = 1 to n
    j = i
    while j <= n
        visit(i, j)
        j = j + i
```

內層造訪 $i, 2i, \ldots, \lfloor n/i \rfloor i$，所以

$$S(n) = \sum_{i=1}^{n} \lfloor n/i \rfloor .$$

對 $x \ge 1$，$x/2 \le \lfloor x \rfloor \le x$，故 $nH_n/2 \le S(n) \le nH_n$，得到 $\Theta(n \lg n)$。$n = 6$ 時各輪是 $6, 3, 2, 1, 1, 1$ 次，共 14 次。

階乘先取對數，再比較成長

對 $n \geq 4$，$n! \leq n^n$，因此 $\lg(n!) \leq n \lg n$。

另一方面，後半至少 $n/2$ 個因子都不小於 $n/2$，所以

$$\lg(n!) \geq \frac{n}{2} \lg(n/2) \geq \frac{n}{4} \lg n.$$

故 $\lg(n!) = \Theta(n \lg n)$。這不代表 $n! = \Theta(n^n)$：事實上兩者比值趨近 0。

對固定 $a > 1$，$\ln(n!/a^n) \geq (n/2)\ln(n/2) - n \ln a \to \infty$，可證 $a^n = o(n!)$。若需精確倍率，再使用 Stirling 公式，不能把對數裡的誤差隨意指數化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

floor、ceiling 與餘數的邊界

對實數 x，$x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$。因此整數 n 滿足

$$\lfloor n/2 \rfloor + \lceil n/2 \rceil = n.$$

對整數 a 與正整數 m，本章採數學餘數

$$a \bmod m = a - m\lfloor a/m \rfloor, \qquad 0 \leq a \bmod m < m.$$

例如 $-7 \bmod 3 = 2$，因為 $-7 = 3(-3) + 2$。某些語言的 `%` 採截向零的商，會得到負餘數；把公式搬成程式時必須核對語義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

連續整數分割，什麼時候能合併？

對 $x \ge 0$、正整數 a, b，有

$$\left\lfloor \frac{\lfloor x/a \rfloor}{b} \right\rfloor = \left\lfloor \frac{x}{ab} \right\rfloor, \qquad \left\lceil \frac{\lceil x/a \rceil}{b} \right\rceil = \left\lceil \frac{x}{ab} \right\rceil.$$

floor 的證明：令 $q = \lfloor x/(ab) \rfloor$，則 $abq \le x < ab(q+1)$。除以 a 再取 floor，得 $bq \le \lfloor x/a \rfloor \le bq + b - 1$；再除以 b 取 floor，結果就是 q。

例如 $x = 17, a = 3, b = 2$，兩次 floor 是 $17 \to 5 \to 2$，直接除以 6 取 floor 也是 2。a, b 為正整數是上述整數區間論證的前提。

ceiling 的端點，要改用嚴格下界

令 $q = \lceil x/(ab) \rceil$，便有 $ab(q-1) < x \le abq$。除以 a 再取 ceiling，得到

$$b(q-1) + 1 \le \lceil x/a \rceil \le bq.$$

再除以 b 取 ceiling，結果也是 q。$x = 17, a = 3, b = 2$ 時，$17 \to 6 \to 3$，與 $\lceil 17/6 \rceil = 3$ 一致。

對正整數 u, v，另有 $\lceil u/v \rceil \le (u+v-1)/v$、$\lfloor u/v \rfloor \ge (u-v+1)/v$：寫成 $u = qv + r, 0 \le r < v$，分 $r = 0$ 與 $r > 0$ 即可核對。實作 `(u+v-1)/v` 時仍須避免加法溢位。

空間成本：同時活著多少，才是峰值

額外空間不含輸入本身，但通常包括暫存陣列與遞迴呼叫堆疊；若另排除輸出，須明寫。

情境	累積配置量	同時存活的峰值
依次配置並釋放 n 個單字區塊	$\Theta(n)$	$\Theta(1)$
配置 n 個單字且全部保留	$\Theta(n)$	$\Theta(n)$
共 K 次呼叫，峰值有 d 個常數大小框架同時存活	$\Theta(K)$	$\Theta(d)$

插入排序只用固定數量的索引與暫存資料，在單筆資料可用常數個字表示的假設下，額外空間為 $\Theta(1)$。若每筆是大型物件，還要說明搬的是物件還是參照。

配過多少與同時保留多少：畫成存活區間

依次配置 1、2、3、4 個字：立即釋放時峰值 4，全部保留時峰值 10；兩者累積配置量同為 10。只計圖中存活區塊，不含配置器碎片、快取或作業系統常駐記憶體。

常數與交會點：成長階不能回答全部

同一成本單位下，兩種方法的精確模型是 $T_A(n) = 8n^2$、$T_B(n) = 64n \lg n$，$n \geq 2$。A 較快等價於 $n < 8 \lg n$。

n	$8n^2$	$64n \lg n$	比較
16	2048	4096	A 較快
32	8192	10240	A 較快
64	32768	24576	B 較快

對實數 $n \geq 4$，$n / \lg n$ 嚴格遞增；檢查 $n = 43, 44$ 分別落在兩側，可知正整數 $n \geq 2$ 的切換點在 43 與 44 之間。若模型來自量測，這個結論也只適用於該模型與環境。

同階也可能差很多，快硬體也有單位

若 A 的速度是每秒 10^9 個模型步驟，B 是每秒 10^8 步，則應比較

$$t_A(n) = T_A(n)/10^9, \qquad t_B(n) = T_B(n)/10^8,$$

不能把步驟數與秒數直接混在同一個不等式。換硬體改變常數，換演算法可能改變成長階；兩者都需要明確成本模型。

空間上限也會限制可用方法。例如需要 n^2 個項目的表格，即使時間估算可接受，也可能放不進記憶體。實際選擇須同時檢查時間、空間與資料特性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

綜合分析：從程式走到一般結論

分析一段新程式時，可以依下列順序建立答案，而不是先猜一個 Big-O：

1. 固定合法輸入、輸入規模、比較與算術模型。
2. 用小輸入列完整狀態，找到每輪實際做的事。
3. 寫不變量、邊界與遞減量，處理正確性及終止性。
4. 將每次成本寫成一般求和，或建立遞迴式。
5. 分開證上界與下界，註明最好／最壞／平均的對象。
6. 算峰值空間，核對溢位、別名與分布等前提。

本章的插入排序得到 $T(S) = \Theta(n + I(S))$，比只寫最壞 $O(n^2)$ 更能說明輸入特性如何改變成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：多項式最高次項為什麼主導？

令 $p(n) = a_d n^d + \sum_{i<d} a_i n^i$，固定 $d \geq 1$、$a_d > 0$，並設 $M = \sum_{i<d} |a_i|$。

對 $n \geq 1$，低階項的絕對值總和不超過 $M n^{d-1}$。再取 $n \geq 2M/a_d$，便有

$$\frac{a_d}{2} n^d \leq p(n) \leq \frac{3a_d}{2} n^d.$$

因此 $p(n) = \Theta(n^d)$，即使低階係數有負數仍成立；若最高次係數為負，就不符合本章時間函數最終非負的條件。$d = 0, a_0 > 0$ 時另得 $\Theta(1)$。

延伸：Big-O 式子的等號不能反推

式子 $n^2 + n = n^2 + O(n)$ 表示右側存在一個屬於 $O(n)$ 的函數，使等式成立；本例選 $h(n) = n$ 即可。

由此可推出 $n^2 + n = \Theta(n^2)$，卻不能從 $f = \Theta(n^2)$ 反推 $f = n^2 + O(n)$：反例 $f = 2n^2$，差值 n^2 不在 $O(n)$。

若要逐項運算，先把匿名函數命名。例如 $u \in O(n), v \in O(n)$，用三角不等式得 $|u + v| \leq (c_u + c_v) n$。若涉及相減，需允許帶符號的誤差界，不能忽略原來非負的定義慣例。

延伸：e、ln 與近似的誤差

對 $x > -1$，$\ln(1 + x) \leq x$。因此正整數 n 有 $(1 + 1/n)^n \leq e$。

更精確地，對 $x \geq 0$，

$$x - \frac{x^2}{2} \leq \ln(1 + x) \leq x.$$

下界可由導數得到：$h(x) = \ln(1 + x) - x + x^2/2$，$h(0) = 0$ 且 $h'(x) = x^2/(1 + x) \geq 0$。代入 $x = 1/n$，得到

$$e^{1 - 1/(2n)} \leq (1 + 1/n)^n \leq e.$$

所以不只知道它趨向 e，也知道有限 n 時的一側誤差。近似式應附適用範圍，不能任意丟掉高階項。

延伸：指數近似也能給明確餘項

Taylor 定理給 $e^x = 1 + x + (e^\xi/2)x^2$，其中 ξ 介於 0 與 x。因此 $|x| \le 1$ 時，

$$\frac{e^{-1}}{2}x^2 \le e^x - 1 - x \le \frac{e}{2}x^2.$$

所以 $e^x = 1 + x + \Theta(x^2)$ 的極限方向是 $x \to 0$，不是 $x \to \infty$。另由凸性，$e^x \ge 1 + x$ 對所有實數成立。

標準冪級數 $e^x = \sum_{k=0}^{\infty} x^k/k!$ 對所有實數收斂；若只取前幾項，還須像上式一樣控制餘項。本節使用基礎微積分的 Taylor 定理，不在此重新推導該定理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：對數界也要涵蓋負的 x

對 $x > -1$，較一般的雙界為

$$\frac{x}{1+x} \le \ln(1+x) \le x.$$

上界令 $h = x - \ln(1+x)$，導數為 $x/(1+x)$，最小值在 0，且 $h(0) = 0$。下界令 $q = \ln(1+x) - x/(1+x)$，導數為 $x/(1+x)^2$，也在 0 取得最小值 0；兩個論證都涵蓋正負兩側。

對固定實數 z，充分大時代入 $x = z/n$，得 $z/(1+z/n) \le n\ln(1+z/n) \le z$。兩側都趨近 z，因此 $(1+z/n)^n \to e^z$。

當 $|x| < 1$，標準展開為 $\ln(1+x) = \sum_{k=1}^{\infty} (-1)^{k+1}x^k/k$；不能省略其適用範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：Stirling 公式要保留什麼精度？

對正整數 n，Stirling 公式為

$$n! = \sqrt{2\pi n}(n/e)^n\big(1 + O(1/n)\big).$$

Stirling 定理給出主近似的相對誤差。取自然對數可得

$$\ln(n!) = n\ln n - n + \tfrac{1}{2}\ln(2\pi n) + O(1/n).$$

相對誤差 $O(1/n)$ 經取對數仍為 $O(1/n)$，因 $\ln(1+u) = O(u)$ 對充分小的 u 成立。反過來，若只知道對數相差 $O(n)$，指數化後可能相差指數倍率，不能說階乘值只有常數誤差。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：把 Stirling 誤差變成數值區間

Stirling 定理較精確的形式為：

$$n! = \sqrt{2\pi n}(n/e)^n e^{\alpha_n}, \qquad \frac{1}{12n+1} < \alpha_n < \frac{1}{12n} \quad (n \ge 1).$$

取 $n = 6$，主近似 $L = \sqrt{12\pi}(6/e)^6 \approx 710.078$，小於 $6! = 720$。代入誤差界得到

$$Le^{1/73} < 720 < Le^{1/72}.$$

修正項有明確範圍，不是任意四捨五入。由 $\alpha_n = \Theta(1/n)$ 與零附近 $e^u - 1 = \Theta(u)$，可得 $e^{\alpha_n} = 1 + \Theta(1/n)$。粗略乘積界只決定 $\lg(n!)$ 的成長階，這個定理則進一步控制主近似的相對誤差。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸：反覆取對數，停止條件不能省

令 $\lg^{(0)} n = n$，$\lg^{(k+1)} n = \lg(\lg^{(k)} n)$；只在前一步大於 1 時再取對數。定義

$$\lg^{*} n = \min\{k \geq 0 : \lg^{(k)} n \leq 1\}.$$

輸入	依次得到的值	$\lg^{*} n$
1	1	0
2	$2 \to 1$	1
16	$16 \to 4 \to 2 \to 1$	3
65536	$65536 \to 16 \to 4 \to 2 \to 1$	4

$\lg^{k} n$ 是對數值的 k 次方；$\lg^{(k)} n$ 是函數反覆套用。兩者記號不同，成長率也不同。

延伸：Fibonacci 的閉式與驗證

定義 $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$。設 $\varphi = (1+\sqrt{5})/2$、$\psi = (1-\sqrt{5})/2$，兩者都滿足 $r^2 = r + 1$。

因此 $G_n = (\varphi^n - \psi^n)/\sqrt{5}$ 滿足同一遞迴，且 $G_0 = 0, G_1 = 1$。依數學歸納法，$G_n = F_n$ 對每個非負整數成立。

因 $|\psi| < 1 < \varphi$，有 $F_n = \Theta(\varphi^n)$。但 F_n 的**數值大小**不等於計算它的演算法時間：樸素遞迴、表格法與快速倍增必須各自分析，後續遞迴章再比較。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

遞迴、遞迴式與分治

遞迴不只是「函式呼叫自己」。要先說清楚每次呼叫負責什麼、為什麼一定停，再把所有呼叫與合併的成本加起來。

本章依序讀：**一條遞迴路徑 → 合併排序 → 遞迴式的證明工具 → 最大子陣列與矩陣乘法 → 不等分割的延伸工具**。

先備知識：第 1 章的不變量、歸納法、漸近符號與級數；矩陣例子另用基本矩陣乘法，延伸定理使用積分。

參考用書：Cormen 等，*Introduction to Algorithms*, 3rd Edition，Chapters 2、4；另參考 MIT 6.046J（2005）Lecture 2–3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

子呼叫的答案，是歸納假設的接口

設 $P(n)$ 表示：「對所有大小為 n 的合法輸入，程序會終止，且回傳符合規格的結果。」

1. **基例**：直接核對最小輸入，包括空輸入是否允許。
2. **縮小**：每個子問題都合法，大小嚴格小於 n。
3. **組合**：假設較小呼叫的結果正確，證明目前呼叫也正確。

這是強歸納法。不能先假設「目前這次呼叫正確」，也不能只證子問題變小就當作結果正確。

成本另寫 $T(n) = \sum_i T(n_i) + f(n)$：$f$ 只計本次的比較、分割、複製與合併，**不能把子樹的成本再算一次**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二分搜尋只繼續走一邊

前提：$A[1..N]$ 已非遞減排序，鍵值具有全序；回傳**任一**匹配位置，若沒有則回傳 NIL。整體呼叫為 $(1, N)$，$N = 0$ 時直接返回。

```
BINARY-SEARCH(A, low, high, x)
    if low > high: return NIL
    mid = low + floor((high-low)/2)
    if A[mid] == x: return mid
    if A[mid] < x:
        return BINARY-SEARCH(A, mid+1, high, x)
    return BINARY-SEARCH(A, low, mid-1, x)
```

本章採包含兩端點的區間。非空呼叫滿足 $1 \le low \le high \le N$；空區間不讀取陣列。這裡沒有保證回傳第一個重複鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為什麼可以整段排除？

若 $A[mid] < x$，則所有 $i \le mid$ 都有 $A[i] \le A[mid] < x$，所以答案若存在，必在右半。若 $A[mid] > x$，同理只能留左半；相等就已符合規格。

目前長度 $n = high - low + 1 \ge 1$；繼續搜尋的長度至多 $\lfloor n/2 \rfloor < n$。空區間是失敗基例，故遞迴會終止。以長度強歸納，即得整體正確性。

固定成本比較、只傳索引時，最壞成本滿足

$$W(0) = \Theta(1), \qquad W(n) = W(\lfloor n/2 \rfloor) + \Theta(1).$$

沿最長路徑只有 $\Theta(\lg(n+1))$ 個呼叫；最好情況為根即匹配，成本 $\Theta(1)$。順序執行的堆疊峰值為 $\Theta(\lg(n+1))$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

相同陣列，找到與找不到各走一遍

找 9：保留白格；目前中點為青框

	1	2	3	4	5	6	7	
1..7	3	5	7	**8**	9	12	15	向右
5..7	3	5	7	8	9	**12**	15	向左
5..5	3	5	7	8	**9**	12	15	回傳 5

若找 10：最後再呼叫空區間 6..5 → NIL。

$A = [3, 5, 7, 8, 9, 12, 15]$。找 9：$(low, mid, high)$ 依序為 $(1, 4, 7)$、$(5, 6, 7)$、$(5, 5, 5)$，回傳 5。

找 10 的前三次檢查相同；最後因 $9 < 10$ 呼叫 $(6, 5)$，空區間回傳 NIL。圖中灰格只表示已排除，不是把原陣列刪掉。

同一個二分邏輯，複製成本可以差很多

固定原始長度為 N，目前區間長度為 n。考慮最壞的搜尋路徑：

傳參方式	每次呼叫的額外工作	整次最壞成本
只傳索引或 view	$\Theta(1)$	$\Theta(\lg(N+1))$
每次複製目前區間	$\Theta(n)$	$\Theta(N)$
每次複製整個原始陣列	$\Theta(N)$	$\Theta(N \lg(N+1))$

第二列沿路求和 $N + N/2 + N/4 + \cdots = \Theta(N)$。第三列中，$N$ 不隨遞迴縮小，不能偷換成 n。

子問題只有一個，也能畫遞迴樹：此時樹退化成一條路。是否適合這個工具，與呼叫數必須大於一無關。

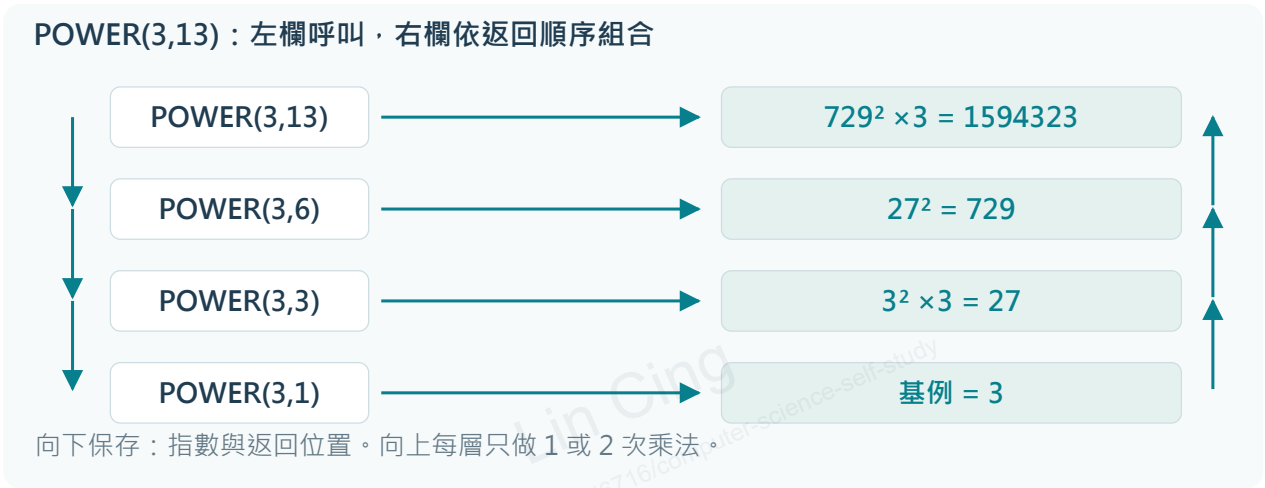
快速冪：相同子問題只算一次

輸入為非負整數 n，輸出 a^n；約定 $a^0 = 1$。本例先計乘法次數，不把大整數乘法一律視為固定時間。

```
POWER(a, n)
    if n == 0: return 1
    if n == 1: return a
    y = POWER(a, floor(n/2))
    z = y*y
    if n is odd: z = z*a
    return z
```

若 $n = 2k$，由 $y = a^k$ 得 $y^2 = a^n$；若 $n = 2k + 1$，再乘 a。$\lfloor n/2 \rfloor < n$ 與兩個基例完成強歸納證明。

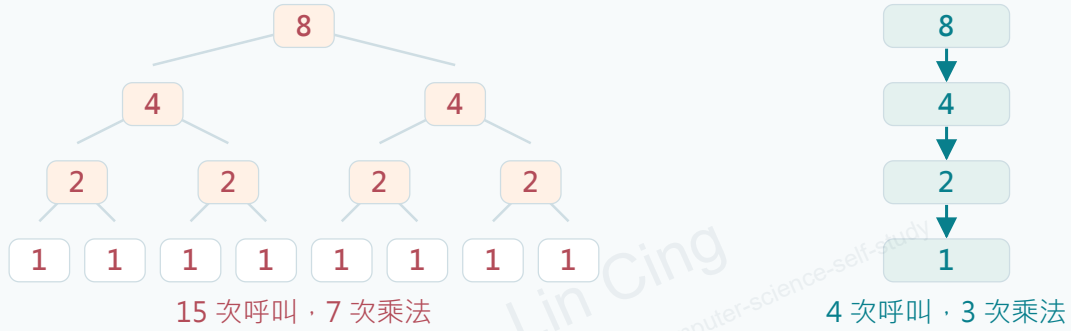
往下保留返回位置，往上完成乘法



對 3^{13}，向下的指數為 $13 \to 6 \to 3 \to 1$。向上得到 $3 \to 27 \to 729 \to 1594323$；分別做 0、2、1、2 次乘法，共 5 次。

對 $n \geq 1$，平方次數為 $\lfloor \lg n \rfloor$；額外乘 a 的次數為二進位 1 的個數減 1。因此乘法總數為 $\lfloor \lg n \rfloor + \mathrm{popcount}(n) - 1$；$n = 0$ 則為 0。

代數式出現兩次，不代表要呼叫兩次



若把 $a^{n/2}a^{n/2}$ 寫成兩次獨立的 POWER 呼叫，即使兩次答案相同，成本仍滿足 $D(n) = 2D(n/2) + \Theta(1) = \Theta(n)$，先限 n 為二冪。

保存一次結果再平方，則 $R(n) = R(\lfloor n/2 \rfloor) + \Theta(1)$，在單位成本乘法模型下為 $\Theta(\lg n)$。差別在**實際執行了幾個呼叫**，不是怎麼簡化乘積。

固定寬度實作還要處理乘法溢位；若只需 $a^n \bmod m$，可每次乘法後取模，但乘積本身仍須可安全計算。

Fibonacci：呼叫樹的重複與輸出大小

定義 $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$。直接遞迴每次都重算；呼叫總數滿足 $C_0 = C_1 = 1$、$C_n = C_{n-1} + C_{n-2} + 1$。

令 $D_n = C_n + 1$，由同一遞迴與基例可證 $C_n = 2F_{n+1} - 1$。例如算 $F_5 = 5$ 需要 15 次呼叫；時間為 $\Theta(\varphi^n)$，但堆疊深度只有 $\Theta(n)$，其中 $\varphi = (1 + \sqrt{5})/2$。

由小到大只保存前兩項，做 $n - 1$ 次加法即可算 F_n（$n \geq 2$）。這是避免重算的基本想法，完整動態規劃設計留到後續章節。

算術步數與位元成本要分開：F_n 有 $\Theta(n)$ 個位元；「保存兩個數」是兩個大整數，並非固定數量的機器字。逐項做普通整數加法的位元時間為 $\Theta(n^2)$。

用矩陣冪精確計算 Fibonacci

令 $Q = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$。對 $n \geq 1$，

$$Q^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}.$$

$n = 1$ 直接成立。右乘 Q 後，第一欄是原本兩欄之和、第二欄是原第一欄，恰好得到下一組 Fibonacci，歸納完成。

例如 $Q^2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$、$Q^4 = \begin{bmatrix} 5 & 3 \\ 3 & 2 \end{bmatrix}$，故 $Q^5 = \begin{bmatrix} 8 & 5 \\ 5 & 3 \end{bmatrix}$，右上角為 $F_5 = 5$。$n = 0$ 直接回傳 0。

對矩陣套快速冪，只需 $O(\lg(n+1))$ 次 2×2 矩陣乘法，且全程可用精確整數。這是**算術操作數**；輸出有 $\Theta(n)$ 個位元，因此位元時間不可能是 $O(\lg n)$。用浮點數計閉式再四捨五入也不能無條件保證正確。

合併排序與分治入門

分治（divide and conquer）把問題拆成較小的同類問題，遞迴求解，再合併答案。以下排序包含兩端點的 $A[p..r]$，長度 $m = r - p + 1$；$p > r$ 表示空區間。

```
MERGE-SORT(A,p,r)
    if p >= r: return
    q = p + floor((r-p)/2)
    MERGE-SORT(A,p,q)
    MERGE-SORT(A,q+1,r)
    MERGE(A,p,q,r)
```

非空呼叫要求 $1 \leq p \leq r \leq n$；整體呼叫 $(1, n)$，空陣列的 $(1, 0)$ 直接返回。子區間長度為 $\lceil m/2 \rceil$、$\lfloor m/2 \rfloor$，$m \geq 2$ 時都嚴格小於 m。

中點寫成差值式，可避免直接計算 $p + r$ 的溢位；索引、長度與加減仍須在實作型別可表示的範圍內。

不依賴額外鍵值的 MERGE

前提為 $1 \le p \le q < r \le n$，兩半都已穩定排序。L, R 必須是**獨立副本**，長度分別為 $q-p+1$ 與 $r-q$，不能被後續回寫 A 破壞。

```
MERGE(A,p,q,r)
    L = copy(A[p..q]); R = copy(A[q+1..r])
    i = 1; j = 1; k = p
    while i <= L.length and j <= R.length
        if L[i].key <= R[j].key: A[k] = L[i]; i = i+1
        else: A[k] = R[j]; j = j+1
        k = k+1
    while i <= L.length
        A[k] = L[i]; i = i+1; k = k+1
    while j <= R.length
        A[k] = R[j]; j = j+1; k = k+1
```

鍵值相等時先取 L：原輸入左半的資料本來就在右半之前。以長度判斷耗盡，不需要鍵值域之外的無窮大哨兵。

MERGE 的保存、順序與終止證明

令 $m = r - p + 1$。主迴圈每次比較前，維持

$$k - p = (i - 1) + (j - 1).$$

$A[p..k-1]$ 恰是已消耗的 $L[1..i-1]$ 與 $R[1..j-1]$ 的穩定合併，已排序，且其中每個鍵都不大於任何未消耗的鍵。其餘資料仍在獨立副本中。

初始化時輸出為空，條件成立。兩邊都非空時，各自首筆就是該邊最小的剩餘鍵；取較小者、相等先取左，便是全體下一筆。消耗一筆並令 k 加一，維持長度、保存、順序與穩定性；剩餘筆數嚴格減一，故會終止。

主迴圈結束後，一邊耗盡；另一邊已排序，且不小於輸出尾端，可依序接上。最後 $i = |L| + 1, j = |R| + 1, k = r + 1$，全部 m 筆恰輸出一次。若只用原陣列的 view，回寫可能覆蓋尚未讀取的資料，破壞這個證明的前提。

有重複 key 的完整合併

令 $L = [1a, 4b, 4c]$、$R = [2d, 4e, 6f]$，左半所有資料原本都早於右半。下表的 i, j 是比較前的索引；只比較鍵值，字母用來辨識資料身分。

i, j	比較與選擇	輸出前綴
1,1	$1 \le 2$，取 1a	1a
2,1	$4 > 2$，取 2d	1a,2d
2,2	$4 \le 4$，取左側 4b	1a,2d,4b
3,2	$4 \le 4$，取左側 4c	1a,2d,4b,4c
4,2	左半耗盡，接上右半	1a,2d,4b,4c,4e,6f

共 4 次鍵值比較、6 次輸出寫入，另有 6 筆資料複製到 L, R。最壞比較數為 $m - 1 = 5$；$\Theta(m)$ 描述成本階，並不表示每次都恰比較 m 次。

有重複 key 的完整合併：圖解

相等取左：4b、4c 都必須早於 4e

	L			R			輸出前綴
start	1a	4b	4c	2d	4e	6f	ø
1≤2	1a	4b	4c	2d	4e	6f	1a
4>2	1a	4b	4c	2d	4e	6f	1a, 2d
4≤4	1a	4b	4c	2d	4e	6f	1a, 2d, 4b
4≤4	1a	4b	4c	2d	4e	6f	1a, 2d, 4b, 4c
tail	1a	4b	4c	2d	4e	6f	1a, 2d, 4b, 4c, 4e, 6f

綠底 = 已消耗；所有未消耗資料仍保存在獨立的 L、R 副本。

Sentinel 是有前提的另一種表示

哨兵（sentinel）是在每個暫存陣列尾端加一個特殊標記，讓合併可固定輸出 m 次，不必每輪判斷哪邊耗盡。但標記的排序順位必須**嚴格高於所有合法鍵值**，而且不是可輸出的真資料。

不能直接把最大合法整數當作哨兵。若合法鍵包含 $+\infty$，取 $L = [+\infty]$、$R = [+\infty, +\infty]$，再加相同值的哨兵：等值先取左可能依序取到左邊真資料、左哨兵，第三次便讀超過 L。

本章的長度檢查版適用於任意全序鍵值域。另一個安全做法是使用帶標籤的哨兵，讓它在比較時嚴格高於全部真資料；仍須保證哨兵不會被輸出成答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從局部合併推到整個排序

對區間長度 m 作強歸納。$m \le 1$ 時已排序且不需修改。$m \ge 2$ 時，兩個子區間都較短；歸納假設保證兩半各自穩定排序，並保留所有原資料。

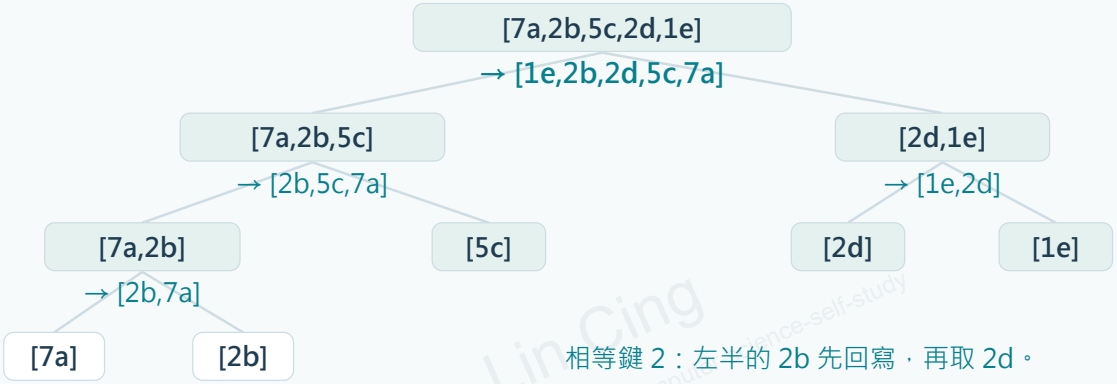
於是 MERGE 的前提成立。合併證明使整段穩定排序且不遺失、不重複資料；區間外未改動，完成目前呼叫的規格。子問題長度嚴格縮小，另保證終止。

此論證不要求 n 是二冪。例如 $[7a, 2b, 5c, 2d, 1e]$ 先分成長 3、2，再分成 2/1 與 1/1。根節點合併遇到相等鍵 2 時，仍先取左邊的 2b，再取 2d。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從局部合併推到整個排序：圖解

往下分割；每個箭頭旁列出該子呼叫的排序結果



相等鍵 2：左半的 2b 先回寫，再取 2d。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴式的成本不能漏掉合併

對 $m \geq 2$，計算中點為 $\Theta(1)$；兩個子呼叫依序執行，成本相加。MERGE 複製 m 筆並回寫 m 筆，為 $\Theta(m)$，故

$$T(m) = T(\lceil m/2 \rceil) + T(\lfloor m/2 \rfloor) + \Theta(m), \quad T(0), T(1) = \Theta(1).$$

先看二冪 $m = 2^h$ 上的精確代理式 $U(1) = d$、$U(m) = 2U(m/2) + cm$，固定 $c, d > 0$。深度 $\ell < h$ 有 2^ℓ 個內部節點，每個局部成本 $cm/2^\ell$，所以每層總成本 cm。

共有 h 個內部層，最底 m 個葉各花 d，得到

$$U(m) = cm \lg m + dm.$$

真成本不必恰為同一個 c：把各節點的固定上下界分別代入，即得 $\Theta(m \lg m)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

任意長度的界與一個六筆成本表

任意 $m \geq 2$ 平衡拆分後，深度 ℓ 的區間長度不超過 $\lceil m/2^{\ell} \rceil$，所以樹高至多 $\lceil \lg m \rceil$。同層區間互不重疊、總長至多 m，給出 $O(m \lg m)$ 上界。

若 $\ell < \lfloor \lg m \rfloor$，每段至少有 $\lfloor m/2^{\ell} \rfloor \geq 2$ 筆，尚無葉；這些完整層的總長恰為 m。又 $\lfloor \lg m \rfloor \geq \frac{1}{2} \lg m$，故內部工作為 $\Omega(m \lg m)$。另有 m 個單元素基例。

$m = 6$ 的深度	區間長度	仍需合併的長度總和
0	6	6
1	3,3	6
2	2,1,2,1	4
3	1,1,1,1（另兩葉已停止）	0

本例複製到暫存陣列與回寫各 16 筆，基例共 6 次。不能把每一層都算成 6 筆。

峰值空間不是所有配置次數的總和

本版 MERGE 等兩個子呼叫返回後，才建立總長 m 的 L, R 暫存陣列；完成合併就可釋放。因此子呼叫的暫存陣列不會與父層 MERGE 的暫存陣列同時存活。

根層需要 n 筆暫存資料，其他層不會更大，所以暫存陣列峰值為 $\Theta(n)$。另有 $\Theta(\lg n)$ 個呼叫框架，合計仍為 $\Theta(n)$，這裡取 $n \geq 2$。

全程累計配置量可達 $\Theta(n \lg n)$，但那不是同時需要的記憶體。若保留已完成呼叫的暫存陣列不釋放，就改變了生命期模型；也可預先配置 n 筆 scratch 空間重複使用，但仍須維持未讀取資料不被覆寫的前提。

比較第 1 章：插入排序用常數個額外機器字；此合併排序需要線性額外空間。兩者都可穩定排序，前提分別是嚴格大於才右移、等值時先取左。

代入法：把猜測證成同一個界

代入法先猜候選界，再用強歸納找出同一組固定常數，使較小輸入的假設推出目前的結論。

令 $T(1)=1$、$T(n)=2T(\lfloor n/2\rfloor)+n$，$n\ge 2$。猜 $T(n)\le cn\lg n$。對 $n\ge 4$，令 $m=\lfloor n/2\rfloor\ge 2$，則

$$\begin{aligned}T(n)&\le 2cm\lg m+n\\&\le cn\lg(n/2)+n\\&=cn\lg n-(c-1)n\le cn\lg n\end{aligned}$$

只需固定 $c\ge 1$。但 $n=1$ 的候選右側為 0，不能作歸納基礎。改用遞迴算出的 $T(2)=4, T(3)=5$；固定 $c=2$ 同時覆蓋兩者，而 $n\ge 4$ 的子引數至少為 2，歸納閉合。

遞迴的基例與證明使用的歸納基礎可以不同，但到達小輸入時的依據都必須交代。

同一常數的下界與強化假設

先限二冪 n，令 $T(1)=1, T(n)=2T(n/2)+n$。若子問題滿足 $T(n/2)\ge d(n/2)\lg(n/2)$，則

$$T(n)\ge dn\lg n+(1-d)n\ge dn\lg n,\qquad 0<d\le 1.$$

基例右側為 0、左側為 1，成立；歸納得 $\Omega(n\lg n)$。任意整數的版本可用前面的平衡分層成本證明。

再看 $T(n)=T(\lfloor n/2\rfloor)+T(\lceil n/2\rceil)+1$，$T(1)=1$。猜 $T(n)\le cn$ 只得 $cn+1$，無法閉合；改猜

$$T(n)\le cn-d.$$

代入得到 $T(n)\le cn-2d+1\le cn-d$，只要 $d\ge 1$。取 $c=2, d=1$，基例也成立，並可歸納得到精確解 $2n-1$。

子問題中保留的低階扣項，正好用來支付父節點的新工作。「$cn+n$ 仍是 $O(n)$」卻沒有證出原先同一個 cn，不能作為歸納步驟。

換變數與真正縮小的條件

先在 $n = 2^{2^k}$ 上考慮 $T(n) = 2T(\sqrt{n}) + \lg n$、$T(2) = \Theta(1)$。令 $m = \lg n$、$S(m) = T(2^m)$，則

$$S(m) = 2S(m/2) + m = \Theta(m \lg m).$$

還原得 $T(n) = \Theta(\lg n \lg \lg n)$；不是把新變數 m 又當成原輸入 n。接下來另證帶 floor 的一般整數版本。

固定平移也要檢查是否縮小。例如 $T(n) = 2T(n/2 + 17) + n$，只有 $n > 34$ 時子引數小於 n。令 $n = 34 + x$、$U(x) = T(34 + x)$，得到

$$U(x) = 2U(x/2) + x + 34.$$

採足夠大的固定區間為基例，幾何分層求和得 $\Theta(x \lg x) = \Theta(n \lg n)$。這裡先處理實數引數；整數版還須控制取整誤差。若只給 $T(1)$，$n = 34$ 會自我呼叫，遞迴尚未定義完成。

平方根裡的 floor，如何補上一般整數證明？

定義 $T(n) = 1$ 於整數 $1 \le n \le 3$，$n \ge 4$ 時

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n.$$

對 $n \ge 4$，$2 \le \lfloor \sqrt{n} \rfloor < n$，所以遞迴有定義且會停止。

先取 $a_k = 2^{2^k}$、$U_k = T(a_k)$。因 $\sqrt{a_k} = a_{k-1}$，這些點沒有取整誤差；$U_0 = T(2) = 1$，且 $U_k = 2U_{k-1} + 2^k$。

除以 2^k，得到 $U_k/2^k = U_{k-1}/2^{k-1} + 1$。累加得 $U_k = (k+1)2^k$，也就是這些特殊點上 $T(n) = \lg n(1 + \lg \lg n)$。

這還只涵蓋特殊點，不能到此就宣稱所有整數的界成立；下一步要補單調性與包夾。

單調性先證明，再用相鄰特殊點包夾

$T(1)=T(2)=T(3)=1<T(4)$。假設較小引數的相鄰值已非遞減；由 $\lfloor\sqrt{n+1}\rfloor\ge\lfloor\sqrt{n}\rfloor$、兩個子引數均小於 n，以及 $\lg(n+1)>\lg n$，得到 $T(n+1)\ge T(n)$。強歸納證明單調性。

選 k 使 $a_k\le n<a_{k+1}$，等價於 $2^k\le\lg n<2^{k+1}$。由單調性，

$$(k+1)2^k\le T(n)\le(k+2)2^{k+1}.$$

此區間內 $\lg n$ 與 2^k 在兩倍內，$1+\lg\lg n$ 與 $k+1$ 也在固定倍數內，因此

$$T(n)=\Theta\big(\lg n(1+\lg\lg n)\big),\qquad n\ge 2.$$

對充分大的 n 可寫成 $\Theta(\lg n\lg\lg n)$。這個包夾使用已證的單調性；不是把 floor 的誤差憑直覺丟掉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴樹：每個節點只記當次工作

遞迴樹每個內部節點只記本次局部成本 $f(\text{size})$；子呼叫由子樹另計。若節點已記完整 $T(\text{size})$，再加子樹就重複計費。

取 $T(n)=3T(n/4)+n^2$、$T(1)=1$，先限 $n=4^L$。深度 j 有 3^j 個節點，每個局部成本 $(n/4^j)^2$，層成本為 $n^2(3/16)^j$。葉數為 $3^L=n^{\log_4 3}$，故

$$T(n)=n^2\sum_{j=0}^{L-1}(3/16)^j+n^{\log_4 3}.$$

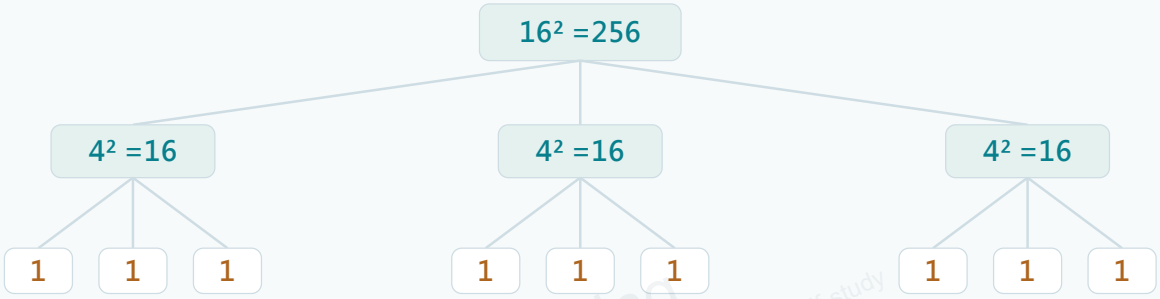
等比和至多 $16/13$，葉成本也不超過 n^2，給出上界 $O(n^2)$；根的局部工作給出下界 $\Omega(n^2)$。

$n=16$ 的各層成本為 256、48、9，總計 313。最後的 9 已是九個基例的成本，不能再加九次 $f(1)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴樹：每個節點只記當次工作：圖解

T(n)=3T(n/4)+n²，T(1)=1；n=16



層成本：256 → 48 → 9（九個基例）

總成本 313；葉不再另外加上 f(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每層縮成七分之八，還要把葉成本算進去

取整數 $n \geq 8$，

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lfloor n/4 \rfloor) + T(\lfloor n/8 \rfloor) + n,$$

並令 $0 \leq n \leq 7$ 時 $T(n) = 1$。內部節點的子問題總長至多 $7n/8$，所以深度 j 的內部成本至多 $n(7/8)^j$，合計至多 $8n$。

若內部節點數為 I、葉數為 L，每個內部節點都有三個子節點，數邊得 $3I = I + L - 1$，故 $L = 2I + 1$。每個內部節點至少有成本 8，因而 $8I \leq 8n$，給 $L \leq 2n + 1$。

所以 $T(n) \leq 8n + (2n + 1) = O(n)$；根本身提供 $T(n) \geq n$，得到 $\Theta(n)$。**無窮等比和先支付內部工作，葉成本由節點計數另外支付**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不等分割：先界深度，再界每層質量

令 $a=\lfloor n/3\rfloor$、$b=n-a$，且 $T(n)=T(a)+T(b)+cn$，固定 $c>0$；小輸入的成本有固定正上下界。

沿任一路徑，第 j 層的引數 q_j 滿足

$$\frac{n}{3^j}-\frac{3}{2}\le q_j\le n(2/3)^j+3.$$

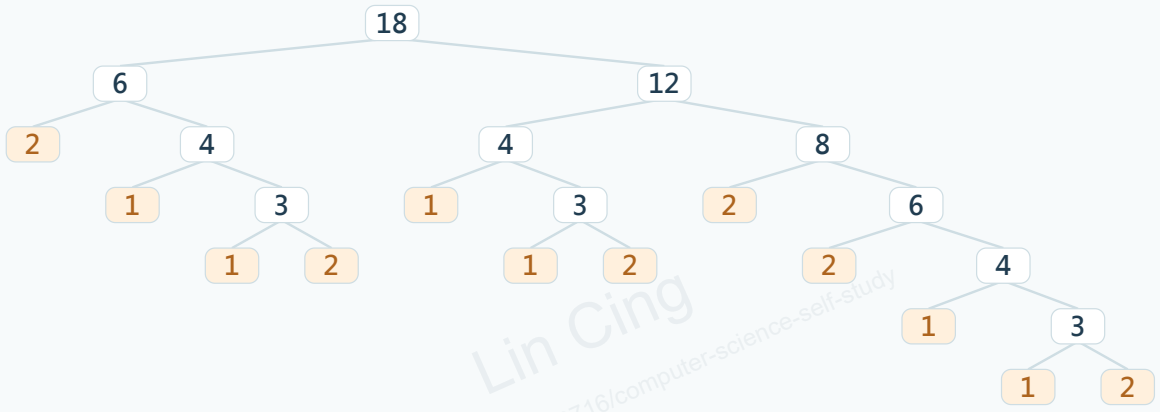
這由每步 $q/3-1\le q_{\text{next}}\le 2q/3+1$ 及等比誤差和歸納得到。把大小至多固定 $K\ge 6$ 的整段底層運算視為基例，不改漸近階。右側界給高度 $O(\lg n)$；同層子問題互不重疊，總長度至多 n，工作至多 cn。葉的正整數長度總和是 n，所以至多 n 個葉，合計上界 $O(n\lg n)$。

若 $0\le j\le\lfloor\log_3(n/(K+2))\rfloor$，左側界使每個 $q_j>K$，尚無路徑停止。這些完整層都保有總長度 n，各成本 cn，層數為 $\Omega(\lg n)$，所以有相同下界。

若只知局部工作為 $O(n)$，則仍有上界，卻不能保證下界；零合併成本便可能只剩線性葉成本。

不等分割：完整呼叫樹的葉不在同一層

n=18；白底繼續分割，橘底為 n≤2 的基例

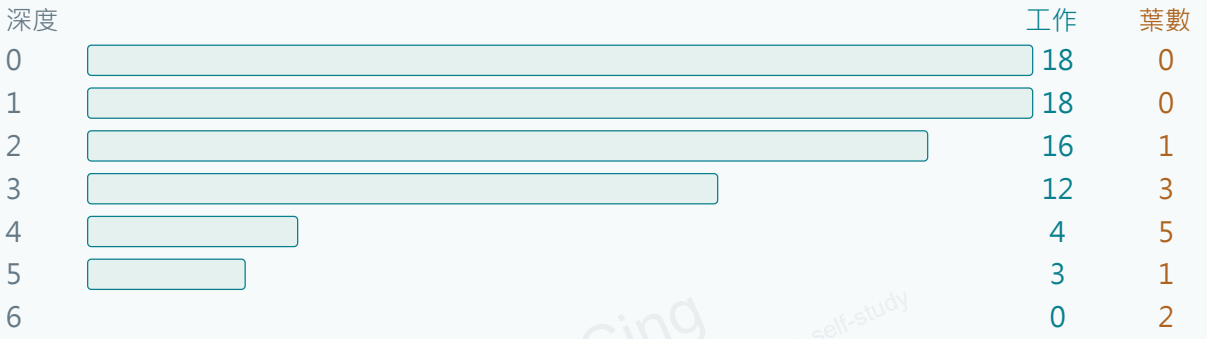


共 11 個內部節點、12 個葉；葉在深度 2 到 6 之間。

每條邊是一次實際子呼叫。橘色葉在不同深度停止；最長路徑為18 →12 →8 →6 →4 →3 →2。

不等分割：完整層與提早停止要分清

n=18；n≤2 停止；條長表示該層內部節點的總量



葉節點成本另計；到達基例後不會繼續產生下一層。

本圖固定 $n \le 2$ 為基例，其餘拆成 $\lfloor n/3 \rfloor$ 與 $n - \lfloor n/3 \rfloor$。$n = 18$ 的完整層總量起初都是 18；出現葉之後，**只有仍繼續分割的節點**才支付合併工作。

最短路徑決定下界能保留幾層；最長路徑決定上界要涵蓋幾層。不能用最深路徑的高度，乘上每一層都等於 n 的假想成本來證下界。

先看每層工作，再看三種主導方式：圖解

各欄採獨立比例；標籤為每層總成本

先看每層工作，再看三種主導方式

三例都取 $n = 16$、$a = b = 2$、$T(1) = 1$。非遞迴項依序為 $1, n, n^2$，各層（含葉）的成本為：

$f(n)$	深度 0、1、2、3、4	總成本
1	1、2、4、8、16	31
n	16、16、16、16、16	80
n^2	256、128、64、32、16	496

這三個數例分別呈現葉端、等高、根端主導；一般定理還需要固定參數、多項式差距與情形 3 的正則性，不能只靠圖的外觀判定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

主定理：先核對前提

令固定 $a \geq 1, b > 1$，局部成本 $f \geq 0$，常數大小基例的成本有固定正上下界。考慮

$$T(n) = aT(n/b) + f(n), \qquad p = \log_b a.$$

先在精確冪上使用下表；取整的延伸條件在後面說明。若 a 是實際呼叫數，它必須為整數；實數 a 可解讀為遞迴權重。

情形	需要核對的前提	結論
1	固定 $\epsilon > 0$，$f(n) = O(n^{p-\epsilon})$	$\Theta(n^p)$
2	$f(n) = \Theta(n^p)$	$\Theta(n^p \lg n)$
3	固定 $\epsilon > 0$，$f(n) = \Omega(n^{p+\epsilon})$；且充分大 n 有 $af(n/b) \leq cf(n)$，固定 $c < 1$	$\Theta(f(n))$

只說「比 n^p 小／大」並不夠：情形 1、3 要有固定的多項式差距，情形 3 還要檢查正則性（regularity）。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一張表的完整判定數例

每一列都使用固定正基例。表中的 p 是 $\log_b a$，不是從圖上猜出的指數。

$(a, b, f(n))$	前提的具體核對	緊界
$(9, 3, n)$	$p = 2$，取 $\epsilon = 1$	$\Theta(n^2)$
$(1, 3/2, 1)$	$p = 0$，$f(n) = \Theta(n^0)$	$\Theta(\lg n)$
$(3, 4, n \lg n)$	$p \approx 0.7925$；取 $\epsilon = 0.1$，正則性比值至多 $3/4$	$\Theta(n \lg n)$
$(8, 2, n^2)$	$p = 3$，取 $\epsilon = 1$	$\Theta(n^3)$
$(7, 2, n^2)$	$p = \lg 7$，取 $\epsilon = \lg 7 - 2 > 0$	$\Theta(n^{\lg 7})$

第三列的正則性由

$$\frac{3f(n/4)}{f(n)} = \frac{3}{4}\frac{\lg n - 2}{\lg n} \le \frac{3}{4} \quad (n \ge 4)$$

直接得到。相較之下，$2T(n/2) + n \lg n$ 的對數因子沒有固定多項式差距，不能直接套經典三情形；下一個層和仍能解它。

對數邊界必須真的求和

令 $n = b^L, p = \log_b a$，且 $f(n) = \Theta(n^p (\lg n)^k)$，固定實數 k。在固定大小的基例以上，把深度 j 改編成離葉端的距離 $r = L - j$，層成本為 $\Theta(n^p r^k)$。

所以總成本是 $\Theta(n^p)$ 的葉成本，加上 n^p 乘一個冪次和：

$$T(n) = \begin{cases} \Theta(n^p (\lg n)^{k+1}) & k > -1, \\ \Theta(n^p \lg \lg n) & k = -1, \\ \Theta(n^p) & k < -1. \end{cases}$$

當 $k > -1$，積分比較給 $\sum_{r=1}^{L} r^k = \Theta(L^{k+1})$；$k = -1$ 是調和和；$k < -1$ 時和收斂到正常數。若 $k < 0$，須在大於 1 的固定基例之外使用公式，避免對 $\lg 1$ 取負次方。

因此 $2T(n/2) + n/\lg n$ 為 $\Theta(n \lg \lg n)$；換成 $n/(\lg n)^2$ 則為 $\Theta(n)$。這是層和的結論，不是經典三情形中的情形 1。

正則性缺失的明確反例

只看 $n = 2^L$，令 $T(1) = 1$，且

$$f(n) = \begin{cases} n^2 & L \text{ 為偶數}, \\ n^3 & L \text{ 為奇數}, \end{cases} \qquad T(n) = 2T(n/2) + f(n).$$

每個輸入都有 $f(n) \geq n^2$，對 $a = b = 2, p = 1$ 已有固定多項式差距。但在偶數 $L \geq 2$ 上，子問題的指數 $L - 1$ 是奇數，故

$$T(n) \geq 2f(n/2) = n^3/4, \qquad \frac{T(n)}{f(n)} \geq n/4 \to \infty.$$

所以 $T(n)$ 不屬於 $\Theta(f(n))$。同時計算得 $2f(n/2)/f(n) = n/4$，不存在共同的 $c < 1$ 讓正則性最終成立。

這個例子說明缺少前提會怎樣，沒有推翻主定理。對振盪函數也不能因引數只差一個常數，就假設函數值接近。

主定理為什麼成立

先令 $n = b^L, T(1) = t > 0$。展開 L 次可得

$$T(n) = tn^p + \sum_{j=0}^{L-1} a^j f(n/b^j).$$

第 j 層總權重是 a^j；a 為整數呼叫數時，它也是節點數。葉項 tn^p 在三種情形中都提供正下界。

情形 1：充分大 x 有 $f(x) \leq Cx^{p-\epsilon}$，因此大規模層的總和至多

$$Cn^{p-\epsilon} \sum_{j=0}^{L-1} (b^\epsilon)^j = O(n^{p-\epsilon} b^{\epsilon L}) = O(n^p).$$

因 $b^\epsilon > 1$，這是往葉端增大的幾何和。上式可補上任意有限個小引數項來作上界；這些底層的總權重為 $O(n^p)$，固定函數值可吸收到常數。

加上葉成本得到 $O(n^p)$，再用 tn^p 的下界，證得 $\Theta(n^p)$。

主定理等高與根主導的兩個證明

情形 2。 因 $a = b^p$，充分大引數的每層總成本為

$$a^j \Theta((n/b^j)^p) = \Theta(n^p).$$

有 $L - O(1) = \Theta(\lg n)$ 個這樣的層；將上下界分別相加，得到 $\Theta(n^p \lg n)$，也涵蓋葉成本。

情形 3。 正則性可以逐層使用：只要引數仍充分大，就有

$$a^j f(n/b^j) \le c^j f(n), \qquad 0 < c < 1.$$

這些層合計不超過 $f(n)/(1-c)$。剩餘固定大小層與葉的總成本為 $O(n^p)$；因 $f(n) = \Omega(n^{p+\epsilon})$，也屬於 $O(f(n))$。根的局部工作 $f(n)$ 給出下界，合成 $\Theta(f(n))$。

固定小輸入不是整棵樹的常數成本：可能有 $\Theta(n^p)$ 個小節點，必須像這裡一樣計入底層總量。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

取整的誤差不會逐層線性累積

沿任一實際呼叫路徑，令 $q_0 = n$，q_{j+1} 為 $\lfloor q_j/b \rfloor$ 或 $\lceil q_j/b \rceil$。兩者都有 $|q_{j+1} - q_j/b| \le 1$。取固定 $C \ge b/(b-1)$，歸納得

$$|q_j - n/b^j| \le C.$$

因下一步誤差至多 $C/b + 1 \le C$，取整誤差不會每層累加成 j。

選固定基例範圍 $q \le n_0$，其中 $n_0 > C + 2$，且足夠大使子問題嚴格縮小。上、下誤差界使各路徑進入基例的深度都為 $\log_b n + O(1)$。在 $n/b^j \ge 2C$ 的大規模層，實際 q_j 與 n/b^j 在固定倍數內。

實際有整數 a 個子呼叫時，葉深度只差常數，故葉數為 $\Theta(a^{\log_b n}) = \Theta(n^p)$；固定個底層的總成本為 $O(n^p)$。$a = 1$ 時是一條路，葉與固定個底層的成本都是常數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

從精確冪推到所有整數的成本條件

以下處理整數 a 個實際子呼叫；精確冪上的實數權重版本已由前面的展開式處理。

情形 1、2： 大規模層的 q_j 與 n/b^j 在固定倍數內。固定次方即使為負，仍有

$$q_j^{p-\epsilon} = \Theta((n/b^j)^{p-\epsilon}).$$

因此情形 1 可沿用多項式上界與幾何和；情形 2 的每個完整大規模層仍為 $\Theta(n^p)$，層數為 $\Theta(\lg n)$。固定個不完整底層另計 $O(n^p)$。對數擴充在遠離固定基例的層，$\log q_j$ 與離底層深度同階，也可沿用冪次和。

情形 3： 正則性必須涵蓋實際取整後的引數。充分條件是

$$a \max\{f(\lfloor x/b \rfloor), f(\lceil x/b \rceil)\} \le c f(x), \qquad c < 1.$$

各層總成本才會以 c 倍收縮；底層仍由多項式差距吸收。對任意 f，只證引數相差常數並不足以省略此條件。

正則性也能推出多項式分離

在精確冪上，若 $af(x/b) \le cf(x)$、固定 $0 < c < 1$，且某個固定底層點 $f(x_0) > 0$，向上反推 k 層可得

$$f(b^k x_0) \ge (a/c)^k f(x_0).$$

因 $\log_b(a/c) = p + \log_b(1/c)$，令 $\epsilon = \log_b(1/c) > 0$，便有沿這條精確冪序列的 $f(n) = \Omega(n^{p+\epsilon})$。

這解釋為什麼有些版本在正且良好的底層條件下，不另外列出多項式差距。本章保留三情形的完整假設，方便逐項核對。

此推導的域是 $n = b^k x_0$；推到全部實數或整數，還需控制最後進入的固定區間，以及該區間上 f 的正下界。不能只證一條序列，就宣稱所有輸入都成立。

最大非空連續子陣列

給定 $n \geq 1$ 個有限實數 $A[1..n]$，求非空、連續區間 $[i, j]$，使

$$\sum_{k=i}^{j} A[k]$$

最大，回傳 (i, j, sum)。相同最大值可以有多個答案；此規格允許任選其中一個。先以精確加法與比較的操作數分析；浮點誤差、NaN 與溢位不是這個模型的合法輸入行為。

若每天價格為 $P_0, \ldots, P_n$，令 $A_i = P_i - P_{i-1}$，便有 $P_j - P_{i-1} = \sum_{k=i}^{j} A_k$。區間 $i..j$ 對應第 $i-1$ 天買、第 j 天賣。

若允許不交易，才可加入空區間、收益 0；全負輸入時答案會改變。逐一起點、延長終點並累加，可在 $\Theta(n^2)$ 時間與 $O(1)$ 額外空間枚舉；每個區間若重新加總，才變成 $\Theta(n^3)$。

跨中線必須由 後綴 與 前綴 組成

對 $low < high$，令 $mid = low + \lfloor (high - low)/2 \rfloor$。任一非空區間恰屬三類：

1. 全在 $low..mid$。
2. 全在 $mid + 1..high$。
3. 跨過中線，滿足 $i \leq mid < j$。

第三類必包含 mid 與 $mid + 1$，可唯一拆成左半的後綴（suffix）$i..mid$ 與右半的前綴（prefix）$mid + 1..j$。

兩側端點可獨立選擇，故最佳跨線和等於**最佳非空左後綴和＋最佳非空右前綴和**。分別掃描兩側即可，不必枚舉全部端點配對。左右兩半各自的最佳子陣列則未必接到中線，不能直接相加。

跨中線必須由 後綴 與 前綴 組成：圖解

mid=5：跨線區間必須包含第 5 與第 6 格

左側向外累加：−1, 3, 0, 1, −1 → 最大 3，起點 4

右側向外累加：2, 3, −2, 2 → 最大 3，終點 7

組合為 A[4..7]=[4,−1,2,1]，總和 6。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨線掃描的完整程序

```
CROSS(A, low, mid, high):
    left = -infinity; total = 0
    for i = mid downto low:
        total += A[i]
        if total > left: left = total; best_i = i
    right = -infinity; total = 0
    for j = mid+1 to high:
        total += A[j]
        if total > right: right = total; best_j = j
    return (best_i, best_j, left+right)
```

每次左迴圈更新後，`total` 恰是 $A[i..mid]$ 的和；`left` 是已掃過全部左後綴的最大和。初值 $-\infty$ 是比較標記，不是輸入資料；第一個有限候選一定更新它，並設定起點。

每輪新增恰一個候選、取最大，保持不變量；結束時左後綴已全部納入。右邊的前綴同理，兩者相加得到最佳跨線和。掃描總次數為

$$(mid - low + 1) + (high - mid) = high - low + 1.$$

最大子陣列的遞迴與一般證明

```
MAX-SUBARRAY(A, low, high):
    if low == high: return (low, high, A[low])
    mid = low + floor((high-low)/2)
    L = MAX-SUBARRAY(A, low, mid)
    R = MAX-SUBARRAY(A, mid+1, high)
    C = CROSS(A, low, mid, high)
    return a tuple with largest sum among L, R, C
```

對區間長度作強歸納。單元素只有自身；較長時兩個子區間均非空且較短，子呼叫分別給前兩類的最大值，CROSS 給第三類最大值。三類互斥且完整，取三者最大即正確；長度嚴格縮小也保證終止。

因此操作成本滿足

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n) = \Theta(n \lg n).$$

依序執行子呼叫、只傳索引時，每層僅保存常數個索引與和，堆疊峰值為 $O(\lg n)$。若額外複製子陣列，必須另算配置與存活空間。

九元素數例：每次跨線比較都可核對

對 $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$，取 $low = 1, high = 9, mid = 5$。

左索引 i	$A[i..5]$ 累計和	已知最佳和、起點
5	-1	-1，5
4	3	3，4
3	0	3，4
2	1	3，4
1	-1	3，4

右邊 $j = 6, 7, 8, 9$ 的前綴和是 $2, 3, -2, 2$，最佳為 $6..7$、和 3。跨線解為 $4..7$、和 6；左半最佳 $4..4$、和 4，右半最佳 $9..9$、和 4，所以本次回傳 $(4, 7, 6)$。

對全負輸入 $[-8, -2, -5]$，非空規格回傳 $(2, 2, -2)$，不是 0。

線性方法的狀態與不變量

令 E_i 為必須以 i 結尾的最佳非空區間和，B_i 為 $1..i$ 中最佳非空區間和。則

$$E_1 = B_1 = A_1, \qquad E_i = \max(A_i, E_{i-1} + A_i), \quad B_i = \max(B_{i-1}, E_i).$$

以 i 結尾的區間不是只有 A_i，就是接在以 $i-1$ 結尾的區間後；要延伸時，選先前最佳便足夠。這個分類與基例給出兩個狀態的歸納證明。

若選單元素 A_i，起點重設為 i；若選延伸，保留舊起點。全域最佳改善時保存該起點與終點 i。每輪固定工作，故時間 $\Theta(n)$、額外空間 $O(1)$。

九元素例的 E_i 為 $[-2, 1, -2, 4, 3, 5, 6, 1, 5]$，$B_i$ 為 $[-2, 1, 1, 4, 4, 5, 6, 6, 6]$。分治法不是這題最快的解法；它提供的是另一個清楚的分類與組合證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從純量乘積到區塊矩陣乘法

標準矩陣乘法逐一計算

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

內迴圈在處理第 k 項後，累積值恰為前 k 個乘積之和。n^2 個輸出各做 n 次乘加，算術操作數為 $\Theta(n^3)$；這沒有證明所有矩陣乘法都必須是立方成本。

對偶數階矩陣，寫成區塊

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, \qquad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix}.$$

每個字母是 $n/2$ 階矩陣，乘法給出

$$C_{11} = ae + bg, \quad C_{12} = af + bh, \quad C_{21} = ce + dg, \quad C_{22} = cf + dh.$$

這裡保持乘法次序；矩陣區塊一般不交換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

八次遞迴乘法的完整成本

先補零到二冪階數，最後擷取原尺寸結果。每次呼叫從自己的矩陣 view 讀出階數，不能一直沿用原始階數。

```
BLOCK-MULTIPLY(A,B):
    n = A.rows
    if n == 1: return [[A[1,1]*B[1,1]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    C11 = BLOCK-MULTIPLY(a,e)+BLOCK-MULTIPLY(b,g)
    C12 = BLOCK-MULTIPLY(a,f)+BLOCK-MULTIPLY(b,h)
    C21 = BLOCK-MULTIPLY(c,e)+BLOCK-MULTIPLY(d,g)
    C22 = BLOCK-MULTIPLY(c,f)+BLOCK-MULTIPLY(d,h)
    return the four blocks assembled into C
```

索引 view 只保存起點、stride 與階數，可用常數時間分割。即使真的複製區塊，其 $\Theta(n^2)$ 成本也與固定個區塊加法同階，因此 $T(n) = 8T(n/2) + \Theta(n^2)$，基例為 $\Theta(1)$。

不能把八個子呼叫吸收到一個子呼叫的常數裡。第 j 層有 8^j 次呼叫，葉數是 $8^{\lg n} = n^3$；分支數會改變整棵樹的成本指數。

Strassen 的七個乘積全部列出

以下各乘積由遞迴 STRASSEN 計算；括號內的加減是逐元素的區塊運算。

$$
\begin{aligned}
M_1 &= (a + d)(e + h), & M_2 &= (c + d)e, \\
M_3 &= a(f - h), & M_4 &= d(g - e), \\
M_5 &= (a + b)h, & M_6 &= (c - a)(e + f), \\
M_7 &= (b - d)(g + h).
\end{aligned}
$$

再組合四個結果：

$$
\begin{aligned}
C_{11} &= M_1 + M_4 - M_5 + M_7, & C_{12} &= M_3 + M_5, \\
C_{21} &= M_2 + M_4, & C_{22} &= M_1 - M_2 + M_3 + M_6.
\end{aligned}
$$

$n = 1$ 時直接做純量乘法；$n > 1$ 時依此分割、計算七個乘積，再組成四塊。這些步驟與基例一起定義完整遞迴程序。

四個輸出區塊為何正確

只用分配律與加法消去，保持矩陣乘法的因子順序：

$$\begin{aligned}
C_{11} &= (ae + ah + de + dh) + (dg - de) \\
&\quad - (ah + bh) + (bg + bh - dg - dh) = ae + bg, \\
C_{12} &= (af - ah) + (ah + bh) = af + bh, \\
C_{21} &= (ce + de) + (dg - de) = ce + dg, \\
C_{22} &= (ae + ah + de + dh) - (ce + de) \\
&\quad + (af - ah) + (ce + cf - ae - af) = cf + dh.
\end{aligned}$$

基例的純量乘法正確。若七個較小乘積均由歸納假設正確算出，上述恆等式便組成 AB 的全部四塊，完成歸納證明。

這裡需要加法、減法與分配律。只有加法、沒有減法的一般半環（semiring），不能直接套用這七式。

七乘積的數值與四塊回收

取 $A = \begin{bmatrix} 2 & -1 \\ 3 & 4 \end{bmatrix}$、$B = \begin{bmatrix} 5 & 2 \\ -2 & 1 \end{bmatrix}$。

乘積	代入	值
M_1	$(2+4)(5+1)$	36
M_2	$(3+4)5$	35
M_3	$2(2-1)$	2
M_4	$4(-2-5)$	-28
M_5	$(2-1)1$	1
M_6	$(3-2)(5+2)$	7
M_7	$(-1-4)(-2+1)$	5

四塊依序為 $36 - 28 - 1 + 5 = 12$、$2 + 1 = 3$、$35 - 28 = 7$、$36 - 35 + 2 + 7 = 10$。直接乘法核對：

$$AB = \begin{bmatrix} 10 + 2 & 4 - 1 \\ 15 - 8 & 6 + 4 \end{bmatrix} = \begin{bmatrix} 12 & 3 \\ 7 & 10 \end{bmatrix}.$$

七乘積的數值與四塊回收：圖解

數例的四塊位置：C11、C12、C21、C22 不可互換

A			B			C	
2	-1	×	5	2	=	12	3
3	4		-2	1		7	10

C11: 36−28−1+5=12

C12: 2+1=3

C21: 35−28=7

C22: 36−35+2+7=10

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strassen 的時間、補零與記憶體

Strassen 的七個子乘積需要 10 次輸入區塊加減，組合四個結果還需 8 次。view 分割為 $O(1)$，所以

$$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}), \qquad \lg 7 \approx 2.80735.$$

加減的常數較大，小矩陣未必較快；可在固定門檻改用標準乘法，不改漸近指數。代數恆等式的正確性也不等於浮點誤差保證。

任意 $n \geq 1$ 可補零到 $m = 2^{\lceil \lg n \rceil}$，$n \leq m < 2n$。左上 $n \times n$ 結果仍是原乘積，且 $m^{\lg 7} < 7n^{\lg 7}$，故同階。空矩陣直接返回，不遞迴。

依序完成七個子呼叫、適時釋放暫存時，每層只保留固定個當層區塊，峰值空間滿足

$$S(n) \leq S(n/2) + O(n^2) = O(n^2).$$

另有 $O(\lg n)$ 個框架。若同時保留所有子問題工作區，就必須重建空間遞迴式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延伸工具：Akra–Bazzi 的適用條件

這裡引用 MIT *Mathematics for Computer Science*（2010）§10.4.3 的定理，採用正且有界的基例作充分條件；**不把下列應用當成定理的完整證明**。

$$T(x) = \sum_{i=1}^{k} a_i T(b_i x + h_i(x)) + g(x), \qquad x > x_0.$$

- k 有限固定，$a_i > 0$、$0 < b_i < 1$ 固定。
- $0 \le x \le x_0$ 時，固定常數 $0 < d_1 \le T(x) \le d_2$。
- $g \ge 0$ 可微，$|g'(x)|$ 至多以多項式成長；$|h_i(x)| = O(x/\log^2 x)$。
- x_0 足夠大，所有子引數都在 $[0, \rho x]$，固定 $\rho < 1$，使遞迴有定義且會停止。

上述條件是本章使用的版本。若基例與 g 全為零，成本恆零，就不能得到帶正葉成本的結論。

先求唯一指數，再做積分

令 p 滿足 $\sum_i a_i b_i^p = 1$，則引用定理給出

$$T(x) = \Theta\left(x^p \left[1 + \int_1^x \frac{g(u)}{u^{p+1}}\, du \right] \right).$$

因 $0 < b_i < 1$，函數 $F(p) = \sum_i a_i b_i^p$ 連續且嚴格遞減；當 $p \to -\infty$ 時趨向無窮大，$p \to +\infty$ 時趨向 0，所以恰有一個實根。

對 $T(x) = T(x/3) + T(2x/3) + x$，取固定小輸入為正常數，比例方程為 $(1/3)^p + (2/3)^p = 1$，故 $p = 1$。代入得

$$T(x) = \Theta\left(x \left[1 + \int_1^x \frac{du}{u} \right] \right) = \Theta(x \log x).$$

這和前面用完整層與最深層得到的兩側界一致；兩種推導的適用前提都要保留。

取整與比例改變，是兩種不同的事

floor、ceiling 與固定平移的誤差至多為常數，符合 $O(x/\log^2 x)$；仍須選夠大的基例範圍，確保子問題縮小。

但 $T(n) = T(n/2 + n/4) + T(n/2) + n$ 的第一個子問題其實是 $3n/4$。$n/4$ 不符合小擾動條件；不能沿用兩個 $1/2$ 比例。

正確方程為 $(3/4)^p + (1/2)^p = 1$，唯一根 $p \approx 1.507126591639 > 1$。由

$$\int_1^n u^{-p}\,du = \frac{1 - n^{1-p}}{p-1} = \Theta(1),$$

得到 $T(n) = \Theta(n^p)$。嚴格指數由方程定義，小數只是近似值。

若其他教材把子問題寫成 n/c_i，則本章 $b_i = 1/c_i$。轉換表示法時，方程也必須一起換。

延伸閱讀與定理來源

- Cormen、Leiserson、Rivest、Stein，*Introduction to Algorithms*, 3rd Edition：Chapter 2（合併排序）、Chapter 4（分治、代入法、遞迴樹、主定理及其證明）。
- Demaine、Leiserson，MIT 6.046J（Fall 2005），Lecture 2–3：遞迴式，以及二分搜尋、快速冪、Fibonacci 與矩陣乘法的連結。
- Lehman、Leighton、Meyer，*Mathematics for Computer Science*（2010），§ 10.4.3，Theorem 10.4.1：MIT 官方講義。本章使用正且有界基例的充分條件版本。

主定理的三情形在正文給出證明；Akra–Bazzi 為引用的延伸定理，正文負責核對條件、求指數與計算積分。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

排序、選擇與隨機分析

排序要決定所有資料的先後；選擇只問其中一個秩。兩者可以共用分割與堆積，卻不必付出相同成本。

本章依序學：**比較模型的限制 → 堆積與排序 → 隨機分析工具 → 分割與快速排序 → 計數、基數與桶排序 → 次序統計量**。最後用生日問題、負載、等待與線上遴選練習機率建模。

先備知識：第 1 章的規格、不變量、級數，第 2 章的分治與遞迴式。正文先補齊本章需要的機率工具，再使用期望成本。

參考用書：Cormen 等，*Introduction to Algorithms*, 3rd Edition，Chapters 5–9；補充參考 MIT 6.046J（2005）Lectures 4–6。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先定義排序的對象與保證

每筆資料由 **key（鍵值）** 與隨它搬動的欄位組成。另加身分標籤 a, b, c，方便辨識相同鍵的不同資料；除非特別說明，比較只看 key。

正確排序必須同時保證：輸出鍵值非遞減，而且輸出是輸入資料的同一個多重集合。**穩定性**另要求同鍵資料維持原先順序。

例輸入 $[2_a, 1_b, 2_c]$：$[1_b, 2_a, 2_c]$ 穩定；$[1_b, 2_c, 2_a]$ 排序正確但不穩定；$[1_b, 2_a, 2_a]$ 遺失 2_c，連排序規格都不符合。

本章陣列採 **1 起算、閉區間** $A[p..r]$，$p > r$ 表示空區間；鍵值計數表則依鍵值從 0 起算。選擇的 k 永遠是目前子區間內、1 起算的局部秩。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較數、執行時間與空間各自計算

預設比較與搬移一筆固定大小資料、讀寫一個索引均為常數成本。若鍵是長字串、大整數或外部儲存物件，必須把實際比較、取位與資料搬移成本加回來。

要回答的問題	必須固定的條件
最壞時間 $\max_A T(A)$	長度與合法輸入集合
平均時間 $\mathbb{E}_A[T(A)]$	輸入的聯合分布
隨機演算法 $\mathbb{E}_R[T(A,R)]$	固定輸入 A，對內部亂數 R 平均
額外空間	輸出陣列、暫存區及同時存活的呼叫框架是否計入

「分割原地完成」只說分割程序的儲存需求；快速排序仍可能累積遞迴堆疊。「期望很快」也沒有排除某條亂數路徑很慢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

比較決策樹：每片葉要辨識一種次序

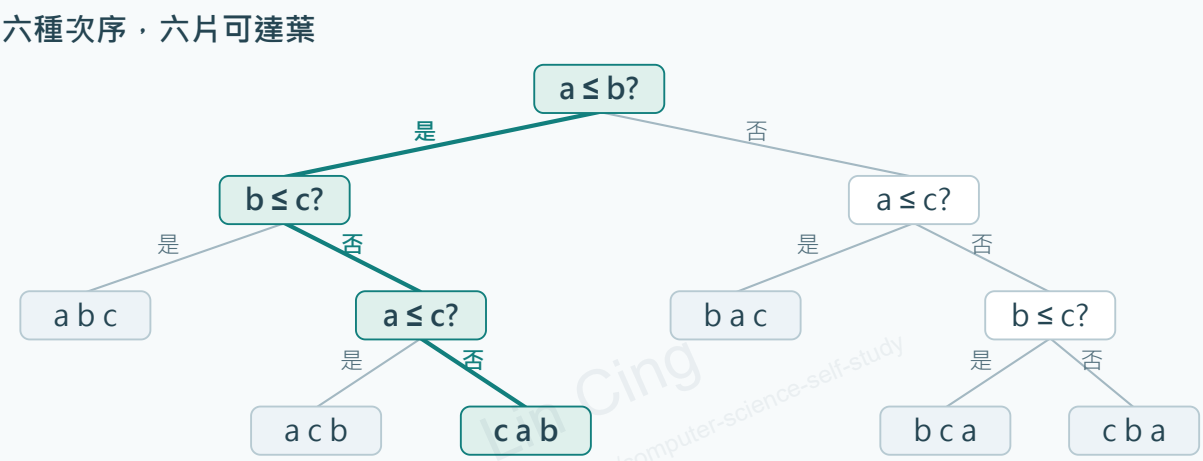
先限制 n 個鍵互異。比較排序只能由兩鍵比較取得次序資訊，不能讀鍵值當作陣列索引；固定輸入長度後，用二元樹表示所有執行。

內節點標示被比較的兩個原始資料身分；左右邊分別表示 $\le$、$>$ 的結果。根到葉記錄本次觀察，葉必須決定這 n 個身分的完整排序。

$n!$ 種相對次序必須導向不同的可達葉。若兩種次序到同一葉，演算法觀察完全相同，只能輸出同一個身分排列，至少有一種會錯。因此可達葉數至少 $n!$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

沿著比較結果，走到資料的次序



圖中的葉標示的是身分次序；例如 c, a, b 表示 $c < a < b$。取 $a = 4, b = 7, c = 1$，依比較結果追蹤到 c, a, b，即可核對一條執行路徑。

最壞下界說的是「至少有一個輸入」

高度 h 的二元樹最多有 2^h 片葉，所以

$$n! \le 2^h, \qquad h \ge \lceil \lg(n!) \rceil.$$

對 $n \ge 2$，階乘的後半至少有 $\lfloor n/2 \rfloor$ 個因子，各不小於 $n/2$；另一方面每個因子至多 n。因此

$$\lfloor n/2 \rfloor \lg(n/2) \le \lg(n!) \le n \lg n.$$

對充分大 n，兩側給出 $\lg(n!) = \Theta(n \lg n)$；小規模另以常數涵蓋。

結論：**每個正確的比較排序，都有最壞輸入需要 $\Omega(n \lg n)$ 次比較。** 這沒有宣稱每個輸入都慢；插入排序遇已排序輸入只做 $n - 1$ 次鍵比較。

允許重複鍵仍包含互異鍵這個困難子集。計數與基數排序則利用鍵值表示，已離開比較模型，必須另外計鍵範圍與位元數的成本。

平均下界不能只看最深的葉

令 $N = n!$ 種相對次序均勻，各選一片代表葉，深度為 $d_1, \dots, d_N$。不同葉的根路徑彼此不是前綴。把每個深度 d_i 的葉對應到 $[0,1)$ 中長度 2^{-d_i} 的二元區間，這些區間互不相交，故

$$\sum_{i=1}^{N} 2^{-d_i} \le 1.$$

由幾何平均不大於算術平均，

$$2^{-\frac{1}{N}\sum_i d_i} \le \frac{1}{N}\sum_i 2^{-d_i} \le \frac{1}{N}.$$

取對數得到 $\frac{1}{N}\sum_i d_i \ge \lg N = \Omega(n \lg n)$。這才是均勻輸入的平均下界；不能由最深路徑直接推到平均路徑。

對每條亂數路徑皆正確的排序，固定亂數後仍有此下界。再對亂數平均，因最大值不小於均勻輸入平均，可得 $\max_A E_r[C(A,r)] \ge \lg(n!)$；允許錯答的排序不在此論證範圍。

三鍵數例：區分輸入平均與亂數平均

用三鍵數例區分兩種平均。L 採前述的三鍵決策樹；R 採相同排序程序，但將原始身分順序改為 a, c, b。排序前等機率選 L 或 R：

鍵值遞增的身分次序	L 比較數	R 比較數	兩者平均
a, b, c	2	3	$5/2$
a, c, b	3	2	$5/2$
b, a, c	2	3	$5/2$
b, c, a	3	3	3
c, a, b	3	2	$5/2$
c, b, a	3	3	3

各欄對均勻輸入平均皆為 $16/6 = 8/3 \ge \lg 6$，各列期望的最大值為三。均勻輸入平均、單一輸入對亂數的平均、最壞期望成本，三者不可混用。

堆積：形狀限制與鍵值限制

二元堆積的形狀是**完全二元樹**：最後一層以前全滿，最後一層由左往右填。1 起算陣列的位置 i 有

$$parent(i) = \lfloor i/2 \rfloor, \quad left(i) = 2i, \quad right(i) = 2i + 1.$$

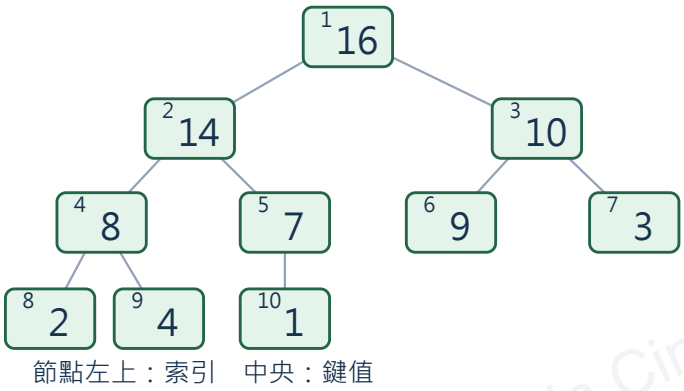
只有 $1 \le i \le s$ 的位置屬於目前堆積，s 是 heap-size（有效堆積大小）。陣列容量、陣列長度與 s 是不同的量。

max-heap 要求每個非根位置 i 都有 $A[parent(i)] \ge A[i]$；此處 $A[i]$ 簡寫其鍵值，交換時仍搬動整筆資料。沿祖先鏈可知根為全體最大鍵；同層或左右子樹之間沒有完整排序關係。

高度計最長向下路徑的**邊數**；單節點高度 0，非空 n 節點堆積高 $\lfloor \lg n \rfloor$。若改計層數，則是 $\lfloor \lg n \rfloor + 1 = \lceil \lg(n+1) \rceil$。空堆積另行處理，不代入 $\lg 0$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一棵堆積的樹與陣列表示



節點左上：索引　中央：鍵值

索引固定，鍵值移動

1	2	3	4	5
16	14	10	8	7
6	**7**	**8**	**9**	**10**
9	3	2	4	1

每條邊都滿足：父鍵 ≥ 子鍵
同一列的節點不保證有序

節點旁標示陣列索引。交換時鍵值移動，位置的父子索引關係不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

向下修復的前提不能省略

前提：$1 \le i \le s$，位置 i 的左右子樹已各自為 max-heap，只有 i 與子節點之間可能違規。不存在的子節點不讀取；and 採短路求值。

```
MAX-HEAPIFY(A,i,s):
    while true:
        largest = i
        if 2*i <= s and A[2*i] > A[largest]:
            largest = 2*i
        if 2*i+1 <= s and A[2*i+1] > A[largest]:
            largest = 2*i+1
        if largest == i: return
        swap(A[i],A[largest])
        i = largest
```

每輪在目前節點與存在的兩個子節點中找最大者，再決定是否交換。只比較兩個子節點、沒有比較父節點，並不能判斷是否需要修復。

向下修復為何正確，又為何會停止

若最大者就是目前節點，兩條父子邊合法；兩棵子樹原本合法，因此整棵子樹完成。

否則把較大子節點的根鍵提升。它不小於自己的後代，也不小於另一個子節點的根與原父鍵，所以提升後父節點的兩條邊都合法。唯一可能的新違規，在舊父鍵下移的位置；另一棵子樹保持不變。

對剩餘高度歸納即可證明整棵子樹最後合法。每次 i 變為 $2i$ 或 $2i+1$，嚴格下降一層；高度 h 至多下降 h 次，所以時間 $O(h+1)$，迭代版額外空間 $O(1)$。

例 $A=[16,4,10,8,7,9,3]$、$i=2, s=7$：選子鍵 8，交換位置 $2,4$，得 $[16,8,10,4,7,9,3]$。位置 4 的子索引 $8,9$ 已超界，停止。索引 7 的鍵 3 屬於另一棵子樹，這一輪不會拿它比較。

由下而上建堆：先讓子樹符合前提

```
BUILD-MAX-HEAP(A):
    s = length(A)
    for i = floor(s/2) downto 1:
        MAX-HEAPIFY(A,i,s)
```

不變量：每輪開始時，索引大於 i 的每個節點都是合法 max-heap 的根。

初始化 $i = \lfloor n/2 \rfloor$ 時，較大索引全是葉，成立。兩個子節點索引都比 i 大，因此呼叫滿足 MAX-HEAPIFY 的前提；完成後 i 也合法，而且其後代仍是合法子堆。令 i 減一，保持下一輪條件。

終止 $i = 0$ 時，根 1 也合法，整棵堆積完成。$n = 0, 1$ 時不執行迴圈，直接得到合法空堆或單節點堆。

$\lfloor n/2 \rfloor$ 是最後一個可能有子節點的位置，**不是第一片葉**；例如 $n = 10$ 時位置 5 還有左子節點 10，不能漏掉這一輪。

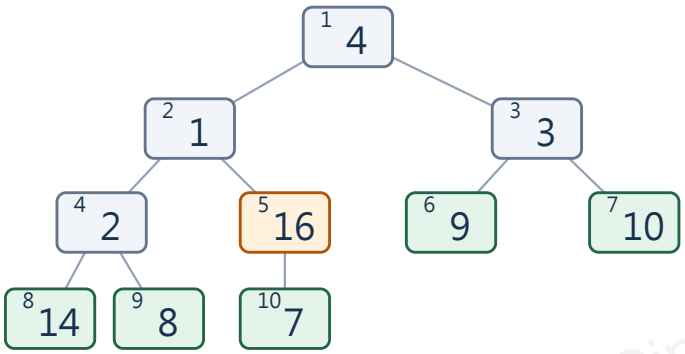
由位置五往前建堆：每輪完整狀態

輸入 $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$，有效大小 $s = 10$。表格採 **1 起算索引**；每輪的兩棵子樹已先修復，才處理本輪的根。

根 i	選子節點與交換索引	本輪修復後 $A[1..10]$
5	唯一子鍵 7；$16 \geq 7$，不交換	$[4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$
4	子鍵 $14, 8$，選 14；$4 \leftrightarrow 8$	$[4, 1, 3, 14, 16, 9, 10, 2, 8, 7]$
3	子鍵 $9, 10$，選 10；$3 \leftrightarrow 7$	$[4, 1, 10, 14, 16, 9, 3, 2, 8, 7]$
2	選 16，再選 7；$2 \leftrightarrow 5$、$5 \leftrightarrow 10$	$[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$
1	選 $16, 14, 8$；$1 \leftrightarrow 2$、$2 \leftrightarrow 4$、$4 \leftrightarrow 9$	$[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

接下來的樹與陣列圖使用同一組資料。橘框為目前處理的根，綠框為已保證合法的子樹根；位置的父子關係固定，交換只移動鍵值及其資料。

位置五：只有左子節點



節點左上：索引　中央：鍵值

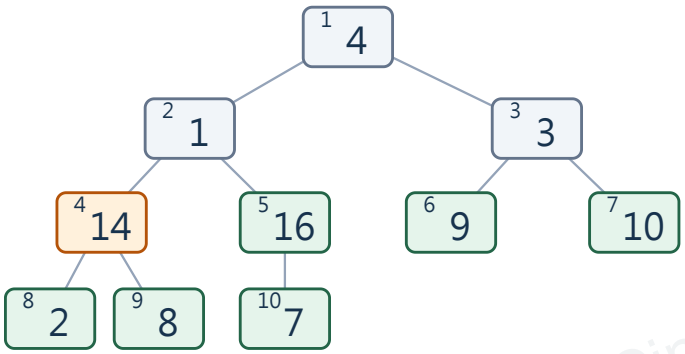
索引固定，鍵值移動

1	2	3	4	5
4	1	3	2	16
6	7	8	9	10
9	10	14	8	7

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

位置 5 的鍵 16 與左子節點 10 的鍵 7 比較，已滿足 $16 \geq 7$，不交換。右子索引 $11 > s$，不能讀取。位置 $6..10$ 都是葉；加上本輪完成的子樹根 5，由下往上的不變量繼續成立。

位置四：較大的子鍵十四上移



節點左上：索引　中央：鍵值

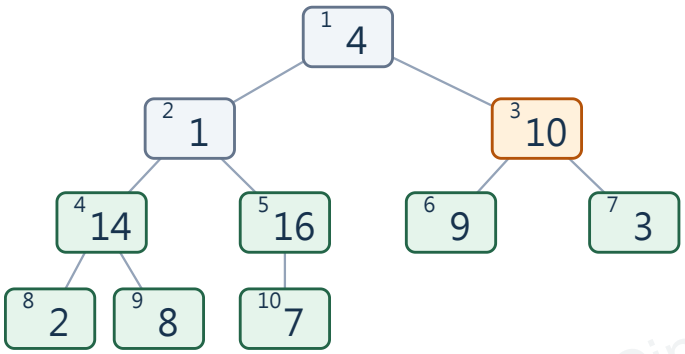
索引固定，鍵值移動

1	2	3	4	5
4	1	3	14	16
6	7	8	9	10
9	10	2	8	7

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

位置 4 原鍵為 2，子索引 $8, 9$ 的鍵為 $14, 8$。先選較大子鍵 14，再判斷 $14 > 2$，交換位置 $4, 8$。舊鍵 2 到葉位置 8 後停止，位置 9 的鍵 8 不變。

位置三：從兩個子節點中選十



節點左上：索引　中央：鍵值

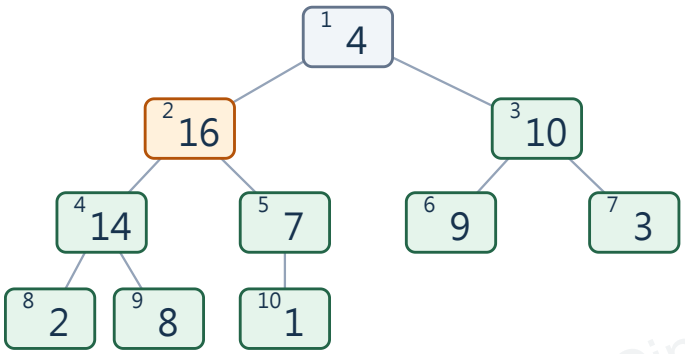
索引固定，鍵值移動

1	2	3	4	5
4	1	10	14	16
6	7	8	9	10
9	3	2	8	7

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

位置 3 的鍵為 3，子索引 $6, 7$ 的鍵為 $9, 10$。選較大鍵 10，交換位置 $3, 7$。舊鍵 3 到葉位置 7 後停止，位置 6 的鍵 9 不變。兩個子樹原先合法，交換後以位置 3 為根的整棵子樹也合法。

位置二：舊鍵一沿五、十下沉



節點左上：索引　中央：鍵值

索引固定，鍵值移動

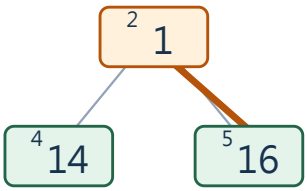
1	2	3	4	5
4	16	10	14	7
6	7	8	9	10
9	3	2	8	1

橘：本輪已完成的子樹根
綠：先前已合法；灰：待處理

位置 2 的子鍵為 $14, 16$，先與較大的 16 交換。舊鍵 1 到位置 5 後，仍小於唯一子鍵 7，再交換位置 $5, 10$。路徑為 $2 \to 5 \to 10$；這輪不改動位置 4 的鍵 14。

追蹤同一個舊鍵：每一步只選當前子節點

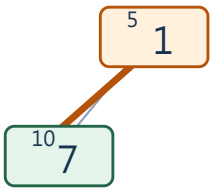
比較 1



16 > 1 → swap(2,5)

A[2]=16; A[5]=1

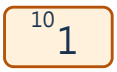
比較 2



7 > 1 → swap(5,10)

A[5]=7; A[10]=1

到葉，停止



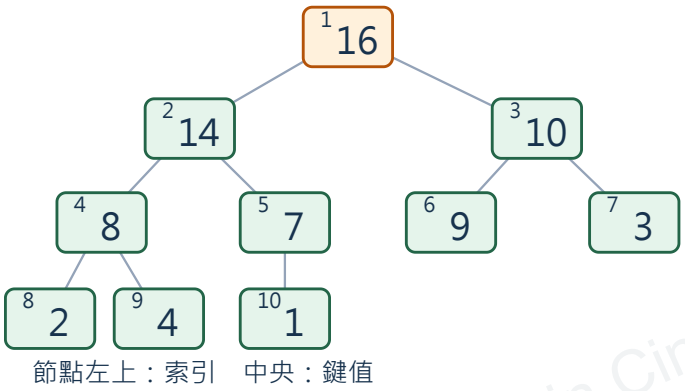
20, 21 > heap-size 10

不再讀取 child

舊鍵 1 留在索引 10

橘框始終追蹤舊鍵 1；粗線指出本次選中的較大子節點。每個節點左上角是索引、中央是鍵。第一次在子索引 $4, 5$ 中選 5，第二次在位置 5 只有子索引 10。未畫出的子樹保持合法，不會因為未被走訪就需要重排。

位置一：根鍵四沿二、四、九下沉



節點左上：索引　中央：鍵值

索引固定，鍵值移動

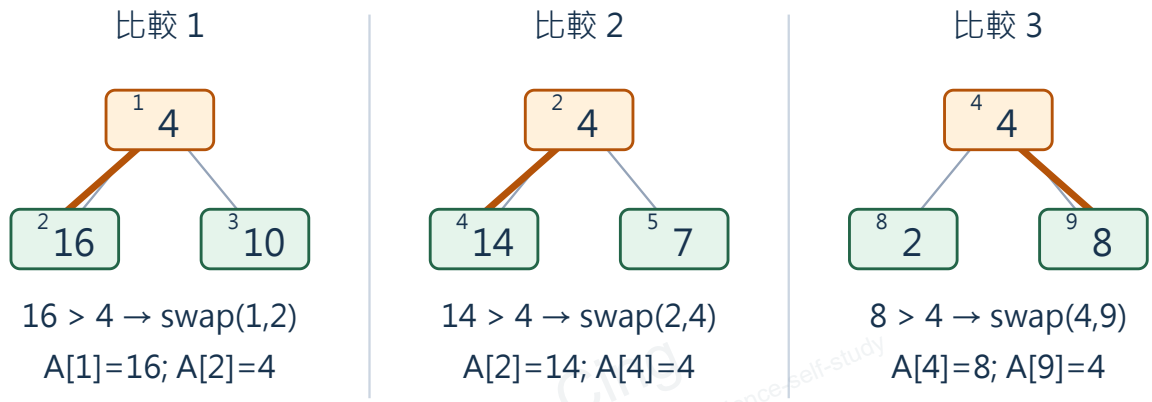
1	2	3	4	5
16	14	10	8	7
6	7	8	9	10
9	3	2	4	1

橘：本輪已完成的子樹根

綠：先前已合法；灰：待處理

舊根鍵 4 先後與 $16, 14, 8$ 交換，停在葉位置 9。最終各非葉的父子關係為 $16 \geq 14, 10$；$14 \geq 8, 7$；$10 \geq 9, 3$；$8 \geq 2, 4$；$7 \geq 1$。這些關係涵蓋所有九條樹邊，因此整棵樹為 max-heap；十筆資料的多重集合不變。

根的三次下降：索引路徑與鍵值分開看



每框畫的是交換前的局部狀態，下方列出交換後的鍵值。路徑索引為 $1 \to 2 \to 4 \to 9$，同一個舊鍵始終是 4；各步選中的子鍵則為 $16, 14, 8$。到位置 9 後，子索引 $18, 19$ 都超過 $s = 10$，所以停止。

逐次交換後的完整陣列

接續位置 3 修復完成後的陣列，所有索引固定為 $1..10$：

本輪根	交換索引	交換後 $A[1..10]$
2	$2 \leftrightarrow 5$	$[4, 16, 10, 14, 1, 9, 3, 2, 8, 7]$
2	$5 \leftrightarrow 10$	$[4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$
1	$1 \leftrightarrow 2$	$[16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$
1	$2 \leftrightarrow 4$	$[16, 14, 10, 4, 7, 9, 3, 2, 8, 1]$
1	$4 \leftrightarrow 9$	$[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$

核對方法：每次先在目前節點的有效子節點中選最大，再判斷它是否大於父鍵。一次交換只改兩格，其餘八格與資料多重集合保持不變。要追蹤下降路徑，應保存這些中間狀態；最終陣列本身沒有記錄完整操作歷史。

線性建堆：把節點高度逐層加總

先假設 $n \geq 1$。位置 i 至少高 h，當且僅當它最左的第 h 代後裔仍存在，也就是 $i2^h \leq n$。所以高度至少 h 的節點數**恰為** $\lfloor n/2^h \rfloor$。

把每個高度拆成 h 個單位，再交換有限求和次序：

$$\sum_{i=1}^{n} height(i) = \sum_{h=1}^{\lfloor \lg n \rfloor} \#\{i : height(i) \geq h\} = \sum_{h=1}^{\lfloor \lg n \rfloor} \left\lfloor \frac{n}{2^h} \right\rfloor < n.$$

一次修復花 $O(height(i)+1)$，呼叫共 $\lfloor n/2 \rfloor$ 次，因此總成本 $O(n)$。$n \geq 2$ 時，迴圈本身至少做 $\lfloor n/2 \rfloor$ 次固定工作，給 $\Omega(n)$，合為 $\Theta(n)$。

例如 $n = 15$，高度和為 $7+3+1 = 11$；不能把每個節點都當成根的高度。空堆積的高度和為零，沒有修復呼叫，直接返回；不代入 $\lg 0$，也不聲稱 $0 < 0$。逐筆插入的新鍵可能一路上浮，其最壞總成本 $\Theta(n \lg n)$；那是另一個建堆程序。

堆積排序：有效前綴逐步縮小

```
HEAPSORT(A):
    BUILD-MAX-HEAP(A)
    for end = length(A) downto 2:
        swap(A[1],A[end])
        MAX-HEAPIFY(A,1,end-1)
```

每輪開始時，$A[1..end]$ 為 max-heap；尾端 $A[end+1..n]$ 已遞增，而且尾端每個鍵都不小於堆積中的任一鍵。

交換把前綴最大鍵送到 $A[end]$，使排序尾端再長一格。有效大小改為 $end-1$，原根的兩棵子樹仍合法，所以只需修復新根。交換保留資料多重集合；終止時剩一筆前綴，整體有序。

建堆 $O(n)$，其後至多 $n-1$ 次修復各花 $O(\lg(n+1))$，故時間 $O(n \lg(n+1))$。比較排序下界給最壞 $\Theta(n \lg n)$。本節相等即停的實作，遇全相等鍵可只花 $\Theta(n)$，不能把最壞界套到每份輸入。

迭代修復使額外空間 $O(1)$；若改用遞迴修復，須另計呼叫堆疊。

十筆堆積排序：九次取出最大鍵

從建堆結果 $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ 開始。每輪先交換位置 1 與舊末端 $s+1$，把最大鍵送到排序尾端，再修復新大小 s 的前綴。表中的路徑只列後續下沉，不含根與末端的交換。

新大小 s	修復路徑	修復後堆積前綴	已排序尾端
9	1→2→4→9	$[14, 8, 10, 4, 7, 9, 3, 2, 1]$	$[16]$
8	1→3→6	$[10, 8, 9, 4, 7, 1, 3, 2]$	$[14, 16]$
7	1→3→7	$[9, 8, 3, 4, 7, 1, 2]$	$[10, 14, 16]$
6	1→2→5	$[8, 7, 3, 4, 2, 1]$	$[9, 10, 14, 16]$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最後五輪：前綴縮到只剩一筆

新大小 s	修復路徑	修復後堆積前綴	已排序尾端
5	1→2→4	$[7, 4, 3, 1, 2]$	$[8, 9, 10, 14, 16]$
4	1→2	$[4, 2, 3, 1]$	$[7, 8, 9, 10, 14, 16]$
3	1→3	$[3, 2, 1]$	$[4, 7, 8, 9, 10, 14, 16]$
2	1→2	$[2, 1]$	$[3, 4, 7, 8, 9, 10, 14, 16]$
1	—	$[1]$	$[2, 3, 4, 7, 8, 9, 10, 14, 16]$

例如第二輪，舊末端的鍵 1 到根後沿 $1 \to 3 \to 6$ 下沉，先與 10、再與 9 交換。每輪的尾端皆遞增，而且所有尾端鍵都不小於有效堆積的任一鍵；最後合併得到完整遞增序列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

優先權佇列：最大值、取出、升鍵與插入

$Q.A[1..Q.s]$ 是 max-heap，陣列可寫容量為 `Q.capacity`。以下固定容量程序先檢查空／滿；所有交換搬動整筆資料，鍵值只決定優先權。

```
MAXIMUM(Q):
    if Q.s == 0: return EMPTY
    return Q.A[1]

EXTRACT-MAX(Q):
    if Q.s == 0: return EMPTY
    result = Q.A[1]; Q.A[1] = Q.A[Q.s]; Q.s -= 1
    if Q.s > 0: MAX-HEAPIFY(Q.A,1,Q.s)
    return result
```

`MAXIMUM` 為 $O(1)$；`EXTRACT-MAX` 保存最大資料，以末筆補根、縮小有效大小，再向下修復，最壞 $O(\lg(s+1))$。單筆取出後大小為零，不讀新的根。

升鍵：只沿祖先方向修復

```
SIFT-UP(Q,i):
    while i > 1 and key(Q.A[floor(i/2)]) < key(Q.A[i]):
        swap(Q.A[i],Q.A[floor(i/2)])
        i = floor(i/2)

INCREASE-KEY(Q,i,k):
    require 1 <= i <= Q.s and k >= key(Q.A[i])
    set-key(Q.A[i],k)
    SIFT-UP(Q,i)
```

升鍵可能破壞的只有祖先方向：新鍵仍不小於原有子鍵。每次交換修好目前父子邊，候選鍵繼續向上；到根或父鍵不小於它時停止，最壞 $O(\lg(s+1))$。`and` 從左往右短路，先確認 $i > 1$。

直接插入新資料，不需要哨兵鍵

```
INSERT(Q,record):
    if Q.s == Q.capacity: return FULL
    Q.s += 1
    Q.A[Q.s] = record
    SIFT-UP(Q,Q.s)
```

直接把新資料寫在新的最後位置，再上浮，不需額外的負無窮哨兵。固定容量且未滿時，插入最壞為 $O(\lg(s+1))$；若動態陣列需要重新配置，單次搬移可能為 $\Theta(s)$，須另外分析擴容的攤銷成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從位置八開始，核對每次上浮

升鍵沿父索引 8 → 4 → 2 上浮

	1	2	3	4	5	6	7	8
原堆積	20	15	18	7	12	9	17	2
位置 8 升鍵	20	15	18	7	12	9	17	**16**
交換 8 ↔ 4	20	15	18	**16**	12	9	17	7
交換 4 ↔ 2	20	**16**	18	15	12	9	17	7

16 先後與父鍵 7、15、20 比較；最後 20 ≥ 16，停止上浮。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

例中先在位置 8 寫入 16，依序比較父鍵 $7, 15, 20$；交換 $8 \leftrightarrow 4$、$4 \leftrightarrow 2$ 後停止。位置索引固定，資料身分隨交換移動。

降鍵、指定刪除與相等優先權

max-heap 的降鍵只可能破壞往子節點的邊，故向下修復。刪除位置 i 時，以末筆補洞並縮小大小；若 i 已是舊末端，直接縮小即可。

設替補鍵為 z。若 $i > 1$ 且 $z > A[parent(i)]$，則因原父鍵不小於原被刪鍵、原被刪鍵不小於其子鍵，z 也支配那些子鍵，只需上浮。否則父邊合法，只需下沉。各操作最壞 $O(\lg(s+1))$，前提是已知位置；搜尋任意鍵仍可能要 $\Theta(s)$。

若用 handle（資料身分代號）定位，交換時必須同步更新 handle 到索引的對照表，移除被刪資料的對照。陣列索引本身不會永遠指向同一筆資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

排序與佇列的同鍵次序，使用不同的序號方向

堆積排序並不穩定：$[2a, 2b, 1c]$ 可排成 $[1c, 2b, 2a]$。若要遞增排序時保留同鍵原順序，可比較 $(key, original_index)$；較晚的同鍵資料較大，會先被放到尾端。若 max-priority queue 要讓同優先權者先到先服務，則比較 $(priority, -arrival_index)$。兩者的序號方向不同，且都改變了比較與儲存規格。

同優先權資料 a, b 依序抵達，原序號分別為 $1, 2$，鍵皆為 5：

用途	a 的比較鍵	b 的比較鍵	先取出最大者	所需次序
穩定遞增排序	$(5, 1)$	$(5, 2)$	b，先放到尾端	輸出 a, b
同優先權先到先服務	$(5, -1)$	$(5, -2)$	a，立即服務	服務 a, b

遞增排序會把較大的鍵從右往左放；佇列則直接服務被取出的最大者。使用序號前，先確認「取出的先後」如何對應目標次序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雇用問題：比較成本與替換成本要分開

有 n 位能力相異、可比較排名的候選人；排名越大越好。每次面試後，只要比現任者好就立刻替換。虛擬候選人 dummy 比任何人都差。

```
HIRE(A):
    best = dummy; hires = 0
    for i = 1 to n:
        interview A[i]
        if rank(A[i]) > rank(best):
            best = A[i]
            hire best; hires += 1
    return (best, hires)
```

第 i 輪後，best 是前 i 位中能力最好的候選人：初始無人；新來者較好才替換，否則保留原最佳者。因此最後能找出全體最佳者。

同一個程序，三種不同的成本問題

面試費為 c_I、每次雇用費為 c_H，雇用 X 次的總費用為 $nc_I + Xc_H$。空輸入返回 dummy、$X = 0$。對 $n \geq 1$，排名遞增時 $X = n$，逆序時 $X = 1$。接著區分三種問法：

問法	隨機來源或量詞	要算的量
最壞情形	所有合法輸入取最大	$\max_A X(A) = n$
平均情形	明定輸入分布，例如 $n!$ 種順序等機率	$\mathbb{E}_A[X(A)]$
隨機化程序	固定任意輸入，由程序均勻洗牌	$\mathbb{E}_R[X(A, R)]$

均勻洗牌使每個固定的相異排名輸入變成均勻排列；前提是能控制處理順序。期望保證仍容許壞執行：遞增排列的機率為 $1/n!$。雇用次數少，只降低雇用費，並未減少面試次數。

指示變數：把計數拆成可以求機率的事件

先固定有限樣本空間 Ω，各結果的機率非負、總和為 1。隨機變數 X 的期望是 $\mathbb{E}[X] = \sum_{\omega \in \Omega} \Pr(\omega) X(\omega)$。

事件 E 的指示變數 I_E 在 E 成立時為 1，否則為 0，故

$$\mathbb{E}[I_E] = 1 \cdot \Pr(E) + 0 \cdot \Pr(E^c) = \Pr(E).$$

若目標計數恰為 $X = \sum_{i=1}^{m} X_i$，交換兩個有限和可得

$$\mathbb{E}[X] = \sum_{\omega} \Pr(\omega) \sum_{i} X_i(\omega) = \sum_{i} \sum_{\omega} \Pr(\omega) X_i(\omega) = \sum_{i} \mathbb{E}[X_i].$$

這是**期望線性性**，不要求 X_i 獨立。獨立性可能用在計算單項事件的機率，或將交集機率寫成乘積時。

反例：令 $X_1 = X_2$ 都是同一次公平擲幣的正面指示。仍有 $\mathbb{E}[X_1 + X_2] = 1$，但 $\Pr(X_1 = X_2 = 1) = 1/2$，不是 $1/4$。

三個機率工具：聯集界、Markov 界與尾和

先辨認正在求「事件機率」「個數的期望」或「最大值的期望」。以下三個工具都可從指示變數推出。

聯集指示滿足 $I_{\cup_i E_i} \le \sum_i I_{E_i}$，取期望即得

$$\Pr\left(\bigcup_{i} E_i\right) \le \sum_{i} \Pr(E_i).$$

若 $X \ge 0$、$t > 0$，由 $X \ge t \, I_{\{X \ge t\}}$ 得 Markov 界

$$\Pr(X \ge t) \le \frac{\mathbb{E}[X]}{t}.$$

整數尾和：每個結果到達了幾個門檻

若 L 為非負整數，逐個結果皆有 $L = \sum_{j \ge 1} I_{\{L \ge j\}}$，故

$$\mathbb{E}[L] = \sum_{j \ge 1} \Pr(L \ge j).$$

有限上界時可直接交換有限和；無限尾和由非負部分和的單調極限成立，兩側亦可同為無窮大。這三個工具均不需要事件獨立。

獨立性若用在具體例子的機率乘法，仍須另外明列；不能因工具不要求獨立，就省略模型中的獨立條件。

數例：$L = 0, 1, 3$ 的機率分別為 $1/2, 1/4, 1/4$，直接算得期望 $0/2 + 1/4 + 3/4 = 1$。尾機率依序為 $\Pr(L \ge 1) = 1/2$、$\Pr(L \ge 2) = 1/4$、$\Pr(L \ge 3) = 1/4$，加總也等於 1。

雇用次數：先算每個位置刷新紀錄的機率

假設 $n!$ 種相異排名排列等機率，令 X_i 表示第 i 位是否被雇用。固定前 i 位的候選人集合後，這些人的 $i!$ 種相對順序仍等可能，最高者落在最後位置的機率為 $1/i$。

第 i 位被雇用，恰好等價於「它是前 i 位的最高者」，所以

$$\mathbb{E}[X] = \sum_{i=1}^{n} \Pr(X_i = 1) = \sum_{i=1}^{n} \frac{1}{i} = H_n.$$

由 $1/x$ 遞減，矩形面積與積分比較給出

$$\int_{1}^{n+1} \frac{dx}{x} \le H_n \le 1 + \int_{1}^{n} \frac{dx}{x}, \qquad n \ge 1.$$

因此 $\mathbb{E}[X] = \Theta(\ln n)$，而總期望費用仍為 $n c_I + c_H H_n$；面試費這一項並未變成對數。

六人逐輪雇用，與同規模期望值對照

位置 i	排名	面試後最佳排名	本次雇用？	累積次數
1	2	2	是	1
2	5	5	是	2
3	1	5	否	2
4	4	5	否	2
5	6	6	是	3
6	3	6	否	3

數例 $[2,5,1,4,6,3]$ 在位置 $1,2,5$ 刷新紀錄，共雇用三次；同規模平均為 $H_6 = 49/20 = 2.45$。只有一次雇用的機率為 $1/n$；每位都雇用只有一種遞增排列，機率為 $1/n!$。空輸入另取 $H_0 = 0$。

同一個方法，也能算逆序對與拿回原帽的人數

均勻排列 n 個相異鍵。對位置 $i < j$，令 I_{ij} 表示 $A[i] > A[j]$。交換兩位置的值，將逆序與非逆序結果一對一配對，因此 $\Pr(I_{ij} = 1) = 1/2$。

$$V = \sum_{i<j} I_{ij}, \qquad \mathbb{E}[V] = \binom{n}{2}\frac{1}{2} = \frac{n(n-1)}{4}.$$

共享位置的逆序事件可能相依；這不影響期望線性性。當 $n = 2$，逆序數只能是 0 或 1，但期望為 $1/2$。

另一例：將 $n \geq 1$ 頂帽子均勻地一對一還給原主人們。第 i 人拿回自己的帽子的機率為 $1/n$，固定點數 F 滿足

$$F = \sum_{i=1}^{n} I_i, \qquad \mathbb{E}[F] = n\frac{1}{n} = 1.$$

$n = 2$ 時，可能兩人都拿對或都拿錯，各有 $1/2$ 機率。期望為一人，不表示一定恰有一人拿對。$n = 0$ 時直接有 $F = 0$。

均勻整數抽樣：拒絕超出範圍的位元結果

RANDOM(a,b) 的契約是回傳包含兩端的整數，各機率 $1/(b-a+1)$，且新呼叫條件於歷史仍均勻。以下以公平、獨立的位元實作；必須有整數 $a \le b$。

令 $m = b - a + 1$。$m = 1$ 直接回傳 a；否則令 $k = \lceil \lg m \rceil$，反覆讀取 k 個位元作為 $z \in \{0, \dots, 2^k - 1\}$。若 $z < m$，回傳 $a + z$；否則重抽整組。

每輪接受率 $p = m/2^k > 1/2$。對任一合法值 v，回傳它的機率為

$$\sum_{t=0}^{\infty} (1-p)^t \frac{1}{2^k} = \frac{1}{p 2^k} = \frac{1}{m}.$$

重試輪數期望為 $1/p < 2$，因此期望位元成本為 $O(\lg m)$。拒絕抽樣幾乎必然終止，沒有預先固定的重試次數上限。

直接取模會產生偏差：八個三位元結果對 3 取模，餘數 $0, 1, 2$ 的前像數為 $3, 3, 2$，不是等機率。偽亂數產生器與整數取樣方式，也須分別滿足所採模型。

原地洗牌：每輪多固定一個位置

每輪從尚未固定的後綴（suffix）中均勻選一筆，與位置 i 交換。即使鍵相同，也把每筆資料視為不同身分。

```
SHUFFLE(A):
    for i = 1 to n:
        j = RANDOM(i,n)
        swap(A[i],A[j])
```

第 i 輪後，後續交換不再碰 $A[1..i]$。空輸入只有一種空排列；單筆輸入只與自己交換。單位成本整數抽樣模型下，時間 $\Theta(n)$、額外空間 $O(1)$；位元抽樣成本另計。

均勻洗牌：逐輪延伸指定的前綴

主張：第 i 輪前，任一指定的 $i-1$ 個不同身分序列占據 $A[1..i-1]$ 的機率為 $(n-i+1)!/n!$。初始 $i=1$，空前綴的機率為一。

固定此前綴後，下一個指定身分恰占剩餘 $n-i+1$ 格中的一格。條件均勻抽樣以 $1/(n-i+1)$ 的機率選中它，因此指定長 i 前綴的機率為

$$\frac{(n-i+1)!}{n!}\frac{1}{n-i+1}=\frac{(n-i)!}{n!}.$$

歸納至 $i=n$，每個完整排列的機率皆為 $0!/n!=1/n!$。取 $n=4$、指定輸出 $[C,D,A,B]$：

固定長度	0	1	2	3	4
指定前綴	空	C	C,D	C,D,A	C,D,A,B
機率	1	$1/4$	$1/12$	$1/24$	$1/24$

四筆洗牌：逐輪檢查固定前綴

從 $[A,B,C,D]$ 開始，依序抽到 $j=3,4,3,4$。字母是身分，不是比較用的鍵。

輪次 i	合法索引	抽到 j	交換後完整陣列	已固定前綴
1	$1..4$	3	$[C,B,A,D]$	C
2	$2..4$	4	$[C,D,A,B]$	C,D
3	$3..4$	3	$[C,D,A,B]$	C,D,A
4	4	4	$[C,D,A,B]$	C,D,A,B

此路徑機率為 $(1/4)(1/3)(1/2)(1)=1/24$。任意指定輸出也唯一決定一條合法路徑：依次在剩餘後綴找到指定的下一個身分，再將它換到位置 i。

已固定的前綴，後面不再移動

從 [A,B,C,D] 開始：每輪只從尚未固定的 suffix 選一個

	位置1	位置2	位置3	位置4	本輪機率
i=1, j=3	C	B	A	D	1/4
i=2, j=4	C	D	A	B	1/3
i=3, j=3	C	D	A	B	1/2
i=4, j=4	C	D	A	B	1

綠格已固定、不再改動；這條路徑機率 1/4·1/3·1/2·1=1/24。

後面的輪次只交換尚未固定的位置；這是前綴能永久固定的原因。單次追蹤用來核對程序，一般均勻性仍由前面的聯合不變量證明。

每個位置看似均勻，整個排列仍可能有偏

隨機選一個循環平移量，確實能讓每筆以 $1/n$ 機率落在每個位置；但 $n \geq 3$ 時只有 n 種可能輸出，而非 $n!$ 種。均勻邊際分布不等於聯合均勻。

另一個錯誤程序讓 $i = 1..n$，每輪都從整個 $1..n$ 抽 j 再交換。$n = 3$ 時有 $3^3 = 27$ 條等可能亂數路徑；若六種排列均勻，每種須對應 $27/6$ 條路徑，但路徑數必為整數，故不可能。

錯誤程序會重新移動先前的位置，因此不能沿用「已固定前綴」的證明。證明某個程序不均勻，與只說「看起來不夠隨機」不同。

正確洗牌若只做前 m 輪，$0 \leq m \leq n$，每個有序樣本的機率為 $(n-m)!/n!$；忽略順序後，每個 m 元子集合有 $m!$ 種排列，機率為 $1/\binom{n}{m}$。這省下後續交換；若輸入必須先建成可索引陣列，建表成本仍要計入。

隨機優先值：沒有碰撞時才直接得到均勻排列

給 $n \geq 1$ 筆資料獨立均勻的優先值 $P_i \in \{1, \ldots, M\}$，其中整數 $M \geq n$，再按優先值排序。條件於所有優先值不同，交換資料標籤會一對一對應不同次序，且保留機率，因此 $n!$ 種相對順序等可能。

每對碰撞機率為 $1/M$。聯集界（union bound）給

$$\Pr(\exists i < j : P_i = P_j) \leq \frac{\binom{n}{2}}{M}.$$

取 $M = n^3$、$n \geq 2$，上界為 $(n-1)/(2n^2) < 1/(2n)$。碰撞很少，仍不等於不會碰撞。

若平手時保留原順序，無條件輸出有偏：$n = 2$ 時，第一筆先出的機率為 $\Pr(P_1 \leq P_2) = (M+1)/(2M) > 1/2$。

可在任何碰撞時整組重抽，再接受無碰撞結果；上述 M 下接受率有正常數下界，期望輪數為常數。也可在每個平手群中獨立均勻洗牌。用比較排序實作優先值排序需 $\Theta(n \lg n)$ 時間與 $O(n)$ 優先值空間；重抽另須偵測碰撞，不能當成免費。

隨機化之後，仍要說清楚正確性與可控制的部分

`RANDOMIZED-HIRE` 先均勻洗牌，再執行 `HIRE`。對每個固定的相異排名輸入，洗牌後皆為均勻排列，故期望雇用數為 H_n，期望費用為 $nc_I + c_H H_n$。另有 $\Theta(n)$ 洗牌操作，不能把操作時間與雇用費用混成同一單位。

若候選人只能依外部順序到達、不能重新安排，就沒有這個洗牌控制權。此時 H_n 的分析仍須對外部順序假設均勻，不能用一個無法執行的步驟取消輸入假設。

本章的洗牌與隨機 pivot 排序，隨機的是成本；每次返回都符合規格。這類稱 Las Vegas 演算法。Monte Carlo 演算法允許小機率錯答，必須另列錯誤機率與其適用輸入；只給期望時間不能推得正確率。

後面的線上遴選又是不同目標：策略可能沒有選到全域最佳，但仍按規定選出一人。它的成功機率要依遴選模型另證，不能與排序的正確性保證混用。

快速排序：分割後，兩側仍要各自排序

本節使用 1 起算閉區間 $A[p..r]$。選定 pivot（分割基準值）後，分割只建立它與兩側的大小關係，還沒有把兩側內部排好。

```
QUICKSORT(A,p,r):
    if p >= r: return
    q = PARTITION-LAST(A,p,r)
    QUICKSORT(A,p,q-1)
    QUICKSORT(A,q+1,r)
```

空區間 $p > r$、單筆區間 $p = r$ 都直接返回。分割回傳 pivot 的位置 q；兩個遞迴區間都**排除** q，所以長度嚴格縮小。這個回傳契約是排序程序正確與終止的前提。

尾端分割：保存鍵值，交換整筆資料

尾端版本的分割程序如下。鍵值 x 先保存；交換搬動整筆資料，其他欄位隨鍵一起移動。

```
PARTITION-LAST(A,p,r):
    x = key(A[r]); i = p-1
    for j = p to r-1:
        if key(A[j]) <= x:
            i += 1
            swap(A[i],A[j])
    swap(A[i+1],A[r])
    return i+1
```

迴圈邊界：i 是小鍵區尾端，j 是下一筆待分類

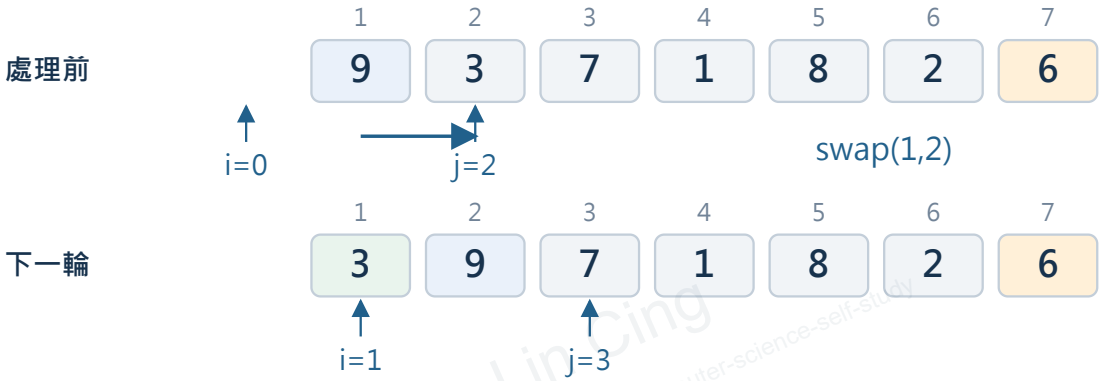
每輪開始時，pivot 固定在 A[r]。本章採 **1 起算索引、閉區間 A[p..r]**；i 可以等於 p−1，表示 ≤ 區為空。

區域	已知關係	初始狀態 i=p−1、j=p
A[p..i]	≤ pivot	空
A[i+1..j−1]	> pivot	空
A[j..r−1]	尚未分類	除 pivot 外全部
A[r]	pivot	全程固定到收尾交換

讀圖方式： 綠色是 ≤ 區，藍色是 > 區，灰色是未知區，橘色是 pivot。以下兩頁都沿用 [9,3,7,1,8,2,6]、pivot=6。

小鍵到來：交換後，兩個邊界都前進

3 ≤ 6：先讓 i 前進，再交換 A[1] 與 A[2]

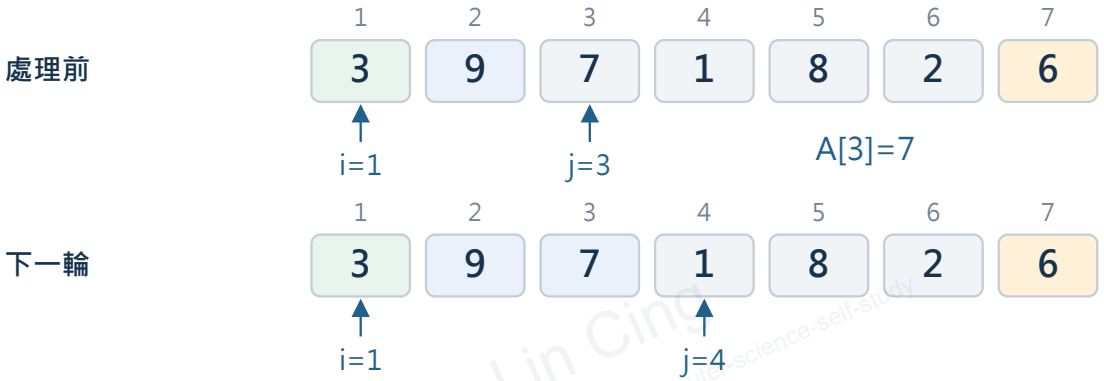


3 加入 ≤ 區；9 回到 > 區。pivot 6 仍在索引 7。

處理 A[2]=3 時，先令 i=1，再交換 A[1]、A[2]。3 加入 ≤ 區，原先的 9 留在 > 區；最後 j 前進。一般情況也可能 i=j，此時是自己與自己交換。

大鍵到來：不搬資料，只擴大已分類區

7 > 6：不交換、不改 i，只讓 j 前進

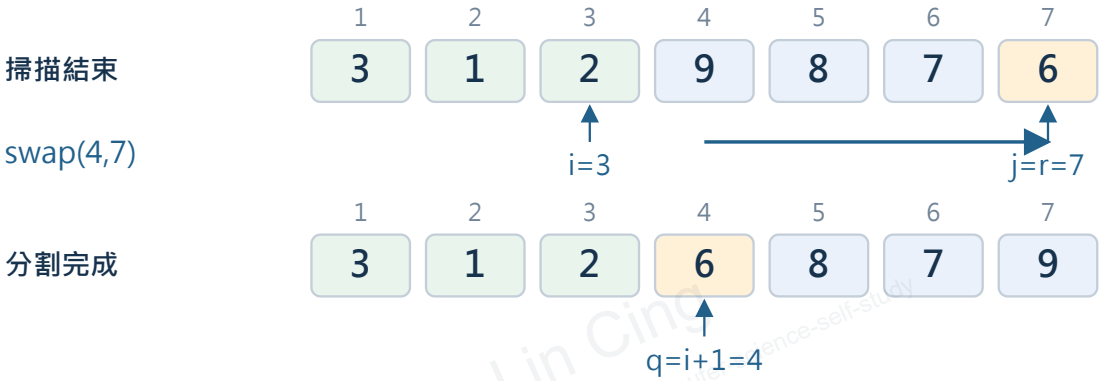


7 從未知區加入 > 區；陣列值不變，但分類邊界改變。

A[3]=7>6，因此 i 不變、陣列不變，只把 j 往右移一格。**不變的是值，改變的是已知資訊：** 7 現在屬於 > 區，不再是未知值。

收尾交換：q 才是 pivot 的最終位置

掃描結束後，pivot 才移到兩區之間



≤6：[3,1,2] | pivot 6 | >6：[8,7,9]；兩側仍未排序。

每輪未知區長 r−j 減一，所以掃描必終止。結束時 j=r；交換 A[i+1] 與 A[r]，得到 q=i+1。左區 ≤ pivot、右區 > pivot，兩側內部仍未排序。

從分割不變量到整體排序正確性

1. **初始化：** i=p−1、j=p，前兩區都是空區間，不變量成立。
2. **保持：** 小鍵交換後擴大 ≤ 區；大鍵只擴大 > 區。兩種情況都使未知區少一筆，pivot 仍固定。
3. **分割終止：** 未知區消失，收尾交換把 pivot 放到 q。交換只重排元素，不會遺失或新增資料。
4. **遞迴歸納：** 空區間、單元素已排序。兩個子區間排除 q，長度都變小；歸納假設使它們各自有序，再接上 pivot，整個區間便有序。

只檢查「左小右大」還不夠：完整正確性同時需要**元素多重集合不變、遞迴嚴格縮小**，以及兩側遞迴回來後確實有序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整分割追蹤：每次比較與交換都列出

輸入 $A = [9, 3, 7, 1, 8, 2, 6]$、$p = 1, r = 7$。尾端鍵 $x = 6$，初始 $i = 0$；表中 i 是本輪判斷後的值。

掃描索引 j	判斷	i	操作後的完整陣列
1	$9 > 6$，不交換	0	$[9, 3, 7, 1, 8, 2, 6]$
2	$3 \le 6$，交換 $1, 2$	1	$[3, 9, 7, 1, 8, 2, 6]$
3	$7 > 6$，不交換	1	$[3, 9, 7, 1, 8, 2, 6]$
4	$1 \le 6$，交換 $2, 4$	2	$[3, 1, 7, 9, 8, 2, 6]$
5	$8 > 6$，不交換	2	$[3, 1, 7, 9, 8, 2, 6]$
6	$2 \le 6$，交換 $3, 6$	3	$[3, 1, 2, 9, 8, 7, 6]$
掃描結束	交換 $4, 7$	3	$[3, 1, 2, 6, 8, 7, 9]$

回傳 $q = i + 1 = 4$。左側 $[3, 1, 2] \le 6$、右側 $[8, 7, 9] > 6$，兩側都還沒排好。掃描恰做 $r - p = 6$ 次鍵比較；連同收尾，時間 $\Theta(r - p + 1)$、額外空間 $O(1)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最壞時間：一整族輸入達到平方成本

已排序的 n 個相異鍵，每次取最後位置作 pivot。pivot 永遠最大，子問題長度是 n−1 與 0。

本輪長度	本輪鍵比較	下一輪非空長度
n	n−1	n−1
n−1	n−2	n−2
…	…	…
2	1	1

比較總數為 $n(n-1)/2$，給出最壞時間的 Ω(n²) 下界。這個結論適用整個輸入族，並非由單次實測推論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

平方上界必須涵蓋每一種分割

令 C(n) 是 n 個鍵的最壞比較數，子問題大小 a+b=n−1。以較小規模的歸納假設可得

$$C(a)+C(b)+(n-1)\le \frac{a(a-1)}{2}+\frac{b(b-1)}{2}+n-1=\frac{n(n-1)}{2}-ab\le \frac{n(n-1)}{2}.$$

C(0)=C(1)=0，故上界涵蓋所有分割；a=0 或 b=0 時恰達界。每個被選作 pivot 的元素不再進入遞迴，所以呼叫總數 O(n)。總時間 O(n+C)，配上下界得到**最壞 Θ(n²)**。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

均分時，先按層數清楚計算比較工作

取 n=15，每次選真正的中位數。此表只計 Lomuto 的鍵比較，單元素不呼叫 partition。

深度 d	子問題個數	每個長度	本層鍵比較
0	1	15	14
1	2	7	2×6=12
2	4	3	4×2=8
3	8	1	0

共 34 次；同樣 15 個鍵，端點分割則做 105 次。差別來自每層保留下來的子問題大小，與 partition 本身仍是線性掃描並不矛盾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不必完全平分，固定比例也能保證對數深度

若每次非空子問題的較大側至多為原長度的 ρ 倍，其中 ρ<1 是固定常數，最長分支深度為 O(log n)。同一層的子區間互不重疊，本層總掃描量至多 n，因此總時間 O(n log n)；固定的 1:9 分割也符合。

完全均分且 $n = 2^k - 1$ 時，深度 d 有 2^d 個長度 $2^{k-d} - 1$ 的子問題。本層比較數為

$$2^d(2^{k-d} - 2) = n + 1 - 2^{d+1}, \qquad 0 \le d \le k - 2.$$

前約 k/2 層，每層仍做 Θ(n) 次比較，故這種執行本身就需 Ω(n log n)，合為 Θ(n log n)。這裡的下界來自實際工作量；比較排序的最壞輸入下界並不保證每個輸入都一樣慢。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

重複鍵：隨機 pivot 不能修好全相等輸入

對 $[5,5,5,5,5]$，每個 <= x 都成立，故每輪 $q=r$。子問題長度依序為 $5,4,3,2$，比較次數 $4+3+2+1=10$；一般 n 筆為 $n(n-1)/2$。

隨機換來的 pivot 仍是 5，不能改變這條執行路徑。因此本節稍後對二路版本推導的期望 $\Theta(n \lg n)$，先限定**鍵互異**。本章次序統計量節會完整給出三路分割：小於、等於、大於 pivot；只有兩個嚴格區繼續遞迴。全相等時只分割一次，總時間 $\Theta(n)$。

穩定性是另一項保證。以身分區別輸入 $[2_a, 2_b, 1_c]$，尾端 pivot 為 1，收尾交換得到 $[1_c, 2_b, 2_a]$；兩筆鍵 2 的順序已反轉。隨機挑 pivot 並不會使這個交換式實作自動變成穩定排序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

只遞迴較小側：控制堆疊，沒有省掉分割

原本兩側都遞迴的程序，最壞有 $\Theta(n)$ 層堆疊；分割原地進行，不代表整個快速排序只用常數空間。可改成只遞迴較小側，較大側留在本層迴圈處理。

```
QUICKSORT-SMALL-STACK(A,p,r):
    while p < r:
        q = PARTITION-LAST(A,p,r)
        if q-p < r-q:
            QUICKSORT-SMALL-STACK(A,p,q-1)
            p = q+1
        else:
            QUICKSORT-SMALL-STACK(A,q+1,r)
            r = q-1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

較小側遞迴，限制尚未返回的呼叫鏈

兩側長度 $a + b = n - 1$，真正遞迴的一側至多 $\lfloor (n-1)/2 \rfloor$。沿尚未返回的呼叫鏈，規模至少減半，故堆疊為最壞 $O(\lg(n+1))$。

若 pivot 每次都是最大值，遞迴側為空，迴圈仍依次分割長度 $n, n-1, \ldots, 2$。此時堆疊甚至為常數，時間卻仍是 $\Theta(n^2)$；空間與時間要分別計算。

每次選中位數：一條尚未返回的呼叫鏈

巢狀框架

15 → 7 → 3 → 1：同時存活四個框架

每次選最大值：同一框架的連續迴圈

同一工作框架

每輪的空遞迴呼叫立即返回；較大側留在同一框架中，仍須執行下一次分割。

隨機化保證的是每個固定輸入的期望成本

每輪在目前閉區間 $p..r$ 均勻抽一個整數 k，交換 $A[k]$ 與 $A[r]$，再用同一個尾端分割。未來選擇條件於當時狀態仍須均勻。

鍵互異時，不論目前排列如何，每個局部秩成為 pivot 的機率都是 $1/n$。接下來取期望的隨機來源是**演算法內部選擇**，不要求使用者提供均勻排列。

任何固定輸入仍可能遇到每次都選端點的亂數路徑，花費 $\Theta(n^2)$。期望界既不等於每次執行的上界，也不能直接當成尚未證明的尾機率界。

數學模型的均勻抽樣須與實作分開。固定 seed 可重現某條偽亂數序列，但「程式呼叫了 random」本身不足以證明抽樣均勻；有界整數抽樣可使用本章前面的拒絕抽樣程序。

比較一對鍵，關鍵在秩區間內的第一個 pivot

將相異鍵依大小命名為 $z_1 < \cdots < z_n$。令 X_{ij} 表示 z_i, z_j 是否直接做過一次鍵比較。它們至多比較一次：一次比較必有一方為 pivot，而 pivot 隨即退出遞迴。

考慮 $Z_{ij} = \{z_i, \ldots, z_j\}$。區間外的 pivot 把這整段留在同一側；區間內的 pivot 若是端點 z_i 或 z_j，本次便比較兩者；若是內部鍵，兩者分居不同遞迴區，往後不再相遇。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

區間外的 pivot，不會分開這一對鍵

分析排序後的秩：$Z_{2,5}$ = {2,3,4,5}；不是陣列快照

先選 7：2..5 一起留在左子問題，$X_{2,5}$ 尚未決定。

區間內先選端點 2（或 5）

該次比較 2 與 5 → $X_{2,5}$=1

區間內先選內部 3（或 4）

2 在左、5 在右 → 永不比較，$X_{2,5}$=0

四個區間鍵等可能先選；其中兩個是端點。Pr($X_{2,5}$=1)=2/4=1/2。

因此 $X_{ij} = 1$，恰好等價於「Z_{ij} 內第一個被選為 pivot 的鍵是兩個端點之一」。它不必是全程第一個 pivot。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從四種首選情況，推到一般比較機率

追蹤 z_2, z_5，只看 $Z_{2,5} = \{z_2, z_3, z_4, z_5\}$ 內誰先成為 pivot。

區間內第一個 pivot	z_2, z_5 的去向	$X_{2,5}$
z_2	本次直接比較	1
z_3	分居兩側，不再相遇	0
z_4	分居兩側，不再相遇	0
z_5	本次直接比較	1

四個鍵各有 $1/4$ 機率成為區間內首選，因此比較機率為 $1/2$。一般區間長 $m = j - i + 1$，故

$$\Pr(X_{ij} = 1) = \frac{2}{m} = \frac{2}{j - i + 1}.$$

證明區間內第一個 pivot 確實均勻

等可能性需要證明。固定區間 Z 的 m 個鍵及其中一個指定鍵 z，對目前包含 Z 的子問題大小 N 歸納。$N = m$ 時，首個 pivot 均勻，指定鍵機率 $1/m$。$N > m$ 時，本次直接選到 z 的機率 $1/N$；選到其餘 Z 內鍵則失敗。

若先選到 Z 外鍵，含 Z 的後續子問題嚴格縮小，依歸納指定鍵首選機率仍是 $1/m$。對 $N - m$ 個外部鍵求和得

$$\frac{1}{N} + \frac{N - m}{N}\frac{1}{m} = \frac{1}{m}.$$

因此區間內首選確實均勻；分母是 m，不是原輸入長度 n。其他遞迴分支可穿插執行，但不會移動或比較此子區間內的鍵。

期望比較數：把秩區間長度逐項加總

總鍵比較數 $C = \sum_{1 \le i < j \le n} X_{ij}$。期望線性性不需要這些事件獨立。長度 $k = j - i + 1$ 的秩區間共有 $n + 1 - k$ 個，因此

$$\begin{aligned}\mathbb{E}[C] &= \sum_{i<j} \frac{2}{j-i+1} = 2\sum_{k=2}^{n} \frac{n+1-k}{k} \\ &= 2(n+1)\sum_{k=2}^{n} \frac{1}{k} - 2(n-1) \\ &= 2(n+1)H_n - 4n, \qquad H_n = \sum_{k=1}^{n} \frac{1}{k}.\end{aligned}$$

由 $\ln(n+1) \le H_n \le 1 + \ln n$，得到期望比較數 $\Theta(n \lg n)$。每輪掃描工作與本輪鍵比較次數同階，所有呼叫另有 $O(n)$ 開銷，所以期望時間也是 $\Theta(n \lg n)$。

此精確式適用鍵互異、各輪條件均勻、每個非 pivot 鍵只做一次 <= 的版本。它不計索引測試、交換或產生亂數的成本。比較事件彼此相依，不能據此直接套用獨立變數的變異數或尾界公式。

首端 pivot 的分割：先看它自己的不變量

以下是首端 pivot 的單向掃描版本；閉區間仍是 $A[p..r]$，但 $x = A[p]$ 固定到收尾。

```
PARTITION-FIRST(A,p,r):
    x = A[p]; i = p
    for j = p+1 to r:
        if key(A[j]) <= key(x):
            i = i+1
            swap(A[i],A[j])
    swap(A[p],A[i])
    return i
```

每輪開始時 $A[p] = x$、$A[p+1..i] \le x$、$A[i+1..j-1] > x$，$A[j..r]$ 尚未分類。小鍵先擴大左區再交換，大鍵只移動掃描邊界；未知區每輪少一筆。

終止時交換 $A[p]$ 與 $A[i]$，回傳位置 i。排序遞迴仍須排除該位置。不能把首端版本的初始 $i = p$，直接抄進尾端版本的 $i = p - 1$ 規格。

同一個 pivot，兩種分割都能核對

首端版輸入 $[6, 10, 13, 5, 8, 3, 2, 11]$，取 $x = 6$、初始 $i = 1$。

j	判斷與操作	操作後陣列
2	$10 > 6$，$i = 1$	$[6, 10, 13, 5, 8, 3, 2, 11]$
3	$13 > 6$，$i = 1$	同前
4	$5 \le 6$，$i = 2$，交換 $2, 4$	$[6, 5, 13, 10, 8, 3, 2, 11]$
5	$8 > 6$，$i = 2$	同前
6	$3 \le 6$，$i = 3$，交換 $3, 6$	$[6, 5, 3, 10, 8, 13, 2, 11]$
7	$2 \le 6$，$i = 4$，交換 $4, 7$	$[6, 5, 3, 2, 8, 13, 10, 11]$
8	$11 > 6$，$i = 4$	同前
結束	交換 $1, 4$，回傳 4	$[2, 5, 3, 6, 8, 13, 10, 11]$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對照兩版的相同 pivot

同一個 pivot 鍵，區內排列可以不同

兩版皆 q=4、pivot=6；左側 ≤6、右側 >6，區內不必排序。

尾端版若也要選同一個 6，須先交換首尾，再分割 $[11, 10, 13, 5, 8, 3, 2, 6]$，可得 $[5, 3, 2, 6, 8, 10, 13, 11]$。兩版都把 6 放在位置 4，各做 7 次掃描鍵比較；兩側的內部排列可以不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

名稱相近時，對照前提、邊界與回傳值

版本	pivot 與初始邊界	回傳與遞迴
CLRS 尾端單向掃描	$x = A[r]$，$i = p - 1$，$j = p$	回傳單點 q；遞迴 $[p, q-1]$、$[q+1, r]$
MIT 2005 首端單向掃描	$x = A[p]$，$i = p$，$j = p + 1$	回傳單點 q；仍排除 q
首端雙向掃描、最後定位	$x = A[p]$，左右掃描遇等值即停	指標交錯後交換 pivot，回傳單點 q
本章三路分割	先保存 pivot 鍵 x，$lt = i = p$，$gt = r$	回傳等值閉區間 $[lt, gt]$；整段排除

雙向掃描仍須再看回傳契約：經典 Hoare partition 回傳分界線，不保證 pivot 已在最終位置；另一種雙向版本會在收尾交換 pivot，回傳定位點。兩者的遞迴範圍不同，不能只憑「Hoare」名稱照搬。後面的雙向定位程序會明定後者的規格。

同一批資料，兩套區間表示

先轉換索引與區間規格，再比較程式。對同一批資料 $[2, 3, 6, 6, 8]$，等值區在 1 起算寫成閉區間 $[3, 4]$，在 0 起算寫成半開區間 $[2, 4)$；兩者都包含兩筆資料。

非空閉區間 $[p, r]$ 對應半開區間 $[p-1, r)$，後者不包含右端點。因此閉區間 $[p, q-1]$ 對應半開區間 $[p-1, q-1)$；兩套規格中的空區間長度都為零。

雙向掃描後定位：遇到等值就停下來

以下使用閉區間 $p \le r$，首端 pivot。and 必須由左往右短路求值，先檢查邊界才讀陣列。

```
PARTITION-INWARD-PLACE(A,p,r):
    x = key(A[p]); i = p+1; j = r
    while i <= j:
        while i <= j and key(A[i]) < x: i += 1
        while i <= j and key(A[j]) > x: j -= 1
        if i >= j: break
        swap(A[i],A[j])
        i += 1; j -= 1
    swap(A[p],A[j])
    return j
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向掃描：不變量與完整交換追蹤

已分類左段 $A[p+1..i-1] \le x$，右段 $A[j+1..r] \ge x$。兩側掃描分別略過嚴格小、嚴格大的鍵；未交錯時交換停住的兩筆，再同時推進。等值鍵不能讓指標永久停住。未知區每輪縮小，最後 pivot 交換到 j；回傳兩側為 $[p, j-1]$、$[j+1, r]$。

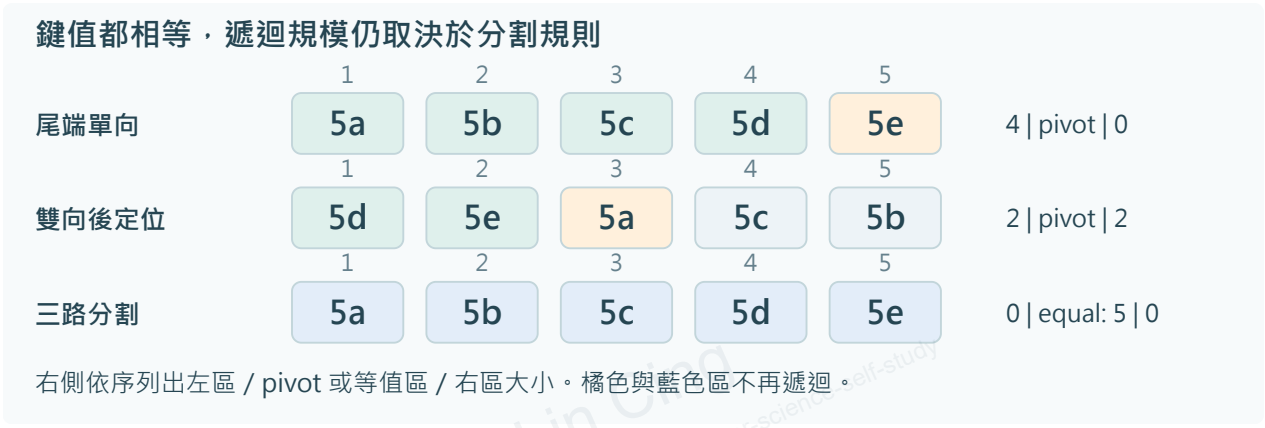
輸入 $[6, 10, 13, 5, 8, 3, 2, 11]$，$x = 6$。表中索引是兩側掃描停住、尚未交換的狀態。

停住的 (i, j)	交換位置	操作後的完整陣列
$(2, 7)$	$2, 7$	$[6, 2, 13, 5, 8, 3, 10, 11]$
$(3, 6)$	$3, 6$	$[6, 2, 3, 5, 8, 13, 10, 11]$
$(5, 4)$，已交錯	收尾 $1, 4$	$[5, 2, 3, 6, 8, 13, 10, 11]$

回傳 $q = 4$，左側皆 ≤ 6、右側皆 ≥ 6；兩側內部仍不必有序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同樣五個相等鍵，三種分割有三種結果



字母只辨識資料。雙向例依序交換 $2,5$、$3,4$、$1,3$。對全相等輸入族，雙向版每輪線性掃描、兩側約各半，總時間 $\Theta(n \lg n)$；尾端單向版為 $\Theta(n^2)$，三路為 $\Theta(n)$。這些是指定輸入族的成本；雙向版在其他輸入仍可能退化。

分類一次，可能做了兩次鍵比較

前面的精確式數的是尾端單向掃描中 `key(A[j]) <= x` 的執行次數，每個非 pivot 鍵恰測一次。三路版本若先測 `< x`，失敗才測 `> x`，一次分類可能做兩次布林比較；遇等值鍵時也會測兩次。

例某輪待分類的鍵為 $[1,4,3]$、pivot 值 $x=3$。只看各鍵與 pivot 的分類判斷，鍵 1 用一次測試，鍵 4、3 各用兩次，共五次，而非三次。原地三路版本可能改變處理次序，但每筆的分支測試規則仍要算清楚。

資料結構章節採 0 起算半開區間 $[l,r)$、等值區 $[lt,gt)$；本章三路版本採 1 起算閉區間 $[p,r]$、等值區 $[lt,gt]$。兩者都排除整個等值區，但程式邊界與精確比較常數須各自核對。

鍵互異、pivot 條件均勻時，三路版本的期望時間仍為 $\Theta(n \lg n)$：每輪至多常數倍的線性分類工作，遞迴規模相同。這個漸近結論不表示布林比較數也恰為 $2(n+1)H_n - 4n$。

四個秩：比較機率與整次執行的成本不同

四個相異鍵依大小命名為 z_1, z_2, z_3, z_4，輸入排列任意。

鍵的秩對	區間長度	直接比較機率
$(1,2),(2,3),(3,4)$	2	各為 1
$(1,3),(2,4)$	3	各為 $2/3$
$(1,4)$	4	$1/2$

所以 $\mathbb{E}[C] = 3 + 4/3 + 1/2 = 29/6$；代入 $H_4 = 25/12$ 也得到 $2 \cdot 5 \cdot 25/12 - 16 = 29/6$。

單次執行的比較數必為整數。首個 pivot 秩為 2 或 3 時共四次，機率 $1/2$；首選端點、接著三筆中選中位數時共五次，機率 $(1/2)(1/3) = 1/6$；其餘共六次，機率 $1/3$。因此 $4/2 + 5/6 + 6/3 = 29/6$，是對執行成本分布的第二種核對。

相鄰秩必比較，不代表所有鍵對都比較；平均 $29/6$ 也不代表某次真的做了分數次比較。

計數排序：從頻率算出每個鍵的區塊

前提：$K \geq 0$ 為整數，每個鍵都在 $0..K$；輸入、輸出位置為 $1..n$。計數表用鍵值當索引，必須能配置 $K+1$ 格，計數欄位也要能表示 $0..n$。

```
COUNTING-SORT(A,K):
    C[0..K] = 0
    for j = 1 to n: C[key(A[j])] += 1
    for v = 1 to K: C[v] += C[v-1]
    for j = n downto 1:
        v = key(A[j])
        B[C[v]] = A[j]
        C[v] -= 1
    return B
```

第一輪掃描後 $C[v]$ 是鍵 v 的頻率；累積後是鍵值 $\leq v$ 的資料數，亦即該鍵區塊的**末位置**。後面的回填會消耗計數；若需要保留區間查詢能力，先複製未消耗的前綴表 P。

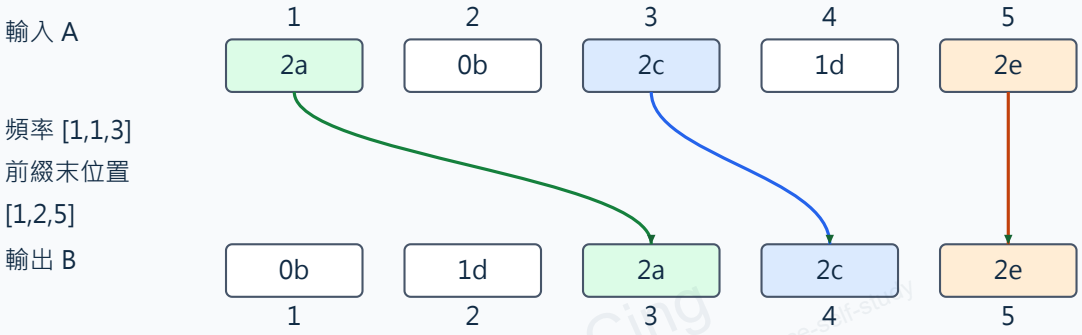
末位置回填：逐筆標出寫入位置

取 $A = [2_a, 0_b, 2_c, 1_d, 2_e]$、$K = 2$。鍵 $0, 1, 2$ 的頻率為 $[1, 1, 3]$；累積末位置為 $[1, 2, 5]$。三個輸出區塊依序是位置 1、位置 2、位置 $3..5$。

掃描位置 j	資料	寫入 B 的位置	寫入後的 C
5	2_e	5	$[1, 2, 4]$
4	1_d	2	$[1, 1, 4]$
3	2_c	4	$[1, 1, 3]$
2	0_b	1	$[0, 1, 3]$
1	2_a	3	$[0, 1, 2]$

同鍵資料的先後順序確實保留下來

計數排序：追蹤同鍵資料的寫入位置



反向處理 e→d→c→b→a；放入位置 5→2→4→1→3。

綠a、藍c、橘e是三筆鍵為 2 的資料；輸出仍保持 a→c→e。

輸出 $[0_b, 1_d, 2_a, 2_c, 2_e]$。鍵 2 的身分仍依 a, c, e 出現，這才是穩定性的證據；只看裸鍵 $[0, 1, 2, 2, 2]$ 無法判斷。

回填不變量、穩定性與成本

固定鍵 v，令 $L = P[v-1]$（$v = 0$ 時 $L = 0$）、$R = P[v]$。已處理的輸入後綴中，鍵 v 的資料恰占區塊右端 $[C[v]+1, R]$；$C[v]$ 指向尚未填入的最右位置。

下一筆鍵 v 尚未寫入，所以 $C[v] > L$，寫入合法且不覆蓋既有資料，再減一便保持不變量。終止時 $C[v] = L$，每個區塊恰填滿其頻率大小；區塊依鍵遞增，因此排序正確且資料無增減。

同鍵 x 在輸入先於 y 時，反向掃描先把 y 放到右邊，再把 x 放到左邊，所以穩定。若保留「末位置減一」卻改正向掃描，會反轉同鍵身分；下一節的「起始位置加一」才與正向掃描相配。

四個迴圈分別計初始化、頻率、前綴與輸出，總時間及含輸出的額外空間皆 $\Theta(n + K + 1)$。當 $K = O(n)$ 且 $n \geq 1$ 才是線性；即使只有少數資料，巨大鍵域仍需付表格成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

回填前保存前綴計數，才能回答區間查詢

對鍵值 $0 \leq a \leq b \leq K$，先算鍵值不大於 b 的筆數，再扣除小於 a 的筆數：

$$Q(a, b) = \begin{cases} P[b], & a = 0, \\ P[b] - P[a-1], & a > 0. \end{cases}$$

前面五筆資料的三種表格如下：

鍵 v	0	1	2
頻率 $F[v]$	1	1	3
保存的前綴 $P[v]$	1	2	5
回填後的計數器 $C[v]$	0	1	2

$Q(1, 2) = 5 - 1 = 4$。若誤用回填後的計數器，會得到錯誤的 $2 - 0 = 2$。另有 $Q(0, 0) = P[0] = 1$，實作不必讀取負索引。

必須在回填前複製 P；預處理時間為 $\Theta(n + K + 1)$，每次查詢為 $O(1)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同樣穩定的計數排序，也能從左往右填

末位置版本從右往左掃描，填入後把計數減一。另一個版本先算每個鍵的**起始位置** $S[v]$，從左往右掃描，填入後加一。

設 $F[v]$ 為鍵 v 的頻率、鍵域 $0..K$：

```
S[0] = 1
for v = 1 to K:
    S[v] = S[v-1] + F[v-1]
for j = 1 to n:
    v = key(A[j])
    B[S[v]] = A[j]
    S[v] = S[v] + 1
```

初始 $S[v] = 1 + \sum_{u<v} F[u]$，是該鍵區塊的第一格。空鍵區塊可以共用同一起始位置，因為不會有資料寫入它。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

起始位置的八筆完整追蹤

取 $A = [3_a, 1_b, 4_c, 2_d, 6_e, 1_f, 2_g, 2_h]$、$K = 6$。鍵 $0..6$ 的頻率為 $[0, 2, 3, 1, 1, 0, 1]$，初始 $S = [1, 1, 3, 6, 7, 8, 8]$。

j	資料	寫入位置	該鍵的下一位置
1	3_a	6	$S[3] = 7$
2	1_b	1	$S[1] = 2$
3	4_c	7	$S[4] = 8$
4	2_d	3	$S[2] = 4$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完成後四筆，核對輸出與最後指標

j	資料	寫入位置	該鍵的下一位置
5	6_e	8	$S[6] = 9$
6	1_f	2	$S[1] = 3$
7	2_g	4	$S[2] = 5$
8	2_h	5	$S[2] = 6$

輸出 $[1_b, 1_f, 2_d, 2_g, 2_h, 3_a, 4_c, 6_e]$。最後指標可等於區塊末端加一；它表示下一個空位，並不表示這一格還會被存取。

同鍵資料沿輸入順序填入同一區塊

1_b 先占位置 1，1_f 再占位置 2；$2_d, 2_g, 2_h$ 依序占位置 3、4、5。顏色連結同鍵區塊，身分標籤用來核對穩定性。

已填前綴與下一空位的不變量

令 $S_0[v]$ 是未消耗的起始位置，$c_j(v)$ 是已掃描的前 $j-1$ 筆中、鍵為 v 的資料數。每輪開始時，

$$S[v] = S_0[v] + c_j(v).$$

區塊 $[S_0[v], S[v]-1]$ 恰好裝著已掃描的鍵 v 資料，而且順序與輸入相同；這些區塊互不重疊。

初始化 $c_1(v)=0$，所有區塊為空。讀到鍵 v 時，尚有這筆未放，所以 $c_j(v) < F[v]$，寫入位置仍在 v 的合法區塊內。只寫下一空位再令 $S[v]$ 加一，保持資料與次序；其他鍵的區塊不變。

終止時 $c_{n+1}(v) = F[v]$，每個區塊恰填滿，區塊依鍵排序，故正確且穩定。掃描、計數及起始位置各自都是完整迴圈，時間 $\Theta(n+K+1)$、含輸出的額外空間 $\Theta(n+K+1)$。

掃描方向與位置定義要成對使用： 正向配起始位置加一；反向配末位置減一。只改掃描方向，可能仍排對裸鍵，卻反轉同鍵資料。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LSD 基數排序：穩定性把低位順序保留下來

設每個鍵是固定 $d \geq 1$ 位、基底 $r \geq 2$ 的非負整數；不足位數補前導零。LSD（最低有效位優先）依序處理第 $1, 2, \ldots, d$ 位，每輪對目前位做**穩定**排序。

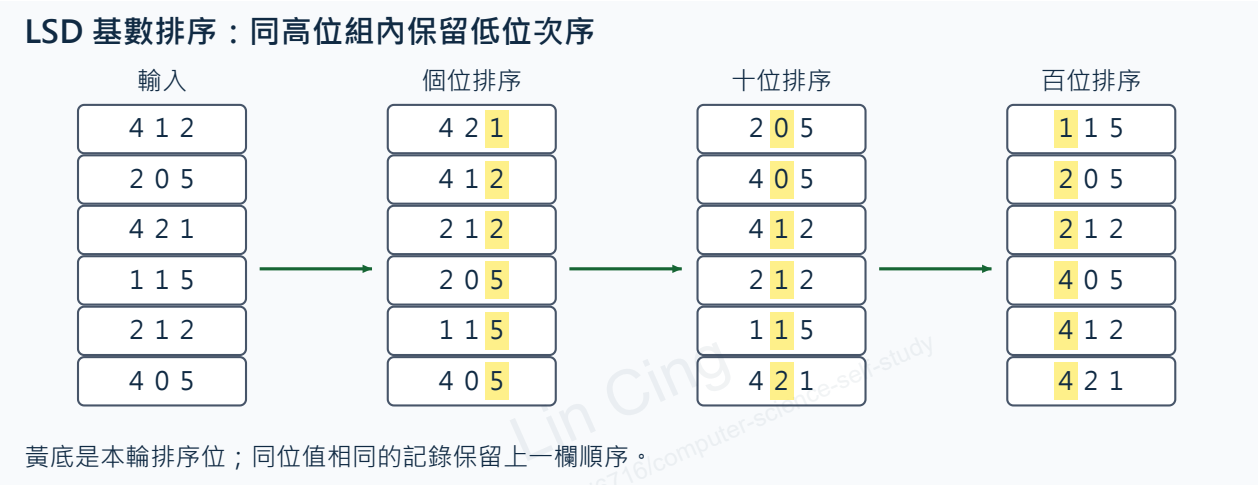
不變量：完成第 t 輪後，序列依最低 t 位代表的數值排序。$t=0$ 無限制。第 t 輪把資料按目前位分組，不同組由該位決定大小；同組內靠穩定性保留前 $t-1$ 位的次序。因此最低 t 位完整有序，$t=d$ 時整體有序。

保留低位順序只發生在**相同高位組內**，不是全域仍依低位遞增。反例：$[12, 11]$ 先按個位得到 $[11, 12]$；十位皆為 1，若不穩定地反轉同鍵，便錯誤地回到 $[12, 11]$。

直接對整個序列按百位、十位、個位依次排序，也會把低位變成最後的主序。MSD 必須在同一高位群內遞迴處理，不能只把 LSD 的迴圈倒過來。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三輪基數排序，每輪都列出完整序列



百位輪中，205 排在 212 前面，是因為同一百位組內保留了低兩位 $05 < 12$ 的次序。每輪都只比較目前位，完整鍵序由不變量逐步建立。

逐輪核對基數排序的完整結果

已處理的位	完整序列
尚未處理	$[412, 205, 421, 115, 212, 405]$
個位	$[421, 412, 212, 205, 115, 405]$
個位與十位	$[205, 405, 412, 212, 115, 421]$
全部三位	$[115, 205, 212, 405, 412, 421]$

十位輪中，$205, 405$ 的十位皆為 0，保留前輪的相對次序。$412, 212, 115$ 的十位皆為 1，也保留此前次序；它們的最低兩位依序為 $12, 12, 15$。

位元數、每輪基底與成本一起算

每輪用穩定計數排序，時間 $\Theta(n+r)$；d 輪共 $\Theta(d(n+r))$，可重用的額外空間 $\Theta(n+r)$。只有 d 固定且 $r=O(n)$ 才直接得到線性時間。

對 $b \geq 1$ 位元鍵，每輪取 s 位，$1 \leq s \leq b$，則

$$r = 2^s, \qquad d = \lceil b/s \rceil, \qquad T = \Theta(\lceil b/s \rceil (n + 2^s)) .$$

最高一組不足 s 位時可補零。假設一個鍵與一個數位可在常數時間讀取；超過機器字長的大整數，必須另外計取位與搬移成本。

$n \geq 2$ 時取 $s = \min(b, \lfloor \lg n \rfloor)$，便有 $2^s \leq n$，得到 $O(n\lceil b/s \rceil)$；$b = O(\lg n)$ 時是 $O(n)$。

在「每輪成本為 $n+r$」的模型下，32 位元鍵取 $s=8$ 需 $4(n+256)$，取 $s=16$ 需 $2(n+65536)$。後者較小當且僅當 $n > 65024$；等號時相同。這是指定模型的比較，實機還有配置、快取與頻寬成本。

分桶是共同技巧，並不代表同一個演算法

方法	分配依據	分配後還做什麼
LSD radix sort	由低位到高位，每次一位	每輪對整個序列穩定排序
MSD radix sort	由最高位開始	只在同一高位群內遞迴處理下一位
區間 bucket sort	$x \in [0,1)$ 映到 $\lfloor nx \rfloor$	每桶完成內部排序，再按桶串接

三者都可以把資料放進 buckets，但**位數、遞迴規則與成本前提不同**。固定十個數位桶，不等於依 n 分成 n 個等長區間；把每桶直接排序，也不等於繼續按下一位遞迴。

若把 $[312, 305, 121, 118, 214, 207]$ 按百位分組，得到 $[121, 118]$、$[214, 207]$、$[312, 305]$。這一步尚未完成排序：MSD 要在各組處理低位；區間桶方法則須用指定的桶內排序，把每組排好。只串接原分組會得到錯誤次序。

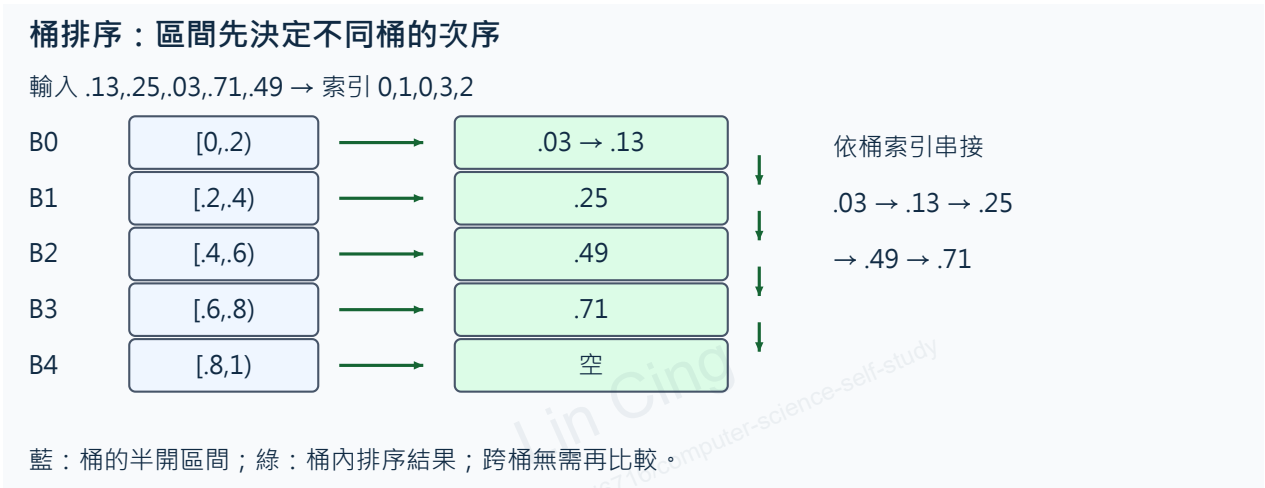
區間桶排序：先分配，再把每桶排好

輸入 $n \geq 1$ 個 $x \in [0, 1)$；建立 n 個桶 $B[0..n-1]$，將 x 依序附加至 $B[\lfloor nx \rfloor]$。每桶用插入排序，最後依桶編號串接。空輸入直接返回；$x = 1$ 不在定義域，不能拿來存取 $B[n]$。

正確性不需要隨機分布：若 $x \leq y$，則 $\lfloor nx \rfloor \leq \lfloor ny \rfloor$。不同桶已依不重疊的半開區間排序，同桶則由內部排序完成。依輸入順序附加，再用穩定插入排序，才能另保證穩定性；頭插可能先改變同鍵順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

五個桶：區間、桶內次序與完整輸出



例 $n = 5$，輸入 $[.13, .25, .03, .71, .49]$，桶編號依序 $[0, 1, 0, 3, 2]$。第 0 桶排成 $[.03, .13]$，第 $1, 2, 3$ 桶分別為 $.25, .49, .71$，第 4 桶為空；輸出 $[.03, .13, .25, .49, .71]$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

期望線性：算平方桶大小，而非平方期望

令 N_i 是桶 i 的大小。建立、分配與串接共 $\Theta(n)$，桶內插入排序總成本至多常數倍的 $\sum_i N_i^2$。因此

$$T \le c_0 n + c_1 \sum_{i=0}^{n-1} N_i^2.$$

現在才加入輸入假設：各筆鍵**互相獨立且均勻**落在 $[0,1)$。令 X_{ij} 表示第 j 筆落入桶 i，則 $N_i = \sum_j X_{ij}$、$X_{ij}^2 = X_{ij}$，且

$$\begin{aligned}\mathbb{E}[N_i^2] &= \sum_j \mathbb{E}[X_{ij}] + \sum_{j \ne k} \mathbb{E}[X_{ij} X_{ik}] \\ &= n\frac{1}{n} + n(n-1)\frac{1}{n^2} = 2 - \frac{1}{n}.\end{aligned}$$

所以 $\mathbb{E}[\sum_i N_i^2] = 2n - 1$，給 $\mathbb{E}[T] = O(n)$；讀取輸入本身給 $\Omega(n)$，合為 $\Theta(n)$。

獨立性用在不同資料 j, k 的落桶事件。不同桶的大小 N_i 彼此相關，因為總和固定為 n；這不妨礙期望線性性。僅知 $\mathbb{E}[N_i] = 1$，不能把 $\mathbb{E}[N_i^2]$ 寫成 1。

分布集中、資料相關與真正的最壞輸入

若各筆仍獨立，但落入桶 i 的機率改為 p_i，$\sum_i p_i = 1$，同樣展開得到

$$\mathbb{E}\left[\sum_i N_i^2\right] = n + n(n-1)\sum_i p_i^2.$$

當 $\sum_i p_i^2 = O(1/n)$ 時仍有線性期望上界。均勻分布給 $1/n$；若所有機率集中單桶，平方桶大小總和就是 n^2。

邊際均勻也不夠：先均勻抽 $J \in \{0, \dots, n-1\}$，令每筆 $x_j = (J + 1/2)/n$。各筆對每個桶的邊際機率都為 $1/n$，卻總是全體同桶，$\sum_i N_i^2 = n^2$。這反駁平方成本的錯誤估計；相等鍵的插入排序本身仍可能只需線性時間。

真正達到平方時間的輸入族可取 $n \ge 2$、$x_j = (n - j + 1)/(2n^2)$。它們互異、嚴格遞減，而且全在第 0 桶；插入排序須搬移 $n(n-1)/2$ 次，故最壞時間 $\Theta(n^2)$。

若每桶改用最壞 $O(m \lg(m+1))$ 的排序，總最壞可降為 $O(n \lg(n+1))$；獨立均勻模型仍保留線性期望，因為 $m \lg(m+1) = O(m^2)$。

次序統計量與極值：先說清楚要找哪一筆

輸入為 $n \geq 1$ 筆可全序比較的鍵；第 k 小是排序後多重集合的第 k 個值，$1 \leq k \leq n$。重複值各占一個位置，答案卻可相同。例如 $[2, 5, 2, 8]$ 的第 $1, 2$ 小都是 2。

本章的下中位數（lower median）取 $k = \lceil n/2 \rceil$。偶數筆時，它是兩個中央值中較小的值，與統計上取兩者平均的定義不同。空輸入與非法秩在入口拒絕。

找最小值可從第一筆開始，依序比較其餘資料；不變量是「目前最小值等於已讀前綴的最小值」，共做 $n - 1$ 次鍵比較。最大值同理。

這也是相異鍵輸入的比較下界：每個非最小值都須至少在一次比較中被證明大於另一個鍵，才能排除最小值資格；一次比較至多排除一個尚未被排除的候選。因此至少 $n - 1$ 次。用相異鍵證明最壞下界，並不要求演算法拒絕重複鍵。

成對找極值：八筆資料只做十次比較

同時找最小與最大時，先比較一對資料：較小者只和目前最小值比較，較大者只和目前最大值比較。每對共三次，比兩筆分別挑戰兩個極值少一次。

取 $[14, 3, 9, 1, 8, 5, 12, 6]$。偶數筆先比較第一對以初始化；後面各對維持「兩個極值涵蓋已處理前綴」的不變量。

本輪資料	組內較小、較大	更新後最小值	更新後最大值	累積比較次數
$14, 3$	$3, 14$	3	14	1
$9, 1$	$1, 9$	1	14	4
$8, 5$	$5, 8$	1	14	7
$12, 6$	$6, 12$	1	14	10

偶數 n 用 $1 + 3(n - 2)/2 = 3n/2 - 2$ 次；奇數 n 先以第一筆同時初始化兩個極值，剩下各對共 $3(n - 1)/2$ 次。統一為 $\lceil 3n/2 \rceil - 2$；$n = 1$ 時為零。平手時仍可任選一方作極值，數值答案正確。

為什麼成對法的最壞比較數已經最少

對相異鍵，每筆最初同時具有最小值、最大值兩種候選資格。最後只保留兩種資格各一個，故總共須排除 $2n-2$ 個資格。

對手（adversary）可依下表回答比較。稱未曾比較者為 U，只剩最大值資格者為 H，只剩最小值資格者為 L，兩者都失去者為 M。

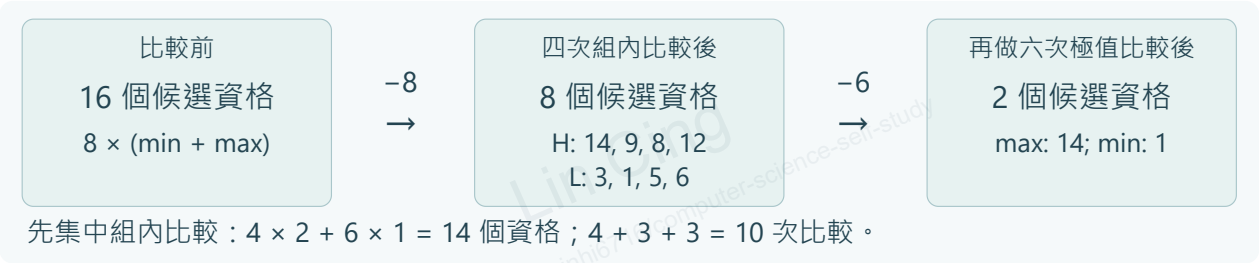
比較對象	對手回答與新排除數
U, U	一大一小，各失一種資格；排除 2 個
H, H 或 L, L	消去一個最大或最小候選；排除 1 個
H, L	回答 $H > L$；排除 0 個
U 與已分類者	若對方是 H，令 $U < H$；是 L，令 $U > L$；是 M，任選一致方向。只排除 U 的一種資格
涉及 M、不涉及 U	保持 $H > M > L$；M, M 選既有偏序的一個線性延伸，至多排除 1 個

把候選資格的排除次數算清楚

這些回答維持一致全序：H 從未被證明小於其他鍵，L 從未被證明大於其他鍵，U 沒有比較約束。按 $H > M > L$ 分層，層內採既有比較偏序的線性延伸，不會產生循環。

只有 U, U 比較能一次排除兩個資格，且每次耗掉兩筆未比較資料，至多 $\lfloor n/2 \rfloor$ 次。其他比較至多排除一個資格。若總比較數為 c，則

$$2n - 2 \le c + \lfloor n/2 \rfloor, \qquad c \ge \lceil 3n/2 \rceil - 2.$$



先集中組內比較：4 × 2 + 6 × 1 = 14 個資格；4 + 3 + 3 = 10 次比較。

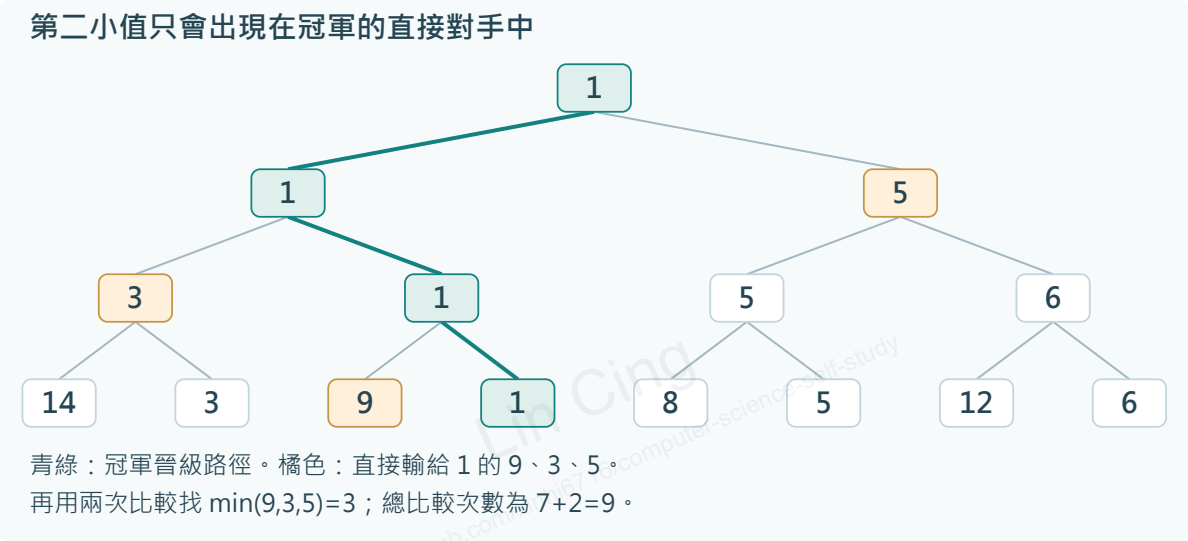
淘汰賽保存了第二小值的候選名單

讓較小鍵晉級，並保存每位勝者直接擊敗的對手。第二小值必須直接輸給最小值：若它輸給另一個非最小鍵，已有兩個鍵比它小，產生矛盾。以下先假設鍵相異。

一般 $n \geq 2$ 可用平衡賽程，不足的席位輪空，無須與虛構鍵比較。每次淘汰一人，共 $n-1$ 次；冠軍實際對手數 $d \leq \lceil \lg n \rceil$，清單再比 $d-1$ 次，最壞至多 $n + \lceil \lg n \rceil - 2$。若允許重複鍵，可按 $(key, identity)$ 打破平手，此時求的是第二筆資料，未必是第二個不同值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

八鍵淘汰賽：冠軍直接擊敗了誰



冠軍 1 直接擊敗 $9, 3, 5$，故第二小是這三者的最小值 3。淘汰賽共 7 次，候選清單再比 2 次，共 9 次。

隨機選擇的三路分割：把等值區一次排除

排序會遞迴處理兩側；選擇只保留含答案的一側。本節陣列索引是 **1 起算閉區間** $A[p..r]$；目標 k 是此子陣列內的局部秩，$1 \le k \le r-p+1$。分割比較鍵值、交換整筆資料，x 為子陣列中實際出現的鍵。

```
PARTITION3(A,p,r,x):
    lt = p; i = p; gt = r
    while i <= gt:
        if key(A[i]) < x:
            swap(A[lt],A[i]); lt += 1; i += 1
        elif key(A[i]) > x:
            swap(A[i],A[gt]); gt -= 1
        else:
            i += 1
    return (lt,gt)
```


三路分割：四區不變量與每輪進度

迴圈開始時，$A[p..lt-1] < x$、$A[lt..i-1] = x$、$A[i..gt]$ 未知、$A[gt+1..r] > x$；空區間合法。小鍵分支擴大小區與等值區；等鍵分支擴大等值區。大鍵分支從右端換來的鍵尚未分類，所以不能推進 i。每輪未知區長度減一，交換保留多重集合；終止時未知區消失，得到完整三區。

< x　= x　尚未分類　> x　pivot x = 3; A[4..10]

	4	5	6	7	8	9	10
lt = 4, i = 6, gt = 9	3	3	5	3	9	1	8
lt = 4, i = 6, gt = 8	3	3	1	3	9	5	8

交換 A[6] 的 5 與 A[9] 的 1：gt 減成 8，i 仍為 6。
下一輪必須分類新換來的 A[6] = 1；p = lt = 4，故小於 x 的區間目前為空。

隨機選擇：只有往右才要換算局部秩

```
RANDOM-SELECT(A,p,r,k):
    require 1 <= k <= r-p+1
    while p < r:
        x = key(A[uniform-integer(p,r)])
        (lt,gt) = PARTITION3(A,p,r,x)
        L = lt-p; E = gt-lt+1
        if k <= L: r = lt-1
        elif k <= L+E: return x
        else: k -= L+E; p = gt+1
    return key(A[p])
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

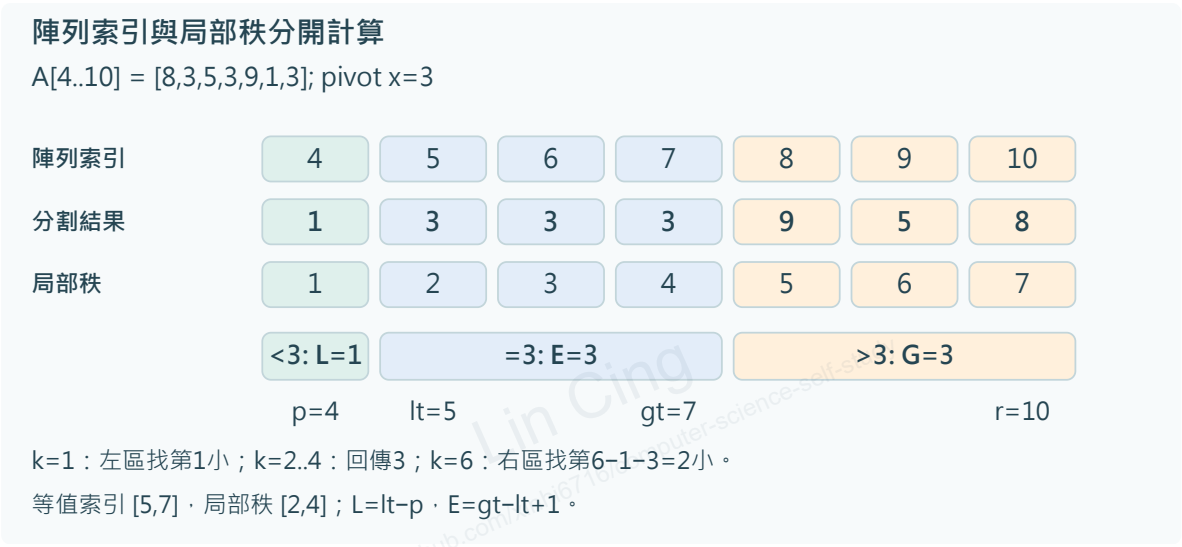
選擇迴圈：答案不變，規模持續縮小

x 取自目前資料，因此 $E \geq 1$。答案落在局部秩 $L+1..L+E$ 時直接回傳 x；往左保留第 k 小，往右則扣除已確定較小的 L 筆與等值的 E 筆。

迴圈不變量：目前子陣列的第 k 小，等於原問題要求的鍵值。左分支滿足 $1 \leq k \leq L$；右分支滿足 $1 \leq k-L-E \leq r-gt$，故不會保留空區間。每次未返回時，規模至少減少 E；最後只剩一筆，回傳該鍵。這同時證明正確性與終止，允許重複鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

局部秩、區長與絕對索引要分開



$A[4..10] = [8,3,5,3,9,1,3]$，取 $x = 3$，分割後 $lt = 5, gt = 7$，所以 $L = 1, E = 3, G = 3$。局部等值秩為 $2..4$。

用區長逐一換算目標局部秩

原局部秩	判斷	下一狀態或答案
1	$1 \le L$	$A[4..4]$ 的第 1 小
4	$L < 4 \le L + E$	回傳 3
6	$6 > L + E$	$A[8..10]$ 的第 $6 - 1 - 3 = 2$ 小

不能直接比較 k 與 lt：前者由目前子陣列的第一筆算起，後者從整個陣列的索引起點算起。若改用絕對目標位置 $t = p + k - 1$，則每輪保留 t，不再同時扣局部秩。

隨機選擇：對所有後續狀態做強歸納

條件於目前的陣列，pivot 均勻選自其 n 筆資料；未來每輪亦在當時的子陣列內重新均勻選取。先假設鍵相異，pivot 的局部秩 R 均勻分布於 $1..n$。無論目標秩為何，後續規模至多

$$m_R = \max(R-1, n-R).$$

令 $F(A,k)$ 為此狀態的剩餘期望成本。本層掃描與固定開銷至多 cn。對**所有較短狀態**假設 $F(B,j) \le a|B|$，條件於本次 pivot 套用歸納，得

$$F(A,k) \le cn + \frac{a}{n}\sum_{r=1}^{n} \max(r-1, n-r).$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把所有較大側長度加總，再完成歸納

偶數 $n = 2h$ 時，較大側長度 $h, h+1, \dots, 2h-1$ 各出現兩次，故和為 $2\sum_{t=h}^{2h-1} t = 3h^2 - h$。奇數 $n = 2h+1$ 時，中央秩給 h，其餘對稱出現兩次，和為 $h + 2\sum_{t=h+1}^{2h} t = 3h^2 + 2h$。除以 n 後均至多 $3n/4$，因此

$$F(A,k) \le (c + 3a/4)n \le an \qquad (a \ge 4c).$$

增大 a 以涵蓋 $n = 1$ 基例，即得 $O(n)$。首次分割須掃描 $n > 1$ 筆，故期望為 $\Theta(n)$。

證明沒有假設未知的成本函數單調，也沒有把相依隨機變數的期望拆乘。$3n/4$ 是較大側長度的期望上界，不能當成每次分割的確定上限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

重複鍵不破壞三路選擇的期望線性界

為分析替同鍵資料附上固定的身分次序，得到唯一秩 $R \in \{1, \dots, n\}$；實際分割仍只比較鍵，不比較身分。均勻選一筆資料，使分析用的 R 均勻。

若答案在 $< x$ 區，其長度至多 $R - 1$；若在 $> x$ 區，其長度至多 $n - R$；若在等值區便終止。故每種 pivot 身分都有同一上界 $m_R = \max(R - 1, n - R)$，前述對所有較短狀態的強歸納照用，得到期望 $\Theta(n)$。

最壞執行仍可為 $\Theta(n^2)$：相異鍵、找最小值，每輪恰抽到當前最大值，掃描規模為 $n, n-1, \dots, 2$。這條路徑有正機率，期望界不等於最壞界。迭代版加原地三路分割只需 $O(1)$ 額外空間，且會改動輸入。

全等資料則一次掃描就返回。若改用把所有等值都送到同一側的二路 Lomuto，找最小秩時會退化，隨機 pivot 也無法補救。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

確定性選擇：讓完整程序與追蹤使用同一規格

以下列表與排序結果皆為 **1 起算**。SORT-COPY 依鍵排序副本；STABLE-SPLIT-COPY 線性掃描輸入，依序加入小於、等於、大於 x 的三個新列表，保持各區的輸入次序。這是便於自學追蹤的 $O(n)$ 額外空間版本；不把它宣稱為原地程序。

每組至多五筆，取下中位數；組數 $g = \lceil n/5 \rceil$，再對組中位數遞迴找第 $\lceil g/2 \rceil$ 小，作為 pivot。

```
MEDIAN-PIVOT(A):
    M = empty list
    for each consecutive group G of at most 5 records in A:
        B = SORT-COPY(G)
        append key(B[ceil(length(B)/2)]) to M
    return DETERMINISTIC-SELECT(M,ceil(length(M)/2))
```

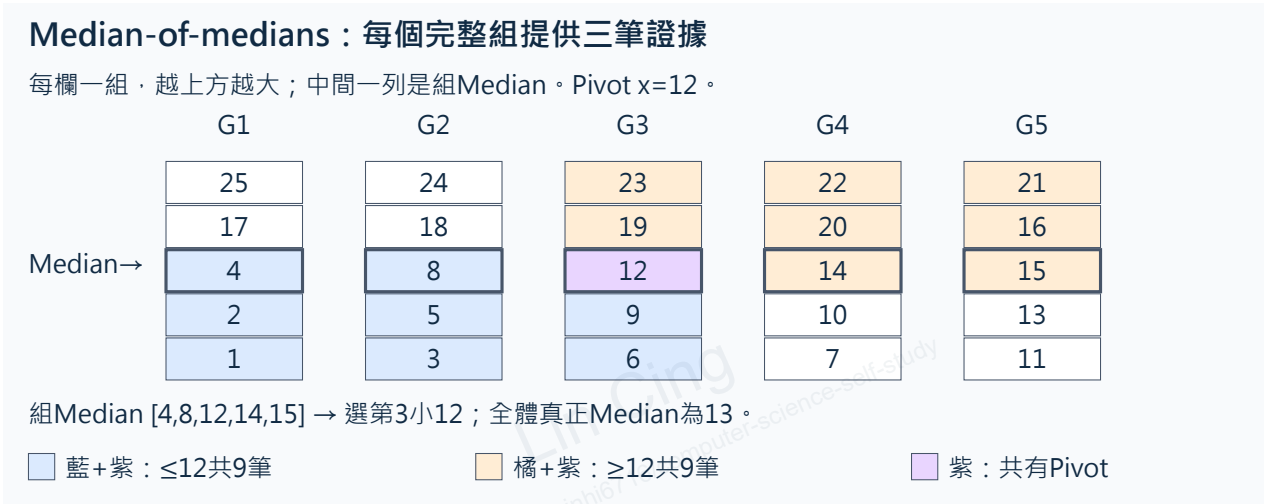
Lin Cing https://github.com/xinhi6716/computer-science-self-study

確定性選擇：回傳等值或遞迴單側

```
DETERMINISTIC-SELECT(A,k):
    n = length(A); require 1 <= k <= n
    if n <= 5: return key(SORT-COPY(A)[k])
    x = MEDIAN-PIVOT(A)
    (L,E,G) = STABLE-SPLIT-COPY(A,x)
    if k <= length(L):
        return DETERMINISTIC-SELECT(L,k)
    elif k <= length(L)+length(E): return x
    else:
        return DETERMINISTIC-SELECT(G,k-length(L)-length(E))
```

正確性由分割的三區次序與規模歸納得到。組中位數的遞迴規模 $g < n$；答案側至多 $n-1$，因 x 必出現，等值區非空。兩種遞迴都會停止，時間分析也必須同時計入兩者。

五元素組提供的是排除證據，不是精確中位數



這 25 筆資料恰為 $1..25$。組中位數為 $[4, 8, 12, 14, 15]$，所以 $x = 12$；全體下中位數卻是第 13 小，也就是 13。

兩側證據，都列出實際鍵值

中位數 ≤ 12 的前三組，各取其最小三筆，可列出 $[1,2,4]$、$[3,5,8]$、$[6,9,12]$，共九筆 ≤ 12。中位數 ≥ 12 的後三組，各取最大三筆，可列出 $[12,19,23]$、$[14,20,22]$、$[15,16,21]$，共九筆 ≥ 12。

兩份證據在 12 重疊沒有問題：若遞迴找嚴格大側，排除的是全部 $\le x$；若找嚴格小側，排除的是全部 $\ge x$。分析的兩個分支不會同時採用。含重複鍵時仍保證非嚴格的大小關係，三路分割正好把等值區一併排除。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

取整與殘組：完成最壞線性時間的歸納

$g = \lceil n/5 \rceil$ 個組中位數的下中位數 x，兩側各至少有 $\lceil g/2 \rceil$ 個中位數滿足 $\le x$ 或 $\ge x$。保守扣掉可能不足五筆的殘組，以及含所選 pivot 身分的那一組，剩至少 $\lceil g/2 \rceil - 2$ 個完整組。

每組提供三筆證據，所以兩側各至少有

$$3(\lceil g/2 \rceil - 2) \ge 3n/10 - 6$$

筆非嚴格大小證據，嚴格遞迴側至多 $7n/10 + 6$。小規模下這個下界可能為負，只代表界太鬆；實際等值區非空，子問題仍嚴格縮小。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

歸納同時付清兩個遞迴的成本

非遞迴工作至多 cn。對所有較短狀態假設成本至多常數 a 乘其長度；$n \ge 140$ 時，

$$\begin{aligned} T(n) &\le a\lceil n/5\rceil + a(7n/10 + 6) + cn \\ &\le (9a/10 + c)n + 7a \le an. \end{aligned}$$

最後一步取 $a \ge 20c$，使 $a/10 - c \ge a/20$，且 $n \ge 140$ 給 $an/20 \ge 7a$。有限的 $1 \le n < 140$ 可增大 a 涵蓋；這是證明門檻，不要求改動程式的五筆基例。初次分組掃描給 $\Omega(n)$，故最壞時間為 $\Theta(n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第十六小：逐次列出分組、區長與下一個秩

輸入按以下五組串接，分組前不先排序全體：

$[1, 2, 4, 17, 25]$、$[3, 5, 8, 18, 24]$、$[6, 9, 12, 19, 23]$、$[7, 10, 14, 20, 22]$、$[11, 13, 15, 16, 21]$。

目前 (n, k)	組中位數 M 與 pivot x	$(\lvert L\rvert, \lvert E\rvert, \lvert G\rvert)$	下一步
$(25, 16)$	$[4, 8, 12, 14, 15] \to 12$	$(11, 1, 13)$	大側第 4 小
$(13, 4)$	$[19, 20, 16] \to 19$	$(6, 1, 6)$	小側第 4 小
$(6, 4)$	$[15, 16] \to 15$	$(2, 1, 3)$	大側第 1 小
$(3, 1)$	基例排序 $[16, 17, 18]$	不再分割	回傳 16

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

由保留的列表，重建下一次分組

第一次大側保留掃描次序，為 $[17, 25, 18, 24, 19, 23, 14, 20, 22, 13, 15, 16, 21]$。分成五、五、三筆，排序副本後分別是 $[17, 18, 19, 24, 25]$、$[13, 14, 20, 22, 23]$、$[15, 16, 21]$，中位數為 $19, 20, 16$；在這三筆上用基例回傳中位數 19。

接下來小側為 $[17, 18, 14, 13, 15, 16]$；前五筆的中位數為 15，單筆殘組為 16，兩筆取下中位數 15。大側為 $[17, 18, 16]$，第 1 小回傳 16。這條路徑符合前頁的穩定列表程序；改成原地交換分割，區內次序及後續分組可能不同，不能要求同一條中間路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

改變組大小，時間與空間保證都要重算

固定奇數組大小 $s = 2t + 1$，每個完整組的中位數可提供 $t + 1$ 筆非嚴格證據。忽略取整與常數項時，兩個遞迴規模比例和為

$$\frac{1}{s} + 1 - \frac{t + 1}{2s} = 1 + \frac{1 - t}{2s}.$$

五筆一組時為 $9/10 < 1$，七筆一組時為 $6/7 < 1$，留有線性工作所需的餘裕；有限殘組與歸納基例仍須另算。三筆一組時比例和為 1，粗界為

$$T(n) \le T(\lceil n/3 \rceil) + T(2n/3 + O(1)) + O(n).$$

此式可證 $O(n \lg n)$，卻不能只憑一個上界便聲稱實際演算法必有 $\Omega(n \lg n)$。下界還須證明相應的壞分割能在各層持續發生。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

副本與尚未返回的呼叫鏈，都要計入空間

本章穩定列表程序的額外空間為 $O(n)$：每次呼叫保存 $O(n)$ 筆副本，兩個遞迴先後執行；對充分大規模，每條尚未返回的呼叫鏈至多按固定小於一的比例縮小，幾何和為 $O(n)$，小規模部分為常數。最壞呼叫深度為 $O(\lg n)$。把分割換成原地版本還不等於整體 $O(1)$ 空間，因為組中位數、副本與遞迴堆疊都必須處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

生日碰撞：用補事件，逐人算不碰撞的機率

模型：$k \geq 0$ 個人各自獨立、均勻選擇 $n \geq 1$ 種日期。這是簡化的機率模型，不直接假設真實生日符合它。

當前 j 人日期都不同，下一人有 $n - j$ 種日期不會碰撞，所以條件機率為 $(n - j)/n$。由條件機率連乘，$k \leq n$ 時

$$\Pr(\text{日期皆不同}) = \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right),$$

$$\Pr(\text{至少一對同日}) = 1 - \prod_{j=0}^{k-1} \left(1 - \frac{j}{n}\right).$$

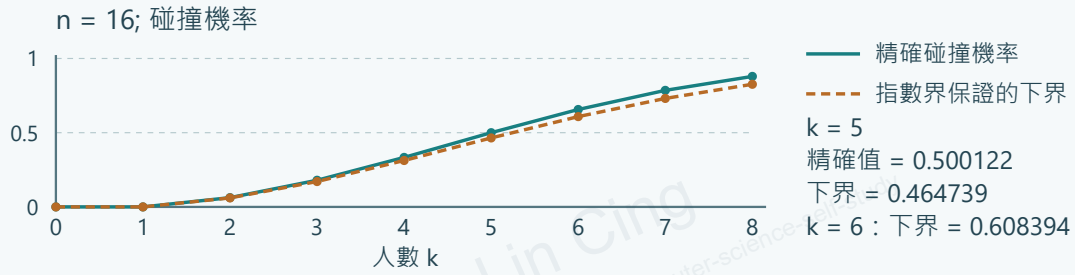
$k = 0, 1$ 無碰撞；空乘積取 1。$k > n$ 時由鴿籠原理必碰撞，直接取機率 1，不要把連乘繼續延伸成負因子。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

充分條件可能比精確門檻保守

對 $2 \le k \le n$，逐項使用 $1 - x \le e^{-x}$，再取補事件：

$$\Pr(\text{碰撞}) \ge 1 - \exp\left(-\sum_{j=0}^{k-1} \frac{j}{n}\right) = 1 - e^{-k(k-1)/(2n)}.$$



下界達到 $1/2$ 的條件是 $k(k-1) \ge 2n \ln 2$；這是**充分條件**，並非精確門檻。$n = 16$ 時，精確機率在 $k = 5$ 已超過 $1/2$，這個下界要到 $k = 6$ 才能保證。

碰撞對數的期望，不等於發生碰撞的機率

令 I_{ij} 表示第 i, j 人同日。兩人獨立均勻，故

$$\Pr(I_{ij} = 1) = \sum_{d=1}^{n} \Pr(B_i = d, B_j = d) = n\frac{1}{n^2} = \frac{1}{n}.$$

碰撞對數 $X = \sum_{i<j} I_{ij}$，所以

$$\mathbb{E}[X] = \frac{\binom{k}{2}}{n}.$$

「有碰撞」是事件 $X \ge 1$。Markov 界只給 $\Pr(X \ge 1) \le \mathbb{E}[X]$，不能反向推論：期望很大，不代表非零機率很大。例如 X 以 $1/M$ 機率等於 M^2，否則為零，則期望為 M，非零機率卻為 $1/M$。

在生日的獨立均勻模型中，$k = o(\sqrt{n})$ 時由上界得碰撞率趨零；$k \ge C\sqrt{n}$、固定 C 夠大時，由補事件指數界得到固定正的碰撞率。兩個方向合起來，才支持 $\sqrt{n}$ 的機率轉折尺度。

十六種日期：把每一個條件機率乘出來

取 $n = 16, k = 5$，逐人累乘不碰撞的機率：

加入第幾人	本次不碰撞的條件機率	到此仍皆不同的機率
1	1	1
2	$15/16$	$15/16$
3	$14/16$	$105/128$
4	$13/16$	$1365/2048$
5	$12/16$	$4095/8192$

碰撞機率為 $4097/8192 \approx 0.500122$，碰撞對數的期望則為 $\binom{5}{2}/16 = 5/8$。$k = 5$ 的指數補事件上界為 $e^{-10/16} > 1/2$，尚不能證明碰撞超過半數；$k = 6$ 時才有 $e^{-15/16} < 1/2$。充分條件較保守，與精確答案沒有矛盾。

一組樣本的碰撞，與整個模型的平均不同

同一日期樣本與整個機率模型：樣本計數和期望要分開

模型：16日，5人。P(有碰撞)=4097/8192≈0.500122。

E[碰撞對數]=10/16=5/8；這個期望不是上面的事件機率。

圖中的日期為 $[2, 7, 10, 7, 4]$，本次只有第 $2, 4$ 人同日，因此 $X = 1$、$I_{\{X \geq 1\}} = 1$。它們是這一個結果的計數，與對全部結果平均的 $5/8$、$4097/8192$ 不同。

兩兩獨立足以算單對，卻不足以連乘全體

令 U, V 獨立均勻於 $\{0, 1, 2\}$，三人的日期為

$$B_1 = U, \qquad B_2 = V, \qquad B_3 = (2U - V) \bmod 3.$$

任取兩個日期，皆能唯一還原 (U, V)：B_1, B_2 直接還原；B_1, B_3 給 $V = 2B_1 - B_3$；B_2, B_3 給 $U = 2(B_2 + B_3)$，均在模 3 下計算。因此每對日期均勻於九種有序組合，兩兩獨立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

九種等可能結果，逐一算出碰撞對數

U	V	(B_1, B_2, B_3)	碰撞對數 X
0	0	$(0, 0, 0)$	3
0	1	$(0, 1, 2)$	0
0	2	$(0, 2, 1)$	0
1	0	$(1, 0, 2)$	0
1	1	$(1, 1, 1)$	3
1	2	$(1, 2, 0)$	0
2	0	$(2, 0, 1)$	0
2	1	$(2, 1, 0)$	0
2	2	$(2, 2, 2)$	3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

期望相同，碰撞機率仍可能不同

模型	$\Pr(X=0)$	$\Pr(X=1)$	$\Pr(X=3)$	$\Pr(X>0)$	$\mathbb{E}[X]$
兩兩獨立的構造	$2/3$	0	$1/3$	$1/3$	1
三個日期完全獨立	$2/9$	$2/3$	$1/9$	$7/9$	1

完全獨立時，27 種等可能三元組中，全異有 6 種、全同有 3 種，恰有一對相同則有 $3 \cdot 3 \cdot 2 = 18$ 種：先選哪一對、再選共同日期，最後選另一個不同日期。恰有兩對相同不可能，因為相等具有遞移性。

此模型碰撞率為 $3/9 = 1/3$；三個完全獨立日期的碰撞率則為 $1 - 6/27 = 7/9$。兩者的碰撞對數期望都為 1。相同期望不代表相同分布；精確補事件連乘需要更強的聯合假設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

球入桶：先固定球數，再問負載與空桶數

$t \geq 0$ 顆球各自獨立、均勻投入 $b \geq 1$ 個桶。固定桶的負載 N 是 t 個指示變數的和，故 $\mathbb{E}[N] = t/b$。恰有 r 球入桶須選定成功的 r 次投球，因此

$$\Pr(N = r) = \binom{t}{r} (1/b)^r (1 - 1/b)^{t-r}, \quad 0 \leq r \leq t.$$

令 Z_0、Z_1 分別為空桶數、恰有一球的桶數。對每桶套用指示變數：

$$\mathbb{E}[Z_0] = b(1 - 1/b)^t,$$

$$\mathbb{E}[Z_1] = b\left[t \frac{1}{b}(1 - 1/b)^{t-1}\right] = t(1 - 1/b)^{t-1}, \quad t \geq 1.$$

$t = 0$ 時直接有 $Z_0 = b, Z_1 = 0$；$b = 1$ 的事件直接判定，或以空乘積慣例 $0^0 = 1$ 解讀邊界式。

例 $t = b = 4$：固定桶平均負載為 1，但 $\mathbb{E}[Z_0] = 81/64$、$\mathbb{E}[Z_1] = 27/16$。各桶負載總和固定為四，不獨立；平均一球不代表每桶恰有一球。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

收集所有桶：每個階段都在等下一個新桶

每次投球獨立、均勻。固定一個桶，令 T 為首次命中所需投球數，含成功的那一次。$\Pr(T > t) = (1-1/b)^t$，$t \geq 0$，故尾和給

$$\mathbb{E}[T] = \sum_{t=0}^{\infty} (1-1/b)^t = b.$$

四個桶：等到第一次命中指定的第 3 桶

1	2	3	4	5	6
2	2	4	1	4	3
未中	未中	未中	未中	未中	命中

T = 6

E[T] = 4

P(T = 6) = $(3/4)^5 \times (1/4) = 243/4096$

前五次未中，第六次命中；單次等待長度不必等於期望值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每多收集一個新桶，要再等多久

改問全部桶皆命中。已見 i 個不同桶時，新桶機率為 $(b-i)/b$；等待該次新命中的投球數 T_i，條件於目前所見集合，其期望皆為 $b/(b-i)$。未來投球仍獨立且均勻，故不取決於已耗時多久。

總停止時間 $C = \sum_{i=0}^{b-1} T_i$，期望線性性給

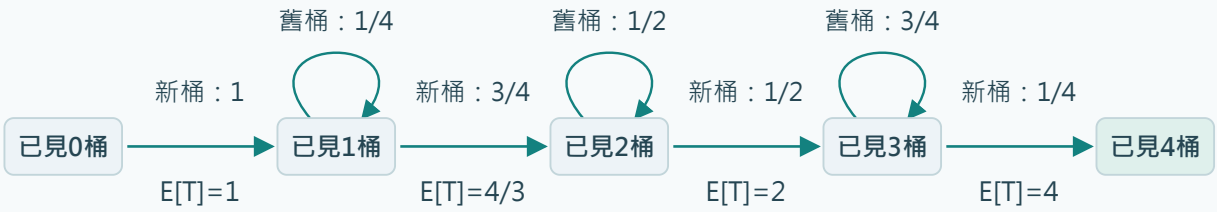
$$\mathbb{E}[C] = \sum_{i=0}^{b-1} \frac{b}{b-i} = bH_b.$$

$b \geq 2$ 時為 $\Theta(b \ln b)$；$b = 1$ 時恰為一次。這個相加步驟不必另證各階段獨立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四桶收集：重複命中時，留在原狀態

命中新桶才前進，命中舊桶留在原狀態



實際投球 2,2,4,1,4,3：第1、3、4、6球命中新桶；各階段長度為1、2、1、2。

階段期望相加為 1+4/3+2+4=25/3；上面的單次投球則在第6球集滿。

四桶各階段期望為 $1, 4/3, 2, 4$，合為 $25/3$。圖中實際投球為 $2, 2, 4, 1, 4, 3$，第 $1, 3, 4, 6$ 球命中新桶，各階段實際長度為 $1, 2, 1, 2$，合為六；單次停止時間不必等於期望。

第一次碰撞與收集完成：停止條件決定時間尺度

先界收集完成時間 C。投完 t 球後，指定桶仍空的機率為 $(1-1/b)^t \le e^{-t/b}$。任一桶空就表示尚未集滿，因此

$$\Pr(C > t) \le \min\{1, be^{-t/b}\}.$$

若整數 $t \ge b(\ln b + c)$、$c \ge 0$，未集滿機率至多 e^{-c}。各桶是否為空可以相依；這裡只用了聯集界。

同一串投球，停止條件不同，答案就不同

t = 1	t = 2	t = 3	t = 4	t = 5	t = 6
2	2	4	1	4	3
{2}	{2}	{2,4}	{1,2,4}	{1,2,4}	{1,2,3,4}

D = 2：首次重複　　C = 6：四桶全部出現

每格下方列出截至當次已見的桶；D 與 C 是同一串投球的兩種停止時間。

首次碰撞時間：用尾和證明平方根期望

令 D 為第一次出現重複桶的投球編號。$D > k$ 恰表示前 k 球都不同；$k > b$ 時機率為零，其餘回到生日補事件。對 $k \ge 2$，$k(k-1) \ge k^2/2$，故

$$\begin{aligned}\mathbb{E}[D] &= \sum_{k \ge 0} \Pr(D > k) \\ &\le 2 + \sum_{k \ge 2} e^{-k^2/(4b)} \\ &\le 2 + \int_0^{\infty} e^{-x^2/(4b)}\,dx = 2 + \sqrt{\pi b}.\end{aligned}$$

下界：對整數 $k \le \sqrt{b}/2$，聯集界給碰撞率至多 $k(k-1)/(2b) \le 1/8$，所以前 $\Theta(\sqrt{b})$ 個尾機率至少 $7/8$，得 $\mathbb{E}[D] = \Omega(\sqrt{b})$。

合為 $\mathbb{E}[D] = \Theta(\sqrt{b})$，而 $\mathbb{E}[C] = \Theta(b \ln b)$。同樣是球入桶，問題問「第一次重複」或「全部出現」，答案就不同。

連續正面：視窗數與最長長度是不同的量

獨立、公平擲硬幣 n 次，L 為最長連續正面的長度；沒有正面時 $L = 0$。對 $1 \le k \le n$，令 $A_{i,k}$ 表示第 i 到 $i+k-1$ 次全為正面，合法起點為 $1 \le i \le n-k+1$。每個視窗成功率為 2^{-k}，故

$$X_k = \sum_{i=1}^{n-k+1} \mathbf{1}_{A_{i,k}}, \qquad \mathbb{E}[X_k] = (n-k+1)2^{-k}.$$

相鄰視窗可能重疊，不能直接當成獨立事件。例如 $\Pr(A_{1,3} \cap A_{2,3}) = 2^{-4} = 1/16$，因為只需前四次都是正面；兩個邊際機率的乘積卻是 $2^{-6} = 1/64$。期望線性不要求獨立，但求聯集的精確機率需要考慮重疊。

重疊視窗如何對應同一段連續正面

同一段四連 H 會包含兩個長3視窗，不能當成獨立事件

長3全H視窗：起點1、5、6，共3個。

最長run：5..8，長度4。

重疊視窗可同時成立；第5與第6視窗共享兩次擲幣。

圖中長三的全正面視窗從 $1, 5, 6$ 開始，共 $X_3 = 3$ 個；第 $5..8$ 次的四連正面貢獻了其中兩個視窗。最長長度是 $L = 4$，與視窗數、正面段數都不同。

最長連續正面的上界：先求尾機率，再求期望

事件 $\{L \geq k\}$ 是所有長 k 全正面視窗的聯集。對 $1 \leq k \leq n$，聯集界給

$$\Pr(L \geq k) \leq \min\{1, (n-k+1)2^{-k}\} \leq \min\{1, n2^{-k}\}.$$

令 $k_0 = \lceil \lg n \rceil$，其中 $n \geq 1$。由整數尾和及等比級數，

$$\begin{aligned} \mathbb{E}[L] &= \sum_{k=1}^{n} \Pr(L \geq k) \\ &\leq k_0 + \sum_{k=k_0+1}^{\infty} n2^{-k} = k_0 + n2^{-k_0} \leq \lceil \lg n \rceil + 1. \end{aligned}$$

這裡 $\lg$ 為以 2 為底的對數。若 $k > n$，事件不可能；$n = 0$ 時直接令 $L = 0$，不代入對數。

對 $r \geq 1$ 取整數門檻 $k = \lceil r \lg n \rceil$，可得 $\Pr(L \geq k) \leq n^{1-r}$。例如 $n = 1000, k = 20$ 的精細上界為 $981/2^{20} < 10^{-3}$；$k = 30$ 時可用較鬆的 $1000/2^{30} < 10^{-6}$。這些界控制單次執行出現異常長段的機率。

下界：把硬幣切成互不重疊的獨立區塊

對充分大的 n，令 $s = \lfloor (\lg n)/2 \rfloor \geq 1$、$m = \lfloor n/s \rfloor$。將前 ms 次切成 m 個長 s 的區塊，尾端不足一塊者不用。每塊全正面的機率為 $2^{-s} \geq n^{-1/2}$；不同區塊使用互不重疊的獨立硬幣，因此這些成功事件相互獨立。

若 $L < s$，所有區塊都必須失敗；反方向未必成立，因為長段可能跨越區塊邊界。因此只有下列不等式：

$$\Pr(L < s) \leq (1 - 2^{-s})^{m} \leq e^{-m 2^{-s}} \leq e^{-m/\sqrt{n}} \longrightarrow 0.$$

由 $m = \Theta(n/\lg n)$，有 $m/\sqrt{n} \to \infty$。再利用非負性，

$$\mathbb{E}[L] \geq s \Pr(L \geq s) = \Omega(\lg n).$$

配合上界即得 $\mathbb{E}[L] = \Theta(\lg n)$。這個證明用到了獨立區塊；不能只因重疊視窗的期望很多，就斷言至少一個成功的機率很高。

有限數例：$n = 16, s = 2, m = 8$。至少一個指定區塊為 HH 的機率為 $1 - (3/4)^8 = 58975/65536$，故 $\mathbb{E}[L] \geq 117950/65536$。這是可核對的下界，並非精確期望。

線上遴選：先固定資訊與決策限制

已知有 $n \geq 2$ 位能力相異的候選人，以均勻隨機排列抵達。每次面試只能與已見者比較；拒絕後不能召回，最多雇用一次。目標為選到全域最佳者，並非讓面試成本最小。

取整數 $1 \leq k < n$，前 k 人只觀察、不雇用，記錄最高分 M；後段遇到第一位超過 M 者便選取。若始終無人超過，則選最後一人。

第二個迴圈不更新 M：在第一個超過門檻者出現以前，其他分數都不高於 M；一旦超過便立即返回。最末面試若未通過比較，仍可直接雇用當前第 n 人，並非召回已離開者。$n = 1$ 直接選唯一者；$k = 0$ 另定義為選第一人。

線上遴選：程序與停止條件

```
ONLINE-SELECT(n,k):
    require n >= 2 and 1 <= k < n
    M = -infinity
    for i = 1 to k:
        M = max(M, score(i))      // interview, then reject
    for i = k+1 to n:
        if score(i) > M: return i
    return n
```

檢查每位後段候選人之前，先前所有後段分數都沒有超過觀察段最高分 M。一旦超過就立即停止；始終沒有超過，才啟用最後一人的備援選擇。

成功機率：條件於最佳者的抵達位置

條件於全域最佳者在位置 i。若 $i \le k$，它已在觀察階段被拒絕，一定失敗。若 $i > k$，成功恰好要求前 $i-1$ 人中的最高分者落在前 k 位：否則後段已有人先超過 M，程序會提早停止；反之，位置 i 就是第一次超過門檻。

最佳者位置均勻，機率為 $1/n$。固定其位置後，前 $i-1$ 人的相對排列仍均勻，因此其中最大者落在前 k 位的條件機率為 $k/(i-1)$。對互斥的位置事件相加，

$$P_k = \sum_{i=k+1}^{n} \frac{1}{n}\frac{k}{i-1} = \frac{k}{n}\sum_{j=k}^{n-1}\frac{1}{j}.$$

推導使用條件機率，沒有直接假設「最佳者位置」與「先前最大者位置」獨立。

邊界也要單獨定義：$n = 1$ 時成功率 1；$k = 0$ 時選第一人，成功率 $1/n$。式中的調和和從 $k \ge 1$ 起才有定義。

最佳門檻：有限差分與漸近極限分開求

令 $q_k = \sum_{j=k}^{n-1} 1/j$，並定義空和 $q_n = 0$。因 $q_k = 1/k + q_{k+1}$，對 $1 \le k \le n-2$ 有

$$P_{k+1} - P_k = \frac{(k+1)q_{k+1} - k q_k}{n} = \frac{q_{k+1} - 1}{n}.$$

q_k 隨 k 嚴格下降，故成功率先增後減。門檻 k 滿足 $q_k \ge 1 \ge q_{k+1}$ 時達最大；等號可能使相鄰門檻並列。$n = 2$ 的唯一正門檻為 $k = 1$，與 $k = 0$ 同為 $1/2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最佳成功率為何趨近 1/e

遞減函數 $1/x$ 的積分夾界給

$$\ln(n/k) \le q_k \le \ln(n/k) + 1/k,$$

因此 $f(k/n) \le P_k \le f(k/n) + 1/n$，其中 $f(x) = x \ln(1/x)$。由 $f'(x) = \ln(1/x) - 1$、$f''(x) = -1/x < 0$，最大值在 $x = 1/e$，等於 $1/e$。

選離 n/e 最近的合法整數，使 k/n 距 $1/e$ 為 $O(1/n)$；連續可微性給 $f(k/n) = 1/e - O(1/n)$。所有門檻又有 $P_k \le 1/e + 1/n$，故最佳成功率趨近 $1/e$。有限 n 的最佳整數門檻仍應用差分求，不能直接把 n/e 當成精確答案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

六人遴選：門檻機率與每次決策都能核對

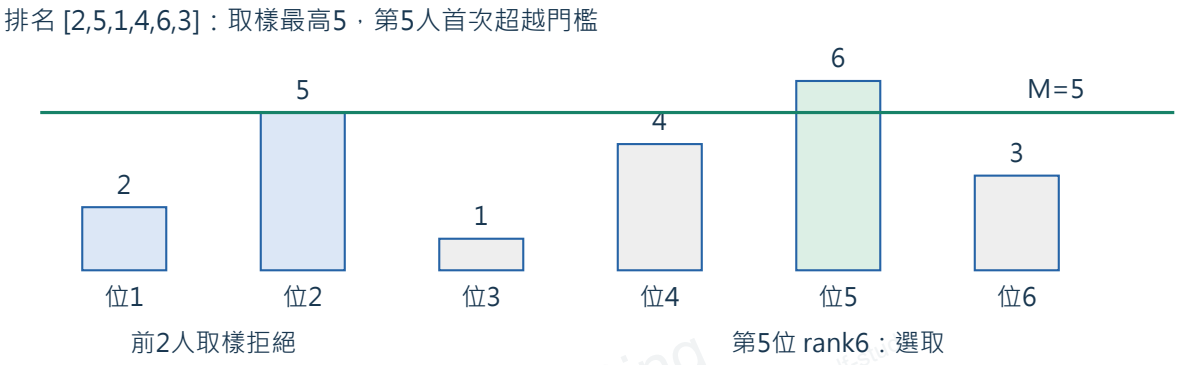
取 $n = 6$，將各門檻代入 $P_k = (k/6)q_k$：

k	q_k	精確成功率 P_k
1	$137/60$	$137/360$
2	$77/60$	$77/180$
3	$47/60$	$47/120$
4	$9/20$	$3/10$
5	$1/5$	$1/6$

$q_2 > 1 > q_3$，故最佳正門檻是 $k = 2$；$k = 0$ 成功率只有 $1/6$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從觀察門檻到每一位候選人的決策



排名 $[2, 5, 1, 4, 6, 3]$：前兩人拒絕，$M = 5$；第三人 $1 \le 5$、第四人 $4 \le 5$，均拒絕；第五人 $6 > 5$，立即雇用，第六人無須面試。失敗例 $[5, 6, 1, 2, 3, 4]$ 則在取樣時拒絕了最佳者，後段無人超過 6，最後按規則選 4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

隨機遞迴分割：條件期望與葉數守恆

定義一個額外的成本模型：大小 $n \geq 2$ 的呼叫先輸出 n 個星號，再均勻選 $p \in \{1, \ldots, n-1\}$，處理大小 $p, n-p$ 的兩個子問題；大小 1 輸出一個驚嘆號後停止。每次抽樣在給定先前歷史後仍均勻。

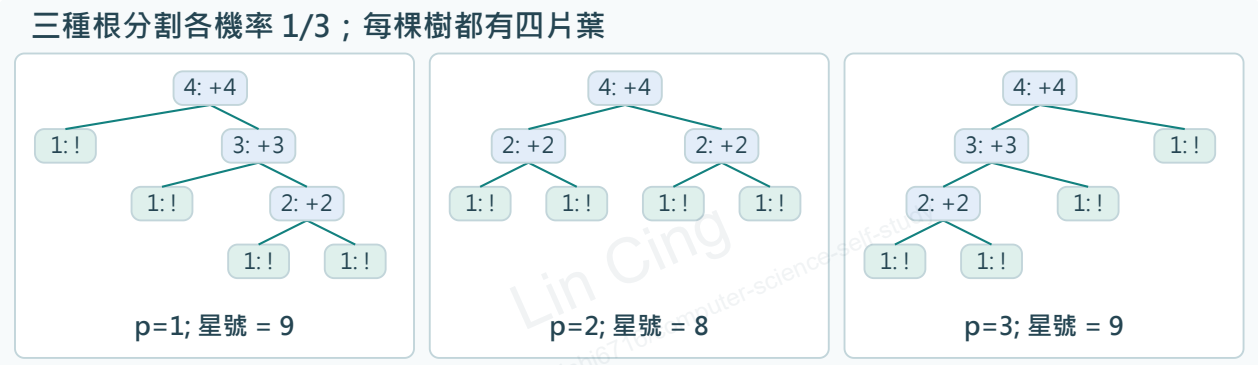
令 L_n, S_n 分別為驚嘆號、星號的期望，$L_1 = 1, S_1 = 0$。固定分割點 p 後，兩側期望相加；再平均所有 p，利用 $j \leftrightarrow n-j$ 的對稱，

$$L_n = \frac{2}{n-1} \sum_{j=1}^{n-1} L_j, \qquad S_n = n + \frac{2}{n-1} \sum_{j=1}^{n-1} S_j.$$

不需要假設兩側成本獨立，因為用的是期望的加法。每條分支的整數規模都嚴格下降，故必然終止。

四筆分割：葉數相同，成本卻不同

每次分割保存大小總和，最終每片葉的大小都是 1，所以**每次執行都恰有 n 片葉**，不只是期望如此。星號數卻會改變：$n = 4$ 時根成本為 4；平衡分成 $2 + 2$，總成本 $4 + 2 + 2 = 8$；分成 $1 + 3$ 時，大小 3 的成本為 $3 + 2 = 5$，總成本為 9。因此期望 $S_4 = (9 + 8 + 9)/3 = 26/3$。



解出隨機分割成本：相鄰規模相減

葉數可用強歸納核對：若 $j<n$ 時 $L_j=j$，則 $L_n=\frac{2}{n-1}\sum_{j<n} j=n$。

星號成本先乘開分母：

$$(n-1)S_n=n(n-1)+2\sum_{j<n}S_j,$$

$$nS_{n+1}=n(n+1)+2\sum_{j<n}S_j+2S_n.$$

兩式相減得 $nS_{n+1}-(n-1)S_n=2n+2S_n$，所以 $S_{n+1}=2+\frac{n+1}{n}S_n$。此差分對 $n\ge 2$ 由上式成立，$n=1$ 可由 $S_1=0, S_2=2$ 直接核對。

令 $Q_n=S_n/n$，則 $Q_{n+1}=Q_n+2/(n+1)$。由 $Q_1=0$ 累加，得

$$Q_n=2(H_n-1),\qquad S_n=2n(H_n-1)=\Theta(n\ln n).$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

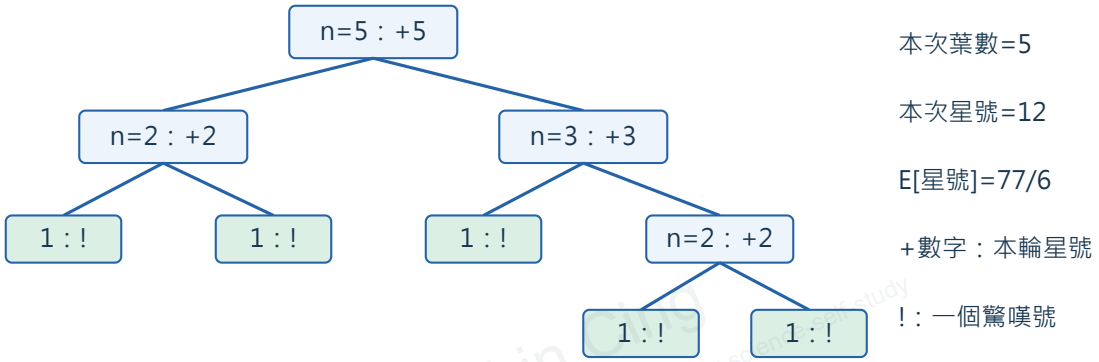
用小規模輸入核對封閉式

n	每次執行葉數	$\mathbb{E}[\text{星號數}]$
2	2	2
3	3	5
4	4	$26/3$
5	5	$77/6$
6	6	$87/5$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

期望成本與最壞執行仍須分開

五個單位葉完整展開：守恆的是葉數，不是星號成本



圖示 p=2、之後 3=1+2；葉數始終5，但星號總數會隨分割而變。

最壞路徑每次切出 1，星號數為 $2+3+\cdots+n=n(n+1)/2-1$；期望的線性對數界並不是每次執行的保證。

參考資料與署名

參考用書：Thomas H. Cormen、Charles E. Leiserson、Ronald L. Rivest、Clifford Stein，Introduction to Algorithms，第 3 版，第 5–9 章。 補充講義：MIT 6.046J/18.401J，Fall 2005。Lecture 4: Quicksort（Charles E. Leiserson，2005-09-21）；Lecture 5: Sorting Lower Bounds / Linear-Time Sorting（Erik Demaine，2005-09-26）；Lecture 6: Order Statistics（Erik Demaine，2005-09-28）。

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

資料結構與攤銷分析

資料結構把抽象操作變成可維護的狀態。先問「哪些狀態合法、操作保證什麼」，再問「一串操作總共花多少工作」。

本章依序學習：**表示與操作規格 → 攤銷分析 → 雜湊 → 搜尋樹與擴充 → B-tree → Fibonacci heap → 整數 universe**。

前置知識為第 1 章的規格與不變量、第 2 章的遞迴分析、第 3 章的機率與 heap。這裡先用兩個 stack 和動態陣列，把後面要用的成本工具建立起來。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

操作名稱相同，前提可能不同

抽象資料型態（ADT）描述可觀察行為；陣列、鏈結與樹則是實作。先固定契約，複雜度才有意義。

操作	必須先回答
DELETE	給的是 key、陣列索引，還是現成的節點 handle？
SEARCH	同鍵可否重複？要回傳哪一筆？
POP／DEQUEUE	空結構回傳錯誤、特殊值，還是根本不允許呼叫？
指標更新	節點物件身分要保留嗎？外部 handle 是否仍有效？

雙向鏈結串列刪除已知節點可為常數時間，但「先按 key 找到節點再刪除」通常還要加搜尋成本。後面的每個定理都必須配合它自己的輸入規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

環狀佇列：實體位置不等於邏輯順序

固定容量 $C \geq 1$，使用陣列 $A[0..C-1]$、首筆索引 h 與元素數 s。合法狀態滿足 $0 \leq h < C$、$0 \leq s \leq C$，第 j 筆資料位於

$$Q[j] = A[(h + j) \bmod C], \qquad 0 \leq j < s.$$

空為 $s = 0$，滿為 $s = C$；不靠兩個相等的索引猜測。下一個插入位置是 $(h + s) \bmod C$，但只有未滿時才能寫入。每個實體格都可使用。

陣列中的舊值不等於仍在 queue 的資料。出列後可清除該格以釋放持有的資源，但成員資格由 h, s 決定。固定容量的 ENQUEUE／DEQUEUE 都是最壞 $O(1)$；配置陣列與元素複製成本須另列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先檢查邊界，再更新佇列狀態

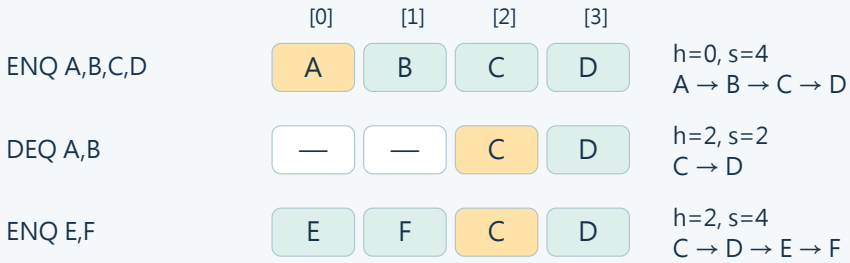
```
ENQUEUE(x):
    if s == C: return overflow
    A[(h+s) mod C] = x
    s = s + 1

DEQUEUE():
    if s == 0: return underflow
    x = A[h]
    h = (h+1) mod C
    s = s - 1
    return x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

假設元素讀寫成功，操作循序執行。失敗呼叫不改 h, s 或任何有效資料；資料值與錯誤狀態用不同回傳型態區分。此處不是多執行緒 lock-free queue，不能省略同步後直接共用。

繞回後，仍然依 C、D、E、F 出列

環狀佇列：先看實體槽，再沿邏輯順序讀



橘色 = 首筆；— = 非成員，不要求清除舊位元

先放入 A～D，取出 A、B，再新增 E、F。實體為 [E,F,C,D]，但 $h = 2, s = 4$，讀取索引為 $2 \to 3 \to 0 \to 1$。再插入 G 必須拒絕：此時下一個插入位置等於 h，卻是滿態。

環狀索引為何維持 FIFO？

插入前 $s < C$，索引 $(h + s) \bmod C$ 不等於任一有效位置 $(h + j) \bmod C$，其中 $0 \le j < s$：若相等，便有 $C \mid (s - j)$，但 $1 \le s - j < C$，矛盾。因此新資料不覆寫舊資料，恰接在邏輯序列尾端。

出列回傳 $A[h] = Q[0]$。更新後 $h' = (h + 1) \bmod C, s' = s - 1$，對 $0 \le j < s'$ 有

$$A[(h' + j) \bmod C] = A[(h + j + 1) \bmod C] = Q[j + 1].$$

所以剩餘序列剛好去掉首筆，次序不變。初態 $s = 0$ 合法；成功操作維持不變量，失敗操作保持原態，歸納便涵蓋所有合法呼叫序列。

另一種表示法：保留一格，換掉 size

CLRS 第 10.1 節用 head 指首筆、tail 指下一個插入格，保留一格：以本節 0 起算記號，空為 $h = t$，滿為 $(t+1) \bmod C = h$，可存 $C-1$ 筆。

另一常見慣例以 f 指首筆之前、r 指末筆，空為 $f = r$、滿為 $(r+1) \bmod C = f$。兩者換算為 $h = (f+1) \bmod C$、$t = (r+1) \bmod C$。不能只把變數名稱替換，卻保留相同讀寫順序。

本章採 h, s，可存滿 C 格；需要時可計算 $t = (h+s) \bmod C$。此時 $t = h$ 同時可能是空或滿，必須查看 s。保留一格、額外計數、full 旗標是不同設計，不能混用判斷式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

單向鏈結插入：先保留原本的後繼

給定有效節點 p，把尚未加入串列的新節點 x 插在它之後。節點身分固定，key 可重複；x 不得已在同一串列裡。

```
q = p.next
x.next = q
p.next = x
```

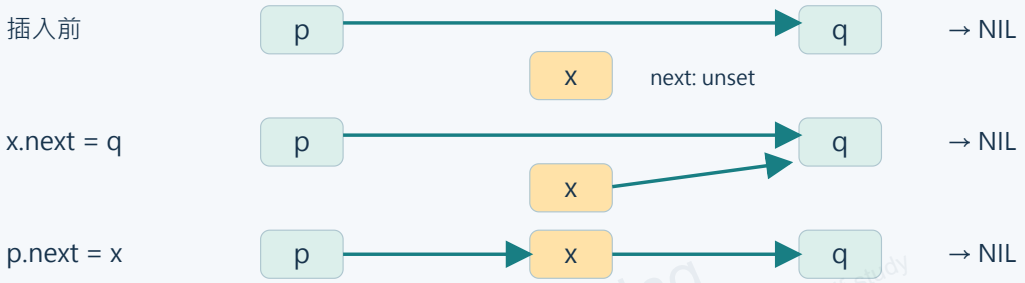
原路徑 $p \to q$ 變成 $p \to x \to q$，其餘 next 不變，元素只增加 x。若 q 為 NIL，同樣適用；若另存 tail，須把原 tail=p 的情況更新為 tail=x。

若先寫 `p.next=x`，再寫 `x.next=p.next`，會得到 x 指向自己的迴圈，原 q 也可能失去可達性。$O(1)$ 指的是已知 p 的改接；尋找 p、配置 x 及複製非定額資料另計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三個狀態看清楚：新增節點如何接回原路徑

先接好新節點，再改原路徑



先讓 x.next 接到 q，再讓 p.next 接到 x。中間態中，x 還不是由 head 可達的成員；完成後才成為串列的一部分。圖中的節點代號是身分，不以 key 值取代。

單向刪除：已知節點不等於已知前驅

刪除 p 後面的節點時，先存 `x=p.next`；若 x=NIL 即無後繼可刪。否則存 `q=x.next`，令 `p.next=q`。若 x 是 tail，令 tail=p；最後才回收 x。只給 x 而不給前驅，刪除尾節點一般需從 head 找前驅，最壞 $\Theta(n)$。

把「把下一筆資料複製到 x，再刪下一筆」當作常數刪除，會改變 x 的資料身分，也不適用尾節點；不符合這裡保留節點身分的規格。

原地反轉：先保留尚未處理的後綴

前提是以 NIL 結束、無環的單向串列：

```
prev = NIL; cur = head
while cur != NIL:
    nxt = cur.next
    cur.next = prev
    prev = cur
    cur = nxt
head = prev
```

若另存 tail，先記 old_head，最後令 tail=old_head。迴圈中 prev 是已處理前綴的反向串列，cur 是尚未處理後綴的首筆；保存 nxt 才能在反向改接後繼續前進。每步少一個未處理節點，故時間 $\Theta(n)$、額外空間 $O(1)$。

雙向環狀串列：sentinel 把端點接起來

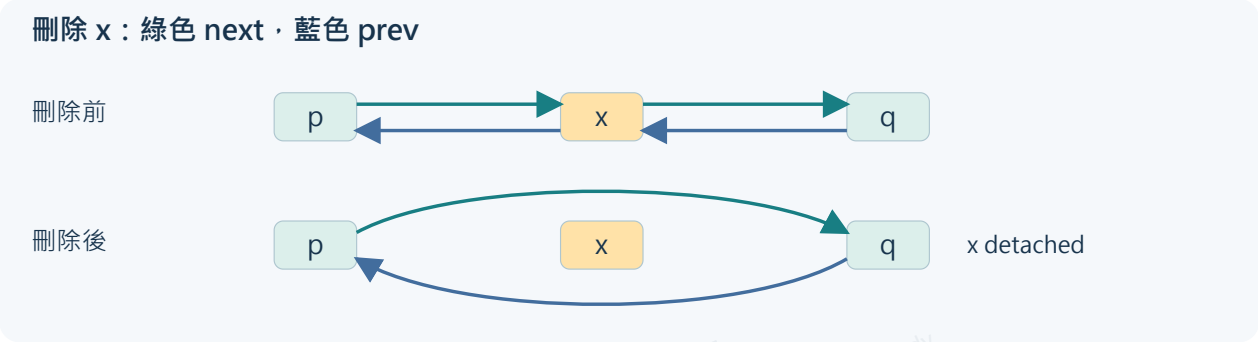
配置固定的虛擬節點 z（sentinel），不算資料元素。空串列滿足 `z.next=z.prev=z`；非空時 z.next 是首筆、z.prev 是末筆。

對每個在環中的節點 v，包含 z，維持 `v.next.prev=v` 與 `v.prev.next=v`。只接受本串列中的有效 handle，禁止刪除 z。檢查陌生 handle 是否屬於本串列的成本，不包含在已知成員的常數改接內。

```
DELETE(x):                  // x != z, valid member
    p = x.prev; q = x.next
    p.next = q
    q.prev = p
```

p 或 q 可以是 z，故同一段涵蓋首筆、末筆與唯一資料節點。改接完成才回收 x；外部持有 x 的 handle 隨後失效。這份規格採循序操作，中間指標狀態不可直接視為可供並行讀取。

雙向刪除：改兩條鄰居連線，保留其餘路徑



原本 p.next=x、q.prev=x，改成 p.next=q、q.prev=p。其餘成員的相鄰關係不變，x 從環中移除；x 的舊欄位本身不代表它仍是成員。

只有一筆資料時 p=q=z，兩次寫入分別使 z.next=z、z.prev=z，正好回到空態。不要把「兩個指標寫入」誤解為「刪除任意 key 都只花兩步」：先找 key 仍可能掃過整條串列。

雙向插入：四個連結都要對得上

把 detached 節點 x 插到已知 p 之後；p 可以是 z。先存 q=p.next：

```
x.prev = p
x.next = q
q.prev = x
p.next = x
```

完成後 p↔x↔q，其餘連結不變。原本空時 p=q=z，z.next、z.prev 都會變為 x。若 p 是末筆，q=z，所以 z.prev 自動更新為新末筆。

每條新的雙向相鄰關係都有對應回鏈，故恢復不變量。這裡的寫入順序只保證循序呼叫完成後正確；不提供 concurrent publication 保證。sentinel 簡化邊界，但每個串列多占一個節點，並不改變搜尋的漸近界。

用陣列索引表示節點：位置和順序分開

不用原生 pointer，也能用平行陣列 key、next、prev 表示節點；同一索引的三個欄位屬於同一物件。索引 0 保留為 NIL，合法物件使用 1～5。

索引	狀態	key	next	prev
1	free	—	4	—
2	live	B	3	5
3	live	C	0	2
4	free	—	0	—
5	live	A	2	0

head=5，依 next 得 $5 \to 2 \to 3 \to 0$，所以資料順序是 A→B→C；free=1，空格鏈為 $1 \to 4 \to 0$。實體相鄰的索引不是邏輯相鄰的保證。

配置與回收：live list 和 free list 不可重疊

固定物件池的槽位分成互斥的三類：live list、free list、呼叫者持有。ALLOCATE 把 free 槽交給呼叫者，初始化並接入後才變成 live。free list 只用 next 欄位，採 stack 方式管理。

```
ALLOCATE():
    if free == 0: return exhausted
    x = free; free = next[x]
    return x                    // initialize before linking

FREE(x):                        // detached, not already free
    next[x] = free
    free = x
```

配置與回收各改常數個欄位，所以在固定物件池內為 $O(1)$；不包含擴充池或配置不定長資料的成本。

物件池數例：從配置到回收

沿用前面的五個槽，head 始終為 5：

完成的操作	live 鏈（到 NIL=0）	free 鏈	呼叫者持有
起態	5(A)→2(B)→3(C)→0	1→4→0	無
ALLOCATE 得 1	5→2→3→0	4→0	1
D 接在 C 之後	5→2→3→1(D)→0	4→0	無
刪除並 FREE(2)	5→3→1→0	2→4→0	無

接入 D：key[1]=D、prev[1]=3、next[1]=0、next[3]=1。刪 B：先改 next[5]=3、prev[3]=5，再令 next[2]=4、free=2。

重複 FREE 會破壞 free list；舊 handle 也可能誤指重用後的新物件。索引在範圍內並不足以證明生命週期有效。

兩個 stack 如何保持 FIFO

用 S 接收新資料，用 T 提供下一筆出列資料。下列序列記號依指定方向讀取：

$$Q = \mathrm{reverse}(T_{\mathrm{bottom}\to\mathrm{top}}) \parallel S_{\mathrm{bottom}\to\mathrm{top}}.$$

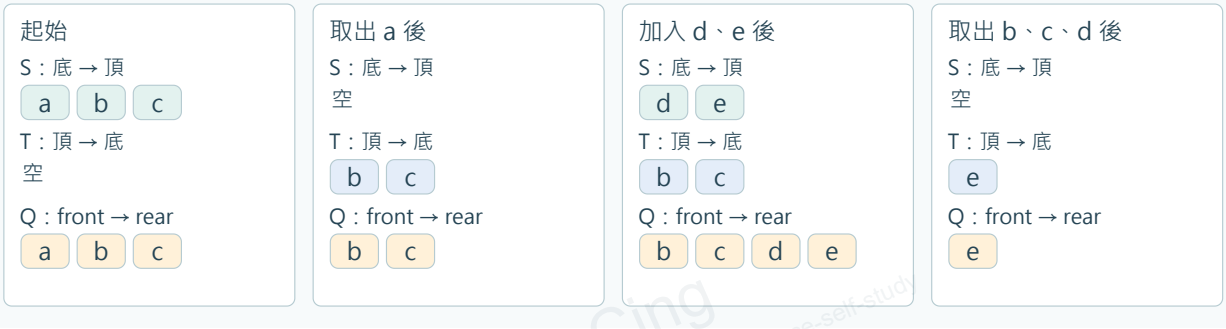
$\parallel$ 表示串接。因此 **T 的 top 是 queue 的 front**；只有 T 空時，才把 S 全部倒進 T。

ENQUEUE 把新資料 push 到 S，等於接在 Q 尾端。若 T 非空，DEQUEUE pop 它的 top，正好移除 Q 首項。若 T 空，倒轉 S 後，最早進入的資料成為 T 的 top；順序仍等於原來的 Q。

兩者皆空時回傳 underflow，狀態不變。**不能每次出列都倒轉 S**：T 尚有舊資料時，新資料必須等在它們後面。

先看資料順序，再看昂貴的搬移

同一個 queue，兩個 stack：舊資料先出列



圖中的 S 由底讀到頂，T 由頂讀到底。加入 d, e 時，b, c 仍在 T 等待，所以不用搬動。只有 b, c 都已出列，才倒轉 S，接著取出 d。

昂貴一次，不等於每次都昂貴

以一次 stack PUSH 或 POP 為 1 單位；暫不計每次 queue 呼叫固定的判斷開銷。ENQUEUE 成本 1。若 DEQUEUE 需搬移 k 筆，成本是 $2k + 1$：每筆先 pop S 再 push T，最後 pop T。

從空 queue 出發，每筆資料至多經過四步：**進 S → 離開 S → 進 T → 離開 T**。所以 e 次入列、d 次成功出列的總成本至多 $3e + d$。

任意長度 m 的合法操作序列都有 $e + d \leq m$，故總成本至多 $3m$；補回每次固定判斷後仍為 $O(m)$。這個結論沒有假設操作隨機，也不需要「大多數情況便宜」。

單次 DEQUEUE 仍可能是 $\Theta(n)$。攤銷界是整段序列的保證，不能用來宣稱每次呼叫有相同延遲。

勢能法：把延後的工作記在狀態上

第 i 次操作把狀態 D_{i-1} 變成 D_i，實際成本為 c_i。選一個只依狀態決定的函數 Φ，定義攤銷成本

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

$\Delta\Phi > 0$ 表示本次多收一點，為以後存下額度；$\Delta\Phi < 0$ 表示用先前累積的額度支付工作。它不是另一種實際執行時間。

把前 m 次相加，中間勢能成對抵消：

$$\sum_{i=1}^{m} c_i = \sum_{i=1}^{m} \hat{c}_i + \Phi(D_0) - \Phi(D_m).$$

若所有可達狀態都滿足 $\Phi(D_i) \geq \Phi(D_0)$，則總實際成本不超過總攤銷成本。常用的充分條件是 $\Phi(D_0) = 0$ 且 $\Phi \geq 0$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

初始狀態與負攤銷費用，都不能略過

若只證明 $\Phi \geq 0$，卻沒有 $\Phi(D_0) = 0$，正確結論是

$$\sum c_i \leq \sum \hat{c}_i + \Phi(D_0).$$

例如 queue 一開始已有 n 筆資料在 S，一次出列就可能搬移 n 筆。不能把這些初始資料當成之前已付費；必須計入建立初態的成本，或保留 $O(n)$ 初始項。

個別 $\hat{c}_i$ 可以為負。只要整體勢能條件成立，望遠鏡等式仍正確；負值表示這次釋放的信用比實際工作還多。

攤銷（amortized）針對所有合法序列給總界；**期望**（expected）必須指定亂數或輸入分布。同一資料結構也可能同時有「期望攤銷」界，兩層量詞不能省略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩個 stack 的勢能，只計尚未搬過的資料

令 $\Phi(S, T) = 2|S|$，初態兩個 stack 皆空，所以 $\Phi_0 = 0$。

操作	實際成本 c	$\Delta\Phi$	$\hat{c}$
ENQUEUE	1	2	3
T 非空時 DEQUEUE	1	0	1
T 空，搬移 $k \geq 1$ 筆後出列	$2k + 1$	$-2k$	1
空 queue 回報 underflow	0	0	0

最後一列僅計 stack primitives；若計呼叫開銷，四列各加固定常數。T 中的資料已支付搬移費，不再需要額外勢能。

每個前綴都滿足總成本 $\leq 3e + d$。這與逐筆資料最多做四次 primitive 的 aggregate 證明相符。

交錯操作的完整帳目

兩個 stack 均以**底→頂**列出；$\Phi = 2|S|$。起終勢能皆為 0，兩種成本均合計 19。

操作	S	T	c	Φ	$\hat{c}$
ENQ a	a	空	1	2	3
ENQ b	a,b	空	1	4	3
ENQ c	a,b,c	空	1	6	3
DEQ → a	空	c,b	7	0	1
ENQ d	d	c,b	1	2	3
ENQ e	d,e	c,b	1	4	3
DEQ → b	d,e	c	1	4	1
DEQ → c	d,e	空	1	4	1
DEQ → d	空	e	5	0	1

計費法：每個前綴都要付得起

第 i 次操作實付 c_i，分析時收取 a_i，餘額為

$$B_t = B_0 + \sum_{i=1}^{t} (a_i - c_i).$$

從空結構開始，取 $B_0 = 0$。若能對任意合法序列的每個前綴證明 $B_t \geq 0$，便得到 $\sum_{i=1}^{t} c_i \leq \sum_{i=1}^{t} a_i$。

不能先透支，再承諾未來補回：呼叫者可能在透支當下停止，該前綴的界就已失效。若初態自帶信用 $B_0 > 0$，必須把它列入總成本界，不能當成免費工作。

信用可附在元素上，方便證明它不會被重複花用；若總信用只由目前狀態決定，取 $\Phi(D) = B(D)$，就得到前面的勢能法。

MULTIPOP：大量刪除的成本由誰支付？

空 stack 支援 PUSH、POP 與 MULTIPOP(k)，其中 $k \geq 0$；後者最多移除 k 個元素，遇空即停。每次成功 push／pop 的基本成本為 1。

PUSH 收 2：付當次成本 1，另存 1 在新元素上。POP 與 MULTIPOP 收 0，用被移除元素的信用付款。**每個仍在 stack 的元素恰有 1 單位信用**，由歸納可知餘額從不為負。

若 m 次呼叫內有 p 次 PUSH，成功移除總數 $q \leq p$，所以實付 $p + q \leq 2p \leq 2m$。同一物件再次 push 是新一次插入，必須重新付費。

此處只數成功的元素操作。若每次 API 呼叫另收固定的檢查／派送成本 1，總界改為 $m + p + q \leq 3m$；因此空 MULTIPOP 不代表程式完全不用時間。

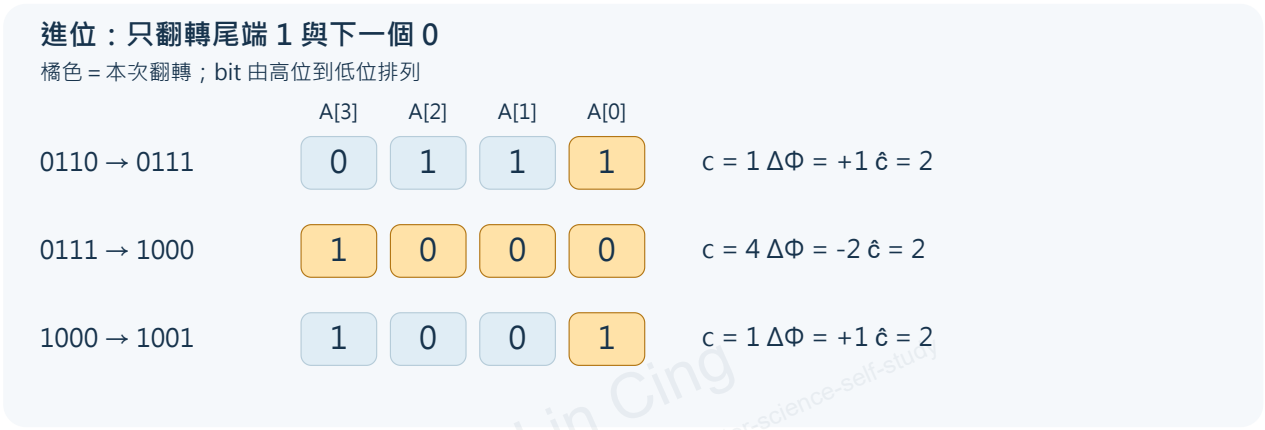
Binary counter：先固定寬度與溢位規則

使用固定 $k \geq 1$ 個 bit，$A[0]$ 是最低位，值為 $\sum_{j=0}^{k-1} A[j]2^j$。初值全 0；INCREMENT 計算模 2^k 的加一，故全 1 之後回到全 0。

```
INCREMENT(A, k):
    j = 0
    while j < k and A[j] == 1:
        A[j] = 0
        j = j + 1
    if j < k:
        A[j] = 1
```

成本 c_i 只計被翻轉的 bit 數。測試與迴圈控制的實際時間為 $\Theta(1 + c_i)$，因此另加每次常數不改變總時間的漸近界。不要把增加位寬的任意精度整數，直接當成這份規格。

進位不是同時改掉所有 bit



0111→1000 雖翻轉四個 bit，卻同時用掉先前存下的勢能。下一頁把一般進位與溢位分開計算。

一般進位與溢位，分開計算

一般進位若遇到 t 個尾端 1，先把它們清零，再把下一個 0 設為 1：$c = t + 1$。取勢能 $\Phi =$ 目前 1 的個數，便有 $\Delta\Phi = 1 - t$、$\hat{c} = 2$。

溢位是獨立分支：$t = k$，全部清零而沒有新的 1，故 $c = k$、$\Delta\Phi = -k$、$\hat{c} = 0$。不能在這個分支仍寫 $t + 1$ 次翻轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一串加一，三種分析得到同一個界

從全 0 開始做 $m \geq 1$ 次加一。第 j 個 bit 每 2^j 次操作翻轉一次，包含歸零的溢位；故總翻轉數恰為

$$T(m, k) = \sum_{j=0}^{k-1} \left\lfloor \frac{m}{2^j} \right\rfloor < 2m.$$

聚合分析直接加總各 bit 的次數。**計費法**每次收 2；把 0 設為 1 時，付 1 並留 1 給未來清零，每個 1 都帶有自己的信用。**勢能法**以 1 的個數記錄同一筆信用，普通加一收 2、溢位收 0。

例如 $k = 4, m = 8$：各 bit 翻轉 $8, 4, 2, 1$ 次，共 15；不是 $8 \times 4 = 32$。一般證明涵蓋任意 m，這個數例只用來核對計數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整帳目：從 0000 到溢位歸零

四位元，起態 $\Phi_0 = 0$；bit 字串依最高位到最低位書寫。前 8 次攤銷總額 16，扣除終態勢能 1，實付 15。

第幾次	操作後狀態	c_i	Φ_i	累積實付
1	0001	1	1	1
2	0010	2	1	3
3	0011	1	2	4
4	0100	3	1	7
5	0101	1	2	8
6	0110	2	2	10
7	0111	1	3	11
8	1000	4	1	15

完整帳目：最高位也要付清

接續同一串操作。第 9～15 次各收 2，第 16 次溢位收 0。全程實付與攤銷總額均為 30，終態沒有剩餘信用。

第幾次	操作後狀態	c_i	Φ_i	累積實付
9	1001	1	2	16
10	1010	2	2	18
11	1011	1	3	19
12	1100	3	2	22
13	1101	1	3	23
14	1110	2	3	25
15	1111	1	4	26
16	0000	4	0	30

哪些改動會破壞常數攤銷界？

非零初態：若從 k 個 1 開始，第一次就花 k。原勢能仍可用，但只能推出 $\sum c_i \le 2m + \Phi_0 \le 2m + k$；不能省掉初始信用。

加入 DECREMENT：對 $k \ge 2$，先從 0 加到 $2^{k-1} - 1$，再反覆交替 INCREMENT、DECREMENT。狀態在 $0\underbrace{11\cdots1}_{k-1}$ 與 $1\underbrace{00\cdots0}_{k-1}$ 間往返，每次都翻轉 k 個 bit。

令準備階段長度 $r = 2^{k-1} - 1$，再做 $2r$ 次交替，總呼叫 $3r$、僅交替部分已花 $2rk$，因此可達 $\Omega(mk)$。每次至多翻 k 個 bit，亦有 $O(mk)$；這個壞序列的成本為 $\Theta(mk)$。

問題在於同一批昂貴進位可以立即被借位反轉，先前「每個 1 預付一次清零」的信用不再足夠。操作集合改了，必須重新證明成本界。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

動態陣列：先把搬移成本算清楚

狀態為 (n, M)：n 是有效元素數，M 是容量。從 $(0, 0)$ 開始；首次插入配置 1 格，滿表時先把容量倍增，再加入新資料。

成本只計**寫入新資料 1 次，加上搬移的舊資料數**。配置、釋放與每次判斷的固定開銷暫略。若配置會初始化每一格，必須另計；漸近結論仍可成立，精確常數會改變。

第幾次插入	插入後 (n, M)	本次成本	累積成本
1	(1,1)	1	1
2	(2,2)	2	3
3	(3,4)	3	6
4	(4,4)	1	7
5	(5,8)	5	12

五次插入共搬移 $1 + 2 + 4 = 7$ 筆，**搬移量不一定小於插入次數**。要證明線性總成本，須用完整幾何級數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

只擴張時，aggregate 與勢能給同一總界

插入 $m \geq 2$ 筆，令 L 為小於 m 的最大 2 冪。搬移總數為 $1 + 2 + \cdots + L = 2L - 1$，所以 $T(m) = m + 2L - 1 < 3m$。$m = 0, 1$ 分別有成本 0、1。

勢能取 $\Phi(0, 0) = 0$，非空表取 $\Phi(n, M) = 2n - M$。僅插入的可達狀態滿足 $M/2 < n \leq M$，故勢能非負。

分支	c	$\Delta\Phi$	$\hat{c}$
首次插入 $(0, 0) \to (1, 1)$	1	1	2
未滿，不搬移	1	2	3
滿表 $(M, M) \to (M + 1, 2M)$	$M + 1$	$2 - M$	3

因此 $T(m) \leq 3m$。**首次插入的攤銷成本是 2**；不能把含 $\log 0$ 的公式直接套用初態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

縮表若太積極，就會反覆搬同一批資料

考慮錯誤策略：滿表時倍增；刪除後使用率一到 $1/2$ 就減半。

取 $k \geq 2$ 為 2 的冪，從滿表 (k, k) 插入一筆，擴成 $(k + 1, 2k)$，搬移 k 筆。再刪除這筆，縮回 (k, k)，又搬移 k 筆。之後反覆插入、刪除，每對操作搬移 $2k$ 筆，成本恰為 $2k + 2$。

建好初態需 $O(k)$；接著做 $2r$ 次交替操作，總成本為 $\Omega(kr)$。取 $r = k$，操作次數 $O(k)$，成本卻為 $\Omega(k^2)$。

修正方法是把兩個門檻分開：**滿表才擴張，刪除後至多四分之一滿才縮表**。縮完回到半滿，下一次反向調整前有足夠操作累積信用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

擴縮規格：門檻、空表與資料順序

本例只教 **APPEND 與 DELETE-LAST**。刪除中間元素若要搬動後綴，必須另計該成本。

- 空容量首次 APPEND 配置 1 格；滿表時容量乘 2，再寫入新資料。
- DELETE-LAST 先刪最後一筆。若 $M \geq 2$ 且刪後 $n \leq M/4$，容量減半，搬移剩下的 n 筆；否則容量不變。
- 容量是 2 的冪；用過後最小保留容量 1。空結構刪除回報 underflow，狀態不變。
- 每次成功操作成本 $1+$ 搬移筆數。搬移保持所有存活資料的順序；外部 raw pointer 可能因重新配置失效。

門檻慣例：本節採 $n \leq M/4$；原書 §17.4 採 $n < M/4$，在 $M \geq 4$ 時會晚一次刪除才縮表。兩者皆可有常數攤銷界，但不能共用同一張逐步帳目。

在這個規格下，一次操作最多觸發一次 resize。對 $M \geq 4$，刪除觸發縮表時恰有 $n = M/4$；因前一個狀態尚未觸發門檻，而且每次只刪一筆。

同一批資料，追蹤到縮表完成

擴縮門檻分開，才能在下一次搬移前存足信用

			成本
滿表	n=4, M=4	a b c d	成本 —
APPEND e	n=5, M=8	a b c d e · · ·	成本 5 = 1 + 4
DELETE e	n=4, M=8	a b c d · · · ·	成本 1
DELETE d	n=3, M=8	a b c · · · · ·	成本 1
DELETE c	n=2, M=4	a b · ·	成本 3 = 1 + 2

半滿先保留容量；四分之一滿才縮表，縮完回到半滿。

擴成 8 格後，刪到 4 筆時先保留容量；刪到 2 筆才縮為 4 格。兩個門檻之間的間隔，就是避免 resize 來回震盪的 **hysteresis（遲滯）**。

稠密與稀疏狀態，分別存下搬移信用

對 $M \geq 2$ 定義

$$\Phi(n, M) = \begin{cases} 2n - M, & n \geq M/2, \\ M/2 - n, & n < M/2. \end{cases}$$

另定義 $\Phi(0,0) = \Phi(0,1) = 0$、$\Phi(1,1) = 1$。兩段在半滿時同為 0，所有可達狀態皆非負。

稠密區的勢能隨插入增加，準備支付擴張；稀疏區的勢能隨刪除增加，準備支付縮表。這不是把同一筆信用算兩次：每個狀態只用其中一段。

下一頁逐一核對分支。$M = 1, 2$ 的空表例外另列，避免把 $M/4$ 當成一定是可達整數元素數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一般容量的四種分支，都有常數攤銷界

下表 M 是操作前容量。未 resize 的分支若跨過半滿邊界，使用兩段公式直接代入。

操作情形	c	$\Delta\Phi$	$\hat{c}$
不擴張的插入	1	-1 或 2	0 或 3
滿表擴張	$M + 1$	$2 - M$	3
不縮表的刪除	1	-2 或 1	-1 或 2
$M \geq 4$，刪至 $M/4$ 後縮表	$1 + M/4$	$1 - M/4$	2

最後一列：刪前 $n = M/4 + 1$，勢能 $M/4 - 1$；刪後容量 $M/2$、元素數 $M/4$，正好半滿，勢能 0。

例 $M = 8$：$(3, 8) \to (2, 4)$，搬移 2 筆，$c = 3$；勢能由 1 降為 0，所以 $\hat{c} = 2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最小容量的例外與總成本結論

一般表格之外，再核對以下合法邊界：

狀態變化	c	$\Delta\Phi$	$\hat{c}$
首次 $(0,0) \to (1,1)$	1	1	2
重用空格 $(0,1) \to (1,1)$	1	1	2
刪空 $(1,1) \to (0,1)$	1	-1	0
最小縮表 $(1,2) \to (0,1)$	1	0	1

所有成功操作都有 $\hat{c} \le 3$，起態勢能 0、終態勢能非負。因此任意 m 次成功 APPEND／DELETE-LAST 的實際總成本 $\le 3m$。

若把 underflow 呼叫的判斷成本計為 1，其勢能不變、攤銷成本也為 1；總成本仍線性。這個界不適用於任意位置刪除、任意縮容政策，或沒有列入模型的非定額搬移成本。

Dictionary：先決定同鍵插入的意義

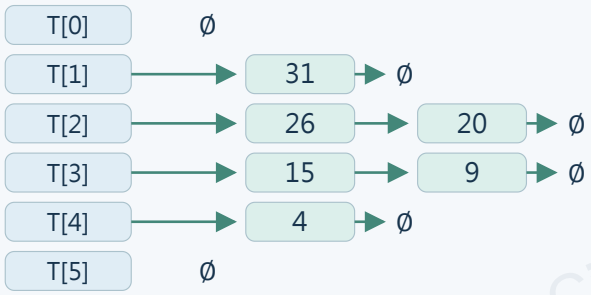
本節維護 key→record 的對應，每個 key 至多一筆；INSERT 遇到既有 key 時更新 value，DELETE 接受 key，找不到則回報不存在。key 的相等判斷與 hash 值相等是兩回事。

若 universe 為 $\{0, \dots, M-1\}$，直接定址表以 key 當索引，三種操作最壞 $O(1)$，空間及一般的全表初始化成本為 $\Theta(M)$。當實際資料只有 $n \ll M$ 筆，才希望把表縮到與 n 相近的大小。

雜湊表使用 $h : U \to \{0, \dots, m-1\}$。$|U| > m$ 時，鴿籠原理保證任何固定 h 都有碰撞；仍須保留完整 key 供核對。以下假設 key、hash 計算與相等比較可在常數個 word 操作內完成；長字串與任意精度整數須另計讀取與運算成本。

Chaining：同槽資料保留在串列裡

m = 6; h(k) = k mod 6
依序頭插：4, 9, 15, 20, 26, 31



查 20
26 → 20 ✓

查 14
26 → 20 → ø

同桶仍要比完整 key

$m = 6, h(k) = k \bmod 6$，依序頭插 4、9、15、20、26、31。槽 2 為 26→20，槽 3 為 15→9。找 20 檢查兩筆；找不存在的 14 也檢查槽 2 的兩筆。

常數頭插，不代表整個 INSERT 都是常數

令 $\alpha = n/m$，chaining 可有 $\alpha > 1$。搜尋掃描 h(k) 指定的整條串列直到找到 key 或到尾端，最壞 $\Theta(n)$。

已知 key 全新時，頭插的改接是 $O(1)$。但本章 INSERT 具有更新既有鍵的語義，必須先搜尋去重，成本為搜尋加常數改接。DELETE(key) 同樣先搜尋；只有持有有效雙向節點 handle 時，才可直接以 $O(1)$ 解鏈。

CLRS §11.2 的常數時間插入與刪除有上述輸入前提：插入假定物件尚未存在，刪除直接給物件。這與本章 dictionary API 不同，並非兩個成本結論互相矛盾。

失敗搜尋：把每一筆碰撞的成本加起來

先採 simple uniform hashing：固定相異 key 的槽位獨立且均勻分布。固定不存在的 query q，令 $I_x = \mathbf{1}[h(x) = h(q)]$，則

$$L_q = \sum_{x \in K} I_x, \qquad E[L_q] = \sum_{x \in K} \frac{1}{m} = \frac{n}{m} = \alpha.$$

搜尋會檢查這條 chain 的所有記錄，再加常數的 hash 與索引成本，因此期望時間為 $\Theta(1 + \alpha)$。$\alpha = 0$ 時仍要判斷空桶，不是零時間。

線性期望本身不需要碰撞事件互相獨立；上述完整均勻模型是足夠前提，稍後的 universal family 只靠鍵對碰撞上界，就能給出期望時間上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成功搜尋：平均的是記錄，不是桶

固定相異鍵的插入順序 $x_1, \ldots, x_n$，採頭插、沒有刪除，且成功 query 等機率選其中一筆。找 x_i 要檢查自己，以及晚於它插入的同槽記錄。

$$E[C_i] = 1 + \frac{n - i}{m}, \qquad E[C] = \frac{1}{n} \sum_{i=1}^{n} E[C_i] = 1 + \frac{n - 1}{2m}.$$

加入 hash 的常數成本後仍為 $\Theta(1 + \alpha)$。較大的桶含更多可被問到的 key，所以成功查詢並不是等機率選桶。

例如六筆表有兩條長度 2 的 chain、兩條長度 1 的 chain；等機率選 key 的檢查總數為 $(1 + 2) + (1 + 2) + 1 + 1 = 8$，平均 $4/3$。這個確定表的平均，不必等於隨機模型的期望 $1 + 5/12$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

除法與字串 hash：公式不等於分布保證

除法法 $h(k) = k \bmod m$ 很直接；若 $m = 2^p$，只保留最低 p 個 bit，例如 0、8 在 m=8 時都到槽 0。選質數可避開某些輸入規律，但不能保證任意 key 集合均勻。

字串可逐字套 Horner：$H_0 = 0$、$H_{i+1} = (BH_i + c_i) \bmod m$。歸納可知結果等於整個 base-B 整數取模，處理長度 ℓ 需 $\Theta(\ell)$ 次定額步驟。

取 B=10、m=7，字元碼序列 [1,2,3] 的中間值是 $1, 5, 4$，對應 $123 \bmod 7 = 4$。若 $B \equiv 1 \pmod m$，結果變成字元碼總和取模，重排字元仍相同；因此正確計算一個公式，不等於它具有低碰撞率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

乘法法：取低 word 裡的高位

$h(k) = \lfloor m \operatorname{frac}(kA) \rfloor$。取 $A = s/2^w, m = 2^p$，則 $r_0 = ks \bmod 2^w$，再取 $h(k) = \lfloor r_0 / 2^{w-p} \rfloor$，其中 $1 \le p \le w$。

寫 $ks = q2^w + r_0$，則 $kA = q + r_0/2^w$，小數部分正是 $r_0/2^w$。乘 2^p 再取 floor，就是把低 word 的最低 w−p 位捨去，留下其中最高 p 位。

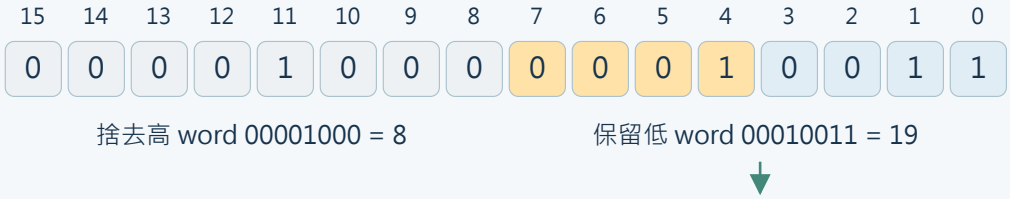
下一頁以八位元完整追蹤：w=8、s=159、k=13、p=4，結果應為 1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

乘法數例：完整乘積到槽號

w = 8; s = 159; k = 13; p = 4

ks = 2067 = 8 × 256 + 19



取低 word 的高四位：0001 → h = 1

$2067 = 8 \cdot 256 + 19$，低 word 是 00010011，取高四位 0001。不是完整乘積的最高四位；運算必須遵守指定的 word 位寬。

Universal family：隨機的是函數選擇

有限函數族 H 稱 universal，若對每對固定相異鍵 x、y，均勻抽取 $h \in H$ 時有

$$\Pr_h[h(x) = h(y)] \leq \frac{1}{m}.$$

鍵集合與 query 必須獨立於這次抽樣。這不要求使用者鍵值天生均勻，也不要求所有輸出或碰撞事件互相獨立；更不保證某次抽到的表完全沒有長 chain。

來源慣例：本章採 CLRS §11.3.3 的「不超過」。MIT 2005 Lecture 8 的定義寫「恰為」，並用模質數的 dot-product family 示範。恰為的版本也滿足本章定義，但不能把較弱定義反推成相等。

Universal chaining：完整的期望上界

固定集合 K 與 query q。令 $Y_q = \sum_{x \in K, x \neq q} \mathbf{1}[h(x) = h(q)]$。線性期望與 universal 定義給出

$$E[Y_q] \le \frac{|K \setminus \{q\}|}{m}.$$

q 不存在時，掃描長度期望至多 n/m；q 存在時，包含它的整條 chain 長度期望至多 $1 + (n-1)/m$，搜尋不會比掃完整條更貴。所以任意固定 q 的期望搜尋時間為 $O(1+\alpha)$，不需要成功 query 等機率。

若固定操作序列的同時存在筆數一直是 $O(m)$，更新式 INSERT、DELETE(key) 皆先搜尋，總 t 次操作期望 $O(t)$，另加初始化 $\Theta(m)$。若 query 依賴已觀察到的 h 或先前回應而自適應選擇，這份固定鍵對證明不能直接套用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Affine family：先證明模質數後的對應

取質數 $P > \max U$；均勻獨立抽 $a \in \{1, \dots, P-1\}$、$b \in \{0, \dots, P-1\}$，令

$$h_{a,b}(k) = ((ak + b) \bmod P) \bmod m.$$

對固定 x≠y，記 $r = (ax+b) \bmod P$、$s = (ay+b) \bmod P$。因 $a(x-y) \not\equiv 0 \pmod P$，必有 r≠s。

反過來，每對 r≠s 都唯一決定 $a = (r-s)(x-y)^{-1} \bmod P$、$b = r - ax \bmod P$。所以均勻的 (a,b) 對應均勻的 $P(P-1)$ 個有序相異 (r,s)。P 是質數，才保證非零的 x−y 有乘法逆元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最後取 mod m，碰撞機率仍不超過 1/m

固定 r 後，其餘 P−1 個 s 等可能。與 r 同 mod m 的 s 至多 $\lceil P/m \rceil - 1 \le (P-1)/m$ 個，因此

$$\Pr[h(x) = h(y) \mid r] \le \frac{\lceil P/m \rceil - 1}{P-1} \le \frac{1}{m}.$$

再對 r 取平均，便證明 universal。若 m≥P，取 mod m 不會合併相異的 r、s，碰撞機率為 0。

不等式可由寫 $P = qm + d$、$0 \le d < m$ 核對：d=0 時左側計數為 q−1；d>0 時為 q。m 不必是質數；需要質數的是 P。以 P、乘積模運算能在常數個 word 步驟完成為成本前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MIT 的 dot-product family 為何也成立？

取質數 m，key 為固定長度向量 $x = (x_0, \ldots, x_r) \in \mathbb{F}_m^{r+1}$。均勻抽係數向量 a，定義 $h_a(x) = \sum_i a_i x_i \bmod m$。

x≠y 時，選一個 $x_j - y_j \neq 0$ 的位置。固定其他係數後，碰撞等價於

$$a_j(x_j - y_j) = -\sum_{i \neq j} a_i(x_i - y_i) \pmod m.$$

因差值可逆，m 種 a_j 中恰有一種成立，故機率恰為 1/m。這個族的 table modulus m 必須是質數，與前頁 affine family 的任意 table size 不同；長度可變時，hash 計算成本也隨長度改變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Open addressing：三種槽位狀態

每槽直接存一筆 record，所以 n≤m。每個 key 的探測序列 $h(k,0),\ldots,h(k,m-1)$ 固定且可重現；要保證有空槽就能插入，這 m 個位置須是全表排列。

狀態	搜尋動作	刪除的影響
OCCUPIED	比完整 key，相等才成功	可改為 DELETED
DELETED（tombstone）	繼續探測	不是停止點
EMPTY	判定不存在並停止	不能由一般刪除製造

EMPTY 是本次建表後未曾使用的槽。若某 key 曾插到更後面，當時不可能跳過 EMPTY；後續刪除只留下 DELETED，才不會切斷搜尋路徑。重建表則重新定義整張表的狀態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋與刪除：最多探測 m 次

```
FIND(k):
    for i = 0 .. m-1:
        j = h(k,i)
        if T[j] is EMPTY: return absent
        if T[j] is OCCUPIED and T[j].key == k:
            return j
    return absent
```

DELETE(k) 先 FIND，找到 j 才把 T[j] 改為 DELETED；不存在則保持原態。key 可以是任何合法值，EMPTY／DELETED 必須有獨立的狀態標記，不能與資料值混淆。

迴圈上限不可省：一張有 tombstone、卻沒有 EMPTY 的表，失敗搜尋必須走完 m 槽後返回；不能無限等待空槽。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

更新式 INSERT：記住 tombstone，但先查完

```
first = NONE
for i = 0 .. m-1:
    j = h(k,i)
    if T[j] is OCCUPIED and T[j].key == k:
        T[j].value = value; return j
    if T[j] is DELETED and first == NONE: first = j
    if T[j] is EMPTY:
        target = first if first != NONE else j
        T[target] = OCCUPIED(k,value); return target
if first != NONE: T[first] = OCCUPIED(k,value); return first
return full
```

第一個 DELETED 後面仍可能有同 key；立即插入會製造重複記錄。只有遇 EMPTY 或走完全部 m 槽，才能確認不存在，再使用記住的空位。覆蓋全表是此處的探測前提。

Tombstone：保留搜尋路徑，再重用空間

m = 7; home(k) = k mod 7

ø = EMPTY; D = DELETED

	0	1	2	3	4	5	6	
插入 1, 8, 15	ø	1	8	15	ø	ø	ø	探測：1 → 2 → 3
刪除 1	ø	D	8	15	ø	ø	ø	查 15：1 → 2 → 3
更新 8	ø	D	8	15	ø	ø	ø	只更新槽 2 的 value
插入 22	ø	22	8	15	ø	ø	ø	探測到 4，再回填槽 1

遇到 D 繼續查；確認沒有同 key，才重用 D。

$m = 7, h(k,i) = (k \bmod 7 + i) \bmod 7$。插入 1、8、15 時分別占槽 1、2、3。刪 1 後，找 15 仍沿 $1 \to 2 \to 3$，不能在 tombstone 停下。

同一份資料的完整探測帳目

承接前圖。最後仍為三筆；INSERT(22) 在槽 4 確認不存在，才回填槽 1。

呼叫	探測槽	寫入／回傳	探測次數
INSERT(1,a)	1	槽 1 存 (1,a)	1
INSERT(8,b)	1,2	槽 2 存 (8,b)	2
INSERT(15,c)	1,2,3	槽 3 存 (15,c)	3
DELETE(1)	1	槽 1=DELETED	1
FIND(15)	1,2,3	找到槽 3	3
INSERT(8,b′)	1,2	更新槽 2，不新增	2
INSERT(22,d)	1,2,3,4	重用槽 1	4
FIND(29)	1,2,3,4	不存在	4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

探測公式：先確認能否覆蓋全表

Linear probing 用 $(h_0(k) + i) \bmod m$，一定覆蓋 m 槽，但連續的占用區段會吸引更多插入，形成 primary clustering。

Quadratic probing 用 $(h_0(k) + c_1 i + c_2 i^2) \bmod m$，參數不可任意選。例如 m=7、偏移 i^2 的前七項為 $0, 1, 4, 2, 2, 4, 1$，只覆蓋四槽；即使其他槽有空間也可能插不進去。

兩個 key 的 home 相同時，以上兩種方法的整條序列相同，稱 secondary clustering。Double hashing 讓步長也依 key 決定，但仍需證明步長與 m 互質；「多一個函數」本身不是覆蓋證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩種可證的全覆蓋探測

Double hashing：$h(k,i) = (h_1(k) + id) \bmod m$。回到 home 等價於 $m \mid id$。設 $g = \gcd(m,d)$，約分後最小正週期是 m/g，所以覆蓋全表恰需 g=1。m 為質數可取 1≤d<m；m 為 2 的冪可取奇數 d。

三角偏移：對 $m = 2^w$，使用 $f(i) = i(i+1)/2 \bmod m$，先做整數除法。m=8 時為 $0, 1, 3, 6, 2, 7, 5, 4$。

若 f(i)=f(j)，則 $2^{w+1} \mid (i-j)(i+j+1)$。兩因子一奇一偶，故偶因子需被 2^{w+1} 整除。當 $0 \le i, j < 2^w$，有 $|i-j| < 2^w$、$1 \le i+j+1 < 2^{w+1}$，唯一可能為 i−j=0。因此 f 無重複，正好覆蓋全表。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniform probing 是較強的理想模型

以下期望探測界假設：每個新 key 的探測序列，是相對於目前占用狀態仍均勻的 m 槽排列；無刪除、無 tombstone，且 n<m。它不是每次隨機選一槽的有放回抽樣。

線性、二次與雙重雜湊只有受限的序列集合，不能因為 home 均勻或函數族 universal，就直接宣稱符合 m! 種排列的理想模型。

因此分開三件事：chaining 的碰撞機率、探測是否覆蓋全表，以及探測排列的分布。前兩者成立，仍不足以推出下一頁的期望公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

失敗搜尋界：由尾機率加總

令 X 是遇到第一個 EMPTY 的探測數，$\alpha = n/m < 1$。前 i−1 槽均占用的機率為

$$\Pr(X \ge i) = \prod_{j=0}^{i-2} \frac{n-j}{m-j} \le \alpha^{i-1}, \quad 1 \le i \le n+1.$$

i=1 為空乘積 1；i>n+1 的機率為 0。各比值不超過 n/m，因 $m \ge n$。由 $X = \sum_{i \ge 1} \mathbf{1}[X \ge i]$ 得

$$E[X] = \sum_{i \ge 1} \Pr(X \ge i) \le \sum_{i \ge 1} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

新 key 插入的探測同樣適用。m=10、n=8 時上界為 5；它不是每次恰好五次，也不是 linear probing 的無條件保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

成功搜尋界：重現插入時的路徑

固定 n 筆相異鍵的插入順序，無刪除或搬移；成功 query 等機率選記錄。第 i+1 筆插入時只有 i 筆，期望探測至多 $m/(m-i)$；日後搜尋重現其插入路徑。

$$E[C] \le \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{1}{\alpha} \sum_{k=m-n+1}^{m} \frac{1}{k}$$

$$\le \frac{1}{\alpha} \int_{m-n}^{m} \frac{dx}{x} = \frac{1}{\alpha} \ln \frac{1}{1-\alpha}.$$

因 1/x 遞減，每段 [k−1,k] 的積分至少 1/k。$\alpha = 0$ 時沒有成功 query，不能直接代入 1/α；右式在 α→0 時極限為 1。這裡同時需要理想探測模型與等機率成功查詢，少一項都不能沿用完整公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

低活躍負載，也可能搜尋整張表

m=7，先插入 home 分別為 0～6 的七筆，再全部刪除。此時 n=0、α=0，但七槽全為 DELETED，找任意不存在的 key 都要探測 7 次。

因此不能把無 tombstone 的 $1/(1-\alpha)$ 界套到這張表。可另追蹤 $\beta = (n+d)/m$，d 為 tombstone 數；本例 β=1，才反映沒有 EMPTY 可提早停止。β 本身也不是任意實作的完整成本證明。

重建會清除 tombstone，但必須列入配置、重新計 hash 與插入的成本；在 open addressing 中，重新插入本身也需要探測。若要主張重建的期望攤銷界，還需固定負載門檻、隨機模型與重建規則，不能只說「定期 rehash」就省略成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Perfect hashing：把隨機重試放在建表

FKS 兩層法處理固定的 n 個相異 key。建表後每次查詢最壞 $O(1)$ 個 word 操作，空間 $O(n)$；建表採隨機重試，只有期望線性成本，不能稱最壞線性。

n=0 時直接建空 dictionary，不做 mod 0。以下 n≥1，第一層使用 m=n，均勻抽 universal h。桶 j 大小為 n_j，準備配置 n_j^2 個次層槽。

令 Y 為碰撞的無序鍵對數。對每桶用 $n_j^2 = n_j + 2\binom{n_j}{2}$，可得

$$\sum_j n_j^2 = n + 2Y, \qquad E\left[\sum_j n_j^2\right] \le n + 2\binom{n}{2}\frac{1}{n} = 2n - 1.$$

平方空間和的控制來自碰撞對數，不是宣稱每個桶都一定很小。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第一層：真的檢查空間，再接受

令 $S = \sum_j n_j^2$。Markov 不等式給出 $\Pr(S \geq 4n) \leq E[S]/(4n) < 1/2$。每次獨立重抽，直到實際計算得到 S<4n。

每輪只需清空 n 個桶的計數並掃過 n 個 key，成本 O(n)；先檢查 S，**不要在不合格的輪次先配置所有平方表**。接受機率大於 1/2，所以期望試驗輪數少於 2，第一層期望成本 O(n)。

接受後有確定的 S<4n。這比只知道 E[S]<2n 更強：最終成品確實占線性空間，而不是把「期望線性」誤寫成每一次都已線性。極端壞抽樣仍可能重試很多次，不能把期望輪數當硬上限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第二層：平方槽數換取無碰撞

條件於已接受的第一層分桶。桶內 s≥2 個 key，配置 s^2 槽，獨立抽 universal h_j。碰撞對數 Z 滿足

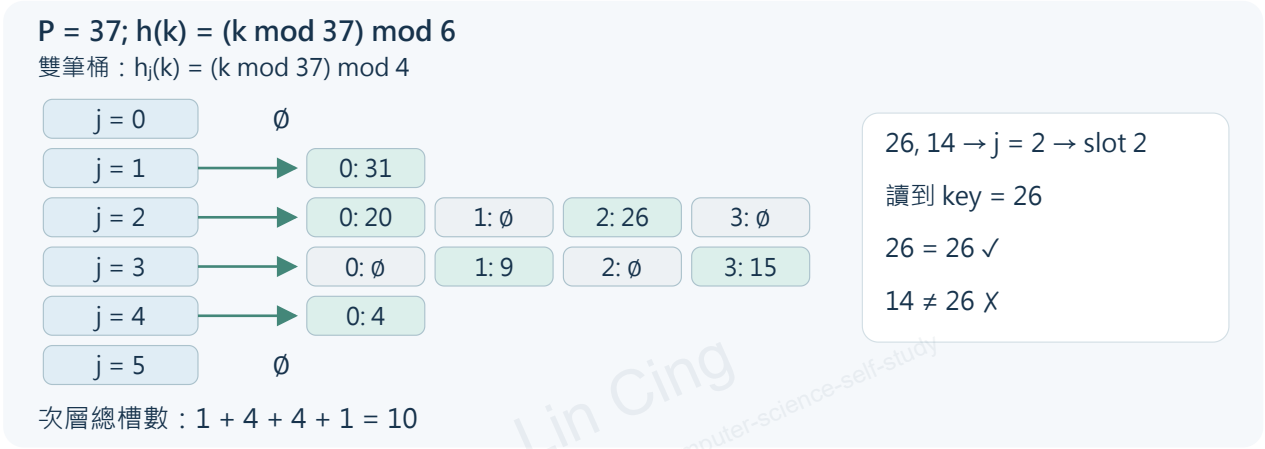
$$E[Z] \leq \binom{s}{2}\frac{1}{s^2} = \frac{s-1}{2s} < \frac{1}{2}, \qquad \Pr(Z \geq 1) \leq E[Z] < \frac{1}{2}.$$

真的檢查無碰撞才接受。每輪清空 s^2 槽、插入 s 筆共 $O(s^2)$，期望少於兩輪。對固定的合格分桶加總，期望成本為 $O(\sum_j s_j^2) = O(n)$；再對第一層取期望，總建表仍 O(n)。

空桶不配置次層；單筆桶可直接存記錄。存每桶大小、hash 參數與指標只另需 O(n)。查詢最壞界依賴定額 hash 與 key 比較，增加新 key 則超出這份靜態規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩層配置：同一組鍵值完整落到槽位



$K = \{4, 9, 15, 20, 26, 31\}$、P=37、$h(k) = (k \bmod 37) \bmod 6$。桶大小 [0,1,2,2,1,0]，平方和 10<4n=24。兩個雙筆桶用 $h_j(k) = (k \bmod 37) \bmod 4$，分別放到槽 (0,2) 與 (1,3)，實際檢查都無碰撞。

沒有存入的 key，仍可能撞到已占用槽

查 26：第一層算得 2，次層算得 2，讀到 key=26，相等才成功。查 14：同樣走 2→2，讀到的卻是 26，所以必須回不存在。

查 5：第一層到空桶 5，直接回不存在，不計算不存在的次層函數。查 10：第一層到單筆桶 4，讀到 4≠10，仍回不存在。

Perfect 只保證已存集合內不互撞，不替未存鍵提供成員資格證明。 Lookup 必須先處理空桶，再檢查次層槽是否占用，最後比完整 key。建表成功後，無論查詢是否存在，都只走常數層；這才是查詢最壞常數時間的保證。

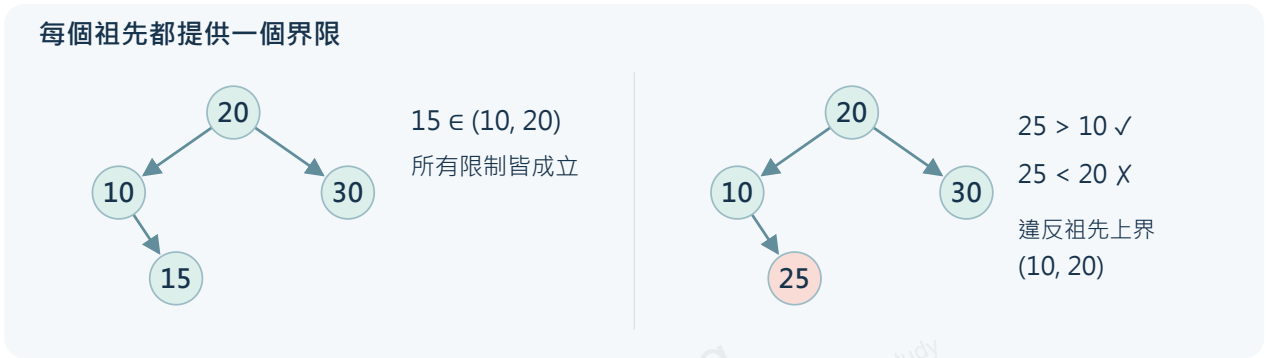
BST 的限制來自所有祖先

BST（binary search tree，二元搜尋樹）把比較結果變成分支。本節延續唯一 key 的 dictionary：每個節點存 key、value、left、right、parent；空連結為 NIL，root.parent=NIL。同 key 插入只更新 value。

每個節點的整棵左子樹都小於它，整棵右子樹都大於它。 只檢查父子大小並不足夠。往左把允許區間的上界縮到目前 key，往右則更新下界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

祖先區間：局部大小正確仍可能違規



CLRS §12.1 也容許重複 key，使用左側 ≤、右側 ≥ 的性質。本節採嚴格次序；若要存重複資料，可另存 count 或以 (key,唯一編號) 排序，不能直接混用兩套規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋：每次比較排除一整棵子樹

```
FIND(T,k):
    x = T.root
    while x != NIL:
        c = compare(k,x.key)   // 三向比較：小於、等於、大於
        if c == 0: return x
        x = x.left if c < 0 else x.right
    return absent
```

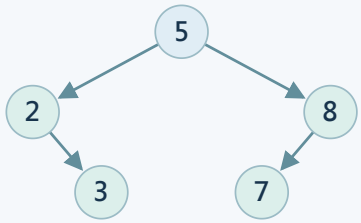
迴圈不變量：若 k 存在，必在目前 x 的子樹中。比較不等時，BST 的全域次序排除另一側；每次嚴格往下一層，有限樹必終止。到 NIL 表示候選子樹已空。

深度 d 按 root 到節點的**邊數**算，root 深度為 0。成功搜尋深度 d 的節點，訪問 d+1 筆；失敗搜尋只計遇到的非 NIL 節點。以下一次三向 key 比較算一次；NIL 測試與比較器內部指令不混入這個數。

高度 h 按最長路徑的邊數算，空樹高度 −1；非空樹搜尋為 O(h+1)，空樹另需 O(1)；統一可寫 O(h+2)。

同一棵樹的成功與失敗搜尋

深度從 0 起；成功比較數 = 深度 + 1



d = 0　查 7：5 → 8 → 7，3 次

d = 1　查 6：5 → 8 → 7 → NIL

d = 1　失敗也比較 3 次

d = 2　查 9、10 都在 8 的右側失敗

查 7 走 5→8→7，共三次 key 比較；查 6 也走同樣三個節點，最後才到 NIL。查 9 或 10 則走 5→8→NIL，只比較兩次。

成功與失敗都要實際數訪問節點。到 NIL 的最後一次指標測試，不是額外一次 key 比較。

平均成本：先決定查詢怎麼抽

若只在五個已存 key 中等機率查詢，成功比較總數為 1+2+2+3+3=11，平均 **11/5**。若 query 改為整數 1～10 均勻抽取，還要加上五個失敗查詢。

query	1	4	6	9	10
訪問 key	5,2	5,2,3	5,8,7	5,8	5,8
比較數	2	3	3	2	2

因此混合平均為 (11+12)/10=**2.3**。失敗間隙不是等機率事件：區間 (8,+∞) 在此包含 9、10，權重 2/10；不能把每個 NIL 連結各算同一權重。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中序走訪與極值

中序（inorder）依「左子樹 → 目前節點 → 右子樹」列出 key。歸納假設兩個子樹各自輸出遞增序列；BST 性質又讓左側每個值 < 根 < 右側每個值，因此串接後仍遞增。每個節點處理一次、空連結測試 O(n) 次，總時間 Θ(n+1)，遞迴堆疊 O(h+2)。

找 MINIMUM(x) 時，前提為 x≠NIL；沿 left 直到沒有 left child。若還有左子樹，最小值一定在其中；沒有時，目前 key 小於所有右子樹 key，故答案就是 x。MAXIMUM 對稱。空 dictionary 的極值操作應直接回 absent。

用 BST 排序，不能只算最後的走訪。 依序插入 1,2,⋯,n 會得到右鏈；建樹比較數為 0+1+⋯+(n−1)=n(n−1)/2，再加 Θ(n) 的走訪，仍是 $\Theta(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

後繼：有右子樹就向下，否則向上找

```
SUCCESSOR(x):                    // x 是有效節點
    if x.right != NIL: return MINIMUM(x.right)
    p = x.parent
    while p != NIL and x == p.right:
        x = p; p = p.parent
    return p                    // NIL 表示沒有後繼
```

有右子樹時，其中最小 key 就是下一個；它比 x 大，且不大於任何其他右子樹 key。沒有右子樹時，向上經過「右孩子 → 父」所遇到的父鍵都更小；第一次由左側回到的祖先才是最小的較大 key。

在樹 5(左 2、右 8)，2.right=3、8.left=7 中：succ(5)=7，路徑 8→7；succ(3)=5，路徑 3→2→5；succ(8)=NIL。PREDECESSOR 交換 left/right 即得。

每次呼叫只沿單一路徑，最壞 O(h+1)。從最小值開始反覆找 successor 可列出全部節點；每條樹邊至多向下、向上各經過一次，整段是 O(n)，不是 n 次最壞界相乘後的必然 Θ(nh)。

插入：找到的是空連結，不是任意葉節點

```
INSERT(T,k,v):
    p = NIL; x = T.root
    while x != NIL:
        p = x; c = compare(k,x.key)
        if c == 0: x.value = v; return x
        x = x.left if c < 0 else x.right
    z = Node(k,v,NIL,NIL,p)
    if p == NIL: T.root = z
    elif k < p.key: p.left = z
    else: p.right = z
    return z
```

迴圈保留 k 的祖先區間；到 NIL 後，新節點恰好落在所有祖先允許的範圍內。既有 key 的分支只改 value，不改 key 或節點身分；在樹內任意改 key 可能破壞次序。

最壞 O(h+2)，包含空樹插入；新增節點空間 O(1)。

Transplant：先換父節點的連結

```
TRANSPLANT(T,u,v):              // u 必須是樹內有效節點
    p = u.parent
    if p == NIL: T.root = v
    elif u == p.left: p.left = v
    else: p.right = v
    if v != NIL: v.parent = p
```

此程序只把「指向 u 的連結」改成指向 v，並更新非空 v 的 parent；**不會替呼叫者接好 v.left、v.right**。v=NIL 合法，但本節的 NIL 不是可寫入 parent 的節點。

刪除 z 若沒有左孩子，直接 TRANSPLANT(T,z,z.right)，也包含 z 是葉節點的情況。若只有左孩子，則改用 z.left。所有被保留的子樹仍滿足 z 的祖先區間。

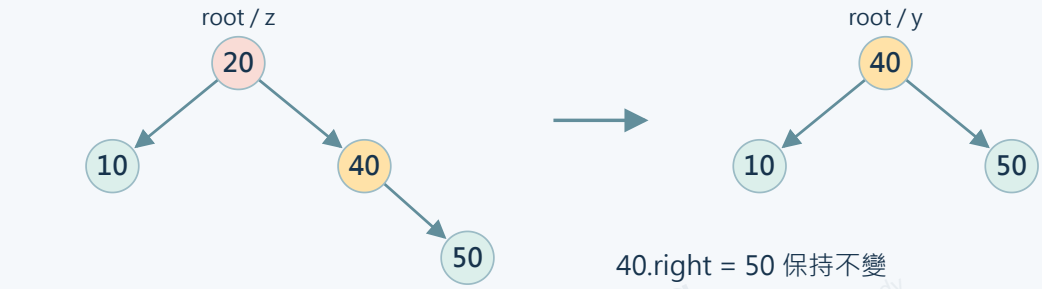
例如 root=20 只有右孩子 40：移接後 root=40、40.parent=NIL。若 20 是最後一筆，則 root=NIL。這些分支都只改常數個連結；DELETE(key) 尚須先付搜尋成本。

兩個孩子：先把後繼從原位取出

```
DELETE-NODE(T,z):
    if z.left == NIL: TRANSPLANT(T,z,z.right)
    elif z.right == NIL: TRANSPLANT(T,z,z.left)
    else:
        y = MINIMUM(z.right)
        if y.parent != z:
            TRANSPLANT(T,y,y.right)
            y.right = z.right; y.right.parent = y
        TRANSPLANT(T,z,y)
        y.left = z.left; y.left.parent = y
```

y 是右子樹最小值，所以沒有左孩子，但**可能有右孩子**。深處的 y 要先由 y.right 接替，才能把整棵原右子樹接到 y。若 y 就是 z.right，則保留原本 y.right；不可執行 y.right=z.right，否則會得到 y.right=y 的自環。

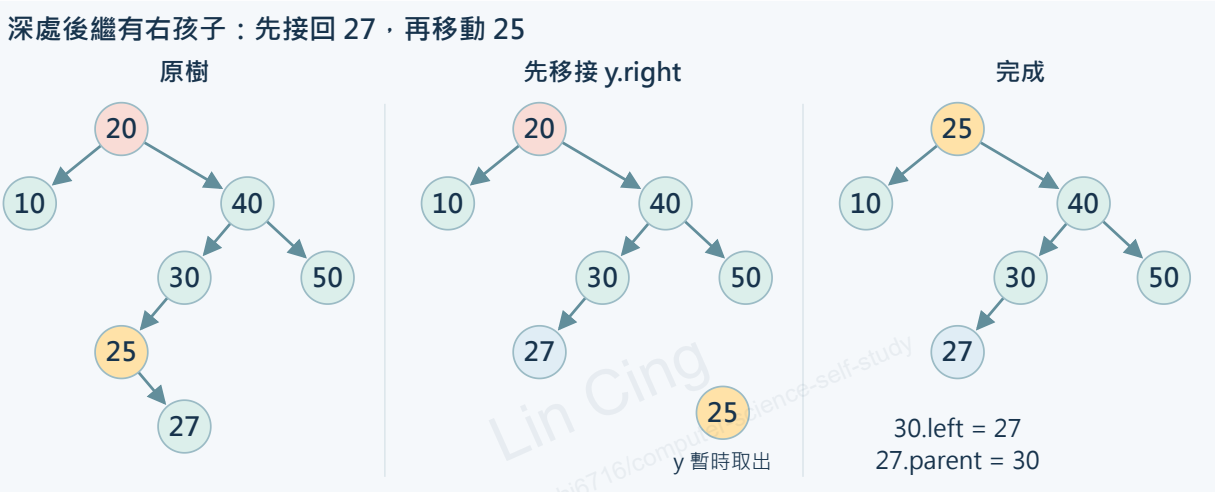
後繼就是右孩子：保留它的右連結



刪 z=20：y=40，且 y.parent=z。先讓 root 指向 y，y.parent=NIL；再接 y.left=10、10.parent=y。**y.right=50 從頭到尾不變。**

中序由 [10,20,40,50] 變為 [10,40,50]。保留下來的所有鍵值、value 與節點身分不變；只有 z 離開樹。非根的 z 也同樣處理，只是改接的是 z 原父的 left 或 right。

深處後繼：它的右孩子不能遺失



刪 root z=20，右子樹最小節點 y=25，且 y.right=27。依序更新：① 30.left=27、27.parent=30；② y.right=40、40.parent=y；③ root=y、y.parent=NIL；④ y.left=10、10.parent=y。

深處後繼：核對次序與可達性

圖中的中間態把 y 畫成暫時取出的物件；它的舊連結要清理或覆寫，不能在完成前把整個物件池當成一棵合法 BST。完成後 root=25、25.right=40、40.left=30、30.left=27。

原中序 [10,20,25,27,30,40,50] 只刪去 20。若第一步誤寫 30.left=NIL，27 會遺失；若忘了 parent 更新，後續 successor 的向上搜尋也會壞掉。

為何新連結仍符合次序？y=25 是原右子樹的最小值，所以原左子樹的 key 全小於 y，剩下右子樹的 key 全大於 y。27 原在 25 的右側、30 的左側，符合 25<27<30，接成 30.left 仍合法。

移接保留原來除 z 以外的每個物件恰好一次；沒有新建副本。最後還要核對 root.parent=NIL、每個非空 child 的 parent 指回擁有它的節點，且沒有循環。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪鍵與刪物件：兩種實作的差別

以 z.key=20、y.key=25 為例，兩種做法都能留下正確的鍵集合，但對 handle 的效果不同。

做法	留在 z 原位的物件	離開樹的物件	外部 handle 的影響
複製後繼 payload，再刪 y	原 z，內容改成 25 的記錄	y	z 的意義改變；y 的 handle 失效
移接後繼節點 y	原 y，位置改到 z 原位	z	z 的 handle 失效；y 的身分保留

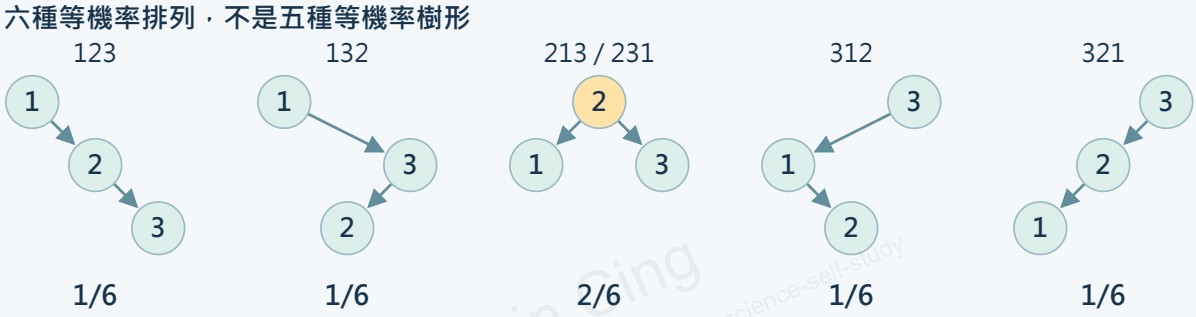
來源慣例：紙本筆記示範前者，CLRS 第三版 §12.3 採後者。本節以移接為準。複製法若採 key→value 記錄，必須搬對應 value，不能只抄 key；附加欄位、資源所有權與外部索引也要維護。

兩種方法都可改用前驅（左子樹最大值），但指標方向要整組對稱更換。不要從前驅版取圖、再套後繼版程式的左右分支。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

隨機順序不等於隨機樹形

固定 n 個相異 key，均勻抽取 n! 個插入排列。首筆為 root；給定 root 秩 R=r，兩側的相對排列仍各自均勻，故子樹可遞迴套同一模型。此處只分析插入建樹。



root=2 的兩個排列得到同一平衡樹，其機率為 2/6；其餘四個排列各得到不同樹形，各為 1/6。若直接把五種形狀各設為 1/5，就換了機率模型。

固定秩的期望深度：逐一數祖先

將鍵依大小記為 $x_1<\cdots<x_n$，D_i 是 x_i 的深度。對 $j\neq i$，x_j 成為 x_i 的祖先，**當且僅當閉區間 $[x_{\min(i,j)}, x_{\max(i,j)}]$ 中最先插入的是 x_j**。

區間外先插入的 key 只能把兩者一起帶往同側；區間內第一筆若是 x_j，它會成為通往 x_i 的路徑節點。若先出現別筆，就會先到 x_i，或把 x_j 與 x_i 分開。因此

$$\Pr(x_j \text{ 是 } x_i \text{ 祖先}) = \frac{1}{|j-i|+1}.$$

深度就是祖先數，套線性期望（不需各事件獨立）得

$$E[D_i] = \sum_{j<i} \frac{1}{i-j+1} + \sum_{j>i} \frac{1}{j-i+1} = H_i + H_{n+1-i} - 2.$$

其中 $H_t = \sum_{k=1}^{t} 1/k$。n=5、i=3 時，期望深度為 $2H_3 - 2 = 5/3$，成功搜尋的期望比較數再加 1，為 8/3。

期望建樹成本：把所有深度加起來

節點插入時比較過的節點，恰好是它在完成樹中的祖先；後續插入不改動既有節點的深度。因此建樹總 key 比較數為 $C = \sum_i D_i$。利用

$$\sum_{i=1}^{n} H_i = \sum_{k=1}^{n} \frac{n-k+1}{k} = (n+1)H_n - n,$$

可得

$$E[C] = 2(n+1)H_n - 4n = \Theta(n \log n), \qquad n \geq 2.$$

常數節點配置與連結另共 Θ(n)。等機率成功查詢的平均比較數，其期望為 $E[C]/n + 1$。n=3 時 E[C]=8/3，平均成功比較數為 17/9；n=1 時建樹比較 0 次、成功查詢 1 次。

這也連到前章的 quicksort：同一相對順序遞迴以首筆為 pivot 時，兩個 key 是否比較，取決於它們之間誰最先被選到。但**平均深度為對數，尚不能推出最大深度也有同樣期望界**。

高度分析：先處理 max，再取期望

令 X_n 為高度（邊數），n≥1 時設 $Y_n = 2^{X_n}$；另設 **$Y_0=0$ 作為輔助值**，不是 $2^{X_0} = 2^{-1}$。單節點 $Y_1=1$。n≥2、條件在 root 秩 R=r 時，至少一側非空，故

$$Y_n = 2\max(Y_{r-1}^{L}, Y_{n-r}^{R}).$$

令 $a_j=E[Y_j]$。給定 R 後，兩側相對排列均勻，配合 max(u,v)≤u+v 得

$$a_n \leq \frac{2}{n}\sum_{r=1}^{n}(a_{r-1} + a_{n-r}) = \frac{4}{n}\sum_{j=0}^{n-1} a_j, \qquad n \geq 2.$$

這裡用的是**條件期望**，沒有把隨機大小的子樹與 R 當成無條件獨立。也沒有使用錯誤等式 E[max(U,V)]=max(E[U],E[V])。

例如 $Y_1=1$、$a_2=2$；n=3 的兩個平衡排列高度 1、四個鏈狀排列高度 2，所以 $a_3=(2\cdot2+4\cdot4)/6=10/3$。

解出指數高度界，再回到原高度

強歸納證明 $a_n \le n^3$（$n \ge 1$）。$a_1=1$；對 $n \ge 2$，$a_0=0$，且 $\sum_{j=1}^{n-1} j^3 = [n(n-1)/2]^2$，所以

$$a_n \le \frac{4}{n}\sum_{j=1}^{n-1} j^3 = n(n-1)^2 \le n^3.$$

為何可以從這裡推 $E[X_n]$？令 $\mu = E[X_n]$。由 $e^u \ge 1+u$ 得 $2^x \ge 2^\mu[1+(\ln 2)(x-\mu)]$；取期望，線性項消失，得到 Jensen 在此所需的形式：

$$2^{E[X_n]} \le E[2^{X_n}] = a_n \le n^3 \quad \Longrightarrow \quad E[X_n] \le 3\log_2 n.$$

$e^u - 1 - u$ 的導數是 $e^u - 1$，在 $u=0$ 取最小值 0，故上述切線界成立。此處高度的支集有限，期望均存在。

另一方面，h 高的二元樹至多 $2^{h+1}-1$ 個節點，故每棵樹 $h \ge \log_2(n+1) - 1$。兩側合起來，隨機建樹的期望高度為 $\Theta(\log n)$，但單次最壞仍為 $n-1$。

紅黑樹：用路徑不變量限制高度

紅黑樹仍是嚴格次序的 BST，只多存 color。以下 B 表示黑、R 表示紅；圖中的顏色與 B／R 標籤並用，避免只能靠顏色辨識。

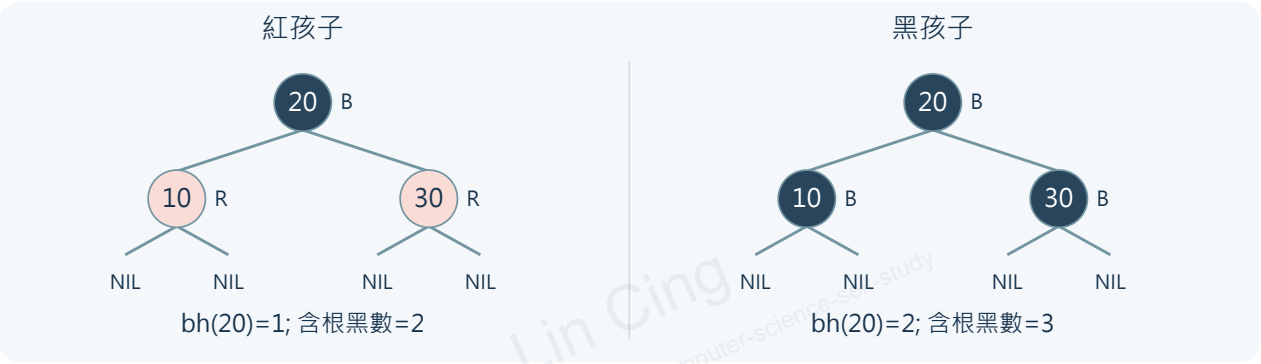
1. 每個節點非紅即黑，根節點 為黑。
2. 所有外部空位置 NIL 為黑；資料節點沒有 孩子，也仍連到 NIL。
3. 紅節點的兩個 孩子 都黑，故不會連續兩個紅節點。
4. **從任一節點往下到各個 NIL 的路徑，黑節點數都相同。** 不能只檢查少數 根節點–NIL 路徑。

用一個共享 哨兵節點 表示所有 NIL，令 NIL.left=NIL.right=NIL、NIL.color=B，root.parent=NIL。一般 child.parent 一致性只檢查非 NIL 的 孩子。同鍵 插入只更新 value，不改形狀或顏色。

前節 BST 不寫 NIL.parent；本節刪除修復將暫借它記錄一個缺黑位置的父節點。它不是所有空位置同時有效的 parent。旋轉遇到 β=NIL 時仍不寫 NIL.parent。

黑高先固定計數範圍

定義 $bh(x)$：從 x 往下到任一 NIL 的黑節點數，**不含 x，含末端 NIL**；另定 $bh(NIL)=0$。紅黑性質保證此數不因選哪條路而變。



若 c 是非 NIL 的 孩子，c 紅時 $bh(c)=bh(x)$；c 黑時 $bh(c)=bh(x)-1$。這是因為父的計數含 c，而 孩子 自己的 bh 不含自己。c=NIL 時同樣有 $bh(x)=1+bh(NIL)$。

高度界：先數最少需要多少節點

本章沿用內部節點邊數高度 h：單節點 h=0、空樹 h=−1。另以 H 計 根節點 到外部 NIL 的最長邊數，非空時 H=h+1，空樹 H=0；高度證明使用 H，不偷偷換掉 h 的定義。

引理： x 的子樹至少含 $2^{bh(x)}-1$ 個內部節點。對子樹高度歸納：x=NIL 時，$0=2^0-1$。若 x 非 NIL，兩個 孩子 的黑高各至少 bh(x)−1；套歸納假設再加上 x：

$$n_x \geq 2\left(2^{bh(x)-1}-1\right)+1=2^{bh(x)}-1.$$

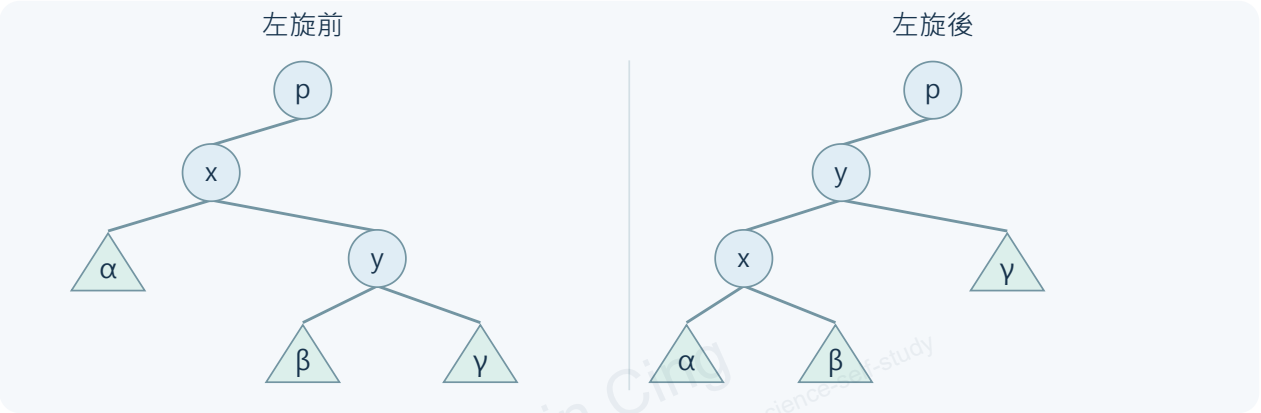
再取一條最長 根節點–NIL 路。含 根節點 與 NIL 共 B 個黑點、R 個紅點。兩端黑，沒有相鄰紅，所以 $R \leq B-1$；路徑邊數

$$H=B+R-1 \leq 2(B-1)=2bh(root).$$

因此 $n \geq 2^{bh(root)}-1 \geq 2^{H/2}-1$，得到 $H \leq 2\log_2(n+1)$。空樹 H=0 也成立。搜尋路徑加終止測試需 O(H+1)，統一寫 O(log(n+2))，才能涵蓋空樹常數成本。

這只保證搜尋高度；插刪還須證明修復會終止且不超出同一時間界。紅黑樹也不要求每個節點左右高度差至多 1，那是 AVL 的條件。

左旋：β 換父，舊父改接新根



前提 x≠NIL、y=x.right≠NIL。α、β、γ 表示整棵子樹，允許為 NIL；p 是 x 的**舊父**。旋轉後 y 取代 x 的局部根位置，β 從 y.left 移到 x.right。

左右旋轉：完整且對稱的接線

```
LEFT-ROTATE(T,x):                RIGHT-ROTATE(T,y):
    p=x.parent; y=x.right           p=y.parent; x=y.left
    beta=y.left                     beta=x.right
    x.right=beta                    y.left=beta
    if beta!=NIL: beta.parent=x     if beta!=NIL: beta.parent=y
    y.parent=p                      x.parent=p
    if p==NIL: T.root=y             if p==NIL: T.root=x
    elif p.left==x: p.left=y         elif p.left==y: p.left=x
    else: p.right=y                 else: p.right=x
    y.left=x                        x.right=y
    x.parent=y                      y.parent=x
```

先保存舊父 p；β=NIL 時不改 sentinel.parent。前提分別為 x.right≠NIL 與 y.left≠NIL。

旋轉接線：逐條核對數值與次序

取 p=30、x=10、y=20、β=15，且 p.left=x。其餘子樹內容不變。

順序	寫入	目的
1	10.right=15	接回 β
2	15.parent=10	讓 β 認新父
3	20.parent=30	新局部根接外父
4	30.left=20	更新原父指向局部根的 連結欄位
5	20.left=10	舊根變成左 孩子
6	10.parent=20	舊根認新父

若 x 原為 根節點，第 3 項改為 20.parent=NIL，第 4 項改為 T.root=20。

鍵值區間 α<x<β<y<γ 不變，故中序與 BST 次序保持；節點身分及 鍵值與資料 也不變。**旋轉本身不保證紅黑性質**。

插入紅葉：先辨認唯一可能的違規

先依前節 INSERT 搜尋；同鍵 則更新 value 並返回。新節點 z 以 R 插入，left=right=NIL，parent 指向搜尋到的父節點。

原本的一個黑 NIL 被「紅 z 加黑 NIL」取代，任何穿過此位置的路徑黑數不變。若插成黑色，只有經此位置的路增加一黑，通常會破壞黑數相等。

修復迴圈入口保持三件事：**z 紅；若 z.parent 是 根節點，該父必黑；最多只剩根紅或 z 與父同紅其中一種違規。** BST 與各路黑數相等均維持。

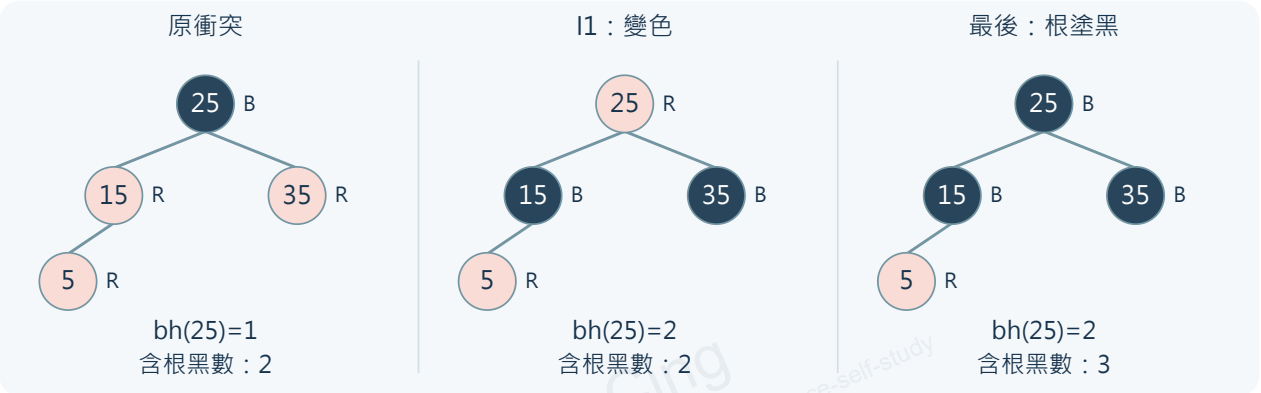
若父黑，直接結束；若 z 是新根，最後塗黑即可。只有父 p 紅才進迴圈；p 不可能是根，因此祖父 g 存在且黑。叔 u 是 g 的另一 孩子，可能為 NIL，但 color 仍可安全讀取。

I2 不是另一個互斥結局：執行後必接 I3。旋轉會改 parent；I3 使用的是當下的 z.parent 與祖父，不能沿用旋轉前快取的 p、g。

插入修復：父在左側的三個分支

```
while z.parent.color==R:
    if z.parent==z.parent.parent.left:
        u=z.parent.parent.right
        if u.color==R:                          // I1
            z.parent.color=B; u.color=B
            z.parent.parent.color=R
            z=z.parent.parent
        else:
            if z==z.parent.right:               // I2
                z=z.parent
                LEFT-ROTATE(T,z)
            z.parent.color=B                    // I3
            z.parent.parent.color=R
            RIGHT-ROTATE(T,z.parent.parent)
    else: apply the explicitly mirrored branch
T.root.color=B
```


I1：重新著色不改祖先路徑的黑數



先插 25、15、35，再插 5。新增的 5R 與父 15R 衝突，叔 35R 也紅。把 15、35 塗黑、25 塗紅，z 移到 25；此時兩條子路徑的黑數仍相同。

I1：哪些黑數改變，哪些不變？

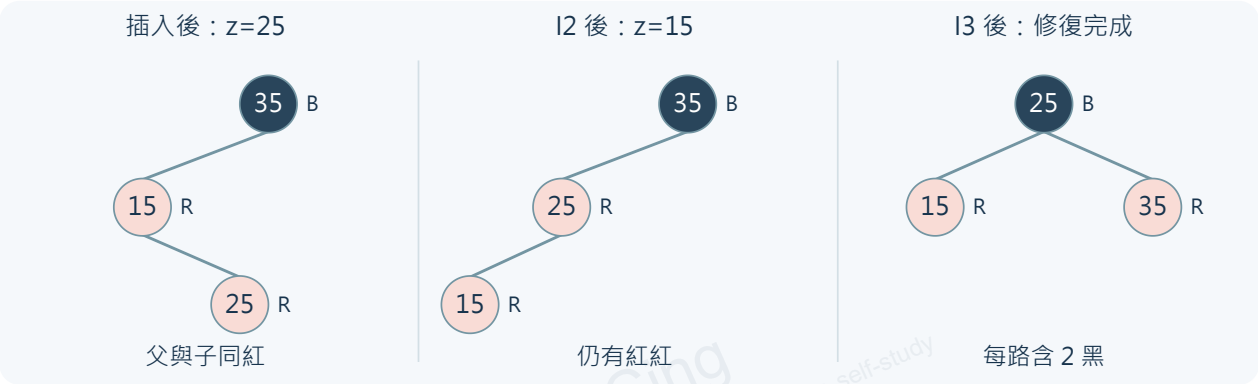
令 p／u 以下（不含 p／u）的每條路有 b 黑點。原先經 g 的完整路有 1+b 黑；改成 gR、p/uB 後仍是 1+b。**穿過 g 的祖先到 NIL 路徑黑數保持不變**，但不含 g 的 bh(g) 從 b 變為 b+1。

違規只可能移到 g 與它的父。若 g 恰為 根節點，迴圈結束後才把它塗黑，使所有 根節點–NIL 路一起加一黑。這個最後步驟不能套到非根 g。

計數範圍	I1 前	I1 後
不含 g 的 bh(g)	b	b+1
含 g 到 NIL	1+b	1+b
任一祖先經 g 到 NIL	外部黑數+1+b	外部黑數+1+b

若 g=25 為根且 b=1，I1 後 bh(25)=2、含根黑數=2；最後根塗黑才把含根黑數改為3。

I2 接 I3：折線先拉直，再修復

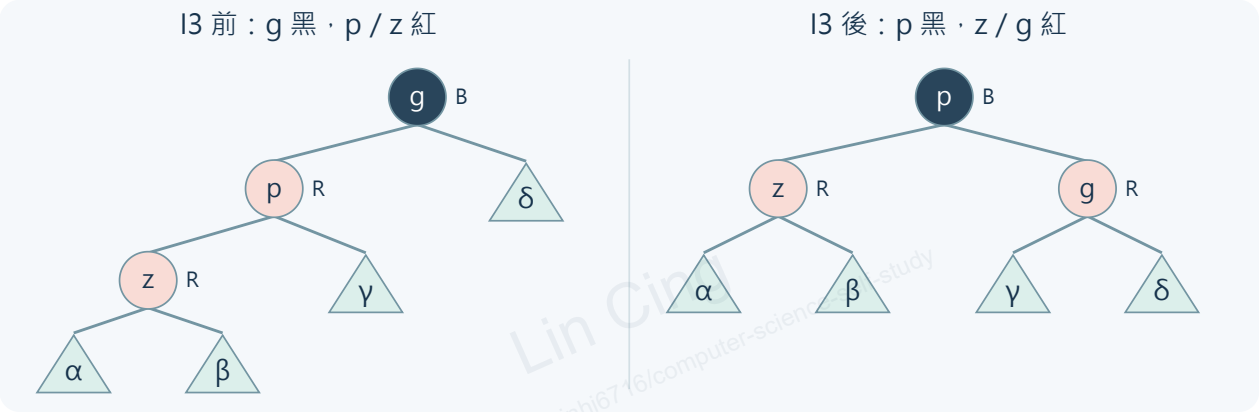


插 35、15、25，叔為黑 NIL，且父 15 在祖父 35 左側、新 25 在父右側。I2 先令 z 指舊父 15，再左旋 15；此時 25R 成為 15R 的父，**紅紅仍存在**。

I3 把目前父 25 塗黑、祖父 35 塗紅，再右旋 35。結果為 25B、15R、35R，每路含 2 黑；I2 後不能返回。

I3 為何保留黑數，也消除紅紅

設祖父 g 黑、其左孩子 p 紅，而 p 的左孩子 z 也紅；叔 u 黑。z 的兩個孩子為 α、β，p 的原右子樹為 γ，g 的原右子樹為 δ。把 p 塗黑、g 塗紅，再右旋 g：



I3：逐條驗證修復後的不變量

α、β 因 z 紅而為黑根；γ 因 p 紅而為黑根；δ 就是黑叔。各邊界子樹從根到 NIL（**含子樹根**）的黑數皆為 b；原經 g 的路貢獻 1+b，改後經 p 也貢獻 1+b。

新局部根 p 黑，不會與外父形成紅紅。z 仍紅但父 p 黑；g 改紅，其 孩子 γ、δ 都黑。故局部所有紅黑性質恢復，接回原父也不改外部黑數，修復可以結束。

若原本已是外側鏈（插 35、25、15），跳過 I2，直接 I3 即可；黑數證明完全相同。

通往哪棵子樹	修復前的局部黑節點	修復後的局部黑節點
α、β	gB；再加 b	pB；再加 b
γ	gB；再加 b	pB；gR 不計；再加 b
δ	gB；再加 b	pB；gR 不計；再加 b

四種目的地前後皆 1+b。子樹內部連結及顏色不變，因此每個內部節點原有的路徑相等性也保留。

父在右側：把鏡射動作寫出來

以下取代主修復程式的 else；條件是 z.parent=祖父.right。u 這次取左叔，旋轉方向也交換。

```
u=z.parent.parent.left
if u.color==R:                              // I1 mirrored
    z.parent.color=B; u.color=B
    z.parent.parent.color=R
    z=z.parent.parent
else:
    if z==z.parent.left:                     // I2 mirrored
        z=z.parent
        RIGHT-ROTATE(T,z)
    z.parent.color=B                        // I3 mirrored
    z.parent.parent.color=R
    LEFT-ROTATE(T,z.parent.parent)
```


插入為何至多旋轉兩次？

I1 不旋轉，每次把 z 上移兩層，最多 O(H) 次。I2 接 I3 最多兩次基本旋轉，隨即結束；只做 I3 則一次。因此 CLRS 這套 bottom-up 插入的**旋轉總數至多 2 次**，不是每層各兩次。連搜尋、重新著色一起算，總 O(log(n+2))，新增節點空間 O(1)。

第一種離開迴圈的情況是父黑：沒有紅紅，最後只須確保根黑。第二種是 I3 把局部根塗黑：它與外父也不會衝突。I1 則嚴格往上走，不能無限重複。

修復分支	基本旋轉	下一步
I1：叔紅	0	z 上移兩層，可能重複
I2：叔黑、內側	1	立刻接 I3
I3：叔黑、外側	1	局部根黑，修復結束
最後根塗黑	0	完成

可能的旋轉段只有 I3 或 I2→I3；I1 可以出現很多次，但不會增加旋轉數。

兩種修復時機，不能混接程式

比較項目	本節 bottom-up	top-down 2-3-4 分裂式插入
何時處理雙紅 孩子	先插紅葉，再從違規處向上修復	搜尋途中遇 4-node 先分裂、變色
何時可能旋轉	叔黑的 I2／I3	向下處理時若產生相鄰紅便局部修復
共同不變量	BST 次序、無相鄰紅、各路黑數相等	相同，但修復入口條件不同

兩者可得到不同的合法樹形；不能把 top-down 的變色時機插入本節程式卻沿用原證明。「一次雙旋調整」可能包含兩次基本旋轉；比較旋轉數前，先固定計數單位與具體演算法。

也可把紅邊視為紅 孩子 與黑父的連結，收縮後對應 2-3-4 tree；一般紅黑樹允許左紅或右紅。額外要求紅邊左傾的是另一套規格，不能默認。

刪除前先分清 z、y、x 與原色

z 是要刪除的節點。若 z 至多有一個孩子，令 y=z；若有兩個孩子，令 y 為右子樹的最小節點，稍後把 **y 這個物件移到 z 的位置**。

x 負責填補 y 的舊位置：後繼 y 沒有左孩子，因此取 x=y.right；若 y=z，則取 z 唯一的非空孩子，沒有孩子時取 NIL。先保存 y 的原色，再讓移來的 y 繼承 z 的顏色。**是否需要修復，取決於 y 搬動前的顏色。**

```
RB-TRANSPLANT(T,u,v):
    若 u.parent==NIL: T.root=v
    否則若 u==u.parent.left: u.parent.left=v
    否則: u.parent.right=v
    v.parent=u.parent              // 即使 v==NIL 也寫
```

本節允許寫入 NIL.parent，只用它保存當前缺黑位置 x 的父節點，供修復時尋找兄弟；它不是所有空連結共用的正確父節點。旋轉遇到 β=NIL 時不可覆寫此欄位。修復完成後才重設 NIL.parent=NIL；NIL 始終為黑色。

RB-DELETE：先移接，再依原色決定修復

```
RB-DELETE(T,z):                  // z is an existing non-NIL record
    original=z.color
    if z.left==NIL:
        x=z.right; RB-TRANSPLANT(T,z,x)
    elif z.right==NIL:
        x=z.left; RB-TRANSPLANT(T,z,x)
    else:
        x,original=RELOCATE-SUCCESSOR(T,z)
    if original==B: DELETE-FIXUP(T,x)
    NIL.parent=NIL                // only after repair finishes
    reclaim z
```

刪除物件 z；其餘物件的 鍵值與資料 與身分保持不變。原色為紅時不減黑數；原色為黑時，從填補舊位置的 x 開始處理。若外部 API 接收 key，先搜尋，找不到便回傳 absent，不能對 NIL 呼叫此程序。

兩個孩子：移動後繼前先保存原色

```
RELOCATE-SUCCESSOR(T,z):          // requires two non-NIL children
    y=MINIMUM(z.right); original=y.color; x=y.right
    if y.parent==z:
        x.parent=y                // even when x==NIL
    else:
        RB-TRANSPLANT(T,y,x)
        y.right=z.right; y.right.parent=y
    RB-TRANSPLANT(T,z,y)
    y.left=z.left; y.left.parent=y
    y.color=z.color
    return x,original
```

後繼移接：資料跟著物件走

依序插入 40、20、60、10、30、50、70、45、55，得到根 40B：左子樹為 20B(10R,30R)，右子樹為 60R，其左側是 50B(45R,55R)，右側是 70B。

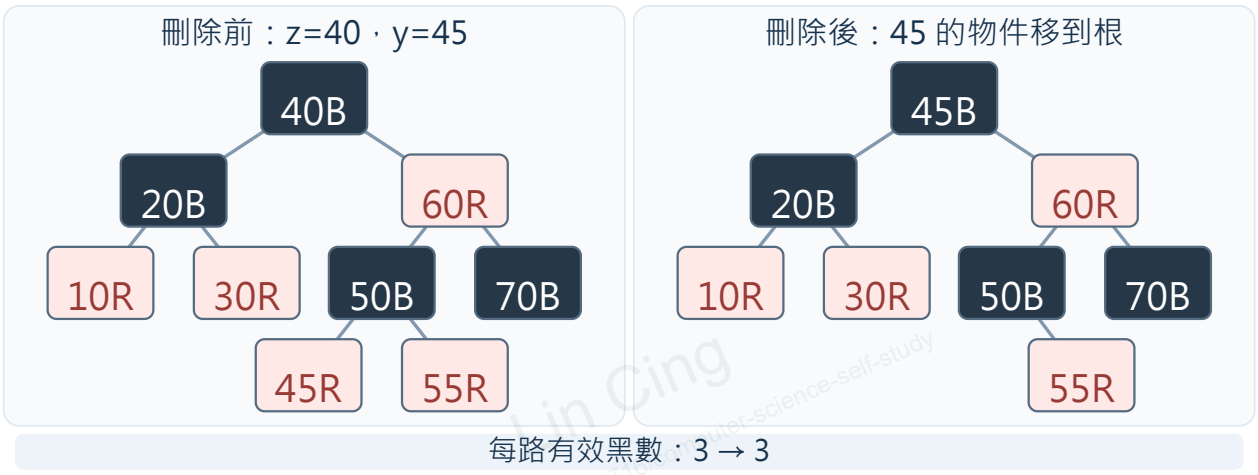
刪除 z=40 時，後繼 y=45R、x=NIL，且 y 的舊父為 50。依序執行：

1. 令 50.left=NIL、NIL.parent=50，移出原來的 45。
2. 令 45.right=60、60.parent=45，接上原右子樹。
3. 以 45 取代根 40，令 45.parent=NIL。
4. 令 45.left=20、20.parent=45，並讓 45 繼承 40 的黑色。

結果為根 45B。45 的原色是紅色，舊位置沒有少掉黑節點，因此無須修復；45 原有的資料與物件身分都保留，只有節點 40 被移除。

後繼若就是 z.right，x 的新父應為 y；若後繼更深，第一次移接已保存 x.parent，後續搬動 y 時不可破壞它。這兩種情況對 x=NIL 也必須成立。

深處後繼：同一棵樹的移接前後



兩圖是同一數例的移接前後：45 從 50 的左側搬到根。原位置由 x=NIL 補上；第一次移接後 NIL.parent=50，搬動 45 不可改寫它。修復或清理完成後才重設 NIL.parent=NIL。未畫出的空孩子皆為黑 NIL。

額外黑記在位置上，不是第三種顏色

若 y 的原色為紅，移除它不會減少路徑黑數；移到 z 位置的節點又已繼承 z 的顏色，所以不必修復。若 y 原為黑，則在填補舊位置的 **x 處暫記一份額外黑（extra black）**，讓經過該位置的路徑仍具有原來的有效黑數。

實際的 x.color 仍只有紅、黑兩種。若 x 紅，最後把它塗黑便吸收額外黑；若 x 已是根，則從全樹所有路徑同時取消這一份額外黑。只有 x 為黑且不是根時，才進入修復迴圈。

此時兄弟 w 不可能為 NIL：若 w=NIL，兄弟側只有末端 NIL 的一份黑；x 側至少已有 NIL 再加額外黑，共兩份，與兩側有效黑數相等矛盾。

x 在父 p 的左側時，w=p.right；稱 w.left 為**近側孩子 near**，w.right 為**遠側孩子 far**。x 在右側時，這兩個角色及旋轉方向全部鏡射。圖中的 x +1 表示額外黑；未畫出的空孩子皆為黑 NIL，路徑計數含起點及末端 NIL。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

修復迴圈：x 表示目前唯一的額外黑位置

```
DELETE-FIXUP(T,x):
    while x!=T.root and x.color==B:
        if x==x.parent.left: x=REPAIR-LEFT(T,x)
        else: x=REPAIR-RIGHT(T,x)
    x.color=B
```

兩個 輔助程序 回傳下一個修復位置：D2 回傳父節點，D4 回傳 根節點 使迴圈結束。最後的塗黑有兩種意義：x 紅時吸收額外黑；x 已是黑根時，把證明中的額外黑從全部路徑一起去掉，實際 color 不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

左側缺黑：D1～D4 的完整接續

```
REPAIR-LEFT(T,x):
    p=x.parent; w=p.right
    if w.color==R:                          // D1
        w.color=B; p.color=R; LEFT-ROTATE(T,p)
        w=p.right
    if w.left.color==B and w.right.color==B:  // D2
        w.color=R
        return p
    if w.right.color==B:                    // D3
        w.left.color=B; w.color=R; RIGHT-ROTATE(T,w)
        w=p.right
    w.color=p.color                         // D4
    p.color=B; w.right.color=B; LEFT-ROTATE(T,p)
    return T.root
```


右側缺黑：明列鏡射，避免近側與遠側搞反

```
REPAIR-RIGHT(T,x):
    p=x.parent; w=p.left
    if w.color==R:                          // D1 mirrored
        w.color=B; p.color=R; RIGHT-ROTATE(T,p)
        w=p.left
    if w.left.color==B and w.right.color==B:  // D2 mirrored
        w.color=R
        return p
    if w.left.color==B:                     // D3 mirrored
        w.right.color=B; w.color=R; LEFT-ROTATE(T,w)
        w=p.left
    w.color=p.color                         // D4 mirrored
    p.color=B; w.left.color=B; RIGHT-ROTATE(T,p)
    return T.root
```

先看兄弟，再看近側與遠側顏色

分支	前提（先處理紅兄弟）	去向
D1	兄弟紅	變成黑兄弟，接 D2／D3／D4
D2	黑兄弟，兩個 孩子 黑	缺黑移到父；可能被紅父吸收
D3	黑兄弟，near 紅、far 黑	轉成 D4
D4	黑兄弟，far 紅	消除額外黑並結束

x 在父左側時，near=w.left，far=w.right。D1／D3 後都要重新取 w；D3 不會返回，必接 D4。D2 的 return 是把父交回外層迴圈，不是宣告整個刪除已完成。

x 在父右側時，near=w.right、far=w.left。幾何角色以「靠近／遠離 x」定義，不能把 near 永遠記成 left。兩個 輔助程序 都不更新 NIL.parent；它由移接步驟保存，直到本次修復結束。

共享 NIL 不會讓此處左右判斷含糊：進入迴圈時兄弟必非 NIL，因此父不可能兩側同時為 NIL。

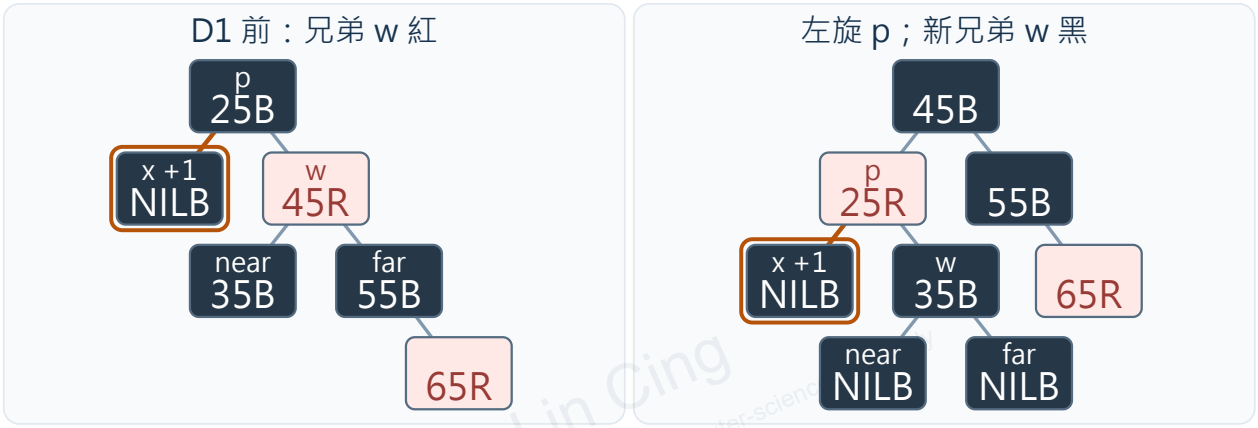
D1：兄弟轉黑，缺黑位置不動

先以 b 表示 x 子樹從根到 NIL 的**實際黑數**，含 x 與末端 NIL；加上額外黑後，有效黑數為 b+1。D1 的兄弟 w 紅，因此父 p 及 w 的兩個孩子必黑。w 紅而整棵 w 子樹的黑數為 b+1，故它的兩個孩子也各有 b+1 黑。

把 w 塗黑、p 塗紅，再左旋 p。新局部根為黑色 w；其左側的紅色 p 下，x 的有效黑數為 b+1，原近側孩子也為 b+1。w 右側原遠側孩子仍為 b+1。因此旋轉前後，從局部根開始的有效黑數均為 b+2。

額外黑仍留在 x，尚未消除。重新取得 x 的兄弟後，它已是黑色，接著依兩個孩子的顏色進入 D2、D3 或 D4。

D1 數例：重新取得旋轉後的兄弟



每路有效黑數：3 → 3

依序插入 15、25、35、45、55、65，再刪 15：x=NIL 位於 25B 左側，兄弟為 45R。把 45 塗黑、25 塗紅，左旋 25，得到新根 45B、左側 25R（右孩子 35B）、右側 55B（右孩子 65R）。x 仍是 25.left，**新兄弟改為 35B**；缺黑位置與祖先的有效黑數不變。

D2：把額外黑交給父節點

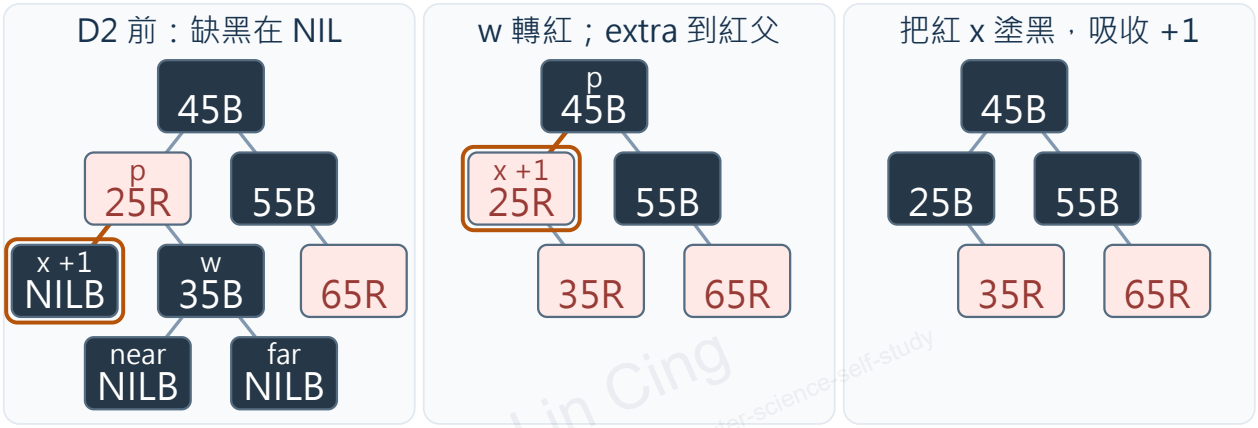
兄弟 w 黑，近側與遠側孩子也都黑。設 x 的實際黑數為 b；含額外黑後為 b+1。w 子樹的完整黑數為 b+1，而 w 的兩個孩子各為 b。

將 w 改為紅色後，w 子樹的黑數降為 b，與 x 側的**實際**黑數相等。但整棵 p 子樹對外仍少一黑，因此改把額外黑記在 p，令 x=p：

p 原來的狀態	接下來的處理
紅色	離開迴圈，最後塗黑，吸收額外黑
黑色且非根	沿祖先繼續修復
根節點	從全樹路徑同時取消額外黑，結束

D1 剛把 p 塗紅。因此 D1 後若接 D2，會由紅父吸收，不會再向上遇到第二次 D1。

D2 數例：紅父吸收額外黑



每路有效黑數：3 → 3 → 3

承接 D1：35B 的兩個孩子都是黑 NIL。把 35 改紅，將修復位置從 NIL 上移到 25R；迴圈停止，最後把 25 塗黑。結果為根 45B、左側 25B（右孩子 35R）、右側 55B（右孩子 65R）。額外黑由 25 的紅轉黑補齊，無須再旋轉。

D2 到達根：所有路徑一起少一黑

原樹為根 25B、兩個黑葉 15B 與 35B。刪除 15 後，D2 把 35 改紅，再令 x=25。x 已是根，因此從全樹所有路徑同時取消額外黑：

每路有效黑數：3 → 3 → 2

最後為 25B，其右孩子為 35R。根的黑高由 2 降為 1，各條路徑仍相等。這裡是**整棵樹一起少一黑**，與上一例「紅父塗黑以補回一黑」是不同的結束方式。

D3：近側紅，先轉成遠側紅

兄弟 w 黑，近側孩子 N 紅、遠側孩子黑。設 x 的實際黑數為 b。w 的兩個孩子從根到 NIL 均有 b 黑；因 N 紅，N 的兩個孩子也各有 b 黑。

把 N 塗黑、w 塗紅，再右旋 w。新兄弟為 N：左側是原 N.left，黑數為 b；右側是紅色 w，它的兩個孩子為原 N.right 與原遠側孩子，黑數也都為 b。因此新兄弟的完整黑數仍為 b+1，父 p 及祖先看到的有效黑數不變。

新兄弟的遠側孩子就是原來的 w，現在為紅色，故**立即接 D4**。額外黑仍在 x；D3 既沒有把缺黑上移，也沒有完成修復。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D3 數例：右旋兄弟，再接 D4

依序插入 15、25、45、35，得到根 25B、左孩子 15B，以及右孩子 45B（左孩子 35R）。刪除 15 後，x=NIL、兄弟 w=45B、近側為 35R、遠側為黑 NIL。

把 35 塗黑、45 塗紅，再右旋 45，得到根 25B、右孩子 35B，其右孩子為 45R。新兄弟 35 的遠側已紅，下一步必須執行 D4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D4：兩側補齊，消除額外黑

設父 p 原色的黑色貢獻為 c∈{0,1}，x 的實際黑數為 b。兄弟 w 黑，其近側與遠側孩子的完整黑數皆為 b，且遠側孩子原為紅色。

讓 w 繼承 p 的原色，把 p 與遠側孩子塗黑，再左旋 p。新局部根為 w：

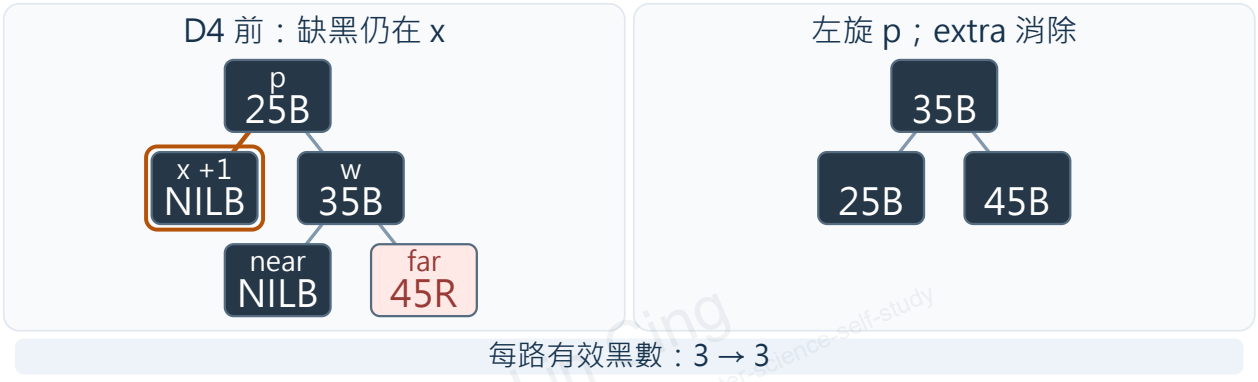
新根的哪一側	從該側根到 NIL 的黑數
左側 p	p 黑，加上 x 或近側子樹的 b，得到 b+1
右側原遠側孩子	由紅轉黑，從 b 增為 b+1

兩側實際黑數已相等，不再需要額外黑。新局部總黑數為 c+b+1，等於原本從 p 開始、含額外黑的有效總數。

若新根 w 紅，它的兩個孩子 p 與原遠側孩子都黑；w 對外又保留 p 的原色，因此不會與外父新增紅紅衝突。令 x=T.root 結束迴圈，最後確保根黑。

D4 數例：完成同一次刪除

承接 D3 的數例：兄弟 35 繼承父 25 的黑色，25 與遠側孩子 45 也塗黑，再左旋 25。



結果為根 35B、左孩子 25B、右孩子 45B。留下的鍵為 25、35、45；每條根到 NIL 的路徑均有 3 黑，額外黑已完全消除。

右側缺黑：把 D3、D4 完整鏡射

依序插入 15、35、45、25，再刪除 35。後繼 45B 就是右孩子，移接後成為新根；x=NIL 位於 45.right，故必須令 NIL.parent=45。

兄弟為 15B，近側是 25R，遠側是黑 NIL。右側 D3 **左旋 15**，接著右側 D4 **右旋 45**；最後為根 25B、兩個黑孩子 15B 與 45B。近側、遠側是相對於 x 的位置，不是固定的左、右方向。

同一數例：逐路加總實際黑數與額外黑

以下每條路**含起點與 NIL**。D3 不改 x=NIL 的位置；D4 才消除 extra。

階段與路徑	實際黑數	extra	有效總數
D3 前：25B → NIL(x)	2	1	3
D3 前：25B → 45B → 35R → NIL	3	0	3
D3 後：25B → 35B → 45R → NIL	3	0	3
D4 後：35B → 25B → NIL	3	0	3
D4 後：35B → 45B → NIL	3	0	3

黑點各加 1、紅點加 0，只有穿過 x 的路再加 extra。未列出的另一側 NIL 路依同一規則也為 3；可對照前後圖逐一驗算。D4 後無須 extra，兩側已各補齊一黑。

終止與總成本：旋轉少，不代表刪除為常數

BST 移接先保住鍵序與所有未刪除的節點身分。移除紅色位置不減黑數；移除黑色位置時，先在 x 記額外黑，使祖先看到的有效黑數維持平衡。

修復步驟	額外黑的位置	是否可能繼續向上
D1、D3	留在原 x	只轉換局部情況
D2	移到父節點	黑父且非根時繼續
D4	消除	結束

D2 每次上移一層，最多 O(log(n+2)) 次。D1 後父節點變紅：若接 D2 就由紅父吸收，否則經 D3／D4 結束。因此 D1、D3、D4 各至多出現一次，**整次刪除至多 3 次基本旋轉**。

各分支已分別證明黑數及紅紅關係的維護；最後塗黑處理紅 x 或黑根的結束情況，紅黑性質全部恢復。刪除最後一個節點則得到 T.root=NIL。

搜尋 z、找後繼、向上修復合計仍需 **O(log(n+2))**。至多三次旋轉只限制結構重排；D2 的傳播與重新著色仍可能經過整條祖先路徑。

用子樹大小支援動態順序統計

資料反覆插入、刪除後，若還要查「第 k 筆」或「這個節點排第幾」，可以在平衡搜尋樹加入**子樹大小 size**。size 包含節點本身，NIL.size=0：

$$\text{size}(x) = 1 + \text{size}(x.left) + \text{size}(x.right).$$

秩（rank）採中序次序的 1-based 位置；select(k) 回傳第 k 個節點。本節數例的鍵互異。若允許重複鍵，可用 (key,identity) 定義全序；只給一個重複值，無法指定唯一節點的秩。

擴充時必須一起說清楚**摘要怎麼算、修改後怎麼修、查詢為何正確**。本節摘要由自身資料與兩個孩子在 O(1) 時間算出，且不依節點顏色；依顏色的摘要，須另算重新著色的維護成本。查詢與維護皆先依樹高 h 分析為 O(h+1)；只有底層維持紅黑性質時，才能換成最壞 O(log(n+2))。加一個欄位不會自動讓普通 BST 平衡。

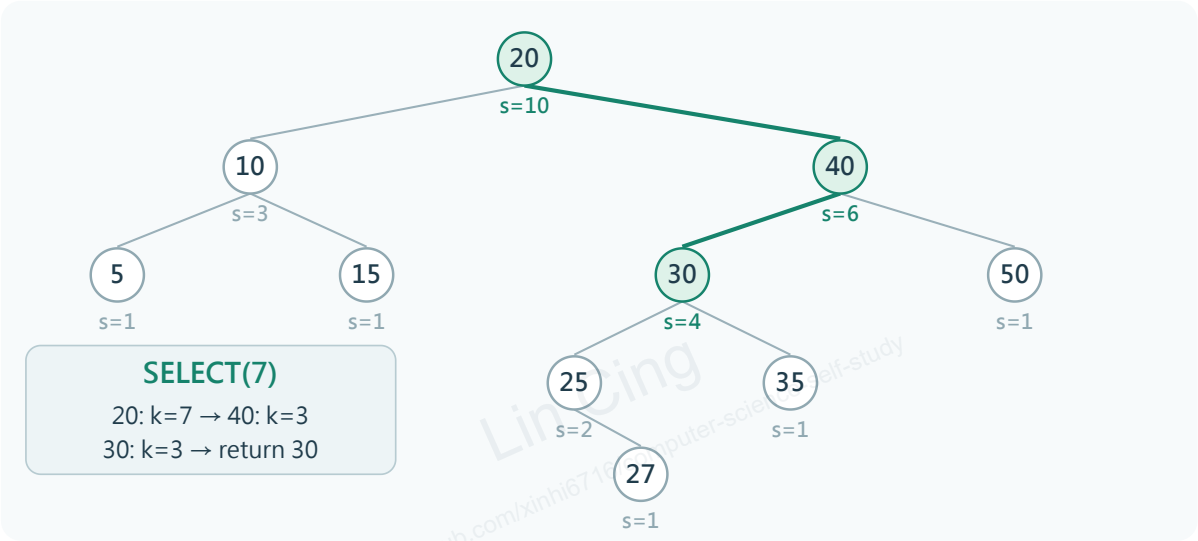
Select：每層都使用目前子樹的局部秩

```
SELECT(x,k):
    require 1<=k<=x.size
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
```

迴圈不變量是「答案仍為目前 x 子樹中的第 k 個節點」。左子樹佔前 r−1 位，x 佔第 r 位；往左時 k 不變，往右時扣掉左子樹與 x，共 r 筆。

合法的 k 保證選到的子樹非空，且新 k 仍合法。每次下降一層，因此會在有限步驟內找到答案，不會先走到 NIL。公開介面須先拒絕 k=0、k>root.size；空樹沒有合法的 k。

同一棵樹，同時核對鍵值、size 與路徑



圓內是鍵，圓下 s 是包含自身的子樹大小。綠色路徑追蹤 SELECT(7)：20→40→30。接下來的 select、rank 與刪除數例，都以這棵樹為共同起點。

Select(7)：往右才扣掉已略過的前綴

目前鍵	局部 k	left.size	r	選擇與新 k
20	7	3	4	往右到 40；7−4=3
40	3	4	5	往左到 30；仍為 3
30	3	2	3	回傳節點 30

中序為 **5,10,15,20,25,27,30,35,40,50**，第 7 筆確為 30。第一步略過 {5,10,15,20}，剩餘子樹中要找的是第 3 筆；不能把全樹的 7 原封不動拿去和每一層的 r 比較。

往左未略過目前子樹中的任何前綴，所以不扣 k；往右略過的正是 r 筆。這也說明為何 k−r 不會是 0：只有 k>r 時才往右。

Rank：向上累加互不重疊的前綴

```
RANK(T,x):                    # x is a valid node in T
    r=x.left.size+1; u=x
    while u!=T.root:
        if u==u.parent.right:
            r+=u.parent.left.size+1
        u=u.parent
    return r
```

不變量：r 是原節點 x 在目前 u 子樹中的中序位置。u 若是左孩子，父節點與右側資料都排在 x 後，不新增前綴；u 若是右孩子，就加上父節點及其整棵左子樹。

新增的資料都在先前 u 子樹之外，又都排在 x 之前，因此不重複。到達根時，r 就是全樹的秩。此介面接受樹內的有效節點，不能傳 NIL、已刪除的物件或另一棵樹的節點。

Rank(27)：哪些祖先需要加，哪些不用？

初始 r=size(27.left)+1=1。

u→父節點	u 位於哪側	新增量	更新後 r
27→25	右側	size(NIL)+1=1	2
25→30	左側	0	2
30→40	左側	0	2
40→20	右側	size(10)+1=4	6

最後得到第 6 筆，和中序次序一致。第一次加入節點 25，第二次加入 {5,10,15,20}；加上原本的 27，共 6 筆。這些前綴彼此不重疊，30、40 則都在 27 後面，不能算進來。

摘要由下往上重算，旋轉也要同步維護

```
PULL(v):                        # v != NIL
    v.size=1+v.left.size+v.right.size
    v.max=max(v.high,v.left.max,v.right.max)
REPAIR-UP(v):
    while v!=NIL:
        PULL(v); v=v.parent
```

此版本同時保存 size 與區間的最大上端點 max；只有順序統計時，可省略 max。NIL.size=0、NIL.max=−∞。

插入新葉時 size=1；接好連結後，從其父節點向根重算。刪除則先完成 BST 移接，再從正確起點沿**最終的父鏈**重算，之後進入紅黑修復。旋轉改好指標後，先 PULL 下移節點，再 PULL 升起節點。

size 與 max 都不依顏色，重新著色不必重算。刪除區間或縮短上端點時，max 可能下降，不能只保留歷史最大值。

刪除後，從哪裡開始修摘要？

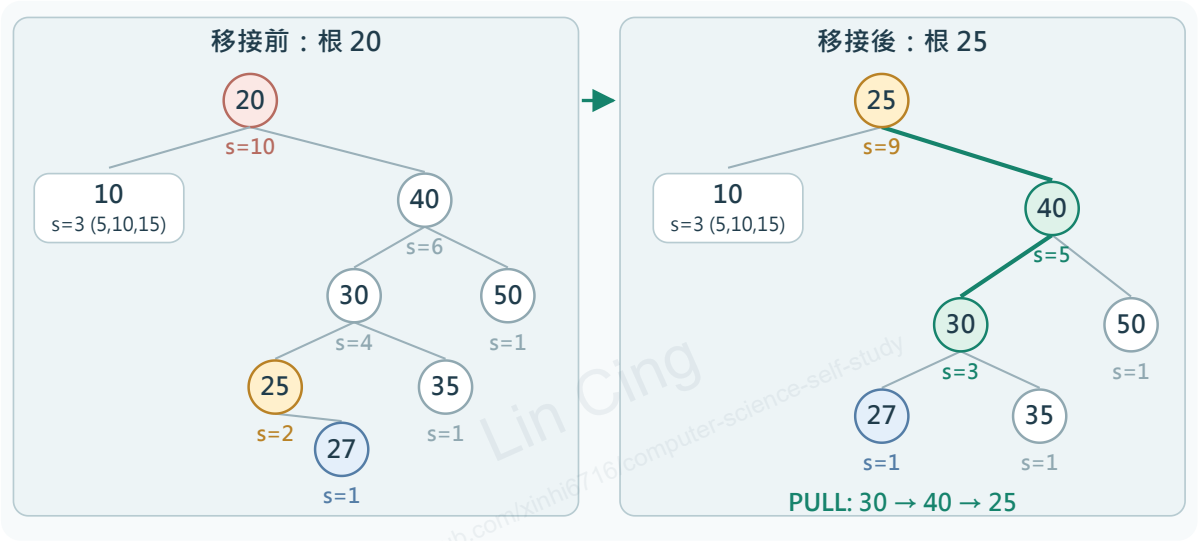
在移接前先保存起點 a；完成全部連結後，執行 REPAIR-UP(a)，再做紅黑顏色修復。

刪除分支	保存的 a	最終父鏈涵蓋的變動
z 至多一個孩子	z 的原父節點	孩子的子樹不變；祖先失去一筆
後繼 y 就是 z.right	y	y 接上新的左子樹，再修祖先
後繼 y 位於更深處	y 的原父節點	原右子樹、搬上去的 y、z 的舊祖先

必須保存真正的節點，不能在釋放 z 後再讀 z.parent。共享的 NIL.parent 也不是所有空連結的共同父節點；修復摘要的 a 應另外保存。若 a=NIL，代表沒有祖先需要重算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除 20、移接 25：連結與摘要一起看



25 的物件身分保留，27 接到 30 的左側；25 再接上原左子樹 10、原右子樹 40。色塊表示移接角色，不是紅黑顏色。圖中先完成 BST 移接與摘要重算，顏色修復依前節程序接續。

先算 30，再算 40，最後才算新根 25

移接前保存 a=30。移接後，這條父鏈為 **30→40→25**，依序重算：

節點	使用已更新的孩子	新 size
30	1+size(27)+size(35)=1+1+1	3
40	1+size(30)+size(50)=1+3+1	5
25	1+size(10)+size(40)=1+3+5	9

25 原本的 size=2 已失效。若先重算 25，會讀到尚未更新的 40.size=6，誤得 10。若只從舊根 20 的父節點開始，起點是 NIL，更會完全漏掉這三個需要更新的節點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

旋轉後只修兩格，需要額外理由

左旋時 y=x.right 升起，β=y.left 改接給 x。完成指標改接後，**先 PULL(x)，再 PULL(y)**，因為新 y 的摘要依賴新 x。

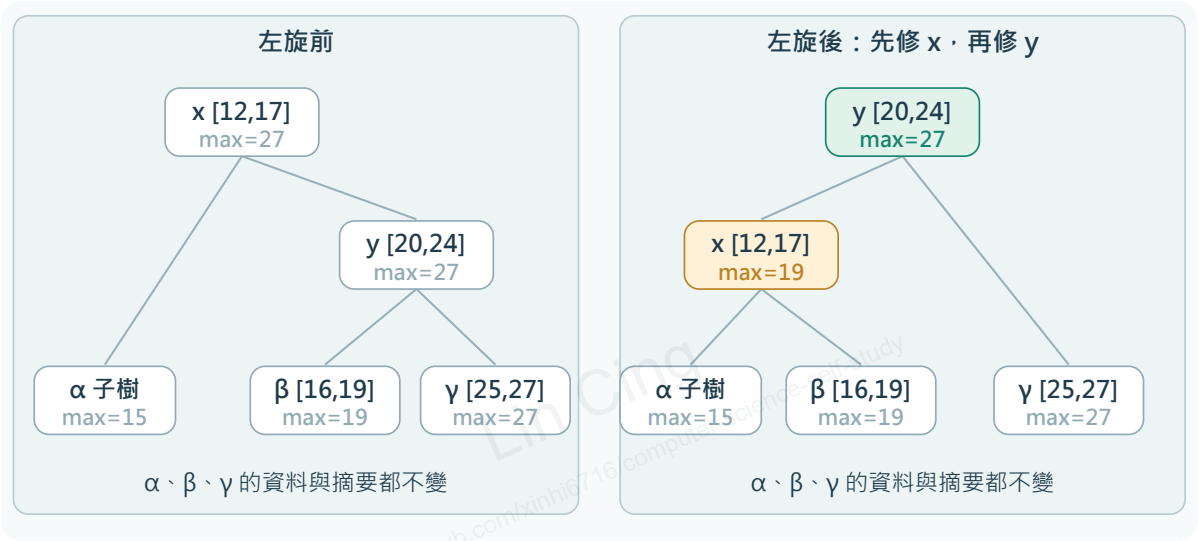
size 與子樹最大 high 都只依「子樹含有哪些資料」。旋轉後，新 y 包含的資料恰為舊 x 的整棵子樹，因此新 y 的摘要等於舊 x，外部祖先看到的摘要不變，只需 O(1) 局部重算。

由孩子算得出，不代表只影響兩格。 height 也有局部公式，旋轉後局部樹高卻可能改變，仍須向祖先傳播。一般摘要每次旋轉最多增加 O(h+1) 工作；紅黑插入最多 2 次、刪除最多 3 次旋轉，所以仍為 O(log(n+2))。

若直接在每個節點保存全域 rank，插入新最小鍵會讓所有舊節點的 rank 加 1，需要 Θ(n) 次明確寫入。選子樹摘要，正是為了把更新留在少數祖先路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一組區間左旋：x.max 下降，整體 max 不變



先算 x.max=max(17,15,19)=19，再算 y.max=max(24,19,27)=27。新 y 仍涵蓋原局部子樹的全部資料；先前最大上端點 27 沒有消失，只是內部連結改變。

區間樹：依下端點排序，保存最大上端點

本節使用閉區間 [l,h]，允許 l=h 的單點區間。兩區間 i、q 重疊，當且僅當：

$$i.low \le q.high \quad \text{且} \quad q.low \le i.high.$$

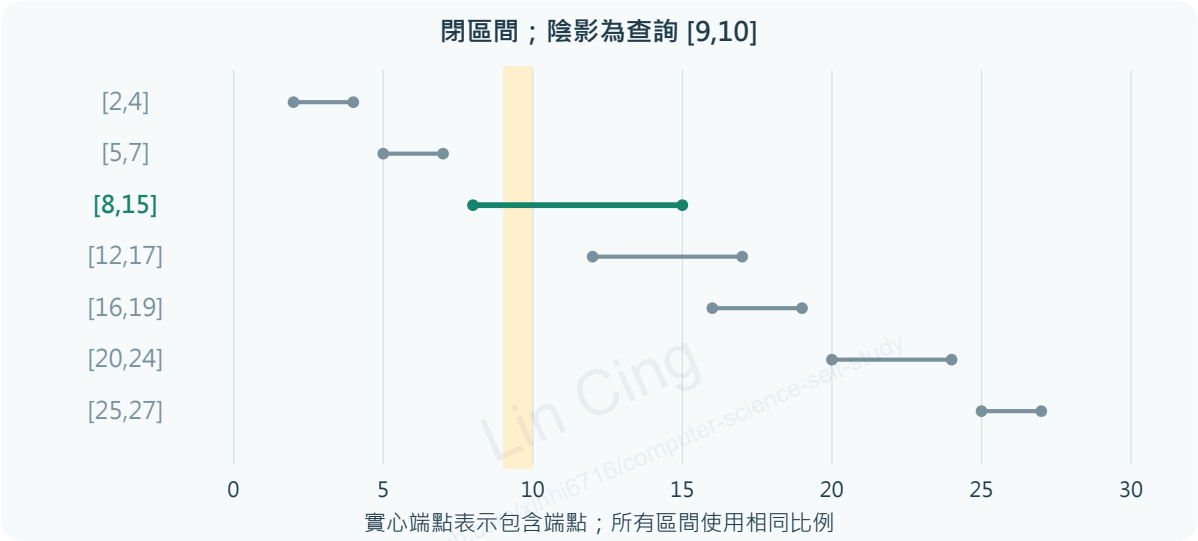
也就是排除 i 完全在 q 左方或右方的兩種情況。

BST 依下端點 low 排序；同 low 可再用物件識別碼決定先後，仍有 left.low≤x.low≤right.low。每節點保存

$$x.max = \max\{x.high,\ x.left.max,\ x.right.max\}, \qquad NIL.max = -\infty.$$

max 是整棵子樹的最大 high，不是目前節點的 high，更不是最大的 low。數例的根 [12,17] 雖然 high=17，max 卻是右側後代 [25,27] 提供的 27。

把區間放在同一條座標軸上



陰影是查詢 [9,10]。只有 [8,15] 穿過陰影；[5,7] 在它左側，[12,17] 在右側。端點位置與後面的搜尋樹使用的資料完全一致。

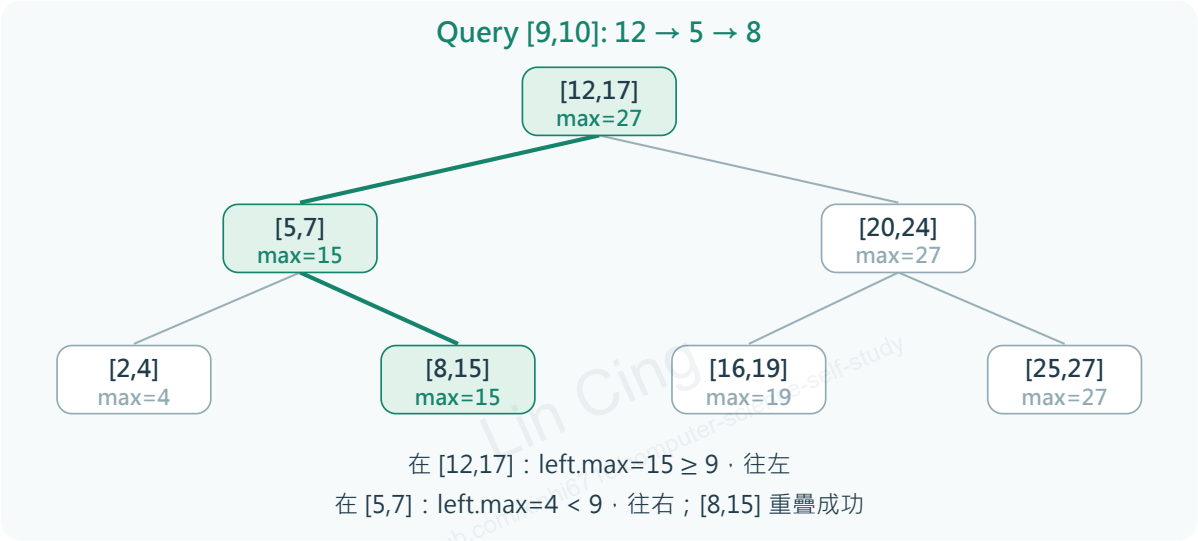
重疊搜尋：每一步只選一棵子樹

```
INTERVAL-SEARCH(T,q):
    x=T.root
    while x!=NIL and not overlaps(x.interval,q):
        if x.left!=NIL and x.left.max>=q.low:
            x=x.left
        else:
            x=x.right
    return x
```

找到目前節點重疊就回傳；否則依左子樹的 max 決定方向。此程序只找**任意一個**重疊區間，不保證 low 最小，也不列出全部答案。

真正需要證明的是：若目前子樹有答案，選出的那一側仍保留至少一個答案。left.max≥q.low 本身不保證左側一定重疊，不能省略後面的正確性論證。

同一棵區間樹：綠線追蹤查詢 [9,10]



第一步讀到左側 max=15≥9；到 [5,7] 時，改讀其左側 max=4<9，因此轉向右側，找到 [8,15]。每一步讀的是「當前節點左孩子的子樹摘要」。

三次查詢：成功與失敗都能逐步核對

查詢	訪問順序	分支依據／結果
[9,10]	[12,17]→[5,7]→[8,15]	15≥9 往左；4<9 往右；命中
[18,18]	[12,17]→[20,24]→[16,19]	15<18 往右；19≥18 往左；命中
[28,29]	[12,17]→[20,24]→[25,27]→NIL	左側 max 依序不足；最後無解

查詢 [28,29] 走到 [25,27] 時，目前區間的 high=27<28，且左右孩子皆空，故最後回傳 NIL。**目前節點不重疊**與**某棵子樹可排除**是不同判斷，不能只看 x.high 就決定剪掉左側。

往右：先證明左側完全沒有答案

維持不變量：「若原樹存在重疊區間，目前 x 子樹也至少有一個。」起點為根，顯然成立；若 x 重疊就直接回傳。

當左孩子為 NIL，左側沒有候選。否則若 left.max<q.low，對任何左側後代 j 都有

$$j.high \leq left.max < q.low.$$

因此它們全在查詢左方，不可能重疊。加上 x 已經不重疊，若仍有答案，只能在右側，往右保持不變量。

這裡用的是**整棵左子樹的 high 上界**。若只將根改成 [12,13]，查詢 [14,14]，雖然根的 high=13<14，左側 [8,15] 仍是答案；單看根的 high 不足以剪枝。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

往左：為什麼不必回頭搜尋右側？

若 left.max≥q.low，左子樹中必有一個區間 j，其 high **恰等於**這個 max。左側若本來就有答案，走左當然安全。

若左側完全無解，j.high≥q.low 卻仍不重疊，只可能是 j.low>q.high。對任一右側後代 k，BST 的鍵序給出

$$k.low \geq x.low \geq j.low > q.high.$$

所以右側也全在查詢右方；加上 x 不重疊，此時目前子樹根本沒有答案。因此不論左側有沒有解，都不會丟掉原本存在的所有解。

走到 NIL 時，由不變量得知原樹也無解。每次下降一層，成本為 O(h+1)，紅黑底層則為 O(log(n+2))。關鍵是**精確 max 對應實際存在的區間**，不是隨意給的上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

過大的舊 max，也可能讓搜尋漏掉答案

將 [8,15] 改成 [8,27] 後再刪掉。左子樹只剩 [5,7]、[2,4]，精確 max 應為 7；若錯留 27，查詢 [22,22]：

位置	使用的比較	錯誤路徑
[12,17]	舊 left.max=27≥22	往左，放棄有答案的右側
[5,7]	left.max=4<22	往空的右孩子
NIL	迴圈結束	錯誤回報找不到

真正答案是右側 [20,24]。精確 left.max=7 會讓第一步往右找到它。**舊上界 27 沒有左側區間作見證**，因此上一頁「左無解則右亦無解」的論證不再成立。修改後必須由下往上重新取 max，允許摘要下降。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

端點慣例與資料修改，也屬於演算法規格

區間慣例	非空條件	overlap(i,q)	左側可能有解
閉區間 [l,h]	l≤h	i.low≤q.high 且 q.low≤i.high	left.max≥q.low
半開 [l,h)	l<h	i.low<q.high 且 q.low<i.high	left.max>q.low

[4,5] 與 [5,7] 在 5 重疊；[4,5) 與 [5,7) 則沒有交集。[5,5) 是空集合，不能把單點閉區間的結論直接套過來。

修改 low 會改變排序鍵，應刪除舊資料再插入；不能留在原位置只改數值。只改 high 且仍滿足非空條件時，鍵序不變，但須從該節點往上重算 max；縮短區間也不能省略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

找一個、找最左與列出全部，是不同問題

本節搜尋只回傳任一重疊區間。若要列出全部，可在 left.max≥q.low 時訪問左側、檢查 x，再於 x.low≤q.high 且 right.max≥q.low 時訪問右側。每個訪問節點 O(1)，最壞仍可為 O(n)，不能直接沿用單一路徑的對數界。

另一個容易完整分析的方法，是反覆搜尋、暫時刪除命中資料，記錄後最後插回。若共有 k 個答案，包含 k 次成功及最後一次失敗搜尋，總成本為 **O((k+1)log(n+2))**，另需 O(k) 儲存結果。這會暫時修改集合，必須符合介面契約。

要得到 O(log n+k)，還需要另一個演算法或更細的訪問節點數證明。size、max 可同時維護，但這裡的 rank 是依 (low,identity) 排序的位置，並非依 high 排序，也不是重疊區間的數量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-tree：把一頁內的空間換成較少的層數

一個節點保存多個已排序的鍵，以及分隔各鍵範圍的子指標。搜尋先在頁內選出範圍，再讀取下一頁；成本要分成**頁存取（I/O）與頁內比較、搬移（CPU）**。

本節採鍵互異的集合。每個鍵與它的資料或 record pointer 一起搬移；內節點也能保存實際資料。重複插入回傳 EXISTS，不新增一筆。這與 B+ tree「資料集中在葉、父節點另存 separator」的表示不同。

一節點能否放進一頁，取決於頁大小、鍵與資料指標大小、子指標及頁頭。以下的 I/O 分析假設這個配置已成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最小度 t：先固定每種節點的容量

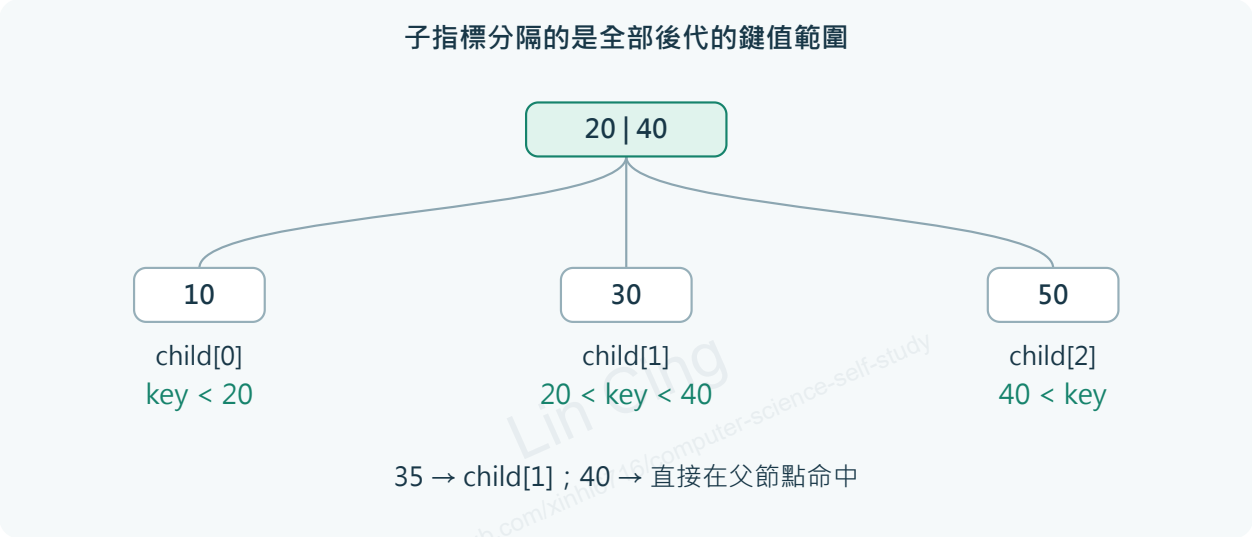
固定整數 $t \geq 2$；鍵陣列與子指標陣列皆採 0-based。

節點種類	鍵數 k	子指標數
非根節點	$t-1 \leq k \leq 2t-1$	葉為 0；內節點恰為 $k+1$
非空根	$1 \leq k \leq 2t-1$	葉為 0；內節點至少 2
空樹	零鍵葉根	0

所有葉深度相同；根深度為 0，葉深度為高度 h。非空樹 h=0 表示資料全部放在根；空樹也約定 h=0，但下文的非空高度定理不能直接套 n=0。

Lin Ching
https://github.com/xinhi6716/computer-science-self-study

每個子指標負責整段鍵值範圍



鍵 $a_0 < \cdots < a_{k-1}$ 將整個子樹分成 $k+1$ 段。範圍限制適用於**全部後代**，不是只比較子節點的第一個鍵。搜尋 35 要走中間；搜尋 40 直接在父命中。

order m 與 minimum degree t 不能混用

本節採用的定義	非根內節點的分支數	非根鍵數
order m：最大分支數為 m	$\lceil m/2 \rceil \cdots m$	$\lceil m/2 \rceil - 1 \cdots m-1$
minimum degree t	$t \cdots 2t$	$t-1 \cdots 2t-1$

m=2t 時容量才完全對應。 m=3 是 2-3 tree，不能令 t=1.5，也不能令 t=2：後者是最多 4 個分支的 2-3-4 tree。

一般「order」的定義並非各書一致；先確認它指最大分支數還是其他容量。上表固定採最大分支數，不從名稱猜公式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先分裂與溢位後分裂：前提不同

本節插入先分裂已有 $2t-1$ 鍵的滿節點：提升一鍵，左右各 $t-1$ 鍵，兩邊立即合法。

order m 的溢位後分裂則先讓節點暫存 m 鍵，再提升中位鍵。例如 m=3：葉 [2,8] 插 5，暫成 [2,5,8]，提升 5，留下 [2]、[8]。

若 m=3 卻在滿葉 [2,8] 插入前就分裂，提升任何一鍵都會讓一側零鍵，違反下限 1。因此**容量公式、修復時機與正確性證明必須配套**。刪除同樣有「先修好再下降」與「刪後處理不足」兩種安排。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

高度上界：先數最稀疏的合法樹

假設 n≥1。高度 h 的根至少有 1 鍵；h≥1 時至少有 2 個子節點。深度 i≥1 至少有 $2t^{i-1}$ 個節點，每個至少 t−1 鍵，所以

$$n \geq 1 + \sum_{i=1}^{h} 2t^{i-1}(t-1) = 2t^h - 1.$$

移項並取以 t 為底的對數：

$$h \leq \log_t \frac{n+1}{2}.$$

h=0 時總和為空，得到 n≥1，仍符合。根的最少鍵數是 1，不能誤把根也算成 t−1 鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

高度下界與十億筆資料的頁數

每節點最多 2t−1 鍵、最多 2t 個分支，因此

$$n \leq (2t-1)\sum_{i=0}^{h}(2t)^i = (2t)^{h+1} - 1.$$

取 t=100、n=10^9：h=2 最多 7,999,999 鍵，不夠；h=3 最多 1,599,999,999 鍵，容量足夠。另一方面 h=4 最少 199,999,999 鍵；h=5 最少 19,999,999,999 鍵，太多。故合法樹只能有 h=3 或 4。

一次搜尋走 4 或 5 個節點；若根常駐記憶體、其他路徑頁皆未快取，便是 **3 或 4 次頁讀取**。若根也不在記憶體，再多讀一頁。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

搜尋：頁內定位，再走唯一可能的子樹

```
SEARCH(x,k):
    i=lower_bound(x.keys,k)       // first key >= k, or len(keys)
    if i<len(x.keys) and x.keys[i]==k: return (x,i)
    if x.children is empty: return NOT_FOUND
    read x.children[i] if not cached
    return SEARCH(x.children[i],k)
```

若沒有相等鍵，全域範圍限制保證只有 child i 可能包含 k；其餘全部排除。每次下降一層，最後在葉命中或失敗，故終止且正確。lower_bound 可用線性掃描或二分搜尋；兩者不改變頁路徑。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

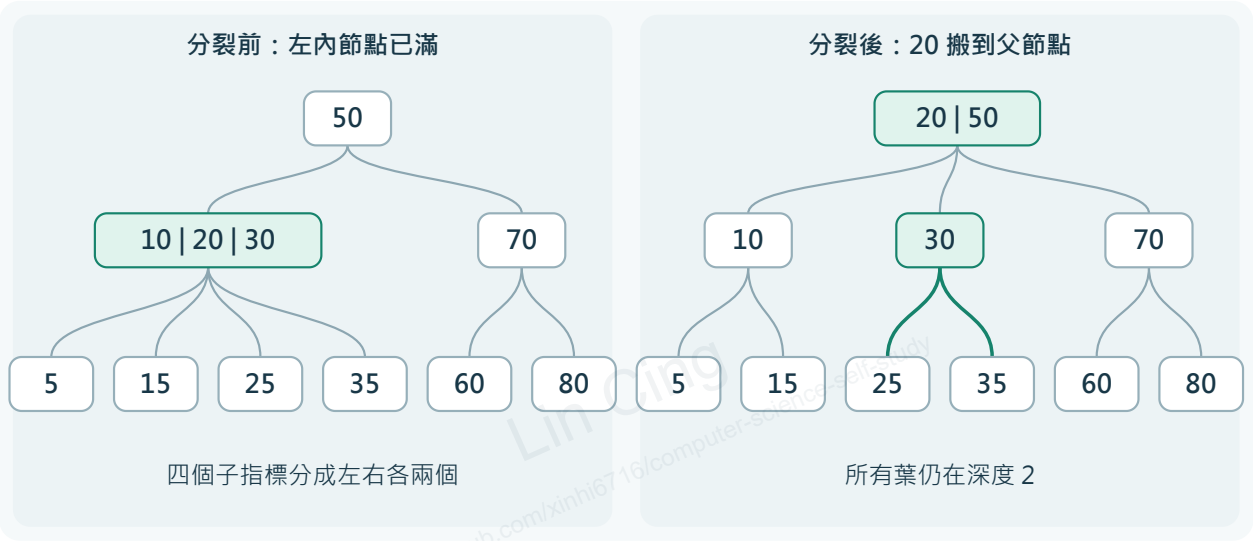
SPLIT：提升中位鍵，也切開子指標

```
SPLIT(p,i):                       // p nonfull; child i full
    y=p.children[i]; old=y.keys; kids=y.children
    median=old[t-1]
    z=Node(keys=old[t:], children=kids[t:] if kids else [])
    y.keys=old[:t-1]
    y.children=kids[:t] if kids else []
    p.keys.insert(i,median); p.children.insert(i+1,z)
    write y,z,p
```

切片採左含右不含，且 old、kids 代表改動前的陣列。左右各 t−1 鍵；若為內節點，各有 t 個子指標。median 搬到父節點，**不在子節點保留副本**。父原先未滿，增加 1 鍵仍合法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分裂內節點：四個舊子樹不能遺失

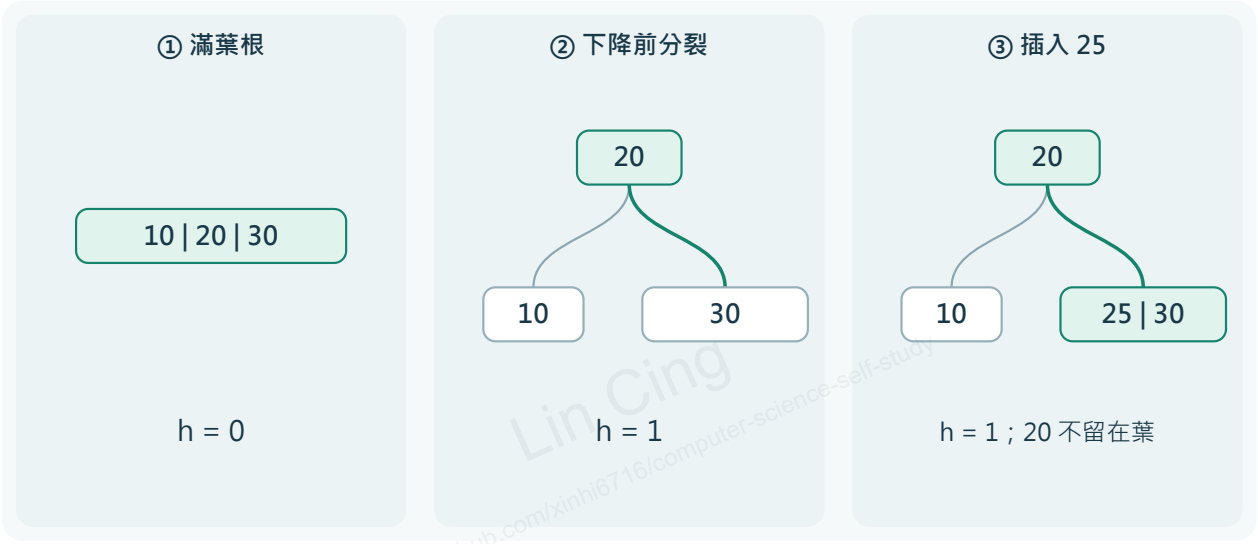


t=2，原父 [50] 的左子節點為 [10,20,30]。提升 20 後，父為 [20,50]；[10] 接 [5]、[15]，[30] 接 [25]、[35]。原父的右子樹 [70] 不變。所有葉仍深度 2。

插入：下降時永遠選未滿節點

```
INSERT(k):
    if SEARCH(root,k) succeeds: return EXISTS
    if len(root.keys)==2*t-1:
        root=Node([], [root]); SPLIT(root,0)
    x=root
    while x.children is not empty:
        i=lower_bound(x.keys,k)
        if len(x.children[i].keys)==2*t-1:
            SPLIT(x,i)
            if k>x.keys[i]: i+=1
        x=x.children[i]
    insert k in sorted order into x.keys; write x
    return INSERTED
```

根分裂：高度從頂端增加



t=2，滿葉根 [10,20,30] 插 25：先提升 20，再因 25>20 選右葉，得到 [25,30]。只有根分裂新增一層；所有舊葉一起加深，葉深度仍相等。

插入正確性：為何不會向上連鎖補救？

迴圈開始時 x 未滿。根若原本滿，先建立新根並分裂，故初始化成立。若下一個子節點滿，因 x 未滿，可安全把一鍵提升到 x；分裂後的兩個子節點各有 t−1 鍵，選中的一側仍未滿。

每次下降一層；到葉後只增加一鍵，最多從 2t−2 變 2t−1，不溢位。SPLIT 保存全部鍵及子樹範圍，最後插入的 k 也落在搜尋選定的範圍，因此鍵集合恰增加 k。

與溢位後分裂相比，這裡把修復提前到下降之前，所以無須回到已離開的父頁處理溢位。插入前查重另走一條路徑，仍不改變漸近成本。

刪除：先替即將下降的子問題留一格餘裕

非根節點通常只需 t−1 鍵；本節刪除加強條件：**每次遞迴進入非根節點時，先保證它至少有 t 鍵**。這樣在該層失去一鍵，仍有合法的 t−1 鍵。

情況	處理
在葉找到 k	直接刪除
在內節點找到 k，左側至少 t 鍵	用前驅取代，從左側刪前驅
左側不足 t、右側至少 t	用後繼取代，從右側刪後繼
相鄰兩側皆 t−1	兩側與父鍵合併，再刪 k

未找到 k 時，也要先把目標子節點修到至少 t 鍵再下降。根可例外較少；若最後變成零鍵內根，改用唯一子節點當新根。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

向左兄弟借：父鍵下降，兄弟最大鍵上升

```
BORROW-LEFT(p,i):                // i>0
    y=p.children[i]; s=p.children[i-1]
    y.keys.insert(0,p.keys[i-1])
    p.keys[i-1]=s.keys.pop()
    if s.children: y.children.insert(0,s.children.pop())
    write s,y,p
```

前提：y 有 t−1 鍵、s 至少 t 鍵；兩者皆在同一層。s 的最右子樹介於新、舊分隔鍵之間，必須一起移到 y 的最左側。完成後 y 有 t 鍵，s 仍至少 t−1 鍵；父鍵數不變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

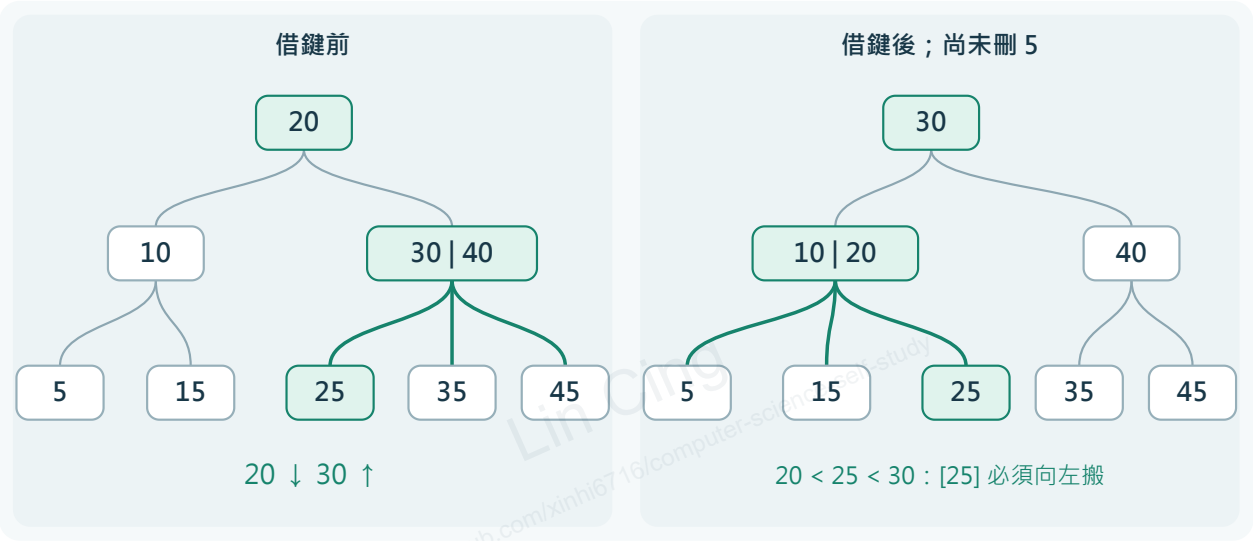
向右兄弟借：子指標也要移到正確一側

```
BORROW-RIGHT(p,i):                // i<len(p.keys)
    y=p.children[i]; s=p.children[i+1]
    y.keys.append(p.keys[i])
    p.keys[i]=s.keys.pop(0)
    if s.children: y.children.append(s.children.pop(0))
    write s,y,p
```

前提同樣是 y 有 t−1 鍵、s 至少 t 鍵。s 的最左子樹移到 y 最右側。只搬鍵、不搬該子指標，會使整段鍵值範圍落在錯誤子樹；即使每個節點內仍排序，也已不再是合法搜尋樹。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

借鍵數例：25 的子樹為何必須向左搬？



t=2，要刪 5。左內節點 [10] 只有 1 鍵，右兄弟 [30,40] 有 2 鍵。先借：20 下降，30 上升；[25] 介於 20 與 30，必須成為 [10,20] 的最後一棵子樹。圖中只完成下降前修復，尚未刪 5。

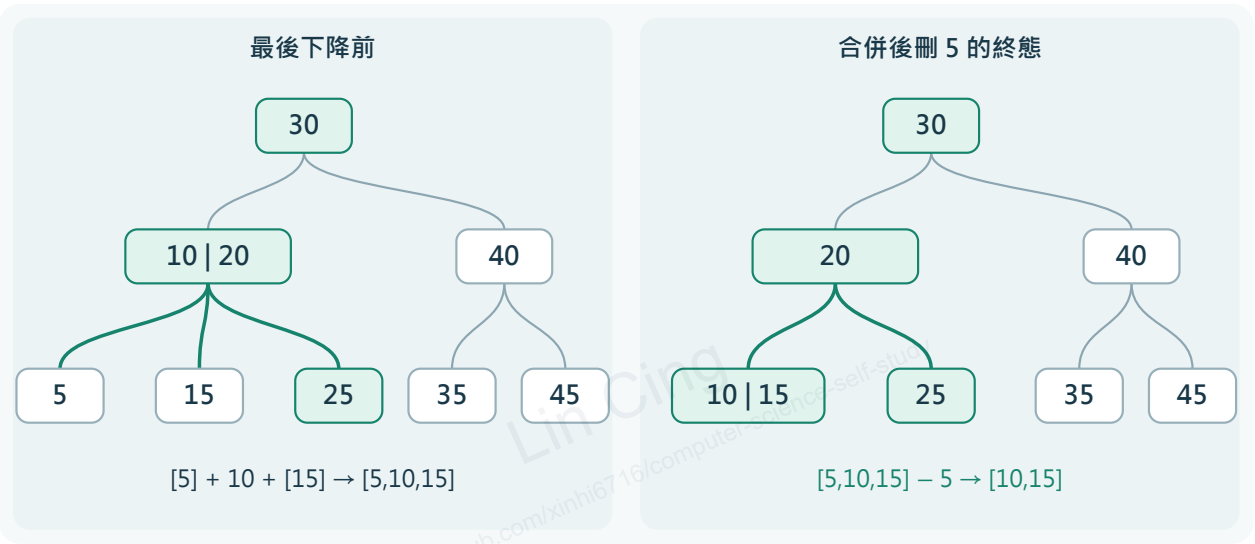
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

MERGE：父分隔鍵也是合併內容的一部分

```
MERGE(p,i):                      // adjacent children both t-1 keys
    y=p.children[i]; z=p.children[i+1]
    y.keys.append(p.keys.pop(i))
    y.keys.extend(z.keys)
    y.children.extend(z.children)
    p.children.pop(i+1)
    write y,p; free z
    return y
```

新 y 有 (t−1)+1+(t−1)=2t−1 鍵，恰好滿而不溢位；若為內節點，有 t+t=2t 個子指標。父少一鍵及一個子指標。非根父進入時至少 t 鍵，現在仍有 t−1；若空的是根，最後統一縮高。

接續刪 5：先合併兩個最小葉



在 [10,20] 內，目標葉 [5] 與右兄弟 [15] 各只有 1 鍵。將父鍵 10 一起合入 [5,10,15]，再刪 5 得 [10,15]。左內節點剩 [20]，連 [10,15]、[25]；根 [30] 和右內節點 [40] 不變。中序恰少 5，所有葉仍深度 2。

內節點命中：前驅與後繼都搬整筆資料

t=2 的初始樹：根及兩個葉	刪除	結果
根 [20]；左 [5,10]、右 [30]	20	前驅 10 上移；根 [10]，葉 [5]、[30]
根 [20]；左 [5]、右 [30,40]	20	後繼 30 上移；根 [30]，葉 [5]、[40]

取前驅時，沿左子樹的最右子指標到葉，取最後一鍵；後繼則沿右子樹的最左子指標到葉，取第一鍵。選取後必須從原子樹刪掉那筆資料，不能讓同一鍵在父、葉各留一份。

記錄指標也要同步搬移；只改鍵、保留父的舊資料，會讓查詢回傳錯誤的 record。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

根縮減：只有根能拿掉整層

根 [20] 與葉 [5]、[30] 合成 [5,20,30]，再刪 20 得 [5,30]。舊根零鍵且只有一個子節點，用該子節點取代，h 從 1 變 0。一般內節點不能照做，否則只縮短一側路徑，破壞葉等深度。

刪除主程序：命中鍵的三種內部情況

```
DELETE-NODE(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k:
        if not x.children: erase x.keys[i]; write x; return FOUND
        L=x.children[i]; R=x.children[i+1]
        if len(L.keys)>=t:
            q=MAX-RECORD(L); DELETE-NODE(L,q.key); x.keys[i]=q
        elif len(R.keys)>=t:
            q=MIN-RECORD(R); DELETE-NODE(R,q.key); x.keys[i]=q
        else:
            y=MERGE(x,i); DELETE-NODE(y,k)
        write x; return FOUND
    return DELETE-DESCEND(x,i,k)  // next card
```

此處 keys 陣列存整筆記錄，比較使用記錄的 key；q 是保存下來的記錄值或穩定指標，不可在刪除原位置後讀取已釋放的物件。

刪除下降：借鍵優先，合併時修正索引

```
DELETE-DESCEND(x,i,k):
    if not x.children: return NOT_FOUND
    if len(x.children[i].keys)==t-1:
        if i>0 and len(x.children[i-1].keys)>=t:
            BORROW-LEFT(x,i)
        elif i<len(x.keys) and len(x.children[i+1].keys)>=t:
            BORROW-RIGHT(x,i)
        elif i<len(x.keys): MERGE(x,i)
        else: MERGE(x,i-1); i-=1
    return DELETE-NODE(x.children[i],k)
```

往左合併時，原 child i 被併入 child i−1，必須 i−=1。對外 DELETE(k) 呼叫 DELETE-NODE(root,k)，**不論 FOUND 或 NOT_FOUND，最後都檢查零鍵內根並縮高**，然後回傳原本的搜尋結果。

找不到鍵，也可能合法地改變樹形

根 [20]、葉 [5]、[30]，t=2，要求刪 99。先選右子節點；右葉只有 t−1=1 鍵，左兄弟也無法借，所以先合併成 [5,20,30]，索引從 1 改為 0。

在新葉找不到 99，回傳 NOT_FOUND；外層仍須縮掉零鍵內根。最後是葉根 [5,20,30]，集合完全不變，高度從 1 降到 0。

NOT_FOUND 表示集合沒有刪掉元素，不保證樹形完全不變。 若 API 要求失敗時連樹形都不動，可在開始前另做存在性搜尋，並將這條額外路徑算入成本。空樹與刪最後一鍵後皆保留零鍵葉根。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除的一般證明與兩種成本

以剩餘高度歸納：葉直接刪除正確；內部替換後遞迴刪掉前驅或後繼，集合只少 k。借鍵與合併保持全域鍵序、子指標數及葉等深度，並讓下一個子問題至少有 t 鍵。非根父即使因合併少一鍵，仍保有 t−1。根例外由外層縮高處理。每次遞迴嚴格下降，故終止。

每層只碰常數個相鄰頁，I/O 為 $O(h+1)=O(1+\log_t(n+1))$。頁內陣列搬移最多 O(t)，插入、刪除 CPU 上界 O(t(h+1))。前驅／後繼若先找再刪，多走的路徑仍總計 O(h+1)，不是每層都重走整棵樹。

純搜尋在頁內二分，可用 O((h+1)log t) 比較；這不會把插入、刪除的陣列搬移也降為 O(log t)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci heap：把整理留到取出最小值時

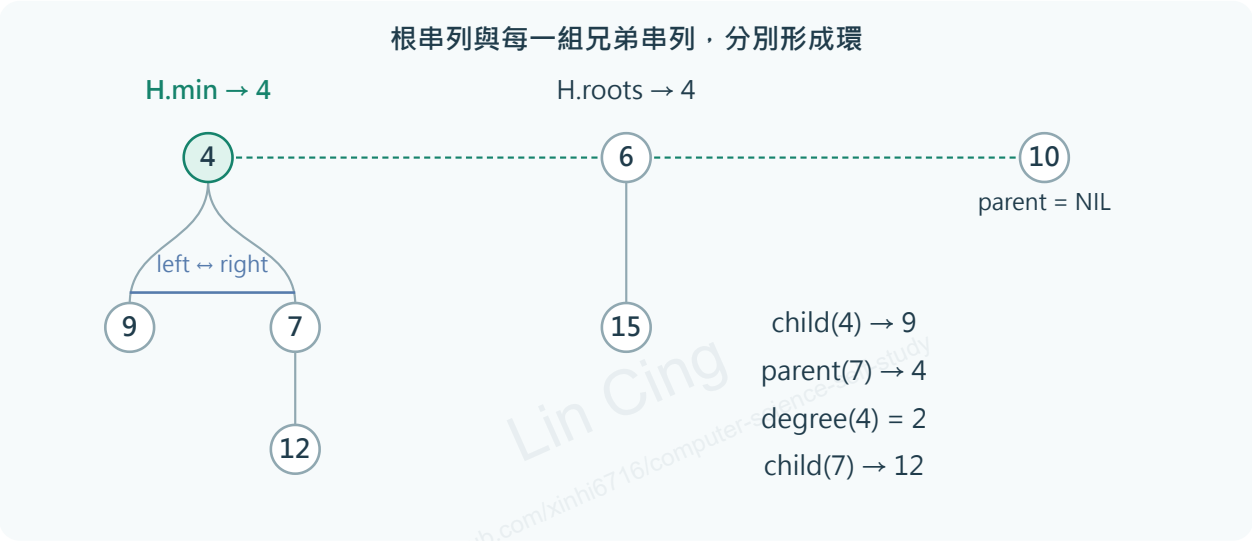
最小堆積只要求父鍵不大於子鍵，不要求不同子樹彼此排序。Fibonacci heap 保留一串滿足堆積序的樹：插入或合併時先接上根串列，直到 EXTRACT-MIN 才合併度數相同的根。

度數（degree）是直接孩子的個數，不是樹高。 經過切割，樹不再保證是完整二項樹。後面將證明最大度數為 O(log(n+1))；這個結論不代表根到葉的路徑也只有對數長度。

DECREASE-KEY(H,x,k) 與 DELETE(H,x) 都要求給定有效的節點參照（handle）。只給鍵值、再尋找對應節點，可能需要走遍整個堆積；這筆搜尋成本另計。相同鍵可以共存，但節點身分不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

指標各管一件事：根串列、子串列與最小值



圖中根為4、6、10；4的孩子是9、7，7還接著12。根串列入口與最小值指標用途不同；每組兄弟另有自己的環狀串列。

欄位保存什麼？標記從哪一次連結開始算？

節點保存 `key`、`parent`、`child`、`left`、`right`、`degree`、`mark`。`child` 指向任一孩子，其兄弟以環狀雙向串列相連；只有一個孩子時，該節點的 left、right 都指向自己。H.roots 是根串列入口，H.min 指向最小根，H.n 記錄總節點數；空堆積的兩個入口皆為 NIL。

標記 記錄非根節點在「這一次接到目前父節點之後」，是否已失去一個孩子。 第一次失去孩子先標記，第二次就切離父節點。這不是終生累積次數；節點切成根、之後重新接到另一個父節點時，紀錄重新開始。

本節統一讓根的 mark=false；提升為根，或重新連結成孩子時，都清除標記。這項慣例也用於後面的勢能計數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙向環串列：能常數時間合併的實際原因

刪除已知節點 x，只要接 x.left.right=x.right、x.right.left=x.left；若 x 是入口就換入口，若是獨生元素就設 NIL，最後讓 x 左右指回自己。只用 O(1) 次指標更新。

兩個非空環入口 a、b，先存 al=a.left、bl=b.left，再接 al.right=b、b.left=al、bl.right=a、a.left=bl，即把兩環接成一環。空環直接取另一入口。

以下 ATTACH-ROOT(H,x) 將已脫離原串列的 x 接到根串列，設 parent=NIL、mark=false；REMOVE-CHILD(p,x) 維護 p.child、環指標及 度數。這些基本操作都不搜尋鍵，也不走訪全部兄弟。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

INSERT、MINIMUM 與破壞式 UNION

操作	完整處理	實際成本
MAKE	roots=min=NIL，n=0	O(1)
INSERT(x)	新節點 degree=0、child=NIL；接根、比較 min、n 加一	O(1)
MINIMUM	回 min；空堆積回 NIL	O(1)
UNION(H1,H2)	接兩根環，取兩個非空 min 的較小者，n 相加	O(1)

UNION 消耗兩個互不重疊的舊堆積；結果取得全部節點，原入口不可再拿來獨立操作。H1=H2 不允許。兩者皆空時結果仍空。這個界不包含逐節點重寫「屬於哪個 heap」欄位；若實作要求那種更新，不能沿用 O(1) 合併界。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

EXTRACT-MIN：先提升孩子，再整理根

```
EXTRACT-MIN(H):
    z=H.min
    if z==NIL: return NIL
    while z.child!=NIL:
        c=z.child
        REMOVE-CHILD(z,c)
        ATTACH-ROOT(H,c)
    REMOVE-ROOT(H,z)
    H.n-=1
    if H.roots==NIL: H.min=NIL
    else: CONSOLIDATE(H)
    return z
```

每次取目前 孩子 再移除，避免一邊修改環串列、一邊用失效的 next 繼續走。所有提升節點清 父節點 與 標記；z 被隔離後才交回呼叫者。若只有一個根但它還有孩子，提升後堆積仍非空，不能只憑「原來一個根」就回空。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CONSOLIDATE：讓每個 度數 最後只留一個根

```
CONSOLIDATE(H):
    A=empty degree table; H.min=NIL
    while H.roots!=NIL:
        x=POP-ROOT(H)              // isolate one root
        d=x.degree
        while A[d]!=NIL:           // absent slot means NIL
            y=A[d]; A[d]=NIL
            if y.key<x.key: swap(x,y)
            LINK-ISOLATED(y,x)      // y.parent=x; y.mark=false
            d=x.degree
        A[d]=x
    for each non-NIL x in A:
        ATTACH-ROOT(H,x)
        if H.min==NIL or x.key<H.min.key: H.min=x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LINK 與 度數 表的實作契約

LINK-ISOLATED 把同 度數 的兩個獨立根合成一棵：較大鍵 y 成為 x 的 孩子，x.degree 加一；相等鍵選任一根。A 可用按需延伸的陣列，最多 O(D(n)+1) 槽。上面採逐根取出的版本，避免重複處理已被連結的舊根。

```
LINK-ISOLATED(y,x):
    require x!=y; x.degree==y.degree
    require x.key<=y.key; both roots isolated
    y.parent=x; y.mark=false
    splice y into x.child circular list
    x.degree+=1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

整理根的不變量與終止理由

每次外層開始時，尚未處理的根留在 H.roots，已處理節點則分布在 A 指向的互不重疊子樹中；A 每個槽至多存一棵 度數 等於槽號的樹。所有原節點恰好出現在這兩部分之一。

內層維持 d=x.degree；遇到同 度數 的 y 就連結，節點集合與堆積序不變，兩根變一根，d 增一。原有根數有限，每次 link 都讓森林根數減一，故不可能無限循環。

外層每次取掉一個未處理根，因此結束時 A 包含全部剩餘樹，且 度數 各異。重建根串列並逐根比較最小值，得到正確 H.min。注意「度數 各異」只在整理完成後保證；INSERT、UNION、CUT 都可再次造成重複。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

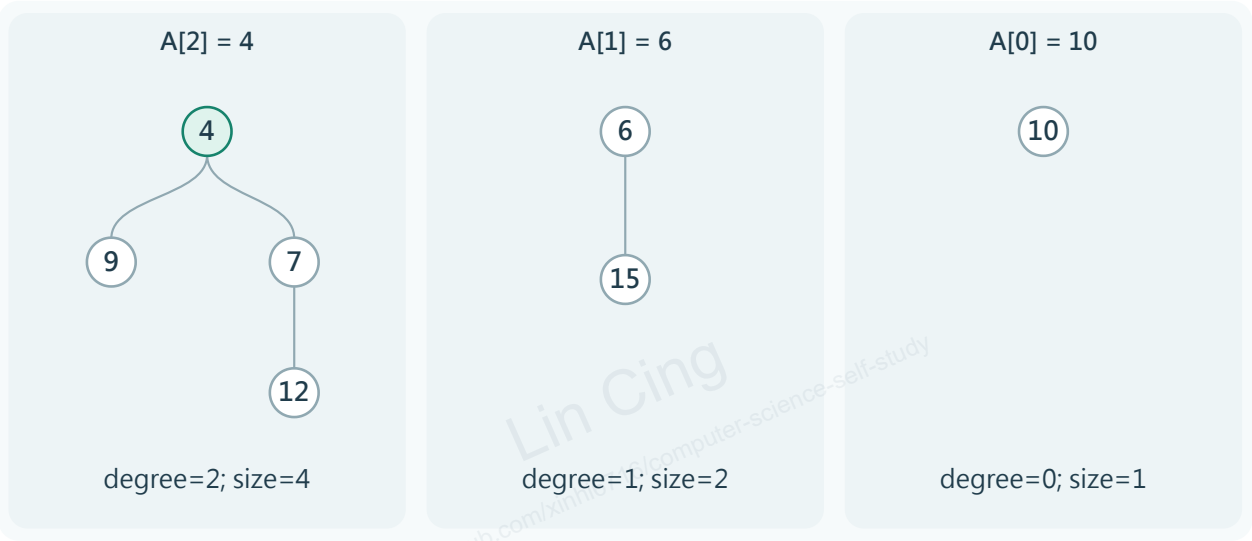
數例：四次 link，逐槽追蹤整理過程

從八個單節點根 1、4、9、7、12、6、15、10 開始，先取出最小根 1，再按 4、9、7、12、6、15、10 的順序整理；它們的初始度數皆為 0。

取出根	這一輪的 link	非空 A 槽（度數:root）
4	無	0:4
9	9 掛到 4	1:4
7	無	0:7，1:4
12	12 掛到 7；再把 7 掛到 4	2:4
6	無	0:6，2:4
15	15 掛到 6	1:6，2:4
10	無	0:10，1:6，2:4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

數例終態：樹形、度數 與根數一起核對



根 度數 依序為2、1、0；節點數4+2+1=7，min=4。綠色根4是最小值，其餘根不必按鍵值排序。

同一數例：根數下降如何釋出勢能

根 4 的孩子是 9、7，7 的孩子是 12；根 6 的孩子是 15；根 10 沒有孩子。三個根的度數分別為 2、1、0，總節點數為 4+2+1=7，每條父子邊都滿足父鍵≤子鍵。

階段	根數 r	標記數 m	勢能 K(r+2m)
取出 1 之前：八個單節點根	8	0	8K
移除 1、完成四次連結之後	3	0	3K

這次勢能下降 5K：移除最小根使根數少一，再由四次連結少四。它把「先前插入時存下的額度」接回後面的攤銷公式；**5K 是勢能變化，不是五條機器指令的實際成本。**

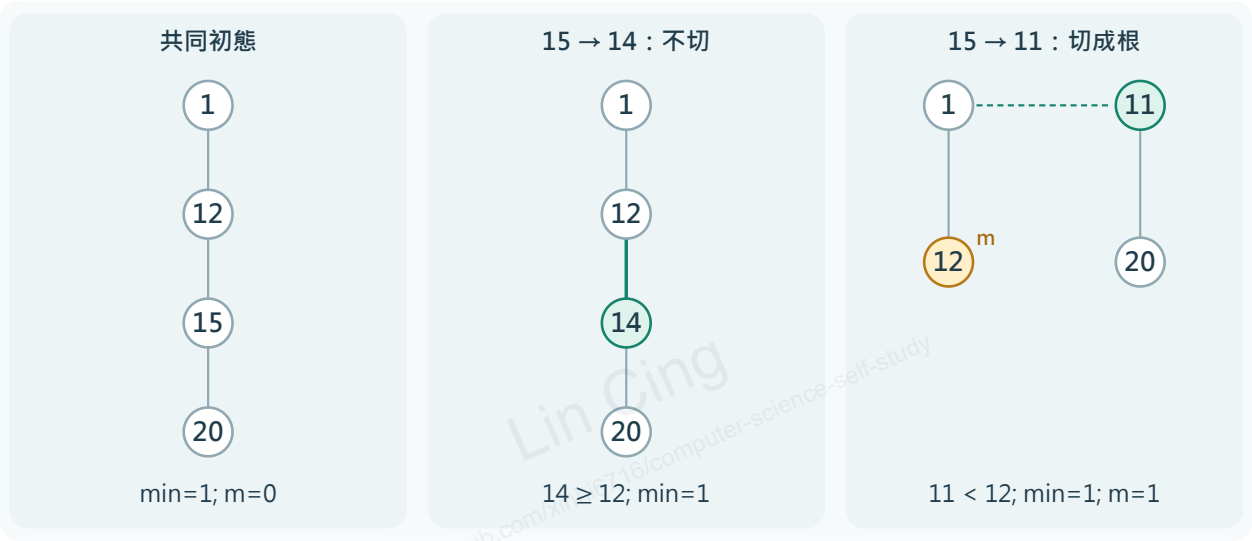
本例恰為未經切割的二項樹。發生 CUT 後，度數 d 不再保證有 2^d 個節點；後面的通用下界改為 F_{d+2}。

DECREASE-KEY：只有違反父子順序才切

```
DECREASE-KEY(H,x,k):
    if k>x.key: error
    x.key=k; p=x.parent
    if p!=NIL and x.key<p.key:
        CUT(H,x,p)
        CASCADING-CUT(H,p)
    if x.key<H.min.key: H.min=x

CUT(H,x,p):
    REMOVE-CHILD(p,x)       // degree decreases by one
    ATTACH-ROOT(H,x)       // parent=NIL; mark=false
```

只降低 x 的鍵，不會破壞 x 與孩子的順序；可能壞掉的只有 父節點→x。若 x 已是根，或新鍵仍≥父鍵，皆不切。min 只可能保留舊值或改成 x，其他節點鍵值沒有改變。

數例：15→14 不動；15→11 帶著子樹上來



兩個情況都從左圖開始。降低至14不切；降低至11則連孩子20一起提升，並標記第一次失去孩子的非根12。

兩種遞減鍵值，從同一初態分開追蹤

根 1 的孩子 12 有孩子 15，15 又有孩子 20。兩個情況各自從這個初態開始：

新鍵	與父鍵 12 比較	指標與標記的變化
14	14≥12	不切割、不改標記
11	11<12	連同孩子 20 一起切成新根；12 若原本未標記，便標記

切割搬動的是**以該節點為根的整棵子樹**，不是只取出一個鍵；20 仍是新鍵 11 的孩子。第二種情況中，12 是非根且首次失去孩子，因此留下標記，根 1 不受影響。

兩種結果的最小值都仍是 1。若降低的是既有根，例如把根 3 改為 0，只更新最小值指標，不會切割。

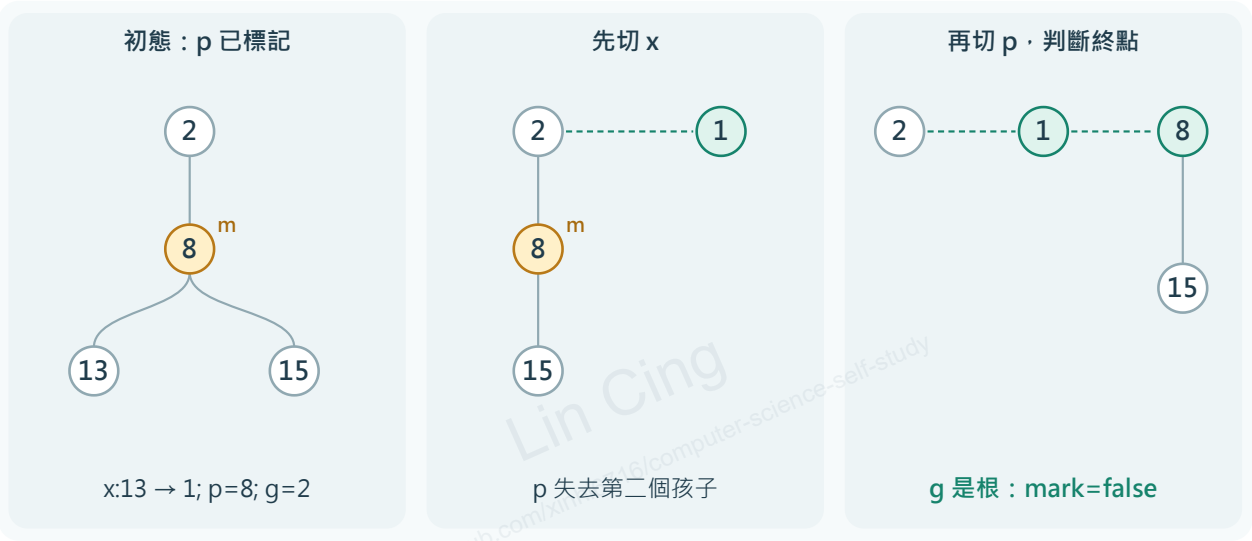
CASCADING-CUT：第一次失去孩子先標記，第二次才切走

```
CASCADING-CUT(H,y):
    while y.parent!=NIL:
        p=y.parent
        if y.mark==false:
            y.mark=true
            return
        CUT(H,y,p)
        y=p
    return                      // root: stop, do not mark
```

呼叫時 y 剛失去一個 孩子。未標記表示這是本次成為 孩子 後的第一次損失，留下記號便停；已標記表示第二次損失，把 y 切成根並清 標記，父節點 p 也因此失去孩子，所以繼續處理 p。

堆積序仍正確：切掉父子邊不會讓剩餘邊失序。級聯的作用是限制非根能失去多少孩子，保住 度數 與子樹大小之間的關係，而不只是修復鍵值順序。

級聯分支一：祖父是根，就停在根



先切x，再切已標記的p。到g時它已是根，所以停止；不新增g的標記。p原有的孩子15仍跟著p。

到根就停：根串列與勢能逐項核對

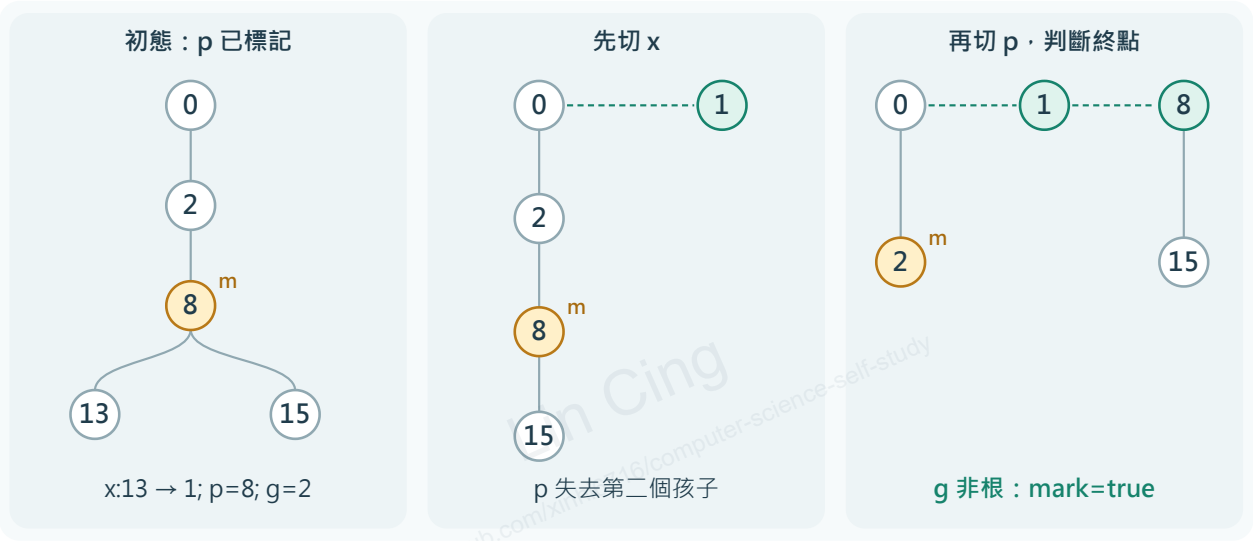
初態只有根 g=2，其孩子 p=8 已標記；p 的孩子為 x=13、s=15。p 的標記表示它在這次成為孩子之後，先前已失去另一個孩子。現在把 x 從 13 降到 1。

先切 x，再切已標記的 p。到 g 時，g.parent=NIL，因此停止，**不把根 g 標記**。最後根為 2、1、8；8 仍帶著孩子 15，最小值改為 1。

狀態	根數 r	標記數 m	未縮放勢能 r+2m
操作前	1	1	3
切 x、切 p，抵達根 g 後	3	0	3

新增兩個根增加 2，清除 p 的標記減少 2，勢能淨變化為 0。這裡先核對 r+2m；實際採用的勢能還會乘上一個固定常數 K。

級聯分支二：祖父非根且未標記，才留下記號



這次g還有父節點q，且原本未標記。切x、p之後，標記g而停；因為根q=0仍最小，min不變。

尚未到根：清除 p 的標記，再標記 g

在前一個局部結構上方接根 q=0，令 g=2 為未標記非根，而 p=8 仍已標記。把 x=13 降到 1：先切 x、再切 p；抵達 g 時，g.parent=q 且 g.mark=false，所以標記 g 後停止。

最後根為 q=0、x=1、p=8；q 仍接 g=2，p 仍接 s=15，最小值保持 0。

狀態	根數 r	標記數 m	未縮放勢能 r+2m
操作前	1	1	3
p 清標記、g 新增標記後	3	1	5

若 g 原本也已標記，就把 g 一起切走，繼續處理 q。**停止位置由父節點與標記共同決定**；「祖父被標記」只適用於祖父非根且先前未標記的分支。

同一個勢能同時支付所有操作

取固定常數 K>0，對所有現存且互不重疊的堆積定義

$$\Phi = K \sum_{H} \big(r(H) + 2m(H) \big), \qquad \hat{c}_i = c_i + \Phi_i - \Phi_{i-1}.$$

K 只需選一次，大到能支付以下常數時間指標操作，之後不可隨操作更換。由空堆積開始 Φ₀=0 且始終非負，故 Σcᵢ≤Σĉᵢ。若從已有森林開始，總界須加初始 Φ₀。

INSERT 增一根、不增 標記，因此 ΔΦ=K，攤銷仍 O(1)。MINIMUM 與 MAKE 不改勢能。UNION 將兩個舊堆積替換成一個，根數與標記數相加，總勢能不變，因此實際與攤銷都 O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何一串級聯切割只收常數攤銷成本？

若沒有切割，實際與勢能變化都 O(1)。否則令 c≥1 為總切割次數，包括最初切 x；其餘 c−1 次都切掉已標記的非根節點，並清除它們的 標記。結尾至多新標記一個非根，x 的舊標記若存在也被清除，因此

$$\Delta r = c, \qquad \Delta m \le 1 - (c-1) = 2 - c,$$
$$\Delta \Phi \le K(4 - c).$$

存在固定 a，使實際成本≤a(c+1)。選 K≥a，得到

$$\hat{c} \le a(c+1) + K(4-c) \le a + 4K = O(1).$$

這是對任何合法操作序列的保證，不需要隨機輸入。單次實際切割仍可能沿長路徑花線性時間；O(1) 是攤銷界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

EXTRACT-MIN 的成本：根多時，用先前存的勢能付款

設操作前有 n 節點、r 個根，最大 度數 上界為 D=D(n)。取出最小根後，待整理根數 R≤r−1+D。每次 link 讓根數減一，所以全部 link 次數≤R；處理孩子、掃根、清空與重建 度數 表，實際成本 ≤a(r+D+1)。

整理後至多 D+1 個根。提升與 link 都只清 標記、不新增，因此

$$\Delta\Phi \le K(D+1-r).$$

$$\hat{c} \le a(r+D+1) + K(D+1-r) \le (a+K)(D+1)$$

最後一步用同一個 K≥a；空堆積與單節點另為 O(1)。這裡先得到 O(D+1)，接下來必須證明 D=O(log(n+1)) 才能宣稱 O(log(n+1)) 攤銷時間。連續插入 n 個根後第一次取出，實際可達 Θ(n)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

度數 證明的關鍵：孩子最多只少一個孩子

令 x 現在有 d 個孩子，按它們**最近一次接到 x 的時間**排序為 $y_1, \dots, y_d$。對 i≥2，在 y_i 接上當時，目前仍在的較早孩子 $y_1,\cdots,y_{i-1}$ 都已在 x 下面，因此 x 當時 度數 至少 i−1。

LINK 只合併相同 度數 的根，所以 y_i 當時也至少有 i−1 個孩子。之後若失去第二個孩子就會被切離 x；既然 y_i 現在仍是 x 的孩子，至多只少一個。因此

$$\deg(y_1) \ge 0, \qquad \deg(y_i) \ge i-2 \quad (2 \le i \le d).$$

這不是說兄弟的目前 度數 已排序，也不是說每個非根終生只能失去一個孩子；切成根再被連結後，歷史會重新計算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從孩子下界推到 Fibonacci 子樹大小

對子樹節點數做強歸納，證明任意 degree=d 的節點 x 都有 size(x)≥F_{d+2}。孩子子樹比 x 小，可套歸納假設；F 數列非遞減，配合上一頁的 度數 下界。

$$F_0 = 0, \quad F_1 = 1, \quad F_j = F_{j-1} + F_{j-2} \ (j \geq 2).$$

d=0 時 size≥1=F₂；d=1 時至少自己與一個孩子，size≥2=F₃。d≥2 時，第一個孩子至少一節點，其餘依序給出

$$\text{size}(x) \geq 2 + \sum_{i=2}^{d} F_i = F_{d+2}.$$

最後的恆等式由 $\sum_{i=0}^{d} F_i$=F_{d+2}−1 得到：d=0 為 0=F₂−1；歸納時加上 F_{d+1}，再用 Fibonacci 遞迴即可。這個證法以子樹大小歸納，不偷用尚未證明的「最小大小隨 度數 單調」。

完成 度數 界：指數大小換成對數 度數

令 ϕ=(1+√5)/2，ϕ²=ϕ+1。證明 F_{d+2}≥ϕᵈ：d=0 為 1≥1，d=1 為 2≥ϕ；d≥2 時

$$F_{d+2} = F_{d+1} + F_d \geq \varphi^{d-1} + \varphi^{d-2} = \varphi^d.$$

所以對 n≥1 的堆積中任意節點，n≥size(x)≥F_{d+2}≥ϕᵈ，得到 $d \leq \lfloor \log_\varphi n \rfloor$。因此最大 度數 為 O(log(n+1))，度數 表大小、整理後根數與 EXTRACT-MIN 攤銷成本皆為 O(log(n+1))。

度數 d	0	1	2	3	4	5	6
子樹節點數下界：F_{d+2}	1	2	3	5	8	13	21

這是保證下界，不宣稱每個 d 都能由某個特定操作序列恰好達到等號。空堆積不取 log 0，另行處理。

DELETE：指定節點必須真的成為唯一最小

DELETE(H,x) 可先將 x 的比較鍵改成比所有一般鍵都小的專用哨兵，再 EXTRACT-MIN。原本若為非根，DECREASE-KEY 會正確處理父節點失去孩子及級聯切割；取出後連帶提升 x 的所有孩子並整理。

不能只寫「改成負無限大」卻允許其他節點也有相同比較值；否則 min 可能仍指向別的節點，刪錯身分。可把比較鍵擴充為 (tag,key)，平常 tag=1，僅本次目標 tag=0，確保它唯一最小；刪完再把回傳節點的原鍵還原。

以同一勢能把兩操作相加，中間勢能相消，攤銷成本為 O(1)+O(log(n+1))。直接拆除內節點的另一種實作也可以正確，但必須另處理孩子提升、父的 標記 與級聯；不能只把孩子放到根串列就算完成。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

操作成本總表：先確認輸入與計費範圍

操作	單次實際上界	攤銷上界
MAKE、MINIMUM、INSERT、UNION	O(1)	O(1)
DECREASE-KEY（已知 handle）	O(n+1)	O(1)
EXTRACT-MIN、DELETE（已知 handle）	O(n+1)	O(log(n+2))
只給任意鍵的 SEARCH	O(n+1)	堆積序不會自動改善搜尋界

表中使用 log(n+2)，讓 n=0 的空堆積也保留常數成本；非空時與慣用的 O(log n) 同階。DELETE、DECREASE-KEY 要求有效節點，因此不能在空堆積提供這種 handle；空 EXTRACT-MIN 則合法，直接回傳 NIL。

n 為相關堆積的節點數。UNION 要求兩個輸入互不重疊，並消耗舊入口。成本不包含配置器、逐節點所有權重標記，或尋找 handle 的額外工作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把操作次數換成圖演算法的總成本

圖演算法若有 O(V) 次插入／取出、O(E) 次已知 handle 的遞減鍵值，便得到 O(E+V log(V+1)) 的堆積總成本。是否值得實作仍取決於常數、記憶體與存取模式；漸近界不是實測速度。

例如 V=1000、E=8000，且每個頂點至多插入與取出各一次，每條邊至多觸發一次遞減鍵值：

工作	最多次數	Fibonacci heap 的總攤銷上界
INSERT	1000	O(V)
EXTRACT-MIN	1000	O(V log(V+1))
DECREASE-KEY	8000	O(E)

相加得到 O(E+V log(V+1))；若使用一般二元堆積，對應上界為 O((E+V)log(V+1))。數字是操作次數上限，並非毫秒或可直接比較的精確成本；兩種結構的常數並不相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

利用整數位元，換一種搜尋模型

紅黑樹只比較鍵值；vEB（van Emde Boas tree）還使用鍵的二進位表示。固定集合 $S \subseteq \{0, \ldots, u-1\}$，$u = 2^m$、$m \geq 1$，**n=|S| 與鍵值範圍（universe）大小 u 是不同參數**。本節不存重複鍵。

假設 word RAM 的一個 word 可容納鍵與位址；位移、遮罩、比較、陣列索引皆為常數時間。這不適用於任意精度整數的位元成本模型。

目標是最壞 $O(1 + \log\log u)$ 的查找、插入、刪除、前驅與後繼，最小／最大值為 O(1)。這不違反比較排序下界：演算法使用了額外的整數運算，也須支付初始化與記憶體成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先看少了什麼：直接定址與遞迴分組

直接定址的 bit vector 能在 O(1) 時間查找、插入與刪除，但若 $S = \{0, u-1\}$，從 0 向右掃描後繼可能花 Θ(u)。在 bit vector 上建立二元摘要樹，可把此查詢降為 O(log u)。

把 universe 分成約 $\sqrt{u}$ 組，再把各組與非空組號都遞迴表示，仍**不保證** O(log log u)：若先遞迴找組號，再遞迴找該組最小值，會有

$$T(u) = 2T(\sqrt{u}) + O(1), \quad R(m) = 2R(m/2) + O(1) = \Theta(m).$$

因此成本仍是 Θ(log u)。關鍵不只在縮小 universe，還要讓每層最多留下**一次非定值成本的遞迴**；min/max 快取就是為此而設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

奇數位元寬度：高位與低位不能各除一半

令 $\ell = \lfloor m/2 \rfloor$、$h = \lceil m/2 \rceil$，$L = 2^\ell$、$H = 2^h$，所以 u=HL。對 u>2：

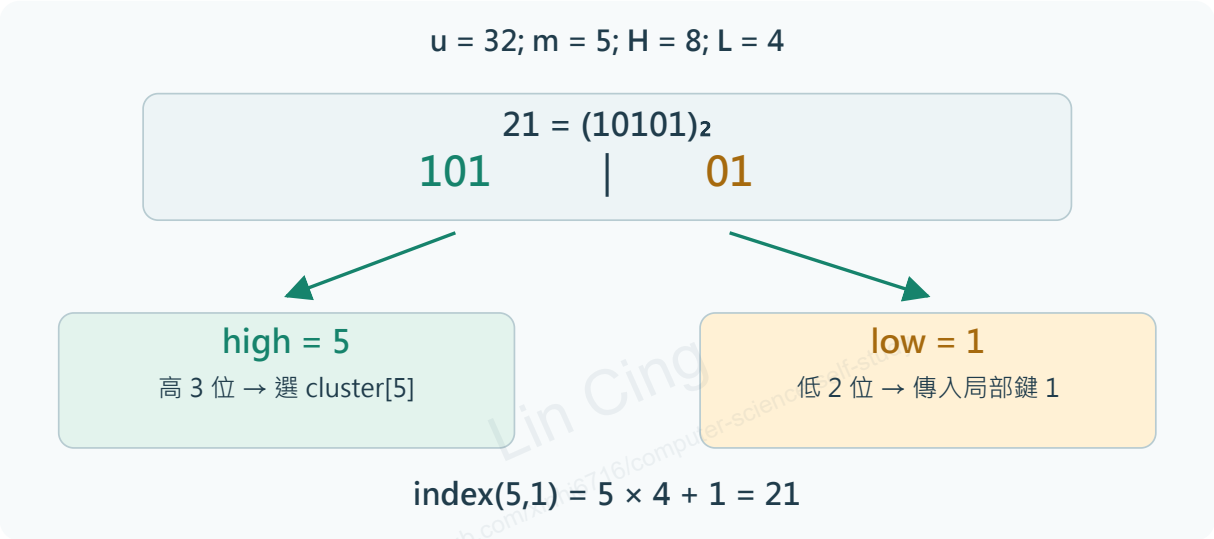
$$\text{high}(x) = \lfloor x/L \rfloor, \quad \text{low}(x) = x \bmod L, \quad \text{index}(i,j) = iL + j.$$

有 H 個分組（cluster），每組的 universe 大小為 L；摘要（summary）存 cluster 編號，其 universe 大小為 H。**每層都依該節點的 u 重新計算**，不能沿用頂層遮罩。

u=32 時，m=5、h=3、ℓ=2、H=8、L=4。$x = 21 = (10101)_2$ 切成 $101 \mid 01$，所以 high=5、low=1，還原為 5×4+1=21。u=2 直接處理，不建立下一層。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一個鍵的兩個座標



high 選 cluster；low 是傳給該 cluster 的鍵。summary 裡的 5 表示「第 5 組非空」，不是原集合真的存了鍵 5。這兩種資料不能混成同一組鍵值。

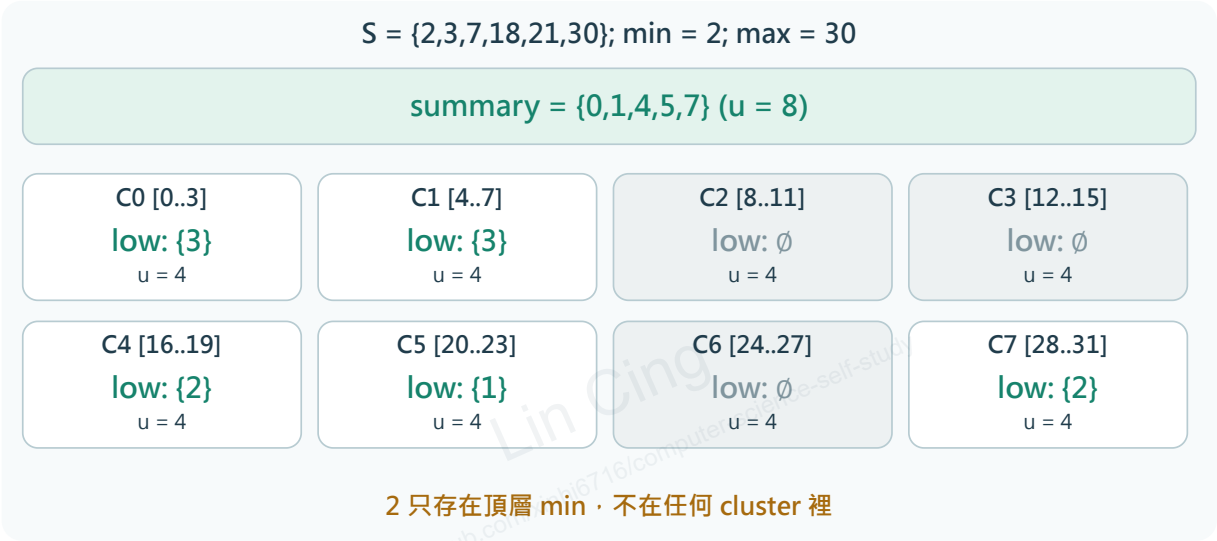
表示不變量：最小值不放進任何 cluster

每個節點保存 u、min、max；u>2 時另有 summary 與 cluster[0..H−1]。採用預先建立的完整結構，空節點的 min=max=NIL。

非空時，min 是該集合最小值；所有其他鍵依 high／low 分到 cluster。**min 本身不存入 cluster**，所以單一元素時所有 cluster 與 summary 都空。當至少兩鍵時，max 也存在某個 cluster。

summary 恰存非空 cluster 的編號：$i \in \text{summary} \iff C_i.\min \neq \text{NIL}$。因此 min=max 可在 O(1) 判定單一元素；向空節點插入只需設定 min=max=x，不必往下填滿一條路。

u=32 的集合：原鍵、組內鍵與摘要分開看



原集合為 {2,3,7,18,21,30}。頂層 min=2、max=30；C_0 只存 low=3，不再存 2。summary={0,1,4,5,7}。圖只展開頂層；每個 C_i 與 summary 仍各自遵守相同不變量。

查找與邊界：先保證參數在 universe 內

```
MEMBER(V,x):                  # require 0<=x<V.u
    if V.min==NIL: return false
    if x==V.min or x==V.max: return true
    if V.u==2: return false
    return MEMBER(V.cluster[high(x)],low(x))
MINIMUM(V): return V.min
MAXIMUM(V): return V.max
```

high／low 皆使用 V 的尺寸。查找沿一個 cluster 向下；快取命中時提早返回。NIL 必須是獨立狀態，不能用合法鍵 0 代替。

本節後繼定義為集合中**嚴格大於 x** 的最小鍵，前驅為嚴格小於 x 的最大鍵；x 不必在集合中，但須在 universe 內。越界參數由公開介面拒絕，避免索引負值或 H。

後繼：用 max 決定遞迴走哪一邊

```
SUCCESSOR(V,x):
    if V.min==NIL: return NIL
    if x<V.min: return V.min
    if x>=V.max: return NIL
    if V.u==2: return 1
    i=high(x); j=low(x); C=V.cluster[i]
    if C.max!=NIL and j<C.max:
        return index(i,SUCCESSOR(C,j))
    k=SUCCESSOR(V.summary,i)
    if k==NIL: return NIL
    return index(k,V.cluster[k].min)
```

若同組 max 比 j 大，後繼就在同組；否則同組沒有答案，直接問 summary 的下一個非空組。**不用先遞迴失敗一次再換組**。找到下一組後讀 min 是 O(1)。

後繼數例：同組與跨組共用一份狀態

同組先看快取 max；跨組只問 summary。兩條路徑互斥。

逐步核對後繼查詢的座標

以 {2,3,7,18,21,30} 為準。SUCCESSOR(3)：i=0、j=3，C_0.max=3，不可留在同組；summary 的後繼組號是 1，讀 C_1.min=3，還原 1×4+3=7。

SUCCESSOR(20)：i=5、j=0，C_5.max=1>0，因此只遞迴 C_5，得 offset=1，還原為 21。20 不在集合中仍可查詢。SUCCESSOR(30)=NIL，因為沒有嚴格更大的鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

前驅：不能忘記獨立保存的 min

```
PREDECESSOR(V,x):
    if V.min==NIL: return NIL
    if x>V.max: return V.max
    if x<=V.min: return NIL
    if V.u==2: return 0
    i=high(x); j=low(x); C=V.cluster[i]
    if C.min!=NIL and j>C.min:
        return index(i,PREDECESSOR(C,j))
    k=PREDECESSOR(V.summary,i)
    if k==NIL: return V.min
    return index(k,V.cluster[k].max)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最後的 k=NIL 分支不是無條件回傳 NIL：前面已知 x>V.min，且更小的非空 cluster 不存在，所以答案正是未放入 cluster 的 min。例如 PREDECESSOR(3)=2；即使 C_0 裡沒有更小的 low，也不能漏掉頂層的 2。

插入：換出舊 min，並只更新必要的摘要

```
INSERT-ABSENT(V,x):          # require x not in V
    if V.min==NIL: V.min=V.max=x; return
    if x<V.min: swap(x,V.min)
    if V.u>2:
        i=high(x); j=low(x); C=V.cluster[i]
        if C.min==NIL:
            INSERT-ABSENT(V.summary,i)
            C.min=C.max=j
        else: INSERT-ABSENT(C,j)
    if x>V.max: V.max=x
```

新鍵若小於 min，先交換，接著插入的是**舊 min**。空 cluster 的組號尚未在 summary，故遞迴插入組號，再以 O(1) 建立單一元素 cluster；非空 cluster 的組號已存在，只遞迴該 cluster。兩條昂貴路徑互斥。

插入 1：新 min 在外，舊 min 回到 C_0

由原集合 {2,3,7,18,21,30} 插入 1：

步驟	頂層／局部狀態	要維護的不變量
交換	min=1；待插入 x=2	新 min 不放 cluster
分組	high(2)=0、low(2)=2	用頂層 L=4
進 C_0	原 C_0={3}，插入後 {2,3}	C_0 自己的 min=2、max=3
摘要	仍為 {0,1,4,5,7}	C_0 原本就非空
結束	max=30	原最大值未改

C_0 的 u=4，所以它的下一層改用 L=2：舊局部 min=3 被放到其 C_1 的 low=1。不可把頂層 low=2 再當成「整棵樹的鍵 2」而混用座標。

刪除：先處理單一元素與 u=2

DELETE-PRESENT 的前提是 x 確實存在。先處理以下兩種狀態，再進入下一頁的一般情況：

```
DELETE-PRESENT(V,x):
    if V.min==V.max:
        V.min=V.max=NIL; return
    if V.u==2:
        V.min=V.max=1-x; return
    DELETE-GENERAL(V,x)
```

第一個分支中 min=max 必為有效鍵，因為呼叫契約排除了空集合；刪除後所有 cluster 原本就空。u=2 且不是單一元素時，集合必為 {0,1}，刪除 x 後只剩 1−x。

若直接對空集合或不存在的鍵呼叫，這段程式可能清掉合法資料。公開介面先 MEMBER，未找到就回傳 NOT_FOUND；不能把這項前提藏在解答裡。

一般刪除：提升下一個 min，再清掉其 cluster 副本

```
DELETE-GENERAL(V,x):         # u>2; at least two keys
    if x==V.min:
        k=V.summary.min
        x=index(k,V.cluster[k].min); V.min=x
    i=high(x); j=low(x); C=V.cluster[i]
    DELETE-PRESENT(C,j)
    if C.min==NIL:
        DELETE-PRESENT(V.summary,i)
        if x==V.max:
            k=V.summary.max
            if k==NIL: V.max=V.min
            else: V.max=index(k,V.cluster[k].max)
    elif x==V.max:
        V.max=index(i,C.max)
```

刪除時的 x，為什麼會改成新 min？

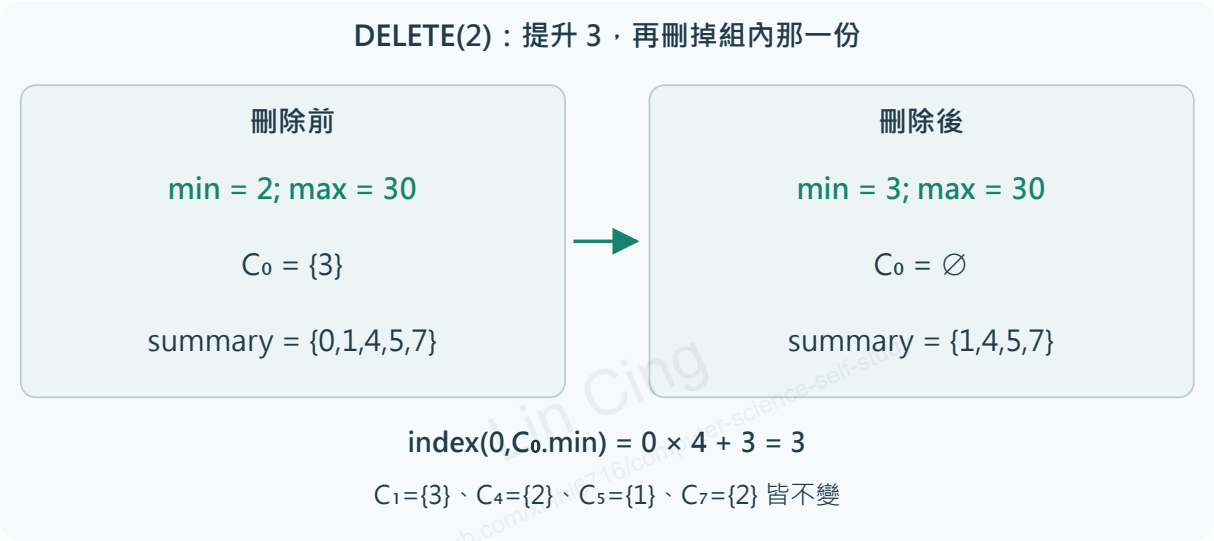
變數 x 在提升時改成新 min 的值；接下來刪的是它在 cluster 中的那份表示。先更新 cluster 與 summary，再重算 max，不能讀取已經失效的舊最大組號。

若集合只有 {2,3} 且刪除 2：提升後 x=3、min=3；C_0 的 3 被刪空，summary 也清空。此時 x 等於舊 max=3，所以設定 max=min=3，留下合法的單一元素集合。

若還有 7、18、21、30，提升後的 x=3 不等於 max=30，最大值便不變。這裡比較的是程式目前的 x，不能在某一行偷換回原刪除參數 2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除 2：提升 3，不是把所有鍵往前搬



3 提升成頂層 min，C_0 清空，summary 刪掉組號 0。

逐步核對提升與摘要刪除

從原集合重新開始：summary.min=0、C_0.min=3，故新 min=0×4+3=3。刪除 C_0 裡的 3 後 C_0 變空，再從 summary 刪掉組號 0，得到 {1,4,5,7}。max=30 不變。

新 min=3 **只在頂層保存**；若還留在 C_0，就破壞表示不變量，也會讓後續刪除／單一元素判斷錯誤。此例中 cluster 的刪除是單一元素分支，只花 O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除最大值與最後兩個元素

仍由原集合出發，連續刪除 30、21、18、7、3、2：

刪除鍵	刪除後 min／max	頂層 summary
30	2／21	{0,1,4,5}
21	2／18	{0,1,4}
18	2／7	{0,1}
7	2／3	{0}
3	2／2	空
2	NIL／NIL	空

刪除 3 後沒有非空 cluster，max 必須回到獨立的 min=2；不能從空 summary 取得組號再索引陣列。刪除最後的 2 只清 min/max，已經沒有其他表示需要清理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

刪除有兩次呼叫，為何仍只剩一條昂貴路徑？

一般刪除確實可能先呼叫 DELETE(cluster)，再呼叫 DELETE(summary)。但第二次只有在 cluster 被刪空時才發生，而刪空表示該 cluster **原本只有一個元素**，第一次呼叫立即走 O(1) 的單一元素分支。

cluster 原本狀態	cluster 刪除成本	是否刪 summary 組號
單一元素	O(1)	是，可能遞迴
至少兩個元素	可能遞迴	否，刪除後仍非空

因此成本是 $\max\{T(L), T(H)\} + O(1)$，不是 $T(L) + T(H) + O(1)$。這是由表示不變量與分支條件推出的**最壞界**，不需要用機率或攤銷分析掩蓋第二次呼叫。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用位元寬度解遞迴式

設 T(u) 為任一操作的最壞上界，$R(m) = T(2^m)$。每層只有一次非定值遞迴，子節點位元數最多為 $\lceil m/2 \rceil$：

$$R(m) \le R(\lceil m/2 \rceil) + c, \qquad R(1) = O(1).$$

對 m≥2，$\lceil m/2 \rceil \le 2m/3$；所以至多 $\lceil \log_{3/2} m \rceil$ 個遞迴層後抵達基例。每層 O(1)，得到 $O(1 + \log m) = O(1 + \log \log u)$。

m=5 的最長位元路徑為 5→3→2→1；cluster 路徑也可能是 5→2→1。**取決於走 summary 或 cluster**，不能把兩種 universe 大小都寫成精確的 √u。u=2 時成本是常數，而不是 log log 2=0 次工作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正確性：每個操作如何維持集合表示

以 m 作歸納。m=1 可列出空、{0}、{1}、{0,1} 四種合法集合。較大節點的集合由獨立 min 與各 cluster 還原值的聯集唯一決定。

查詢：min/max 守住邊界；同組存在答案時，其餘更高組不可能更小。否則 summary 找到下一個非空組，再讀該組 min。前驅另外納入未放進組內的 min。

插入：新 min 與舊 min 交換，只改變保存位置，不遺失鍵；將待插入鍵加入正確 cluster，且只在該組首次非空時加入摘要。

刪除：刪 min 時提升目前最小的 cluster 鍵，再刪其組內副本；其餘情況只刪目標鍵。空組同步退出摘要，max 由更新後狀態重建。歸納假設保證子操作正確，因此每步恰好實現目標集合變化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

建立空結構也有成本：Θ(u) 空間與初始化

本節採預先配置的完整樹，不把配置藏進 O(1) 的空節點插入。u>2 時，H 個 cluster 各有 universe L，另有 universe H 的 summary；本層指標與欄位成本至多 cH：

$$P(u) \le HP(L) + P(H) + cH, \qquad P(2) \le p_0.$$

可用強歸納證明 $P(u) \le A(u-1) - B$。選 B≥c、A≥B+p₀，則子項代入後：

$$P(u) \le A[H(L-1) + (H-1)] - B(H+1) + cH \le A(u-1) - B.$$

因此 P(u)=O(u)。只計 cluster 分支的底層節點已需 Ω(u) 空間，故為 Θ(u)；逐節點建立與清空也需 Θ(u) 時間。此推導同時容許 H≠L。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

公開介面、重複鍵與空間取捨

情境	本節的完整規格
重複 INSERT(x)	先 MEMBER；存在就回傳 ALREADY_PRESENT
不存在的 DELETE(x)	先 MEMBER；不存在就回傳 NOT_FOUND
x<0 或 x≥u	在任何索引前拒絕
需要重複鍵	另維護 multiplicity；計數由 0↔1 時才改鍵集合
需要附帶資料	另定義 key→record 對應與所有權

多一次 MEMBER 仍是兩個 O(1+log log u) 的相加，不改漸近界。重複鍵的計數假設可在常數時間讀寫、且不溢位；不能直接把重複鍵丟入 INSERT-ABSENT。

完整樹適合可負擔 u 的情境。若改成按需配置與 hash table，需要重新分析空間、配置與期望時間；**不能直接沿用本節的確定性最壞界**。n 很小而 u 很大時，O(n) 空間的平衡樹通常更合理。

把攤銷、期望與最壞保證放回各自的前提

結構／操作	保證	主要前提
紅黑樹查詢／更新	最壞 O(log(n+2))	比較鍵；O(n) 空間
雜湊查找	期望 O(1+α)	宣告的雜湊機率模型；不支援排序鄰居
Fibonacci 遞減鍵值	攤銷 O(1)	已知 handle；同一序列勢能
完整 vEB 前驅／更新	最壞 O(1+log log u)	整數 word RAM；Θ(u) 空間／建立

若先建立完整 vEB 再做 q 次操作，總上界為 $O(u + q(1 + \log\log u))$；初始化不能被省略。一般鍵的比較模型、整數 universe 模型與隨機模型各有適用條件，沒有一張脫離模型的「最快資料結構」排名。

參考資料與署名

Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms, 3rd ed. (2009), Chapters 10–14, 17–20. MIT 6.046J/18.401J (2004), L09: Balanced Search Trees, Piotr Indyk. MIT 6.046J/18.401J (2005), L08: Hashing II; L09: Randomly Built BSTs; L11: Augmenting Data Structures; L13: Amortized Analysis. Charles E. Leiserson and Erik D. Demaine. Lin Cing，個人手寫資料結構筆記：佇列、搜尋樹、紅黑樹、多路搜尋樹與堆積。

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

動態規劃

遞迴把問題拆開；動態規劃進一步辨認哪些子問題其實相同，讓每個狀態只求解一次。

本章從狀態與依賴開始，依序處理 **鋼條切割 → 背包 → 最長共同子序列與序列比對 → 矩陣鏈與最佳二元搜尋樹**。

前置知識為遞迴、漸近分析、陣列與樹。重點是把問題定義、遞迴式、填表順序與答案重建連成同一個演算法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先決定狀態代表什麼

設計問題	必須說清楚的內容
狀態是否足夠？	相同狀態的未來選擇與成本必須相同。
遞迴是否完整？	每種合法最優解都要落入某個候選分支。
先算哪些狀態？	邊界與依賴決定合法的填表順序。
如何交回答案？	最優值、選擇紀錄與重建路徑各有用途。

每個模型都先固定合法解與成本，再比較時間、空間及是否能重建一個最優解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

動態規劃：先決定什麼才算同一問題

動態規劃（dynamic programming, DP）把重複出現的子問題合併為狀態，保存答案，再組合出原問題答案。

- **狀態語義**：此格究竟最佳化哪一組合法解？
- **轉移與證明**：候選能否涵蓋所有最佳解，而且都合法？
- **計算與重建**：依賴先算，並記錄達成最佳值的選擇。

最佳子結構支持正確組合；子問題重疊解釋為何快取能節省重算。兩者不能代替狀態定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

狀態充分性：忘記歷史之前先檢查

若兩段歷史被合併為同一狀態，它們必須具有相同的後續合法選擇、後續成本規則與剩餘目標。

已累積價值可以分開記帳；狀態需保留會改變未來的資訊。

例如切鋼條：每段價格只取決於長度、切割免費、沒有段數上限。剩餘長度同為 j，先前如何切不影響未來，因此 $r[j]$ 足夠。

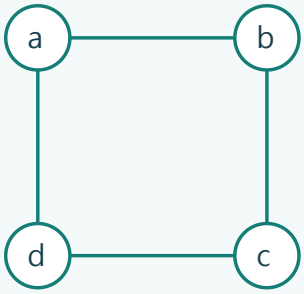
若新增「最多切成 k 段」，只記 j 便不足，還需記剩餘可用段數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最長簡單路：端點狀態會漏掉資源限制

無向圖的邊為 ab, bc, cd, da，每條邊長為1；簡單路不可重複頂點。

端點相同，不代表餘下資源相同



最佳整路 / Whole optimum: a → b → c → d

後綴 / Suffix: c → d (1)

獨立最長 / Independent: c → b → a → d (3)

拼接 / Splice: a → b → c → b → a → d

重複 a, b：非法

$a \to b \to c \to d$ 是 a 到 d 的最長簡單路，長度3。但其中 $c \to d$ 並非端點 c, d 間最長路；$c \to b \to a \to d$ 長度3。直接替換得到 a, b, c, b, a, d，已不合法。

失敗原因：端點沒有記錄前綴已使用的頂點，不能任意拼接各端點間最佳解。

擴充狀態可以修復，但可能付出指數成本

固定起點 s，令 $D[S, v]$ 為「恰好使用頂點集合 S、從 s 到 v 的最長簡單路長度」，不存在則為 $-\infty$。

$$D[\{s\}, s] = 0, \qquad D[S, v] = \max_{\substack{u \in S \setminus \{v\} \\ uv \in E}} (D[S \setminus \{v\}, u] + 1).$$

其他基底為 $-\infty$；僅計算 $s, v \in S$、$v \neq s$ 的非基底狀態。答案為 $\max_{S \ni s,t} D[S, t]$。

集合大小嚴格增加，形成狀態 DAG；至多 $O(2^{|V|}|V|^2)$ 時間。這是指數 DP，不是多項式解。若原圖本身是 DAG，端點最長路可直接按拓撲序計算。

切鋼條：問題條件與狀態

給定整數長度 $n \geq 0$，價格 $p[1..n]$。將整條鋼條切成正整數長度的段，每段都出售；切割免費，同長度可以出現多次。

$$r[j] = \text{恰好用完長度 } j \text{ 的最大總收益}, \qquad r[0] = 0.$$

本例價格（收益單位）：

長度 i	1	2	3	4
$p[i]$	1	5	8	9

求最佳值 $r[4]$，並輸出每段長度。整段出售也是合法方案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第一段選擇導出遞迴式

任何非空切法都有第一段長度 i，$1 \leq i \leq j$；第一段直接出售，只有餘段繼續切。

$$r[j] = \max_{1 \leq i \leq j} \{p[i] + r[j-i]\}, \qquad j \geq 1.$$

- $i = j$：不再切割，餘段為空，收益 $p[j] + r[0]$。
- 不把第一段寫成 $r[i]$：此處 i 表示一段成品的長度。
- 只要長度下降，就能以 j 歸納證明計算正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩向證明：不漏解，也不引入假解

令 $OPT(j)$ 為真正最佳收益，並假設所有更短狀態均已正確。

上界方向：任一最佳切法第一段為 i，其餘段收益不超過 $r[j-i]$。所以 $OPT(j) \le p[i] + r[j-i] \le \max_i(p[i] + r[j-i])$。

下界方向：對每個 i，一段長度 i 接上長度 $j-i$ 的最佳切法，長度恰為 j，全部合法。因此 $p[i] + r[j-i] \le OPT(j)$；取最大仍成立。

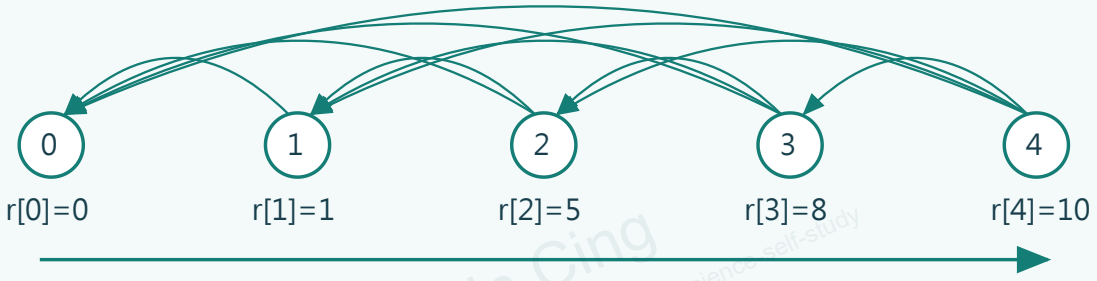
兩式合起來得到等號。替換餘段不會改變第一段合法性，是最佳子結構成立的關鍵。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

重疊子問題與依賴 DAG

箭頭 $j \to k$ 表示計算 $r[j]$ 需要 $r[k]$。

箭頭：狀態 → 所需子問題



計算次序：0 → 1 → 2 → 3 → 4

5 個狀態；10 條依賴；j 嚴格下降

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先完成較短依賴，再計算較長狀態

所有 $k < j$ 都是 $r[j]$ 的候選依賴。例如 $r[3]$、$r[4]$ 都需要 $r[2]$。合併同長度呼叫後，這份答案只需保存一次。

長度4的依賴 DAG 有5個狀態 $0, 1, 2, 3, 4$。狀態 j 有 j 條向外的依賴邊，因此總數為 $1 + 2 + 3 + 4 = 10$。

依 $0, 1, 2, 3, 4$ 填表，方向與依賴箭頭相反。計算 $r[j]$ 時，所有需要的 $r[k]$ 都已完成。依賴長度嚴格下降，因此不會形成循環。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

記憶化：每個狀態只展開一次

```
memo[0] = 0; 其餘為 UNKNOWN
solve(j):
    if memo[j] is not UNKNOWN: return memo[j]
    q = -∞
    for i = 1 .. j:
        q = max(q, p[i] + solve(j-i))
    memo[j] = q
    return q
```

呼叫從目標往下展開；重複呼叫直接讀表。以獨立的 `UNKNOWN` 記未計算，不能把合法的負收益當作未知。

本題所有長度都會到達，故仍需 $\Theta(n^2)$ 時間、$\Theta(n)$ 表格與至多 $O(n)$ 呼叫堆疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

由小到大填表，同時記下第一段

```
r[0] = 0
for j = 1 .. n:
    r[j] = -∞
    for i = 1 .. j:           # i 由小到大
        q = p[i] + r[j-i]
        if q > r[j]:          # 同值保留較小的第一段
            r[j] = q
            first[j] = i
```

迴圈不變量：開始算 j 時，$r[0..j-1]$ 全部正確。每個候選都只讀這些已完成的值。

時間為 $\sum_{j=1}^{n} j = n(n+1)/2 = \Theta(n^2)$；值表與選擇表共 $\Theta(n)$ 空間。

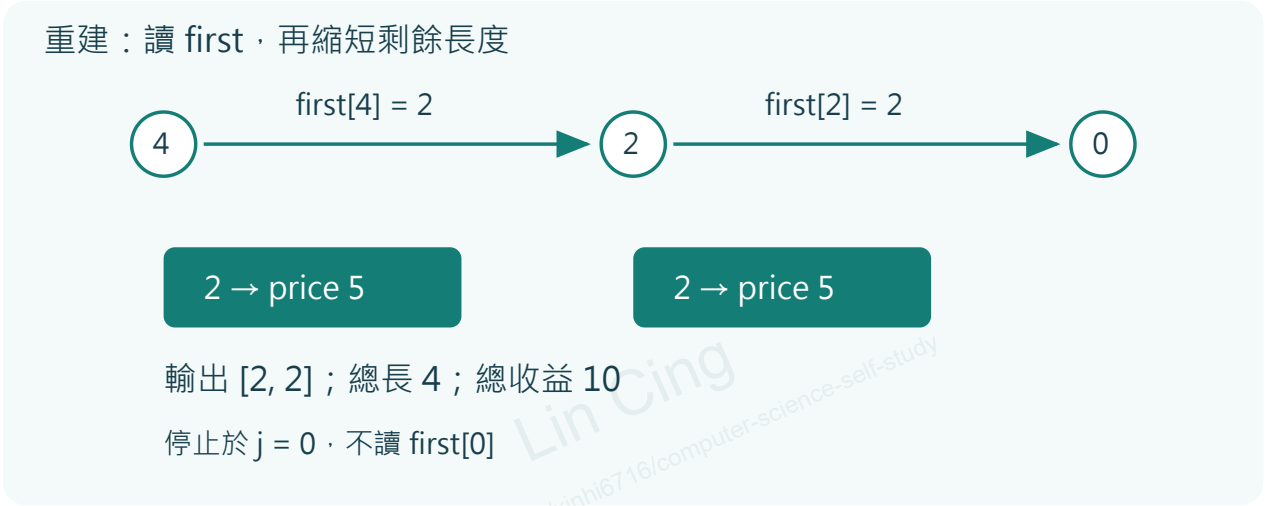
完整數例：每個候選都可追查

依 $i = 1, 2, \ldots, j$ 列出候選 $p[i] + r[j-i]$。

j	候選收益（依 i 遞增）	掃描中的最大值	$r[j]$	first[j]
0	空切法	—	0	—
1	$1+0=1$	1	1	1
2	$1+1=2,\ 5+0=5$	2, 5	5	2
3	$1+5=6,\ 5+1=6,\ 8+0=8$	6, 6, 8	8	3
4	$1+8=9,\ 5+5=10,\ 8+1=9,\ 9+0=9$	9, 10, 10, 10	10	2

收益最大的第一段不一定是售價最高的段：$p[4]=9$，而兩段長度2可得10。

沿記錄的選擇還原最佳切法



此例 $first[4]=2$、$first[2]=2$。剩餘長度依序為 $4\to2\to0$，輸出 $[2,2]$。核對長度和 $2+2=4$，收益 $5+5=10=r[4]$。

記錄的切法為何維持最佳收益

剩餘長度為 j 時，已輸出各段的長度和為 $n-j$，且其收益加上 $r[j]$ 等於 $r[n]$。初始 $j=n$、尚未輸出任何段，因此兩項性質都成立。

令 $i=first[j]$。輸出 i 並把 j 換成 $j-i$，已輸出長度恰增加 i。由 $r[j]=p[i]+r[j-i]$，已輸出收益的增加量，恰等於剩餘最佳收益的減少量，因此兩項性質都維持。

到達 $j=0$ 時，已輸出長度為 n，且 $r[0]=0$，故已輸出收益為 $r[n]$。本例輸出兩段長度2，總長度4、收益10。

重建迴圈在剩餘長度為零時停止

填表完成後，每個正長度 j 的 `first[j]` 都記錄一個最佳第一段。沿這些選擇即可輸出完整切法。

```
j = n; pieces = []
while j > 0:
    i = first[j]
    append i to pieces
    j = j - i
return pieces
```

每個選擇皆滿足 $1 \le first[j] \le j$。每次至少減1，故迴圈至多執行 n 次便在零停止，重建需 $O(n)$ 時間。此時不變量保證總長度為 n、總收益為 $r[n]$。

$n = 0$ 時不進入迴圈，直接回傳空串列。

記憶化與填表是在排同一張圖

面向	Top-down memoization	Bottom-up tabulation
啟動方式	從目標遞迴要求依賴	按合法順序逐狀態計算
實際狀態	僅展開可達狀態	通常填預先指定的範圍
額外成本	呼叫堆疊、快取查詢	通常無遞迴堆疊
本例時間／表格	$\Theta(n^2)$／$\Theta(n)$	$\Theta(n^2)$／$\Theta(n)$

記憶化不保證比填表快；填表也不必是二維矩形。兩者都必須先有正確轉移與無循環的依賴順序。

邊界條件會改變答案

必須用完整條：若 $p[1]=-2, p[2]=-5$，則 $r[1]=-2$、$r[2]=\max(-4,-5)=-4$。初始最大值必須為 $-\infty$，不能為0。

允許丟棄剩餘材料：改成 $r[j]=\max(0,\max_i(p[i]+r[j-i]))$，同一負價例答案為0；重建也需能記錄「停止並丟棄」。這是另一個問題。

空輸入：$n=0$，價格表可為空，收益0、切法空。若 $n>0$，本章假設每個長度 $1..n$ 都有價格；缺價長度不能直接當成0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

換條件時，重新檢查轉移

固定每次實際切割費用 $c\ge 0$：最後整段出售時不收切割費。

$$r[j]=\max_{1\le i\le j}\left(p[i]+r[j-i]-c\,\mathbf{1}_{i<j}\right).$$

原例 $c=2$：兩段2收益 $5+5-2=8$，整段4收益9，因此最佳改為不切。

這仍只需長度狀態；若限制成品段數或價格依先前選擇改變，就應擴充狀態。判斷依據是未來是否合法、成本能否只由狀態決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用可檢查的五件事設計 DP

1. 精確寫出每個狀態的合法解集合與基底。
2. 指定第一個／最後一個決策，枚舉全部合法候選。
3. 證明最佳解落在候選中，而且候選都能組成合法解。
4. 找出嚴格下降的尺度，安排依賴順序。
5. 以「狀態數 × 每狀態候選成本」估算，並另外驗證重建。

本例：長度狀態、第一段決策、兩向證明、遞減長度、二次時間；輸出切法再核對總長度與收益。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

0/1 背包：每件只能選一次

有 n 件物品，重量 w_i 為正整數，價值 v_i，容量 W 為非負整數。

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{使} \quad \sum_{i=1}^{n} w_i x_i \le W, \quad x_i \in \{0,1\}.$$

允許空背包，所以最優值至少為 0；負價值物品不必取。物品可有相同重量或價值，但仍是不同的件。 整數容量是這個容量索引 DP 的前提；任意實數重量不能直接當陣列索引。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

狀態為何要保留物品範圍

定義 $D[i, c]$：只用物品 $1, \ldots, i$、總重至多 c 的最大價值。答案為 $D[n, W]$。 只記「剩餘容量」不夠：相同容量下，可用物品不同，答案也可能不同。以物品前綴限制可用集合，才不會重複拿同一件。 對固定 (i, c)，未來選擇只取決於此前綴與容量；不同歷史若到達相同狀態，較差的價值不可能改善任何延伸。這裡假設價值可相加，沒有物品間相依限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩個互斥分支導出遞迴

$$D[0, c] = 0, \qquad D[i, 0] = 0.$$

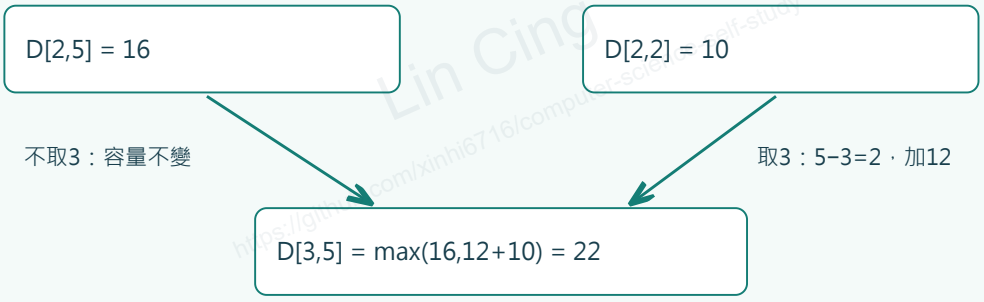
若 $c < w_i$，則 $D[i, c] = D[i-1, c]$；否則

$$D[i, c] = \max\{D[i-1, c],\ v_i + D[i-1, c - w_i]\}.$$

不取 i 保留容量；取 i 要扣去 w_i，且兩分支都只剩前 $i-1$ 件。

同一格的兩個前一列依賴

i = 3, c = 5, (w₃,v₃) = (3,12)



箭頭 = 資料流；取或不取都把物品前綴從3降到2

遞迴的上下界證明

任取 (i, c) 的可行集合 S。若 $i \notin S$，其價值至多 $D[i-1, c]$；若 $i \in S$，移除 i 後只用前 $i-1$ 件，重量至多 $c - w_i$，故價值至多 $v_i + D[i-1, c-w_i]$。因此最優值不超過候選最大值。 反方向：不取分支的最優集合可直接使用；取分支的最優集合不含 i，加上 i 後仍不超重，故兩個候選都是可達值。最大候選不超過最優值。 配合 $i = 0$ 的基底，對 i 歸納即得正確性。若移除 i 後不是剩餘問題的最優解，可換成更好的解並加回 i，與原解最優矛盾。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一數例：輸入與填表順序

容量 $W = 5$；物品 (w_i, v_i) 為 $(2, 10), (1, 6), (3, 12)$。

列是可用前綴，欄是容量上限，並非必須填滿的重量。

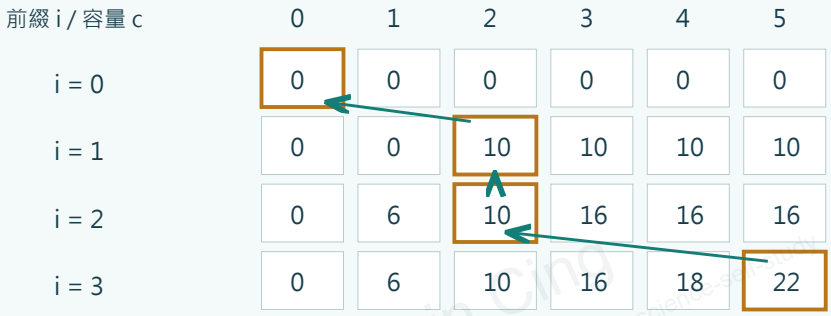
每格只讀前一列；依 $i = 1, 2, 3$ 填列，同列容量順序可任意。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一數例：完整狀態表

完整表與回溯：W = 5

物品 (w,v)：1=(2,10), 2=(1,6), 3=(3,12)



箭頭 = 回溯：取3 → 不取2 → 取1；總重5，總值22

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

看懂表格中的三次比較

完整表給出 $D[3,5]=22$。

物品 1：$D[1,1]=0$（太重）；$D[1,2]=\max(0,10+0)=10$。 物品 2：$D[2,3]=\max(D[1,3],6+D[1,2])=\max(10,16)=16$。 物品 3：$D[3,3]=\max(16,12+D[2,0])=16$，能裝不代表必須取；$D[3,4]=\max(16,12+6)=18$；$D[3,5]=\max(16,12+10)=22$。 每個「取」候選皆是當前物品價值加上可行的前綴最優值，不能加同列任意最大值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

回溯把最優值變成物品集合

從 $(i,c)=(n,W)$ 開始；若 $D[i,c]=D[i-1,c]$，選擇不取並令 $i\leftarrow i-1$。否則取 i，令 $c\leftarrow c-w_i$、$i\leftarrow i-1$。相等時固定不取，輸出一組最優解。 本例 $(3,5):22>16$，取 3，至 $(2,2)$；$10=10$，不取 2，至 $(1,2)$；$10>0$，取 1，至 $(0,0)$。 輸出 $\{1,3\}$：重量 $2+3=5$，價值 $10+12=22$。每步 i 減一，回溯 $O(n)$；不要把容量遞減當作唯一停止條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

可直接實作的二維版本

```
D[0..n][0..W] = 0
for i = 1..n:
    for c = 0..W:
        D[i,c] = D[i-1,c]
        if c >= w[i]:
            D[i,c] = max(D[i,c], v[i] + D[i-1,c-w[i]])
```

表格有 $(n+1)(W+1)$ 格，每格常數次運算。時間 $O(n(W+1))$，完整表空間 $O(n(W+1))$；通常在 $W\geq 1$ 時寫作 $O(nW)$。保留表格即可依前頁程序重建。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

壓成一維：倒序維持前一列

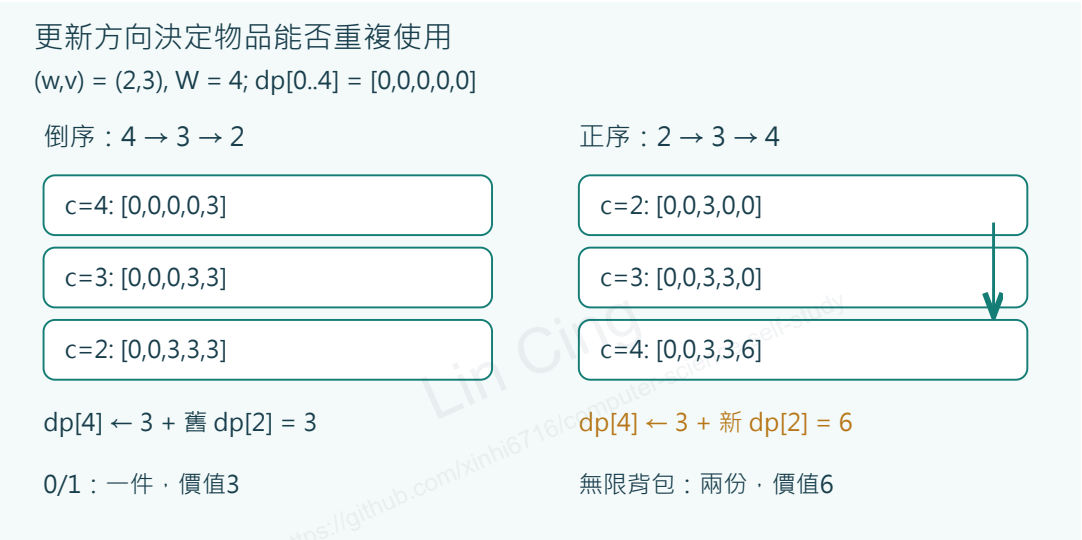
```
dp[0..W] = 0
for i = 1..n:
    for c = W down to w[i]:
        dp[c] = max(dp[c], v[i] + dp[c-w[i]])
```

處理 i 時，尚未更新的容量存 $D[i-1,\cdot]$。倒序且 $w_i > 0$，使 $c - w_i < c$ 尚未更新；右側讀到的仍是前一列。更新後 $dp[c] = D[i,c]$。 時間仍 $O(nW)$，值的空間降至 $O(W)$。被覆寫的歷史不足以直接做二維回溯；需要解集合時，可保留二維表或另設帶版本的決策紀錄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正序反例：一件物品被拿了兩次

只有一件物品 $(w,v) = (2,3)$，容量 $W = 4$。0/1 最優值應為 3。



正序在 $c = 2$ 寫入 3，之後 $c = 4$ 讀到剛寫的 $dp[2] = 3$，得到 6，等於使用兩份同一物品。更新方向是在選擇依賴哪個版本的狀態。

無限背包：同列依賴正好需要正序

若每種物品可取任意非負整數份，定義 $U[i,c]$ 為前 i 種、重量至多 c 的最大價值。可放時

$$U[i,c]=\max\{U[i-1,c],\ v_i+U[i,c-w_i]\}.$$

取走一份後仍能取第 i 種，所以第二項是同列。以容量正序更新一維陣列即可；因 $w_i>0$，同列依賴的容量嚴格較小。 例 $(2,3),W=4$：可取兩份，答案 6；此時前頁的正序結果才符合模型。本節排除零重物品；零重正價值且無限供應會使目標無上界，不能套用此有限遞迴。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

恰好裝滿：不可行不是零分

令 $E[i,c]$ 為前 i 件、總重**恰為** c 的最大價值。遞迴形式與 0/1 相同，但

$$E[0,0]=0,\quad E[0,c]=-\infty\ (c>0).$$

不可行候選保持 $-\infty$；實作先檢查可達再相加，避免整數哨兵溢位。正重量使 $E[i,0]=0$。 例單件 $(2,-3),W=2$：至多容量答案為 0（空集合）；恰好容量答案為 -3。若 $W=1$，恰好容量不可行。全部初始化為 0 會把不存在的重量誤作可行解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

偽多項式取決於數值大小

$O(nW)$ 對容量數值 W 是多項式，對二進位輸入長度卻未必。記錄容量約需 $\log_2(W+1)$ 位元；若 $W = 2^b$，掃描容量要 2^b 次。 例如同樣 20 件物品，$W = 10^3$ 約有兩萬格更新，$W = 10^9$ 約有兩百億格。整數重量可索引不等於容量索引一定實用。 此分析把每次數值加法與比較視為常數成本；若價值整數位數很大，還需計入算術位元成本。0/1 背包的決定版是 NP-complete；最佳化版稱 NP-hard。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分數背包的貪心為何不同

若改成 $0 \le x_i \le 1$，每件可切分，對正價值物品按 v_i/w_i 由大到小取；負價值可略過。若較低密度被選而較高密度尚未用完，交換相同重量 δ 會使價值不減；反覆交換可得到密度貪心解。 用相同三件物品、容量改為 $W = 4$：密度依序是物品 2 的 6、1 的 5、3 的 4。 分數解取 2、1、$1/3$ 件 3，價值 $6 + 10 + 4 = 20$。0/1 密度貪心取 2、1 得 16；0/1 最優取 2、3 得 18。交換重量若不能切分就未必可行。排序實作為 $O(n \log n)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

分枝限界的上界不是 DP 格值

對最大化問題，分枝限界法（branch and bound）在取／不取的搜尋樹中，保留已找到的可行解價值 L。令已取物品價值為 V、剩餘容量為 C，只用尚未決定物品的分數背包最優值為 $F(C)$。節點的總上界為 $B = V + F(C)$。 因放寬限制只會增加可行集合，節點任何 0/1 完成解價值都不超過 B；若 $B \le L$，可剪去該節點（只求一組最優解）。 上一例根節點 $B = 20$，整數最優值為 18：上界不一定可達。DP 的 $D[i, c]$ 則是所定義子問題的**確切**最優值；兩者不可混用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

子序列保留次序，允許跳過字元

子序列的索引必須嚴格遞增；子字串則必須連續。

固定 $X =$ `ABCBDAB`、$Y =$ `BDCABA`。`BCBA` 在 X 使用索引 $(2, 3, 4, 6)$，在 Y 使用 $(1, 3, 5, 6)$。它的長度為 4；`BCA` 雖也是共同子序列，卻不是最長。

目標先求最長共同子序列（LCS）的**長度**，再重建其中一個解。

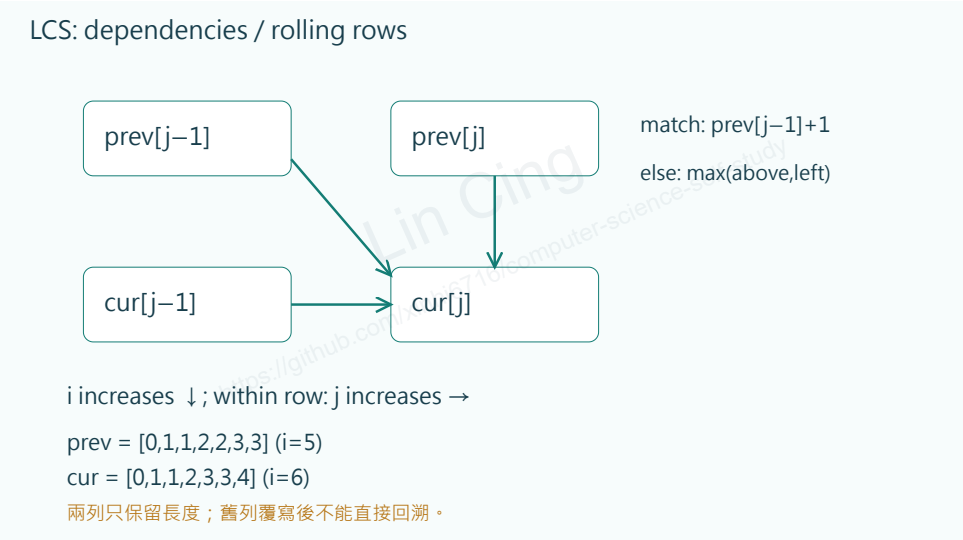
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

狀態必須同時記住兩個前綴

令 $C[i, j]$ 為 $X[1..i]$ 與 $Y[1..j]$ 的 LCS 長度。答案是 $C[7, 6]$。

$C[0, j] = C[i, 0] = 0$：任一前綴為空，只能取空子序列。

依賴只往較小的 $i + j$，形成 DAG。按照 $i = 1..m$、每列 $j = 1..n$ 填表時，所需的左、上、左上格皆已完成。



相同末字元：先證明可達下界

若 $x_i = y_j = a$，取較短前綴的一個 LCS，再附加 a。索引合法，故

$$C[i, j] \geq C[i - 1, j - 1] + 1.$$

這證明「候選解做得到」。例如 $(6, 6)$ 的末字元皆為 A，$C[5, 5] = 3$，故有長度 4 的共同子序列。單有下界尚不能斷言它最優。

相同末字元：再證明沒有更長的解

取任一 LCS Z，長度 k。其末字元必為 a；若不是，兩邊最後位置都未用，可再附加 a，與最長矛盾。

刪掉 Z 最後的 a，其餘字元在兩邊都可嵌入前 $i-1, j-1$ 個位置。因此 $k-1 \le C[i-1, j-1]$，得到上界。結合前頁：

$$C[i, j] = C[i-1, j-1] + 1.$$

這保證存在使用兩個末位置的最佳嵌入；不代表每一種最佳匹配的索引都必須使用它們。例如 A 對 AA 可匹配第一個或第二個 A。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不同末字元：兩個方向的界

若 $x_i \neq y_j$，共同子序列的一組嵌入不可能同時使用兩邊最後位置：若兩者都使用，必須是同一個末字元。

所以任一最佳解至少略過 x_i 或 y_j，其長度至多為 $\max(C[i-1, j], C[i, j-1])$。這是上界。

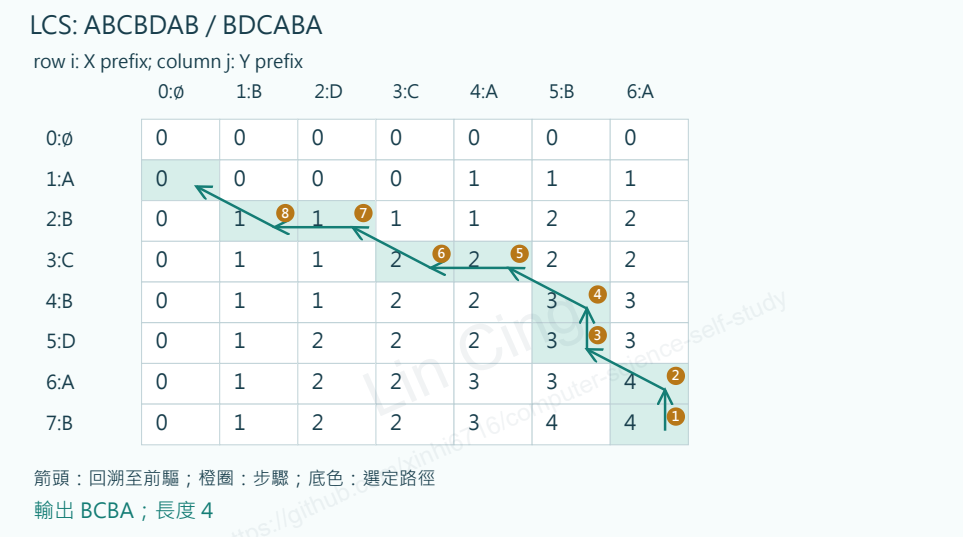
反向看，兩個較短前綴的共同子序列，在完整前綴中仍合法；取較長者即可達到該最大值。故

$$C[i, j] = \max\{C[i-1, j], C[i, j-1]\}.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整值表：數值代表長度

相同取左上加 1，不同取上、左最大值。零邊界不可省略。



例如 $C[3,3] = C[2,2] + 1 = 2$；$C[5,5] = \max(3,2) = 3$；$C[7,6] = \max(4,4) = 4$。編號箭頭是下一頁逐步使用的回溯路線。

逐步回溯：反轉 ABCB 得 BCBA

不同字元且上、左同值時，本例固定先往上；相同則往左上並記錄字元。

步	目前格 → 前驅格	理由／記錄
1	$(7,6) \to (6,6)$	上、左皆 4，先上
2	$(6,6) \to (5,5)$	A=A，記 A
3	$(5,5) \to (4,5)$	上 3 > 左 2
4	$(4,5) \to (3,4)$	B=B，記 B
5	$(3,4) \to (3,3)$	左 2 > 上 1
6	$(3,3) \to (2,2)$	C=C，記 C
7	$(2,2) \to (2,1)$	左 1 > 上 0
8	$(2,1) \to (1,0)$	B=B，記 B；停止

ties 決定選哪一個最佳解

倒序記錄 ABCB，反轉得到 BCBA。

BCBA、BCAB、BDAB 都是長度 4 的 LCS。不同 tie 規則可能回傳不同字串，長度不變。

在 $(7, 6)$ 往左到 $(7, 5)$ 也是合法最佳選擇；往上只是本例的確定性規則。

只存一個前驅只能重建一個解。若要列出所有不同 LCS，必須探索所有合法最佳分支並去重；分支路徑與不同字串的數量也不相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

填表與重建有不同成本

共有 $(m+1)(n+1)$ 個狀態，每個內部格只做常數次比較：填完整表時間 $\Theta(mn)$、空間 $\Theta(mn)$。

從 (m, n) 回溯，每一步至少減少一個索引，時間 $O(m+n)$。可存 choice 表，也可保留完整值表，回溯時重新比較鄰格；兩者都能重建一個解。

暴力枚舉一邊的索引子集有 2^m 種候選；許多不同遞迴路徑會重複要求同一對前綴。DP 合併的就是這些重複狀態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩列滾動：保留長度，丟掉歷史

讓較短的字串放在欄方向。prev 是上一列，cur 是當前列；每列由左至右更新。

```
prev[0..n] = 0
cur[0..n] = 0
for i = 1..m:
    cur[0] = 0
    for j = 1..n:
        if X[i] == Y[j]: cur[j] = prev[j-1] + 1
        else: cur[j] = max(prev[j], cur[j-1])
    swap(prev, cur)
return prev[n]
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩列滾動與解的重建

時間仍為 $\Theta(mn)$，額外空間 $O(\min(m,n))$。此程式只回傳長度。

每次更新讀取上一列與當前列的左鄰格。交換兩列後，舊歷史可以覆寫，因為後續長度計算不再需要它。

過去列被覆寫後，不能直接照完整表路線回溯。若要輸出序列，保留完整值／choice 表，或另外使用分治重建等演算法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

全域對齊：每個字元都必須交代

給定 $X = \texttt{ACGTA}$、$Y = \texttt{AGCAC}$。可插入缺口 `-`，使兩列等長；不得有 `-/-` 欄。刪去缺口後須恢復原序列。

一欄的類型	分數
相同字元	+3
不同字元	−2
一個字元對一個缺口	−3

目標是所有欄分數總和最大，開頭與結尾的缺口也照常扣分。這是線性缺口成本的全域對齊；LCS 長度不是本問題的分數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對齊狀態與完整邊界

$F[i, j]$ 是前綴 $X[1..i]$ 與 $Y[1..j]$ 的最佳全域對齊分數。

$$F[0,0] = 0, \qquad F[i,0] = -3i, \qquad F[0,j] = -3j.$$

只有一邊有字元時，每個字元都必須對缺口，因此邊界為 `0, -3, -6, -9, -12, -15`。

不能把邊界全設 0；那會把未對齊的前綴免費略過，改變問題。也不能加上「與 0 取最大」而仍稱為同一個全域對齊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最後一欄只有三種可能

令 $s(a,b)=3$（相同）或 -2（不同）。當 $i,j>0$：

$$F[i,j]=\max\begin{cases}F[i-1,j-1]+s(x_i,y_j) & \text{字元對字元}\\ F[i-1,j]-3 & x_i \text{ 對缺口}\\ F[i,j-1]-3 & \text{缺口對 } y_j.\end{cases}$$

任一合法對齊的最後一欄必屬其中一類；刪去後得到相應前綴對齊，故不會超過右側最大值。反過來，任一前驅最佳對齊附加該欄仍合法，能達到該候選值。兩方向共同證明遞推式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對齊完整數例：最佳分數為 1

Global alignment: ACGTA / AGCAC

row i: X prefix; column j: Y prefix

	0:ø	1:A	2:G	3:C	4:A	5:C
0:ø	0	-3	-6	-9	-12	-15
1:A	-3	3	0	-3	-6	-9
2:C	-6	0	1	3	0	-3
3:G	-9	-3	3	0	1	-2
4:T	-12	-6	0	1	-2	-1
5:A	-15	-9	-3	-2	4	1

箭頭：回溯至前驅；橙圈：步驟；底色：選定路徑

輸出 ACGTA- / A-GCAC；總分 1

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

$F[2,3]=\max(0+3,-3-3,1-3)=3$。

重建對齊：每一個欄都有代價

從 $(5,5)$ 回溯：

目前格 → 前驅格	附加欄	該步加分
$(5,5) \to (5,4)$	`-/C`	−3
$(5,4) \to (4,3)$	`A/A`	+3
$(4,3) \to (3,2)$	`T/C`	−2
$(3,2) \to (2,1)$	`G/G`	+3
$(2,1) \to (1,1)$	`C/-`	−3
$(1,1) \to (0,0)$	`A/A`	+3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正向讀出重建的對齊

反轉欄順序：

```
ACGTA-
A-GCAC
```

總分 $3-3+3-2+3-3=1$。兩個原序列都完整保留。

兩個缺口欄分別交代 X 的 C 與 Y 最後的 C。T/C 不匹配貢獻 -2，三個匹配欄貢獻 $3+3+3$。

$F[5,5]=\max(-2-2,-1-3,4-3)=1$，故最後一欄選 `-/C`。即使最終分數為正，中間格仍可為負；全域對齊不把它們重設為 0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

相似的表格，需要不同的語義

問題	狀態值	相同字元	不同字元	邊界
LCS	最長長度	左上 +1	上、左最大	0
全域對齊	最大總分	三候選仍都比較	三候選仍都比較	累計缺口成本

LCS 可以略過字元而不計費；對齊必須把所有字元放到某一欄。對齊即使字元相同，也依既定三分支規則比較候選，不能直接套用 LCS 的 if/else。

兩者都能用前綴 DAG 填表；完整重建則需要足夠的前驅資訊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

區間 DP：最後一次決策

矩陣鏈最後一次相乘決定切點 k；BST 的根 r 決定左右鍵區間。兩者都保留原本順序，不能任意重排。

狀態 $[i, j]$ 記錄連續區間；依長度遞增填表，因為每個決策只依賴更短區間。值表回答「多好」，決策表回答「如何做」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

矩陣鏈：維度決定合併成本

給定 $p = [5, 10, 3, 12, 5]$，矩陣 A_i 的維度是 $p_{i-1} \times p_i$。

矩陣	A_1	A_2	A_3	A_4
維度	5×10	10×3	3×12	12×5

本節用傳統乘法計純量乘法數：$(a \times b)(b \times c)$ 要 abc 次，結果 $a \times c$。只改括號，不改矩陣順序；DP 規劃成本不包含實際相乘。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

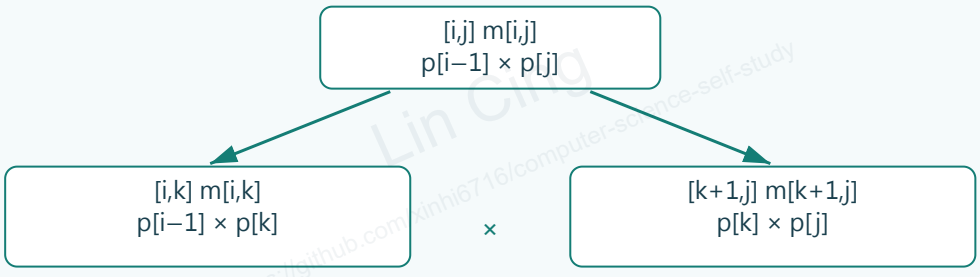
矩陣鏈遞迴與狀態充分性

$m[i, j]$ 是計算 $A_i \cdots A_j$ 的最少乘法數，$m[i, i] = 0$。

$$m[i, j] = \min_{i \le k < j} \{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \}.$$

任何括號化的區間結果都為 $p_{i-1} \times p_j$，所以外部只需知道端點與最小成本，不需記內部括號。固定切點後兩個子區間的維度都固定。

固定切點：兩側輸出維度不隨括號改變



merge = p[i−1] · p[k] · p[j]

箭頭：父狀態依賴兩個更短區間；替換子解只降低內部成本

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

切開、替換、再合併的證明

任一完整括號化的最後乘法都有某個 $i \le k < j$。若其中左子括號化非最優，換成更便宜的左解，輸出維度不變，右成本與合併成本也不變，總成本下降，矛盾；右側同理。

反方向，每個 k 的兩個可行子解都能合法相乘，因此每個候選都是可行完整解。涵蓋最優解且沒有加入不可行解，取最小值就是最優。以區間長度歸納，已算子表正確即可推出當前格正確。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

依長度填表：所有短區間候選

所有對角線 $m[i,i]=0$；表列每個切點的「左＋右＋合併」。

長度	區間	候選與計算	m；s
2	$[1,2]$	$k=1: 0+0+5\cdot 10\cdot 3=150$	150；1
2	$[2,3]$	$k=2: 0+0+10\cdot 3\cdot 12=360$	360；2
2	$[3,4]$	$k=3: 0+0+3\cdot 12\cdot 5=180$	180；3
3	$[1,3]$	$k=1: 0+360+600=960$；$k=2: 150+0+180=330$	330；2
3	$[2,4]$	$k=2: 0+180+150=330$；$k=3: 360+0+600=960$	330；2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整鏈：三個切點都必須比較

$[1,4]$ 的切點	左成本	右成本	最後相乘	候選總成本
$k=1$	0	330	$5\cdot10\cdot5=250$	580
$k=2$	150	180	$5\cdot3\cdot5=75$	**405**
$k=3$	330	0	$5\cdot12\cdot5=300$	630

最小合併成本本身不是一般性的貪婪規則，必須加上兩邊的最佳成本。此例巧合由 $k=2$ 同時得到最便宜合併與最便宜總成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整鏈：三個切點都必須比較 （續）

矩陣鏈完整表：m[i,j] / s[i,j]

	j=1	j=2	j=3	j=4
i=1	0 / –	150 / 1	330 / 2	405 / 2
i=2		0 / –	360 / 2	330 / 2
i=3			0 / –	180 / 3
i=4				0 / –

填表順序：L=1 → 2 → 3 → 4；每格只讀更短區間

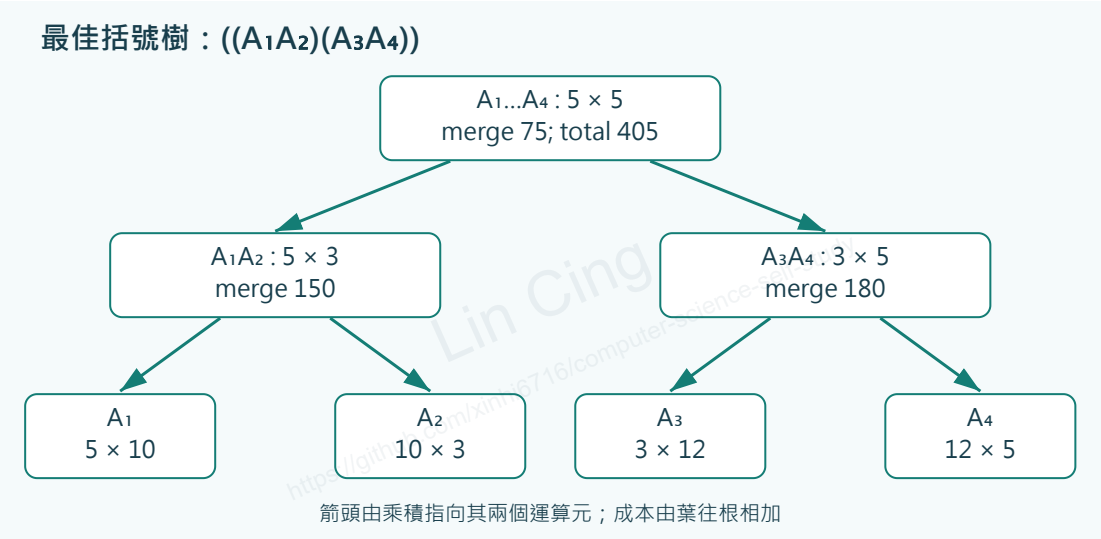
L=4: 580, 405, 630 → min = 405

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

由切點表還原括號樹

$s[1,4]=2$，接著 $s[1,2]=1$、$s[3,4]=3$，得 $((A_1A_2)(A_3A_4))$。

先算 A_1A_2：5×3，成本150；算 A_3A_4：3×5，成本180；最後得到 5×5，成本75，總計405。



遞迴：若 $i=j$ 輸出 A_i；否則輸出括號內的 `build(i,s[i,j])` 與 `build(s[i,j]+1,j)`。每個葉與合併節點各訪問一次，重建 $\Theta(n)$。

矩陣鏈演算法與精確工作量

```
for i=1..n: m[i,i]=0
for L=2..n:
  for i=1..n-L+1:
    j=i+L-1; m[i,j]=infinity
    for k=i..j-1:
      t=m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]
      if t<m[i,j]: m[i,j]=t; s[i,j]=k
```

長度 L 有 $n-L+1$ 格、各有 $L-1$ 個候選，所以候選數

$$\sum_{L=2}^{n}(n-L+1)(L-1)=\binom{n+1}{3}=\frac{n^3-n}{6}.$$

故時間 $\Theta(n^3)$；值表與切點表各 $\Theta(n^2)$ 空間。這是規劃的複雜度，不是實際矩陣乘法耗時。

OBST：成功與失敗都要建模

有序真鍵 $k_1 < \cdots < k_n$，成功查詢權重 $p_1, \ldots, p_n$。外部葉 $d_0, \ldots, d_n$ 代表查無資料的間隙，權重 $q_0, \ldots, q_n$。

d_0 表示小於 k_1；d_i 表示 $k_i < x < k_{i+1}$；d_n 表示大於 k_n。一棵含 n 個真鍵的擴充 BST 有 $n+1$ 個外部葉。

機率模型有 $\sum p + \sum q = 1$；非負次數權重也能直接使用，總成本除以全域總權重才得到每次查詢期望。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩種成本：差一個固定常數

根深度為0。$C(T)$ 只數實際比較的真鍵；$E(T)$ 額外數到達外部葉的一次終止檢查。

$$C(T) = \sum_i p_i(\operatorname{depth}(k_i) + 1) + \sum_j q_j \operatorname{depth}(d_j).$$

$$E(T) = \sum_i p_i(\operatorname{depth}(k_i) + 1) + \sum_j q_j(\operatorname{depth}(d_j) + 1).$$

所以 $E(T) - C(T) = \sum_j q_j$，與樹形無關，最優樹集合相同。空段只有 d_{i-1}：$C[i, i-1] = 0$，$E[i, i-1] = q_{i-1}$。初始化與報出的成本必須用同一慣例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OBST 區間：保留全域機率質量

區間 $[i, j]$ 包含鍵 $k_i, \ldots, k_j$ 與外部葉 $d_{i-1}, \ldots, d_j$。

$$w[i, j] = \sum_{t=i}^{j} p_t + \sum_{t=i-1}^{j} q_t.$$

子問題仍用原始全域權重，不除以區間質量。因為子樹降低一層時，每個屬於此子樹的查詢都多一次檢查，增加的是全域質量 w。

若改存條件期望，合併時必須重新乘回各子樹質量，不能把不同分母的條件期望直接相加。空段的 $w[i, i-1] = q_{i-1}$ 也不能漏掉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OBST 遞迴與替換證明

$$E[i, j] = w[i, j] + \min_{i \le r \le j} \{E[i, r-1] + E[r+1, j]\}.$$

C 也滿足相同形狀的遞迴，但空段成本為0。根 r 的左右鍵範圍由中序順序唯一決定；合併增加 $w[i, j]$。

任一最優樹都必有某個根。若其中子樹非最優，以同一鍵／間隙集合的較便宜樹替換，仍為 BST；子樹全域質量不變，所以移至相同深度所增成本不變，得到更小總成本而矛盾。反向，每個候選根搭配兩棵可行子樹都形成可行 BST，因此枚舉所有根後取最小即為最優。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三鍵例 B：初始化與單鍵區間

固定 $p = (.20, .30, .10)$、$q = (.10, .05, .15, .10)$，總質量1。

空段 $E[1,0] = .10$、$E[2,1] = .05$、$E[3,2] = .15$、$E[4,3] = .10$。

區間	w	唯一候選 $E_L + E_R + w$	E	C
$[1,1]$	.35	$.10 + .05 + .35$	.50	.35
$[2,2]$	.50	$.05 + .15 + .50$	.70	.50
$[3,3]$	.35	$.15 + .10 + .35$	.60	.35

單鍵區間的 C 已含其左右間隙的失敗比較，故不是單純 p_i。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

例 B：兩鍵與三鍵所有根候選

區間；質量	候選根與完整計算	最小 E；C
$[1,2]$；.80	$r=1: .10 + .70 + .80 = 1.60$；$r=2: .50 + .15 + .80 = 1.45$	1.45；1.15
$[2,3]$；.70	$r=2: .05 + .60 + .70 = 1.35$；$r=3: .70 + .10 + .70 = 1.50$	1.35；1.05
$[1,3]$；1.00	$r=1: .10 + 1.35 + 1 = 2.45$；$r=2: .50 + .60 + 1 = 2.10$；$r=3: 1.45 + .10 + 1 = 2.55$	**2.10；1.70**

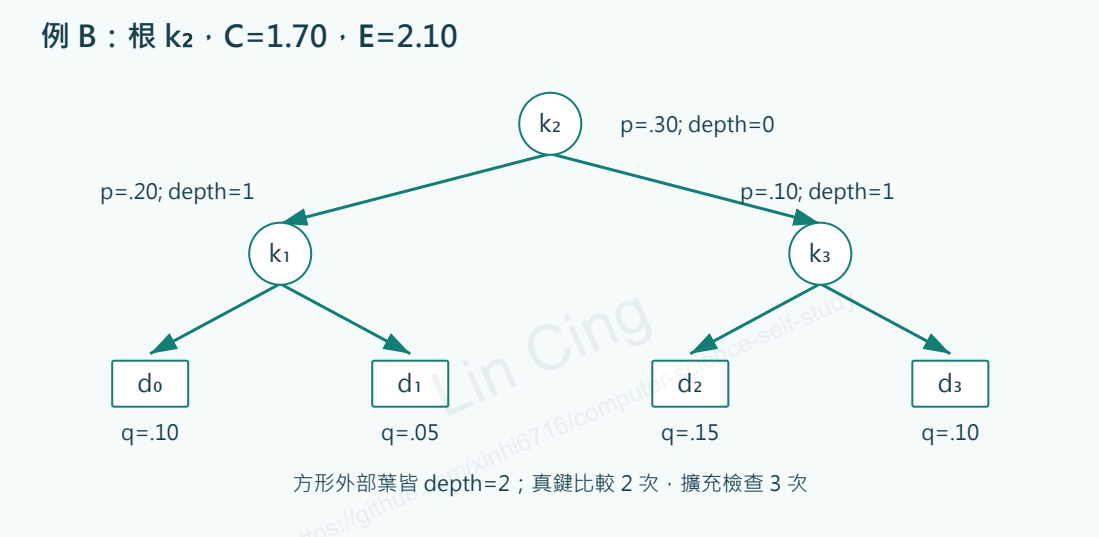
這三個區間的最佳根皆為 $r = 2$。

各區間 $C = E - \sum_{t=i-1}^{j} q_t$；扣除的是該區間的間隙總質量。全表共10個根候選，不能只算最後3個。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

例 B：重建後直接驗算期望

根為 k_2，左子 k_1，右子 k_3。真鍵深度分別1、0、1；四個外部葉深度皆2。



例 B：逐項核對查詢成本

真鍵比較次數依序為2、1、2，成功查詢的加權成本為 $.20 \cdot 2 + .30 \cdot 1 + .10 \cdot 2 = .90$。

四個失敗間隙的總機率為 $.10 + .05 + .15 + .10 = .40$。每次失敗查詢比較2個真鍵；若將外部葉的終止檢查也計入，則成本為3。

$C = .90 + (.10 + .05 + .15 + .10) \cdot 2 = 1.70$；$E = .90 + .40 \cdot 3 = 2.10$。兩值差.40；這是每次全域查詢的期望，不是「成功條件下」期望。

OBST 填表、重建與精確成本

```
for i=1..n+1: E[i,i-1]=w[i,i-1]=q[i-1]
for L=1..n:
  for i=1..n-L+1:
    j=i+L-1; w[i,j]=w[i,j-1]+p[j]+q[j]
    E[i,j]=infinity
    for r=i..j:
      t=E[i,r-1]+E[r+1,j]+w[i,j]
      if t<E[i,j]: E[i,j]=t; root[i,j]=r
```

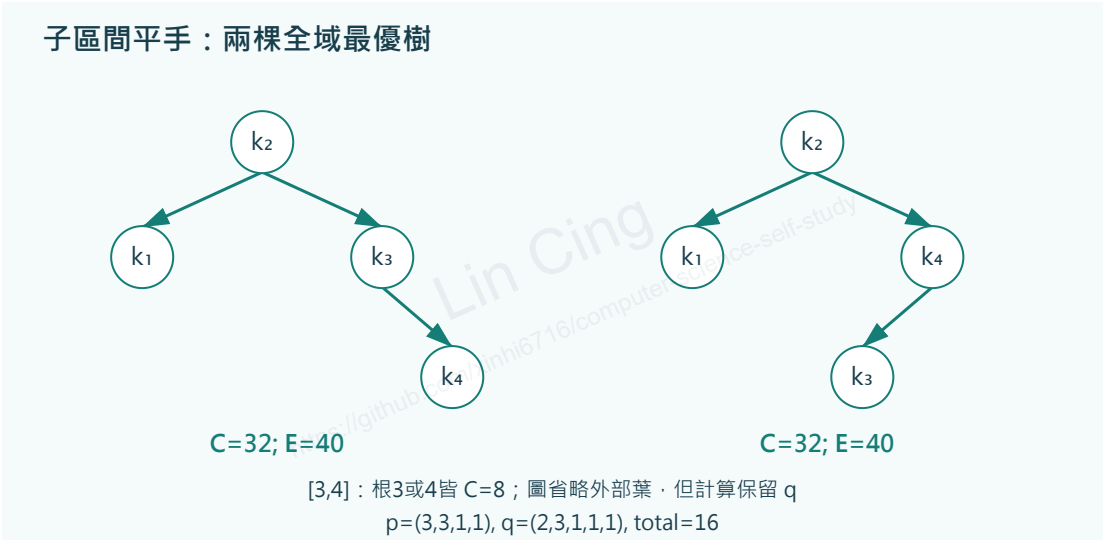
候選數 $\sum_{L=1}^{n}(n-L+1)L=\binom{n+2}{3}=n(n+1)(n+2)/6$，因此基本演算法 $\Theta(n^3)$ 時間、$\Theta(n^2)$ 空間。`build(i,j)` 若空則回傳 d_{i-1}，否則以 `root[i,j]` 接左右子樹；輸出 $2n+1$ 節點，$\Theta(n)$ 時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

整數例 C：相同最小成本可以有兩棵樹

取次數權重 $p=(3,3,1,1)$、$q=(2,3,1,1,1)$，全域總權重16。

區間 $[3,4]$ 的 $w=5$；C 候選根3為 $0+3+5=8$，根4為 $3+0+5=8$，兩者皆最優。全域根2的候選為 $C[1,1]+C[3,4]+16=8+8+16=32$；其他根1、3、4的候選分別35、38、41。



全域 $C=32$、$E=40$；除以16得期望2、2.5。只存第一個最小根可重建一棵；要求所有最優樹時須存所有平手根，輸出數量可能指數成長。

僅成功查詢與完整模型如何銜接

模型	權重與邊界	最佳化目標
資料結構課第10章成功版	只有 p；空段0；$W = \sum p$	成功查詢加權真鍵比較數
本節完整模型	p, q；C 空段0，E 空段 q；$w = \sum p + \sum q$	成功與失敗的全域加權成本

現行資料結構教材也另外列出完整 p/q 模型與 $E - C$ 差；不能把「成功版」當成它唯一的慣例。原始資料結構筆記使用真鍵成本 C；CLRS §15.5 使用擴充成本 E，本節明確連接兩者。

將所有 q 設0才退化為僅成功查詢的模型；直接刪去非零 q 會改變最佳化問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

區間 DP 的檢查清單

矩陣鏈：端點決定輸出維度；切點必須 $i \le k < j$；最小的是完整候選總成本；重建沿記錄的切點。

OBST：鍵順序固定；間隙隨子區間保留；空段成本與 C/E 慣例一致；保留全域權重；所有候選根都比較；平手策略只影響選哪棵最優樹。

矩陣鏈與 OBST 的「同為區間 DP」不代表可互換合併成本或邊界。此處分析基本立方演算法；更快的 OBST 最佳化須另外證明根單調性，不能套到任意區間 DP。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 15, Dynamic Programming. MIT 6.046J, Fall 2004, Lecture 12, Dynamic Programming. 講者 Charles E. Leiserson。 MIT 6.046J, Fall 2005, Lecture 15, Dynamic Programming. 講者 Charles E. Leiserson；投影片署名 Erik D. Demaine、Charles E. Leiserson。 原始掃描筆記《演算法_wjungle》動態規劃相關頁；《資料結構》實體第 166–170 頁用於比較最佳搜尋樹模型。原始筆記涵蓋演算法與資料結構，並非單一本教科書全文。

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16 貪婪演算法

Part IV Advanced Design and Analysis Techniques

本章定位

貪婪演算法每步只保留一個選擇，節省枚舉其他候選的成本。本章用活動交換、Huffman葉交換與matroid增廣性證明這一步何時安全，再處理加權目標與容量限制如何使原規則失效。

前置知識

排序、交換論證、樹結構與選擇公理性質。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用交換法證明 activity selection。
- 辨識 greedy-choice 與最佳子結構。
- 建 Huffman code 並以 matroid 理解貪婪正確性。
- 在安全選擇失效時給出反例，並把一般證明中的交換步驟寫完整。
- 16.1 An activity-selection problem
- 16.2 Elements of the greedy strategy
- 16.3 Huffman codes
- 16.4 Matroids and greedy methods
- 16.5 A task-scheduling problem as a matroid

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

16.1 An activity-selection problem

已知所有活動的開始、結束時間，要求選最多個兩兩不重疊活動。假設 s_i<f_i，活動已按 finish time 遞增排序，區間採半開形式 `[s_i,f_i)`，故一個活動開始時刻等於另一個結束時刻也相容。

定理。 對任一非空子問題，令 `a_m` 是 finish 最早的活動，則存在最大相容集合包含 `a_m`。

證明。 令 A^* 為子問題的最大解，a_j 為 A^* 中最早結束的活動。因 a_m 是全域最早結束，`f_m<=f_j`。把 a_j 換成 a_m：

- A^* 中原本在 a_j 之後的活動都滿足 `start>=f_j`，所以也與 a_m 相容；
- 子問題沒有更早的 activity 可受影響；
- 集合大小不減，故替換後仍是最大解。

因此 greedy-choice 成立。選 a_m 後只剩 `start>=f_m` 的子問題；對剩餘集合重複同一交換，歸納得到最優。

活動選擇的線性掃描與邊界

```
activity-select(按結束時間排序的活動):
    result=[]; last_finish=-∞
    for (s,f) in 活動:
        if s>=last_finish:
            append (s,f) to result
            last_finish=f
    return result
```

每次處理活動 j 前，result 是前 j−1 個中依此規則取出的相容集合，last_finish 是最後所選活動的終點；交換定理使每次選擇可延伸為某個最優解。掃描每活動一次，已排序時 Θ(n)；若尚未排序，另付 O(n log n)。

本章假設 s<f 且區間半開，端點相等可以相接；零長活動需另訂同點使用規則。空輸入直接輸出空集合。最早開始或最短時長不保證最優，例如活動[0,10)、[1,2)、[2,3)使最早開始只選一個，卻有兩個相容短活動。

最短時長先取的反例為 [0,3)、[3,6)、[2,4)。先選長度2的中間活動會擋住兩個長度3的活動，只有一個；最優則選前兩個，得到兩個。

活動選擇的完整掃描

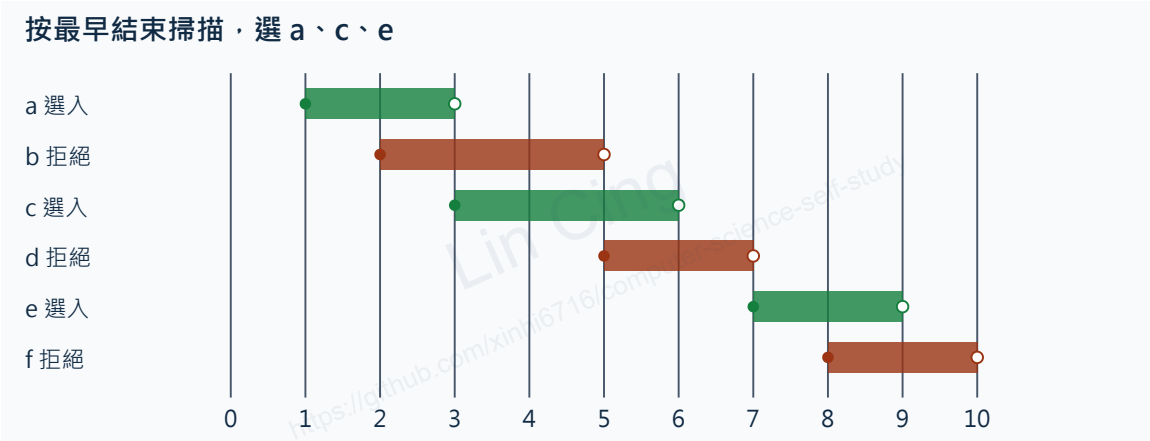
取六個活動：

活動	start	finish
a	1	3
b	2	5
c	3	6
d	5	7
e	7	9
f	8	10

依 finish 排序掃描：選 a；b 的 $2<3$ 拒絕；c 的 $3>=3$ 選入；d 的 $5<6$ 拒絕；e 的 $7>=6$ 選入；f 的 $8<9$ 拒絕。結果 `{a,c,e}`。

活動選擇的完整掃描 （續）

最優解也不只一個。`{b,d,e}` 與 `{a,c,e}` 都有 3 個；交換定理只保證存在一個含最早結束者的最優解，沒有說所有最優解都必須包含它。



16.2 Elements of the greedy strategy

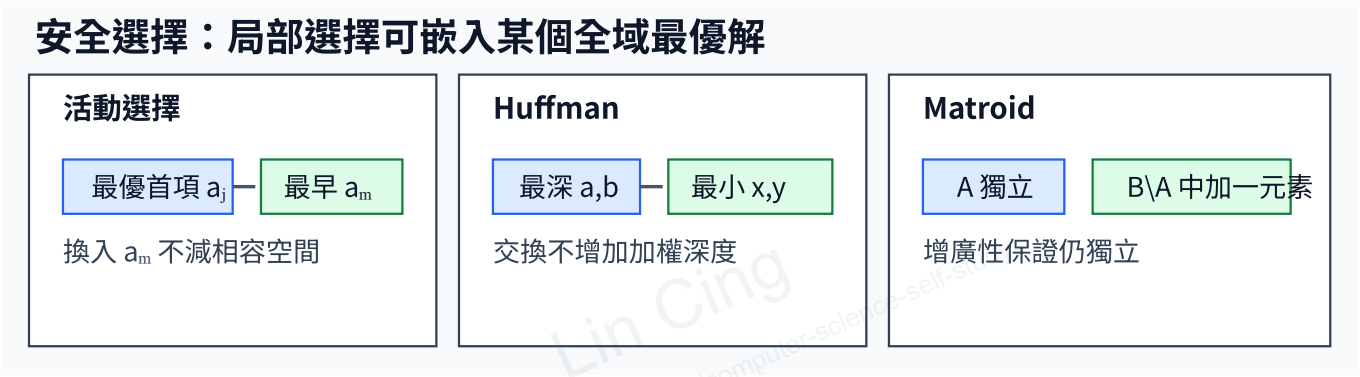
本章的交換型 greedy proof 要回答下列問題：

1. **Greedy choice 安全。** 取 greedy 的 g，對任意最優解 A^* 找第一個不同決策 x，構造 $A' = A^* - x + g$；逐項驗證 A' 可行且價值不低於 A^*。
2. **剩餘問題相同。** 移除 g 後，A^* 的其餘部分必須是明確定義的子問題的可行解，且子問題沒有遺失原來的候選。
3. **歸納終止。** 基底（空集合或單一活動）正確，每次決策使問題嚴格縮小。

這三項成立才可由「每步都安全」推到全域最優。局部立即利益、平均表現或若干成功例都不足夠。

例如任意面額的找零不能只取最大硬幣。面額 {1,3,4} 找6，greedy 得4+1+1三枚，最佳3+3兩枚；此反例否定一般規則。熟悉面額的成功經驗不能代替針對面額族的證明。

16.2 Elements of the greedy strategy （續）



為何加權活動不能用同一交換

把目標改成最大總 value，就不再是最大基數。取

活動	區間	value
short	[0,2)	1
long	[1,3)	100
tail	[3,4)	1

最早 finish greedy 選 short，再選 tail，總 value 2；最優為 long+tail，總 value 101。交換 A^* 中原本最早結束的 long 為 short 後，雖然區間仍相容，但 value 從 100 降到 1，不能保持最優值。

加權版本有最佳子結構。以 1-based 索引按 finish 排序，令 p(j) 為 j 之前 finish 不晚於 s_j 的最大索引，不存在時 p(j)=0；D[j] 表示前 j 個活動的最佳總價值。則 `D[0]=0`，

`D[j]=max(D[j-1], value_j+D[p(j)])`。

為何加權活動不能用同一交換 （續）

排除 j 就只看前 j−1 個；包含 j 則其餘活動必來自前 p(j) 個，而且可獨立最佳化。此二分涵蓋所有解。數例 p=(0,0,2)、D=(0,1,100,101)，由 D3 選 tail，再由 D2 選 long。finish-time greedy 只保證未加權最大基數，不能沿用到這個目標。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

0–1 與 Fractional Knapsack 的不同證明

假設物品重量與價值皆正、容量 C>0。fractional 版本允許取比例 x_i∈[0,1]，其餘目標是最大化 Σv_i x_i。依密度 ρ_i=v_i/w_i 下降排序，先裝滿密度高的物品，至多一件只裝部分。

若全部物品總重不超過 C，全部取走即最優。否則任何最優解必裝滿 C；若仍有空間且有正價值物品未取完，增加一點比例就能改善目標，矛盾。

若某可行最優解在高密度 i 尚未取滿時，卻使用較低密度 j 的正重量，令 δ=min(w_i(1−x_i),w_j x_j)>0，把 δ 重量由 j 移給 i。容量不變，價值增加 δ(ρ_i−ρ_j)≥0。反覆交換直到第一個 greedy 選擇相同，再對剩餘容量歸納，得到某最優解符合整個 greedy 排序。相同密度可任意打破平手。

但在 0–1 版本 x_i 只能是 0 或 1，δ 重量交換通常不合法。例 C=10，物品 (w,v)=(6,12),(5,9),(5,9)。密度 greedy 先取 (6,12)，剩 4 放不下其他，價值 12；0–1 最優取兩件 (5,9)，價值 18。fractional 則先取 (6,12)，再取一件 (5,9) 的 4/5，價值 19.2。

0–1 DP 用 D[i,c] 表示前 i 件、容量至多 c 的最佳值，D[0,c]=D[i,0]=0，D[i,c]=max(D[i−1,c],v_i+D[i−1,c−w_i])，第二項僅在 w_i≤c 時存在。整數容量的時間 Θ(nC) 是偽多項式，因輸入 C 本身只需 Θ(log C) 位元。

16.3 Huffman codes

二元 prefix tree 的葉深度就是碼長。給定頻率 `f_c`，總位元數為

`B(T)=sum_c f_c d_T(c)`。

Huffman 每次從 priority queue 取兩個最小權重樹 x,y，建立權重為 `f_x+f_y` 的父節點。若有 n 個符號，進行 n−1 輪；每輪兩次 extract-min、一次 insert，共 3(n−1) 次 heap 操作，加線性建堆，時間 O(n log n)。

交換左、右 child 只改變 codeword 的 0/1，不改變長度與總成本。頻率相同時可形成多棵不同的最優樹。

Huffman 選擇引理：把最小頻率放到最深兄弟

假設至少兩個字元且頻率皆正。任何最佳 prefix tree 可取成每個內節點恰有兩個 child，因刪掉只有一個 child 的內節點會縮短其下所有碼字而降低成本。取最深葉，其兄弟也必為同深葉，否則兄弟的後代會更深。

令 x、y 為頻率最低的兩字元。先把 x 換到這對最深葉的一個位置；若其中一個位置原本是 y，就選另一位置。這使 y 在第二次交換前仍留在別處，或已經在所需的兄弟位置。

交換 a 與 x 的成本差只涉及這兩個葉：

`B(T')-B(T)=(f_a-f_x)(d_x-d_a)<=0`。

因 `f_a>=f_x` 且 `d_x<=d_a`，差值非正。此時 x 已放妥，剩餘位置的頻率都不小於 y，故再把 y 換到 x 的最深兄弟位置，同樣不增加成本，且不會移走 x。若 x 或 y 原本已在目標位置，該次交換為零。頻率相同時任何相應安排成本相等。

故總存在最佳 prefix code 含這個 greedy choice。這裡需使用「最低頻率」與「最深葉」的兩個交換不等式，不能只說看起來合理。

Huffman 收縮與最優子結構

把 x、y 從字母表刪除，加入虛擬字元 z，令 `f_z=f_x+f_y`。若 T' 是收縮後的最佳樹，把 z 展開成 x、y，得到原樹 T。

除 x、y 以外的葉深度不變；x、y 的深度都是 `d_T'(z)+1`，故

`B(T)=B(T')+f_x+f_y`。

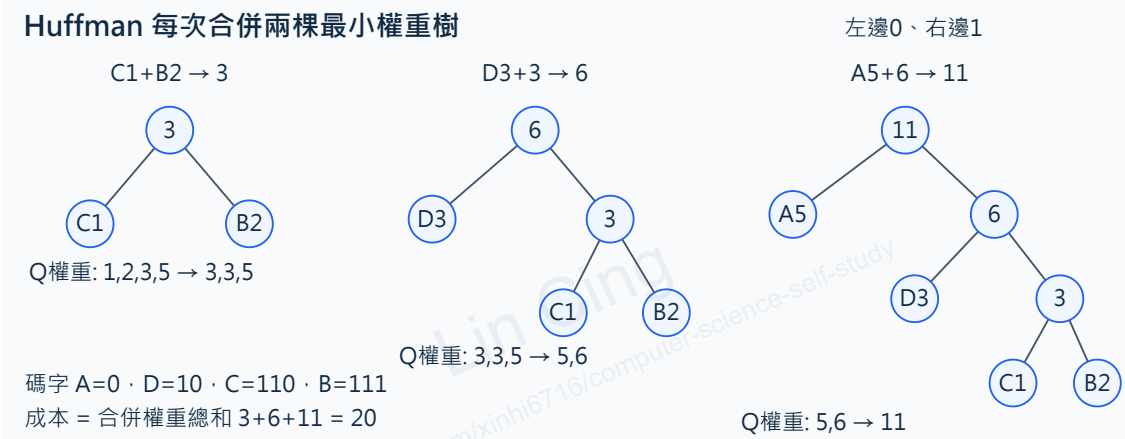
若原問題有更優樹 T*，依上一頁可令 x、y 為最深兄弟；將兄弟收縮成 z，得到

`B(T**)=B(T*)-f_x-f_y<B(T')`，

與 T' 最優矛盾。因此展開收縮樹即原問題最優；再以符號數歸納，Huffman 正確。

頻率 A5、B2、C1、D3 的合併權重為 3、6、11；可取 A=0、D=10、C=110、B=111，成本 `5+6+3+6=20=3+6+11`。

Huffman 收縮與最優子結構 （續）



Huffman 的演算法與單字元邊界

```
Q = 所有正頻率單葉樹組成的 min-heap
while size(Q)>1:
    x=extract_min(Q); y=extract_min(Q)
    insert(Q, node(x.weight+y.weight, x, y))
return Q 的唯一樹 · 若 Q 空則回傳空
```

解碼從根依0、1走向左、右 child；到葉就輸出字元並重回根。prefix 性質保證沒有某個完整碼字同時是更長碼字的前綴，因而字界明確。最後位元若停在內節點，表示輸入截斷，不能默認成功解碼。

空字母表沒有碼樹。只有一個字元時，樹論可給空碼，實際串流通常約定碼0並另傳字元數。本章成本與合併權重總和的等式採至少兩個字元的完整二元樹，不能把單字元實作約定偷換成合併成本。

16.4 Matroids and greedy methods

matroid $M=(S,I)$ 有三條：

- 空集屬於 I；
- 遺傳性：若 A 屬於 I，任何 A 的子集也屬於 I；
- 增廣性：若 A,B 屬於 I 且 $|A|<|B|$，存在 $x \text{ in } B\setminus A$ 使 $A\cup\{x\}$ 屬於 I。

假設 S 有限，w(x)>0。greedy 依 weight 降冪掃描，加入後仍獨立就取。令 x 是第一個單點獨立的元素。若不存在，遺傳性使所有非空集合都不獨立，空集就是最優解。

取任一最優集 B。若 $x\notin B$，從 $A=\{x\}$ 開始，當 $|A|<|B|$ 就以增廣性選 $b\in B\setminus A$ 加入 A。每次只從 B 加入，最後 $|A|=|B|$，故恰有一個 $y\in B$ 被省去，$A=B-\{y\}+\{x\}$。由 B 獨立可知 {y} 獨立，x 是權重最高的可行單點，故 $w(x)\geq w(y)$，而 $w(A)=w(B)-w(y)+w(x)\geq w(B)$。因此 A 也最優且含 x。

這裡交換性不是直接宣告「換任一 y 都可行」。必須先以逐步增廣構造出可行的 A，再由等大小推得唯一被省去的 y。

收縮後仍為 Matroid 的完整證明

固定可行的 x，令 $S'=S\setminus\{x\}$，$I'=\{A\subseteq S' : A\cup\{x\}\in I\}$。S' 可以包含在新系統中永不獨立的元素，這些元素不影響結論。

- 空集在 I'，因 $\{x\}\in I$。
- 若 $A\in I'$、$C\subseteq A$，則 $C\cup\{x\}\subseteq A\cup\{x\}$，由原系統遺傳性可得 $C\in I'$。
- 若 $A,B\in I'$ 且 $|A|<|B|$，原系統中的 $A\cup\{x\}$、$B\cup\{x\}$ 都獨立且前者較小。增廣元素 y 必在 $(B\cup\{x\})\setminus(A\cup\{x\})=B\setminus A$，因此 $A\cup\{y\}\in I'$。

對任何含 x 的可行解，其餘部分屬 I'，且權重恰少 w(x)；反之亦然。所以原問題在選定 x 後精確化為收縮系統的最大權獨立集。對 |S| 歸納，後續掃描的可行性正是 I' 的 oracle，greedy 全程最優。

若 x 在某階段不可加入，之後也不會可加入。否則較大集合 $A\cup\{x\}$ 獨立會由遺傳性推出當時較小的集合也可加 x，矛盾。排序 $O(n \log n)$，每次 oracle 成本 $O(f(n))$ 時，總成本 $O(n \log n+n f(n))$。允許負權且目標為任意獨立集時，須略過負權元素；不能無條件取成最大基底。

Graphic Matroid 與非 matroid 反例

對固定無向圖 G=(V,E)，令 S=E，I 為所有不含 cycle 的邊集合，孤立頂點仍列入 V。遺傳性成立。令 A 的分量為 V1..Vk；若 B 每條邊都留在某個 Vi 內，因 B 在 Vi 內仍是森林，邊數至多 |Vi|−1。相加得 |B|≤Σ(|Vi|−1)=|V|−k=|A|，與 |B|>|A| 矛盾。所以存在 B 中一條跨 A 分量的邊，加入 A 不形成 cycle，增廣性成立。

連通圖的基底都是 n−1 邊的生成樹。若要由最大正權定理推得最小成本 MST，可令新權重 w'(e)=K−cost(e)>0；所有基底有相同邊數，故最大化 Σw' 等價於最小化原成本。這使 Kruskal 升冪規則與此定理精確相接，並沒有把最大任意森林與最小任意森林混為一談。

但一個 hereditary system 不一定有增廣性。取 S={a,b,c}、權重 10,9,9，I={∅,{a},{b},{c},{b,c}}。A={a}、B={b,c} 有 |A|<|B|，卻無法把 b 或 c 加入 A；這不是 matroid。Greedy 選 a 得 10，最優 {b,c} 得 18。

16.5 A task-scheduling problem as a matroid

工作 j 處理時間 1、deadline d_j、延遲罰金 w_j。集合 A 可全部準時，當且僅當依 deadline 排好後 `d_(k)>=k`；等價於對每個 t，

`N_t(A)=|{j in A:d_j<=t}|<=t`。

必要性是前 t 個時間槽最多容納 t 個工作。充分性是把 A 按 deadline 遞增排；第 k 個工作的 deadline 至少 k，故不遲到。

所有工作在時間 0 就緒、不可搶占、單機，deadline 是正整數，完成於 d_j 視為準時。n 個工作只需前 n 格，故先將大於 n 的 deadline 截為 n，不改變可行性。

這族獨立集有遺傳性。為證增廣，取 |A|<|B|，令 k 是滿足 N_k(A)=k 的最大整數，0 一定符合。由 |A|<|B|≤n，k<n。A 中有 k 個工作 deadline≤k，而 B 這類工作至多 k 個，所以 B 中 deadline>k 的工作數至少 |B|−k>|A|−k。因此存在 x∈B\A 且 d_x>k。

把 x 加入 A。對 t≤k，新增工作不被計入，所以 N_t 不變；對 t>k，最大性保證 N_t(A)<t，整數性使 N_t(A)≤t−1，新增至多 1 後仍≤t。因此 A∪{x} 可行，增廣性成立。

單位時間排程的 greedy 與完整追蹤

工作：

工作	deadline	penalty
a	3	9
b	1	8
c	3	7
d	2	6
e	1	5

按 penalty 由大到小加入：a 可行；b 使 deadlines 排序 1,3，可行；c 使 1,3,3，可行；d 使 1,2,3,3，第 4 個 deadline 3<4，拒絕；e 使 `N_1=2>1`，拒絕。

準時集為 {a,b,c}，依 deadline 排成 b,a,c，分別於 1、2、3 完成。其餘 d,e 放在 4、5，確實遲到。延遲罰金 6+5=11；總 penalty 35，準時工作所保住的 penalty 為 24，達最大。

單位時間排程的 greedy 與完整追蹤 （續）

使用長度 n 的 deadline 次數陣列，計算各前綴 N_t，可在 O(n) 內檢查一次獨立性，n 次檢查共 $O(n^2)$，加最初 penalty 排序 O(n log n)。若每次另作比較排序，oracle 是 O(n log n)，總時間會變成 $O(n^2 \log n)$，不能把排序叫作線性。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 16。

Cormen et al., *Introduction to Algorithms*, 3rd ed., § §16.1-16.5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22 基礎圖演算法

Part VI Graph Algorithms

本章定位

圖的搜尋順序同時決定距離、遞迴祖先與完成時間；本章用同一六點有向圖，從FIFO層序與白路徑定理推導BFS、DFS、拓撲序及SCC的正確性與線性成本。

前置知識

圖表示、顏色、時間戳與線性走訪。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 選擇 adjacency matrix 或 list。
- 推導 BFS 最短路徑性質。
- 用 DFS 完成 topological sort 與 SCC。
- 22.1 Representations of graphs
- 22.2 Breadth-first search
- 22.3 Depth-first search
- 22.4 Topological sort
- 22.5 Strongly connected components

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

22.1 Representations of graphs

本章的圖是有限有向圖或無向圖；每個 vertex（頂點）有穩定識別碼，edge（邊）的方向必須明定。路徑可以長度零，因此每個頂點可達自身。令 n=|V|、m=|E|；無向邊只在 E 計一次。除特別註明，本章的最短距離是**邊數**，不是任意權重的總和。

Adjacency list（鄰接串列）有 n 個表頭。有向圖的所有表長之和為 m；無向簡單圖每條邊在兩端各存一次，總長為 2m。空間 Θ(n+m)，列出 u 的鄰點需 Θ(1+outdegree(u))，掃描普通未排序串列查詢 (u,v) 最壞 Θ(1+outdegree(u))。雜湊或排序的鄰接集合可以改變查詢成本，不能把它們的保證套到普通 list。

Adjacency matrix（鄰接矩陣）A 的 row 是起點、column 是終點，A[u,v]=1 表示 u→v；空間 $\Theta(n^2)$、查邊 Θ(1)，但枚舉 u 的鄰點一般要掃 n 個位置。無向圖滿足 $A=A^{\mathsf{T}}$。有權矩陣需另有「不存在」標記：權重 0 可以是合法邊，不能直接與不存在混為一談。

同一個六頂點有向圖

後面的 BFS、DFS 與 SCC 都使用下列**同一組有向邊**，鄰接與外層頂點順序均按表排列。

頂點	Adj[u]	outdegree	indegree
a	b,c	2	1
b	c	1	2
c	a	1	2
d	b,e	2	1
e	d	1	2
f	e	1	0

n=6、m=8，表長總和 8，入度總和也為 8；不是 16。轉置圖 G^{T} 把每個 u→v 放入 $\mathrm{Adj}^{\mathsf{T}}[v]$ 的 u 項：先建立 n 個空表，再掃全部 m 項，時間與新增空間皆 Θ(n+m)。原圖不需破壞。

同一個六頂點有向圖 （續）

一張圖的 List 與 Matrix

Adj[u]：row u → column v；這是有向圖，m=8。

頂點	鄰接串列
a	b,c
b	c
c	a
d	b,e
e	d
f	e

A 的 column

	a	b	c	d	e	f
a	0	1	1	0	0	0
b	0	0	1	0	0	0
c	1	0	0	0	0	0
d	0	1	0	0	1	0
e	0	0	0	1	0	0
f	0	0	0	0	1	0

綠格：存在的有向邊；8個綠格與8個list項逐一對應。

22.2 Breadth-first search

BFS（廣度優先搜尋）以 FIFO queue 維持距離的層序。白色是未發現，灰色是已發現而尚未掃完，黑色是鄰接表已掃完。π[v] 是發現 v 的前驅；∞ 與 NIL 分別表示尚無距離與無前驅，不能用 0 取代。

```
BFS(G,s):
  for v in V: color[v]=WHITE; d[v]=∞; π[v]=NIL
  color[s]=GRAY; d[s]=0; Q=empty queue; enqueue(Q,s)
  while Q is not empty:
    u=dequeue(Q)
    for v in Adj[u]:
      if color[v]==WHITE:
        color[v]=GRAY; d[v]=d[u]+1; π[v]=u
        enqueue(Q,v)
    color[u]=BLACK
  return d,π
```

入列之前就改色，同一頂點才不會因兩個父候選重複入列。在每輪 while 開始時，queue 恰為全部灰色頂點；掃描 u 的期間，u 已出列但仍灰色，因此不要把這個不變量誤稱為每一條指令後都成立。

FIFO 為何保證先處理較近的點

把 queue 依隊首到隊尾寫為 $q_1,\cdots,q_r$。其距離非遞減，且 $d[q_r]\le d[q_1]+1$；空 queue 時條件視為成立。

初始只有 s。若本輪取出 u、$d[u]=k$，剩餘項依歸納只有 k 或 k+1，且非遞減；新發現的每個 v 都取得 k+1 並接在尾端，因此順序保持。本輪結束若 k 層已清空，隊首變成 k+1，下一輪才可能產生 k+2。由此每次 dequeue 的距離也非遞減。

每個有限 d[v] 都有一條實際 π 路徑：s 的長度 0；發現 v 時把 u 的路徑接 $u\to v$，長度恰 $d[u]+1$。因此真最短距離 $\delta(s,v)\le d[v]$，而 π 鏈的 d 每次下降 1，不可能形成 cycle。

但「找到一條路」本身只給上界，還不足以證明最短；下一步需要排除先找到較長路徑的可能。

BFS 最短距離的完整論證

假設有可達 v 最後 $d[v]\neq\delta(s,v)$，選 δ 最小的這種 v。$v\neq s$；取最短路徑中 v 的前一點 u，則 $\delta(s,u)=\delta(s,v)-1$，且依選法 $d[u]=\delta(s,u)$。u 有限距離，必定入列並最終出列。

掃描 $u\to v$ 時，若 v 仍白，就設 $d[v]=d[u]+1=\delta(s,v)$，矛盾。若 v 已發現，設它最初由 w 掃描發現。w 不晚於 u 出列；由 queue 層序，$d[w]\le d[u]$，所以 $d[v]=d[w]+1\le d[u]+1=\delta(s,v)$。結合上一頁 $\delta\le d$，仍得到相等。兩種情況都矛盾，故所有可達點的 d 均正確；不可達點若曾被發現就有實際路徑，亦矛盾，故其 d 保持 ∞。

每個可達非根點有一個 π，反向回溯恰 $d[v]$ 條邊後到 s；它們構成 BFS tree。鄰接順序可以改變 π 與最短路徑的選擇，但不能改變 d。

初始化 $\Theta(n)$，每個頂點最多入列與出列一次，每個可達點的鄰接表掃一次，總時間 $O(n+m)$，queue 與 d、π、color 的額外空間 $O(n)$。使用 matrix 時，每個已出列頂點掃 n 格，最壞 $\Theta(n^2)$。

從 d 出發的完整 Queue Trace

表中的 queue 是本輪**掃描結束後**的內容；括號內是 d 值，新增點同時取得前驅。

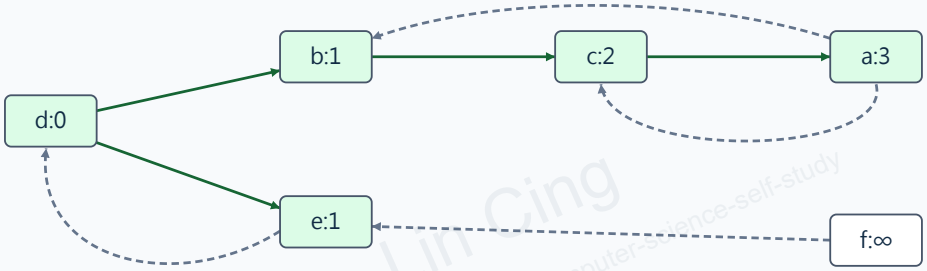
取出的 u	掃描結果	輪末 queue
尚未開始	初始化 d[d]=0	d(0)
d	b 白→d[b]=1,π[b]=d；e 白→d[e]=1,π[e]=d	b(1),e(1)
b	c 白→d[c]=2,π[c]=b	e(1),c(2)
e	d 已黑，不再入列	c(2)
c	a 白→d[a]=3,π[a]=c	a(3)
a	b、c 都已發現，不變	空

最後 (d[a],d[b],d[c],d[d],d[e],d[f])=(3,1,2,0,1,∞)，到 a 的最短路徑是 d→b→c→a。f→e 的存在不表示 d 可逆向到 f。

從 d 出發的完整 Queue Trace （續）

BFS：同一六點圖從 d 出發

格內 = 頂點:距離；綠實線 = π邊；灰虛線 = 其他原圖邊。



隊首順序 d(0) → b(1) → e(1) → c(2) → a(3)；f 保持∞。

有向邊距離、等權前提與最短路徑計數

對任一來源可達的 u→v，δ(s,v)≤δ(s,u)+1；有向圖沒有反向不等式。本例 a→b 從距離 3 指到 1，差為 2。無向邊兩方向都存在時，才有 |d[u]−d[v]|≤1。

等正權 w 時，最短權重是 w・d[v]。一般權重不可用 BFS：s→t 權 9，s→x 權 1，x→t 權 1，BFS 的 1 邊路徑權 9 大於 2 邊路徑權 2。權重全為零時可達路徑都權 0；有負權或負 cycle 更需另行定義問題。

若要算單位權圖最短路徑數，設 ways[s]=1，其餘 0。發現白 v 時照常設 d 與入列；之後對**每條**滿足 d[v]=d[u]+1 的 u→v 加上 ways[u]。同一條簡單圖邊只算一次。所有距離 k−1 的點都在距離 k 的點出列前完成，故出列 u 時 ways[u] 已收到所有最短前驅貢獻。

例如 0→1,0→2,1→3,2→3,1→4,2→4,3→5,4→5：d=(0,1,1,2,2,3)，ways=(1,1,1,2,2,4)。這是 O(n+m) 次加法；若計入大整數成本，路徑數可能有 Θ(n) 位元，不能把每次加法無條件視為固定時間。

22.3 Depth-first search

DFS（深度優先搜尋）先完成目前遞迴分支，再處理下一鄰點。外層必須掃全部頂點，才能包含不從第一個根可達的部分。以下 time 是**發現與完成事件**的計數，不是處理邊的計數。

```
DFS(G,order):
  for v in V: color[v]=WHITE; π[v]=NIL
  time=0; finished=empty list
  for u in order:
    if color[u]==WHITE: Visit(u)
Visit(u):
  time=time+1; tin[u]=time; color[u]=GRAY
  for v in Adj[u]:
    if color[v]==WHITE: π[v]=u; Visit(v)
  color[u]=BLACK; time=time+1; tout[u]=time
  append(finished,u)
```

每個點只被 Visit 一次，每個鄰接項掃一次：Θ(n+m) 時間，遞迴深度最壞 n、額外空間 O(n)。實際寫迭代版本時，stack frame 要保存「下一個待掃鄰點」或完成事件；只把鄰點一次全 push 的 visited 走訪不自動給相同 DFS forest 或完成時間。

發現與完成區間為何只能巢狀或分離

考慮不同頂點 u、v，設 tin[u]<tin[v]。若 v 在 u 完成前發現，當時 u 的 Visit 還在 stack 中；v 是 u 的後代，且 v 的 Visit 必先返回，故 tin[u]<tin[v]<tout[v]<tout[u]。若 v 在 u 完成後才發現，則 tin[u]<tout[u]<tin[v]<tout[v]，兩區間分離。

這排除了交錯 tin[u]<tin[v]<tout[u]<tout[v]。反過來，嚴格包含的時間區間意味發現 v 時 u 仍在 stack，故 v 是 u 的真後代。這就是 parenthesis theorem（括號定理）；它依賴遞迴必須後進先出地返回。

六點圖按 a,b,c,d,e,f 做 DFS，得到下表。括號串為 (a(b(c c)b)a)(d(e e)d)(f f)，所有數字 1 到 12 各使用一次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

發現與完成區間為何只能巢狀或分離 （續）

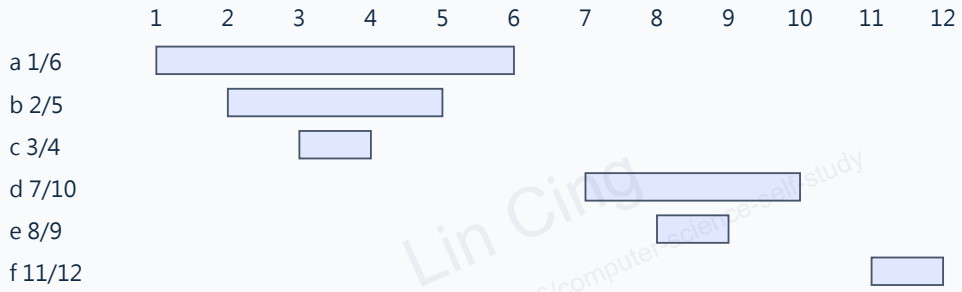
點	π	tin	tout
a	NIL	1	6
b	a	2	5
c	b	3	4
d	NIL	7	10
e	d	8	9
f	NIL	11	12

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

發現與完成區間為何只能巢狀或分離 （續）

DFS 的發現 / 完成事件

每條橫帶從 tin 到 tout；子帶巢狀，另兩棵根的區間分離。



a 包住 b,c；d 包住 e；f 是第三根。不同點區間不可能交錯。

White-path Theorem 的時間條件

在 Visit(u) **尚未把 u 改灰之前**，v 是本次 DFS 中 u 的後代（允許 v=u），當且僅當當時有 u 到 v 的路徑，其所有點都為白。這個時間切點讓 u 本身也符合白色條件。

「後代⇒白路徑」：forest 上 u 到 v 的 tree path 是原圖的路徑；u 的各真後代都在 tin[u] 之後才發現，故當時整條 path 為白。

「白路徑⇒後代」：沿白路徑找第一個假設沒有成為 u 後代的點 y，其前點 x 已是 u 的後代（可能就是 u）。Visit(x) 掃 x→y 時，若 y 仍白就遞迴，y 必為後代；若已發現，因 y 在 tin[u] 切點仍白，它是在 u 的尚未結束遞迴期間發現，也必屬 u 的子樹。兩種情況都矛盾。

必須保留「當時全白」：圖 a→b,a→c,c→b，先掃 b。到 c 被發現時 b 已黑；雖然 c 可達 b，b 不會變成 c 的後代。一般可達性不能直接取代 white-path 條件。

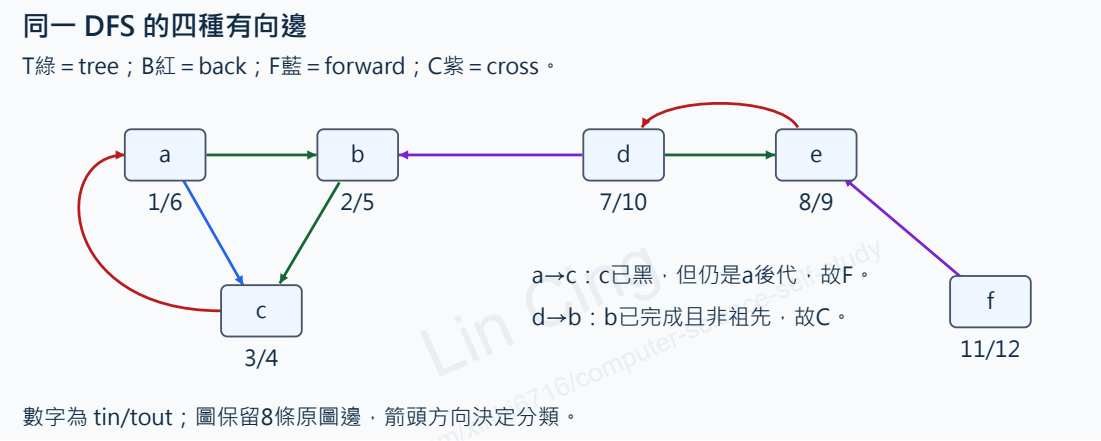
有向 DFS 的四種邊

掃描 u→v 時，u 是當下最深的灰色點，全部灰點恰為一條 stack 祖先鏈。v 白則是 tree edge，v 灰則是 back edge（回邊，含 self-loop）；v 黑時，用 tin 比較：tin[u]<tin[v] 是 forward edge，反之是 cross edge。

邊型	forest 關係	完成後可核對的時間式
tree	π[v]=u	tin[u]<tin[v]<tout[v]<tout[u]
forward	非 tree 的真後代邊	同上；只靠時間不能區分 tree/forward
back	v 為 u 的祖先，或 v=u	祖先時 tin[v]<tin[u]<tout[u]<tout[v]
cross	彼此無祖先關係	tout[v]<tin[u]；掃描方向不能倒過來讀

本例依實際掃描順序：a→b(T)、b→c(T)、c→a(B)、a→c(F)、d→b(C)、d→e(T)、e→d(B)、f→e(C)。a→c 是在 c 完成後才由 a 掃到，故看到黑色不代表一定是 cross。

有向 DFS 的四種邊 （續）



無向圖的邊型與 Cycle 判斷

無向邊會在兩個 adjacency list 出現，但作為一條邊分類時，取**第一次掃到該邊**的方向。設 tin[u] <tin[v]；因 u 的 list 中有 v，v 必在 u 完成前發現。第一次掃到時，若是 u→v，v 尚白，否則早已由 v→u 掃到，所以是 tree；若先掃 v→u，u 仍灰，是 back。故無向 DFS 只有 tree/back，沒有真正的 forward/cross。

對無向簡單圖檢查 cycle，遇到灰色父節點的反向鄰接項不能當成額外 cycle。若允許平行邊，則須用**邊識別碼**略過同一條 parent edge，不能略過所有指向 parent 的邊；第二條平行邊可形成長度 2 的 cycle（依允許 multigraph cycle 的慣例）。

有向圖則有 cycle ⇔ DFS 出現 back edge。回邊 u→v 搭配 forest 中 v 到 u 的有向 tree path 構成 cycle。反向證明：取 cycle 中最早發現的 v，當時其餘 cycle 點皆白；循 cycle 到 v 的前點 u 是白路徑，由 white-path theorem，u 是 v 的後代，所以最後一條 u→v 為 back edge。self-loop 直接落在同一結論。

22.4 Topological sort

Topological order（拓撲序）是所有頂點的排列，使每條 u→v 都滿足 pos[u]<pos[v]。有向 cycle 要求沿圈嚴格遞增又回到原位，不可能成立；DAG（有向無環圖）才存在此排列。

在 DAG 上做 DFS，把完成順序反轉即可，不必用比較排序。對任一掃描的 u→v：v 不可能灰，否則有 back edge/cycle；v 白時成為後代，tout[v]<tout[u]；v 黑時早已完成，仍有 tout[v]<tout[u]。故按完成時間遞減，所有邊都由前指後。時間 Θ(n+m)，額外空間 O(n)。

實作可在掃邊遇灰時回報 cycle，最後才交付拓撲序。不能對一般有向圖不檢查就把 reverse-finish list 標為合法排序；完成時間總能列出，但邊限制不一定成立。

例 a→c,b→c,c→d：按 a,b,c,d 做 DFS，tin/tout 分別 a=1/6,c=2/5,d=3/4,b=7/8，反轉完成序得 b,a,c,d。a,b,c,d 也合法；**a,c,b,d 不合法，因 b→c 被逆置**。

Kahn 的入度更新與失敗判定

先算 indegree[v]，把所有入度零點放 queue。每次取 u 輸出，對每條 u→v 執行 indegree[v]−=1；當且僅當變成零時入列。計數始終等於尚未輸出頂點誘導子圖中的入度，因此輸出 u 時沒有仍未輸出的前驅，已輸出的 prefix 永遠符合邊的順序。

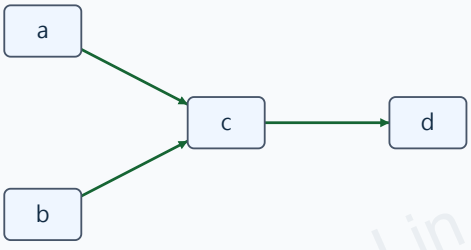
若剩餘非空卻無入度零點，任取殘餘點，反覆沿一條殘餘入邊走；有限頂點必重複，得到 cycle。反之 DAG 的非空誘導子圖必有入度零點，所以演算法會輸出全部 n 點。只輸出少於 n 個就回報失敗；**殘餘點可能只是 cycle 的下游，不代表每個殘餘點都在 cycle 上**。

目前輸出 u	c 的剩餘入度	d 的剩餘入度	輸末 queue
初始	2	1	a,b
a	1	1	b
b	0	1	c
c	0	0	d
d	0	0	空

Kahn 的入度更新與失敗判定 （續）

Kahn：入度與 Queue 的同步狀態

同一 DAG：a→c、b→c、c→d；移除起點後才減各出邊終點入度。



操作	in(c)	in(d)	Queue
初始	2	1	a,b
移除a	1	1	b
移除b	0	1	c
移除c	0	0	d
移除d	0	0	空

輸出 a,b,c,d；a,c,b,d 把 b→c 逆置，不合法。

22.5 Strongly connected components

定義 u~v ⇔ u 可達 v 且 v 可達 u。零長路徑給自反性，定義給對稱性，串接路徑給遞移性，因此它是等價關係；其等價類是 SCC（強連通分量），也就是按包含關係極大的互相可達集合，**不是最大大小的那一類**。

縮合圖 condensation 的點是各 SCC，不同分量間只要有一條原圖邊就加邊。它必為 DAG：若不同類沿圈可達，圈上各類彼此可達，應合為同一等價類，矛盾。把所有邊反向會同時反轉兩方向的可達路徑，所以 G 和 G^T 有完全相同的 SCC。

六點圖的 SCC 為 A={a,b,c}、D={d,e}、F={f}，縮合邊 F→D→A。注意 G^T 有 A→D→F；從 a 隨意在 G^T 起跑可以把三個 SCC 都走到。**轉置本身不會阻止跨分量，必須配合正確根順序與已訪問標記。**

為何不同 SCC 的完成時間有方向

以下 tin、tout 都是**第一趟 G 的 DFS**。令 tin(C)=min tin[v]、F(C)=max tout[v]，v∈C。若不同 SCC 間有 C→D，則 F(C)>F(D)。證明分兩種首次發現先後：

1. 若 tin(C)<tin(D)，C 第一個被發現的 x 當時能沿 C 內部白路徑到邊尾，再跨到 D 並走 D 內部白路徑。由 white-path theorem，C 其餘點及 D 全成 x 的後代，所以 F(C)=tout[x]>F(D)。C 內兩點間可取完全位於 C 的路徑：若路徑經外點，該外點也與 C 互相可達，會屬 C。
2. 若 tin(D)<tin(C)，D 第一點 y 可以在全白的 D 中走到全部 D，所以 F(D)=tout[y]。D 不可能到 C，否則搭配 C→D 使兩類合併；因此 y 的整棵遞迴走不到 C，C 要等 y 完成後才可能發現，故 F(C)>F(D)。

所以 G^T 的跨分量邊 C→D 一定滿足 F(C)<F(D)。這是**分量最大完成時間**的比較；不能要求 C 中每個點都比 D 中每個點晚完成。例如 a↔b、a→c，先掃 a→b，再掃 a→c，得到 tout[b]=3、tout[c]=5、tout[a]=6：F({a,b})=6>F({c})=5，但 b 比 c 早完成。真正跨類的那條 a→c 仍有 tout[a]>tout[c]；不同 SCC 的邊不可能是 back edge。

兩趟 DFS 的完整演算法與不跨界證明

```
SCC(G):
  first=DFS(G,所有頂點的固定順序)
  roots=reverse(first.finished)    // 第一趟的完成序，先保存
  GT=transpose(G)
  第二趟重新設所有頂點為WHITE，π=NIL
  按roots依序，遇WHITE u才在GT做Visit(u)
  每次新Visit根的整棵tree輸出為一個分量
```

歸納已輸出的每棵 tree 恰為一個完整 SCC。下一個白根 u 所在 C 有全部未處理分量中最大的 F(C)：如果 C 的最大完成點已處理，依歸納 C 整類都已處理，與 u 白矛盾。

u 可沿 C 內部路徑走到全部 C；其餘 C 仍白，故第二趟會收齊。若 G^T 有 C→D 的跨界邊，F(D)>F(C)，所以 D 必已處理為黑色，Visit 不遞迴；更不可能經此跳往另一白類。因此此 tree 既不少收也不多收，完成歸納。

兩次 DFS 各 Θ(n+m)，轉置 Θ(n+m)。反轉 append 得到的完成序為 Θ(n)，不必用 O(n log n) 排序；總時間 Θ(n+m)，新增轉置與各陣列空間 O(n+m)。

同一六點圖的兩趟結果

第一趟完成序 c,b,a,e,d,f；因此第二趟候選根序 f,d,e,a,b,c。轉置的表為 a:[c]、b:[a,d]、c:[a,b]、d:[e]、e:[d,f]、f:[]（各表按字母排列）。

候選根	G^T 的實際新發現序	遇到的跨類邊	本次輸出
f	f	無	{f}
d	d,e	e→f，f 已黑	{d,e}
e	跳過；已黑	—	不新增
a	a,c,b	b→d，d 已黑	{a,b,c}
b,c	跳過；已黑	—	不新增

F(A)=6、F(D)=10、F(F)=12，與原縮合邊 F→D→A 一致遞減。若改用第一趟 discovery 遞減，c 可能在 a 前，但這個例子未必失敗；規格正確性不能靠某次剛好成功。最小反例 x→y：第一趟 tin[x]=1,tin[y]=2，若第二趟先從 y 在 G^T 走，會沿 y→x 合併兩個本應分離的 SCC。

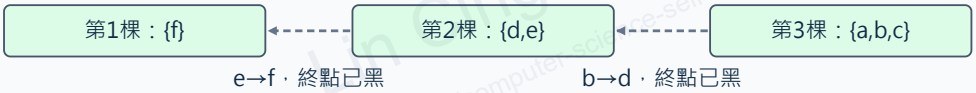
同一六點圖的兩趟結果 （續）

SCC：完成值排序如何阻止第二趟多收

第一趟 G 的分量最大完成值：F({f})=12、F({d,e})=10、F({a,b,c})=6。

原圖縮合 DAG

GT 的根順序與已黑邊界



綠框依序收齊一類；灰虛箭頭可跨類，但不能遞迴到已黑頂點。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 22。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不相交集合：只合併，不拆分

同一組元素分成哪些集合，會隨合併改變；**查詢要回答的是「兩個元素現在是否同組」**。

本章要解決的問題	對應方法
如何代表一個集合？	串列的集合紀錄，或森林的根
合併時要改多少資料？	短串列接長串列；低 rank 根接高 rank 根
查找為何愈做愈便宜？	路徑壓縮，加上整段操作的攤銷分析

先備知識：指標、根樹、漸近記號，以及位能法。rank、level、iter 都會在本章定義。

三個操作，維護同一份集合分割

集合分割（partition）要求：每個已建立元素恰屬一組；各組非空且兩兩不交。

操作	輸入前提	結果
MAKE-SET(x)	x 尚未建立	建立單點集合 {x}
FIND(x)	x 已存在	回傳 x 所在集合的代表
UNION(x,y)	x、y 已存在	合併兩者所在集合；同組時不變

代表是識別集合的代號，不保證是最小鍵。 同組元素的 FIND 必須回傳同一代表；不同組則回傳不同代表。

呼叫次數與成功合併，要分開計算

本章先建立 n 個單點。m 計入使用者呼叫的 MAKE-SET、FIND、UNION，所以 m≥n；UNION 內部呼叫的 FIND 不再重複加進 m。

名稱	規格與計數
成功合併	集合數減一，因此最多 n−1 次
UNION 呼叫	可以重複，次數不受 n−1 限制
LINK(r,s)	只接受兩個**不同的根**，不接受任意元素

> UNION 先 FIND 兩端；代表相同就返回。**不能再 LINK 同一根，也不能因此增加 rank。**

FIND 可以改 parent，但不改集合分割。刪邊、拆集合與撤銷合併，需要另外的結構與成本分析。

無向連通：新邊會合併哪些組？

先為每個頂點 MAKE-SET，再逐邊 UNION。圖解使用這一串輸入：ab、cd、bc、ab、ef、de。

邊串流：partition 的每次改變

每列為處理新邊後的partition；第4列重複邊不改集合數。

			集合數
ab	⇒	{a,b} {c} {d} {e} {f}	5
cd	⇒	{a,b} {c,d} {e} {f}	4
bc	⇒	{a,b,c,d} {e} {f}	3
ab	⇒	{a,b,c,d} {e} {f}	3
ef	⇒	{a,b,c,d} {e,f}	2
de	⇒	{a,b,c,d,e,f}	1

成功5次合併：6→5→4→3→3→2→1；全部6次 UNION 仍各需做代表檢查。

第四條 ab 是重複邊：呼叫照算，集合不再改變。 六次 UNION 中只有五次成功合併。

為什麼這個方法恰好得到連通分量？

不變量：處理過的邊所形成的子圖，其連通分量恰好等於目前的集合分割。

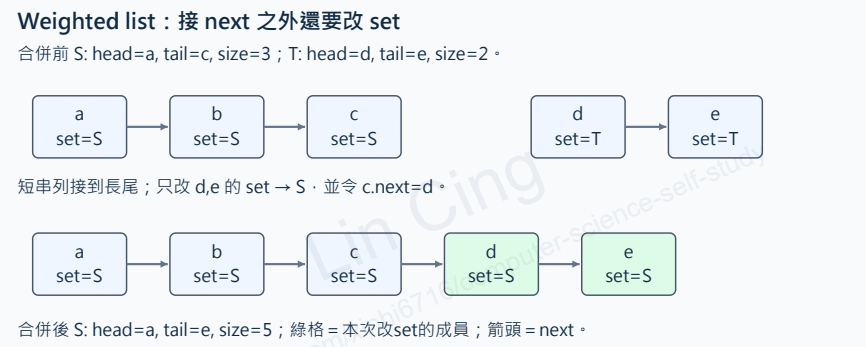
步驟	圖的連通性	集合分割
還沒有邊	每個頂點獨立	每點各成一組
新邊兩端已同組	原本就連通	UNION 不改分割
新邊兩端不同組	這條邊接起兩個分量	UNION 恰好合併這兩組

由歸納可知，最後 FIND(u)=FIND(v) 若且唯若存在無向 u–v 路徑。

靜態圖可用 DFS/BFS 在 Θ(n+|E|) 內找分量；DSU 也能處理**逐步加邊的連通查詢**。有向圖忽略方向後只得到弱連通分量，不能用來求 SCC。

串列表示：接上尾端，還沒完成合併

每組有一份集合紀錄（set record）：head、tail、size。每個成員另存 next 與 set 指標；FIND(x) 直接回傳 x.set.head，因此是 O(1)。



> a→b→c 與 d→e 合併後，tail 由 c 改為 e、size 由 3 改為 5。**d、e 的 set 也要改指新的紀錄**，FIND 才會回傳 a。

加權串列的 UNION，逐項做完

令 S=x.set、T=y.set。同組直接返回；不同組時，讓 S 是較長的串列，同長保留第一組。

次序	修改	目的
1	S.tail.next=T.head	把兩串列接起來
2	保存 T 的原 head；更新 S.tail、S.size	保留走訪起點，更新集合紀錄
3	沿 T 的 next，把每個成員的 set 改為 S	讓短串列的 FIND 改回傳 S.head
4	完成上述改寫後才廢棄 T 紀錄	不留下指向失效紀錄的成員

走訪時不改 T 內部的 next，因而不會遺失後繼。成本為 $\Theta(1+\min(|S|,|T|))$，**有 tail 只讓接線是 O(1)，不是整個 UNION。**

不加權，為什麼會退化成平方成本？

若固定把第二組接到第一組，依序做 UNION(x_2,x_1)、UNION(x_3,x_2)、…，每次新單點都要吸收舊長串列。

合併	必須重寫 set 的成員數
UNION(x_2,x_1)	1
UNION(x_3,x_2)	2
…	…
UNION(x_n,x_{n-1})	n−1

總改寫數為 $n(n-1)/2$。外部只有 n 次建立與 n−1 次合併，即 $m=2n-1$，實際成本卻是 $\Theta(n^2)$。

每次移短組，同一成員最多搬幾次？

固定一個成員 x。每次它的 set 被改寫，表示它在較短的集合；合併後所在組的大小**至少加倍**。

改 k 次後至少有 2^k 個成員，而總數不超過 n，所以 $k \le \lfloor \log_2 n \rfloor$。加總所有成員的改寫，再加上 m 次呼叫的常數開銷，得到 **$O(m+n \log n)$**；涵蓋小 n 可寫 $O(m+n \log(n+1))$。

要分清楚的量	結論
整段操作總成本	$O(m+n \log n)$
單次合併兩個各 n/2 的集合	仍需 $\Theta(n)$
$n=2^k$，同大小集合分輪合併	每輪改 n/2 筆，共 $(n/2)\log_2 n$ 筆

> 攤銷總界不能直接當成單次最壞界。短組策略也不保證所有序列都只有線性成本。

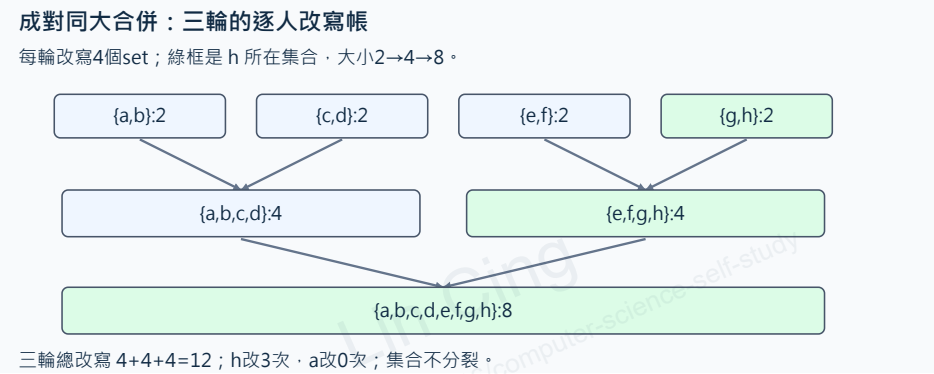
八元素：每次合併到底改了誰？

同長保留第一組。這一組操作稍後也會用來比較森林表示。

UNION	原大小	改 set 的成員	新代表／大小	累計改寫
a,b	1,1	b	a／2	1
c,d	1,1	d	c／2	2
e,f	1,1	f	e／2	3
g,h	1,1	h	g／2	4
a,c	2,2	c,d	a／4	6
e,g	2,2	g,h	e／4	8
a,e	4,4	e,f,g,h	a／8	12

h 的代表依 h→g→e→a，所屬組大小由 1→2→4→8；a 一次也不用搬。**合計 12 次，不代表每人都搬三次。**

用同一張圖，對照逐輪成本與個別成員



每輪的總改寫數與「固定一人搬幾次」是不同分母。表格逐輪加總為 12；h 個人則恰好改寫三次。

若反過來每次只把新單點接到大集合，總改寫可以只有 n−1 次；這不影響上界，也不代表所有序列皆線性。

森林表示：先找根，再接線

每個節點存 parent、rank。MAKE-SET(x) 設 parent[x]=x、rank[x]=0；根的 parent 指向自己。

```
UNION(x,y):
    r=FIND(x); s=FIND(y)
    若 r==s: 回傳 r
    回傳 LINK(r,s)
LINK(r,s):                    // r,s 是不同的根
    若 rank[r]<rank[s]: 交換 r,s
    parent[s]=r
    若 rank[r]==rank[s]: rank[r]+=1
    回傳 r
```

LINK 只接受兩個不同的根，所以不會形成環，也不會拆開舊集合。不同 rank 不加一；相同 rank 只讓保留根加一。

串列比較 size；森林比較 rank，後者不是精確的成員數。

兩趟 FIND：先找到根，再沿原路改接

從 x 出發，沿 parent 往上走，遇到自指的根就停。第一趟只找根 r，不改指標；第二趟再把**原查找路徑**上的每個節點直接接到 r。

```
FIND(x):
    r=x
    當 parent[r]!=r: r=parent[r]
    當 parent[x]!=x:
        old=parent[x]
        parent[x]=r
        x=old
    回傳 r
```

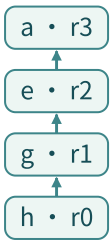
old 保存的是「覆寫前要走的下一站」。若改用 x=parent[x]，讀到的已是剛寫入的根，便跳過中間節點。遞迴版 parent[x]=FIND(parent[x]) 則在呼叫回傳時逐點改接。

兩趟迭代額外空間為 O(1)；遞迴堆疊與原路徑高度同階。走 s 點需 Θ(s)，只改本次路徑，不改 rank。

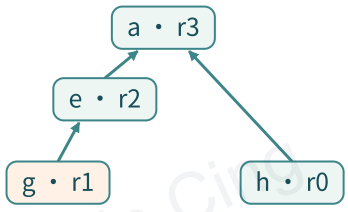
找根正確，不代表整條路徑都已壓縮

箭頭由**子指向父**；只畫 h→g→e→a，根 a 自指，其他節點不變。

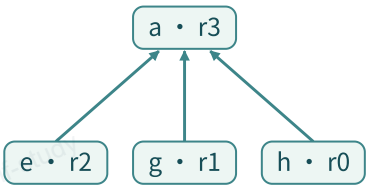
原路徑



覆寫後直接跳根



保存舊父，完成壓縮



第二趟目前節點	先存 old	再改 parent	繼續到
h	g	a	g
g	e	a	e
e	a	a（不變）	a，然後停止

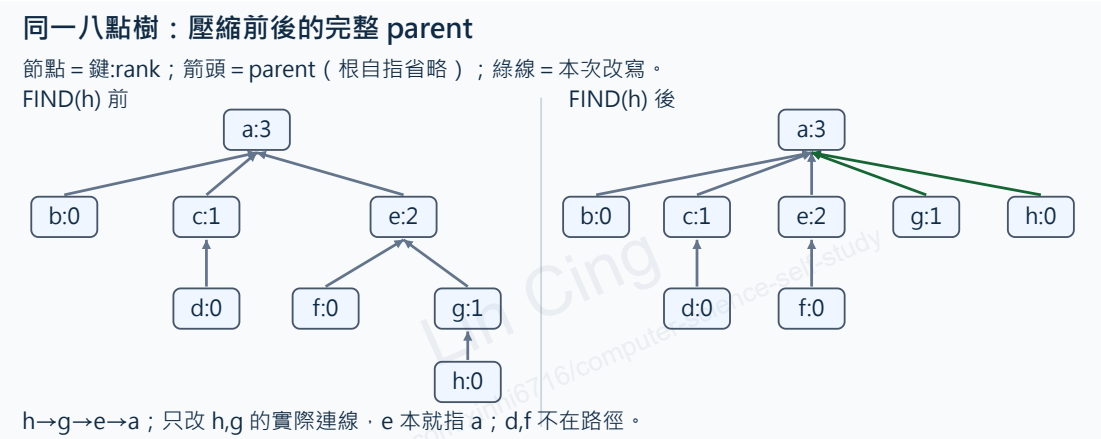
同一八元素的 rank 與壓縮前後

沿用前面七次 UNION；同 rank 時保留第一個根。先記錄每個節點的 rank 與 parent，再看 FIND(h) 改了哪些欄位。

點	a	b	c	d	e	f	g	h
rank	3	0	1	0	2	0	1	0
FIND(h)前parent	a	a	a	c	a	e	e	g
FIND(h)後parent	a	a	a	c	a	e	a	a

- 第一趟走 h→g→e→a，共三條父邊。第二趟把沿途的 parent 設為 a。
- 真正改變的是 **h、g 的 parent**；e 原本就接 a，其他節點不在路徑上。
- **rank 全部不變**。它記錄合併歷史，不能在壓縮後改填目前高度。

看圖核對：改接路徑，保留其餘分支



> 再查 h 只走一條邊；查 d 仍走 d→c→a。**一次 FIND 只壓縮自己的查找路徑。**

Rank 為什麼能限制樹高？

rank 不是「目前高度」，但能當作它的**上界**。對操作次數歸納：

操作	樹高如何改變	為何仍有 height ≤ rank
建立單點	高度為 0	初始 rank 也是 0
不同 rank 合併	低 rank 的根接到高 rank 的根	低 rank 至少差 1，多接一條邊仍不超過大 rank
相同 rank 合併	高度最多增加 1	保留根的 rank 同時加 1
路徑壓縮	改接到祖先，只會縮短路徑	rank 不變，上界仍成立

> **壓縮後不重算 rank。** 例如根的 rank=2，壓縮後高度可以是 1；兩者不必相等。

Rank 也記得合併時累積的成員

根的 rank 每升一級，都必須合併兩個同 rank 的不同集合：

- rank=0：至少 1 個成員。
- 兩個 rank=r−1 的根合併：各至少有 2^{r-1} 個成員，新根至少有 2^r 個。
- 不同 rank 合併只增加成員；FIND 不改變集合成員。

非根的 rank 固定在它失去根身分之時。**當時的集合至少有 2^r 個成員**，所以仍有 $2^r \le n$，即 $r \le \lfloor \log_2 n \rfloor$。這不是目前子樹大小的下界。

> 只按 rank 合併，單次 FIND 為 $O(\log n)$；再加路徑壓縮，改進的是**整段操作的攤銷界**。第一個深葉查找仍可能走 $\Theta(\log n)$ 條邊。

21.4 Rank 與路徑壓縮的精細分析

Ackermann 的迭代是函數合成

固定整段序列最終的元素數 n，令 $\alpha = \alpha(n)$。分析時不隨目前已建立的元素數重算 α。對 $j \geq 1$，定義

$$f^{(0)}(j) = j, \qquad f^{(i+1)}(j) = f(f^{(i)}(j)), \qquad A_0(j) = j + 1, \quad A_k(j) = A_{k-1}^{(j+1)}(j)\ (k \geq 1).$$

上標表示**反覆把輸出當成下一次輸入**。由 $A_0^{(i)}(j) = j + i$ 得 $A_1(j) = 2j + 1$；對 i 歸納可得 $A_1^{(i)}(j) = 2^i(j + 1) - 1$，所以

$$A_2(j) = 2^{j+1}(j + 1) - 1, \qquad A_3(1) = A_2(A_2(1)) = A_2(7) = 2047.$$

各 A_k 對輸入嚴格遞增，且 $A_k(j) \geq j + 1$：由 A_0 開始歸納，較大的輸入會經過更多次、且每次都增加的迭代。又 $j + 1 \geq 2$，故 $A_k(j) > A_{k-1}(j)$。後面的分層界會用到這兩種單調性。

本章採用的 inverse-Ackermann 定義

採 Cormen 第三版 §21.4 的單參數版本：

$$\alpha(n) = \min\{k \geq 0 : A_k(1) \geq n\}.$$

k	0	1	2	3	4
$A_k(1)$	2	3	7	2047	大於 10^{80}

所以 $\alpha(2) = 0$、$\alpha(8) = 3$、$\alpha(2048) = 4$。不同教材可能採雙參數定義；不能直接混用其數值。

完整總界是 $O(m(1 + \alpha(n)))$，其中 m 次外部操作包含 n 次 MAKE-SET。只有 $n \geq 3$ 時才能簡寫為 $O(m\alpha(n))$；$n \leq 2$ 仍有常數操作成本。

哪些量會隨操作改變？

對操作數歸納可證：根以 parent 指向自己；非根的 rank 嚴格小於父 rank。相同 rank 的兩根合併時，保留根的 rank 加一；不同 rank 合併時，把小 rank 根接到大 rank 根。壓縮只改 parent，不改 rank。

節點一旦成為非根，其 rank 就固定。 後續它所指父節點的 rank 可能增加，或因壓縮改指更高的祖先；兩者都不會降低父 rank。這才是 level 不下降的依據。

rank 為 r 的根至少包含 2^r 個元素。非根沿用失去根身分時的歷史下界，故任一節點的 rank 至多 $\lfloor \log_2 n \rfloor < n$。這不是壓縮後的目前子樹大小。

接下來只對 rank 正、且不是根的節點定義 level 與 iter；rank 0 和根另外計費。

level 決定層級，iter 記錄層內進度

令 $r = \operatorname{rank}(x) \geq 1$、$p = \operatorname{rank}(\operatorname{parent}(x))$，定義

$$\ell(x) = \max\{k \geq 0 : p \geq A_k(r)\}, \qquad i(x) = \max\{j \geq 1 : p \geq A^{(j)}_{\ell(x)}(r)\}.$$

正文以 $\ell = \operatorname{level}$、$i = \operatorname{iter}$ 縮短公式。先找父 rank 能跨過的最高層，再算該層可迭代幾次。

因 $p \geq r + 1 = A_0(r)$，有 $\ell \geq 0$。又 $A_\alpha(r) \geq A_\alpha(1) \geq n > p$，故 $\ell < \alpha$。定義給 $i \geq 1$；若 $i \geq r + 1$，就會有

$$p \geq A^{(r+1)}_\ell(r) = A_{\ell+1}(r),$$

與 ℓ 的最大性矛盾。因此 $0 \leq \ell < \alpha$、$1 \leq i \leq r$。

用非負位能記錄尚可支付的工作

逐節點定義位能，再加總成整座森林的位能：

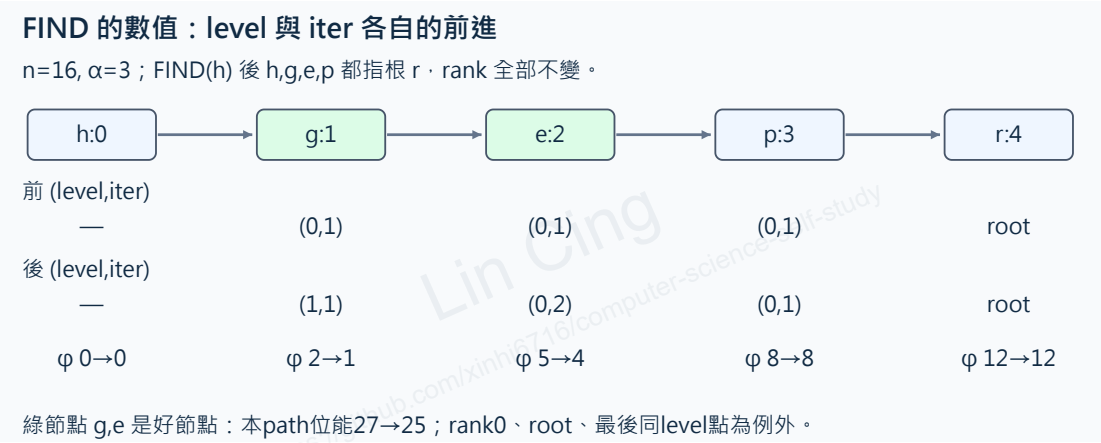
$$\phi(x)=\begin{cases}\alpha r, & x\text{ 是根或 }r=0,\\ (\alpha-\ell(x))r-i(x), & x\text{ 非根且 }r\ge 1,\end{cases}\qquad \Phi=\sum_{x}\phi(x).$$

非根的下界為 $(\alpha-(\alpha-1))r-r=0$，上界為 $\alpha r-1$。根的位能是 αr，rank 0 的位能為 0；所以 Φ 永遠非負。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

五點路徑的位能帳

取 $n=16$、$\alpha=3$，壓縮 rank 依序為 0、1、2、3、4 的路徑。圖中的根保持自指。



兩個好節點各釋放 1：g 靠 level 上升，e 靠同層 iter 上升。其餘三點位能不變，所以整條路徑位能由 27 降至 25。

level 上升時，iter 為何可以下降？

固定非根的 rank r。同一層內，父 rank 增加只會讓 iter 不變或增加；每多一次迭代，ϕ 就減一。

若層級由 k 升到 $k' > k$，iter 可能由 i 重設為較小的 i'。然而

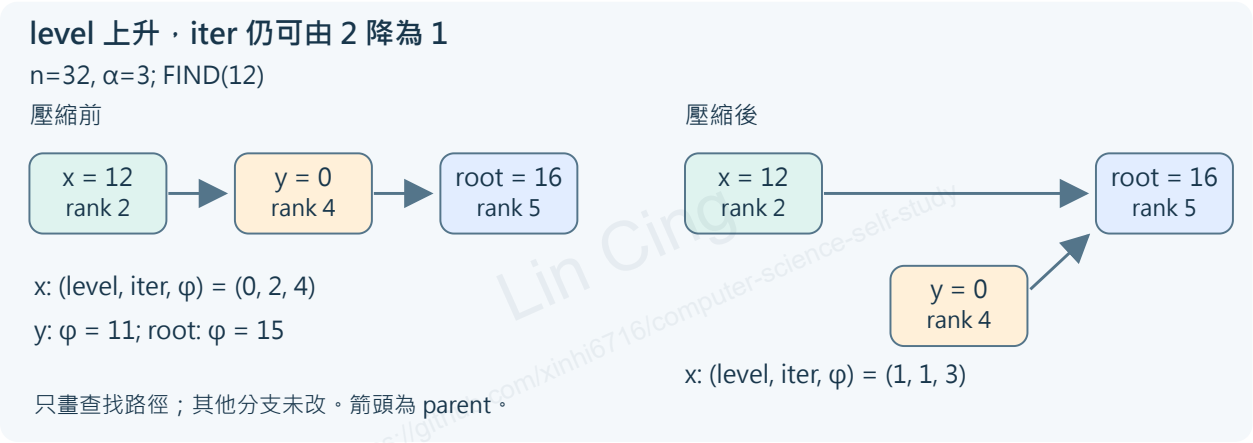
$$\Delta\phi = -(k' - k)r + (i - i') \le -r + (r - 1) = -1.$$

不等式用了 $k' - k \ge 1$、$i \le r$、$i' \ge 1$。**跨層的位能下降，足以支付 iter 重設。** 因此需要單調的是「先比較 level，再比較 iter」的字典序（lexicographic order），不是兩個座標各自遞增。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一個 iter 確實下降的合法 FIND

取 $n = 32$、$\alpha = 3$。查找路徑是節點 $12 \to 0 \to 16$，rank 依序為 $2, 4, 5$。圖只保留這次路徑，其他分支不變。



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨層數例的逐項計算

對 $x = 12$，固定 rank 為 2；壓縮只把父 rank 由 4 換成 5。

量	壓縮前	壓縮後
level	$A_0(2) = 3 \le 4 < A_1(2) = 5$，故為 0	$A_1(2) = 5 \le 5 < A_2(2) = 23$，故為 1
iter	$A_0^{(2)}(2) = 4 \le 4 < A_0^{(3)}(2) = 5$，故為 2	$A_1(2) = 5 \le 5 < A_1^{(2)}(2) = 11$，故為 1
$\phi(x)$	$(3-0)2 - 2 = 4$	$(3-1)2 - 1 = 3$

節點 0 的位能保持 11，根的位能保持 15；其餘分支都不變。因此全森林 $\Delta\Phi = (3 + 11 + 15) - (4 + 11 + 15) = -1$。這裡只列路徑的加總；並未假設其他 29 個節點的位能為零。

如何真的建立這條路徑？

下列只用 MAKE-SET、UNION、FIND；同 rank 保留第一個根。先建立鍵 0 到 31 的單點。

操作	可核對的結果
對 0..15 分輪合併相鄰、同大小區塊	根 0 的 rank 為 4；節點 12 的 rank 為 2
FIND(12)	把 12 的 parent 改為 0；不改 rank
同法合併 16..31	根 16 的 rank 為 4
UNION(16,0)	根 16 升為 rank 5；0 的 parent 為 16
再次 FIND(12)	沿 $12 \to 0 \to 16$；12 的 parent 改為 16

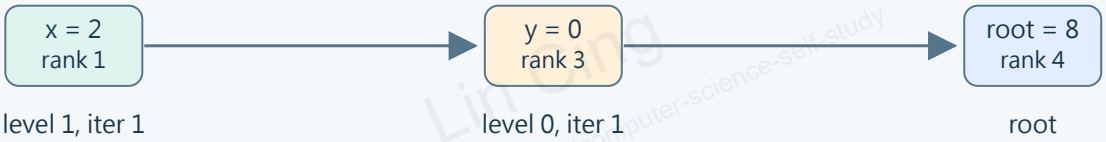
例如前半部的輪次是先合併 (0,1)、(2,3)、…、(14,15)，再合併 (0,2)、(4,6)、…，接著 (0,4)、(8,12)，最後 (0,8)。這說明 rank 數值來自合法歷史，沒有任意指定節點的 rank。

沿路的 level 不必遞增

取 $n = 16$。先把 0..7 合成根 0、rank 3 的集合並執行 FIND(2)；再把 8..15 合成根 8、rank 3 的集合，執行 UNION(8,0)。得到以下合法路徑。

rank 遞增，不代表 level 也遞增

n=16, α=3; FIND(2)



$A_1(1)=3 \le \text{rank}(y) < A_2(1)=7$; $A_0(3)=4 \le \text{rank}(\text{root}) < A_1(3)=7$

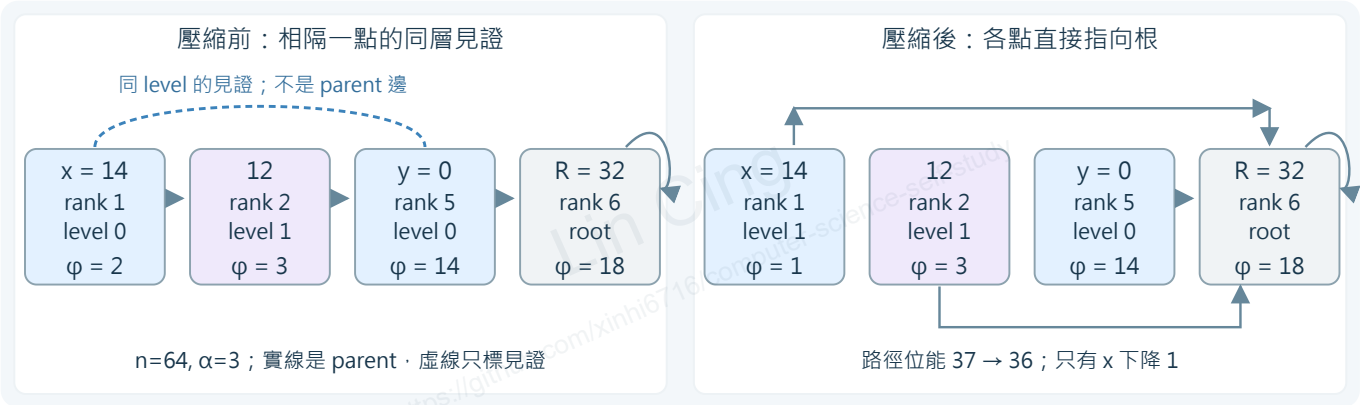
沿路 rank: 1 < 3 < 4；level: 1 > 0。

同 level 的最後一點才是例外；不用假設 level 沿路排序。

不同節點的 rank 也變了；不能把沿路比較，混同「同一節點隨時間」的單調性。

好節點的同層見證可以隔著其他 level

在壓縮前的路徑上，rank 大於 0 的非根節點 x 的後方若有另一個非根 y，且 $\ell(x) = \ell(y)$，就稱 x 為好節點。**找同層的後繼，不是只比較下一個節點。**



路徑 $14 \to 12 \to 0 \to 32$ 的 rank 為 1、2、5、6，level 為 0、1、0（根不定義 level）。14 的見證是 0；中間的 12 不必同層。兩個同層點不必在原樹中相鄰，壓縮後仍能替 14 帶來更高的父 rank。

先用合法合併建立這條路徑

取 64 個單點，同 rank 保留 UNION 的第一個根。「分輪建立區間」表示先合併相鄰單點，再合併相鄰同大小區塊。

操作	能從操作歷史核對的事實
分輪建立 0..31	根 0 的 rank=5；節點 12 的 rank=2，14 的 rank=1
FIND(12)	原路徑 $12 \to 8 \to 0$；只把 12 改接 0，14 仍接 12
分輪建立 32..63	根 32 的 rank=5
UNION(32,0)	兩根同 rank，32 升到 6；0 改接 32
尚未 FIND(14)	路徑為 $14 \to 12 \to 0 \to 32$，rank 為 1、2、5、6

FIND(12) 沿 $12 \to 8 \to 0$ 往上找到根，再把這條路上的父指標直接接到根。它不走進子樹，所以 14 仍接 12。之後的合併才把根 0 接到根 32；這些 rank 都來自合法操作，根 32 保持自指。

把每個節點的 level、iter、位能算出來

固定 $n = 64$、$\alpha = 3$。只對rank 大於 0 的非根節點計算 level、iter；根用 $\phi = \alpha r$。

節點	rank	父 rank：前→後	level：前→後	iter：前→後	ϕ：前→後
14	1	2→6	0→1	1→1	2→1
12	2	5→6	1→1	1→1	3→3
0	5	6→6	0→0	1→1	14→14
32（根）	6	6→6	不適用	不適用	18→18

對 14，壓縮前 $A_0(1) = 2 \le 2 < A_1(1) = 3$，故 level=0、iter=1。壓縮後 $A_1(1) = 3 \le 6 < A_2(1) = 7$，且 $A_1^{(2)}(1) = 7 > 6$，故 level=1、iter=1。位能由 $(3-0)1 - 1 = 2$ 降至 $(3-1)1 - 1 = 1$。

對 12，兩次皆有 $5 = A_1(2) \le p < 11 = A_1^{(2)}(2) < A_2(2) = 23$，故 level=1、iter=1。對 0，$A_0(5) = 6 \le 6 < A_0^{(2)}(5) = 7 < A_1(5) = 11$，故 level=0、iter=1、位能 14。路徑總和 $37 \to 36$；其餘點的 rank、父 rank 與根身分不變，故**全森林位能也恰減 1**。

好節點證明：每一步不等式各用了什麼？

所有量先固定在壓縮前。記 $\rho(v) = \text{rank}(v)$、$p_x = \text{parent}(x)$、$p_y = \text{parent}(y)$，根為 R。令 $r = \rho(x)$、$k = \ell(x) = \ell(y)$、$i = i(x)$。

逐步推導	理由
$\rho(R) \geq \rho(p_y)$	p_y 在通往根的路上，rank 沿非根 parent 邊嚴格增加
$\rho(p_y) \geq A_k(\rho(y))$	$\ell(y) = k$ 的定義
$A_k(\rho(y)) \geq A_k(\rho(p_x))$	y 就是 p_x 或其祖先，故 $\rho(y) \geq \rho(p_x)$；A_k 遞增
$A_k(\rho(p_x)) \geq A_k(A_k^{(i)}(r))$	iter 定義給 $\rho(p_x) \geq A_k^{(i)}(r)$，再用 A_k 遞增
$A_k(A_k^{(i)}(r)) = A_k^{(i+1)}(r)$	函數再合成一次，不是數值相乘

因此新父的 rank 至少為 $A_k^{(i+1)}(r)$。這裡沒有使用「路徑上的 level 遞增」；只使用 rank 的祖先次序與同一個 k 的函數單調性。

跨過下一個門檻，為何一定能支付工作？

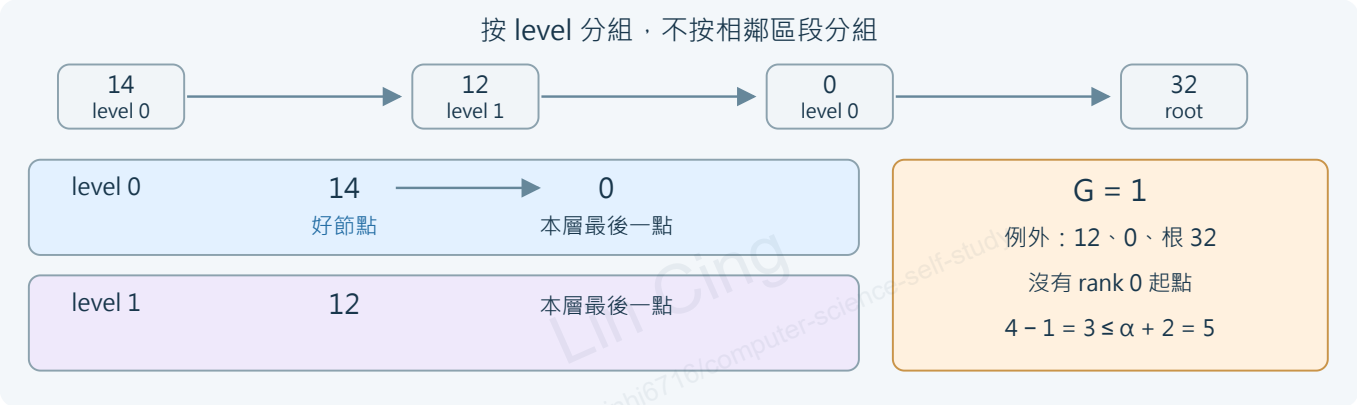
壓縮使 x 的父變成 R，但 r 不變。前頁證明的是新父 rank 跨過**舊 level 的下一次迭代門檻**，不保證新的 iter 必定比舊值大；必須分兩種情況。

壓縮後	位能的差	可以支付的原因
level 仍為 k	$\Delta\phi = -(i' - i) \leq -1$	iter 至少變成 $i + 1$
level 升為 $k' > k$	$\Delta\phi = -(k' - k)r + i - i'$	$k' - k \geq 1, i \leq r, i' \geq 1$，故 $\Delta\phi \leq -r + (r - 1) = -1$

在 64 點例子取 $x = 14, y = 0, k = 0, i = 1$，不等式數值為 $6 \geq 6 \geq 6 \geq 3 \geq 3$。舊層下一個門檻是 $A_0^{(2)}(1) = 3$；新父 rank=6 更跨過 $A_1(1) = 3$，所以落在第二種情況，位能下降 1。

這筆帳按「被壓縮的好節點 x」計費，不向見證 y 再收一次，也不要求 y 的位能下降。所有非好節點的位能亦不增加，故可以把各個好節點的下降量相加。

每個 level 只留下最後一點



沿原路徑向根看，同一 level 除了最後一點外，每點都有後方的同層見證。因此 14 可計費，12 與 0 分別是 level 1、0 的最後一點；再加根 32，共 3 個例外。若把三個相鄰區段 0／1／0 當成三組，就會漏算好節點 14。

從分組計數回到位能總下降

一般路徑含 s 個節點。rank 大於 0 的非根節點的 level 只能是 $0, \dots, \alpha - 1$，故各層最後一點至多 α 個；另加根與可能的 rank 0 起點，例外至多 $\alpha + 2$。rank 沿路嚴格增加，rank 0 起點最多一個。

$$G \geq g := \max(0, s - (\alpha + 2)), \qquad \Delta\Phi \leq -G \leq -g.$$

G 是實際好節點數，g 只是保守下界。64 點例子 $s = 4, \alpha = 3$，所以 $g = 0$，實際卻有 $G = 1$、$\Delta\Phi = -1$。短路徑的下界可以不緊；不等式並未要求實際下降量必須等於下界。後面會用這個一般下界支付長路徑的成本。

用固定常數把路徑工作付清

設兩趟查找的實際成本至多 cs，固定 $C > c$ 並用放大位能 $\Psi = C\Phi$。則攤銷成本

$$\hat{c} = c_{\text{actual}} + \Delta\Psi \le cs - Cg.$$

路徑情況	代入 g 後的上界
$s \le \alpha + 2$	$cs \le c(\alpha + 2)$
$s > \alpha + 2$	$(c - C)s + C(\alpha + 2) \le C(\alpha + 2)$

因此每次 FIND 的攤銷成本為 $O(1 + \alpha)$。例如 $s = 7$、$\alpha = 3$、$c = 2$、$C = 3$，下界 $g = 2$，故 $\hat{c} \le 14 - 6 = 8$。C 在整段分析中固定，不能每次臨時換值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LINK 把位能增加限制在新根

MAKE-SET 新增 rank 0 節點，其位能為 0，攤銷成本仍為 $O(1)$。對 LINK，設舊根 u 接到保留根 v：

可能改變的點	位能變化的原因
u	從根變非根；rank 不變，位能不增
v 原有的孩子	父 rank 不降；前述單調性保證位能不增
v	不同 rank 時不變；同 rank 時加一，位能只增 α

其他點的 rank、父 rank 與根身分皆不變，所以位能不變。於是 $\Delta\Psi \le C\alpha$，LINK 的攤銷成本為 $O(1) + C\alpha = O(1 + \alpha)$。UNION 先做兩次 FIND，根不同才 LINK；最多拆成三次操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從每次攤銷界回到整段實際成本

令第 j 次實際成本為 c_j，攤銷成本為 $a_j = c_j + \Psi_j - \Psi_{j-1}$。相鄰位能相消後，

$$\sum_{j=1}^{m'} c_j = \sum_{j=1}^{m'} a_j + \Psi_0 - \Psi_{m'} \le \sum_{j=1}^{m'} a_j.$$

不等號用了初始空森林 $\Psi_0 = 0$ 與終態位能非負。拆開 UNION 後 $m' \le 3m$，所以原本 m 次操作的總成本為 $O(m(1+\alpha(n)))$。

這是整段序列的最壞總界。rank 與壓縮並用時，單次 FIND 仍可能走 $\Theta(\log n)$；先建立平衡的 rank 樹、再首次查找深葉即可達到。線性長鏈需要移除 rank 規則，不能當成本定理的單次反例。

位能分析參考：Cormen 等，第三版 §21.4，印刷頁 576–581。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

23 最小生成樹

Part VI Graph Algorithms

本章定位

最小生成樹以割上的安全邊逐步降低連接成本；本章從交換證明推導Kruskal與Prim，再用同一五點圖追蹤分量、key及parent，建立可獨立檢查的最優與唯一性證書。

前置知識

連通無向圖、cut 與權重。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用 cut 性質推導 MST 安全性。
- 比較 Kruskal 與 Prim 的資料結構與成本。
- 辨識負權、非連通與唯一性條件。
- 23.1 Growing a minimum spanning tree
- 23.2 The algorithms of Kruskal and Prim

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

23.1 Growing a minimum spanning tree

目標是樹的總權，並非每一條路的長度

給有限無向圖G=(V,E)，n=|V|、m=|E|；每條邊有可比較的有限實數權w(e)。先假設n≥1且G連通。Spanning tree（生成樹）T使用全部頂點、連通且無cycle，恰有n−1條邊；MST使$w(T) = \sum_{e \in T} w(e)$最小。有限個候選樹保證最小值存在，權可以為負、為零或重複。

負權不會破壞MST，但不能刪去「無cycle」條件：三角形三邊皆−1，樹成本−2；若容許一般連通子圖，取三邊成本−3。MST也不是最短路徑樹：在ab:4、ac:5、bc:2中，MST={ab,bc}成本6，a到c沿樹長6；最短路徑卻是ac長5。

n=1時回傳空邊集合、成本0；n=0約定空forest。不連通圖沒有生成樹，minimum spanning forest（MSF）則在每個原連通分量取一棵MST。自環不屬於任何樹；平行邊保留各自edge ID，不能僅以端點辨識刪除對象。

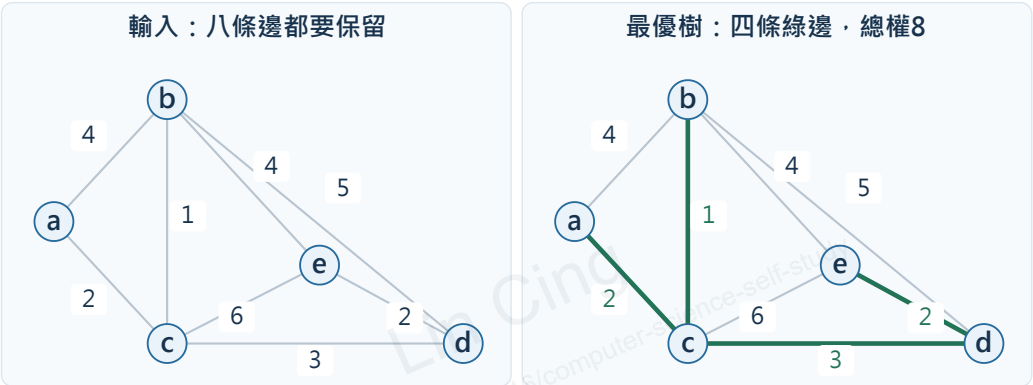
同一五點圖與兩種不同成長順序

本章固定邊表如下，所有後續狀態皆使用同一輸入。Kruskal同權依edge ID先後處理。

ID	0	1	2	3	4	5	6	7
邊	ab	ac	bc	bd	cd	ce	de	be
權	4	2	1	5	3	6	2	4

Kruskal按權選bc、ac、de、cd；Prim從a選ac、cb、cd、de。兩者最後都是{bc,ac,de,cd}，成本1+2+2+3=8，但中途的forest與grown set不同。圖中的灰邊也屬於輸入，不能只看最後綠樹就猜出選取過程。

同一五點圖與兩種不同成長順序 （續）



bc:1 + ac:2 + de:2 + cd:3 = 8

Safe 的意思是仍能延伸成某棵最優樹

維護不變量：目前A⊆E包含在至少一棵MST內。e∉A稱為safe for A，若A∪{e}也包含在某棵MST內。這是最優性的延伸條件，比「加入不成cycle」強；後者只保證還是一片forest。

Cut（割）(S,V−S)要求∅≠S≠V；邊的兩端分居兩側才叫crossing。割respect A表示沒有A中的邊跨割。Light edge是該割跨邊中的最小權者，可以並列。

通用算法由A=∅開始，反覆找safe edge加入，直到|A|=n−1。每次加一條且不變量保證A無cycle；n點n−1邊的forest必連通，所以回傳MST。問題在於如何有效地證明下一條邊safe，不能把「每次選便宜邊」當作證明。

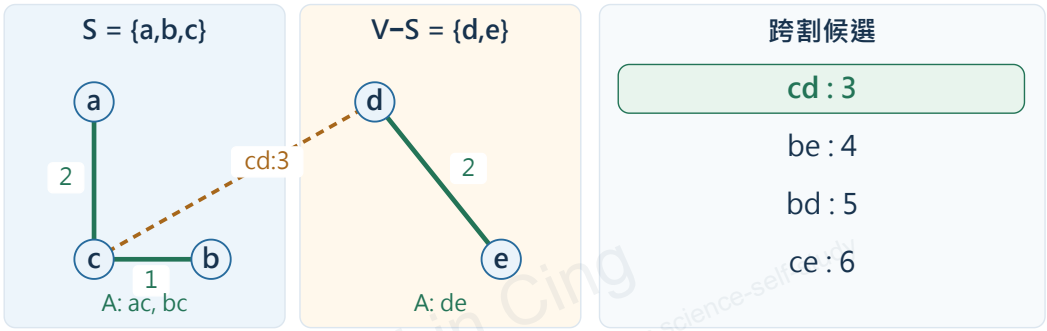
安全邊定理：交換時為何不會丟掉已選的邊

設A可延伸成MST，割(S,V−S)respect A，e是此割的一條light edge。取含A的MST T。若e∈T便已成立；否則T+e恰形成一個cycle。e跨割，cycle要回到起側必另經跨邊f。因割respect A，f∉A。

刪f後cycle被打斷且仍連通，所以T′=T−{f}+{e}仍是生成樹。由w(e)≤w(f)，w(T′)≤w(T)；T已最優，故T′也是MST，且A∪{e}⊆T′。這同時交代結構合法、權不增加、A不受損三件事。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

安全邊定理：交換時為何不會丟掉已選的邊 （續）



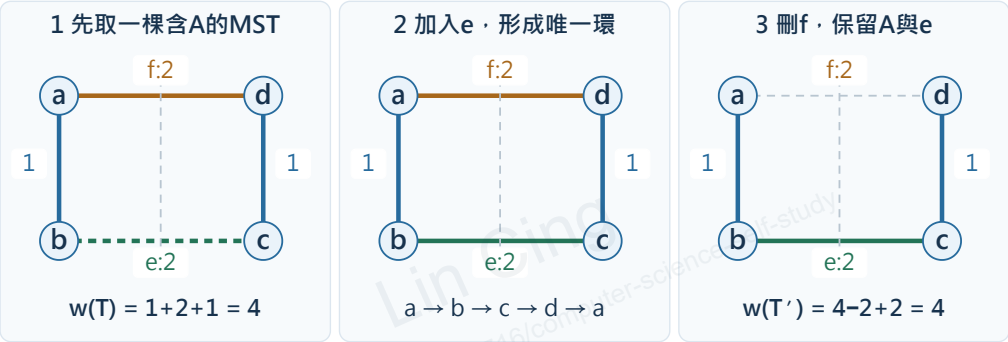
A內三條邊都沒有跨割；cd是最輕跨邊。

respect A：保留已選邊，再加入安全邊 cd

此例S={a,b,c}，A={ac,bc,de}；cd:3、be:4、bd:5、ce:6跨割，故cd safe。de位於另一側內部，並沒有跨割。

交換的三個時刻：一條邊進來，另一條出去

四點圖：ab:1、cd:1、ad:2、bc:2、ac:4、bd:5。A={ab,cd}，割為{a,b}｜{c,d}。



藍邊 A={ab,cd} 始終保留；f不在A中。綠虛線是待加入的e。

同權交換：最優值不變，但找到一棵包含e的MST

T用三條邊達到權重下界1+1+2=4。加入e=bc後，沿a-b-c-d-a的環刪去f=ad，成本4不變且保留A。**只保證存在含A與e的MST；原T就沒有e。**

並列、必選與cycle性質的量詞

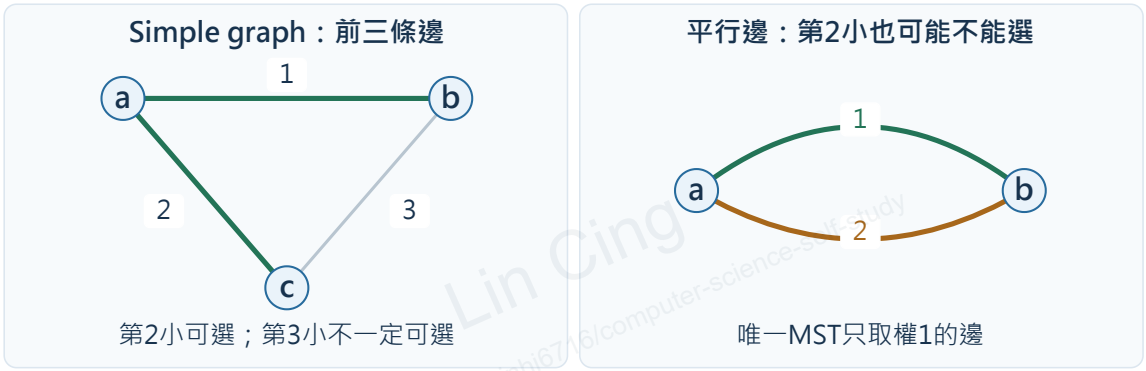
若e是某割的**唯一**light edge，任何不含e的MST都可用上述交換嚴格降低成本，矛盾；所以每棵MST都含e。若只是並列light，只能保證至少有一棵MST含e。三角形各邊權1時有三棵MST，每條邊都是某割的light，卻不能把三條一併選進樹。

Cycle性質：若e是某cycle的唯一最重邊，則e不屬於任何MST。假若MST T含e，刪e分成割；原cycle去e的路必有另一跨邊f，且w(f)<w(e)，交換便嚴格改善。若e只是並列最重，仍可證「存在一棵MST不含e」：從含e的MST做不增成本交換；不能說e一律不可選。

Forest A中任一連通分量C都給出respect A的割(C,V−C)。因此C對外最輕邊safe；這是Kruskal與Prim共同的依據，不是要求每輪全圖最小邊。

全圖第2小邊：結論取決於圖的模型

連通simple graph至少有兩條邊，按非下降權重排序，同權任意排。第2條e必屬某棵MST。



第2小邊的結論要求連通、無自環且無平行邊

若MST T未含e，T+e的環長至少3。除e外至少兩條邊，而全圖嚴格輕於e的邊至多一條，故環上有f不比e輕。刪f後仍最優，得到含e的MST。**平行邊使「環長至少3」失效；第3小邊也沒有相同保證。**

23.2 The algorithms of Kruskal and Prim

Kruskal：讓分量依全域權序合併

```
KRUSKAL(V, E):
    A = 空集合
    對每個 v∈V: MAKE-SET(v)
    將 E 按 (weight, edge-ID) 非下降排序
    對排序後每條 e=(u,v):
        ru = FIND(u); rv = FIND(v)
        若 ru != rv:
            A.add(e); UNION(ru,rv)
    回傳 A
```

DSU代表目前(V,A)的分量，不是原圖的最終連通分量。相同root表示已有一條A路徑，加入e會形成cycle（自環亦在此被拒）。不同root時兩棵樹用一邊連起，仍無cycle。連通輸入接受n−1邊即可提前返回；不必再掃剩餘cycle邊。若要支援MSF，亦可掃到輸入結束。

Kruskal 的一般正確性：被略過的便宜邊去了哪裡

假設接受前A可延伸成MST。對下一條接受的e=(u,v)，令S為u所在的當前DSU分量。S割respect A；若有比e更早的跨邊f，它早已處理過。當時若接受，兩端會合併且此後只合不分，不可能目前仍跨S；當時若拒絕，兩端當時已同分量，現在也仍同分量，也不可能跨S。因此沒有較早跨邊。

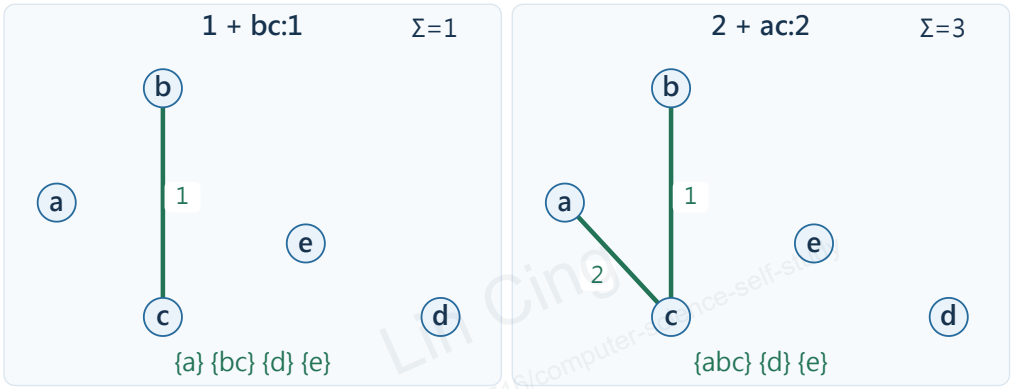
所有尚未掃的邊權不小於e，所以e是這一割的light edge。安全邊定理維持最優延伸不變量。結束時若原圖某分量仍被分成至少兩個A分量，原圖連通路上必有跨A分量邊；掃它時不會被拒，矛盾。故每個原分量都得到生成樹且最優，即MSF。

這個證明不假設權重皆不同，也不假設DSU root剛好是最小頂點；root是實作細節。

Kruskal 真正的每一步

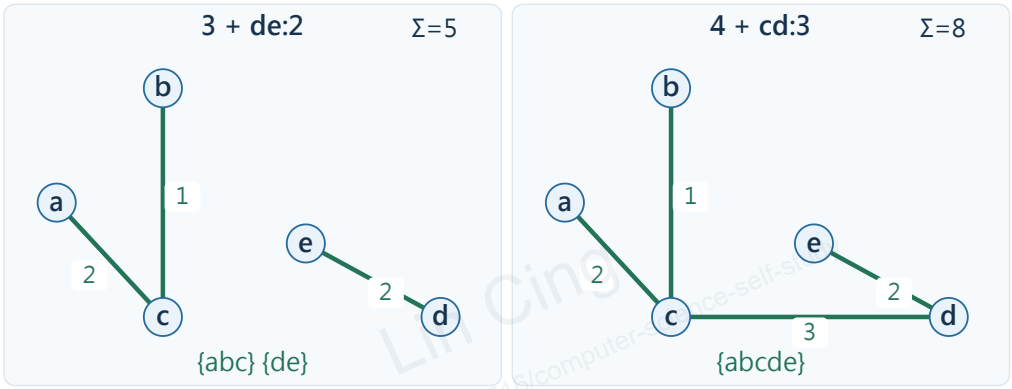
掃描	兩端原分量	決定	處理後分量	累計成本
bc:1	{b},{c}	接受	{bc},{a},{d},{e}	1
ac:2	{a},{bc}	接受	{abc},{d},{e}	3
de:2	{d},{e}	接受	{abc},{de}	5
cd:3	{abc},{de}	接受	{abcde}	8
ab:4	同一分量	拒絕	不變	8
be:4	同一分量	拒絕	不變	8
bd:5	同一分量	拒絕	不變	8
ce:6	同一分量	拒絕	不變	8

Kruskal 真正的每一步 (續)



僅畫已選邊；框下的集合列出所有分量

先長出兩棵樹，再用一條邊合併



第3輪de另成一棵樹；Kruskal先長出forest

第4步已取n−1=4邊，可直接停止；後四列只用來核對完整掃描版本。第3步的de雖不接到{abc}，仍是合法的forest成長，這與Prim不同。

Kruskal 的成本與圖表示前提

比較排序O(m log(m+1))；DSU以union by rank及path compression處理n次MAKE-SET、2m次FIND、至多n−1次UNION，總O((n+m)(1+α(n)))。依第21章固定的版本，α(2)=0；只有n≥3時才能將1+α(n)簡寫成O(α(n))。加上讀入，完整成本為O(m log(m+1)+(n+m)(1+α(n)))；空圖直接返回空森林。排序邊表O(m)空間，DSU O(n)，輸出O(n)。

對連通**simple graph**，n−1≤m<n^2，log m=O(log n)，因此常寫O(m log n)。若允許任意多平行邊，不能再用m<n^2，必保留log m或先用額外演算法去除冗餘邊。負權與同權不影響比較排序。

整數權落在0..W時可用counting sort取代比較排序，成本O(m+W)，但DSU成本仍要加上；W很大時不會更好。這是改變鍵表示的假設，並非推翻比較排序下界。

Prim：key 是跨割的一條邊，不是路徑距離

設已取頂點S。每個v∉S維護$key[v] = \min_{u \in S, (u,v) \in E} w(u,v)$，沒有此邊則∞；parent[v]記實現該值的S端點及edge ID。所有未取頂點各有一個priority-queue項目。第一根r的key特設0，只為先取出，不能把0當輸出樹邊權。

每輪extract最小key。除第一根外，最小項u的parent邊恰是割(S,V−S)的light edge，所以可加入。取u後只需把新出現的候選w(u,v)與key[v]比較，不是key[u]+w(u,v)。

在ab:4、ac:5、bc:2例，取a後key[b]=4、key[c]=5；取b後key[c]=2，但a到c沿樹長4+2=6。Dijkstra則比較d[b]+2=6，保留d[c]=5。Prim允許負權；Dijkstra的正確性另需非負邊。

Prim 的完整 queue 契約

```
PRIM-CONNECTED(V, Adj, r):
    對每個 v: key[v]=∞; parent[v]=NIL; inQ[v]=true
    key[r]=0; Q=BUILD-INDEXED-MIN-HEAP(V,key)
    A=空集合
    當 Q 非空:
        u=EXTRACT-MIN(Q); inQ[u]=false
        若 key[u]==∞: 回報輸入不連通
        若 parent[u]!=NIL: A.add(parent[u]所記的邊)
        對 Adj[u]每條 e=(u,v):
            若 inQ[v] 且 w(e)<key[v]:
                parent[v]=(u,e.ID)
                DECREASE-KEY(Q,v,w(e)); key[v]=w(e)
    回傳 A
```

DECREASE-KEY視為同步更新heap與key儲存。inQ用陣列O(1)查，不可每次掃heap找會員。要輸出MSF時，extract∞代表新原分量：將該u設新根、key[u]=0、不加parent邊，再正常掃鄰居；不能把∞當樹成本。

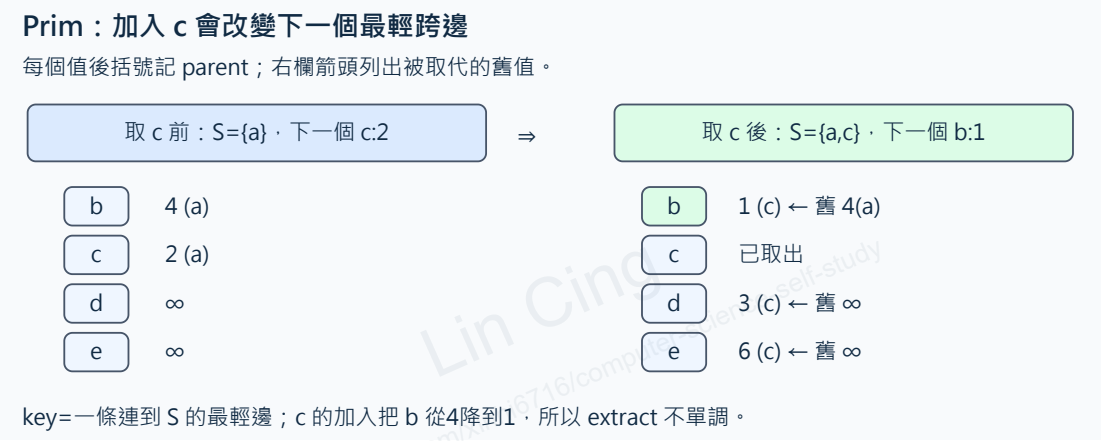
Prim 的不變量、終止與完整數值

每輪開始，S=V−Q，A在S上為一棵樹（第一輪前為空），且包含在某MST中；對Q中各點，key與parent恰表示至S最輕單邊。初始化以根的特例成立。extract後用割定理加入安全邊；掃u的鄰接串列恰補足所有由新點產生的跨邊候選，故不變量恢復。每次Q少一點，n輪終止，輸出n−1邊MST。

本輪extract	掃完後仍在Q的 key(parent)	已接受邊	成本
a:0	b4(a), c2(a), d∞, e∞	無	0
c:2	b1(c), d3(c), e6(c)	ac	2
b:1	d3(c), e4(b)	ac,cb	3
d:3	e2(d)	ac,cb,cd	6
e:2	空	ac,cb,cd,de	8

key extract序列0,2,1,3,2不是遞增。新點能提供更便宜跨邊；最優性靠cut而非key單調。

從新候選到下一條安全邊



圖左S={a}時ac:2最輕；取c後，b的候選由ab:4降成cb:1，d與e才第一次有有限key。下一步必取b:1，不能因「剛取過2」而排除權1。b被取後be:4取代ce:6；再取d才由de:2取代be:4。

每次parent更新要和同輪key一起保存；最後parent不是「第一次碰到的鄰居」。只把頂點順序列出，無法重建這些競爭候選。

Prim 的實作成本不能混用

queue／圖表示	工作拆解	總成本
indexed binary heap＋adjacency list	n次extract、至多2m次掃邊與m次decrease	$O((n+m)\log n)$
Fibonacci heap＋adjacency list	extract共$O(n \log n)$攤銷、decrease共$O(m)$攤銷	$O(m+n \log n)$
線性找最小key＋adjacency list	n輪各掃最多n個key，掃邊共2m	$O(n^2+m)$
adjacency matrix＋線性key	n輪各掃n個key與n個鄰位	$\Theta(n^2)$

對simple graph，$m=O(n^2)$，第三列亦$O(n^2)$。n=1的heap邊界直接返回空樹。heap需indexed handle讓decrease定位O(1)；改成每次push新項、取出時丟舊項的lazy版本，項目數可到O(m)，不可直接套只有n個項目的證明。

$m=\Theta(n)$時binary與Fibonacci均為$O(n \log n)$；$m=\Theta(n^2)$時兩者分別$O(n^2 \log n)$、$O(n^2)$，線性掃key也$O(n^2)$。實際例程用dense Prim驗算選邊，不假裝測量過Fibonacci速度。

不重跑算法也能檢核一棵候選樹

設T是生成樹。以下證書只量化**非自環的非樹邊**e=(u,v)、u≠v，令M_T(u,v)為T中唯一u-v路的最大邊權。自環永不能入樹，直接忽略；不對空的u-u路定最大邊。則

$$T \text{ 是MST} \iff \forall e=(u,v)\in E\setminus T,\ u\neq v:\ w(e)\geq M_T(u,v).$$

必要性：若某w(e)<M，加入e並刪路上最大邊f，仍是樹且嚴格便宜，矛盾。充分性：對任一f∈T，刪f形成割；其他跨邊e都不在T，且T上的e端點路經f，所以w(e)≥M_T≥w(f)。故f為此割light edge。依任意順序加入T的邊時，該割沒有其他T邊跨越，必respect已選prefix；反覆用安全邊定理，整棵T最優。

同一圖非樹邊ab、bd、ce、be的(w,M)依序為(4,2)、(5,3)、(6,3)、(4,3)，全部嚴格大於。因此此例MST唯一。

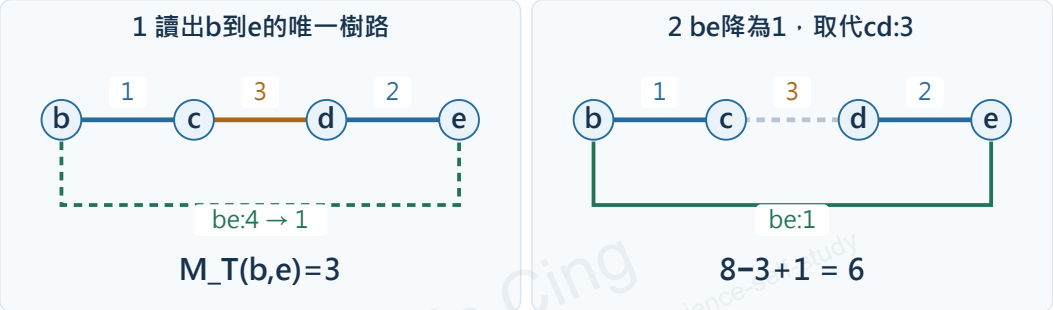
唯一性與邊權變動：證書的兩個用途

T唯一當且僅當每條非自環的非樹邊都有w(e)>M_T。若等號成立，交換一條最大路徑邊得到另一棵同成本樹。反之若全嚴格，前述每個T−f割上，f是唯一light edge，故每棵MST都含全部T邊。這不要求**所有輸入邊權互異**；本章兩條權2仍得到唯一MST。

若只將一條非自環的非樹邊e的權降成w′，其他不變，令M為原T路徑最大值。w′≥M時原T仍通過全部證書；w′<M時刪一條最大邊f並加e。其最優值為w(T)+min(0,w′−M)：不含e的樹權重未變，至少w(T)；任一含e的樹R，刪e分割後，T中u-v路有跨邊g；R−e+g是原權的樹，故w(T)≤w(R)−w′+w(g)≤w(R)−w′+M。這給與新構造相同的下界。

本圖把be由4降到1：T路b-c-d-e權1,3,2，刪cd:3，成本8−3+1=6。若只降到3，舊樹仍最優且出現並列。逐條DFS找路的證書成本O(nm)；這是單邊權變動的更新保證。自環無論多負都不能入樹，改其權不影響MST，亦不執行刪除樹路最大邊。

唯一性與邊權變動：證書的兩個用途 (續)



ac:2保持不變；圖只放大發生交換的b-c-d-e路徑。

新邊比路上最大權更小，才需要替換：$w' < M_T$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 23。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24 單源最短路徑

Part VI Graph Algorithms

本章定位

最短路徑以relax的上界及前驅重建串聯算法；本章逐步證明Bellman–Ford、DAG與Dijkstra各自的適用條件，再用負環證書、差分限制及根樹驗證區分有限距離、不可達與無下界。

前置知識

Edge relaxation、負權與可達性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 維持 shortest-path 與 triangle-inequality 不變量。
- 分別分析 Bellman-Ford、DAG 與 Dijkstra。
- 把 difference constraints 化為負權圖。
- 24.1 The Bellman-Ford algorithm
- 24.2 Single-source shortest paths in directed acyclic graphs
- 24.3 Dijkstra’ s algorithm
- 24.4 Difference constraints and shortest paths
- 24.5 Proofs of shortest-paths properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

24.1 The Bellman-Ford algorithm

距離有三種語義，負邊與負環不同

有限有向圖G=(V,E)，n≥1、s∈V，每條邊為有限實數權。路徑成本是邊權和；容許重複頂點的walk可以包含cycle。定義δ(s,v)為所有s-v walk成本的下確界：不可達為∞；可經過負cycle再到v則為−∞；其餘有有限最小值。源點的零長walk成本0，所以δ(s,s)≤0。

若到v的walk沒有可用負cycle，刪去正cycle會改善，刪去零cycle不變，因此存在至多n−1邊的簡單最短路。負邊本身不使問題無解；負cycle也只影響其**由s可達且能往後到達**的點，其他分支仍可能有有限距離。

單目的地可反轉全部邊後以目的地為source；單點對可取單源輸出的指定項。無向負邊若表示成兩條反向arc，便形成可往返的負closed walk，不能沿用「只是單一負邊」的直覺。

Relax：只改進暫定上界

```
INITIALIZE(s):
    對每個 v: d[v]=∞; parent[v]=NIL
    d[s]=0
RELAX(e=(u,v)):
    若 d[u]!=∞ 且 d[v]>d[u]+w(e):
        d[v]=d[u]+w(e)
        parent[v]=(u,e.ID)
        回傳 changed=true
    否則回傳 false
```

每個有限d[v]是實際s-v walk成本（對relax次數歸納），故d[v]≥δ(s,v)。只做嚴格改善，等值不改parent。平行邊必記ID；初始化Θ(n)，每次relax在精確算術單位成本模型下O(1)。有限整數實作須防溢位，∞要獨立標記；任意把負值截到某下限，會毀掉負環偵測。

一旦d達到有限δ，之後既不能增加，也不能低於δ，所以永遠固定。此處使用「有限」避免把−∞當成某次普通有限relax的輸出。

Bellman–Ford 的操作、早停與成本

```
BELLMAN-FORD(G,s):
    INITIALIZE(s)
    重複最多 n−1 輪:
        changed=false
        對固定順序每條 e∈E:
            若 RELAX(e)成功: changed=true
        若 changed==false: 回傳 d,parent,「無可達負環」
    對每條 (u,v)∈E:
        若 d[u]!=∞ 且 d[v]>d[u]+w(u,v):
            回傳「存在可達負環」
    回傳 d,parent,「無可達負環」
```

最後一輪是測試，不把某個仍在下降的有限d當最短距離交付。若一整輪沒有更新，所有可達邊已滿足d[v]≤d[u]+w；之後同樣輸入再掃也無變，故可早停。n=1仍要測自環，負自環會在最後檢查被發現。

固定全輪版本成本Θ(n+nm)，在m≥1、n≥2時常寫Θ(nm)；早停最好Θ(n+m)，最壞不變。額外d、parent為O(n)，輸入邊表O(n+m)。負環的完整分類與證書可另加下述DP版本。

完整追蹤：逆向邊序需要多輪傳播

頂點依(s,a,b,c,t,z)。邊按下列**固定順序**掃描：ct:3、at:8、ac:1、bt:12、bc:5、ba:−2、sa:4、sb:1；箭頭方向均由第一字母至第二字母。z孤立。

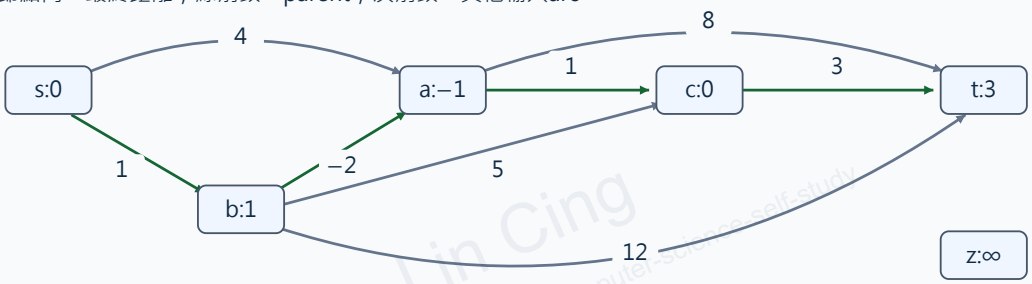
輪末	s	a	b	c	t	z
0	0	∞	∞	∞	∞	∞
1	0	4	1	∞	∞	∞
2	0	−1	1	5	12	∞
3	0	−1	1	0	7	∞
4	0	−1	1	0	3	∞
5	0	−1	1	0	3	∞

第2輪先以舊a=4產生t=12、c=5，稍後ba才把a改成−1。第3輪先ct得8，再at改7，ac才使c=0；第4輪ct才得3。第5輪無改動早停。最終parent為s→b→a→c→t，成本1−2+1+3=3。

完整追蹤：逆向邊序需要多輪傳播 （續）

Bellman–Ford：同一輸入與最終前驅

節點內 = 最終距離；綠箭頭 = parent；灰箭頭 = 其他輸入arc。



逆向掃描輪末：a 4→−1；c ∞→5→0；t ∞→12→7→3。

樹路 s→b→a→c→t：1−2+1+3=3；z 從 s 不可達。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何 n−1 輪足夠，為何負環一定被看見

若無可達負環，任一可達v有簡單最短路$s=v_0,\cdots,v_k=v$，$k\le n-1$。第i輪必掃(v_{i-1},v_i)，且第i−1輪末$d[v_{i-1}]$已達δ；convergence給$d[v_i]=\delta$。即使輪內又處理更多邊，也不會破壞真值，所以到n−1輪全部正確。這只說原地版本能至少涵蓋i邊路徑，不說其值恰等於「最多i邊」的DP。

若有可達負cycle，各cycle頂點在n−1輪後都有限（任一可達點有≤n−1邊的可達簡單路，不要求該路最短）。若最後所有cycle邊都不能改善，沿圈相加$d[v]\le d[u]+w$，d項消去得到$0\le w(\text{cycle})<0$，矛盾。因此至少一條可達邊仍可改善。反之無負環時所有d已等δ，三角不等式排除任何改善。這是偵測的兩方向，不只以「值繼續降」作描述。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

要交付負環證書：用明確的邊數 DP 重建

令$D_i[v]$為最多i邊的最小walk成本，$D_0[s]=0$、其餘∞。

$$D_i[v] = \min\left(D_{i-1}[v],\ \min_{(u,v)\in E}(D_{i-1}[u]+w(u,v))\right).$$

每一層只讀上一層；若保留舊值就記「skip」，若選一條邊就記其ID及上一層u。每個≤i邊walk不是≤i−1邊，就是刪去末邊後的一個≤i−1邊walk；反過來每個候選均合法，故兩向證明recurrence。

計算至第n層。若$D_n[v]<D_{n-1}[v]$，由記錄倒退n層得到**恰n邊**walk：少一邊便屬前一層，不會嚴格改善。分解該walk為一條簡單路及重複點截出的cycles。若所有cycles非負，刪掉它們得到≤n−1邊且成本不大的路，與嚴格改善矛盾。故至少一個截出的cycle權負，輸出其edge ID順序與總權就是可核對證書。

此完整記錄版時間O(n+nm)，$O(n^2+m)$空間；只存兩層可分類但不能保留上述回溯表。不要替此版本聲稱原地BF的O(n)輔助空間。

哪些點應標成 −∞

取seed集合$B=\{v:D_n[v]<D_{n-1}[v]\}$，沿**原圖方向**從B做BFS／DFS，所有到達點標−∞；其餘可達點取D_{n-1}，不可達保∞。

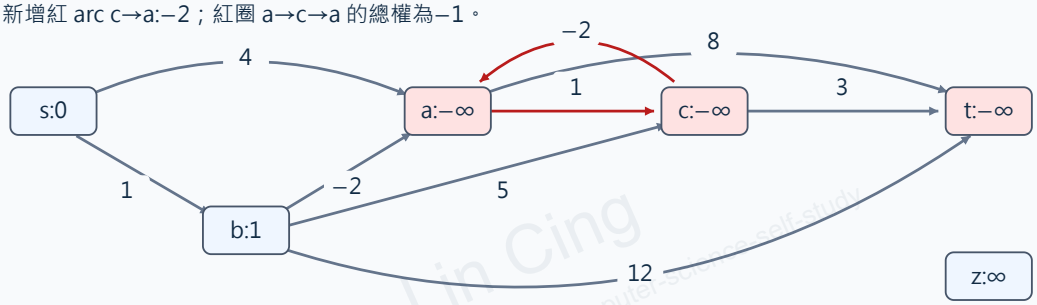
每個seed的n邊walk含負cycle，故seed及其後代都受影響。另一方面，任一可達負cycle的所有點在D_{n-1}已有限。若圈上沒有seed，則每邊$D_{n-1}[v]=D_n[v]\le D_{n-1}[u]+w$，相加矛盾；故圈上至少一seed，所有受其影響的後代都會被遍歷。這證明標記範圍恰好，並非只標cycle本身。

沿前例加c→a:−2，a→c→a權1−2=−1，則(s,a,b,c,t,z)距離為(0,−∞,1,−∞,−∞,∞)。s、b到不了由cycle往回的出口，仍有限；z仍不可達。附帶一個獨立不可達負圈也不能讓s的BF誤報。

哪些點應標成 −∞ （續）

可達負環只影響其向前後代

新增紅 arc c→a:−2；紅圈 a→c→a 的總權為−1。



圈的後代 a,c,t 標−∞；b=1、s=0，因圈沒有回 s 或 b 的出口。

同一次負環偵測，不可把仍有限的分支或不可達 z 一起抹成−∞。

24.2 Single-source shortest paths in directed acyclic graphs

拓撲序讓每個候選都準時到達

```
DAG-SHORTEST(G,s):
    order=TOPOLOGICAL-SORT(G)
    若發現cycle: 拒絕此算法的輸入
    INITIALIZE(s)
    對 order 中每個 u:
        對 Adj[u] 每條 e: RELAX(e)
    回傳 d,parent
```

拓撲序保證任何s-v路上各邊依路徑先後被處理；path-relaxation使路徑終點取得δ。亦可對拓撲位置歸納：v被處理前，所有入邊起點均已固定且掃過，所以沒有未來才到的更好候選。不可達點保持∞。負邊可以有，因無cycle便無負cycle。拓撲排序及掃邊共Θ(n+m)，輔助O(n)。

前例本來就是DAG，取合法次序s,b,a,c,t,z：處理s後a4,b1；處理b後a−1,c6,t13；處理a後c0,t7；處理c後t3。一輪與BF四輪同結果，差在**頂點拓撲順序**而非改了成本定義。

關鍵路徑：最長路只在 DAG 中直接翻號

在DAG把每個權改−w，最短成本的負值便是原最長成本；或以L[s]=0、其他−∞並用max relax。一般有cycle圖不能這樣直接套DAG算法；正cycle會使最長walk無界。

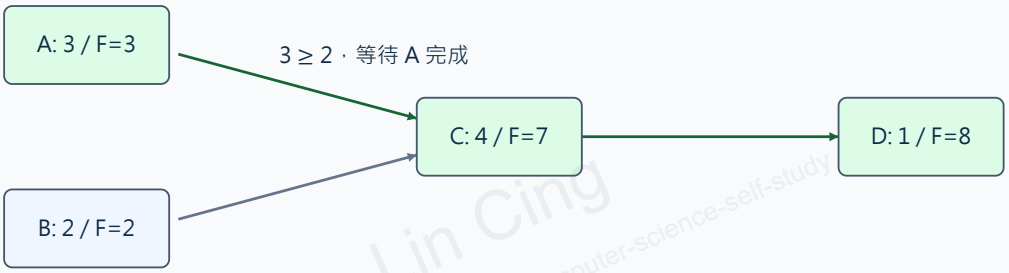
若工作是頂點，duration[v]≥0，依賴u→v表示u完成後v才能開始。最早完成F[v]=duration[v]+max({F[u]:u→v}∪{0})，依拓撲順序算；回溯實現最大值的前驅給critical path。

例如工作A:3、B:2、C:4、D:1，依賴A→C、B→C、C→D。F[A]=3、F[B]=2、F[C]=4+max(3,2)=7、F[D]=1+7=8；critical path A,C,D成本8。無限處理器且只受這些先後條件時，讓每工作一滿足前驅就開始，可達此下界8；有限處理器另有資源限制，不能只以此數當完整排程。

關鍵路徑：最長路只在 DAG 中直接翻號 （續）

工作 DAG：完成時間由較慢前驅決定

格內：工作 / duration / 最早完成 F；綠箭頭在 critical path。



F[C]=4+max(3,2)=7；F[D]=1+7=8。

無限處理器最早完成8；總工量10，一顆處理器仍至少需要10。

24.3 Dijkstra’s algorithm

取最小暫定距離後不再重新打開

此節明列所有邊w≥0。以indexed min-heap保存每個未取頂點各一項，key=d。初始化d[s]=0、其他∞、parent=NIL、inQ=true。

```
當 Q 非空:
    u=EXTRACT-MIN(Q); inQ[u]=false
    若 d[u]==∞: 停止（餘下皆不可達）
    對 Adj[u] 的 e=(u,v):
        若 inQ[v] 且 d[u]+w(e)<d[v]:
            d[v]=d[u]+w(e); parent[v]=(u,e.ID)
            DECREASE-KEY(Q,v,d[v])
```

已取S各點的d已是δ；與Prim相比，更新是**總路長d[u]+w**。已取點不更新也不重入Q。若要提早結束單點對查詢，須等目標真的extract，不能在它第一次被發現就停。

非負權究竟用在哪一行

對下一個有限extract u，若它為第一個錯誤，取一條真最短路s→⋯→x→y→⋯→u，y是路上第一個未取點，x已取。s第一個被取且d[s]=0；其餘情況x確已達δ並掃xy，故convergence保證d[y]=δ(s,y)。

從y到u的後綴權非負，得到δ(s,y)≤δ(s,u)。選最小d給d[u]≤d[y]；上界性給δ(s,u)≤d[u]。合併

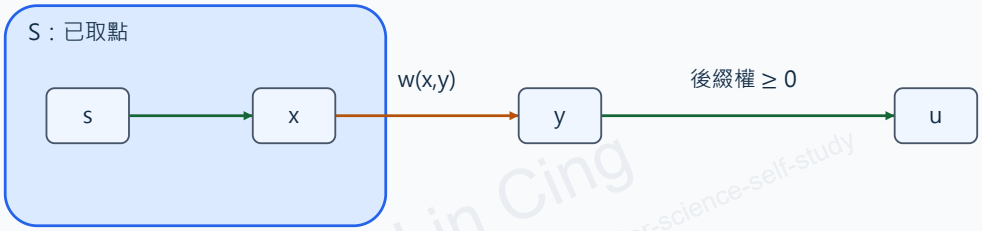
$$\delta(s,u) \le d[u] \le d[y] = \delta(s,y) \le \delta(s,u),$$

全部相等，矛盾，所以extract不會錯。零權邊與相同距離均可，無須嚴格遞增。∞被取時，若仍有可達未取點，該路第一未取點已有有限候選，與取到∞矛盾。

非負權究竟用在哪一行 （續）

Dijkstra：用第一個未取點夾住 u

只追蹤一條真最短路；橘箭頭是它第一次跨出 S 的 arc。



x 已固定且掃過 xy ⇒ d[y]=δ(s,y)。

最小提取 d[u]≤d[y]，非負後綴 δ(s,y)≤δ(s,u)，上下界夾成相等。

數值追蹤與負邊的真正反例

將BF前例只有ba改為+2。逐輪掃完：

extract	未取的有限 d(parent)	已固定距離
s0	a4(s),b1(s)	s0
b1	a3(b),c6(b),t13(b)	s0,b1
a3	c4(a),t11(a)	再加a3
c4	t7(c)	再加c4
t7	無；z∞	再加t7

最短樹s→b→a→c→t，路長1+2+1+3=7。候選t曾13→11→7，第一次發現t不能當最短值。

負邊反例用s→a:2、s→b:5、b→a:−4、a→t:2。取s後先固定a2並令t4，再固定t4，最後b5才提供a1與真正t3。標準不重開版本維持錯值；即使容許更新已取a而不重掃其出邊，t仍錯。此圖為DAG、沒有負環，失敗恰在δ(s,b)=5>δ(s,a)=1，使證明的非負後綴比較失效。

Queue、輸入表示與精確算術成本

indexed binary heap：建heap O(n)、n次extract共O(n log n)、每邊最多一次decrease共O(m log n)，總O((n+m)log n)，n=1特判。Fibonacci heap將decrease降至O(1)攤銷，總O(m+n log n)。每個heap項須能以handle定位，inQ用O(1)陣列查。

線性掃全部未取d配adjacency list為$O(n^2+m)$，simple graph可簡化$O(n^2)$。若以lazy heap推入所有新候選，項目可達m，需另證O((n+m)log(m+1))及O(n+m)空間，不能套只有n項的indexed版本成本。例程使用線性掃key及adjacency list做數值驗算。

上述是實數RAM模型；B位元整數、無負環的簡單路成本需O(B+log n)位元，精確加法／比較成本應另外計入。浮點誤差與任意epsilon可能改變等值和負環判斷；數學正確性並非自動等同浮點穩定性。

24.4 Difference constraints and shortest paths

限制的方向來自移項，不憑語感猜

限制$x_j-x_i\le b$對應arc $i\to j:b$，因為它等同$x_j\le x_i+b$。每列係數矩陣只有一個+1與一個−1，變數可為任意實數。若寫$x_j-x_i\ge b$，先轉成$x_i-x_j\le -b$。等式拆成兩個反向不等式。

加super-source q→每個變數點，權0；q無入邊，確保所有原負cycle都可達。無負cycle時設$x_i=\delta(q,i)$，由每邊三角不等式得$x_j-x_i\le b$，所有限制成立。若存在負cycle，把對應限制相加，左邊每個變數正負各一次全消，得到$0\le \Sigma b<0$，矛盾。這建立可行⇔無負cycle。

原n變數m限制，加q後n+1點、m+n邊，BF為$O(n^2+nm)$。改為原圖所有d初設0，相當已完成q的零邊，跑至多n−1輪原邊及一輪檢查，成本O(n+nm)，包括m=0時仍需初始化，不寫成0。

完整可行與不可行數例

限制為$x_2-x_1\le -2$、$x_3-x_2\le -1$、$x_1-x_3\le 4$，邊為1→2:−2、2→3:−1、3→1:4。全零開始依此序：第一次掃得到(0,−2,−3)，因−3+4=1≥0不改x_1；第二輪不變。

逐條驗證：−2−0=−2≤−2，−3−(−2)=−1≤−1，0−(−3)=3≤4。全體加3得(3,1,0)仍可行，因差值不變。一般可行域沿全一向量平移，沒有固定原點。

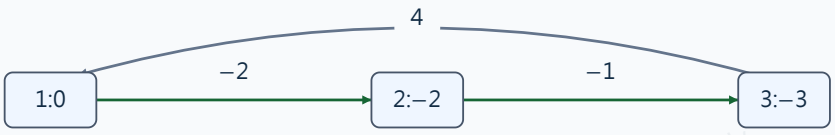
若把最後限制改$x_1-x_3\le 2$，三式相加為$0\le -2-1+2=-1$，無解。負cycle為1→2→3→1；不能只報某個候選不符合，就推所有候選都不存在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整可行與不可行數例 (續)

同一組限制：可行向量與負環矛盾

每條 arc i→j:b 對應 xj−xi≤b；圖內值為可行 x。



逐條差值：−2−0=−2；−3−(−2)=−1；0−(−3)=3≤4。

若最後權4改成2：三式相加得 0≤−2−1+2=−1，矛盾。

原圈總權1可行；改後總權−1無解。整體平移不改任何差值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

哪些延伸仍保留這個證明

等式$x_j - x_i = b$拆成$i \to j : b$與$j \to i : -b$，兩向合起恰強制相等。欲表示$x_i \le c$及$-x_i \le c$，增加一個reference點r，用$r \to i : c$及$i \to r : c$；求解後整體平移使$x_r = 0$，才能把它當絕對界，不能獨立令全部d任意平移卻忘記此基準。

整數右端時BF從整數0用加法/min計算，所得可行解自動整數。若要求**所有**變數整數且右端為實數，先將每個b換成floor(b)：因$x_j - x_i$為整數，兩個限制等價；再檢查負cycle。若只有部分變數要求整數，這個逐列floor方法不再自動適用。

所有$d \le 0$的規範下，最短距離還是逐座標最大的可行向量：任意可行x沿$q \to \cdots \to v$路反覆用限制，$x_v \le$路徑成本，故$x_v \le \delta(q,v)$。這是加入$x_i \le 0$後的結論，原平移不受限時Σx_i可任意增加。

24.5 Proofs of shortest-paths properties

最優子結構、三角不等式與收斂的完整鏈

假設本段沒有從s可達的負cycle。若一條有限最短路的子段不是最短，用更便宜子段替換便產生更便宜s-v walk，矛盾。對任何arc $u \to v$，接一條s-u最短路再接該arc是候選，故$\delta(s,v) \le \delta(s,u) + w$；u不可達時右邊$\infty$不增加限制。

上界性由初始化及每次有限d為walk成本證明；不可達點若曾變有限，就有實際walk，矛盾，故永保∞。Relax當下結束必有$d[v] \le d[u] + w$；**只保證此當下**，u日後再降，該邊可能要重掃。

若u已真最短且$u \to v$接在某條最短路上，該次relax後$d[v] \le \delta(s,u) + w = \delta(s,v)$，又上界性給反向，故v固定於真值（convergence）。依最短路的arc順序套此論證，第一點s已真值，歸納每次把下一點固定；插入其他relax亦只降上界且不破真值，故path-relaxation成立。

Parent 沒有cycle：靠更新時間，不靠距離嚴格遞減

假設某次嚴格relax u→v第一次把parent子圖變成cycle。此前已有parent路v→⋯→u。每條舊parent arc x→y，在最後設定parent[y]時有d[y]=d[x]+w；其後y未更新，x只可能下降，所以本次更新前 d[y]≥d[x]+w。沿舊路相加得d[u]≥d[v]+w(v→⋯→u)。

新更新又要求d[v]>d[u]+w(u,v)，合併得cycle總權<0。路上各點皆有限，故由s可達，與假設矛盾。負自環亦直接由嚴格改善推出權負。所以parent子圖始終無cycle。

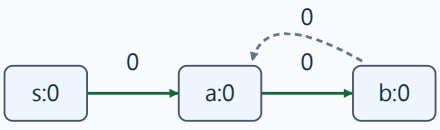
任何有限v≠s必由relax得到非NIL parent；s因d[s]=δ(s,s)=0固定，parent[s]=NIL。沿parent回溯，有限點的parent也有限；有限圖且無cycle，必停在唯一沒有parent的s。每個點只有一個parent，所以得到以s為根、包含所有已發現有限點的樹。

Parent 沒有cycle：靠更新時間，不靠距離嚴格遞減 （續）

零權邊：距離相等仍須檢查 parent 根結構

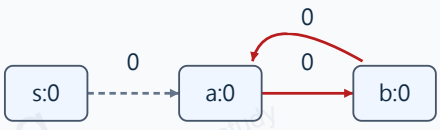
原圖 s→a:0、a→b:0、b→a:0；兩側全部距離都是0。

合法 parent：由 s 有根路



parent[a]=s，parent[b]=a

錯誤 parent：a,b 互為父



parent[a]=b，parent[b]=a

綠 = 合法樹 parent；紅 = 錯誤cycle；灰虛線 = 不在所選parent中的原arc。

Tight等式只驗成本，仍要從 s 遍歷 parent children，確認根連通且無環。

正確距離加合法 parent 才能交付最短路樹

當全部有限d已等δ，對當前parent arc u→v，最後更新時間給d[v]≥d[u]+w；三角不等式給δ(s,v)≤δ(s,u)+w，代入d後兩向合成等號。沿s-v樹路相加，內部d項望遠鏡消去，路權=d[v]−d[s]=δ(s,v)。不可達點仍∞且parent=NIL，故三項最短路樹條件全部成立。

不能改說沿parent距離嚴格下降：零權arc可以相等，負權arc沿正向還會下降；無cycle依賴剛才的嚴格更新時間證明。若不是由這種relax歷史產生，而是事後對每點任選tight predecessor，零權圈可被選成parent cycle。例如s→a:0、a→b:0、b→a:0，d均0；設parent[a]=b、parent[b]=a每邊都tight，卻沒有根路。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性時間驗證輸出，連最優性也有證書

給候選d與parent edge IDs，先在原圖從s算可達集合R；要求R內d有限、d[s]=0，R外d=∞且parent=NIL。驗每個v∈R−{s}的parent確為輸入arc u→v，且d[v]=d[u]+w；用parent children做一次DFS，確認從s恰好遍歷R而無重複，parent[s]=NIL。再掃所有u∈R的輸入arc，驗d[v]≤d[u]+w。

樹路的tight性給一條成本d[v]的真路，故δ(s,v)≤d[v]。另一方面沿任意s-v walk把所有邊不等式相加得d[v]≤該walk成本，故d[v]≤δ(s,v)。兩向相等；可達負cycle也會違反相加不等式，所以證書拒絕它。驗證時間Θ(n+m)、空間O(n)，不需重跑BF或Dijkstra。

這同時解釋為何只驗邊不等式不夠（數值可能太小、沒有實現路徑），只驗tight parent不夠（可能零圈或另有更便宜輸入arc）；兩種證據各負責一個方向。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 24。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

25 全點對最短路徑

Part VI Graph Algorithms

本章定位

全點對最短路徑有兩種不同DP狀態：邊數與中介點；本章推導min-plus、Floyd與Johnson，連結可重建路徑、零權圈、負環影響範圍及重加權的正負號與成本。

前置知識

矩陣最小值加法與前驅表示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 把 min-plus 矩陣乘法接到最短路徑。
- 用 Floyd-Warshall 的中介點 DP 推導轉移。
- 用 Johnson 重加權處理負邊與稀疏圖。
- 25.1 Shortest paths and matrix multiplication
- 25.2 The Floyd-Warshall algorithm
- 25.3 Johnson’ s algorithm for sparse graphs

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

25.1 Shortest paths and matrix multiplication

APSP（全點對最短路徑）的輸入是 n 個頂點的有向加權圖，邊權可為負。輸出每一對 (i,j) 的距離；沒有路徑為 $+\infty$。若存在 i 可達、且能走到 j 的負 cycle，任意重走該圈使成本無下界，這一對的距離定為 $-\infty$，並沒有可交付的有限最短路徑。

本節先假設無負 cycle。任何重複頂點的 walk 都包含非負 cycle，刪圈不增加成本；可反覆刪到 simple path。因此 $i \neq j$ 的最短路徑可取至多 $n-1$ 邊，$i=j$ 則可取零長路徑、成本0。

權矩陣 W[i,j] 是直接邊最小權，無邊為∞；W[i,i]=0 表示可不走邊。若要偵測負 self-loop，初始化必取 min(0,w(i,i))，不能把負 self-loop 覆蓋成0。下面的無負環數例對角線均0。

以最多邊數為狀態

令 $L^{(m)}[i,j]$ 為 i 到 j **最多 m 條邊的 walk** 最小權重；不是恰好 m 條邊。$L^{(0)}=I$，其中 I[i,i]=0、非對角為∞。按最後一條邊拆解，有

$L^{(m+1)}[i,j]=\min_k(L^{(m)}[i,k]+W[k,j])$。

任何非空候選可拆成最多 m 邊前綴加最後一邊；反過來每個有限項都可實際串接。較短的舊路徑也保留，因 k=j 時 W[j,j]=0。因此兩方向涵蓋同一組可行 walk，這是轉移等式的理由。即使有負 cycle，固定 m 的定義仍有限且有意義；只是不能在 $m=n-1$ 宣稱已得無限制的最短距離。

定義 min-plus product： $(A\otimes B)[i,j]=\min_k(A[i,k]+B[k,j])$。若 A、B 分別對應最多 a、b 邊，串接最多 a+b 邊；反向沿任一最多 a+b 邊 walk，在第 min(a,實際長度) 邊切開即可，故 $L^{(a)}\otimes L^{(b)}=L^{(a+b)}$。

Min-plus 的代數與兩種算法

結合律來自同一個有限最小值：((A⊗B)⊗C)[i,j]=$\min_{p,\varphi}$(A[i,p]+B[p,q]+C[q,j])=(A⊗(B⊗C))[i,j]。I 是單位矩陣；∞是 min 的中性元，∞+有限值=∞。本階段不把−∞放入乘法，以免遇到∞+(−∞)沒有定義。

```
Extend(A,B):
  C=全∞的新 n×n 矩陣
  for i,j,k: C[i,j]=min(C[i,j],A[i,k]+B[k,j])
  return C
Slow(W): L=I; 重複n−1次 L=Extend(L,W); return L
Square(W): L=W; m=1
  while m<n−1: L=Extend(L,L); m=2m
  return L
```

Slow 做 Θ(n) 次 Θ(n^3) product，n≥2 時 Θ(n^4)；Square 做 ceil($\log_2$(n−1)) 次 squaring，O(n^3 log n)。保留兩個輸入與一個新輸出即可 O(n^2) 工作空間；不能一邊覆寫輸入而仍宣稱每次 product 精確對應同一舊邊數。n=0回空矩陣，n=1直接處理對角線。

普通乘法把 min 換成加法、把加法換成乘法，但 Strassen 的減法公式不能直接套入 min-plus；上述類比並未證明一般環上的快速乘法界也成立。

四點六邊的共同數例

以下全部矩陣、Floyd stages 與 Johnson 都使用：1→2:4、1→3:7、2→3:−2、2→4:6、3→4:3、4→2:1。頂點1無入邊，其餘三點 cycle 2→3→4→2 成本2，沒有負 cycle。

W 的 rows 為 [0,4,7,∞]、[∞,0,−2,6]、[∞,∞,0,3]、[∞,1,∞,0]。

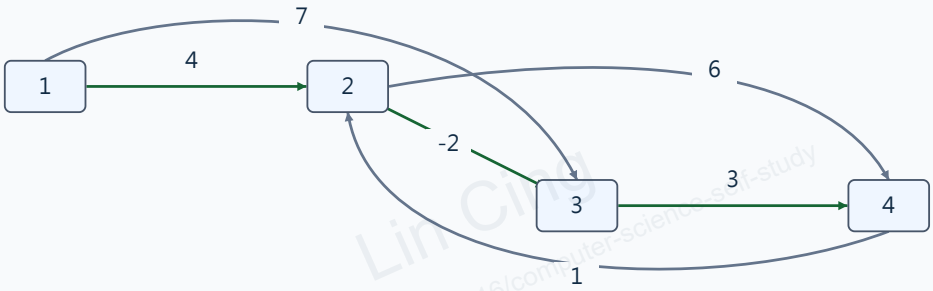
$L^{(2)}$ 的 rows 為 [0,4,2,10]、[∞,0,−2,1]、[∞,4,0,3]、[∞,1,−1,0]。

例如 $L^{(2)}$[1,4]=min(0+∞,4+6,7+3,∞+0)=10，兩條2邊路徑並列；$L^{(2)}$[2,4]=−2+3=1。第三次 Extend 才允許1→2→3→4，$L^{(3)}$[1,4]=5。其餘格不變；n=4，所以 $L^{(3)}$即答案，squaring 的 $L^{(4)}$也相同。

四點六邊的共同數例 （續）

六條有向邊與邊數受限的路徑

綠線 = 最佳3邊路徑；灰線 = 其餘原圖邊。邊旁數字是原權重。



1→2→4=10（2邊）；1→3→4=10（2邊）；1→2→3→4=5（3邊）。

25.2 The Floyd-Warshall algorithm

Floyd–Warshall 的 k 是**允許的中介頂點編號上界**，不是允許的邊數。$D^{(k)}[i,j]$ 是所有中介點在 $\{1,\cdots,k\}$ 的 i→j walk 最小成本；端點 i,j 不受這個集合限制。無負 cycle 時可取 simple path，因此 k 最多出現一次。

最優路徑若不經 k，屬於舊狀態 $D^{(k-1)}$；若經 k，在 k 切開，兩段內部不含 k，因此候選為 $D^{(k-1)}[i,k]+D^{(k-1)}[k,j]$。最優子結構可用替換論證：若任一子段有更便宜的合法替代，串接後成本更小，刪除非負 cycle 也不變差，與最優矛盾。反向串接兩個合法舊子路徑只增加可用中介點 k，故不會取得比狀態定義還小的不合法成本。

所以 $D^{(0)}=W$，且 $D^{(k)}[i,j]=\min(D^{(k-1)}[i,j],D^{(k-1)}[i,k]+D^{(k-1)}[k,j])$。到 k=n 全部中介點皆允許，得到 APSP。逐 k 建新矩陣需 $\Theta(n^3)$ 時間；保留全部 stages 需 $\Theta(n^3)$ 空間，僅留舊／新兩份則 $\Theta(n^2)$。

原地更新為何成立；Loop Order 為何不能任換

無負 cycle 時，每個 stage 的 D[k,k]=0。固定 k 更新 row k 時，候選 D[k,k]+D[k,j]=D[k,j]，不會改值；column k 同理。因此本輪所有 D[i,k]、D[k,j] 都與上一輪相同，各 D[i,j] 一次更新恰實作原轉移，可以只用一個矩陣。

```
D=copy(W)
for k=1..n:
  for i=1..n:
    for j=1..n:
      if D[i,k]!=∞ and D[k,j]!=∞:
        D[i,j]=min(D[i,j],D[i,k]+D[k,j])
```

$\Theta(n^3)$ 時間、$\Theta(n^2)$ 空間。有限整數實作須避免INF sentinel加負數變成「可達」，以及加法溢位；上面的可達判斷先做，精確算術的數學定理才適用。

反例：單位權鏈1→2→4→3，若依 i,j,k 順序只跑一遍，處理 i=1,j=3 時 D[1,4] 與 D[2,3] 都尚為∞，所以 D[1,3] 留∞；稍後 j=4 才算出 D[1,4]=2，卻不會回頭更新 j=3。正確 k,i,j 得 D[1,3]=3。

每個 k 的完整矩陣

rows/columns 都依1,2,3,4。$D^{(0)}=W$；k=1 因1無入邊，不新增路徑，$D^{(1)}=W$。

stage	row1	row2	row3	row4
$D^{(2)}$	0,4,2,10	∞,0,−2,6	∞,∞,0,3	∞,1,−1,0
$D^{(3)}$	0,4,2,5	∞,0,−2,1	∞,∞,0,3	∞,1,−1,0
$D^{(4)}$	0,4,2,5	∞,0,−2,1	∞,4,0,3	∞,1,−1,0

k=2 更新三格：(1,3)取4−2=2、(1,4)取4+6=10、(4,3)取1−2=−1。k=3 更新(1,4)取2+3=5、(2,4)取−2+3=1。k=4 更新(3,2)取3+1=4。其餘格的舊值不大於候選，所以保留。

對照 $L^{(2)}$：它已允許2→3→4與3→4→2，但 $D^{(2)}$尚不允許中介3或4，所以兩格分別仍是6與∞。兩者都叫「第二步」，卻不是同一狀態。

每個 k 的完整矩陣 （續）

Floyd 的六次更新：兩個輸入到一個結果

每列只記嚴格改善；右兩個輸入都來自上一個 stage。其餘格不變。

k	(i,j)	舊值	D[i,k]	D[k,j]	新值
2	(1,3)	7	4	-2	2
2	(1,4)	∞	4	6	10
2	(4,3)	∞	1	-2	-1
3	(1,4)	10	2	3	5
3	(2,4)	6	-2	3	1
4	(3,2)	∞	3	1	4

綠格 = min後的新值；k=1沒有更新，k=2三格、k=3兩格、k=4一格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

路徑輸出：前驅不是中介點

Π[i,j] 表示某條 i→j 最短路徑上，**緊接 j 之前**的點；不是任意中介 k。初始化直接邊 i→j 的 Π[i,j]=i；i=j或不可達為 NIL。採兩個 stage buffer 時，嚴格改善經 k 的值，就取 Π新[i,j]=Π舊[k,j]，因為最後一邊來自 k→j 的後半段；否則沿用舊值。k=j時候選不會嚴格改善，所以不會用空路徑的 NIL覆蓋合法前驅。

等成本時保留舊路徑，不可隨意把 < 改成 ≤。最小例：直接邊1→2權1，在k=2時候選1+0=1；若等值也更新Π[1,2]←Π[2,2]=NIL，就把原本可重建的路徑弄斷。對零權 cycle，更不能只從最後的「距離緊邊」任意選父而不證明無環。

本例Π[1,4]的嚴格改善：k=2時取Π[2,4]=2，暫得1→2→4；k=3時取Π[3,4]=3，且Π[1,3]=2、Π[1,2]=1，最後反向鏈4←3←2←1，成本4−2+3=5。下面給一個額外的 tie rule，使完整 predecessor tree 的無環性可直接由整數下降證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

零權 Cycle 下可證無環的前驅建構

把每格改為 pair (cost,hops)，先比 cost，成本相同再選 hops 小者；兩段相加時兩欄分別相加。初始對角(0,0)、直接邊(w,1)、無路(∞,∞)，同樣做 Floyd轉移。無負 cycle 時，正權圈讓cost變差，零權圈讓hops變差，所以 lexicographic（字典序）最優者必有一條simple path，前述分解與原地證明均成立，仍為$\Theta(n^3)$次固定欄運算。

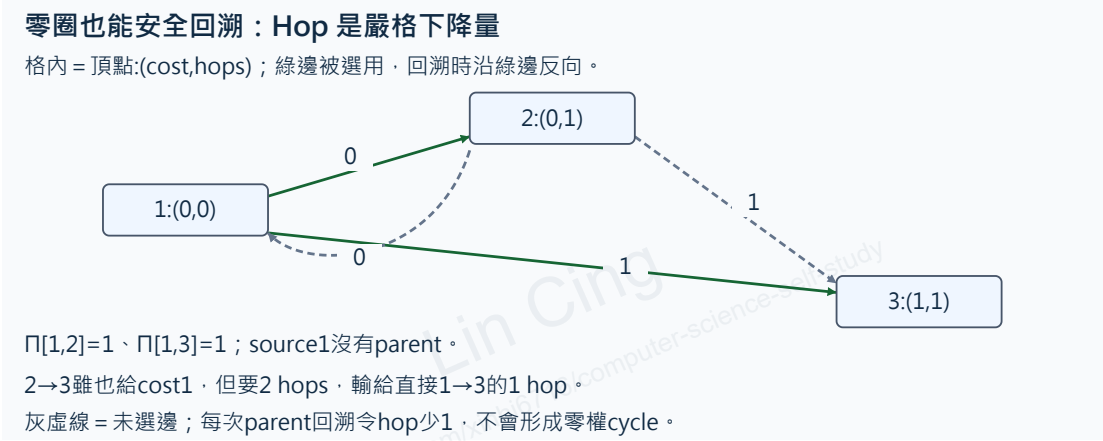
固定source s，對每個有限且v≠s的v，從原圖入邊選一條u→v，使

cost[s,v]=cost[s,u]+w(u,v)，hops[s,v]=hops[s,u]+1。

這樣的u必存在：取一條字典序最優s→v路徑的最後一邊，若其前綴不是字典序最優，可替換而改善整條路徑，矛盾。設定Π[s,v]=u；沿parent每步hops嚴格減1，非負整數不可能無限下降，也不可能有cycle。除了s外有限點都有parent，因此必到s，得到一棵確實重建最短路徑的tree。

對每個s掃全部m邊即可選父，總$O(nm)$，簡單圖$m \le n^2$時不超過Floyd的$\Theta(n^3)$；保存cost/hops/Π為$\Theta(n^2)$。例如1↔2皆權0、1→3與2→3皆權1，source1時hops=(0,1,1)，不能把1的parent設為2，亦不會在1、2之間循環。

零權 Cycle 下可證無環的前驅建構 （續）



負 Cycle 偵測與受影響點對

若跑完Floyd有D[k,k]<0，該值由實際邊串接而成，代表負的closed walk；分解成simple cycles，至少一圈權重負。反向若有負simple cycle，取其最大編號的點k，其餘點都可作k→k中介，Floyd必能找出成本負的closed walk，故某對角線會負。這給偵測的兩方向。

有負環時，固定stage仍能產生一些有限walk值；但它們不能當作受影響pair的最短距離，上一頁依D[k,k]=0的原地等價論證也不適用。標準k,i,j原地更新仍可用於負環偵測：所有有限值都有真walk見證，且不大於相應buffered候選，故不漏掉負對角；只是不同更新細節可得到不同暫存負數。

另算reachability R。只有滿足「存在k，D[k,k]<0、R[i,k]且R[k,j]」的(i,j)標為−∞。充分性是i先到負圈、重複任意多次再到j。必要性是：若i→j相關區域沒有負cycle，刪除其所有非負圈後，只剩有限多simple paths，最小成本有下界，不可能為−∞。不可達仍為∞；其餘pair的Floyd值維持有效。

標記後才交付結果，不把−∞再投入一般∞加法。 Johnson可直接拒絕整張有負環的圖；若要交付部分有限結果，則需這種pair層級語義。

不是整張圖都受負環影響

圖有1→2權2、2→3權−3、3→2權1、3→4權4，點5孤立。負圈2→3→2成本−2。

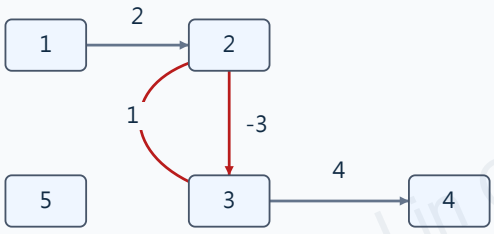
起點＼終點	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

1可以進入負圈再離開到4，所以δ(1,4)=−∞；但沒有回1的路，δ(1,1)=0。4、5不能進負圈，自己的零長距離不變。這比「發現負對角就把所有格標錯誤」更精確。

不是整張圖都受負環影響 （續）

負環影響的是9個點對，並非整張矩陣

紅邊負圈2→3→2成本−2；紅格只標能進入且能離開負圈的點對。



	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

1能進圈但不能回1，故δ(1,1)=0；4與5不能進圈。

Transitive Closure：相同分解，不同代數

$R^{(0)}[i,j]=(i=j)$或存在$i\to j$；保留零長路徑，因此對角線true。把 min/+ 換成 OR/AND，得到

$R^{(k)}[i,j]=R^{(k-1)}[i,j]$ OR ($R^{(k-1)}[i,k]$ AND $R^{(k-1)}[k,j]$)。

有路徑時可刪除重複頂點，不管圈的權重；依是否使用中介k切開，兩方向涵蓋全部可達路徑。這是布林版本的正確性，不依賴無負cycle。原地更新也安全，因R[k,k]=true，而R[i,k] OR (R[i,k] AND true)仍是R[i,k]，row k同理。

六邊共同例的終值rows為 [1,1,1,1]、[0,1,1,1]、[0,1,1,1]、[0,1,1,1]。時間$\Theta(n^3)$布林操作，空間$\Theta(n^2)$布林值；bit packing可減儲存與利用word並行，但須明說word大小及採用的實作，不能無條件把bit操作當任意長字串一次完成。

25.3 Johnson’ s algorithm for sparse graphs

Johnson要把每條負邊改成非負，同時保持同一對端點所有路徑的相對成本。給任意有限potential $h:V\to R$，定義 $\hat{w}(u,v)=w(u,v)+h(u)-h(v)$。對路徑$v_0,\cdots,v_r$相加，內部h消去：

$\hat{w}(path)=w(path)+h(v_0)-h(v_r)$。

對固定起終點，增加量完全相同，所以成本的嚴格大小與等值都保留，最短路徑**集合**相同；不同tie-breaking仍可能挑出不同predecessor tree。對cycle起終同點，增加量0，故負cycle也完全保留；重加權不可能把原負cycle「修成可解」。

反例：對每條邊一律加常數C，長r的路徑會多rC，不是固定端點常數，可能改變最短路徑。1→4直達權6、1→2權2、2→4權2，原本後者權4較短；每邊加3後直達9而兩段10，答案反轉。

Potential 的存在、計算與負環拒絕

新增super-source s，向每個原點加權0邊，且沒有邊回s。若原圖無負cycle，新圖也沒有；每個v可達，Bellman–Ford得到有限$h(v)=\delta(s,v)$。對任一$u\to v$，接上邊可得$h(v)\le h(u)+w(u,v)$，重排就是$\hat{w}(u,v)\ge 0$，這才允許Dijkstra。

可不用實際存s的n條邊：先設每個h[v]=0（等價先放鬆所有s→v），再同步掃原圖邊n−1輪，最後檢查還有無h[v]>h[u]+w(u,v)。它相當於從任一原點零成本起跑、最多n−1條原邊；無負cycle的最短路徑可取simple，因此足夠。有任意負cycle時，各邊potential不等式不可能同時成立，沿cycle相加會推出$0\le$負數，所以最後必拒絕。

共同例初始h=(0,0,0,0)；第一輪只有2→3把h[3]降為−2，得到(0,0,−2,0)；第二、三輪不變。六條邊依序重加權為4、9、0、6、1、1，全部非負。這裡的h不是從原頂點1起跑的距離；h[2]=0而$\delta(1,2)=4$。

Potential 的存在、計算與負環拒絕 （續）

Johnson 的 Potential 與六條重加權

	h[1]=0		h[2]=0		h[3]=-2		h[4]=0
原邊		w		h[u]		h[v]	$\hat{w}=w+h[u]-h[v]$
1→2		4		0		0	4+(0)−(0)=4
1→3		7		0		-2	7+(0)−(-2)=9
2→3		-2		0		-2	-2+(0)−(-2)=0
2→4		6		0		0	6+(0)−(0)=6
3→4		3		-2		0	3+(-2)−(0)=1
4→2		1		0		0	1+(0)−(0)=1

綠格全非負；h來自新增零邊source，不是從頂點1出發的原距離。

每個來源跑 Dijkstra，再還原距離

```
Johnson(G):
  h=前頁全零初始化的Bellman-Ford potential
  若仍可relax則回報負cycle並停止
  對每條原邊設 ŵ(u,v)=w(u,v)+h[u]-h[v]
  for source u in V:
    以非負ŵ在原圖頂點上做Dijkstra，得distHat[v]
    for v in V:
      D[u,v]=∞ if distHat[v]==∞ else distHat[v]-h[u]+h[v]
  return D
```

從1出發，初始distHat=(0,∞,∞,∞)。取出1後為(0,4,9,∞)；取出2後，3由9降4、4由∞降10；取出3後4由10降5；最後取4不改善2。故依序取出1:0、2:4、3:4、4:5（等距採編號小者先），終值(0,4,4,5)，還原為(0,4,2,5)。

其餘來源的distHat rows依序為 [∞,0,0,1]、[∞,2,0,1]、[∞,1,1,0]。各格減h[source]再加h[target]，得到前述Floyd最終矩陣。例如3→2原距離2−(−2)+0=4，4→3原距離1−0+(−2)=−1。不可達仍∞，不要對sentinel做有限數的還原。

每個來源跑 Dijkstra，再還原距離 （續）

非負圖的 Dijkstra 與原距離還原

重加權皆非負；格內是distHat。相同距離以編號小者先取。

動作	到1	到2	到3	到4
初始化	0	∞	∞	∞
取出1	0	4	9	∞
取出2	0	4	4	10
取出3	0	4	4	5
取出4	0	4	4	5

綠格 = 本輪嚴格改善；原距離 = distHat − h[1] + h[v]。

終值 (0,4,4,5) 還原為 (0,4,2,5)，與Floyd row1一致。

稀疏性、Priority Queue 與輸出成本

全零potential實作初始化O(n)、n−1輪各掃m邊、最後再掃m邊：O(nm+n)。若直接在擴增圖做一般Bellman–Ford，則為O((n+1)(m+n))，不要忘記新加的n條邊。

Fibonacci heap的Dijkstra為O(m+n log(n+1))攤銷時間；做n次並填n^2輸出，Johnson總O(nm+n^2 log(n+1))。Binary heap則是O(n(m+n)log(n+1))，不能引用前一個去掉m的log因子的界。額外圖與單趟狀態O(n+m)，若保存完整輸出另需Θ(n^2)；逐row串流可省輸出常駐空間，但沒有交付少於n^2格的同一輸出。

n=1000,m=5000時：Floyd有10^9個(i,j,k)更新；potential至多約5×10^6次原邊掃描，binary-heap重複Dijkstra的量級項約$1000\cdot6000\cdot\log_2 1000\approx5.98\times10^7$，Fibonacci-heap界的量級項則$5\times10^6+10^6\cdot\log_2 1000\approx1.50\times10^7$。這些是不同基本操作的成本模型，**不是實測執行時間或可直接相除的加速比**。稠密m=Θ(n^2)時Floyd簡單且Θ(n^3)，Johnson的稀疏優勢不再成立。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 25。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26 最大流

Part VI Graph Algorithms

本章定位

從可行流與割的上下界出發，說明增廣路為何需要撤回舊決策，再連到配對、節點容量與push–relabel。數例同時核對流量守恆、殘餘容量及最優性證書。

前置知識

有向圖、BFS與求和記號；本章會建立流量守恆、殘餘網路及割的定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 建立 flow network 與 flow value。
- 用 residual graph 修正 Ford-Fulkerson 的增廣。
- 推導 max-flow min-cut 並連接 bipartite matching。
- 26.1 Flow networks
- 26.2 The Ford-Fulkerson method
- 26.3 Maximum bipartite matching
- 26.4 Push-relabel algorithms
- 26.5 The relabel-to-front algorithm

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.1 Flow networks

容量網路有有限頂點V、不同的source s與sink t、容量c(u,v)≥0。無self-loop，非邊容量0；平行同向邊可合併容量。記n=|V|≥2、m為正容量有向邊數；允許孤點與不在s-t路上的點。

本章統一採反對稱淨流f(u,v)=−f(v,u)，並要求f(u,v)≤c(u,v)，所以完整範圍是−c(v,u)≤f(u,v)≤c(u,v)。內點u≠s,t滿足Σ_v f(u,v)=0；流值|f|=Σ_v f(s,v)，是source淨流出，不是邊流絕對值的和。

不能再對全部有序對要求f≥0，否則反對稱會迫使f全為0。容量可為實數；時間先按精確算術操作數計，整數／有理數的位元成本另列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非負 Arc Flow 與淨流不能混算

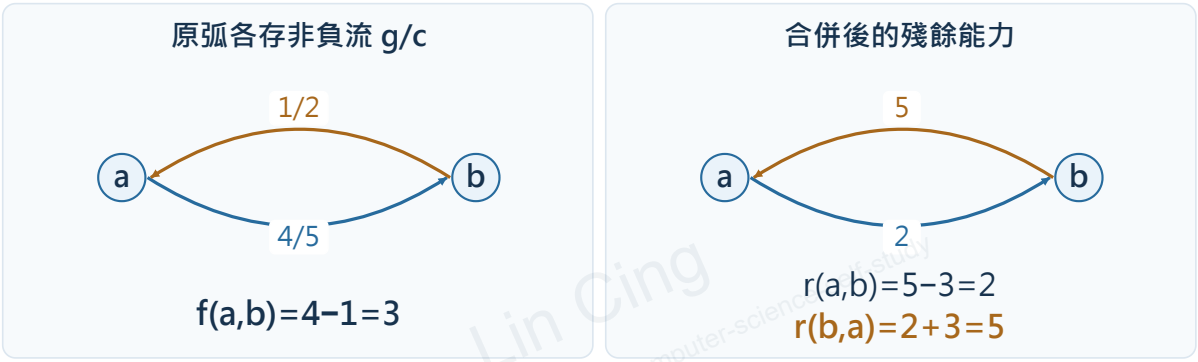
另一表示對原邊存0≤g(u,v)≤c(u,v)，不要求g反對稱。令f(u,v)=g(u,v)−g(v,u)，守恆與淨值不變；雙向都有流時，可取消min(g(u,v),g(v,u))而不改任何點的淨收支。

反過來，給淨流f，令g(u,v)=max(f(u,v),0)。容量上界使g≤c，且g(u,v)−g(v,u)=f(u,v)，故可恢復非負arc flow，兩模型的最大淨值相同。

例c(a,b)=5、c(b,a)=2，若g(a,b)=4、g(b,a)=1，淨f(a,b)=3、f(b,a)=−3。取消雙向各1後，原arc流成3與0。淨流的正向殘餘5−3=2，反向殘餘2−(−3)=5，包含撤回3與新增反向2的可能。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

反向容量與撤回舊流，各自貢獻多少？



反向5 = 原本可反向送2 + 可撤回淨流3

左圖只看一對反平行弧：c(a,b)=5、c(b,a)=2，原弧流為4與1，淨流為3。右圖b→a可增加5：先撤回淨流3，還能使用原本反向容量2。**殘餘弧不等於原圖有同方向的邊；也不能把反向殘餘一律當作只有撤流。**

Cut 的上界來自內部相消

Cut(S,T)分割全部V，s∈S、t∈T；c(S,T)=Σ_(u∈S,v∈T)c(u,v)，只算S→T方向。把S內所有點的淨流出相加，source貢獻|f|，其他內點守恆為0；S內每對f(u,v)+f(v,u)=0相消，所以

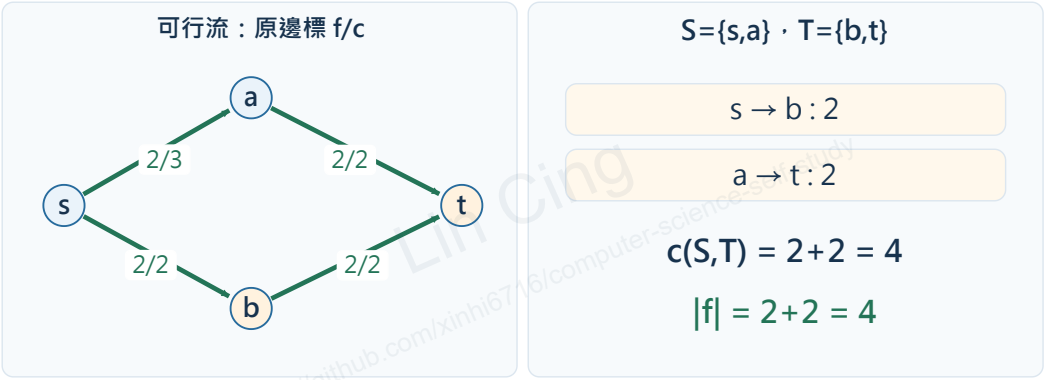
$$|f| = \sum_{u \in S, v \in T} f(u,v) \le \sum_{u \in S, v \in T} c(u,v) = c(S,T).$$

跨割必算淨值，不是只加正流。任何可行流給最大值下界，任何cut給上界；兩者相等便是可核驗的最佳性證書。

同一網路交付流與割兩份證書

容量s→a=3、a→t=2、s→b=2、b→t=2。各沿s→a→t、s→b→t送2，內點守恆且不超容量，流值4。

取S={s,a}、T={b,t}，跨割容量為s→b的2加a→t的2，總4。因此最大流同時≥4且≤4，答案就是4。



流給下界、割給上界；兩者相等，便證明最優

只給source總出容量或sink總入容量不能推定中間可達性；此例給了全部邊、可行流與等值cut，才足以證最優。

多 Source／Sink 與有限超邊

若供給端A、需求端B各無個別上限，且A∩B=∅，加supersource s'→各a∈A、各b∈B→supersink t'。超邊容量可用原邊容量總和C，不必實作∞；任何有效運輸量都不超過全部原容量和。若端點有各自上限，就用其上限，不能默認無限。

把原供需淨量接到超邊即得新守恆流；移去超邊則恢復原供需量，總值不變。這是兩向對應，不只是畫一個新source。

若使用非負arc表示且想消除antiparallel原邊，可將u→v容量c改成u→x→v、兩段各容量c。新點x守恆迫使兩段同流，與原邊一一對應。容量不能各改成c/2。

26.2 The Ford-Fulkerson method

殘餘容量$c_f(u,v)=c(u,v)-f(u,v)\geq 0$，residual graph只保留$c_f>0$的有向邊；鄰居候選必含原邊的反向，即使反向原容量0。

對residual簡單s-t路p，取$\delta=\min_{(uv\in p)}c_f(u,v)>0$。沿每條uv同時加$f(u,v)+=\delta$、$f(v,u)-=\delta$。正向不超容量，反向下降也不超其上界；反向下界則等價於正向上界。路內每點的入／出改變抵消，source淨流出增δ，故新f仍可行且值嚴格增加δ。

```
f=zero net flow
while a residual s-t path p exists:
    delta=min(residual(u,v) for (u,v) in p)
    for (u,v) in p:
        f(u,v)+=delta; f(v,u)-=delta
return f
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

必須撤回舊決策的兩次增廣

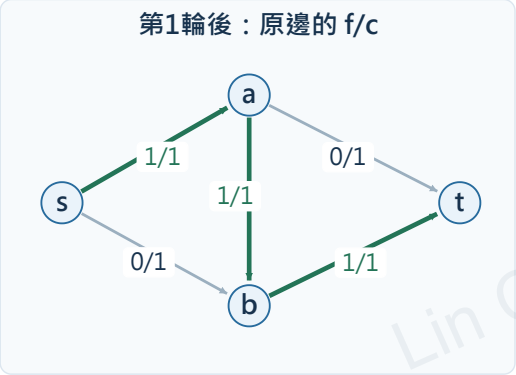
原邊s→a、s→b、a→b、a→t、b→t容量皆1：

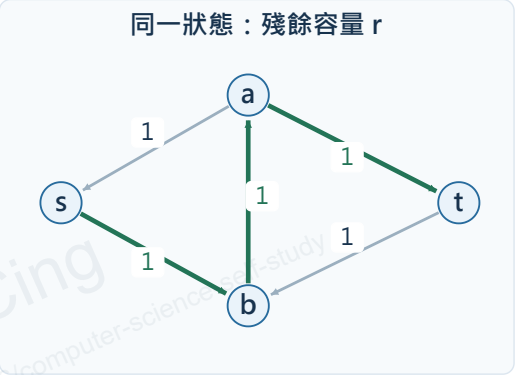
階段	路徑、δ	(f_sa,f_sb,f_ab,f_at,f_bt)	值
初始	—	0, 0, 0, 0, 0	0
第1次	s→a→b→t，1	1, 0, 1, 0, 1	1
第2次	s→b→a→t，1	1, 1, 0, 1, 1	2

第二路的b→a取消舊a→b流，使它由1回0。最後是s→a→t與s→b→t各1。若只保留原方向，第一輪後會錯把「沒有原方向路」當成「沒有residual路」，誤答1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

必須撤回舊決策的兩次增廣 （續）

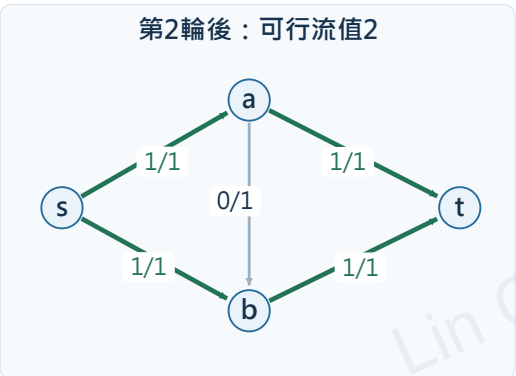




右圖綠路 s→b→a→t 的瓶頸為1；b→a 是撤回能力

若直接儲存殘餘邊，等價更新是r(u,v)−=δ、r(v,u)+=δ；只減正向容量會丟失撤回能力。

撤流後，逐條核對原邊與最優值



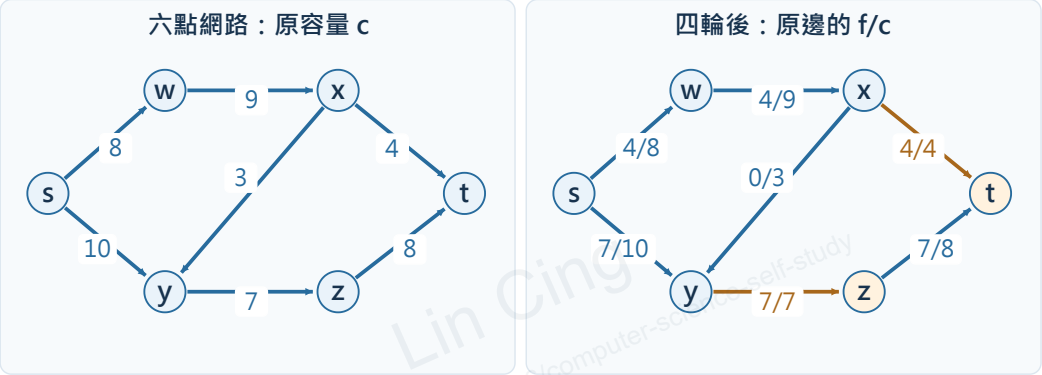
逐條原邊核對

s→a	1 → 1
s→b	0 → 1
a→b	1 → 0
a→t	0 → 1
b→t	1 → 1

a→b由1退到0；source兩條出邊已飽和，cut {s} 容量2

第二次增廣只更新s→b、b→a、a→t。b→a使淨f(a,b)減1，不是把a→b的原容量刪掉。最後a、b各收1送1；流值2等於割{s}容量2，故已最優。剩下的殘餘a→b仍有容量1，但source到不了a，不能用它再增廣。

六點網路：同一個最優值，可以怎樣走到？



S={s,w,x,y}：跨割容量 x→t:4 加 y→z:7，等於流值11

原圖只有x→y:3，沒有y→x。右圖是四輪增廣後的原邊流量f/c，並非殘餘容量。下面固定流量向量順序為(sw,wx,xy,xt,sy,yz,zt)，逐輪核對中間的撤流。

前兩輪：先走x→y，再沿y→x撤回

輪次	路徑與增廣前殘餘容量	δ	增廣後原邊流量向量
初始	—	—	(0,0,0,0,0,0,0)
1	s-w-x-y-z-t；(8,9,3,7,8)	3	(3,3,3,0,0,3,3)
2	s-y-x-t；(10,3,4)	3	(3,3,0,3,3,3,3)

第1輪才創造出殘餘y→x:3。第2輪沿它送3，使原f(x,y)由3變0；不是在原圖額外送一條y→x流。此時x的收支是wx:3流入、xt:3流出；y的收支是sy:3流入、yz:3流出。source淨流出3+3=6，兩輪都增加3。

後兩輪：用流量守恆與割完成檢核

向量仍依(sw,wx,xy,xt,sy,yz,zt)排列。

輪次	路徑與增廣前殘餘容量	δ	增廣後原邊流量向量
3	s-y-z-t；(7,4,5)	4	(3,3,0,3,7,7,7)
4	s-w-x-t；(5,6,1)	1	(4,4,0,4,7,7,7)

流值6→10→11。最終w收4送4，x收4送4，y收7送7，z收7送7。殘餘從s可達S={s,w,x,y}，到不了z或t；原圖跨割只有x→t:4與y→z:7，合計11。**可行流11給下界，割11給上界，所以最大流恰為11。** 只看source原出容量8+10=18，無法得到這個答案。

Max-flow Min-cut 的三向等價

對可行f，以下等價：(i)f最大；(ii)residual無s-t路；(iii)存在cut使|f|=c(S,T)。

(i)⇒(ii)：若有路，增廣得到更大可行流，矛盾。

(ii)⇒(iii)：令S為residual從s可達點、T=V\S。t不可達，所以為合法cut。對u∈S,v∈T，c_f(u,v)=0，否則v也可達；故f(u,v)=c(u,v)，由cut淨流等式得|f|=c(S,T)。

(iii)⇒(i)：任一可行g都有|g|≤c(S,T)=|f|，所以f最大。

定理不要求整數容量，也不要求特定選路。整數性用於終止與成本，不是這個等價定理的前提。

整數性與偽多項式成本

整數容量從f=0開始，residual與δ皆整數，更新後仍整數。每次值至少增1，且有有限cut上界，所以必終止，並得到逐邊整數的最大淨流。設最優值F*，增廣至多F*次。

以adjacency list、一次O(n+m)配置及visited時間戳記，找路只初始化本輪觸及的點；每個新見非source點對應一條掃過的邊，每輪O(m+1)。總O(n+m+(m+1)F*)。F*可能對binary輸入長度指數大，因此是pseudopolynomial。

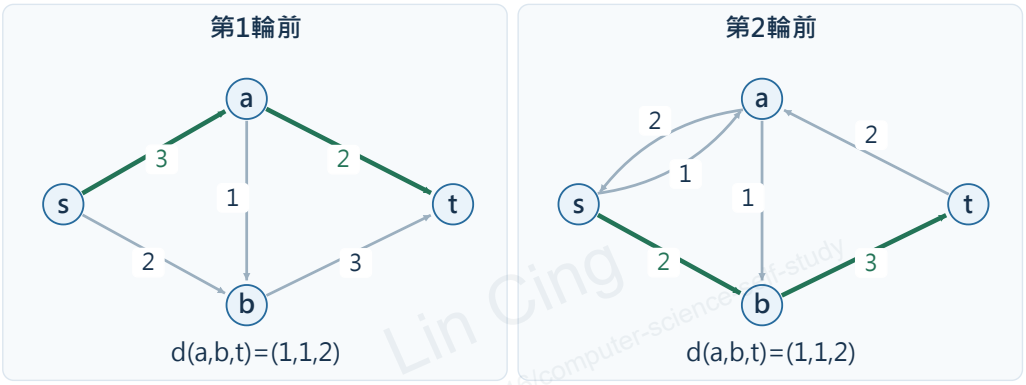
有理容量同乘分母乘積D可整數化；bitlen(D)不超各分母位元數總和，卻不代表D的數值小，所以仍可能有巨大增廣次數。無理容量下「每次至少1」不成立，不能由此證明宣稱任意選路終止。以下BFS版本改用容量值無關的輪數界。

Edmonds–Karp：殘餘邊數最短，不是容量最小

每輪以BFS找最少邊數s-t路，parent回溯增廣。d_f(v)為從s到v的最短邊數，不可達為∞。容量s→a=3、s→b=2、a→b=1、a→t=2、b→t=3，鄰居a先於b：

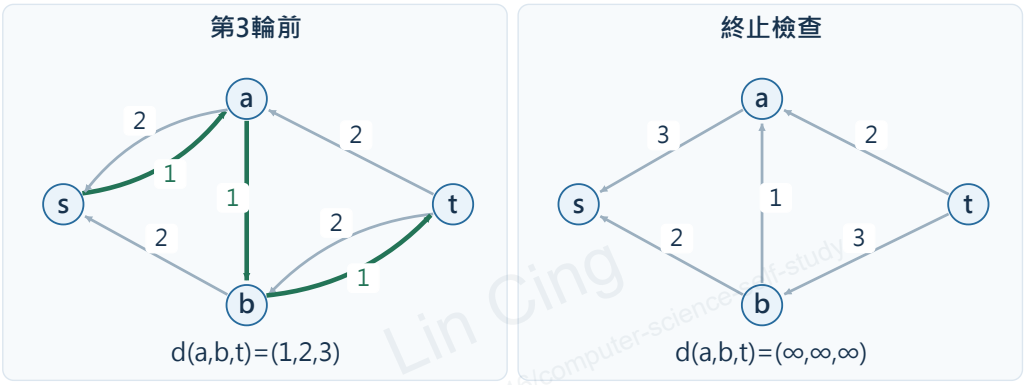
增廣前	d(a),d(b),d(t)	路徑與δ	增廣後f_sa,f_sb,f_ab,f_at,f_bt
第1輪	1, 1, 2	s→a→t，2	2, 0, 0, 2, 0
第2輪	1, 1, 2	s→b→t，2	2, 2, 0, 2, 2
第3輪	1, 2, 3	s→a→b→t，1	3, 2, 1, 2, 3
終止檢查	∞, ∞, ∞	無路	值5，cut {s}容量5

Edmonds–Karp：殘餘邊數最短，不是容量最小 (續)



僅畫r>0的弧；綠色是本輪最少邊數路，δ依次為2、2

第3輪與終止：從殘餘圖讀出割



第3輪δ=1；最終s無出弧，流值5等於cut {s} 容量5

第3輪走s→a→b→t，瓶頸min(1,1,1)=1，流值4→5。更新後只剩a→s:3、b→s:2、b→a:1、t→a:2、t→b:3。從s只能到自己，所以S={s}；原圖跨割容量3+2=5，與流值相等。BFS的∞表示不可達，不是容量為∞。

BFS 距離不下降的反證

增廣前後記d、d'。假設有d'(v)<d(v)，選這類點中d'(v)最小者；令u為新最短路的前驅，d'(u)=d'(v)−1。u不是更小的下降反例，所以d(u)≤d'(u)。

若uv本來就在residual，則d(v)≤d(u)+1≤d'(v)，矛盾。因此uv是剛出現的反向邊，vu在剛才BFS增廣路上。舊最短路的前綴也最短，故d(u)=d(v)+1。

合併得d(v)=d(u)−1≤d'(u)−1=d'(v)−2，與d'(v)<d(v)矛盾。原不可達、新可達的∞情形也適用：上述有限前驅關係會迫使舊v可達。所以每點距離單調不減。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

臨界邊再次飽和之前，距離至少加二

每次增廣至少一條critical邊uv達到瓶頸δ，殘餘變0；此時d(v)=d(u)+1。它要重現，之後必沿vu反向增廣；該輪d'(u)=d'(v)+1。由距離不下降，d'(u)≥d(v)+1=d(u)+2。

因此同一方向每次重新成為critical，尾點距離至少增加2；仍在s-t簡單路上時距離≤n−2，所以每方向O(n)次。候選方向至多2m，總O(nm)次增廣。

每輪BFS／更新O(m+1)，m≥1時總$O(n+m+nm^2)$，通常省初始化寫$O(nm^2)$；m=0直接零流O(n)。有理容量共分母整數化後，所有值的bit長受原輸入長度多項式控制，輪數又與數值無關，因此也是多項式bit演算法。$O(nm^2)$先指算術操作界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逃生問題：限制邊，和限制格點是兩回事

只限制邊會算出2；限制每個格點的通過人數才得到1

座標採(row,column)。可走格點為{(3,2),(2,3),(3,3),(3,4),(3,5),(4,3),(5,3)}，其他格點封閉。兩人從u=(3,2)、v=(2,3)出發，出口為邊界x=(3,5)、y=(5,3)。只限制每條邊容量1，兩條不同路可各送1，卻都經過z=(3,3)。**若題目禁止共用格點，這個流值2並不代表兩人都能逃出。**

把每個格點拆成「進入」與「離開」

對k×k網格，令q為可走格點數、p為不同起點的人數、b為可用邊界出口數。禁止共用任何格點時，建構如下：

原限制	網路表示
每格至多通過一人	每個v拆成v_in→v_out，容量1
可移到相鄰可走格	u_out→v_in與v_out→u_in，各容量1
每個起點有一人	source→起點_in，容量1
從可走邊界離開	邊界_out→sink，容量1

障礙格不建立頂點或邊。起點若已在邊界，也要經自己的內部容量邊再離開。網路有N=2q+2個頂點，邊數M≤q+4k(k−1)+p+b；完整無障礙網格的b=4k−4只適用k≥2，k=1時b=1。

為何最大流等於p，才能讓所有人逃出？

若有p條格點互不重複的逃生路，沿對應網路路徑各送1，所有容量與守恆都滿足。反過來，整數容量保證存在整數最大流；將流分解成source–sink路徑與循環，移除循環不改流值。值為p時，p條source出邊全部飽和；每條內部容量1使任何兩條路都不能使用同一格點。投回原網格便得到每個人的逃生路。這證明兩方向都成立。

本例所有出口路都經z。拆點後的z_in→z_out容量1構成一個割，故最多一人；路u→z→(3,4)→x又給可行流1，所以最大值恰為1。只做邊容量的模型有兩條邊不共用的路，最大值為2；**兩個答案不同，是因為限制不同。**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.3 Maximum bipartite matching

二分圖L∪R的matching是不共端點的邊集合。建立s→每個左點容量1、原邊l→r容量1、每個右點→t容量1。

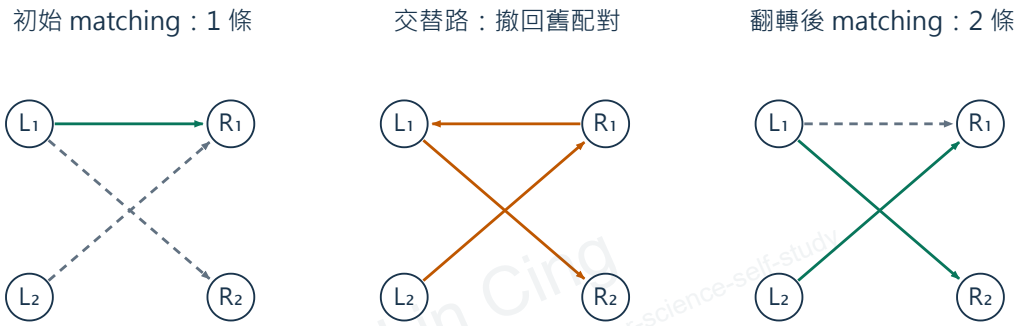
給matching M，沿每條s→l→r→t送1，端點互斥使容量不超，值|M|。反向給整數流：每個左點只有一條容量1入邊，守恆與整數性使其至多一條原匹配出邊為1；右點同理。因此f(l,r)=1的邊形成matching，大小等於流值。

整數增廣保證存在整數最大流，所以最大matching=最大flow。不能說每個最優流皆逐邊整數：$K_{2,2}$的四條左→右邊各送1/2、source與sink邊各1，也有最優值2，卻不能把四條正流邊全選作matching。

最大matching≤min(|L|,|R|)，每輪增1。新網路m'=|E|+|L|+|R|，直接Ford–Fulkerson成本O(|V|+m'+m'・min(|L|,|R|))。有孤點時不能把m'無條件寫成Θ(|E|)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一條交替路實際翻轉哪些邊



$L_2 \to R_1 \to L_1 \to R_2$：未配、已配反向、未配

綠實線 = matching；灰虛線 = 未選原邊；橘箭頭 = residual 交替路

L_1可配R_1/R_2，L_2只可配R_1。先配L_1R_1後仍有residual路$s\to L_2\to R_1\to L_1\to R_2\to t$。反向$R_1\to L_1$撤回舊匹配，另取$L_2R_1$、$L_1R_2$，大小由1增2。

Alternating path的未配／已配邊交替；翻轉後每個內點仍接一條matching邊，兩個原未配端點各新增一條。

Kőnig 等式的可重建證明

取最大matching的residual，Z為從s可達點，$C=(L\setminus Z)\cup(R\cap Z)$。對原邊lr，若$l\in Z, r\notin Z$：未配邊有正向residual使r可達；已配時$s\to l$已飽和，能到l的最後邊只能從其匹配右點r反向而來，仍使r可達。兩種都矛盾，所以C覆蓋全部原邊。

每條matching邊的兩端同時可達或不可達：右端可沿反向到左；已配左端可達必先經它的右端。因此C對每條matching恰取一端。未配左點都可由s到達，不入C；未配右點不可達，否則能增廣，也不入C。所以$|C|=|M|$。

任一cover至少為matching每邊取一個不同端點，故二分圖最小cover大小=最大matching大小。一般圖不成立，例如三角形兩值為2與1。

Hall 條件的充分性

要匹配L全部點，必要條件是對每個A⊆L都有|N(A)|≥|A|，因matching給A不同鄰居。

若最大matching未覆蓋L，從未配左點做交替可達搜尋，得到左集合A、右集合B。前頁正反向論證給B=N(A)。所有B點已配，否則可增廣；它們的匹配左端都在A，而A還有至少一個起始未配左點。因此|A|>|B|=|N(A)|，違反Hall條件。

所以必要條件也充分。L空時空matching已覆蓋L，無須尋找未配起點。此結論講的是覆蓋全部左側；若左右同大，才同時成為兩側的perfect matching。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.4 Push-relabel algorithms

Preflow仍滿足反對稱與容量界，但允許暫存excess e(u)=Σ_v f(v,u)≥0，u≠s。Active只指u≠s,t且e(u)>0；sink雖累積excess，仍不discharge。

有效height滿足h(s)=n、h(t)=0，每條residual uv有h(u)≤h(v)+1。初始化其他height與f/e全0，設h(s)=n並飽和source出邊：f(s,v)=c(s,v)、f(v,s)=−c(s,v)，e(v)+=c(s,v)、e(s)−=c(s,v)。

初態所有非source excess非負；source無正殘餘出邊，其他邊從height0出發也滿足標號。Height不是每輪BFS的真距離，而是約束殘餘邊的有效標號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Push／Relabel 的完整條件

```
push(u,v): # u active, residual>0, h[u]==h[v]+1
    d=min(e[u],c(u,v)-f(u,v))
    f(u,v)+=d; f(v,u)-=d; e[u]-=d; e[v]+=d
relabel(u): # u active, no admissible outgoing edge
    h[u]=1+min(h[v] for v with residual(u,v)>0)
```

Admissible指正殘餘、恰下降1層。Push後殘餘0稱saturating；否則nonsaturating，必使e(u)=0；兩者相等按saturating計。

Active必有正殘餘出邊，否則每個f(u,v)=c(u,v)≥0，導致e(u)=−Σf(u,v)≤0，矛盾。故relabel的min非空。無admissible時，整數有效標號使h(u)≤所有殘餘鄰居height，新height至少增1。

基本操作保持 Preflow 與有效 Height

Push不使e(u)變負，其他非source excess只增；容量／反對稱同增廣證明。Height不變，唯一可能新增的反向vu滿足h(v)=h(u)−1，所以仍有效，但不會admissible。

Relabel不改f/e。新h(u)=1+min h(v)使u出邊有效；對入邊wu，舊有效性有h(w)≤old_h(u)+1≤new_h(u)，所以入邊仍有效，且不admissible。Relabel後至少一條取到最小鄰居height的出邊成admissible。

Admissible network是DAG：沿每邊height減1，繞長k>0的cycle會推出h=h−k，矛盾。Push不新增admissible邊，relabel只可能新增自己的出邊，這兩點支撐後面的list拓撲不變量。

Active 點總有路回 Source，故 Height 有界

令R為active點u在residual可達集合，假設s∉R。R內非source excess皆非負、e(u)>0，所以 Σ_(v∈R)e(v)>0。但R沒有正殘餘出邊，故對v∈R,w∉R，f(v,w)=c(v,w)≥0；內部反對稱抵消給

Σ_(v∈R)e(v)=−Σ_(v∈R,w∉R)f(v,w)≤0，矛盾。

所以u有至多n−1邊的residual簡單路回s。沿路相加有效標號得h(u)≤h(s)+n−1=2n−1。每次relabel時u仍active，故新height同受此界；其餘時height不變。每內點最多2n−1次relabel，全圖O(n^2)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

終止時為最大流，還需另證一定終止

若全部內點excess0，preflow恢復守恆，是可行flow。若residual有s-t簡單路、長k≤n−1，沿有效標號得到n=h(s)≤h(t)+k=k≤n−1，矛盾。因此無增廣路，由max-flow min-cut得最大流。

Sink得到的excess是流值；一開始推出的多餘流可以回source，不必全進sink。

「終止時正確」不等於「必會終止」。接著分開限制relabel、saturating push與nonsaturating push數量，取得容量值無關的有限界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

飽和 Push 總數 O(nm)

固定方向uv。首次飽和時h(u)=h(v)+1且殘餘歸0；要再正向推送，必先有vu反向push恢復殘餘。該步h(v)=h(u)+1，height又不降，所以v至少比前次增2。

到uv再次飽和，u又需比v高1，因此u也至少比前次正向飽和時高2。Height≤2n−1，每個方向最多O(n)次；候選方向≤2m，總O(nm)。

此界依高度差與單調性，而非每次δ大小，對小數容量也成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非飽和 Push 的勢能計費

取Φ=Σ_(active u)h(u)，初始0、始終非負。Relabel增加Φ恰為該height增加量，所有點全程height總增加≤(n−2)(2n−1)=O(n^2)。

飽和push只可能使目的點新active，Φ增加至多2n−1；共O(nm)次，正增加共O(n^2m)。非飽和push清空u；若v新active，最多加h(v)=h(u)−1，若v已active或為終端則不加。因此Φ至少減1。

由非負勢能，這類下降次數不超全部正增加，為O(n^2+n^2m)，m≥1時O(n^2m)。連同relabel與飽和push，generic基本操作共O(n^2m)；m=0時初始化後已無active，直接終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

26.5 The relabel-to-front algorithm

標準linked list L始終含V\{s,t}全部點，包括e=0者；不是只存active。固定N(u)={v：c(u,v)>0或c(v,u)>0}，含反向候選。current跨discharge保存：

```
discharge(u):
    while e[u]>0:
        v=current[u]
        if v==NIL:
            relabel(u); current[u]=head(N[u])
        elif residual(u,v)>0 and h[u]==h[v]+1:
            push(u,v) # 此步不前進current
        else: current[u]=next-neighbor(v)
```

飽和後下一迭代才略過該邊；若推送已清空u，立刻返回，current仍指原鄰居。不能把「每push自動前進」的另一版程式與此trace混搭。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

略過的 Current Arc 為何不用立即重看

固定u高度不變的一段。若uv因殘餘0略過，未來只能經vu反向push恢復；當時h(v)=h(u)+1，新uv往高處走，仍非admissible。若有殘餘但不admissible，有效標號給h(u)≤h(v)，v之後只升高，也不會成下降1層。

亦可從一般性質看：push不新增admissible邊；鄰居relabel後無admissible入邊。只有u自身relabel才可能新增自己的admissible出邊，屆時current重設。

所以掃到NIL表示所有已略過邊現在仍不可推，relabel確實合法；不是因為「曾經看過」就忽略邊的狀態改變。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Move-front 後讀取新位置的 Next

```
initialize_preflow()
L=all vertices except s,t, in any fixed order
current[u]=head(N[u]) for u in L
u=head(L)
while u!=NIL:
    old=h[u]; discharge(u)
    if h[u]>old: move u to front of L
    u=next(u) # 移動後的新位置
```

Move在整次discharge結束後做，不是每次內部relabel就做。中間u移到head後，next是舊head，會重查先前可能收到excess的點。不能保存舊next直接跳過它們。

List 不變量的兩種分支

每次外圈測試：(1)L為非s/t點的admissible network拓撲序；(2)u之前的點皆無excess。初態非source高度皆0，彼此無admissible邊，任意L符合，u在head。

若discharge無relabel，push不新增admissible邊，L仍拓撲有序；只把excess推到後方點或終端，不影響前綴。u最後清空，下一個u以前仍全無excess。

若曾relabel，最後一次relabel後u無admissible入邊，其後push也不新增；移到head保持拓撲序。新next之前只有剛清空的u，前綴條件恢復，其他點相對順序不變。

到u=NIL時，全部L皆為無excess前綴，故守恆，結合generic正確性即最大流。這個證明不能直接套到另一種只存active點的排程。

五點網路的初始化與 a 的完整 Discharge

容量s→a=4、s→b=3、s→c=2、a→b=2、a→t=1、b→c=2、b→t=4、c→t=5。固定N(a)=[b,t,s]、N(b)=[c,t,a,s]、N(c)=[t,b,s]，L=[a,b,c]。

初始化h(s)=5、其餘0，飽和source出邊。以下在每個push/relabel剛完成時記current，未合併下一次掃描：

動作	e(a,b,c,t)	h(a,b,c)	current(a,b,c)
初始化	4, 3, 2, 0	0, 0, 0	b,c,t
a掃b,t,s到NIL；relabel	4, 3, 2, 0	1, 0, 0	b,c,t
a→b推2	2, 5, 2, 0	1, 0, 0	b,c,t
略b到t；a→t推1	1, 5, 2, 1	1, 0, 0	t,c,t
掃t,s到NIL；再relabel	1, 5, 2, 1	6, 0, 0	b,c,t
略b,t到s；a→s退1	0, 5, 2, 1	6, 0, 0	s,c,t

五點網路的初始化與 a 的完整 Discharge （續）

a的height從0變6，結束後move-front；因原在head，L不變，接著掃b。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

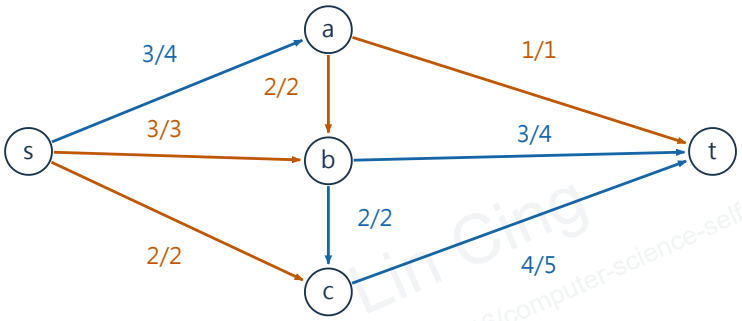
b、c 的 Discharge 與真正移動時點

動作	e(a,b,c,t)	h(a,b,c)	current(a,b,c)	L
b掃c,t,a,s到NIL；relabel	0, 5, 2, 1	6, 1, 0	s,c,t	a,b,c
b→c推2	0, 3, 4, 1	6, 1, 0	s,c,t	a,b,c
略c到t；b→t推3	0, 0, 4, 4	6, 1, 0	s,t,t	a,b,c
b結束後移front	0, 0, 4, 4	6, 1, 0	s,t,t	b,a,c
再掃a，e=0無動作	0, 0, 4, 4	6, 1, 0	s,t,t	b,a,c
c掃t,b,s到NIL；relabel	0, 0, 4, 4	6, 1, 1	s,t,t	b,a,c
c→t推4；結束後移front	0, 0, 0, 8	6, 1, 1	s,t,t	c,b,a

下一點是新位置的b，接著a，皆無excess，最後NIL。b→c後t仍1，只有b→t後才變4。最終source淨流3+3+2=8、sink淨入1+3+4=8。

以 Cut 獨立核對最後的最大流

最後的可行流：每邊標 f/c



h(a,b,c)=(6,1,1)

e(a,b,c)=(0,0,0)

current=(s,t,t)

L=[c,b,a]

e(s)=−8 · e(t)=8

residual 可達 S={s,a}；橘色跨割容量 3+2+2+1=8

a收3送2+1；b收3+2送2+3；c收2+2送4。殘餘可達S={s,a}，跨割原容量3+2+2+1=8，等於可行流值8。初始推出9，其中1由a退回source，所以最後淨流值為8；這不是漏流。割證書獨立於height和排程。

$O(n^3)$ 的完整計費

總relabel $O(n^2)$。兩次相鄰relabel間稱一個phase；沒有relabel時discharge使u沿L向後走，最多n次，含relabel的一次使進入新phase。因此discharge呼叫$O(n^3)$；move-front持有效雙向串列handle可$O(1)$。

每點current在兩次自身relabel之間掃鄰居一次，自身relabel $O(n)$次，全部advances與重標找min合計$O(n\Sigma deg(u))=O(nm)$。飽和push $O(nm)$。每次discharge中非飽和push一旦發生就清空u並返回，所以至多一次，總$O(n^3)$。

合計$O(n^3+nm)=O(n^3)$，因簡單容量網路$m \le n(n-1)$。稀疏鄰接／反向記錄、height/excess/current與L共$O(n+m)$空間。這先是算術次數界，容量加法與比較仍依實際bit長付費。

Generic 操作次數不是免費的選步成本

若每步都掃全部頂點找active，再重掃全部鄰居，不能僅由基本操作$O(n^2m)$就宣稱同一時間界。可另維護active集合：push只改兩端excess，以handle與標記$O(1)$加入／移除，任取active點處理。

鄰居用同一current規則，全部advances與relabel掃描$O(nm)$，push本身$O(1)$，才能實現初始化後$O(n^2m)$算術時間。Relabel-to-front以固定全體串列及discharge另得$O(n^3)$，不是把active集合改個名字。

整數Ford–Fulkerson依F^*、Edmonds–Karp依nm^2、generic依n^2m、relabel-to-front依n^3，是不同機制的最壞操作界，不直接代表某台機器的實測速度。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 26。

Cormen et al., *Introduction to Algorithms*, 3rd ed., § 26.5.

《演算法_wjungle》手寫筆記，網路流、逃生問題與節點容量，PDF實體第37–42頁。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33 計算幾何

Part VII Selected Topics

本章定位

幾何演算法必須把數學判定、退化情形及資料結構順序一致處理；本章以方向測試、交線、凸包及最近點對證明局部判斷如何形成正確全域結果。

前置知識

向量、叉積、排序與掃描線。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用 cross product 判斷線段方向與轉向。
- 以 sweep line 找任兩線段相交。
- 分析 convex hull 與 closest pair。
- 33.1 Line-segment properties
- 33.2 Determining whether any pair of segments intersects
- 33.3 Finding the convex hull
- 33.4 Finding the closest pair of points

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33.1 Line-segment properties

平面點p=(x,y)，閉線段ab={a+t(b−a)：0≤t≤1}。閉線段的交集包含端點接觸、共線重疊及退化為單點的線段，不只真正穿越。

定義方向判定

$$\operatorname{orient}(a,b,c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

本章採一般向右的x軸、向上的y軸：正為a→b→c左轉（counterclockwise），負為右轉，零為共線。這是有向平行四邊形面積；三角形面積為其絕對值的一半。若改用(b−a)與(c−a)的相反叉積順序，正負判準也必反轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

方向判定要精確，不能任意套 Epsilon

若整數座標絕對值≤B，座標差≤2B，兩乘積之差的絕對值≤8B²。儲存原座標的整數型別未必能儲存乘積；必須先擴寬再相減／相乘，或使用任意精度整數。b位元座標的判定涉及O(b)位元整數的乘法，並非任意輸入都能O(1)。

浮點運算可能因相消把很小的非零方向算成0。固定ε不能對任意尺度保證正確：縮放座標λ倍會把orient乘λ²，而拓樸的左／右關係不變。將「差不多共線」當作精確等價，還可能破壞排序比較的傳遞性。

本章正確性採精確實數判定模型；可重算例用整數／有理數精確計算。需要浮點實作時，須使用有誤差界的篩選、失敗後提高精度的精確判定，不能用未證的ε取代符號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線段相交的完整判定

令 o_1=orient(a,b,c)、o_2=orient(a,b,d)、o_3=orient(c,d,a)、o_4=orient(c,d,b)。on(a,b,p)必須同時檢查orient=0與兩軸的閉區間：

```
on(a,b,p):
    return orient(a,b,p)==0 and
           min(ax,bx)<=px<=max(ax,bx) and
           min(ay,by)<=py<=max(ay,by)
intersect(a,b,c,d):
    if sign(o1)*sign(o2)<0 and sign(o3)*sign(o4)<0: return true
    return on(a,b,c) or on(a,b,d) or on(c,d,a) or on(c,d,b)
```

比較符號即可，不必直接乘兩個可能很大的orientation值。垂直線段不需斜率，單點線段也由on處理。若已知共線，on可只檢查bounding box；未知共線時不能省掉第一個條件。

為什麼兩邊都跨越才足夠

在沒有零方向的情形，o_1,o_2異號表示c,d在ab所在直線的不同側；o_3,o_4異號表示a,b在cd直線不同側。前者使cd與ab所在直線的唯一交點落在cd內部；後者使同一個交點也落在ab內部，故兩閉線段相交。

反過來，兩線段若在各自內部穿越，每對端點必分居對方直線兩側。若方向為0，交點不是一般穿越：可能是一端落在另線段上，或兩段共線。兩個共線閉區間有交集時，至少一個端點落在另一區間內；四個on測試完整涵蓋。若其中一段是單點，也正是測該點是否在另段上。

只測一對異號不夠：c,d可以跨過ab的延長線，卻在ab端點之外才相交。

同一判定涵蓋穿越、接觸與分離

取a=(0,0)、b=(4,0)：

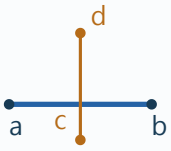
c,d	(o_1,o_2)	(o_3,o_4)	結果
(2,−1),(2,2)	(−4,8)	(6,−6)	內部穿越(2,0)
(4,0),(6,2)	(0,8)	(8,0)	共用端點b
(2,0),(6,0)	(0,0)	(0,0)	重疊[2,4]
(5,0),(6,0)	(0,0)	(0,0)	bounding box分離

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一判定涵蓋穿越、接觸與分離 （續）

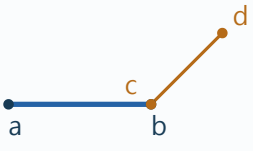
閉線段交集：端點與共線也必判定

內部穿越



$o_1=-4$，$o_2=8$

共用端點



$o_1=0$，$o_2=8$

共線重疊

共同區間 [2,4]

共線分離

bounding boxes 不交

藍段 a=(0,0)→b=(4,0)；四圖 x、y 使用相同單位長度，y 方向向上。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33.2 Determining whether any pair of segments intersects

輸入n條閉線段，目標只回傳是否存在一對相交，不要求列出所有交點。所有配對直接判定需$\Theta(n^2)$；若有$\Theta(n^2)$個交點，完整列舉本來就不能有O(n log n)的總輸出界。

先看非垂直線段。每段整理為左端l、右端r，l_x<r_x。在目前掃描位置x，active集合包含l_x≤x≤r_x的線段；status以交點高度y_s(x)由低至高排序。Event（事件）為左右端點。插入新段只測與上下鄰居；刪除前測將成為鄰居的上下兩段。

比較的key不是線段ID，而是當下的幾何高度。對s=(l,r)，

`y_s(x)=[l_y(r_x−l_x)+(r_y−l_y)(x−l_x)]/(r_x−l_x)`。

分母為正，兩段比較可交叉相乘，避免除法。每次精確算術O(1)的模型下，平衡樹每操作O(log n)。

只在端點更新 Status 的演算法

同x事件先處理所有左端，再右端；同類依y、ID確定次序。高度相同用ID作樹內tie-break，但仍對相鄰段做完整閉線段相交測試。

```
T=empty balanced ordered tree
for each endpoint in (x, left-before-right, y, id) order:
    if left endpoint of s:
        insert s in T at this x
        if intersect(s,below(s)) or intersect(s,above(s)): return true
    else:
        a=below(s); b=above(s)
        if both exist and intersect(a,b): return true
        delete s from T using its stored node handle
return false
```

不存在的鄰居略過。交集測試針對整條線段，所以可能在實際到達交點以前就回傳true。此版本一旦找到就停止；不能把return改成print後照舊繼續，因越過交點後status順序會改變。

鄰居測試為何不漏：最早交點證明

若回傳true，必由精確兩段判定找到真交集，沒有誤報。反證存在交集卻全程回false。選最小x的交點p。p之前沒有任何交叉，active段的高度順序不會自行交換，所以status仍正確。

若兩條在p相交的段在p左側之間還有別段，該段的高度被兩者夾住，連續性使它也通過p；在通過p的有限段群中，至少有一對在p之前相鄰。若相交始於端點，則在該x插入時與相同高度群的鄰居相接；先插後刪保留了恰於同點結束的段。

這對段第一次成為鄰居，只可能由插入其中之一，或刪掉它們之間最後一段造成。兩種事件都做完整交集測試，應已回true，矛盾。共線重疊同樣在重疊開始的端點形成相同高度群，其相鄰段也相交。

這也解釋為何不用對每個x重新排序：尚未找到交集前，順序無改變；若即將改變，先前的鄰居檢查已使演算法停止。

刪除中間段時才出現的新鄰居

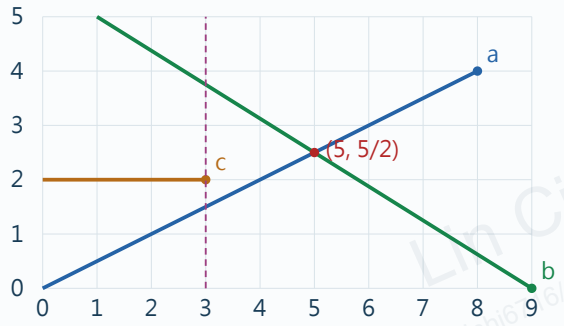
a:(0,0)→(8,4)，b:(1,5)→(9,0)，c:(0,2)→(3,2)。a,b交於(5,5/2)，c與另兩段都不相交。

x與事件	status由低至高	新測試
0插a	a	無
0插c	a,c	a,c不交
1插b	a,c,b	c,b不交
3刪c之前	a,c,b	a,b相交，回true

在x=3三段高度分別3/2、2、15/4，刪c後a,b才相鄰。只測新插入線段會漏掉這類情況。

刪除中間段時才出現的新鄰居 (續)

刪除 c 後，才需要檢查新鄰居 a、b



x=3 的高度：

a：3/2

c：2（將刪除）

b：15/4

刪除前先測 a,b

交點在未來也立即接受

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

垂直線段、端點與成本的完整契約

對一般閉線段，按相同x分批處理：先插入該x開始的非垂直段；再處理該x所有垂直段；最後刪除該x結束的非垂直段。如此端點接觸仍包含於active集合。

垂直段v的y範圍為[L,H]。在status找高度≥L的第一段，若其高度≤H就相交；否則沒有非垂直段在範圍內。這是有序樹lower_bound，O(log n)，不是把整段v硬放成只有一個高度的key。同x的垂直段依L排序；維護已見最大H，若下一個L≤最大H即有重疊，否則更新最大H。單點線段也放入此閉區間群。

排序O(n log n)，2n次非垂直插／刪及n次以內range query各O(log n)，垂直群排序的總成本不超過O(n log n)，故一般版本O(n log n)、空間O(n)。這個界採精確常數成本幾何判定；b位元整數座標的交叉乘積成本須另乘相應位元成本，不能省略。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33.3 Finding the convex hull

凸集包含其任意兩點間整條線段。有限點集Q的凸包CH(Q)是所有凸組合Σλ_ip_i（λ_i≥0、Σλ_i=1）的集合，也是包含Q的最小凸集。

輸出約定：重複座標先去重，只輸出極端頂點，逆時針排列，末尾不重複首點；位於同一條邊中間的點不另列。空集輸出空；單點輸出該點；全部共線且至少兩點時只輸出兩個端點。

Graham scan按極角排序後維護候選stack；Jarvis march每次選下一條支持邊，成本依凸包頂點數h。兩者不需要以浮點atan2計角度：精確orientation與距離平方比較足以決定順序及共線取捨。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Graham Scan：先處理角度相同的點

取p_0為y最小、同y時x最小的點，其餘點都在上半平面或正x方向。對u,v以orient(p_0,u,v)>0判u極角較小；零時比較離p_0距離平方。相同射線只保留最遠點，其餘在p_0和最遠點的線段內，刪掉不改凸包。

```
S=[]
for p in [p0]+points_in_strictly_increasing_polar_angle:
    while len(S)>=2 and orient(S[-2],S[-1],p)<=0:
        pop S
    push p
return S
```

長度檢查使不足三點時也不讀空stack。全部共線先回端點；非共線時掃描結果閉合為凸多邊形。用≤0刪去平角中間點，與本章「只輸出極端頂點」的契約一致；若要求列出每個邊界點，必須另改同射線及最後共線鏈的處理。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pop 的幾何理由：舊頂點已被新三角形包含

固定一次新點p_i。Stack由p_0向上按極角遞增，考慮倒數兩點r,t。若orient(r,t,p_i)≤0，則t不是新凸包頂點。

若$r=p_0$，因t與p_i的極角嚴格遞增且跨度小於π，orient(p_0,t,p_i)>0，這一輪不會pop。因此真正需要pop時$r\neq p_0$。把p_0平移到原點，r,t,p_i的射線按角度依序排列且跨度小於π，所以t位於r與p_i張成的錐內，可寫$t=\alpha r+\beta p_i$，$\alpha,\beta\geq 0$。令D=orient(0,r,p_i)>0，則

`orient(r,t,p_i)=(α+β−1)D`。

非左轉推出$\alpha+\beta\leq 1$，所以$t=\alpha r+\beta p_i+(1-\alpha-\beta)p_0$，是三角形$p_0$,r,p_i的凸組合。刪t不改變加入p_i後的凸包；等號恰為在r,p_i邊上，仍非極端頂點。這是可以反覆pop的證據，不是只以「凸包不能右轉」當充分理由。

Stack 不變量與完整正確性

處理p_i前，stack恰為已處理點凸包的逆時針頂點。初始一／兩點符合退化凸包；首次非共線點形成三角形。

新p_i的極角比先前點大，$p_0\to$p_i是新集合的一條支持邊：所有已處理點都在此有向邊的右側或線上。從舊凸包末端向前看，對p_i不可見的舊邊不變；可見的末端連續鏈被新邊取代。每個pop按上一頁證明是新三角形內的非極端點，不改CH；直到末端(r,t,p_i)為左轉，t→p_i成為另一條切線，留下的舊凸邊與p_i→p_0圍成凸多邊形。

此多邊形包含舊凸包及新點，且頂點全來自輸入，故同時包含CH且包含於CH，兩者相等。歸納到最後即得到CH(Q)。每點push一次、pop至多一次，掃描O(n)；初始排序O(n log n)主導，空間O(n)。

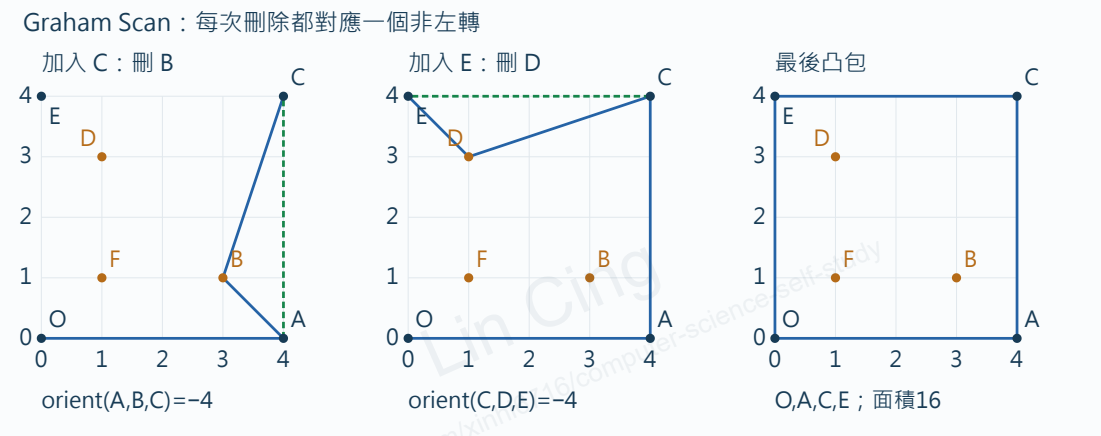
凸包的兩次 Pop 都能用數值核對

p_0=O(0,0)、A(4,0)、B(3,1)、C(4,4)、D(1,3)、E(0,4)，極角次序O,A,B,C,D,E。另有F(1,1)，與C同射線，先刪F。

新點	判定	Pop後／加入後stack
O,A,B	orient(O,A,B)=4	O,A,B
C	orient(A,B,C)=−4，刪B；orient(O,A,C)=16	O,A,C
D	orient(A,C,D)=12	O,A,C,D
E	orient(C,D,E)=−4，刪D；orient(A,C,E)=16	O,A,C,E

B=O/4+A/2+C/4，D=O/4+C/4+E/2，正是被其他點的凸組合包含。最後正方形頂點O,A,C,E，面積16。

凸包的兩次 Pop 都能用數值核對 (續)



Jarvis March：每步找到一條支持邊

從字典序最小點p開始。選任一q≠p，掃描所有r≠p：若orient(p,q,r)<0就令q=r；共線且r更遠也令q=r。掃完後所有點都在p→q左側或線上，q就是逆時針下一個極端頂點。輸出p，再令p=q，回到起點時停止。

為何一次掃描足夠？p是凸包頂點，所有其餘點方向落在一個跨度至多π的支持半平面；在這個半平面內，orientation決定方向的線性次序。每次更新保留已掃點中最靠順時針的射線；同射線保留最遠端。掃完便是一條支持邊，其他點位於同一側。從極端點走到下一極端點，直到繞回，不會走入內點。

每步檢查n−1點，共h條邊，時間O(nh)，空間O(h)含輸出；h≤n，所以最壞O(n^2)。同一正方形例每步選A,C,E,O，h=4。它在h很小時有利，但不能無條件稱比Graham快。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

33.4 Finding the closest pair of points

給n≥2個帶唯一ID的點，求歐氏距離最小的一對。重複座標允許，答案可為0；若n<2回報「無點對」。比較距離平方即可避免為每個候選求平方根。

先依(x,y,ID)排X、依(y,x,ID)排Y。每次按X的前⌈n/2⌉個ID分成L，其餘R；分割線ℓ選在兩組x範圍之間（同x可取該x）。沿Y一次穩定分流成Y_L,Y_R，必用ID membership，不能只用x<ℓ，否則所有點同x時會分不平衡。

遞迴求兩側最佳距離δ_L、δ_R，令$\delta=\min$。若$\delta=0$可直接回傳。任何更近的跨組點對，兩點距分割線都<δ，且縱向距離也<δ，所以只需檢查寬2δ的strip內候選。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最近點對的分治偽碼

```
closest(X,Y):
    if len(X)<=3: return brute-force closest pair
    split X by rank into XL,XR; select dividing x=ell
    split Y by IDs into YL,YR, preserving y order
    best=min(closest(XL,YL), closest(XR,YR))
    delta=distance(best)
    if delta==0: return best
    Z=[p in Y with (px-ell)^2 < delta^2]
    for i=0..len(Z)-1:
        for j=i+1..min(i+7,len(Z)-1):
            if (Z[j].y-Z[i].y)^2 >= distance(best)^2: break
            if distance2(Z[i],Z[j]) < distance2(best): best=(Z[i],Z[j])
    return best
```

Strip範圍使用兩側答案的固定δ；比較時best可變小，但每點最多看後7點。只求一對時，與已知最小值相等的候選不必再更新，因此可以使用嚴格strip與≥break。若要求輸出所有最小點對，需要另設相等距離與輸出規則。

為何只需後七點：分組的裝箱證明

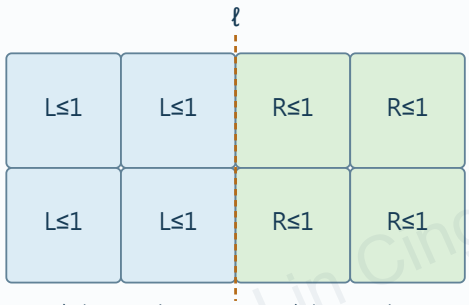
固定strip內一點p，只可能改善答案的後續點q滿足0≤q_y−p_y<δ，故落在寬2δ、高δ的矩形。左組點位於左半δ×δ，右組點位於右半δ×δ；同一組任兩點距離至少δ，這是遞迴最小值的保證。

將每個半正方形分為四個δ/2×δ/2小格，以固定半開邊界分配每點。單格直徑δ/√2<δ，所以同組單格至多一點，每組至多4點，合計至多8點。扣除p本身，其餘至多7點，q在依y排序的後7個位置內。

落在分割線上的點按ID屬於其中一組，各用自己的半正方形計帳；不能把兩組都在分割線的點誤當成同組而排除。若跨組有重複座標，距離0也會由此候選比較找到；若同組已重複，遞迴早已回0。

為何只需後七點：分組的裝箱證明 （續）

寬 2δ、高 δ：依左右組別，各自最多四點



同組任兩點距離 ≥δ

小格邊長 δ/2

小格直徑 δ/√2<δ

含 p 共至多8點

所以只看後7點

分割線上的點仍按 ID 歸組；同一幾何位置可各有一個 L、R 點。

最近點對的完整合併數例

X依序為A(0,0),B(1,4),C(2,1),D(3,7),E(4,1),F(5,5),G(6,0),H(7,8)。左組A–D，右組E–H，ℓ=7/2。

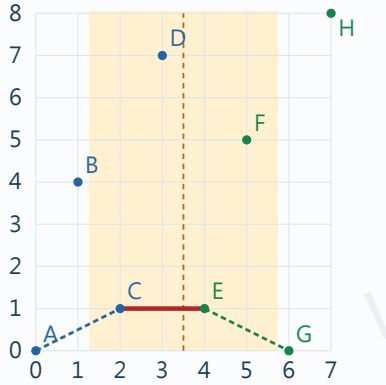
左組最佳A,C，距離平方5；右組最佳E,G，也為5。δ=√5，strip包含C,E,F,D，按y順序正是C(2,1)、E(4,1)、F(5,5)、D(3,7)。

目前點	後續檢查	結果
C	E：平方距離4；下一F的y差4	best改C,E；4≥2，停止此列
E	F的y差4	不可能改善2，停止
F	D的y差2	相等下界也不會改善，停止
D	無後點	結束

跨組C,E的距離2改善兩個子問題的√5。圖中strip按固定√5畫，不隨best縮窄。

最近點對的完整合併數例 （續）

兩側答案 √5，跨組 C、E 的距離 2 更小



ℓ=7/2，固定 δ=√5

左組 A–D，右組 E–H

Strip 依 y 排序：C,E,F,D

$|C-E|^2=(4-2)^2+0^2=4$

兩側最短距離平方都是5

檢查跨組後 $best^2=4$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

複雜度來自穩定分流，不是每層重排

每次分割X用rank，Y以ID穩定分流，建立strip與每點至多7次比較，非遞迴工作O(n)。故

$T(n)=T(\lceil n/2\rceil)+T(\lfloor n/2\rfloor)+O(n)=O(n \log n)$。

最初兩次排序也O(n log n)。若每個子問題重排Y，合併工作變O(n log n)，總界便$O(n \log^2 n)$。ID membership可預先記每點在本層的左右標記；穩定分流不能對每個Y點線性搜尋整個L。

按深度先遞迴、子問題返回後釋放暫存，單條遞迴鏈的陣列總長n+n/2+⋯=O(n)，加當前兄弟與回傳資訊仍O(n)工作空間，stack O(log n)。若刻意保存整棵遞迴樹的所有陣列，會用O(n log n)，那不是這個線性空間實作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 33。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34 NP 完備性

Part VII Selected Topics

本章定位

NP完備性以位元編碼、量詞及可驗證證書界定難度；本章逐步建立核心歸約的構造、雙向語義與多項式成本，讓困難性證明可重建。

前置知識

決策問題、圖靈機、約化與布林電路。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 定義 P、NP、NP-hard、NP-complete。
- 建立 polynomial-time many-one reduction。
- 用 SAT、clique、vertex cover、Hamiltonian cycle 做證明。
- 34.1 Polynomial time
- 34.2 Polynomial-time verification
- 34.3 NP-completeness and reducibility
- 34.4 NP-completeness proofs
- 34.5 NP-complete problems

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.1 Polynomial time

決策問題、編碼與輸入長度

定義與前提

abstract problem 可把 instance 集合映到 answers。決策問題的答案只有 yes 與 no，因此可用語言 `L={x: answer(x)=yes}` 表示。

同一個數學物件可能有不同 encoding。例如整數 t 用 binary 表示約需 `lg(t+1)` 位，用 unary 表示需 t 個符號。多項式時間必須先指定 encoding，因為演算法的界限以實際輸入位元數衡量。

$$L \in \mathrm{P} \iff \exists \text{ deterministic TM } M,\ \exists \text{ polynomial } p : \forall x,\ M(x) \text{ halts in } p(|x|) \text{ and } (M(x) = 1 \iff x \in L).$$

`forall x` 不能寫成「存在某個 x」。P 的演算法必須對每個合格式處理，且錯格式也須在 polynomial time 拒絕或依規格處理。

P 的封閉性與 poly-time 組合

操作與推導

若程序 A 在 `p_A(n)` 時間產生可行 encoding，程序 B 在 `p_B(n)` 時間解第二個問題，先後執行成本為 `p_A(n)+p_B(p_A(n))`。多項式代入多項式仍為多項式。

這提供兩個常用結論。

1. 若 A 可在 polynomial time 直接求解，而 B 可把 instance 直接解算轉成 A，則 B 也可 polynomial-time 求解。
2. 多個 polynomial-time 轉換可串接，因為錯誤方向與長度爆炸都被 poly 界限控制。

但「演算法在某部真實電腦跑得快」不是成員證明。`O(n^100)` 仍屬 P，而常數很低的 exponential algorithm 可能在小 n 更實用。計算類別與工程效能是兩個問題。

bit input、數值參數與 pseudopolynomial

可追蹤數例

令 $\mathrm{bitlen}(t)$ 為 t 的 binary 表示位數。若演算法成本 $O(nt)$，而 $t < 2^{\mathrm{bitlen}(t)}$，成本相對於輸入位元數可能是指數。

考慮 subset-sum：輸入 `n` 個整數與目標 B，DP 表有 `n(B+1)` 格，因此 `O(nB)`。B binary 編碼時，`nB` 不是輸入長度的多項式；這種以數值上界為參數、而非以位元長度為參數的演算法稱為 pseudopolynomial algorithm。

可追蹤族。 令 $n = m$、$t = B = 2^m$，並令 $a_i = 2^{m-1} + i$、$i = 1, \dots, m$、$m \geq 2$，使每個 a_i 都有 $\Theta(m)$ bits。目標有 $m + 1$ bits，總輸入長度 $N = \Theta(m^2)$，但 DP 成本為

$$\Theta(m2^m) = 2^{\Theta(m)} = 2^{\Theta(\sqrt{N})}.$$

若它是 input-bit polynomial，對某個 c 與常數 C 最終應有 $m2^m \leq C(m^2)^c$；然而 $m2^m / m^{2c} \to \infty$，矛盾。

pseudopolynomial 不表示演算法無用。它表示「在 B 小或 unary encoding 下 polynomial」，不能因此宣稱 subset-sum 在 binary 標準編碼下屬於 P。

34.2 Polynomial-time verification

NP 的完整量詞

NP 類常被說成「答案可被快速檢查」。完整定義需要 certificate、長度上界與 deterministic verifier。

$$L \in \mathrm{NP} \iff \exists V,\ \exists\ \text{polynomials}\ p, q : \forall x,\quad x \in L \iff \exists y\ \big(|y| \leq q(|x|) \wedge V(x, y) = 1\big),$$

且 V 是 deterministic algorithm，對所有 x、y 在 `p(|x|+|y|)` 時間內終止。y 稱為 certificate 或 witness。

量詞順序不能交換。對 yes instance，存在 witness；對 no instance，任意 witness 都不能讓 verifier 正確接受。所謂「猜 witness」的 NTM 說法，是把這個存在量詞轉成 nondeterministic computation path。

verifier 的 soundness 與 completeness

何時失效

針對某個關係 `R(x,y)`，可定義 verifier 的兩種方向：

- completeness：若 `x in L`，存在 polynomial-length y 使 `R(x,y)=1`。
- soundness：若存在 y 使 `R(x,y)=1`，則 `x in L`。

合併兩者才是 `x in L iff exists y R(x,y)`。只有 completeness 可能接受錯誤 instance；只有 soundness 可能漏掉某些 yes instance。

以 Hamiltonian cycle 為例，certificate 是 n 個頂點的一條序列。Verifier 必須檢查：序列是所有頂點的一次排列、每個相鄰 pair 都是原圖邊、末點連回首點。四項全部成立才接受。一個只檢查「共有 n 個頂點」的 verifier 不 sound。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P 包含於 NP 的證明

假設 `L in P`，存在 deterministic polynomial-time M 判定 L。定義 `V(x,y)` 忽略 y，直接執行 M(x) 並輸出 M 的答案。取空字串 `epsilon` 為 certificate，長度 0。

若 `x in L`，M(x)=1，因此存在 short witness epsilon 使 V=1。若 x notin L，因 M 對所有 y 都回 0，不存在 witness 使 V 接受。這同時得到 soundness 與 completeness。

因此 `P subseteq NP`。是否有逆包含 `NP subseteq P` 仍是 open problem；教材不能把「尚未找到 polynomial algorithm」誤寫成「已證明不存在」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.3 NP-completeness and reducibility

many-one reduction 的完整定義

`A <=_p B` 表示存在函數 f，滿足：

1. f 對 A 的每個合格式 input 都可在 `poly(|x|)` 時間計算。
2. f 的輸出是 B 的合格式。
3. 對所有 x，`x in A iff f(x) in B`。

方向是「把 source A 的 instance 轉成 target B 的 instance」。若想證明新問題 B 困難，可以把已知困難 A 轉成 B；不能把 B 轉成已知容易問題就下結論。

此 reduction 是 many-one，因為每個 x 只產生一個 B instance。它不允許根據求解器答案再回頭修改 x，也不允許呼叫 B solver 多次，除非那些呼叫本身可被包進合法 polynomial-time 轉換。

雙向語義：completeness 與 soundness

`iff` 要拆成兩個檢查：

- completeness direction：`x in A implies f(x) in B`。A 的每個 yes instance 都要構造出 B 的 yes instance。
- soundness direction：`f(x) in B implies x in A`。任何由 f 產生的 B yes instance 必須能解回 A 的 yes instance。

名稱在文獻中可能因視角互換；解題時應直接寫方向，避免只寫「sound/complete」造成歧義。

以 CLIQUE 為例，轉成的圖若存在 k-clique，具體 clique 必須提供足以還原一組 satisfying assignment 的資訊。若圖的 size、k 或邊的 gadget 允許不存在的 assignment，soundness 就壞掉。

transitivity 定理與完整證明

定理。 若 `A <=_p B` 且 `B <=_p C`，則 `A <=_p C`。

證明。 令 f 是 A 到 B 的 reduction，g 是 B 到 C 的 reduction。定義 `h(x)=g(f(x))`。

若 `x in A`，由 f 的 iff 得 `f(x) in B`；再由 g 的 iff 得 `g(f(x)) in C`。反向若 `g(f(x)) in C`，由 g 得 `f(x) in B`，再由 f 得 `x in A`。因此 `x in A iff h(x) in C`。

令 f 在 `p_f(n)` 時間終止，g 在 `p_g(m)` 時間終止。因 `|f(x)| <= p_f(|x|)`，算 `h(x)` 的時間至多 `p_f(n)+p_g(p_f(n))`。多項式合成仍為多項式。故 A <=_p C。

NP-hard 與 NP-complete

語言 L 是 NP-hard，若對每個 `A in NP` 都有 `A <=_p L`。語言 L 是 NP-complete，若：

1. `L in NP`；
2. L 是 NP-hard。

若已知 C 是 NP-complete，且 `L in NP`，要證明 L 是 NP-complete，只需證 `C <=_p L`。任取 `A in NP`，因 C NP-hard 得 `A <=_p C`；再用已知 `C <=_p L` 與 transitivity 得 `A <=_p L`。

缺少 `L in NP` 只能得到 NP-hard，不能得到 NP-complete。缺少 `C <=_p L` 則沒有把已知困難性傳到 L。

第一個 NP-complete 問題的構造模型

CIRCUIT-SAT 輸入是有限、有向無環的 Boolean circuit，gate 為 fan-in 至多二的 AND、OR 與一輸入 NOT；問是否有一組輸入使指定輸出為 1。certificate 只需原始輸入 bits，依拓撲序計算每個 gate、檢查輸出，時間與 gate/wire 數成線性，故在 NP。

要證 NP-hard，任取 L in NP，固定其 deterministic verifier V、certificate 長度界 q(n) 與時間界。給 x，n=|x|，電路 C_x 的自由輸入必須是 witness y，而不是把一個固定 computation 的接受狀態當作未知。

可將witness規格化為固定長度q(n)+b(n)，其中長度欄b(n)=1+⌈$\log_2(q(n)+1)$⌉，至少1 bit。前b(n)位編碼實際長度ℓ，後q(n)位放witness並補零；新verifier先檢查$0 \le \ell \le q(n)$，再讀前ℓ個witness bits並模擬V。q(n)=0時仍有一位長度欄，只允許$\ell=0$。這不改變「存在可接受witness」，也不會把不同長度默認成相同字串。

由局部 transition 逐 gate 展開計算

固定一部模擬此verifier的單帶deterministic TM，讓它於接受或拒絕後停留在同一終止狀態。多帶轉單帶至多帶來多項式時間損失。令T(n)為包含witness長度後的多項式步數界，初始帶長N=O(n+q(n)+1)，包含前頁至少一位的長度欄，q(n)=0也有定義。

一步最多移動一格，所以只需表示 O(N+T) 個 tape cells，兩端額外保留 T+1 空白格，頭不會碰到外界。每格標籤包含 tape symbol，以及「此格是否有頭、若有則是哪個 state」。機器字母表與狀態集固定，故每格用固定 b bits。

時間 t+1 的第 j 格標籤，只取決於時間 t 的 j-1、j、j+1 三格標籤。這是固定 3b 輸入的 Boolean 函數。逐個輸出 bit，用真值表寫成若干 minterms 的 OR，每個 minterm 為輸入 bit 或 NOT bit 的 AND，再把多輸入 AND/OR 拆成二輸入 gates。因 b 與機器固定，每個局部更新只需 O(1) 個 gates。

逐層接線、規模與 iff 證明

初始層將 x、空白格、初始頭位置與 state 接成常數，只有規格化 witness bits 自由。對 t=0,...,T-1，每個 tape cell 複製上述固定局部電路，將其輸出接到下一層相同 cell。最後用 OR tree 檢查第 T 層是否有接受 state 標記。

總 gate 數為 O(T(N+T))，wire 數同階，gate 索引使用 O(log(T(N+T))) bits；列舉時間層、cell 與固定真值表即可在多項式時間輸出電路，沒有隱藏指數大小的硬體。

對每組 y，依 t 歸納：第零層是合法初始 configuration；局部 transition 產生恰好下一 configuration。因此所有層都是 V(x,y) 的真實執行，無效局部標籤永遠不會由合法初始層出現。接受狀態持續不變，故 C_x(y)=1 iff V(x,y) 接受。

若 x in L，存在接受 y，於是 C_x 可滿足；若 C_x 可滿足，其自由輸入 y 使 V 接受，於是 x in L。每個 NP 語言都如此多項式轉入 CIRCUIT-SAT，加上 NP membership，完成第一個 NP-complete 證明。

34.4 NP-completeness proofs

CIRCUIT-SAT 到 SAT 的局部 gate 約束

為每條 wire（可共享同一 gate output）設一變數，並把輸出 wire z_out 本身作為 unit clause，要求它為真。每個 gate 加入輸出等於 gate 函數的條件：

Gate 關係	等價 CNF clauses
z = u AND v	(¬z∨u) AND (¬z∨v) AND (z∨¬u∨¬v)
z = u OR v	(z∨¬u) AND (z∨¬v) AND (¬z∨u∨v)
z = NOT u	(z∨u) AND (¬z∨¬u)

例如 AND 的前兩 clauses 強迫 z=true 時 u,v 皆真；最後一 clause 強迫 u=v=true 時 z=true。OR 與 NOT 可同樣分真假兩向直接檢查。常數 wires 用 unit clause 固定。

電路接受輸入可延伸為所有 wire 的真值，滿足每個局部 clause 與 output clause。反向滿足公式時，局部等價式沿拓撲序迫使每個 wire 等於實際計算值，output=true，所以原電路可滿足。每 gate 至多三 clauses，總長 O(gates+wires)，不把共享子電路反覆展開成指數長樹狀公式。

SAT 到 3-CNF 的 parse-tree gadget

任意公式先解析成 NOT、一輸入或 AND/OR 二輸入樹；例如 implication 可用 ¬a∨b 改寫，其他固定 arity connective 亦用固定大小 truth-table gadget。為每個內節點設新變數，使用前頁 gate clauses，根變數設真。若同一節點輸入重複，先在所得 clauses 去除重複 literal；含 l 與 ¬l 的 clause 是 tautology，可整個刪掉。

此時 clauses 長度至多三。若要求每 clause 恰含三個相異變數，逐 clause 使用新的 padding variables：

- 兩 literal (a∨b) 改為 (a∨b∨p) AND (a∨b∨¬p)。a∨b 真則任意 p 都可；若假，兩式分別強迫 p 與 ¬p，矛盾。
- 一 literal a 改為四式 (a∨p∨q)、(a∨p∨¬q)、(a∨¬p∨q)、(a∨¬p∨¬q) 的 AND。a 真則皆真；a 假時，任何 p,q 指派都令其中一 clause 為假。
- 空 clause 表示不滿足，直接輸出上述 padding 過的 z AND ¬z；空 AND 表示恆真，輸出固定可滿足的三變數 clause。

原 assignment 可用每個 parse node 的真值延伸；反向沿樹恢復所有節點語義。padding 保留每 clause 的真假要求，因此整體只保留 equisatisfiability，而非把加入新變數後的公式當作原變數的逐字同一函數。每節點常數 clauses，每 clause 至多擴成四個，所以大小與建構時間皆多項式。

長 clause 拆分與可追蹤 gate 例

若已得到長 clause $l_1 \vee \cdots \vee l_r$，$r > 3$，也可用 chain：

$$(l_1 \vee l_2 \vee y_1) \wedge (\neg y_1 \vee l_3 \vee y_2) \\ \wedge \cdots \wedge (\neg y_{r-3} \vee l_{r-1} \vee l_r).$$

若所有 l_i 都假，首式逼 $y_1 = \text{true}$，中間逐次逼下一個 $y_j = \text{true}$，末式卻要求最後 $y_{r-3} = \text{false}$，矛盾。若至少一個 l_i 真，令 y_j 表示前 $j+1$ 個 literals 是否全假，即可逐式滿足。此拆分用 $r-3$ 新變數、$r-2$ clauses；不能直接用 distributivity 把任意公式轉 CNF，那可能產生指數長度。

例 F=(a∨b) AND ¬a，設 u=a∨b、v=¬a、z=u AND v，給三組 gate clauses 並加 z。a=false,b=true 時 u=v=z=true，滿足所有約束。若 a=true，NOT gadget 強制 v=false，而 z 與 AND gadget要求 v=true，故這類指派不能假造接受。這同時示範 witness 延伸與反解。

原書 3-CNF 定義為三個相異 literals；本教材的主要 reduction 進一步可規格化為三個相異變數。出現重複 literal 的輸入仍可先依上述去重、padding 規則處理，不影響後續 CLIQUE occurrence 構造。

3-CNF-SAT <=_p CLIQUE 的構造

令 F 為 3-CNF formula，記 `|F|` 為其實際 binary encoding 長度。若 F 有 k 個 clauses，為每個 clause 中的每個 literal occurrence 各建一個 vertex，並令 CLIQUE 的參數也為 k。非退化情形取 `k>=1`；空 AND formula 沒有 clauses，恆為真，另行映射到 `(空圖,0)`。

邊的規則正好兩條：

1. 同一 clause 內的 vertices 彼此不相連。
2. 不同 clauses 的兩個 vertices 相連，若且唯若它們不是互補 literals。

每個 clause 恰有三個 occurrence，因此 vertices 數是 `3k`。先把實際出現的變數名稱編為n個ID（n≤3k），逐一檢查vertex pairs需$O(k^2)$次O(1) word比較。若逐bit計費，ID比較及edge-list端點各需O(log(n+k)) bits，建圖可用$O(|F| \log|F|+k^2 \log(n+k))$的多項式bit上界；若輸出adjacency matrix則有$\Theta(k^2)$個bits。不能把任意長變數名稱的字串比較無條件當作O(1)。

可追蹤 3-CNF-SAT 例子

令

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

取 assignment `x1=false, x2=false, x3=true`。三個 clauses 都為 true。

選 literal occurrences：第一 clause 的 `x3`、第二 clause 的 `¬x1`、第三 clause 的 `x3`。它們來自不同 clause，因此建立跨群 edges；彼此不是 complement，所以三點兩兩相連，形成 size 3 clique。

這個 assignment 並不是唯一 witness。若改挑第一 clause 的 x1 與第二 clause 的 ¬x1，這兩個 occurrence 會因互補而缺邊，不能一起組 clique；在目前 assignment 下 x1 也為假，不應被選作真 literal。這正好說明 graph 不會把矛盾 literals 同時當成一致 witness。

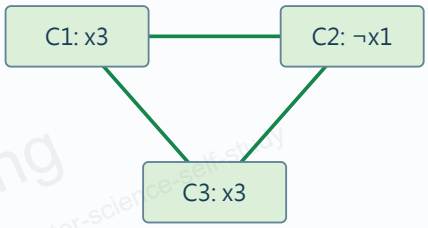
可追蹤 3-CNF-SAT 例子 （續）

不同 clause 且非互補，才連邊

C1 / C2	¬x1	x2	¬x3
x1	×	✓	✓
¬x2	✓	×	✓
x3	✓	✓	×

✓ 相連；× 互補而缺邊

選到的三個 occurrence 彼此一致



x1=0，x2=0，x3=1：每群一個真 literal

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3-CNF-SAT 到 CLIQUE 的 iff 證明

Completeness。 若 F satisfiable，固定一組 satisfying assignment。每個 clause 至少有一個 true literal occurrence，選其中一個。因 assignment 一致，兩個被選 occurrences 不會互為 complement，所以對應 vertices 之間都有邊。k 個 clauses 得到 k-clique。

Soundness。 若 G_F 有 k-clique，同一 clause 的 vertices 沒有邊，所以 clique 必須在每個 clause 中恰好選一個 occurrence。任意兩個被選 literals 都有邊，代表它們不互為 complement。因此同一個 variable 不會同時被要求為 true 與 false。令每個被選 variable 取得使其 literal 為真的值；未選 variables 任意指定；這些要求彼此一致，且每 clause 有一個 true literal，故 F satisfiable。

兩方向合併即 `F satisfiable iff G_F has k-clique`。CLIQUE 也有 polynomial certificate（k 個 vertices）與 checker，所以 CLIQUE 是 NP-complete。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CLIQUE、INDEPENDENT-SET 與 VERTEX-COVER

independent set S 是圖中任兩點都沒有 edge 的集合。vertex cover C 是每條 edge 至少一端在 C 的集合。

對 n 點、無 self-loop 的無向圖 G=(V,E)，令 `Gbar=(V, {uv: u!=v and uv notin E})`。同一 pair 在 G 有 edge 若且唯若在 Gbar 沒有 edge。因此：

$$C \text{ is a clique in } G \iff C \text{ is an independent set in } \overline{G}.$$

若 S 是 independent set，則 V\S 是 vertex cover。假設有 Gbar 的 edge 未被 V\S 覆蓋，它的兩端都在 S，違反 S independent。反向若 V\S 是 vertex cover，任何 Gbar edge 不可能兩端都在 S；所以 S independent，也是在 G 的 clique。

令 `k in {0,...,n}`。把上面兩步串起，得到

`CLIQUE(G,k)` iff `VERTEX-COVER(Gbar,n-k)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CLIQUE、INDEPENDENT-SET 與 VERTEX-COVER （續）

`(G,k) -> (Gbar,n-k)` 與反方向的 `(H,l) -> (Hbar,n-l)` 都以 `O(n^2)` 次配對檢查建立補圖。這裡CLIQUE可說「至少k點」或「恰k點」，因較大clique任取k點仍是clique；VC採「至多n−k點」，較小cover可補點至恰n−k點，所以同一iff成立。這需0≤k≤n；範圍外門檻先直接判定並映射至固定yes/no實例，不能產生負大小certificate。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CLIQUE 與 VERTEX-COVER 的可追蹤例

令 `G` 有頂點 {1,2,3}，edges {12,23}，取 k=2。{1,2} 在 G 是 clique。

補圖 `Gbar` 只有 edge {13}。`{1,2}` 在 Gbar 是 independent set，因為 1 與 2 沒有 edge。其補集 `{3}` 是 Gbar 的 vertex cover，大小為 `3-2=1`，且它覆蓋唯一 edge 13。

轉換是 `VC(Gbar, n-k)`。所以 `CLIQUE(G,k) iff VERTEX-COVER(Gbar,n-k)`。這個等價直接給出雙向約化：complement 本身就是 polynomial-time、可逆的 encoding transformation。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.5 NP-complete problems

VERTEX-COVER 到 HAMILTONIAN-CYCLE 的端口 gadget

HC certificate 是全部頂點的排列，逐邊與閉合檢查給 NP membership。下面完整建立 NP-hardness，不需要平面 crossing gadget；圖中交叉畫線不代表交叉處是頂點。

對原圖每邊 {u,v} 建十二個新點 a1,...,a6、b1,...,b6，其中 ai=[u,v,i]、bi=[v,u,i]。內邊共十四條：

- 兩條鏈 a1-a2-a3-a4-a5-a6 與 b1-b2-b3-b4-b5-b6，共十條。
- 四條交叉連結 a1-b3、a3-b1、a4-b6、a6-b4。

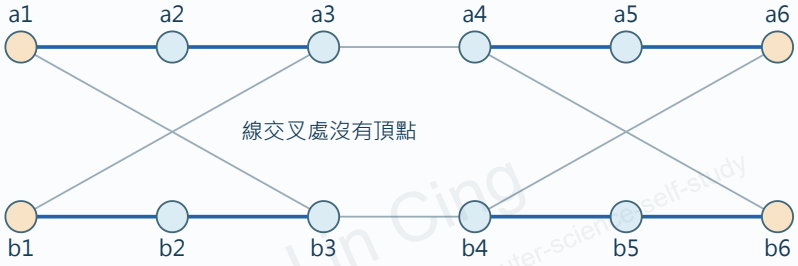
只有四個端口 a1,a6,b1,b6 可接 gadget 外部。其他八點在最後 HC 中必有兩條內邊。a2,b2,a5,b5 在 gadget 本身 degree=2，故它們的八條相鄰鏈邊被迫選入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34.5 NP-complete problems （續）

HC gadget：只有 a1、a6、b1、b6 能接外部

a_i=[u,v,i]；b_i=[v,u,i]



藍色8邊由 a2、a5、b2、b5 的 degree=2 強迫選入；灰色6邊仍待決定。

兩條中邊 a3–a4、b3–b4 決定其餘4條cross edges；橘色為4個端口。

gadget 只有三種合法的覆蓋方式

餘下選擇由中間兩邊 a3-a4、b3-b4 決定。每個內部點仍需 degree=2，故這兩邊一旦選或不選，四條交叉連結的使用亦被迫確定。

中邊選擇	對全部十二點的路徑覆蓋
兩條都選	a1-a2-a3-a4-a5-a6，以及 b1-b2-b3-b4-b5-b6，兩條 disjoint paths
只選 b3-b4	a1-a2-a3-b1-b2-b3-b4-b5-b6-a4-a5-a6，一條 a1 到 a6 的完整 path
只選 a3-a4	對稱的一條 b1 到 b6 的完整 path
兩條都不選	上下各形成一個封閉六點 cycle，與外部無法連成單一 HC，因此禁止

這四種窮盡所有可行內度配置，故合法 HC 只能使用前三種。尤其不能從 a 端口進來卻沿 b 的外部鏈離開；也不能讓十二點完全無人覆蓋。此局部事實正好表示原邊至少有一個端點被選。

中邊如何強迫端口的外部使用

令x表示是否選a3–a4，y表示是否選b3–b4，值為0或1。內點a3已有a2–a3，所以a3–b1必取1–x；a4同理使a4–b6取1–x。b3,b4則使a1–b3與a6–b4取1–y。

因此a1,a6的內部度都是2–y，還需要的外部度為y；b1,b6的外部度為x。端口不可能一邊連外、一邊封閉：同一原頂點側的兩端總是同時接外或同時不接外。

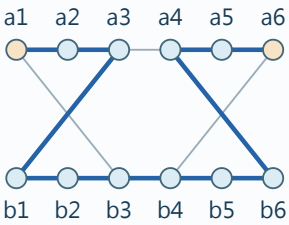
x=y=0時，下列兩個六點cycle已各自用滿度2：a1–a2–a3–b1–b2–b3–a1，以及a4–a5–a6–b4–b5–b6–a4。它們不能接到selector，故不可能是整張圖的單一HC。其餘三種配置的全部路徑如下圖，逐點可核對沒有漏點或換側離開。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

中邊如何強迫端口的外部使用 （續）

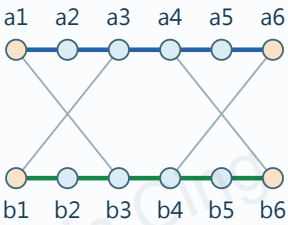
三種合法模式都不會跨原頂點身分離開

只走u側：12點



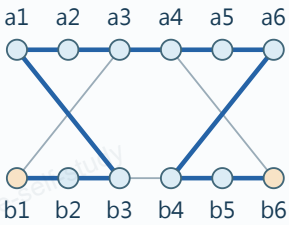
x=0 · y=1

u、v各走6點



x=1 · y=1

只走v側：12點



x=1 · y=0

灰邊未選；藍 / 綠是各條路徑；橘色端口各需一條外邊補到 degree=2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

串接原頂點的 incident gadgets 與 selectors

VERTEX-COVER 輸入採「大小至多 k」。先移除孤點，不影響最小 cover。k<0 或 k=0 且有邊，輸出固定 no HC instance；無邊或 k 大於等於剩餘頂點數，輸出固定 yes triangle。以下可假設無孤點且 1<=k<|V|；一個較小 cover 可以補入其他非孤點成恰 k 點。

對每個原頂點 u，任排其鄰居 u(1),...,u(d)。把各 gadget 的 u 側依序串接：加入 [u,u(i),6]-[u,u(i+1),1]，i=1,...,d-1。這些連結使一條外部路徑一旦選 u，便沿它全部 incident gadgets 走到末端。

另建 k 個 selector s1,...,sk。每個 selector 都接到每個 u 鏈的首端 [u,u(1),1] 與末端 [u,u(d),6]，沒有其他外邊。圖共有 12|E|+k 點；十四內邊乘 |E|，串鏈至多 2|E|，selector 連邊 2k|V|，全為多項式規模。原圖有向無向不可混淆，此構造與目標皆為無向圖。

VC 到 HC 的完整 iff

若 C={u1,...,uk} 是 cover，將每個被選 u 的 incident-gadget 鏈作一條 cover path。原邊若只有 u 被選，使用從 u 側進出的十二點完整 path；若兩端都選，使用兩條各六點 path。因每條原邊至少一端被選，每個 gadget 的十二點恰好各被訪問一次。依序用 s1、u1 鏈、s2、u2 鏈、...、sk、uk 鏈、s1 串成一個 cycle，亦各訪 selector 一次，故為 HC。

反向若新圖有 HC，移去 k selectors 後，cycle 被切成 k 條 disjoint cover paths。局部 gadget 的三模式禁止切換端點身分；每條 path 必沿同一原頂點 u 的 incident 鏈從頭到尾。不同 paths 不能選同一 u，否則會共用其鏈端或 gadget 點。取這 k 個 u 為 C。每個 gadget 的點都被 HC 訪到，而三種模式至少有 u 或 v 的 cover path 經過，所以每條原邊 {u,v} 至少一端在 C。得到大小 k 的 vertex cover。

因此 VC(G,k) iff HC(G')；構造、反解與規模皆多項式。固定 yes/no 例處理前處理的退化情況，不需用不存在的空鏈湊 selectors。

VC gadget 的小例

原圖為 path u-v-w、k=1，可選 cover {v}。建兩個十二點 gadgets W_uv、W_vw；把 v 側第一個 gadget 的端口 6 接第二個 gadget 的端口 1，再用唯一 selector 接整條 v 鏈的首尾。

兩條原邊只有端點 v 被選，所以每個 gadget 都由 v 側完整走過十二點。selector 與兩個 gadgets 共二十五點形成 HC。反之若選 u 的鏈，W_vw 沒有任何被選端點，無法覆蓋，因此不會假造 cover {u}。

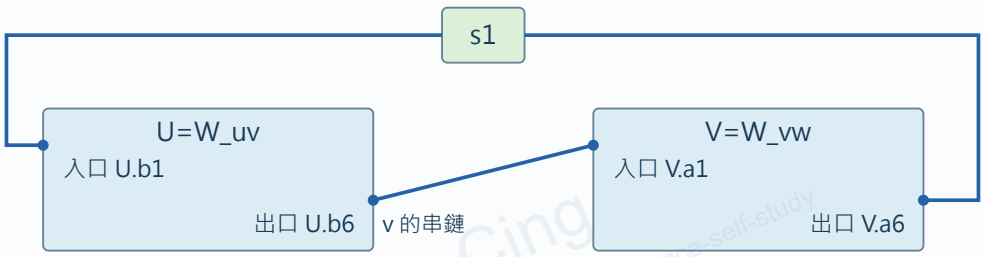
完整二十五點cycle可由以下兩條十二點路徑接成。記U=W_uv、V=W_vw；U的b側代表v，V的a側代表v：

- U：b1,b2,b3,a1,a2,a3,a4,a5,a6,b4,b5,b6。
- V：a1,a2,a3,b1,b2,b3,b4,b5,b6,a4,a5,a6。

由s1→U.b1開始，走完U後經串接邊U.b6→V.a1，再走完V，最後V.a6→s1閉合。U與V的同名點是不同頂點，故24個gadget點加s1恰25點，每點只出現一次。每個相鄰pair都能在十四內邊、串鏈或selector邊中逐項查得。

VC gadget 的小例 （續）

原圖 u–v–w、cover={v}：唯一 selector 串成25點 cycle



各框內是已列出的完整12點路徑．U與V的同名點互不相同。

此圖顯示選定的HC；完整外邊依selector與incident-chain構造定義。

HAMILTONIAN cycle <=_p TSP decision

給 HC instance `G=(V,E)`，`n=|V|`。建立完全圖 `K_n`，邊權重：

`w(u,v)=1` 若 `{u,v} in E`，否則 `w(u,v)=2`。令門檻 `B=n`。

HC 到 tour。 G 的 Hamiltonian cycle 有 n 條原圖 edges，每條 weight 1，因此在 K_n 是 cost n 的 tour。

Tour 到 HC。 K_n 任一個 tour 恰有 n 條邊。若其 cost <= n，且每條 weight 至少 1，則每條邊都必須 weight 1。這些邊都在 G，tour 因而也是 G 的 Hamiltonian cycle。

構造是 `O(n^2)`。所以 `HC(G) iff TSP(K_n,w,n)`，且 TSP decision的 certificate 是一條 tour，檢查排列與總成本即可，故 TSP decision in NP。

TSP 實例追蹤

令G是三角形，edges={12,23,31}，n=3。完全圖的三條邊全部權重1，因此tour 1→2→3→1的成本3，門檻B=3。

門檻 B=3。此 tour cost 恰好等於 B，對應 G 的 Hamiltonian cycle。

若原圖刪去31，構造出的完全圖仍有31，但權重改為2。任何三點tour都含三條邊，成本1+1+2=4>B，所以不存在合格tour，與原圖沒有HC一致。對n<3的簡單無向圖，本章HC無解，直接映射至固定no TSP instance；上述每tour恰n邊的主要論證採n≥3。

optimization 版 TSP 要求真正最小總成本；decision 版只問是否存在 cost <= B。若 optimization algorithm 在 polynomial time 回傳最佳值，決策版可直接比較 B；決策版 NP-hard 因而支持 optimization 版也難以精確 polynomial-time 求解。

優化問題與決策問題

很多 NP-complete 問題有 optimization counterpart。例：TSP 最佳 tour、SET-COVER 最少集合數、MAX-CLIQUE 最大 clique size。

若優化版在多項式時間可精確解，直接把最佳值與門檻比較便能解決策版。另一方向，若已有正確的決策 oracle，可對整數門檻binary search求最佳值。決策版NP-hard且精確優化可解時，便能解全部NP語言；故除非P=NP，這些精確優化問題不會有多項式時間演算法。

binary search 有兩個假設：

- 成本或答案介於可由 input 多項式位元表示的上下界；
- decision procedure 對所有門檻都正確。

得到最佳值不等於直接得到 witness。若也要輸出構造，需額外做 reconstruction，例如逐步固定決策並重跑 decision oracle；不能把「知道最佳值」冒充成「已回傳最佳解」。

classification 的精確邊界

名稱	精確定義	已能推出什麼	不能直接推出什麼
NP-hard language L	每個 `A in NP` 都有 `A <=_p L`	若 `L in P`，則 `P=NP`	L 本身在 NP
NP-complete language L	`L in NP` 且 L NP-hard	它有一個 polynomial verifier，且可承接任一 NP source 的 reduction	一個 optimization 程式自動成為 language
CLIQUE decision	對 input `(G,k)` 回答是否存在 size-k clique	本節已證它在 NP 且 NP-hard	它輸出最大 clique
MAX-CLIQUE optimization	對 G 輸出最大 clique size（可再要求 witness）	若另有 polynomial algorithm，可由答案解 CLIQUE decision	optimization 的問題型別可直接套用 `NP-complete` 定義

`NP-hard` 是 reduction 的普遍性，`NP-complete` 額外要求 membership。decision 回答 yes/no；optimization 輸出最佳值，search 再輸出 witness。證明可互相解出時，要說明程式如何使用一個問題的完整 output，以及是否真的得到另一問題要求的答案。

SUBSET-SUM、bit encoding 與 DP

decision 版 SUBSET-SUM 問：給正整數集合 S 與 t，是否存在子集合元素和恰為 t。

Certificate 是一組 selected indices；verifier 檢查每個索引在範圍內、不重複，且所選整數和恰為 t。長度與檢查時間 polynomial，所以 SUBSET-SUM in NP。

DP 狀態 `dp[i,s]` 表示前 i 個 elements 可否達和 s。轉移為

`dp[i,s]=dp[i-1,s] or (a_i<=s and dp[i-1,s-a_i])`。

計算量 `O(n t)`。這是 pseudopolynomial，不是以 t 的 binary bit length 衡量的 polynomial。NP-hardness 的完整無進位數位表與兩向證明接在下頁；其巨大數值仍有多項式 bit 長，與此 DP 的 pseudopolynomial 界限一致。

3-CNF-SAT 到 SUBSET-SUM 的數位表

先把公式規格化為 n 個變數、k 個 clauses，每 clause 三個相異變數、無互補 tautology，每個變數至少出現一次；前面的去重/padding 處理可達成。恆真與恆假公式另輸出固定 yes/no 整數實例。

建立十進位 n+k 欄，前 n 欄對應變數，後 k 欄對應 clauses。這是緊湊建立整數的方式，最終仍以 binary 編碼交付。

- 每變數 xi 建兩數 T_i、F_i。兩者的第 i 個變數欄為 1，其他變數欄為 0。T_i 在包含正 literal xi 的 clause 欄為 1，F_i 在包含 ¬xi 的欄為 1，其餘為 0。
- 每 clause j 加 slack 數 S_j、S'_j，只在該 clause 欄分別放 1、2，其他欄為 0。
- target 的每個變數欄為 1，每個 clause 欄為 4。

總共 2n+2k 個正整數，每數 n+k 位十進位，故每數 binary 長度 O(n+k)，輸出總 bit 長 O((n+k)^2)，建構為多項式。若輸入要求集合而非 multiset，上述非退化假設保證兩個同變數數字不相同；不同變數的高位欄不同，slack 欄也不混同。

為何無進位，以及兩向解碼

任意子集合在每個變數欄至多加到 2；每個 clause 含三個 distinct literals，對應 T_i/F_i 數最多提供三個 1，slack 最多加 1+2=3，因此 clause 欄至多 6<10。任何子集合都不會進位，不只是正確 witness 才不進位。

若原式可滿足，每變數依真假選 T_i 或 F_i，使變數欄和全為 1。每 clause 的真 literals 數 r 為 1、2 或 3，可用 slack 子集合湊 4-r：r=1 選 1+2，r=2 選 2，r=3 選 1。所有欄符合 target，得到 subset-sum yes。

反向若數字子集合和為 target，無進位迫使每變數欄恰為 1，所以 T_i、F_i 恰選一個，定義一致 assignment。clause target 為 4，但兩 slack 合計至多 3，所以至少一個被選 T_i/F_i 在該欄有 1，對應原 clause 中一個真 literal。每 clause 都滿足。兩向與多項式規模一起完成 NP-hardness，再加索引 certificate 的 NP membership，SUBSET-SUM 為 NP-complete。

可手算的無進位例子

取 F=(x1∨x2∨x3) AND (¬x1∨x2∨¬x3)，欄序 x1,x2,x3,C1,C2：

數	十進位欄表示
T1,F1	10010，10001
T2,F2	01011，01000
T3,F3	00110，00101
S1,S1'	00010，00020
S2,S2'	00001，00002
target	11144

assignment x1=true,x2=true,x3=false 選 T1+T2+F3=11122。每個 clause 真 literal 數都是二，再選 S1'=20、S2'=2，得 11144。沒有進位；高位每欄的 1 也直接證明每個變數只選一個真值數。

可手算的無進位例子 (續)

數值可能與 10^{n+k} 同階，但 binary 表示僅 $O(n+k)$ bits。這正是 reduction 可多項式輸出，而 $O(nt)$ 精確 DP 仍非輸入 bit 長度多項式的原因。

T1+T2+F3=11122；再加 slack 00022，得到 target 11144

	x1	x2	x3	C1	C2	
T1	1	0	0	1	0	變數欄各恰為1
T2	0	1	0	1	1	兩個 clause 各有
F3	0	0	1	0	1	2個真 literal
slack	0	0	0	2	2	各以 slack 2 補到4
target	1	1	1	4	4	

所有子集合都不能進位，才可反向解碼

若只對「依滿足指派選數」證明無進位，尚不足以證明反向：某個錯誤子集合也許靠進位偽造target。本構造對任意子集合都有變數欄≤2、clause欄≤6，故十進位下絕不進位。進位排除是所有候選的全稱命題。

每個數的位元長度O(n+k)，把十進位欄轉成binary可用逐欄乘10加digit的$O((n+k)^2)$簡單bit演算法；2n+2k個數也仍多項式。驗證子集合和時，至多加O(n+k)個O(n+k)位元整數，總和多出O(log(n+k))位元，亦多項式。

例如只取x1=false,x2=false,x3=false，兩clause的真literal數為0、2。第一欄即使兩個slack都取，也只有3，不能達4；不會向其他欄借出或借入一個位值來掩飾不滿足。

編碼不變性、P 的封閉性與 coNP

兩種合理編碼若可相互在多項式時間轉換，P membership不變：先轉碼再求解，輸出長度受轉碼時間限制，組合仍多項式。Binary與unary整數編碼不具此雙向保證，因把b位元數2^(b−1)寫成unary需指數長輸出。

P對補集、聯集、交集封閉：把判定結果取反或依序執行兩個判定器即可。對串接L1L2，枚舉n+1個切點，每個執行兩個多項式判定器。對Kleene star L^*，用reachable[0]=true，對i<j以L判定x[i+1..j]是否可作下一段；$O(n^2)$次多項式判定與DP可決定reachable[n]，空字串直接接受。因此兩者也在P。

coNP={L：補集L在NP}，表示no instance有短證書，不等同「不是NP」。尚未知P=NP或NP=coNP。若NP-complete語言C也在coNP，任取A∈NP經A≤pC，則補集A≤p補集C；把補集C的NP證書接上多項式轉換，可知補集A∈NP，得到NP⊆coNP，再取補集得相等。此為條件推論，不能當成已證分離。

NP-complete 證明鏈與各問題條件

教材使用的證明鏈可整理成：

`CIRCUIT-SAT <=_p SAT <=_p 3-CNF-SAT <=_p CLIQUE`。

再由 complement 得到 `CLIQUE <=_p INDEPENDENT-SET <=_p VERTEX-COVER` 的等價轉換。另由 `VERTEX-COVER <=_p HAMILTONIAN-CYCLE <=_p TSP-decision`。再以 3-CNF-SAT 到 SUBSET-SUM 的無進位十進位欄位構造，完成另一條 NP-complete 證明。

每個箭頭都要保留自己的 hypotheses：HC 用無向圖；TSP reduction 的 complete graph 可由原圖 polynomial-size 建出；subset-sum 的 t binary encoding 不能省；optimization TSP 與 decision TSP 不可混寫成同一語言。

設計新 NP-complete 證明的檢查表

1. 把決策問題精確寫成 yes 集合，列出 instance 參數與 output。
2. 證明它 in NP：指定 certificate、長度上界、verifier、接受／拒絕條件。
3. 選一個已知 NP-complete source A，構造 `A <=_p f -> B`。
4. 證明 completeness direction：source yes 可構造 target yes。
5. 證明 soundness direction：target yes 可解回 source yes。
6. 證明 f 與 target verifier 都是 polynomial time，且 output size 是 polynomial。
7. 檢查退化 edge cases，例如空圖、k=0、重複 literals、圖不連通與 unreachable value。

若只有「看起來像」gadget，沒有第 4、5 步，就沒有約化證明。若只把新問題轉回已知 NP-complete，方向也反了。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 34。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

35 近似演算法

Part VII Selected Topics

本章定位

近似演算法以可證界限交換精確最優解的成本；本章比較cover、metric TSP與subset-sum的下界、誤差和位元成本，區分常數比、對數比與FPTAS。

前置知識

最佳化、近似比與 NP-hard 問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 定義 rho(n) approximation ratio。
- 分析 vertex cover、TSP、set cover、subset sum。
- 說明 randomization、LP rounding 與 PTAS/FPTAS 的邊界。
- 35.1 The vertex-cover problem
- 35.2 The traveling-salesman problem
- 35.3 The set-covering problem
- 35.4 Randomization and linear programming
- 35.5 The subset-sum problem

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

近似保證先說清楚目標、可行性與分母

本章處理非負目標值的最佳化問題。最小化的ρ近似要求每個輸入都回傳可行解，且ALG≤ρ・OPT；最大化則要求ALG≥OPT/ρ，ρ≥1。OPT=0時用不等式定義，不做0/0；最小化必須回0，最大化的非負可行值則已符合。

「對這個例子很好」不是近似保證；演算法須在所有合法輸入上滿足界限，而且計算時間以輸入編碼長度衡量。最小化證明常用可計算下界LB≤OPT，再證ALG≤ρLB；最大化則用上界UB≥OPT，證ALG≥UB/ρ。

PTAS（多項式時間近似方案）對每個固定ε>0，其時間對輸入長度N為多項式；例如N^(1/ε)可符合。FPTAS（完全多項式時間近似方案）還要求同時對N與數值1/ε為多項式。這不是只對log(1/ε)多項式，也不保證ε極小時仍便宜。

35.1 The vertex-cover problem

無向簡單圖G=(V,E)的vertex cover C滿足每條邊至少一端在C。求最小|C|是NP-hard最佳化問題；本節的保證針對未加權版本。空邊集合時回∅、OPT=0。

```
C=empty; matched[v]=false for all v
for each edge (u,v), in any fixed order:
    if not matched[u] and not matched[v]:
        add u,v to C
        matched[u]=matched[v]=true
return C
```

這等價於每次選任一未覆蓋邊，把兩端加入C、刪除所有已覆蓋邊。用端點標記而非反覆重建整張圖，可在O(|V|+|E|)時間、O(|V|)工作空間完成；輸入圖原儲存另計。

Maximal Matching 給了可證的下界

設被選的邊集合為M。因選一邊後兩端立即標記，M中任兩邊不共享端點，是matching（匹配）。每條未被選的邊在掃描時至少一端已標記，因此最後被C覆蓋；演算法回傳可行cover。

M是maximal matching（再也不能加邊），不必是maximum matching（邊數最多）。任一cover，包括C*，都必為M的每條邊選至少一端；這些邊互不共享端點，故至少要|M|個不同頂點。於是

$|C| = 2|M| \leq 2|C^*|$。

證明不需要知道OPT，也不需要求最大匹配。不能把「每條任意邊都需一點」直接相加成|E|下界，因相鄰邊可能共用同一cover頂點；matching才排除了重複計費。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

可達比值二，也可能正好最優

四葉star的中心c、葉a,b,d,e，先選(c,a)：C={c,a}，其餘三條邊因端點c已選而略過。最優只需{c}，|C|/OPT=2，保證可達。

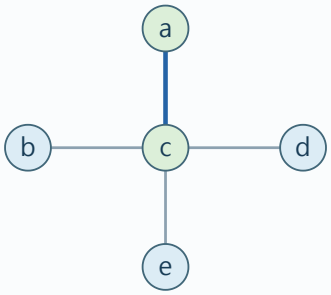
三角形a,b,c先選ab：C={a,b}，其餘邊已覆蓋；任取一點仍漏另一條對邊，所以OPT=2，這次ALG/OPT=1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

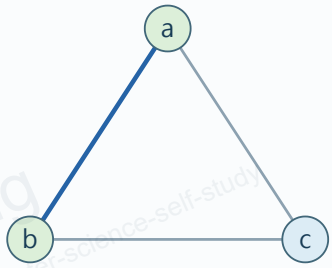
可達比值二，也可能正好最優 （續）

同樣選matching兩端：star達2倍，三角形恰好最優

藍粗邊是選取的 matching；綠點是輸出的 cover。



|M|=1 · ALG=2 · OPT=1



|M|=1 · ALG=2 · OPT=2

若改為頂點有權重，這個選兩端法沒有2倍權重保證。只有一條邊、兩端權重1與100，就會取兩點成本101，最優為1；第35.4節需用權重敏感的LP捨入。

35.2 The traveling-salesman problem

本節主要輸入為n≥3的完全無向圖，邊權c(u,v)≥0、對稱且滿足triangle inequality（三角不等式）c(u,w)≤c(u,v)+c(v,w)。這稱metric TSP；允許不同頂點間成本0時也稱pseudometric，以下證明仍成立。

TSP tour每個頂點恰訪一次後回起點。Double-tree演算法：求最小生成樹T，任選根做完整DFS walk，按頂點第一次出現的順序形成排列，最後連回起點。最後連邊必存在，依賴完全圖前提；一般稀疏圖不能直接這樣shortcut。

在完整權重矩陣上用簡單Prim需$O(n^2)$，DFS／輸出$O(n)$，所以總$O(n^2)$次word算術與比較；若權重為大有理數，須另計每次算術／比較的bit成本。

兩倍保證的三條不等式各用什麼前提

從最佳tour H^*移去任一邊，得到生成樹；移去非負邊不增加成本，MST最小性給c(T)≤c(H^*)=OPT。

完整DFS walk W每條樹邊去、回各一次，故c(W)=2c(T)。W可能重複頂點，所以還不是TSP tour。

若W的某一段為u→v_1→⋯→v_k→w，而中間點之前都已拜訪，可以直接改走u→w。反覆三角不等式給c(u,w)≤這段原成本；每次省去重複拜訪都不增加成本。保留各頂點首次出現及最後回起點，就得到可行tour H，且

c(H)≤c(W)=2c(T)≤2OPT。

下界需要非負，shortcut需要metric，輸出需要完全圖；這些不是可以省略的同一個「TSP」假設。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Double-tree 的每條路與實際成本

取四點Manhattan距離c(p,q)=|p_x−q_x|+|p_y−q_y|：A(0, 0)、B(2, 0)、C(2, 1)、D(0, 1)。三角不等式由各座標絕對值的不等式相加得到。

可取MST邊AD=1、DC=2、CB=1，總4。六條邊成本為1, 1, 2, 2, 3, 3，任何生成樹的三條邊成本至少1+1+2=4，所以此鏈確實最小。根A、child鏈A→D→C→B，完整walk為A,D,C,B,C,D,A，成本1+2+1+1+2+1=8。首次順序A,D,C,B，再回A，tour成本1+2+1+2=6。

每個tour成本至少6：要訪最左與最右x座標，封閉路徑的x總變動至少4；同理y總變動至少2。矩形周邊tour正好6，所以OPT=6。此例不是把walk8誤當成真正tour成本6。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

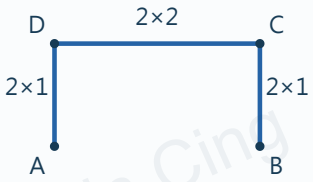
Double-tree 的每條路與實際成本 (續)

同一矩形、同一Manhattan成本：MST 4 → walk 8 → tour 6

最小生成樹

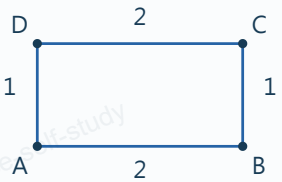
AD+DC+CB=1+2+1=4

每樹邊各走2次



A,D,C,B,C,D,A；成本8

取首次出現、連回A



A,D,C,B,A；成本6

A(0,0)、B(2,0)、C(2,1)、D(0,1)；座標兩軸同尺度，邊標示為Manhattan成本。

Christofides：最小匹配帶來 3/2 保證

沿用metric前提。令O為MST中奇度頂點。度數和為2|E|，所以|O|為偶數；在O上求最小權完美匹配M，將其邊與T組成multigraph。每個奇度點被加一次後變偶，其餘度不變；圖仍連通，故有Euler closed walk，shortcut得到tour。

取最佳tour，按它造訪O的順序shortcut，只保留O，得到總成本≤OPT的封閉循環。把循環邊交替分成兩個完美匹配（|O|=2時視為兩份往返邊），兩者成本和≤OPT，至少一個≤OPT/2。因此最小匹配c(M)≤OPT/2，最後tour成本≤c(T)+c(M)≤3OPT/2。

此演算法使用「一般圖最小權完美匹配可在多項式時間求得」作子程序定理；不能用任意greedy matching替代最小權匹配後仍沿用此證明。求Euler walk與shortcut本身為多項式；證明與double-tree共用metric前提。

非 Metric TSP：常數比會解出 HC

假設存在確定性、多項式時間且對每個非負完全圖都保證ρ近似的演算法A，ρ≥1是固定常數。取固定整數R≥ρ，對HC輸入G建立完全圖：原邊權1、非邊權M=Rn+1。

有HC時OPT=n，故A輸出成本≤ρn≤Rn。無HC時任何tour至少使用一條非邊，成本≥M+(n−1)=Rn+n>Rn。因此只要檢查A輸出是否≤Rn，就能多項式判定HC；將推出P=NP。

M的binary長度O(log n)，完整圖輸出O(n^2 log n) bits，沒有以巨大數值暗藏超多項式輸出。這些權重通常違反三角不等式，所以不抵觸metric近似演算法。結論是「除非P=NP，一般TSP無固定常數比的此類演算法」，不是已證P≠NP，也不把只有期望保證的隨機演算法直接套成確定性判定。

35.3 The set-covering problem

Universe X有N個元素、候選family F={S_1,⋯,S_M}，每個S_j⊆X，聯集須為X。Set cover選若干候選集合覆蓋X；未加權最小化集合數，加權版每組有正有理成本w_j、最小化成本和。空X回∅；聯集不足則回報無可行解。

```
U=X; C=empty
while U is not empty:
    choose S with |S∩U|>0 minimizing w(S)/|S∩U|
    C=C union {S}; U=U minus S
return C
```

未加權w=1時等價於每次選新增覆蓋最多者。每次至少覆蓋一個新元素，且選過的集合之後新增數0不再選，故至多min(N,M)輪。簡單N×M incidence矩陣每輪O(NM)即可算全部新增數，總O(NM min(N,M))次操作，已是多項式；有理成本比用交叉相乘精確比較。

35.3 The set-covering problem （續）

Set-cover的精確最佳化為NP-hard：由vertex-cover輸入G建立X=E，每個原頂點v對應候選集合S_v={與v相接的邊}、成本1。選至多k組覆蓋X，當且僅當原圖有至多k個頂點覆蓋所有邊。Incidence總數2|E|，構造長度與時間均多項式；加權版包含成本全1的特例，因此也NP-hard。此歸約沒有聲稱每個特殊family都困難。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

用實際元素收費，得到 Harmonic 界

某輪選S、成本w，新增k個元素，就給每個新元素charge=w/k。每元素只第一次被覆蓋時付費，故Σ_(e∈X)charge(e)=ALG。

固定任一候選集合A，令|A|=d_A。將其元素按首次被覆蓋次序排$e_1,\dots,e_d$。同輪多個元素任意排。在e_j被覆蓋的那一輪開始，A至少還有d_A−j+1個未覆蓋元素；A本身也是合法候選，所以greedy選擇的每元素成本

`charge(e_j)≤w(A)/(d_A−j+1)`。

即使e_j同輪不是該集合第一個被覆蓋者，當輪開始的未覆蓋數只會更多，此界仍成立。加總得Σ_(e∈A)charge(e)≤w(A)H_(d_A)，其中H_d=1+1/2+⋯+1/d，H_0=0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

對最佳 Cover 相加時可以重複，不能漏元素

設C^*為最佳cover，d=max|S_j|。每個元素至少屬於C^*中的一組，charge非負，故

$$ALG = \sum_{e \in X} charge(e) \leq \sum_{A \in C^*} \sum_{e \in A} charge(e) \leq \sum_{A \in C^*} w(A) H_{|A|} \leq H_d \, OPT.$$

重複覆蓋的元素在右邊可能付多次，只會使上界更大，方向正確。又H_d≤1+ln d：對j≥2，1/j≤∫_(j−1)^j dx/x，逐項相加即可。

未加權與加權版共享此證明，但greedy規則不同；有成本時仍只挑最多新增元素，不符合前頁的每單位成本比較，不能沿用H_d保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy 三組，最優兩組：每個元素都對上

X={1, 2, 3, 4, 5, 6}，S_1={1, 2, 3}、S_2={4, 5, 6}、S_3={1, 2, 4, 5}，成本皆1。先選S_3新增4，再以同額時S_1優先：

輪	進入時U	新增數(S_1,S_2,S_3)	選擇	新元素charge
1	1, 2, 3, 4, 5, 6	3, 3, 4	S_3	1, 2, 4, 5各1/4
2	3, 6	1, 1, 0	S_1	3付1
3	6	0, 1, 0	S_2	6付1

ALG=3；$S_1 \cup S_2$=X，任一單組都不夠，所以OPT=2，比值3/2。charge總和4・(1/4)+1+1=3；在S_1上合計3/2≤H_3=11/6，S_2同樣，符合一般計費證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Greedy 三組，最優兩組：每個元素都對上 (續)

S3先覆蓋4個，但最後需要3組；S1+S2只需2組

	1	2	3	4	5	6
S1	1	1	1	0	0	0
S2	0	0	0	1	1	1
S3	1	1	0	1	1	0

初始：1,2,3,4,5,6

取S3後：3,6

取S1後：6

取S2後：∅

charge(1,2,4,5)=1/4；charge(3)=charge(6)=1

總收費3=greedy成本；最優 S1∪S2={1,...,6}，且不存在單組cover。

35.4 Randomization and linear programming

MAX-3-CNF-SAT最大化滿足clause數m。先規定每clause三個相異變數，且不含同一變數與其否定；各變數獨立、公平地取0或1。對某一clause，三literal全假的機率1/8，故滿足機率7/8。

若能多項式精確求出此最佳化的OPT，檢查OPT=m便能判定第34章的3-CNF-SAT，故精確最佳化NP-hard；這不妨礙以下對所有輸入的近似保證。

令I_j是第j個clause被滿足的指示，$Y=\Sigma I_j$。各clause可共享變數，I_j不必獨立；期望線性仍給

$E[Y]=\Sigma E[I_j]=7m/8\geq 7OPT/8$。

這裡「隨機8/7近似」是指期望目標值保證，不是每次輸出都達7OPT/8，也不是$E[OPT/Y]\leq 8/7$。只有一個clause時，機率1/8會得到Y=0，後者比值甚至無法給有限界。重複literal如$(a\vee a\vee a)$只有1/2滿足率，不能直接套7/8；含a與¬a的clause恆真，可另行計數。

從期望到可控制的成功率

若m=0，直接回傳任意assignment，Y=OPT=0，達到門檻的成功率為1。以下假設m>0；在前述三個相異變數模型，0≤Y≤m且E[Y]=7m/8。設p=Pr[Y≥3m/4]；對未達門檻者用3m/4作上界，得

$7m/8=E[Y]\le(1-p)3m/4+p\cdot m$，所以$p\ge1/2$。

因此獨立重跑r次，保留滿足數最多的assignment，未達3m/4的機率≤2^(−r)。這是明列成功門檻與失敗率的4/3保證；不能把它說成同時以高機率達到7/8。

若需要確定性7/8目標值，可用下一頁的條件期望逐一固定變數，不需要以大量隨機試跑猜出某一個好assignment。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

條件期望：每次固定都不降低保證

在部分指派下，某clause已有真literal則條件滿足率1；已全部指定且全假則0；其餘有r個未指定變數時為1−2^(−r)。逐clause相加即可精確算E[Y｜目前指派]。

對下一變數x，原條件期望是「x=0後期望」與「x=1後期望」的平均，所以選較大的一邊不會下降。固定完n個變數後沒有亂數，條件期望就等於實際滿足數，至少7m/8≥7OPT/8。

例四個clauses：(a∨b∨c)、(¬a∨b∨c)、(a∨¬b∨c)、(a∨b∨¬c)。初始期望7/2；a=0後為13/4，a=1後為15/4，取a=1。之後b=0後為7/2、b=1後為4，取b=1。c任選，例如0，四clause全真。

直接每步重算m個三literal clauses需O(nm)次固定精度分數操作；此例每個條件機率分母至多8，沒有隱藏指數精度。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weighted Vertex Cover 的 LP 下界與捨入

每個頂點有正有理權w_v。整數模型最小化Σw_vx_v，約束每邊uv有x_u+x_v≥1、x_v∈{0, 1}。放寬為0≤x_v≤1得到LP；所有整數cover仍可行，因此LP^*≤OPT。

求最優可行LP解x^*，取C={v：x_v^*≥1/2}。對任一邊，若兩端都未取，兩個值都<1/2，違反$x_u^*+x_v^*$≥1，所以C可行。又逐頂點有1_[v∈C]≤2x_v^*，乘非負w_v相加：

$w(C) \le 2\Sigma w_v x_v^* = 2LP^* \le 2OPT$。

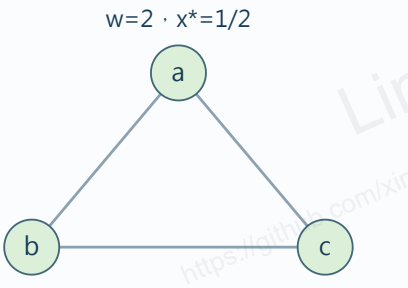
有理LP可用多項式bit複雜度的LP演算法求解；不能僅因simplex常常快就宣稱其最壞多項式。此證明採確實可行的LP解；若數值求解器有容許誤差，須另證可行性與成本界，不能任意將0.4999當成精確1/2。

LP 最佳值與整數最佳值是兩個不同下界

三角形a,b,c權重2, 3, 4。LP的x_a=x_b=x_c=1/2可行，成本9/2。它也是LP最優：將邊限制ab、ac、bc分別乘1/2、3/2、5/2後相加，左邊恰為2x_a+3x_b+4x_c，右邊為9/2，所以任何LP可行解成本≥9/2。

整數cover至少兩點，三種兩點成本5, 6, 7，故OPT=5。Threshold捨入把三點都取，ALG=9=2LP^*≤2OPT=10；實際比值9/5，而不是把LP^*=4.5誤當原問題最優。

加權三角形：LP*=9/2，整數OPT=5，捨入成本9



邊限制乘數：ab 1/2、ac 3/2、bc 5/2

加總：2x_a+3x_b+4x_c ≥9/2

半值達到9/2，所以為LP最優

整數取 a,b：成本2+3=5

≥1/2 捨入取三點：成本9

綠點全被threshold取入；LP下界與原整數最佳值不能混用。

隨機 Set-cover 捨入的成本與可行性分開驗證

作為LP方法的延伸，設N≥2，LP有每元素Σ_(S∋e)x_S≥1、x_S≥0及非負成本。令整數k=3⌈$\log_2 N$⌉≥3 ln N，獨立以p_S=min(1,kx_S)選每個集合。若LP解為有理數，p亦為有理數；以分母為範圍均勻抽整數即可實現，不需精確抽取含ln N的無理數機率。

若包含e的某組p=1，e必覆蓋；否則漏覆機率Π_(S∋e)(1−p_S)≤exp(−Σp_S)≤exp(−k)≤N^(−3)。Union bound得至少一個元素漏覆的機率≤N^(−2)≤1/4。期望成本≤k・LP^*；Markov得成本>2k・LP^*的機率≤1/2。兩種失敗事件相加≤3/4，所以每次同時可行且成本合格的機率至少1/4，成本比2k=O(log N)。

實際檢查覆蓋與成本，不合格就獨立重試，期望至多4次；這是可驗證成功條件的期望多項式演算法，不是固定一次必成功。N=0直接空解；N=1選最便宜包含該元素的集合，另行處理ln1=0。

若LP^*=0，成本非負且期望成本0，故成本必為0，成本檢查自動成立；只需按相同覆蓋規則重試，不以0作Markov的分母。

35.5 The subset-sum problem

本節最佳化版：正整數$a_1,\cdots,a_n$與整數t≥0，找一個子集合使總和≤t而盡可能大。以索引分辨各項，即使數值相同也不能重用同一項。a_i>t可先刪；n=0或t=0直接回空集、值0。

精確清單演算法以$L_0=\{0\}$開始，L_i=merge_unique(L_(i−1),L_(i−1)+a_i)，再刪去>t。兩個輸入皆排序，以兩指標線性合併去重，不必重排。

每個前i項的子集合恰分「不取i」或「取i」，分別對應L_(i−1)與L_(i−1)+a_i。正數使已超過t的和未來不可能再下降，所以刪除安全；歸納得清單保留所有可行精確和。最壞有2^i個不同值，例如二的冪且t足夠大，故通常是指數時間，不因每輪合併線性就成多項式。

TRIM 只刪較大的近值，保留真實可行和

給排序清單L、δ>0：

```
R=[]
for y in L, in increasing order:
    if R is empty or y>(1+delta)*R[-1]: append y to R
return R
```

對每個候選y，若被刪，當時保留的last=z滿足z≤y≤(1+δ)z；若被保留，取z=y。所以每個y都有代表z∈R，y/(1+δ)≤z≤y。0會保留，任何正值都>0，故第一個正值不會被0刪掉。

保留的是候選清單中的真實整數和，不是將y數值除掉或四捨五入。用較小代表才能維持≤t的可行性；若用較大代表，原本可行y加未來項可能被錯誤丟棄。

每輪合併、Trim、刪超額的完整算法

選0<ε<1，n≥1，令δ=ε/(2n)：

```
L=[0]
for i=1..n:
    B=merge_unique(L, [z+a[i] for z in L])
    L=trim(B,delta)
    remove all values >t from L
return max(L)
```

每個保留值仍是前i項某子集合的真實和，故回傳值確實≤t且可行。若需輸出子集合，可為每個保留候選保留不可變的parent pointer與本輪是否取a_i；合併同值時任選一個合法來源。回溯最多n步，各索引嚴格往前，不會重用物品。

只算值時可滾動存清單；若保留各輪的回溯節點，空間需另計到O(nK)個節點，K是單輪清單上界，不能仍稱只需O(K)。

代表誤差的歸納：包括「取本項」分支

對任一前i項的精確可行和y≤t，證明存在z∈L_i使y/(1+δ)^i≤z≤y。i=0只有y=z=0。

若不取a_i，y是上一輪可行和，由歸納取u∈L_(i−1)，y/(1+δ)^(i−1)≤u≤y。候選B有u；trim後有z≥u/(1+δ)，故達所需界且z≤y≤t，不會被刪超額。

若取a_i，寫y=y'+a_i。正數保證y'≤t，由歸納有u≥y'/(1+δ)^(i−1)、u≤y'。候選B有u+a_i，且

`u+a_i≥y'/(1+δ)^(i−1)+a_i≥(y'+a_i)/(1+δ)^(i−1)`。

Trim再損失至多1+δ因子，代表z≤u+a_i≤y≤t，所以也不被超額刪除。這一步明用a_i≥0；若容許負數，早刪超額與此歸納都不能原樣沿用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

累積誤差真的落在 ε 內

把上一頁用於y=OPT，最終回傳z_max至少是其代表，所以OPT/z_max≤(1+δ)^n（OPT=0直接以不等式成立）。由ln(1+x)≤x，

`n ln(1+ε/(2n))≤ε/2`。

又0<ε<1時，ln(1+ε)=∫₀^ε(1+x)^(−1)dx≥ε/(1+ε)≥ε/2。因此(1+ε/(2n))^n≤1+ε，得到

`z_max≥OPT/(1+ε)≥(1−ε)OPT`。

這是每輪誤差累乘後的證明，不是把全局ε直接當每輪δ。若每輪都用δ=ε，誤差可累至(1+ε)^n，不能保證同一個全局1+ε。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四輪整數清單：被刪值與最終答案都可追蹤

a=[112, 108, 205, 103]、t=325、ε=2/5，n=4，δ=1/20：

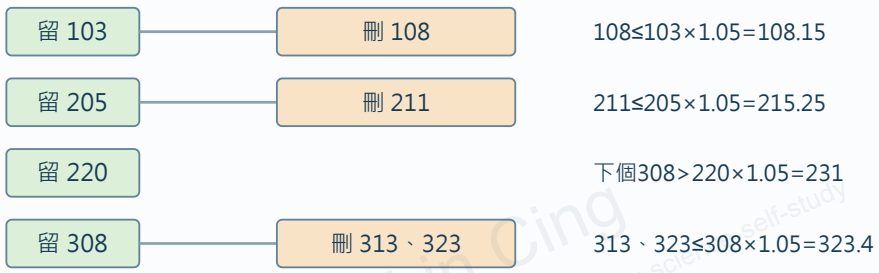
i	合併去重B	TRIM結果	刪去>325之後
1	0, 112	0, 112	0, 112
2	0, 108, 112, 220	0, 108, 220	0, 108, 220
3	0, 108, 205, 220, 313, 425	全保留	0, 108, 205, 220, 313
4	0, 103, 108, 205, 211, 220, 308, 313, 323, 416	0, 103, 205, 220, 308, 416	0, 103, 205, 220, 308

第2輪112≤108・1.05=113.4，刪112；第4輪108≤103・1.05=108.15，211≤205・1.05=215.25，313及323≤308・1.05=323.4，均被較小真實和代表。

輸出308=205+103；精確最優323=112+108+103。實際比值323/308≈1.049≤1.4，且308≤325。被丟掉的112不是從輸入刪除，而是在該輪候選和中被108代表。

四輪整數清單：被刪值與最終答案都可追蹤 （續）

最後一輪：較小代表保留真實和，刪掉其相對近鄰



輸出308=205+103；OPT=323=112+108+103，323/308<1.4。

0保留；416>325於TRIM後刪除。圖只展出最後一輪的正值近鄰。

清單長度、位元成本與 FPTAS

Trim後相鄰正值比率>1+δ，最小正值≥1，最大≤t，所以正值數≤1+ln t/ln(1+δ)，另加0。因 ln(1+δ)≥δ/(1+δ)≥δ/2，單輪長度

$K=O(1+n\ \log(t+1)/\varepsilon)$。

每輪兩條長K清單線性合併、trim及刪除，總$O(nK)=O(n+n^2\ \log(t+1)/\varepsilon)$次算術／比較；只存值時O(K)工作空間。log(t+1)是輸入bit長量級，不是數值t；與精確DP的O(nt)不同。

若ε=α/β為精確有理數，trim判斷可改為$2n\beta\cdot y>(2n\beta+\alpha)\cdot last$，避免浮點相對誤差。另外讀取與刪除a_i>t需O(N)位元工作，N為原輸入長度；之後候選≤2t。中間整數位數由log(t+1)、log n與α/β的編碼長度多項式控制。每次乘法計入相應大整數成本，總時間仍對輸入長度與1/ε多項式，才完成FPTAS的時間證明。

各種保證不可互換

方法	可行性	對目標的保證	額外條件
未加權VC選matching兩端	每次可行	ALG≤2OPT	不適用任意權重
Double-tree TSP	每次可行	ALG≤2OPT	完全、非負、對稱、metric
Weighted set cover greedy	每次可行	ALG≤H_d OPT	按成本／新增數選
隨機MAX-3-CNF	每次有assignment	E[ALG]≥7OPT/8	每clause三相異變數
LP weighted VC threshold	每次可行	ALG≤2OPT	可行最優LP與非負權
Subset-sum TRIM	每次可行	ALG≥OPT/(1+ε)	非負項、δ=ε/(2n)

期望界、每次確定界與帶失敗率的保證回答不同問題。近似方案仍須把精度、清單長度、輸入位元及解的回溯成本一起交代，不能只寫「刪相近值，所以很快」。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 35。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

32 字串匹配

Part VII Selected Topics

本章定位

字串匹配把已知相等的前後綴轉成可重用狀態；本章從逐字比較與模數碰撞，推導有限自動機與KMP的失敗鏈、正確性及線性總成本。

前置知識

模式前綴、雜湊與有限自動機。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 定義 valid shifts 與 prefix 問題。
- 比較 naive、Rabin-Karp、automaton 與 KMP。
- 證明 KMP 前綴函數與線性總時間。
- 32.1 The naive string-matching algorithm
- 32.2 The Rabin-Karp algorithm
- 32.3 String matching with finite automata
- 32.4 The Knuth-Morris-Pratt algorithm

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

32.1 The naive string-matching algorithm

文字（text）T 的長度為 n，模式（pattern）P 的長度為 m；字元來自有限 alphabet Σ，相等比較採 O(1)。本章字元位置一律從1開始，shift s 從0開始：合法位移滿足 0≤s≤n−m 且對每個 j=1,⋯,m 都有 P[j]=T[s+j]。輸出所有位移，允許重疊。

P_k 表示 P[1..k]，P_0=ε 為空字串。Prefix 是前綴，suffix 是後綴；proper prefix/suffix 不得等於整個字串。Border 指同時為前綴與後綴的字串。

若 m=0，規格定為輸出0,⋯,n這 n+1 個邊界；若 m>n，輸出空集合。後續主演算法先處理這兩種情形，才假設1≤m≤n，避免讀取不存在的 P[1]。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐字比較：不漏位移也不接受錯字

```
for s=0..n-m:
    j=1
    while j<=m and P[j]==T[s+j]: j++
    if j==m+1: output s
```

內圈每次檢查前，P[1..j−1]=T[s+1..s+j−1]。相等才延伸不變量；不等就知道此位移不合法；j=m+1 表示全部相等。外圈恰枚舉所有合法範圍，因此聲稱的每個匹配正確，真正匹配也不會漏掉。

共有 n−m+1 個窗口，每個至多 m 次比較，最壞 Θ((n−m+1)m)。取 T=a^n、P=a^m，每個窗口都用滿 m 次，證明此界可達；即使沒有碰撞，也可能有很多真正匹配。額外工作空間 O(1)，不含輸出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一文字的所有窗口與比較次數

T=abababaca、P=aba。遇到第一個不相等就停止：

s	窗口	依序比較	次數	結果
0	aba	a=a，b=b，a=a	3	輸出0
1	bab	a≠b	1	拒絕
2	aba	a=a，b=b，a=a	3	輸出2
3	bac	a≠b	1	拒絕
4	aca	a=a，b≠c	2	拒絕

共10次比較。s=0與s=2共用文字位置3，仍是兩個合法匹配。Naive 重新檢查同一字元對齊；KMP 將利用已知相等的區段排除不可能的位移。

32.2 The Rabin-Karp algorithm

將字元單射編為0,…,d−1，以 radix d 表示長 m 字串。其整數值 H(X)=Σ_(j=1)^m X[j]d^(m−j)，但 m 位數的大整數運算不能當 O(1)。因此保留 h_q(X)=H(X) mod q，q≥2，並在 hash 相等時逐字驗證。

令 p=h_q(P)、t_s=h_q(T[s+1..s+m])、h=d^(m−1) mod q。由移除最高位、左移一位、加入新低位，得到

$$t_{s+1} = \big(d(t_s - T[s+1]h) + T[s+m+1]\big) \bmod q.$$

此恆等式對任意 q≥2 成立，不必是質數；選質數有利於某些隨機化分析，卻不自動保證固定字串的碰撞率為1/q。實作須將負餘數正規化為0,…,q−1，並讓乘法中間值不溢位。

Hash 只排除，字元驗證才接受

```
h=1; p=0; t=0
for j=1..m-1: h=(d*h) mod q
for j=1..m:
    p=(d*p+P[j]) mod q
    t=(d*t+T[j]) mod q
for s=0..n-m:
    if p==t and P[1..m]==T[s+1..s+m]: output s
    if s<n-m:
        t=(d*(t-T[s+1]*h)+T[s+m+1]) mod q
```

Horner 法與 rolling 恆等式歸納保證 t 始終是當前窗口 hash。真正相等的字串必有相同 hash，所以不會漏報；hash 不同可拒絕，hash 相同再比較原字串，所以不會誤報。碰撞影響時間，不影響這個驗證版本的正確性。

若省略原字串驗證，則變成可能誤報的演算法，不能沿用「總是正確」的結論。

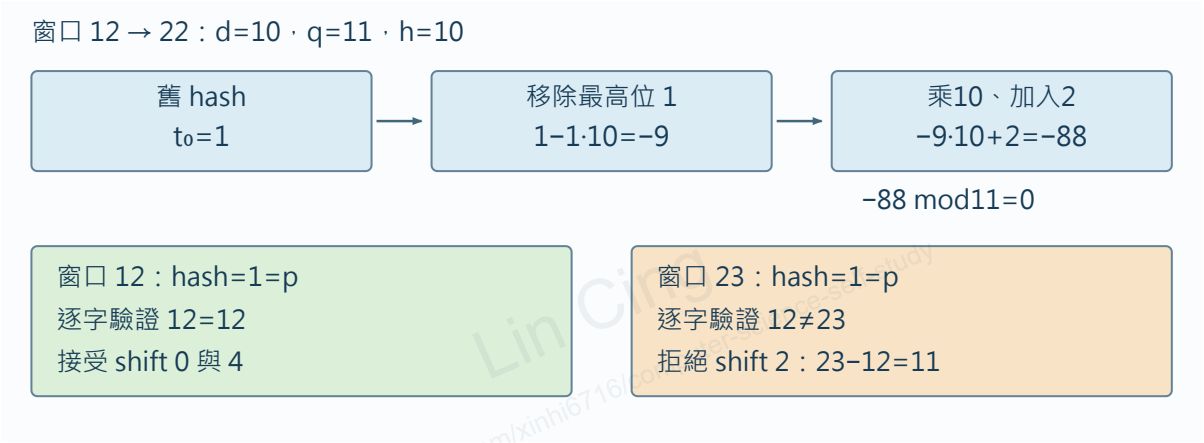
一次真正匹配、一次碰撞、再一次匹配

取 T=122312、P=12、d=10、q=11。h=10，p=12 mod11=1：

s	窗口	t_s	計算下一個 hash	字元驗證
0	12	1	(10(1−1・10)+2) mod11=0	12=12，輸出0
1	22	0	(10(0−2・10)+3) mod11=1	hash不同，略過
2	23	1	(10(1−2・10)+1) mod11=9	12≠23，拒絕碰撞
3	31	9	(10(9−3・10)+2) mod11=1	hash不同，略過
4	12	1	最後窗口，不再讀新字元	12=12，輸出4

Spurious hit（假命中）是 hash 相等而字串不同；此例23−12=11，正好被模數消掉。即使 q 是質數，碰撞仍可發生。

一次真正匹配、一次碰撞、再一次匹配 (續)



成本取決於真匹配與假命中

採 word-RAM：字元、q、所有模乘中間值能在 O(1) word 操作完成，或提供 O(1) 模乘。若直接算 d(t−out・h)，也必保證其中間值可容納；可先把差正規化後再乘 d，使中間值少於 dq。任意精度版本須另乘位元運算成本。

預處理 Θ(m)，rolling 掃描 Θ(n−m+1)。設真匹配數 z、假命中數 f，每次驗證至多 m 字元，總時間 $O(n+m+m(z+f))$。

最壞 z 或 f 可達 Θ(n−m+1)，所以最壞仍 Θ((n−m+1)m)。例如全 a 文字與全 a 模式只有真命中，照樣反覆驗證；不能把退化全歸因於選錯 hash。輸出匹配本身要 Ω(z)，本版本另外花 m 次字元驗證每個真命中。

有證明的隨機 Hash：明寫亂數與條件

另一種可分析的版本：固定質數 q>d−1，字元編碼不同即在域 F_q 不同；在輸入固定後，獨立均勻選 base b∈F_q，以 ΣX[j]b^(m−j) mod q 作 hash。Rolling 公式以 b 取代 d，b=0也依恆等式處理，m=1時 h=b^0=1。

對固定相異字串 X,Y，差多項式 D(b) 非零、次數至多 m−1。非零域上多項式至多有其次數個根：若 r 是根，因式定理提出(b−r)，再對次數歸納。因此

$Pr[h_b(X)=h_b(Y)]\le \min(1,(m-1)/q)$。

對每個非匹配窗口的碰撞指示變數取期望再相加，不需要窗口事件獨立，得到 E[f]≤(n−m+1−z)(m−1)/q。故期望時間為 O(n+m+mz+mnm/q) 的寬鬆界；選 q 足夠大才有低假命中成本，真匹配的 mz 仍在。q 的位元長度與模乘成本也必計入，不能任意增大後仍無條件稱 O(1)。

32.3 String matching with finite automata

有限自動機由(Q,q_0,A,Σ,δ)組成：有限狀態集合、初態、接受狀態集、字母表及每個狀態／字元對的轉移函數。匹配器令 Q={0,⋯,m}、q_0=0、A={m}。

定義 σ(x)=max{k≤m：P_k 是 x 的後綴}。空字串總是後綴，所以最大值存在。狀態 q=σ(目前已讀文字)，意義是「現在可延伸的最長匹配前綴」，不是已經找到幾次完整匹配。

令 δ(q,a)=σ(P_q a)。讀入一個字元後做一次轉移；當狀態等於 m 就輸出目前結尾所決定的位移。接受狀態並非停止狀態，仍有對每個 a 的轉移，才能找到重疊匹配。

為何只記最長前綴長度就夠

要證明 δ(σ(x),a)=σ(xa)。令 q=σ(x)、r=σ(xa)。若 r>0，把匹配的最後一字移去，P_(r−1) 是 x 的後綴，因此 r−1≤q。P_q 也是 x 的後綴；兩個後綴按長度包含，故 P_(r−1) 也是 P_q 的後綴，再接 a，P_r 就是 P_q a 的後綴。因此 r≤σ(P_q a)。r=0時此不等式也成立。

反過來，P_q a 本身是 xa 的後綴，任何同時為 P 前綴的 P_q a 後綴也必為 xa 後綴，故 σ(P_q a)≤σ(xa)=r。兩向合併得等式。

由初態 σ(ε)=0歸納，每次轉移後 q=σ(T[1..i])。q=m 當且僅當 P 是 T[1..i] 的後綴，恰好等價於 shift i−m 匹配；這證明接受判斷的必要性與充分性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整轉移表與重疊匹配

P=abab，Σ={a,b,c}，每格是 δ(q,字元)：

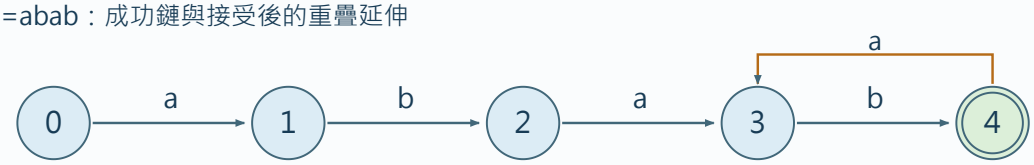
q	a	b	c
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	3	0	0

T=ababab 的狀態依序1,2,3,4,3,4，在 i=4與6輸出 shift0與2。δ(4,a)=3，因 ababa 的最長可用後綴是 aba。若匹配後無條件清零，就會錯過第二次匹配。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完整轉移表與重疊匹配 （續）

P=abab：成功鏈與接受後的重疊延伸



完整轉移表見正文；此圖只畫成功鏈與 δ(4,a)=3。

T[i]	a	b	a	b	a	b
狀態	1	2	3	4	3	4

讀到第4與第6字接受：shift 0、2；完整匹配之後不清零。

建表成本不能混入掃描成本

直接按 δ 的定義，對每個 q,a 從 min(m,q+1) 往下試 k，逐字驗證 P_k 是否為 P_q a 的後綴，可花 $O(m^2)$ 每格，總 $O(m^3|\Sigma|)$。這是正確但慢的建法；掃描仍是 Θ(n)。

有了下一節的 prefix function π，可按 q 遞增建表：

```
for a in Σ: δ(0,a) = (1 if a==P[1] else 0)
for q=1..m:
    for a in Σ:
        if q<m and a==P[q+1]: δ(q,a)=q+1
        else: δ(q,a)=δ(π[q],a)
```

Mismatch 時，下一個候選只能是 P_q 的 proper border；最長的長度為 π[q]。所有較短候選都在它的後綴內，所以同一字元轉移可重用其表格。π[q]<q使依賴格已算完。建 π 要 Θ(m)，建表 $\Theta(m|\Sigma|)$ 時間與空間；只有在可 O(1) 索引 alphabet 的模型下，每字元查表才是 O(1)。

32.4 The Knuth-Morris-Pratt algorithm

Prefix function（前綴函數）定為 π[q]=max{k<q：P_k 是 P_q 的後綴}，q=1,⋯,m；因此 π[1]=0且π[q]<q。它只依模式，與文字無關。

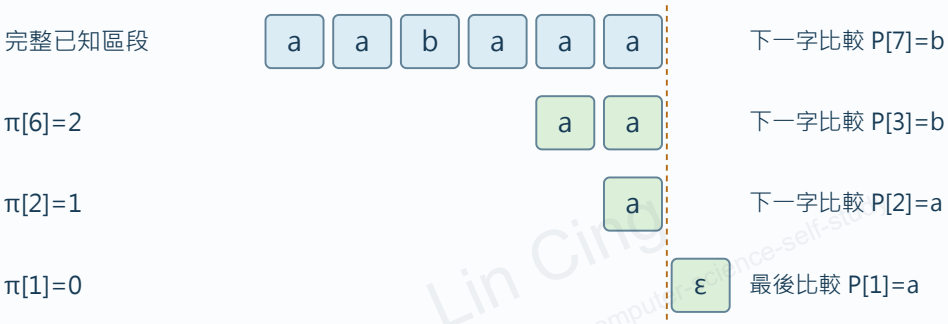
對 P=aabaaab，π=[0,1,0,1,2,2,3]。P_6=aabaaa 的 proper borders 是 aa、a、ε，長度鏈為 6→2→1→0。6本身只是搜尋的起點，不算 proper border。

KMP不儲存整張 δ 表。下一字元無法延伸時，就沿 π 鏈找下一個可能延伸的前綴。每次減少候選長度，同一個新文字字元保持不動，已讀過的文字不重讀。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

32.4 The Knuth-Morris-Pratt algorithm （續）

P=aabaaab，已知文字後綴 aabaaa（q=6）



新字元是 c：6→2→1→0，所有比較皆失敗；同一個 c 尚未向前移動。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

失敗鏈列出了全部候選，不會跳過可行長度

令 B(q) 為 P_q 的所有 proper border 長度。若 b=π[q]>0，任何 k<b 都有

$k \in B(q) \Leftrightarrow k \in B(b)$。

證明：P_b 是 P_q 的後綴，因此它的長 k 後綴就是 P_q 的長 k 後綴；而 P_k 是兩者共用的前綴。任一方向的前後綴相等條件因此相同。b 是最長 proper border，不可能有 b<k<q 的候選。若 b=0，沒有正長度的 proper border。

所以 B(q)={b}∪B(b)，依嚴格遞減長度歸納，反覆 π[q]、π[π[q]]、⋯、0恰好列出所有 proper border。這是 fallback 安全的原因；不是因「短一點大概會匹配」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prefix Function 的逐項建構

```
π[1]=0; k=0
for i=2..m:
    while k>0 and P[k+1]!=P[i]: k=π[k]
    if P[k+1]==P[i]: k++
    π[i]=k
```

在處理 i 前，k=π[i−1]。P_i 若有長 r>0 的 proper border，去掉最後字元後，長 r−1 必是 P_(i−1) 的 proper border；反之，只要該 border 之後的 P[r] 等於 P[i]，就能延長成長 r 的 border。

故由最大候選 π[i−1] 開始，沿完整失敗鏈遞減，第一個可延伸者產生最大合法 r；到0仍不相等則答案0。先前已算正確的 π 值只在較小索引被使用，歸納成立。因 π[i−1]≤i−2，這一步不會把整個 P_i 自身當 proper border。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一模式的建表全軌跡

P=aabaaab：

i	P[i]	初始k	比較與fallback	π[i]
1	a	—	單字元無非空proper border	0
2	a	0	P[1]=a，延長	1
3	b	1	P[2]=a≠b；1→0；P[1]仍不符	0
4	a	0	P[1]=a	1
5	a	1	P[2]=a	2
6	a	2	P[3]=b≠a；2→1；P[2]=a	2
7	b	2	P[3]=b	3

π[6]=2不是3：aabaaa的末三字aaa不等於前三字aab。建表不是在不相等時一律 k−−，而是只保留已知可行的 border。

KMP 掃描與匹配後的狀態

```
π=prefix_function(P); q=0
for i=1..n:
    while q>0 and P[q+1]!=T[i]: q=π[q]
    if P[q+1]==T[i]: q++
    if q==m:
        output i-m
        q=π[m]
```

在接受檢查時，q=σ(T[1..i])，與自動機相同。若未達 m，便保存此 q 供下輪使用；若達 m，先輸出，再保存 π[m]，即已讀文字中最長的 proper 模式前綴後綴。下一輪延伸新匹配只需要 proper border，因目前完整長 m 前綴不可能再增至 m+1。

Mismatch 的所有可行較短候選恰為失敗鏈，第一個能與 T[i] 延伸者就是 δ 的結果；若無候選則為0。這同時證明不漏任何匹配，以及讀取 P[q+1] 前永遠 q<m，不會越界。

完整匹配之後仍能接重疊匹配

P=aabaaab、π[7]=3，取 T=aabaaabaaab：

i	T[i]	比較前q	fallback後／延伸後q	輸出後保存q
1	a	0	1	1
2	a	1	2	2
3	b	2	3	3
4	a	3	4	4
5	a	4	5	5
6	a	5	6	6
7	b	6	7，輸出0	3
8	a	3	4	4
9	a	4	5	5

完整匹配之後仍能接重疊匹配 （續）

i	T[i]	比較前q	fallback後／延伸後q	輸出後保存q
10	a	5	6	6
11	b	6	7，輸出4	3

兩次匹配共用位置5–7的 aab。清零會丟掉這段已知相等的證據。

一個字元可多次 fallback，總成本仍線性

狀態 q=6、下一字元 c 時，依序比較 P[7]=b、P[3]=b、P[2]=a、P[1]=a，都不等 c；鏈6→2→1→0，最後保存0。同一個 c 被比較多次，但文字索引 i 沒有倒退。

掃描中 q 初始0，每處理一字最多增加1，總增加量≤n。每次 while body 至少減1，輸出後的 q←π[m] 也只減不增；q 從不負。因此 while body 的總次數≤n，另加每字常數次外圈檢查及輸出，匹配 Θ(n)。相同記帳適用建 π：總增加≤m−1，while body總次數≤m−1，建表 Θ(m)。

總時間 Θ(n+m)，額外工作空間 Θ(m)，不含輸出。這是整段序列的界，不是說每個字元最壞 O(1)；若需每字固定最壞轉移時間，可改付完整自動機表的空間。

四種方法使用了哪些假設

方法	前處理	掃描最壞時間	模式額外空間
Naive	無	Θ((n−m+1)m)	O(1)
RK，hash相等後驗證	Θ(m)	Θ((n−m+1)m)	O(1)
完整自動機，π建表	Θ(m	Σ	)
KMP	Θ(m)	Θ(n)	Θ(m)

本表採1≤m≤n、固定成本字元及word模算術，預處理與掃描分開列。RK期望快需要額外亂數／碰撞條件；KMP線性不需要隨機輸入或隨機模式。Unicode實作要先選定匹配單位是byte、code point或grapheme cluster；本章不把不同編碼的字元序列自動視為相同字串。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 32。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

28 矩陣運算

Part VII Selected Topics

本章定位

消去法必須同步保存置換與先前乘數；本章由LUP完整算法推導求逆、Schur補及乘法歸約，再以SPD與正交殘差連結Cholesky和最小平方，區分精確算術、rank與數值敏感度。

前置知識

線性代數、高斯消去與最小平方。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用 LUP 解線性系統並解釋置換。
- 用分塊與 LU 分析矩陣反轉。
- 處理 SPD 矩陣與最小平方正規方程。
- 28.1 Solving systems of linear equations
- 28.2 Inverting matrices
- 28.3 Symmetric positive-definite matrices and least-squares approximation

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

28.1 Solving systems of linear equations

本章先在實數、精確算術模型討論。n×n矩陣A若非奇異，每個b都有唯一解Ax=b；若奇異，可能無解或無限多解，不能以「消去遇零」直接斷言所有b都無解。非奇異也不代表不換row就能消去，例如A=[0 1;1 0]可逆但第一個對角pivot是0。

LUP分解的契約是 **PA=LU**：P交換row，L為對角全1的下三角矩陣，U為對角非零的上三角矩陣。P用陣列p表示：目前第i個row來自原第p[i]個row，所以(Pb)[i]=b[p[i]]。這是row置換，不是對未知向量x的column置換。

由PAx=Pb與PA=LU，先解Ly=Pb，再解Ux=y。回代可得$Ax=P^{-1}Ly=P^{-1}Pb=b$，並由非零對角與P可逆得到唯一性。若P=I才可簡寫A=LU；不能一邊假設A=LU，一邊無來由把右端改成Pb。

前向與後向代入的依賴

L為unit lower triangular，依i遞增：$y[i]=b[p[i]]-\Sigma_{j<i} L[i,j]y[j]$。算第i項時，所有右側的y[j]已知；代入保證前i列Ly=Pb成立。

依i遞減：$x[i]=(y[i]-\Sigma_{j>i} U[i,j]x[j])/U[i,i]$。算第i項時，後面的未知已解；回代後最後n−i+1列Ux=y成立。對角非零才可除。兩個不變量分別完成全部列，這不是把未知量「同時猜出」。

```
Solve(L,U,p,b):
  for i=1..n:
    y[i]=b[p[i]]-sum(j=1..i-1,L[i,j]*y[j])
  for i=n..1:
    x[i]=(y[i]-sum(j=i+1..n,U[i,j]*x[j]))/U[i,i]
  return x
```

各做n(n−1)/2個求和乘積，加減與n次除法，時間$\Theta(n^2)$，額外向量空間O(n)。已分解的A可重用於r個右端，總時間$\Theta(n^3+rn^2)$；每個b都重新分解會浪費已得的相同因子。

消去、Schur Complement 與存在性

若目前第一個pivot為$a\neq 0$，把待處理矩陣寫成$[A]=[a\ w^{T};v\ B]$。消去的恆等式是

$[a\ w^{T};v\ B]=[1\ 0;v/a\ I]\cdot[a\ w^{T};0\ B-vw^{T}/a]$。

右下$S=B-vw^{T}/a$稱Schur complement。左因子可逆，因此若原矩陣非奇異，右因子亦非奇異；$a\neq 0$時，若S有非零z滿足$Sz=0$，向量$(-w^{T}z/a,z)$會落在右因子的nullspace，矛盾。因此S非奇異，可繼續遞迴。

若pivot column全部是0，當前S有零column、必奇異，連同已消去的非零pivots推出原A奇異。反之非奇異S的第一column總有非零項，換到pivot位置後，上述推理可持續直到n。這證明**每個非奇異A都存在LUP**，但沒有證明不置換的LU總存在。

原地 LUP：換 Row 必須包含先前因子

work的上三角含U、對角下方儲存L的乘數，L的對角1不另存。以下交換整個row，包含columns $1..k-1$已存的L乘數。

```
work=copy(A); p=[1,..,n]
for k=1..n:
  r=argmax(i=k..n,abs(work[i,k]))
  if work[r,k]==0: 回報singular
  swap(work[k,1..n],work[r,1..n]); swap(p[k],p[r])
  for i=k+1..n:
    work[i,k]=work[i,k]/work[k,k]
    for j=k+1..n:
      work[i,j]=work[i,j]-work[i,k]*work[k,j]
```

為何舊L也要換？第k輪之前已有$PA=[L_{11}\ 0;L_{21}\ I]\cdot[U_{11}\ U_{12};0\ S]$。只交換active rows的置換R會把S變RS，也把L_{21}變RL_{21}，否則兩因子乘積的下半row仍指向錯的原方程。接著本輪消去再把新乘數併入L，維持$PA=LU$的分塊不變量。

$\Sigma_{k}(n-k)^{2}=\Theta(n^{3})$個尾塊更新；找pivot共$\Theta(n^{2})$，不改立方級序。這些是算術操作界，並非任意長有理數的位元成本界。

兩次 Row Swap 的完整數例

取A=[0 2 1;1 1 0;2 3 4]、b=(0,0,7)ᵀ。第一column選abs最大2，交換row1、3，p=(3,2,1)。乘數 $m_{21}=1/2$、$m_{31}=0$，尾塊成[−1/2 −2;2 1]。

第二column選abs最大2，再交換目前row2、3，p=(3,1,2)。**L第一column的1/2也從row2移到row3**。新乘數$m_{32}=(-1/2)/2=-1/4$，最後pivot為$-2-(-1/4)\cdot 1=-7/4$。

L=[1 0 0;0 1 0;1/2 −1/4 1]，U=[2 3 4;0 2 1;0 0 −7/4]。

逐row乘LU得到[2 3 4;0 2 1;1 1 0]，恰為PA依原row3、1、2重排。若只換尾塊而沒換舊L，row2的L乘數會錯留1/2，LU≠PA。

兩次 Row Swap 的完整數例 （續）

兩次 Pivot：尾塊與已存 L 必須同步

綠格 = 已存L乘數；藍格 = U或尚待消去的尾塊。L對角1不另存。

原A

0	2	1
1	1	0
2	3	4

p=(1,2,3)

→ 換row1/3，再消去

2	3	4
1/2	-1/2	-2
0	2	1

p=(3,2,1)

→ 換row2/3，再消去

2	3	4
0	2	1
1/2	-1/4	-7/4

p=(3,1,2)

第2次交換時，第一column的1/2必須從目前row2移到row3。

最後 m31=1/2、m32=−1/4、U33=−7/4，對應PA=LU。

同一數例的前後代入與穩定性界線

Pb=(7,0,0)ᵀ。前向代入：$y_1=7$、$y_2=0$、$y_3=0-(1/2)\cdot 7-(-1/4)\cdot 0=-7/2$。

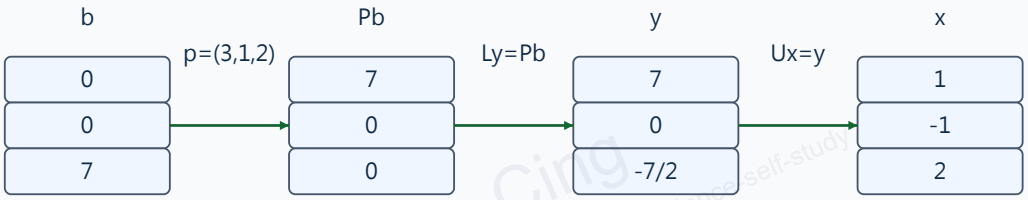
後向代入：$x_3=(-7/2)/(-7/4)=2$；$x_2=(0-1\cdot 2)/2=-1$；$x_1=(7-3\cdot(-1)-4\cdot 2)/2=1$。代回原A，三列分別 $0-2+2=0$、$1-1=0$、$2-3+8=7$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一數例的前後代入與穩定性界線 （續）

從 b 到 Pb、y、x：兩個三角系統

同一A與b；row置換只重排方程右端，沒有重排未知量x。



前向：y3=0−(1/2)·7=−7/2。

後向：x3=2，x2=−1，x1=1；代回原A得(0,0,7)。

Partial pivoting（部分選主元）只在當前column選abs最大值，精確算術保證本輪$|m_{ik}|\le 1$；它防止此column產生過大乘數，**不是對所有矩陣都保證浮點誤差小**。消去仍可能有growth，病態A也可能把小資料擾動放大。數學上的「非零pivot」與程式採用的數值tolerance不是相同定義；近奇異判定需連同尺度和誤差模型說明。

28.2 Inverting matrices

A^{-1}的第j個column是$Ax=e_j$的解；先LUP一次，再做n次代入，$\Theta(n^3)+n\cdot\Theta(n^2)=\Theta(n^3)$。輸出必須核對$AA^{-1}=I$；在精確算術，方陣的右逆就是唯一逆。解單一Ax=b通常直接代入，不必先求逆再乘b。

例A=[1 1;2 5]，U=[1 1;0 3]、L=[1 0;2 1]。對e_1：y=(1,−2)，回代$x_2=-2/3$、$x_1=5/3$。對e_2：y=(0,1)，回代$x_2=1/3$、$x_1=-1/3$。因此A^{-1}=[5/3 −1/3;−2/3 1/3]，不是各entry倒數。

乘回左上5/3−2/3=1、右上−1/3+1/3=0、左下10/3−10/3=0、右下−2/3+5/3=1。若A奇異，某些特定b仍可解，也不會使這n個單位向量都具有唯一解，因此不能交付同一定義的A^{-1}。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Block Inverse 的次序與前提

令A=[B C;D E]，先要求B可逆，設$S=E-DB^{-1}C$。分解A=[I 0;DB^{-1} I]・[B C;0 S]；若A也可逆，S必可逆。把兩因子的逆依**反序**相乘，得到

```
A⁻¹ = [ B⁻¹+B⁻¹CS⁻¹DB⁻¹    −B⁻¹CS⁻¹ ]
      [ −S⁻¹DB⁻¹              S⁻¹     ]
```

矩陣乘法不交換，不能把C、D或逆的位置任意互換。上例B=1,C=1,D=2,E=5，S=3，左上1+1・1・(1/3)・2・1=5/3，得到與逐column求逆完全相同的矩陣。

這個特定分塊程序不能對所有可逆A直接遞迴。例如[0 1;1 0]可逆，但選B=0就不能形成B^{-1}。下一節的SPD性質保證leading block與Schur都可逆；一般矩陣要加置換或先使用下述理論歸約。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Block Inverse 的次序與前提 （續）

Schur Complement：消去後的確切 Block

左乘消去矩陣 [I 0;−DB⁻¹ I]；B需可逆，乘法次序不可交換。

A

B	C
D	E

→ 消去後

B	C
0	E−DB⁻¹C

S=E−DB⁻¹C；A=[I 0;DB⁻¹ I]·[B C;0 S]。

例B=1,C=1,D=2,E=5：S=3，A⁻¹=[5/3 −1/3;−2/3 1/3]。

若B=0，不能先寫B⁻¹；整個A可逆也不能省掉此前提。

乘法與反矩陣的兩向歸約

給兩個n×n矩陣B,C，構造3n×3n的T=[I B 0;0 I C;0 0 I]，則T^{-1}=[I −B BC;0 I −C;0 0 I]，乘回的右上block為BC−BC=0。只要inversion成本I(n)=Ω(n^2)且I(3n)=O(I(n))，建構Θ(n^2)加一次求逆即可在O(I(n))取出BC。必須寫出維度與regularity，不能只說兩問題「看起來相似」。

反方向，先對SPD矩陣分塊。B與S也SPD（下一節證明），可遞迴逆兩個n/2方塊，再做常數個矩陣product：T(n)≤2T(n/2)+O(M(n))。若M(n)=Ω(n^2)、padding到不到2n只改常數，且M(n/2)≤cM(n)、c<1/2，展開後第ℓ層至多(2c)ˡO(M(n))，等比級數收斂，故T(n)=O(M(n))。M(n)=Θ(n^ω)、ω≥2滿足條件。

一般實非奇異A的G=A^{T}A為SPD，因z^{T}Gz=$||Az||^2$>0。A^{-1}=$G^{-1}A^{\mathsf{T}}$，乘右側A可驗為I，總成本仍O(M(n))。非2冪維度可padding成diag(A,I)，逆的原block不變。這是**精確算術複雜度歸約**；浮點實作不宜為了這個證明特意形成A^{T}A，因其condition number會平方。複數版本改用A*，不能用普通轉置宣稱正定。

28.3 Symmetric positive-definite matrices and least-squares approximation

實對稱正定SPD意指$A=A^{\mathsf T}$且對每個非零z皆有$z^{\mathsf T}Az>0$。若A奇異，存在非零z使$Az=0$，會給$z^{\mathsf T}Az=0$，矛盾，因此SPD必非奇異。對任意leading block B，將其非零向量z補零成$(z,0)$，其二次型仍正，故B亦SPD。

對SPD分塊$A=[B\ C;C^{\mathsf T}\ E]$，$S=E-C^{\mathsf T}B^{-1}C$對稱。取任意非零$y$，令$z=-B^{-1}Cy$；則

$(z,y)^{\mathsf T}A(z,y)=z^{\mathsf T}Bz+2z^{\mathsf T}Cy+y^{\mathsf T}Ey=y^{\mathsf T}(E-C^{\mathsf T}B^{-1}C)y=y^{\mathsf T}Sy>0$。

所以S也SPD。第一個pivot為$e_1^{\mathsf T}Ae_1=A_{11}>0$；消去後繼續處理SPD Schur，歸納全部pivot皆正。這才給出SPD不用pivoting也不會除0的完整理由；單憑「對角線原本都正」不夠，例如$[1\ 2;2\ 1]$對角正，第二pivot卻為-3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

$LDL^{\mathsf T}$ 與 Cholesky 的具體計算

對稱消去每步得到正pivot d_j與unit-lower乘數，可歸納$A=L\,D\,L^{\mathsf T}$，其中D是全正對角。令$C=L\,D^{1/2}$，就得Cholesky $A=CC^{\mathsf T}$，C為正對角下三角。也可直接依column計算：

$C[j,j]=\mathrm{sqrt}(A[j,j]-\Sigma_{k<j}\,C[j,k]^2)$，$C[i,j]=(A[i,j]-\Sigma_{k<j}\,C[i,k]C[j,k])/C[j,j]$，$i>j$。

已證的SPD Schur讓每個sqrt內為正，因此算法可以繼續。三重求和為$\Theta(n^3)$，只需保存一個triangle；常見flop計數下約$n^3/3$，與一般LU約$2n^3/3$的領先項不同，這不表示所有機器上精確快兩倍。

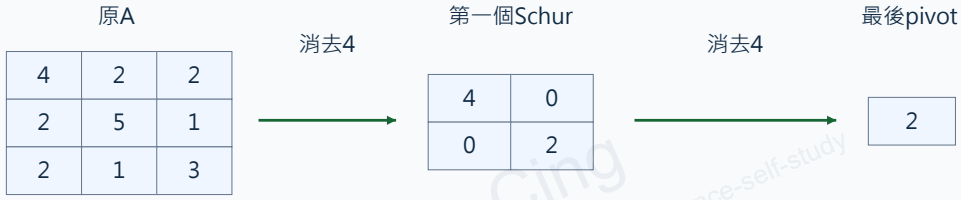
例$A=[4\ 2\ 2;2\ 5\ 1;2\ 1\ 3]$：$C_{11}=2$，$C_{21}=C_{31}=1$；$C_{22}=\mathrm{sqrt}(5-1)=2$；$C_{32}=(1-1\cdot 1)/2=0$；$C_{33}=\mathrm{sqrt}(3-1-0)=\sqrt{2}$。因此$C=[2\ 0\ 0;1\ 2\ 0;1\ 0\ \sqrt{2}]$，乘回三個對角為4、5、3，兩個非對角2、2與另一個1均相符。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LDL^T 與 Cholesky 的具體計算 （續）

正定 Schur 讓 Cholesky 每步合法

SPD消去的每個pivot都正；本例依序4、4、2。



C rows=[2,0,0]、[1,2,0]、[1,0,√2]。

CC^{T}對角為4,5,3，非對角為2,2,1，逐格回到原A。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Least Squares：必要條件與全域最小的充分性

給$A\in\mathbb{R}^{m\times n}$、$b\in\mathbb{R}^m$，目標是$\min_x f(x)=||Ax-b||_2^2$。這個「對係數線性」模型也可用basis functions：$A[i,j]=f_j(t_i)$，不要求f_j對t是直線，例如1,t,t^2仍對係數線性。

沿任意方向z展開：$f(x+tz)=f(x)+2t\,z^TA^T(Ax-b)+t^2||Az||^2$。若x是最小值，所有方向的一次項都必為0，否則選足夠小且相反號的t可下降，所以$A^TAx=A^Tb$。反過來若此正規方程成立，t=1時$f(x+z)=f(x)+||Az||^2\geq f(x)$，證明它確為全域最小，不只是stationary point。

A^TA總為對稱半正定，$z^TA^TAz=||Az||^2$。它是SPD **當且僅當columns線性獨立**。full column rank時唯一解；rank不足時係數有多解，但預測Ax仍唯一：兩個最小解差z滿足Az=0。至少一個解總存在，可取A中獨立columns作basis，先解該full-rank子矩陣的SPD正規方程，其殘差垂直basis，也垂直所有columns；若A全零，則直接取x=0、p=0，殘差為b。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Full-column-rank 的 Pseudoinverse 與投影

若A有full column rank，定義$A^{+}=(A^{T}A)^{-1}A^{T}$，維度n×m。正規方程給$\hat{x}=A^{+}b$。$A^{+}A=I_{n}$，但一般AA^{+}不是I_{m}；它是$Q=A(A^{T}A)^{-1}A^{T}$，滿足$Q^{T}=Q$與$Q^{2}=Q$，且對每個$Az\in Col(A)$，$QAz=Az$。因此Q把b投到Col(A)，殘差$(I-Q)b$滿足$A^{T}(I-Q)b=0$。

對下頁3×2的design matrix，A^{+} rows為(5/6,1/3,−1/6)、(−1/2,0,1/2)。乘b=(0,2,1)得到(1/2,1/2)。Q rows為(5/6,1/3,−1/6)、(1/3,1/3,1/3)、(−1/6,1/3,5/6)，顯然不等於3×3單位矩陣；乘b得到(1/2,1,3/2)，不是原b。

rank不足時$(A^{T}A)^{-1}$未定義，這個公式不能直接使用。更一般的Moore–Penrose pseudoinverse可由SVD構造；本章的公式與唯一係數結論限full column rank，不把奇異Gram硬求逆。

三個不共線資料點的完整擬合

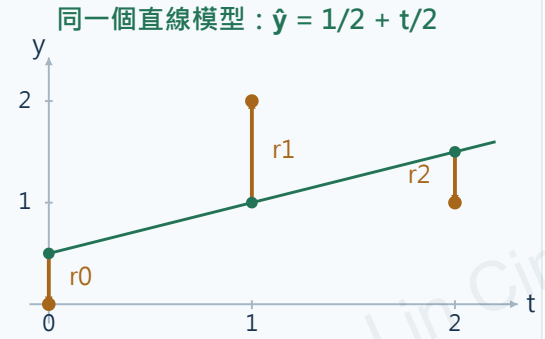
保留三點(t,y)=(0,0),(1,2),(2,1)，設$x=(c,m)^{T}$、模型$\hat{y}=c+mt$。

A=[1 0;1 1;1 2]，b=$(0,2,1)^{T}$，$A^{T}A$=[3 3;3 5]，$A^{T}b=(3,4)^{T}$。方程3c+3m=3、3c+5m=4相減得m=1/2，c=1/2。

t	觀測y	預測$\hat{y}$	$r=y-\hat{y}$
0	0	1/2	−1/2
1	2	1	1
2	1	3/2	−1/2

$A^{T}r$的兩分量為−1/2+1−1/2=0與0・(−1/2)+1・1+2・(−1/2)=0；$||r||^{2}=3/2$。columns(1,1,1)與(0,1,2)獨立，才是唯一係數的原因；**資料點是否共線不決定這個design matrix的column rank**。若仍用相同t、改y=(1,2,3)，三資料共線，columns仍獨立，只是殘差變0。

三個不共線資料點的完整擬合 （續）

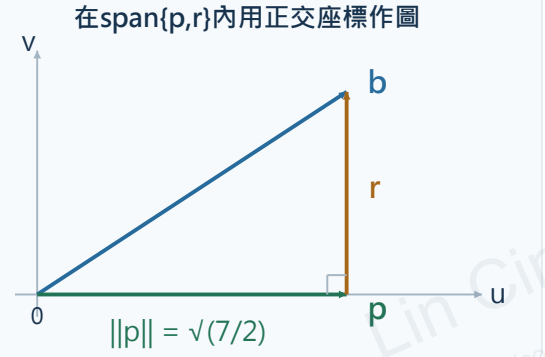


t	y	ŷ	r
0	0	1/2	−1/2
1	2	1	1
2	1	3/2	−1/2

$r^T r = 1/4 + 1 + 1/4 = 3/2$

橘色：固定t的垂直誤差；綠色：預測點與擬合線

把三個觀測值視為R^3中的一個向量



R^3座標

b = (0, 2, 1)
p = (1/2, 1, 3/2)
r = (−1/2, 1, −1/2)

u = p / ||p||
v = r / ||r||

$A^T r = (0, 0)$
$||r||^2 = 3/2$

截面E=span{p,r}；E與Col(A)的交集是水平方向span{p}

本例p、r非零且正交，取u=p/||p||、v=r/||r||作正交單位座標軸；右側列出的b、p、r則是原本的R^3座標。$A^T r=0$使r垂直整個Col(A)。截面E=span{p,r}只沿span{p}與Col(A)相交，圖中的水平線只表示這個交集；Col(A)本身仍是R^3中的二維平面。

正交殘差、Condition Number 與 Rank 缺失

設$p=A\hat{x}$、$r=b-p$。$A^Tr=0$表示r垂直整個Col(A)，且$b=p+r$。對任一係數$\hat{x}+z$，$b-A(\hat{x}+z)=r-Az$，兩項正交，所以平方誤差$=||r||^2+||Az||^2$；這把代數條件連到最小距離幾何。

在(t,y)的散點圖，r_i是固定t_i時的**垂直y誤差**，不是該點到擬合線的最短距離。在觀測向量圖中，整組b是R^3中的一個向量，r才是點b到子空間Col(A)的最短距離向量；兩個圖使用不同的空間。

若columns重複，例如A=[1 1;1 1;1 1]、b=(0,2,1)，只決定$c_1+c_2=1$；全部$(c_1,c_2)=(s,1-s)$都是解，預測仍為(1,1,1)，殘差(−1,1,0)唯一。

full column rank時，2-norm條件數$\kappa_2(A^TA)=\kappa_2(A)^2$，因奇異值變成平方。例diag$(1,10^{-3})$的κ為$10^3$，Gram的κ為$10^6$；這是數值敏感度，不是浮點算術次數。實作常用QR或SVD求least squares，尤其接近rank缺失時；正規方程的精確證明不能當成所有浮點資料都穩定的保證。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 28。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

30 多項式與快速傅立葉轉換

Part VII Selected Topics

本章定位

多項式乘法藉相異點求值與插值轉成逐點乘法；本章從單位根正交性證明DFT與inverse，再逐層核對偶奇分解、bit-reversal、蝶形索引與補零，連結精確算術和浮點限制。

前置知識

複數根、點值表示與遞迴分治。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 比較係數與點值表示的乘法成本。
- 推導 DFT 與逆 DFT。
- 實作 radix-2 FFT 並理解單位根條件。
- 30.1 Representing polynomials
- 30.2 The DFT and FFT
- 30.3 Efficient FFT implementations

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

30.1 Representing polynomials

係數表示$A(x)=\Sigma_{j=0}^{n-1}a_jx^j$的degree bound是n，也就是degree<n；最高係數可以為0，實際degree可能更小。零多項式在程式可用單一係數[0]表示，不把degree bound與向量中第一個非零位置混淆。以下係數依**低次到高次**排列。

同一degree bound的加法逐係數相加，$\Theta(n)$。在一點z求值可用Horner：先$v=a_{n-1}$，再對$j=n-2,\cdots,0$做$v=a_j+zv$，每次保持$v=\Sigma_{r=j}^{n-1}a_rz^{r-j}$，最後得到$A(z)$，共$n-1$次乘與$n-1$次加。任選n個點分別做Horner，則$\Theta(n^2)$，不是$\Theta(n)$。

兩個長度r、s係數陣列相乘，$c_k=\Sigma_{i+j=k}a_ib_j$，輸出長$r+s-1$；直接卷積有rs次係數乘法，$\Theta(rs)$。FFT將改善的是整個表示轉換加乘法流程，不只是中間的pointwise product。

相異點值的存在性與唯一性

給n個相異$x_0,\cdots,x_{n-1}$及值$y_0,\cdots,y_{n-1}$。Lagrange公式$A(x)=\Sigma_k y_k\ell_k(x)$，其中$\ell_k(x)=\Pi_{j\neq k}(x-x_j)/(x_k-x_j)$，每個分母非零、degree<n，且$\ell_k(x_i)$只有i=k時為1，其餘為0，故插值多項式存在。

若兩個degree<n多項式同時吻合，差H有n個不同根。非零degree d多項式至多d根：遇根α可依因式定理寫$H=(x-\alpha)Q$，其餘不同根亦為Q的根，對degree歸納即可。因此H必為零，唯一性成立。少於n點時，可在一個解上加任意常數倍Π已知點$(x-x_j)$，仍吻合，故一般不唯一。

矩陣寫法是$V[i,j]=x_i^j$、$Va=y$，V稱Vandermonde matrix。其$\det=\Pi_{j<k}(x_k-x_j)$：兩個x相同會有相同row，故每個差都是det多項式的因子；總degree正好$n(n-1)/2$，對照單項$x_1x_2^2\cdots x_{n-1}^{n-1}$的係數1便確定常數。因此相異點讓V可逆，也可用一般線性系統求插值，但直接LUP是$\Theta(n^3)$。

公式也可在$O(n^2)$算術操作內轉成係數：先逐因子建$Q(x)=\Pi_j(x-x_j)$，每步更新總$O(n^2)$；對每個k用synthetic division在線性時間求$Q/(x-x_k)$，分母各用$O(n)$算，再逐係數加總。若每次從頭重建每個ℓ_k的多項式，則不能直接宣稱相同的二次成本。

點值相乘需要足夠長度

在**相同且相異的N個點**存A、B的值，$C(x_k)=A(x_k)B(x_k)$只需N次乘法。要唯一重建乘積，須$N \geq r+s-1$，並將兩個係數陣列都補零到N；radix-2選不小於該值的2冪。

例如$A=1+2x$、$B=1-x$，$r=s=2$，乘積$1+x-2x^2$需3係數，所以選$N=4$。不能只因兩個輸入都長2就直接做2點轉換。

若在N次單位根只留N點，$x^N=1$，所以插值得到的是模x^N-1的循環卷積：位置k收到$c_k+c_{k+N}+c_{k+2N}+\cdots$，高次項會繞回。例$(1+x)^2=1+2x+x^2$，$N=2$時$x^2$與常數無法區分，反轉只得[2,2]；[1,2,1]才是普通乘積。補零不是可有可無的格式處理。

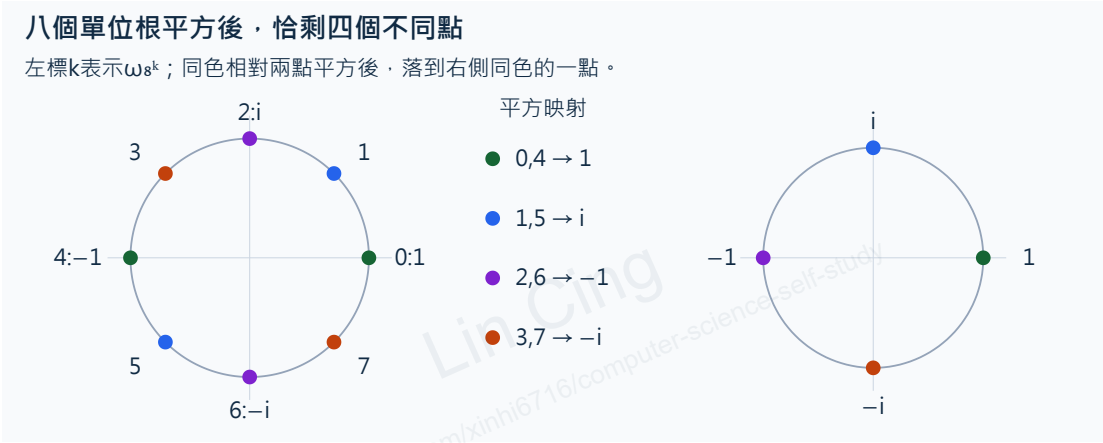
30.2 The DFT and FFT

本章固定**正指數forward**：$\omega_n=e^{2\pi i/n}$，$y_k=\sum_{j=0}^{n-1}a_j\omega_n^{jk}=A(\omega_n^k)$。其他領域常把forward指數設負；只要正逆成對就成立，但不能混用兩種慣例。

ω_n是primitive root，亦即$\omega_n^k=1$當且僅當n整除k。其n個冪相異，構成合法插值點。對正整數d有$\omega_{dn}^{dk}=e^{2\pi ik/n}=\omega_n^k$（cancellation）。n偶數時$\omega_n^{n/2}=-1$，且$(\omega_n^k)^2=\omega_{n/2}^k$；k與k+n/2的平方相同，全部n個點平方後只剩n/2個不同點，各出現兩次（halving）。

幾何級數給正交性：$\sum_{j=0}^{n-1}\omega_n^{jt}$在$n \mid t$時每項1、和n；否則$\omega_n^t \neq 1$，和$=(1-(\omega_n^t)^n)/(1-\omega_n^t)=0$。分母是否非零正是primitive條件要處理的地方，不能把$\omega=1$當成任意n的替代root。

30.2 The DFT and FFT （續）



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正交性推導 Inverse DFT

定義$F[k,j]=\omega_n^{jk}$。令$G[j,k]=(1/n)\omega_n^{-jk}$，則$(GF)[j,\ell]=(1/n)\Sigma_k\omega_n^{k(l-j)}$。j=ℓ時為1，其餘因$0\le j,\ell<n$而n不整除ℓ−j，為0，所以GF=I，G就是$F^{-1}$。

因此$a_j=(1/n)\Sigma_k y_k\omega_n^{-jk}$：使用負指數twiddle，**最後除n一次**。也可每層分攤正規化，但不能每層除n，更不能完全漏除。

若錯把forward再做一次後除n，利用同一和式得到F^2a/n的第j項是a[(-j) mod n]，是循環索引反轉；對任意複數輸入不等於「取共軛」。n=4、a=[2,5+i,7,−1]會變[2,−1,7,5+i]，可直接分辨兩種錯法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

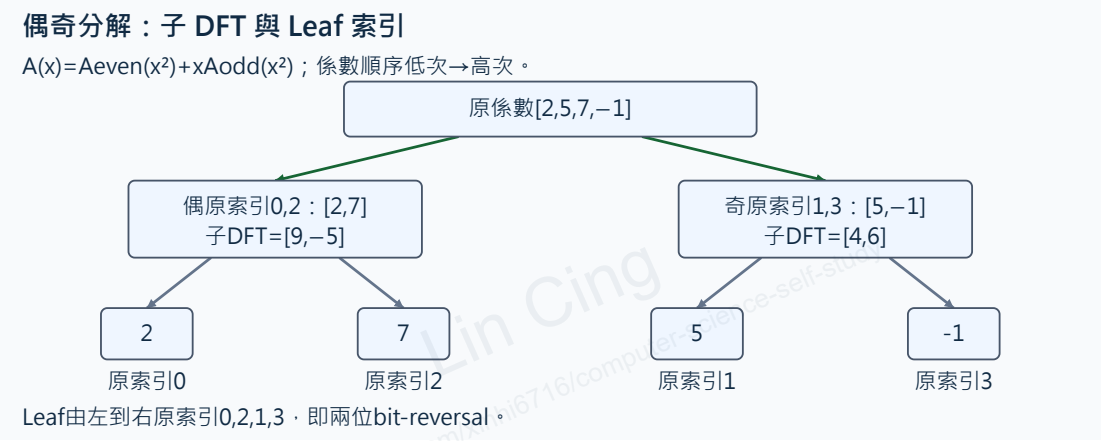
偶奇拆解的兩個子問題

設n是正2冪，$A_e(z)=a_0+a_2z+\cdots$、$A_o(z)=a_1+a_3z+\cdots$，則$A(x)=A_e(x^2)+xA_o(x^2)$。由halving lemma，在全部n個root求A只需兩個n/2點DFT E、O。

對$0\le k<n/2$，$y_k=E_k+\omega_n^kO_k$，$y_{k+n/2}=E_k-\omega_n^kO_k$，因$\omega_n^{k+n/2}=-\omega_n^k$。一次算$t=\omega_n^kO_k$，使用加、減產生兩個輸出，就是butterfly。遞迴基例n=1直接回$[a_0]$；歸納子轉換正確，再用等式合併全部n個值，證明整個FFT正確。

取a=[2,5,7,−1]。偶係數[2,7]的2點DFT為[9,−5]；奇係數[5,−1]為[4,6]。k=0用twiddle1，得$y_0=13$、$y_2=5$；k=1用i，得$y_1=-5+6i$、$y_3=-5-6i$。四個中間值彼此不同，方便分辨接線是否真正正確。

偶奇拆解的兩個子問題 （續）



遞迴算法與完整成本

```
FFT(a,sign):
  n=len(a)；要求n為正2冪
  if n==1: return [a[0]]
  E=FFT([a[0],a[2],...], sign)
  O=FFT([a[1],a[3],...], sign)
  w=1; wm=exp(sign*2πi/n)
  for k=0..n/2-1:
    t=w*O[k]
    y[k]=E[k]+t; y[k+n/2]=E[k]-t
    w=w*wm
  return y
```

forward取sign=+1；inverse取−1，最外層回來後除n。每層Θ(n)拆分/合併，T(n)=2T(n/2)+Θ(n)，$\log_2 n$層，Θ(n log n)複數算術。n/2 butterflies每層，共$(n/2)\log_2 n$個。

複製偶奇陣列且及時釋放子結果的順序實作，可用O(n)峰值額外空間與O(log n)stack；若保存所有層的陣列歷史則O(n log n)。不能只由遞迴深度就宣稱全部記憶體O(log n)，因每層還有輸出與子陣列。

卷積的 Forward、點乘、Inverse 全部算值

A=[1,2,0,0]、B=[1,−1,0,0]，N=4、$\omega_4=i$。

k	DFT(A)	DFT(B)	逐點乘積
0	3	0	0
1	1+2i	1−i	3+i
2	−1	2	−2
3	1−2i	1+i	3−i

對[0,3+i,−2,3−i]做負號DFT，未除4的結果[4,4,−8,0]；除4得[1,1,−2,0]，截取前3項即$1+x-2x^2$。兩次forward加一次inverse與Θ(N)點乘，總Θ(N log N)，輸出degree bound不足N的條件使循環與普通卷積一致。

卷積的 Forward、點乘、Inverse 全部算值 (續)

一次卷積的全部頻點與反插值

A=1+2x，B=1−x，均補到N=4；$\omega_4=i$，forward取正指數。

k	DFT(A)	DFT(B)	逐點乘積
0	3	0	0
1	1+2i	1−i	3+i
2	−1	2	−2
3	1−2i	1+i	3−i

負號DFT先得[4,4,−8,0]，除4→[1,1,−2,0]。

綠格 = 乘積點值；取前3個係數，得到$1+x-2x^2$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

30.3 Efficient FFT implementations

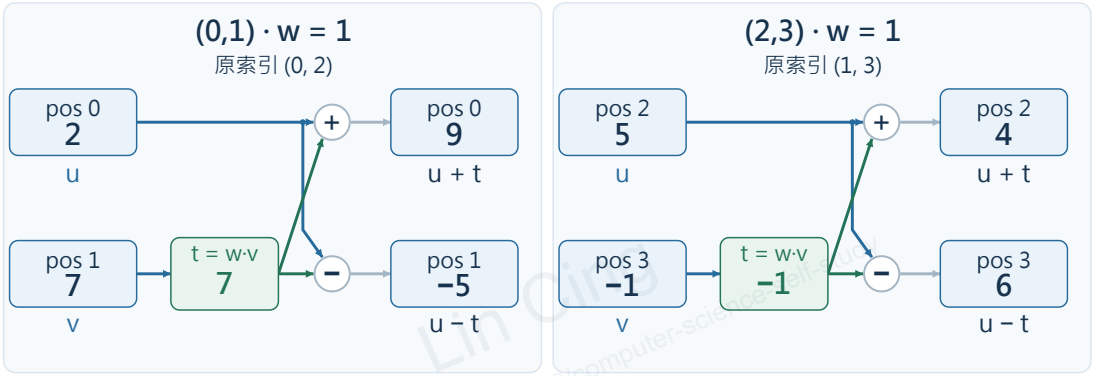
遞迴每層先取偶索引再取奇索引，也就是先看原索引最低位，再看下一位。若$n=2^b$，從左到右的leaf順序就是把原索引b個bits反轉後的順序。rev是自身的逆，因此可寫A[rev(j)]=a[j]，也可寫A[j]=a[rev(j)]。

n=4：原索引00,01,10,11反轉為00,10,01,11，輸入[2,5,7,−1]重排為[2,7,5,−1]。重排後stage1配對的是**實體位置(0,1)、(2,3)**，分別含原索引(0,2)、(1,3)；不能把「原索引」與「目前陣列位置」寫成同一個欄位。

stage1兩個twiddle皆1，實體陣列變[9,−5,4,6]。stage2的m=4，配對實體位置(0,2)與(1,3)，依序使用1與i，輸出按位置0,1,2,3是[13,−5+6i,5,−5−6i]。stage2若又配相鄰位置就是另一張圖、另一個運算。

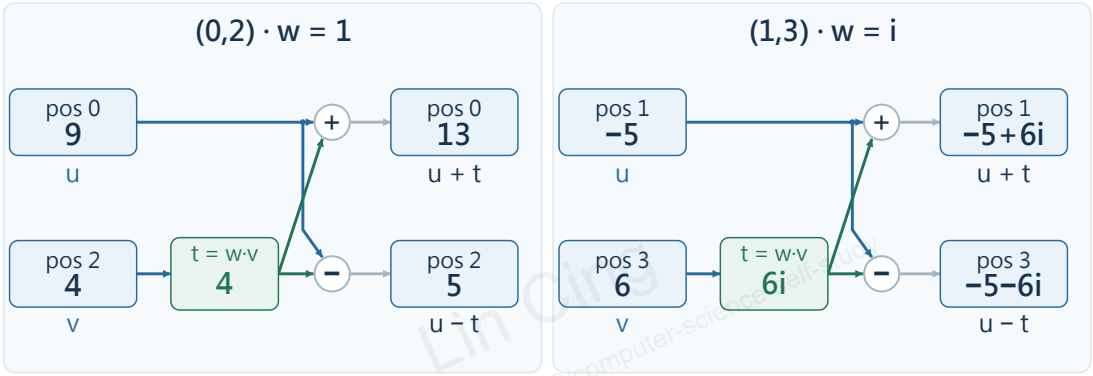
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

30.3 Efficient FFT implementations （續）



Stage1：實體陣列 [2,7,5,−1] → [9,−5,4,6]

Stage2 必須跨半塊合併



Stage2：依實體位置0,1,2,3輸出 [13,−5+6i,5,−5−6i]

j	u=A[j]	t=ω₄ʲA[j+2]	新A[j]	新A[j+2]
0	9	1 • 4=4	13	5
1	−5	i • 6=6i	−5+6i	−5−6i

Stage2 必須跨半塊合併 （續）

另一組a=[1,2,0,−1]依同一規則：bit-reversal→[1,0,2,−1]，stage1→[1,1,1,3]，stage2→[2,1+3i,0,1−3i]。這個例子有多個相同1，單看最終數字可能掩蓋接線錯誤；上面的不同中間值讓每條運算的來源更清楚。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Iterative FFT 的實體索引契約

```
將a按bit-reversal放入A
for s=1..log₂n:
  m=2^s; wm=exp(sign*2πi/m)
  for base=0,m,2m,...,n−m:
    w=1
    for j=0..m/2−1:
      u=A[base+j]; t=w*A[base+j+m/2]
      A[base+j]=u+t
      A[base+j+m/2]=u−t
      w=w*wm
inverse時最後每項除n
```

每輪s開始，長度m區塊的左右半塊各已是相應偶／奇子問題的DFT；butterfly做與遞迴相同的合併，輪末每塊為完整m點DFT。s=1時半塊為單點，基例成立；最後m=n完成整體。先保存u再覆寫上格，否則下格的u−t會誤讀已更新值。

每stage恰n/2個butterflies，共Θ(n log n)。逐索引反轉b位元的簡單重排需O(n log n)位元步驟；可用額外n長陣列，或利用rev的2-cycle就地交換。twiddle只依m與j，不依base，同stage各block可共用其表，若存所有stage表總Θ(n)項；這不改變主要算術級序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Parallel FFT Circuit 的 Work 與 Depth

固定twiddle並把bit-reversal視為接線的bounded-fan-in電路，每stage n/2個互不衝突butterflies可同時做；每stage的輸出只有下一stage才讀。n=8有3stage、共12個butterflies，電路work $\Theta(n \log n)$、depth $\Theta(\log n)$，因每條輸入到輸出的路徑穿過log n次合併。

這個depth計算假設一次複數乘、加各為常數層，且沒有把twiddle串行生成、處理器通訊或實體wire長度藏在其中。軟體多執行緒實作若共用陣列，須在每stage結束同步；同stage各butterfly寫不同兩格，下一stage讀前一stage結果，才能使用相同DAG分析。若僅有P處理器，work下界W/P仍在，不會因電路畫成log n層就自動只需log n時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

精確算術與浮點 FFT 的界線

$\Theta(n \log n)$是複數算術操作模型；浮點root與每次乘加都有誤差。整數係數卷積最後四捨五入要有「每格絕對誤差<1/2」的保證，不能只因少數round-trip吻合就保證任意長度／大係數都精確。

一種精確替代是在質數域F_p用NTT：要求$N \mid (p-1)$，選order恰為N的ω，則N在域中可逆，幾何和與inverse proof仍成立。**複合模數上僅宣稱primitive並不自動保證所需分母都是unit**，這裡的敘述限定質數域。

同一$A=[1,2,0,0]$、$B=[1,-1,0,0]$，$p=17$、$N=4$、$\omega=13$，$\omega^2=16\neq 1$、$\omega^4=1$，且$4^{-1}=13$。$NTT(A)=[3,10,16,9]$，$NTT(B)=[0,5,2,14]$，點乘mod17得$[0,16,15,7]$，inverse得$[1,1,15,0]$。把15解碼為-2，還原原乘積。

解碼也需要幅度界：若輸入整數係數$abs \le B$，長度r,s，每格卷積$abs \le \min(r,s)B^2$；取$p>2\min(r,s)B^2$才可從單模數唯一還原有號係數。本例$r=s=2$、$B=2$，p17>16足夠。單有模數正確結果，不代表任意大整數的原值沒有繞回。

NTT的FFT級序以mod p算術為單位；若p隨資料幅度增長，位元成本還需計入log p位元整數的乘加，不能把任意長整數乘法當作一次固定時間操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 30。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31 數論演算法

Part VII Selected Topics

本章定位

由整除、群與CRT逐步證明模方程、模冪、RSA與隨機質性檢驗，區分確定因數證書和rho時間假設。

前置知識

整除、模算術、群結構與質數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 使用歐幾里得法與擴充 GCD。
- 解模線性方程與 CRT 系統。
- 分析 RSA、隨機質性測試與整數分解。

節次	主題	節次	主題
31.1	Elementary number-theoretic notions	31.2	Greatest common divisor
31.3	Modular arithmetic	31.4	Solving modular linear equations
31.5	The Chinese remainder theorem	31.6	Powers of an element
31.7	The RSA public-key cryptosystem	31.8	Primality testing
31.9	Integer factorization		

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.1 Elementary number-theoretic notions

整數d整除a，記d|a，是存在整數k使a=kd。質數p是p≥2且正因數只有1,p；1不是質數。Gcd(a,b)取非負最大公因數，a,b不全0時為正；另約定gcd(0,0)=0，程式先把負輸入取絕對值。

對n≥1，a≡b(mod n)代表n|(a−b)，a mod n為[0,n−1]唯一代表。負數也適用：−7 mod5=3。若a≡a'、b≡b'，則和、差、積都同餘，因差仍是n的倍數；除法不能直接照搬。

β位元整數的值可達約2^β；跑到√n的迴圈對β是指數。以下分開算術操作數與bit成本，乘除不是無條件O(1)。採普通乘法／長除法時，兩個O(β)位元整數的乘法與取餘上界$O(\beta^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Bezout、消去律與唯一分解的依賴

對a,b不全0，考慮所有正線性組合ax+by的最小值g。以g除a，餘數a−qg也是a,b線性組合；若餘數正且小於g矛盾，所以g|a，同理g|b。所有公因數都整除g，故g=gcd(a,b)，且存在x,y使ax+by=g。

若gcd(a,n)=1，寫au+nv=1，乘b得b=abu+nbv；所以n|ab⇒n|b。這是合法消去a的前提。取n=6,a=2，2・1≡2・4卻1≠4，顯示省互質條件會錯。

質數p若不整除a，則gcd(a,p)=1，上式推出p|ab⇒p|b。每個n≥2可依較小因數遞迴分成質數；兩組分解比較時，第一組任一質數必整除另一組某質數，兩者相等，消去後歸納，得唯一質因數分解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.2 Greatest common divisor

令a=qb+r、0≤r<b。共同整除a,b的整數也整除r=a−qb；共同整除b,r的整數也整除a=qb+r。故gcd(a,b)=gcd(b,r)。

```
euclid(a,b):
    a=abs(a); b=abs(b)
    while b!=0: (a,b)=(b,a mod b)
    return a
```

每輪第二參數嚴格下降至0，所以終止；不變量為當前gcd=原gcd，終態gcd(a,0)=a。若0<b≤a，第一餘數r<b；若r≤b/2已減半，若r>b/2則下一餘數b mod r=b−r<b/2。每兩次使第二參數至少減半，遞除次數O(log b)，β位元輸入為O(β)。

Fibonacci 最壞遞除鏈與 Bit 界

若a>b≥1需要k次遞除才到0，反推每步較早餘數≥後兩餘數之和，得到a≥F_(k+2)、b≥F_(k+1)，$F_1=F_2=1$。F每兩項至少倍增，所以k=O(log b)。相鄰Fibonacci輸入每步商1（末步除外），恰實現同階長鏈。

普通長除法單輪粗估$O(\beta^2)$，乘O(β)輪可安全得$O(\beta^3)$bit上界；這不是最佳界。若追蹤各輪操作數縮短可得到更精細$O(\beta^2)$傳統gcd界，但此處未用較精細界宣稱實作速度。

空間可只保留兩個餘數，O(β)位元；若為教學保存每列trace，便要另付儲存成本。沒有已知質因數分解也能算gcd，不能把「先分解」當成Euclid的前置工作。

Extended Euclid 的一般回推

若遞迴對(b,r)回傳g=b x'+r y'，代r=a−qb可得g=a y'+b(x'−q y')，所以回傳(x,y)=(y',x'−q y')。基例b=0回傳(g,x,y)=(a,1,0)。

```
extended(a,b): # 此處 a,b>=0
    if b==0: return (a,1,0)
    (g,x1,y1)=extended(b,a mod b)
    return (g,y1,x1-floor(a/b)*y1)
```

例252=1・198+54，198=3・54+36，54=1・36+18，36=2・18。回推18=54−36=4・54−198=4・252−5・198。故gcd=18，Bezout係數(4,−5)。負原輸入最後再對對應係數調符號。

Extended Euclid 的一般回推 (續)

Euclid：每個餘數都保留原輸入的整數線性組合

餘數 r	252 的係數 x	198 的係數 y	驗算 252x+198y
252	1	0	252+(0)=252
198	0	1	0+(198)=198
54	1	-1	252+(-198)=54
36	-3	4	-756+(792)=36
18	4	-5	1008+(-990)=18

商 1,3,1,2；最後非零餘數 18 = 4×252 − 5×198

31.3 Modular arithmetic

群G的運算須封閉、有identity、結合律、每元素有inverse。交換群另要求可交換；逆唯一，因若 ab=ac=e，則b=b(ac)=(ba)c=c。

(ℤ_n,+)是有限交換群，identity0、a的逆為−a；(ℤ_n,×)通常不是群，因0無乘法逆。其units U(n)= {a∈[0,n−1]:gcd(a,n)=1}才是乘法群，n≥2。

由Bezout，gcd(a,n)=1⇒au+nv=1⇒u是模逆；反向若au≡1，任何公因數整除au−nv=1，只能為1。Unit 乘積仍可逆，逆為$b^{-1}a^{-1}$，封閉性由此成立。模n的「除以a」只在a為unit時有唯一意義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-s

Lagrange 不是只背一個整除結論

子群H⊑G包含identity、封閉且有逆。左coset gH有|H|個元素，因乘g是雙射。若gH與kH有共同元素 $gh_1=kh_2$，則$k^{-1}g=h_2h_1^{-1}\in H$，兩cosets相同；否則不交。Cosets分割有限G，故|G|是|H|的整數倍，真子群大小≤|G|/2。

有限群的非空乘法封閉子集也自動是子群：任取a，有限性使a^i=a^j，i<j，乘群中的a^(−i)得a^(j−i)=e在子集；a^(j−i−1)給a的逆（若j−i=1，a=e，其逆即e）。

元素a的order是最小正t使a^t=e；$a^0,\cdots,a^{(t-1)}$兩兩不同，所有powers按模t重複，所以生成子群大小 =t。Lagrange給t||G|，進而a^|G|=e。有限性不可刪，否則整數加法群有無限order元素。

Lin Cing
https://github.com/xinhi6716/computer-science-self-s

Totient 的計數與真正的消去

$\phi(n)=|U(n)|$。對p質數，p^α中不互質的代表恰為p的倍數，共$p^{(\alpha-1)}$個，所以$\phi(p^\alpha)=p^{(\alpha-1)}(p-1)$。稍後CRT證兩兩互質乘積的unit集合為各unit集合的Cartesian product，故$\phi(n)=\prod p_i^{(\alpha_i-1)}(p_i-1)$。

例n=15，U(15)={1,2,4,7,8,11,13,14}，$\phi=8$。$7\cdot 13=91\equiv 1$，所以4/7代表$4\cdot 13\equiv 7$，不是整數商0。

加法群$\mathbb{Z}_8$中2生成{0,2,4,6}、order4；乘法群U(7)中2的powers為1,2,4，order3，而3的powers為1,3,2,6,4,5，order6。運算不同時，相同記號〈a〉不是同一組數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.4 Solving modular linear equations

解$ax\equiv b \pmod n$，$n\geq 1$。令$g=\gcd(a,n)$，必要條件$g|b$，因$ax+ny=b$的左側都被g整除。Extended Euclid給$au+nv=g$；若$b=gh$，則$x_0=uh$是一個解，證充分。

把式子除g，得$(a/g)x\equiv b/g \pmod{n/g}$，前係數與新模互質，故唯一解$x_0$模$n/g$。若兩解相減，$n/g$整除$(a/g)(x-x_0)$，互質消去推出$n/g|(x-x_0)$。所以模n的全部g個不同解為

$x=x_0+k(n/g) \bmod n \cdot k=0,\ldots,g-1$。

這給解的存在、個數、完整性與實際演算法。a=0時g=n：若n|b，全部n個代表都是解；否則無解。n=1只有代表0，不在此模數上談一般unit除法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

方程例題與輸出成本

`18x≡30(mod42)`，g=6。Euclid給6=−2・18+1・42，x_0=−2・5=−10≡4(mod7)。全部解為4,11,18,25,32,39；例如18・4=72≡30，增加7使乘積增126=3・42，因此全數成立。

若右邊改31，6∤31，無解。不能強行寫18^{-1} mod42，因它根本不存在。若只回傳解集的壓縮形式$(x_0,n/g,g)$，Euclid後不必枚舉；若要列全g項，就至少有Ω(g)輸出，不能只報O(log n)時間。

先把a mod n歸一化，再求gcd，遞除O(log n)，加g個輸出共O(log n+g)算術操作；每項仍有O(log n)位元，輸出bit數也須計入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.5 The Chinese remainder theorem

模數$m_1,\cdots,m_k$≥2兩兩互質，M=∏m_i，M_i=M/m_i，z_i=M_i^{-1} mod m_i。定義e_i=M_i z_i、x=Σa_i e_i mod M。

對固定j，i≠j時m_j|M_i，故e_i≡0；i=j時e_j≡1。因此x≡a_j(mod m_j)，證存在。若x,x'皆滿足，各m_i都整除差；兩兩互質配合互質消去，逐一得到乘積M整除差，證模M唯一。

映射x↦(x mod $m_1,\cdots$,x mod m_k)因此是雙射，且對加、乘保留運算：整數運算先後取餘等價。Unit也恰對應每個分量都是unit，否則至少與一個質因數不互質；所以totient對互質乘積可乘。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT 的三個單位座標實算

解$x\equiv 2 \pmod 3$、$x\equiv 4 \pmod 5$、$x\equiv 1 \pmod 7$。$M=105$，$M_i=(35,21,15)$，各inverse=(2,1,1)，所以e=(70,21,15)。

$x=2\cdot 70+4\cdot 21+1\cdot 15=239\equiv 29 \pmod{105}$。

檢查29 mod3=2、mod5=4、mod7=1。e_1的分量是(1,0,0)，e_2=(0,1,0)，e_3=(0,0,1)，這些是環的分量選擇器；它們也滿足$e_i^2=e_i$、$i\neq j$時$e_i e_j=0$、$\Sigma e_i=1$，均在mod M下成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CRT 的三個單位座標實算 (續)

CRT selectors：各 e_i 只選中一個同餘分量

e1 = 70	e2 = 21	e3 = 15
mod (3,5,7)	mod (3,5,7)	mod (3,5,7)
(1,0,0)	(0,1,0)	(0,0,1)
a1·e1 = 2×70 = 140	a2·e2 = 4×21 = 84	a3·e3 = 1×15 = 15

$140 + 84 + 15 = 239 \equiv 29 \pmod{105}$

29 → (2,4,1)；重建與取餘互為逆映射

運算的中間整數不能越界截斷；M的bit長是$\Sigma \log m_i$，因此可用大整數與逐步取餘，不能把M的數值大小當成位元記憶體成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非互質系統的相容條件

兩式x≡a(mod m)、x≡b(mod n)，令g=gcd(m,n)。寫x=a+mk，代第二式得到mk≡b−a(mod n)。由前節，恰在g|(b−a)時可解；解的周期為lcm(m,n)=mn/g，而非mn。

例x≡2(mod6)、x≡5(mod9)，g=3、差3可整除。6k≡3(mod9)化2k≡1(mod3)，得k≡2，x=14 mod18。若第二餘數改4，差2不被3整除，無解。

逐次合併可解一般系統；每一步都要檢查g條件並更新lcm。不能把pairwise-coprime的inverse M_i^{-1}公式直接套在m=6,n=9，因這些inverse不存在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.6 Powers of an element

對a∈U(n)，Lagrange給Euler定理a^φ(n)≡1(mod n)；n=p質數時φ=p−1，得Fermat定理。對所有a mod p，a^p≡a：a=0另直接成立。

若g的order為φ(n)，它生成全部units，稱primitive root。此時每個a=g^k，k模φ(n)唯一，稱discrete logarithm ind_(n,g)(a)。g^x=g^y當且僅當g^(x−y)=1，依order恰等價x≡y(modφ(n))。

例mod7以g=3，指數k=0,1,2,3,4,5對應a=1,3,2,6,4,5。所以ind(4)=4。若改g=2，只有1,2,4，對a=3沒有此base的log，不能對非generator保證所有unit都有表示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

為何有限體的乘法群有 Generator

在有限交換群中，若元素order為r,s且互質，乘積order為rs：若(ab)^k=1，則a^k=b^(−k)同時位於兩個互質大小子群，交集只有identity，故r|k且s|k；反向rs次方為1。

對F_p的非零乘法群，把各元素order的lcm記L。每個質數ℓ在L中的最高冪ℓ^v由某個元素order提供；適當取該元素的power，得到order恰ℓ^v。把各質數分量相乘，用前段得order L的元素。

所有p−1個非零元素都是多項式X^L−1的根；體上非零degree L多項式至多L個根（用每個根因式X−a連除歸納）。所以p−1≤L。另一方面每個order整除p−1，由Lagrange得L≤p−1，故L=p−1，generator存在。這不是把任意有限群都說成cyclic。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

奇質數冪的 Generator 也能建構

取mod p的generator g，p為奇質數。比較g與g+p：模p^2有$(g+p)^{(p-1)}\equiv g^{(p-1)}+(p-1)g^{(p-2)}p$，後項除p後非0 mod p。因此至少可選h∈{g,g+p}使h^(p−1)=1+cp且p∤c。

二項式展開給：若v_p(z−1)=j≥1，則對奇p有v_p(z^p−1)=j+1。首項為p(z−1)，其餘項至少多一個p因子（末項也因p≥3而足夠），不會取消首項。因此v_p(h^((p−1)p^k)−1)=k+1。

mod p^α的order需被p−1整除，又整除φ=p^(α−1)(p−1)；上面的valuation排除少一個p因子，故恰φ，證U(p^α) cyclic。這也說明不能任選mod p的generator就不檢查地當mod p^2的generator。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

哪些模數的 Unit Group 是 Cyclic

n>1時恰為2、4、p^α、2p^α，其中p為奇質數。前頁證odd prime power；U(2p^α)由CRT同構U(2)×U(p^α)，第一分量只有1，故仍cyclic；U(2)、U(4)直接可列。

排除其他n：若8|n，U(n)投影到U(8)為滿射，而U(8)每個元素平方都是1、大小4，非cyclic；cyclic群的homomorphic image必cyclic，矛盾。滿射可由odd prime-power的CRT分量及任意odd residue mod2^α得到。

若n含至少兩個odd prime factors，CRT讓每個奇質數冪分量獨立取±1，所以至少有4個平方為1的units；若4|n且另含odd prime，也由U(4)與odd分量給4根。Cyclic群中g^(2k)=1至多兩個k mod order，故這些n不cyclic。上述情況涵蓋所有未列型態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非平凡平方根如何暴露合數

對odd prime power p^α，若$x^2 \equiv 1$，則p^α|(x−1)(x+1)。兩因子的gcd只整除2，p為odd不可能同時分兩側，所以整個p^α只能整除其中一側，得$x \equiv \pm 1$。

若mod n發現$x^2 \equiv 1$但$x \not\equiv \pm 1$，則n不可能質數；而gcd(x−1,n)、gcd(x+1,n)都是非平凡因數。它們都小於n，否則x=±1；都大於1，否則對某側互質消去會迫使另一側被n整除。

例n=77，x=34：$34^2 = 1156 \equiv 1$，$34 \not\equiv 1, 76$；gcd(33,77)=11、gcd(35,77)=7。CRT解讀是$x \equiv 1 \bmod 11$、$x \equiv -1 \bmod 7$，兩種符號分量沒有同步。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Square-and-multiply 的完整迴圈

輸入n≥1、b≥0，先a←a mod n，r←1 mod n。由最高位到最低位讀b的所有bits：

```
r=1 mod n
for bit in binary(b), highest first: # b=0可視為空串
    r=(r*r) mod n
    if bit==1: r=(r*a) mod n
return r
```

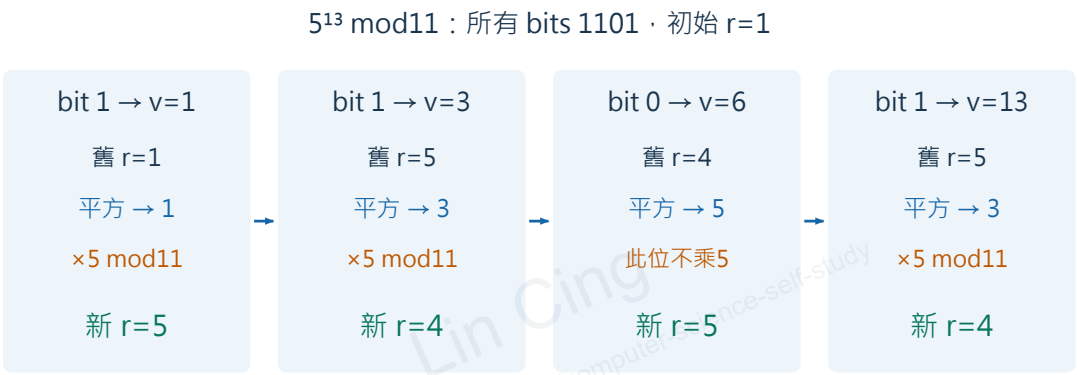
不變量：讀過的前綴值v時r=a^v mod n。下一bit β把v改2v+β，平方與可選乘a正好更新。空前綴v=0正確初始化；終態v=b得到目標。b=0返回1 mod n，包含n=1返回0；不要求gcd(a,n)=1。負指數則需要inverse，是另一輸入契約。

5^{13} mod11 的每一位

13的binary是1101，r從1掃全部四位：

bit	新前綴v	舊r	平方mod11	乘5後／保留
1	1	1	1	5
1	3	5	3	4
0	6	4	5	5
1	13	5	3	4

5^{13} mod11 的每一位 （續）



若r=a代表已讀最高1，只能再掃101；r=a卻掃完整1101會走指數3,7,14,29、模值4,3,9,9，得到錯的9。k位指數至多2k次模乘；若n有β位，普通模乘$O(\beta^2)$bit，總$O(k\beta^2)$，當$k=O(\beta)$為$O(\beta^3)$。另需計首次歸一化超長a的成本。

31.7 The RSA public-key cryptosystem

選相異odd質數p,q，n=pq，$\phi=(p-1)(q-1)$；選$1<e<\phi$且$\gcd(e,\phi)=1$，以Extended Euclid求$d=e^{-1} \bmod \phi$，取$1\le d<\phi$。公開(n,e)，保留d與p,q。對$0\le m<n$，數學轉換$c=m^e \bmod n$、$m'=c^d \bmod n$。

對任意m（不限unit）證$m^{(ed)}\equiv m$：$ed=1+k(p-1)(q-1)$。mod p若$p\mid m$兩側0；否則Fermat使$m^{(p-1)}=1$，得$m^{(ed)}=m$；mod q同理。CRT唯一性給mod n相等。若只用Euler定理會漏掉與n不互質的合法m。

同樣冪運算反向亦還原，所以私鑰冪／公鑰冪構成互逆的數學置換。這個正確性不等於密碼協定的安全性。

非 Unit 訊息也要能解密

取p=11,q=17，n=187、φ=160，e=7；1=23・7−1・160，故d=23。訊息m=11不是unit，仍合法。

加密平方鏈$11^2\equiv121$、$11^4\equiv55$，所以$11^7\equiv55\cdot121\cdot11\equiv88\pmod{187}$。解密$88^2\equiv77$、$88^4\equiv132$、$88^8\equiv33$、$88^{16}\equiv154$；23=16+4+2+1，乘四個對應值：154・132≡132，132・77≡66，66・88≡11。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非 Unit 訊息也要能解密 （續）

n=187 · e=7 · d=23；m=11 不是 unit

mod11 分量

11→0；88→0；$0^{23}\to0$
p | m：此分量直接還原0

mod17 分量

11→11；88→3；$3^{23}\to11$
此分量是 unit · 使用 Fermat

CRT：(0 mod11 · 11 mod17) 唯一重建 11 mod187

若e為固定bit長，公鑰操作$O(\beta^2)$普通bit成本；d有$O(\beta)$位時私鑰$O(\beta^3)$。由n分解出p,q可求φ,d；不能逆稱「RSA逆轉與整數分解已被證明等價」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Textbook RSA 的代數限制

無編碼的RSA是deterministic：相同m總得相同c，可列舉小訊息域。又$E(m_1)E(m_2)\equiv E(m_1m_2)$，因此若c=E(m)，選r∈U(n)，c・r^e對應mr；這是可操作性，不是安全證明。

直接用m^d作「簽章」也不具完整安全性：任取σ再設m=σ^e便有一組驗證會過的(message,signature)，卻非授權簽出的語意訊息。實用方案需另訂隨機化編碼、訊息雜湊與驗證格式；本章只推導基礎置換，不把toy參數或裸冪當部署規格。

這些弱點由明確代數式推出，與「大質數分解很難」可以同時成立。不可用數學解密正確性取代攻擊模型與安全定義。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.8 Primality testing

Trial division只需試到$\lfloor\sqrt{n}\rfloor$：若n=ab合數且兩者都>$\sqrt{n}$會矛盾；若沒有小因數便質數。但對β位n，$\sqrt{n}\approx 2^{(\beta/2)}$，不是多項式bit時間。

質數定理給π(X)~X/ln X，故在大致同量級區間均勻取數，質數密度約1/ln X，只取odd約2/ln X。這是漸近估計，不是每個候選必在ln X次內找到質數的界，也不是下述錯誤率證明。

Fermat測試若$a^{(n-1)}\not\equiv 1$，可確定合數；若等於1只叫probable prime。341=11・31且$2^{10}=1024\equiv 1 \bmod 341$，所以$2^{340}=1$仍合數。Carmichael數甚至讓所有units通過：561=3・11・17，各p−1=2,10,16都整除560，CRT與Fermat給全部units的560次方為1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Miller-Rabin 的完整基例與平方鏈

先處理n<2為非質數；n=2,3為質數；其餘偶數為合數。只對odd n≥5寫n−1=2^s d、d odd、s≥1，均勻取整數a∈[2,n−2]。

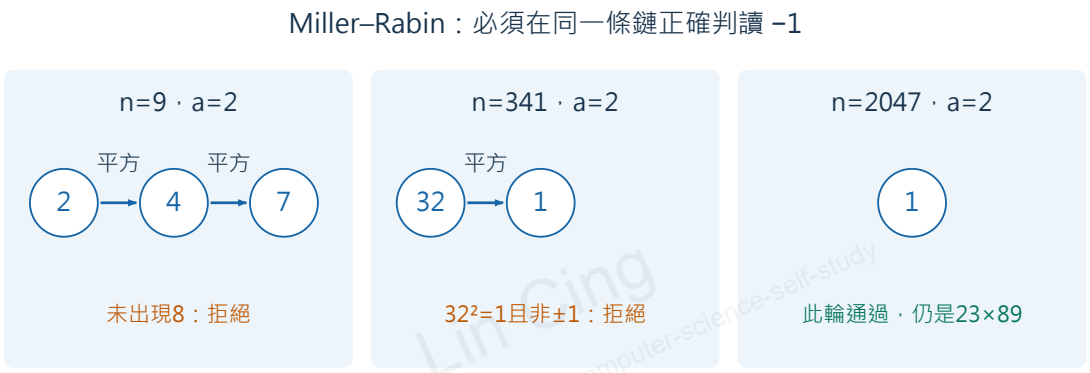
```
x=modular_power(a,d,n)
if x==1 or x==n-1: pass this round
else:
    repeat s-1 times:
        x=x*x mod n
        if x==n-1: pass this round
    if no -1 occurred: return COMPOSITE
```

重做r個獨立基底，皆通過才回probably prime。質數n的非零a滿足a^(n−1)=1；鏈若非從1起，第一次平方到1前的值只能是−1，因體中平方為1只有±1。因此質數必通過，每個被拒者都確實是合數。n=3若未特判會抽空區間[2,1]。

同樣通過 Fermat，強測試能再分辨

n,a	分解n−1	$x_0=a^d$	後續平方	結論
5,2	$4=2^2\cdot 1$	2	4=−1	通過
9,2	$8=2^3\cdot 1$	2	4,7	拒絕，未見−1
341,2	$340=2^2\cdot 85$	32	1	拒絕，32非平凡平方根
2047,2	$2046=2\cdot 1023$	1	不需再平方	本輪通過，仍是23・89

同樣通過 Fermat，強測試能再分辨 （續）



通過只是 probable prime；固定同一 base 反覆測不能放大可信度

2047例說明單個固定base不能證一般整數為質數。錯誤方向是合數被接受，質數不會被拒；不要用未定義positive/negative顛倒這個事件。以下針對每個固定合數證random-base上界。

計算 Strong Liars 先數 Prime-power Roots

固定odd合數n=∏_(i=1)^r p_i^α_i，n−1=2^s d。令p_i−1=2^t_i u_i（u_i odd），D_i=gcd(d,u_i)，τ=min(s,t₁,⋯,t_r)。通過的a必是unit，因若某p_i|a，其所有正冪在此分量都0，不可能成±1。

F_(p_i)^×有generator，寫a=g^k；a^d=1有D_i個解；a^(2^j d)=−1轉成指數同餘，恰在j<t_i時有2^j D_i個解。理由是係數gcd=2^min(j,t_i)D_i，能整除(p_i−1)/2當且僅當j<t_i。

這些根各唯一提升到p_i^α_i：若z已解X^h−η≡0(mod p_i^a)，試z+c p_i^a，下一級只需解 (z^h−η)/p_i^a+h z^(h−1)c≡0(mod p_i)。h=2^j d不被p_i整除（p_i∤n−1），z是unit，導數非零，因此c唯一。故同樣根數在每個prime-power分量成立。

CRT 合併根數，而不是猜一個子群

通過情形互不相交：a^d=1，或某個唯一的j∈[0,s−1]使a^(2^j d)=−1；不同j不可能都為−1，因先平方一次就成1。CRT使各分量獨立，但「同一j、全部−1」必同步。

所以全部a∈[1,n−1]的strong liars個數恰為

$$L = \left(1 + \sum_{j=0}^{\tau-1} 2^{jr}\right) \prod_i D_i.$$

第一項是全部分量的a^d=1，j項是全部分量同時在第j層為−1。不能把每個分量「各自某層為−1」也算進來，那會多算不同步而應拒絕的a。此公式只用質因數分解做**分析**，測試程式不需事先分解n。

1/4 界：一個或至少三個質因數

r=1時n=p^α、α≥2。p−1|p^α−1，故τ=t、D=u，公式得L=p−1。因此`L/(n−1)=(p−1)/(p^α−1)≤1/(p+1)≤1/4`。

r≥3時φ(n)=∏p_i^(α_i−1)2^t_i u_i，且D_i≤u_i、t_i≥τ，所以

`L/φ(n) ≤ [1+Σ_(j=0)^(τ−1)2^(jr)]/2^(rτ)`。

等比和可寫`1/(2^r−1)+(2^r−2)/[(2^r−1)2^(rτ)]`，隨τ≥1遞減，最大在τ=1為2^(1−r)≤1/4。φ(n)≤n−1，得到L≤(n−1)/4。n=9達L=2=(9−1)/4，所以此全代表域的常數不能普遍再縮。

1/4 界：兩個質因數的必要分支

r=2時若$t_1 \neq t_2$，或$s<\min(t_1,t_2)$，則$t_1+t_2 \geq 2\tau+1$。由公式

$L/\phi(n) \leq [1+\Sigma 4^j]/2^{(2\tau+1)}=(4^\tau+2)/(6\cdot 4^\tau) \leq 1/4$。

剩下$t_1=t_2=\tau \leq s$。如果某$\alpha_i \geq 2$，ϕ的prime-power額外因子≥ 3；如果某$D_i<u_i$，因u_i/D_i為odd整數也至少3。至少出現其中一種：否則n=pq squarefree且$D_i=u_i$，會有$p-1|n-1$、$q-1|n-1$；由$pq-1$模$p-1$得到$p-1|q-1$，反向同理，迫使p=q，矛盾。

因此額外除以至少3，$L/\phi(n) \leq (4^\tau+2)/(9\cdot 4^\tau) \leq 1/6<1/4$。所有odd合數情形已覆蓋，一般1/4界不再依賴未指定的最大位階或Carmichael分支。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

抽樣分母、重複次數與條件機率

1與n−1永遠通過。由全域$L \leq (n-1)/4$，排除它們後，[2,n−2]的比例為$(L-2)/(n-3) \leq 1/4$。因此對**固定合數n**，獨立均勻抽r次、全通過機率$\leq 4^{(-r)}$；不是「看到通過後，n為合數的機率」直接等於此值。

若候選的先驗質數率$\pi>0$，質數通過率1，合數接受率$\leq \varepsilon=4^{(-r)}$，Bayes給$Pr(\text{composite} \mid \text{pass}) \leq \varepsilon(1-\pi)/[\pi+\varepsilon(1-\pi)]$。須有候選分布才能使用π；不能替攻擊者挑的固定輸入假設均勻質數密度。

每輪$O(\log n)$次模乘，普通bit界$O(\beta^3)$，r輪$O(r\beta^3)$，另含抽樣成本。固定base清單只有在其適用輸入上限已證明時才可叫deterministic。先隨機抽一個base再反覆沿用，不屬獨立抽樣；每輪IID有放回抽樣則可以偶然抽中相同值，仍符合契約。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

31.9 Integer factorization

判合數不等於得到分解。Trial division可確定找因數，但最壞$\sqrt{n}$次；Pollard rho利用「mod某因數先碰撞」，不需知道該因數。

取整數多項式$f(x)=x^2+c \bmod n$，若$p|n$，則$f(x) \bmod p$等於對$x \bmod p$計同一多項式。只要$x_i \equiv x_j \bmod p$，便有$p|(x_i-x_j)$，所以$d=\gcd(|x_i-x_j|,n)>1$；若$x_i \not\equiv x_j \bmod n$，$d<n$，得到真因數。

這一步的正確性完全確定，且回傳後可直接檢查$1<d<n$與$d|n$。難點是何時碰撞、是否同時在所有分量碰撞；後者會得到$d=n$，不能把它當成功分解。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Floyd 與 Checkpoint 兩種找碰撞方法

```
x=y=x0
repeat until work budget exhausted:
    x=f(x); y=f(f(y))
    d=gcd(abs(x-y),n)
    if 1<d<n: return d
    if d==n: restart with another x0/c
return no factor found within budget
```

Floyd每輪慢指標走1、快指標走2。對有限函數圖，一段tail後進cycle；兩者都進cycle後，相對位置每輪差1，所以至多cycle長λ輪相遇，總$O(\mu+\lambda)$次函數計算，$O(1)$個整數儲存。

另一checkpoint版固定$y=x_{(2^k)}$，讓x繼續走到下個$2^{(k+1)}$；當checkpoint已進cycle且區間長$\geq\lambda$，必在區間內遇到同分量。這是另一排程，不與Floyd的i、j trace混用。兩者都需處理$d=n$與工作上限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

8051 的三輪因數追蹤

取n=8051、x_0=2、$f(x)=x^2+1$ mod8051，使用Floyd。

輪	x=f(x)	y=f(f(y))	差的絕對值	gcd(diff,8051)
1	5	26	21	1
2	26	7474	7448	1
3	677	871	194	97

第3輪677 mod97=95、871 mod97=95，先在97分量碰撞；mod83則13與41尚不同。因此gcd不是8051而是97，除得8051/97=83，兩者經trial division至其平方根皆質數，完整分解8051=83・97。

8051 的三輪因數追蹤 （續）

Floyd 第3輪：x=677，y=871，差194

mod97：先碰撞

mod83：尚未碰撞

677 mod97	871 mod97	677 mod83	871 mod83
95	95	13	41

97 | 194，但83 ∤ 194，所以 gcd(194,8051)=97

驗算：8051 = 83×97；回傳 proper divisor 才算找到因數

Birthday 估計的假設與實際限制

若mod未知質因數p的迭代可近似均勻random mapping，首次碰撞前逐步探索新點，k次仍未碰撞的機率約$\prod_{j=0}^{k-1}(1-j/p)\le\exp(-k(k-1)/(2p))$，所以典型碰撞尺度$\sqrt{p}$；tail/cycle方法在同階步數內檢出。

固定x^2+c不是獨立亂數，這個假設不提供它的最壞時間或每次restart成功率。對平衡semiprime $p\approx\sqrt{n}$，heuristic尺度$n^{1/4}=2^{\beta/4}$，仍非β的多項式。每輪另有β位模平方與gcd成本；依本章安全粗gcd界可報$O(\beta^3)$，不能把一次gcd默認O(1)。

回傳真因數後還要遞迴分解兩側，先排偶數、確認質數或probable-prime的可信條件；若預算用完，只能說尚未找到因數，不能改判質數。Rho的因數證書是確定的，完成時間保證則與該heuristic分析分開。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 31。

M. O. Rabin, "Probabilistic Algorithm for Testing Primality," *Journal of Number Theory* 12(1), 1980, doi:10.1016/0022-314X(80)90084-0.

L. Monier, "Evaluation and Comparison of Two Efficient Probabilistic Primality Testing Algorithms," *Theoretical Computer Science* 12(1), 1980, doi:10.1016/0304-3975(80)90007-9.

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29 線性規劃

Part VII Selected Topics

本章定位

由等價建模、精確pivot與防循環推導two-phase simplex，並以對偶和ray交付三種結果的證書。

前置知識

凸多面體、標準形與對偶。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 將問題轉成 standard form 與 slack form。
- 從基本可行解執行 simplex。
- 使用 weak/strong duality 與可行性啟動方法。
- 29.1 Standard and slack forms
- 29.2 Formulating problems as linear programs
- 29.3 The simplex algorithm
- 29.4 Duality
- 29.5 The initial basic feasible solution

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29.1 Standard and slack forms

本章的standard form固定為 `max cᵀx · Ax≤b · x≥0`，A∈ℝ^(m×n)。所有限制非嚴格，決策變數為連續實數；有理係數時才能以有限binary輸入談bit complexity。其他書也可能把等式形式叫standard，須先確認慣例。

Feasible point滿足全部限制；optimal point在可行點中達最大目標。Infeasible是沒有可行點；unbounded是可行且目標可任意大，不是「可行區域無界」的同義詞。例如x≥0、max(−x)的區域無界，最優值仍是0。

有限個線性半空間的交集是凸多面體：若Ax≤b、Ay≤b，則A(λx+(1−λ)y)≤b，0≤λ≤1。凸性不保證整數解，也不容許把非線性乘積當成線性限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

四種轉換都要能還原原變數

最小化f改最大化−f，輸出目標記得取負；aᵀx≥b乘−1改≤；等式拆成≤與≥。無符號限制的u改u=u⁺−u⁻、u⁺,u⁻≥0：任意u可取(max(u,0),max(−u,0))，反向取差也恢復原點，雖不是一對一，最優值仍對應。

例 `min 2u−v · u+v=3 · u−2v≥−3 · v≥0 · u unrestricted`，改最大化−2u⁺+2u⁻+v，限制

`u⁺−u⁻+v≤3；−u⁺+u⁻−v≤−3；−u⁺+u⁻+2v≤3`。

代u=3−v得v≤2，原目標6−3v，故v=2,u=1、最小0；新目標最大0。若原有n變數、m限制，拆符號至多2n變數、拆等式至多2m限制，係數位元長僅增加常數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Slack Form、Basis 與可行基本解

對Ax≤b引入s=b−Ax≥0，等式矩陣M=[A I_m]、全變數q=(x,s)≥0，Mq=b。Slack有唯一值，不是另一組可任意分配的決策。

選m個線性獨立columns作basis B，令其餘N非基本。由可逆M_B得

$q_B=M_B^{-1}b-M_B^{-1}M_Nq_N$，$z=v+\rho_N^Tq_N$。

非基本全設0為basic solution；可行當且僅當$M_B^{-1}b$逐項≥0。初始slack basis是I，故初始點可行當且僅當b≥0。Basic不等於正：值0的基本變數是退化的來源。給定B後字典唯一，因解出線性方程所用的inverse唯一。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

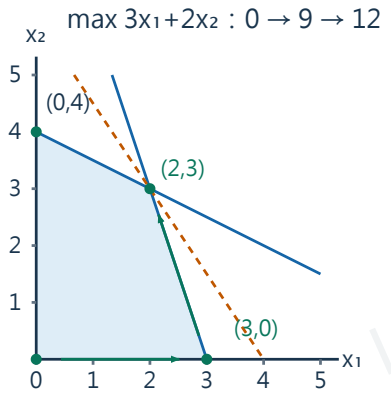
幾何、等值線與 Basis 是同一個例子

以下全部使用 $\max\ 3x_1+2x_2$，$x_1+2x_2\le8$，$3x_1+x_2\le9$，x≥0。可行四邊形頂點(0,0)、(3,0)、(2,3)、(0,4)。目標沿頂點路徑0→9→12改善，最後(2,3)。

初始字典：$x_3=8-x_1-2x_2$；$x_4=9-3x_1-x_2$；$z=3x_1+2x_2$。Basis={x_3,x_4}，初始slack=(8,9)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

幾何、等值線與 Basis 是同一個例子 （續）



藍線：兩個資源限制

$x_1+2x_2=8$

$3x_1+x_2=9$

綠線：simplex 的兩次移動

橘虛線：$3x_1+2x_2=12$

交點 (2,3)，目標 12

最優點也可能形成整條edge，例如改目標x_1+2x_2，邊(0,4)到(2,3)都同值8。頂點法會回其中一個最優基本解，不宣稱最優解唯一。

29.2 Formulating problems as linear programs

建立模型先列「變數代表什麼、單位、允許範圍」，再逐項把可行解映到原問題，也把每個原可行方案映回變數。只有目標相似而漏了容量或守恆，並不是等價模型。

最短路使用potential d_v，固定d_s=0，對每邊uv限制d_v−d_u≤w_uv，最大化d_t。d沒有非負限制，負權路徑可能使d_t<0。

先取全部頂點從s可達且無負cycle、t可達的情形。沿任意s-t路把不等式相加，得d_t≤該路長，所以任何可行d是最短距離的下界；真距離δ(s,v)由三角不等式滿足所有限制，達到上界δ(s,t)。故最大化而非最小化恰得到最短距離。

Potential 的邊界不是程式中的 Infinity

若只研究s可達子圖，無負cycle時所有δ有限，可用前頁模型；t不可達要另外回報，不能把∞當實數LP變數。若保留不可達子圖，當中負cycle會使模型不可行，即使s到t距離完全正常，所以不能漏這個前提。

對無任何負cycle的全圖，可加0權supersource求一組全圖可行potential；若t不從s可達，可在不可達集合整體加任意正K，仍保持所有差限制，因無可達→不可達邊，故d_t無上界。這時LP unbounded對應t不可達，不是「存在負長路」。

數例s→a權2、a→t權−1、s→t權4，限制d_a≤2、d_t≤d_a−1、d_t≤4。取(0,2,1)可行，且兩邊路給d_t≤1，答案1；若錯加d≥0在別的負總長例會排除正確解。

Flow 模型：每個 Arc 的非負量

此節用非負arc flow g_uv，而非上一章的反對稱淨流。最大流：對每個原邊設0≤g_uv≤c_uv，內點滿足Σ_out g−Σ_in g=0，最大化source淨流出。允許antiparallel，只要每個原arc各有自己的變數與容量；不能再加g_uv=−g_vu。

給原flow就滿足模型；給模型解，其容量與守恆正是flow定義。原邊變數m，限制O(n+m)，不必配置n^2個非邊變數。

最小成本流另固定source淨流出D、sink淨流入D，最小化Σ a_uv g_uv。D是「恰好需求」，不可只最大化總流再把成本當附帶值。原邊有有限容量時，負成本cycle可送循環流但不會因而無界；禁止循環需要另外建模。

最小成本流的完整數值模型

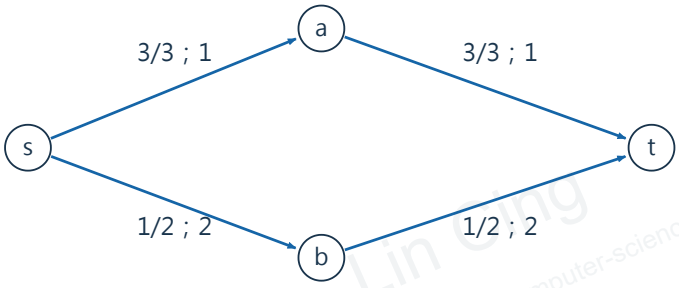
原邊s→a容量3、單位成本1；a→t容量3、成本1；s→b容量2、成本2；b→t容量2、成本2。需求D=4。守恆令上下兩路流分別u,v，得到u+v=4，0≤u≤3、0≤v≤2，最小化2u+4v。

代v=4−u，成本16−2u；可行2≤u≤3，故u=3,v=1，成本10。這同時交付每邊flow、需求、可行區間與最優性，不只列出一條看似便宜的路。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最小成本流的完整數值模型 （續）

需求 D=4；標記為「流 / 容量；單位成本」



u+v=4

2≤u≤3

成本=16−2u

最優 u=3,v=1

上路成本 3×(1+1)=6；下路成本 1×(2+2)=4；總 10

若把需求錯寫u+v≤4，零流成本0會被誤認最佳；若省容量u≤3，會錯選u=4，成本8。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multicommodity 不能拆成互不相干的最大流

對commodity k給(s_k,t_k,D_k)，每原邊有g^k_uv≥0。每種貨物各自滿足內點守恆、端點淨供需±D_k；共享容量是Σ_k g^k_uv≤c_uv。若只問可行，目標可設0；可另加總成本。

同一瓶頸u→v容量1，兩種貨都唯一經過此邊，需求各3/4。各別都可送，但合併需求3/2>1，不可行。兩條獨立最大流會漏掉共享資源。

k種貨、m邊、n點有km變數、O(kn+m)限制，對給定有理D與容量為多項式大小LP。這是可分割貨流；要求每種貨只能選一條路或要求整數量，會改成不同的離散問題，不能沿用一般LP的保證。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29.3 The simplex algorithm

字典固定寫 `x_i=b_i-Σ_(j∈N)a_ij x_j`、`z=v+Σ_(j∈N)c_j x_j`。此處a是「被減去」的係數，與畫在式子中的符號相反；後面所有ratio與pivot都用這個契約。

給可行字典b_i≥0，選c_e>0入基，令x_e=θ≥0、其他非基本0。基本值x_i=b_i−a_ieθ。a_ie≤0不限制增加；a_ie>0要求θ≤b_i/a_ie。因此取這些正分母的最小非負比值，包括0。

沒有正分母時，方向q_e=1、q_i=−a_ie≥0、其他q=0給一條可行ray：x(θ)=x(0)+θq，目標v+c_eθ→∞。不能只寫x_e=∞，∞不是可行實數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pivot 必須保留離基變數的係數

選離基l，a_le≠0，從l列精確解出

x_e=b_l/a_le−x_l/a_le−Σ_(j∈N\{e})(a_lj/a_le)x_j。

這一列代回所有其他列與目標，再交換B、N。原l變成非基本，但不是從方程中消失；漏掉−x_l/a_le會改變可行集合。

新a_el=1/a_le、b_e=b_l/a_le、a_ej=a_lj/a_le。對i≠l：b'_i=b_i−a_ie b_e，a'_il=−a_ie/a_le，a'_ij=a_ij−a_ie a_lj/a_le。目標v'=v+c_e b_e、c'_l=−c_e/a_le、c'_j=c_j−c_e a_lj/a_le。

以上是可逆代入，不只對目前基本解成立，而是保留全部滿足等式的點及其目標；配上所有變數非負，得到同一LP。

Ratio Test 保持可行的一般證明

新入基值b'_e=b_l/a_le=θ≥0。其餘b'_i=b_i−a_ieθ：若a_ie>0，θ≤b_i/a_ie使其非負；若a_ie≤0，則b'_i≥b_i≥0。新非基本全0，所以新基本解可行。

目標增加c_eθ≥0。θ>0時嚴格改善；θ=0時點與目標可能完全不變，只是basis不同。此證明同時說明為何不限制a_ie<0的列，以及為何不能排除零比值。

```
while some reduced cost c[e]>0:
    choose entering e
    candidates={i in B : a[i,e]>0}
    if candidates empty: return feasible point + improving ray
    l=minimum-ratio row, with a specified anti-cycling rule
    pivot(l,e)
return current basic solution
```

同一二維例的兩次 Pivot

初始B={3,4}。先讓x_1入基，比值8/1=8、9/3=3，故x_4離基、θ=3：

$x_1=3-x_2/3-x_4/3$；$x_3=5-5x_2/3+x_4/3$；$z=9+x_2-x_4$。

再選x_2，x_1列比值3/(1/3)=9、x_3列5/(5/3)=3，故x_3離基：

$x_2=3-3x_3/5+x_4/5$；$x_1=2+x_3/5-2x_4/5$；$z=12-3x_3/5-4x_4/5$。

令非基本$x_3=x_4=0$，得到$(x_1,x_2)=(2,3)$，兩限制恰等號。所有reduced costs≤0，任意可行點的目標都≤12，所以此點最優。不必再數值搜尋整個可行多邊形。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

零步長是合法 Pivot

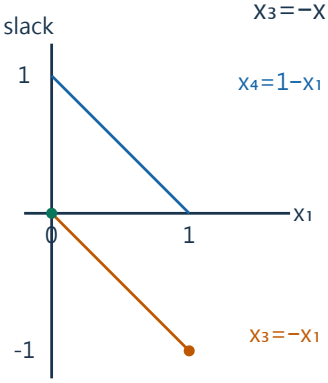
$\max\ z=x_1$；$x_3=-x_1$；$x_4=1-x_1$，所有變數非負，初始B={3,4}、N={1}、基本值(0,1)。

基本列	正分母	比值	θ=0	錯取θ=1
x_3	1	0	0	−1，不可行
x_4	1	1	1	0

令x_3離基，得$x_1=-x_3$、$x_4=1+x_3$、$z=-x_3$。新基本點仍(0,0,1)，目標仍0，卻已能由非正reduced cost證最優。若只取positive ratio就立即越出可行域。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

零步長是合法 Pivot （續）



$x_3=-x_1$，$x_4=1-x_1$：零比值不能略過

$x_1=0$：$x_3=0$、$x_4=1$，可行

ratio：0/1=0 與 1/1=1

合法步長 $\theta=0$，x_3 離基

錯取 $\theta=1$：$x_3=-1$，不可行

可行的 x_1 只有 0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有限 Basis 本身不能排除 Cycling

M=[A I]有m行，任何basis選m個獨立columns；至多C(n+m,m)種。給定basis後字典唯一，所以若永不終止，必重複basis。但退化時目標可以不變，僅說「目標不下降」不能排除重複。

本章用精確lexicographic perturbation給出一個可自足證明的防循環版本。給當前可行起始字典 $q_B=\beta-Dq_N$，把RHS改$\beta_i+\varepsilon^i$，i=1,…,m；ε是正無窮小，**不是**某個任意浮點小數。常數項非負且每行有正的第一個ε係數，所以初始字典在此有序域嚴格可行。

計算與比較改用係數向量：先比常數，再比ε、ε^2,…；首個非零係數決定正負。後續pivot仍用正分母的最小比值，若實數比值相同，用後面的係數分出順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lexicographic 規則為何有限終止

把初始字典等式矩陣記$M_0=[I\ D]$，perturbed RHS $\beta+\Sigma e_i \varepsilon^i$。任何可逆basis K的基本值是$M_0,K^{-1}\beta + \Sigma(M_0,K^{-1}e_i)\varepsilon^i$。

Inverse每行不全零，因此每個基本值作為ε多項式都不恒等於0；可行性又保證它非負，故其實嚴格正。兩個正分母ratio不可能完全相同：否則inverse兩行成比例，違反可逆。每次合法pivot的θ因此嚴格正，c_e>0使目標在lex順序嚴格增加。

同一basis的字典／目標唯一，不能重訪。有限basis保證終止。取所有係數的常數項，就回到原LP的可行基本解；reduced costs與RHS無關，若全≤0則常數解也最優。若出現改善ray，其係數同樣不依ε，從常數可行點出發便證原LP無界。故不是靠挑一個「夠小」的浮點ε猜測正確性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lexicographic 平手的可重算例子

$x_3=2-x_1$；$x_4=2-x_1-x_2$；max x_1+x_2。初始把兩個RHS改$2+\varepsilon$、$2+\varepsilon^2$。x_1入基的兩比值是$2+\varepsilon$與$2+\varepsilon^2$，後者較小，所以x_4離基。

新列 $x_1=2+\varepsilon^2-x_2-x_4$、$x_3=\varepsilon-\varepsilon^2+x_2+x_4$、$z=2+\varepsilon^2-x_4$。後者已無正reduced cost，常數解$x_1=2$、$x_2=x_3=x_4=0$，目標2，符合原限制。$x_3$雖常數0，perturbed值$\varepsilon-\varepsilon^2$嚴格正，正是退化被符號分開的地方。

另一常用防循環法Bland規則是選最小索引的正reduced-cost入基，最小比值平手時選最小索引離基；兩處規則都要給。以上完整終止證明採lex版本，不把任意選取也說成必終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每次 Pivot 的成本與全程成本要分開

普通dense字典一次更新O(mn)個係數，不能由此稱整個simplex多項式；basis數C(n+m,m)對維度可能指數大。Lex版本另存每個RHS的m+1個係數，更新與比較增加多項式因子，仍只有有限／指數的全程最壞界。

有理輸入用精確分數做pivot。Basis解由矩陣子行列式的比值表示；清除原分母後，行列式有至多m!項，bit長受維度與輸入bit長的多項式界控制。這說明單一basis的精確表示不必無限制膨脹，不代表pivot次數變多項式。

浮點實作要另訂feasibility／reduced-cost容許值、ratio近零與重算殘差；把負的b無條件截成0可能改題。Ellipsoid／interior-point屬不同演算法，其多項式理論不由上述pivot計數推出。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29.4 Duality

Primal $\max\ c^{\mathsf{T}}x \cdot Ax \le b \cdot x \ge 0$ 對偶為 $\min\ b^{\mathsf{T}}y \cdot A^{\mathsf{T}}y \ge c \cdot y \ge 0$。m個資源限制對應m個價格y；n個原變數對應n個價格下界。A需轉置，不能只交換max/min。

給兩側可行解，依$x \ge 0$、$A^{\mathsf{T}}y \ge c$與$y \ge 0$、$Ax \le b$依序得

$c^{\mathsf{T}}x \le x^{\mathsf{T}}A^{\mathsf{T}}y = y^{\mathsf{T}}Ax \le y^{\mathsf{T}}b$。

這是weak duality。任一可行dual給primal上界；若上下界相等，兩者同最優。只有相同目標值而不檢查兩側可行，不能當最佳性證書。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從終止 Basis 構造 Dual，一般證明

令M=[A I]、d=(c,0)，終止basis B的矩陣可逆。定義row vector $y^T=d_B^T M_B^{-1}$。代入 $q_B=M_B^{-1}b-M_B^{-1}M_N q_N$得

$z=y^Tb+\Sigma_{(j\in N)}(d_j-y^TM_j)q_j$。

所有columns的reduced cost $\rho_j=d_j-y^TM_j$，基本columns恰0。最優字典的$\rho\le 0$：對slack column e_i有$\rho_{n+i}=-y_i\le 0$，所以$y\ge 0$；對decision column A_j有$\rho_j=c_j-y^TA_j\le 0$，所以$A^Ty\ge c$。

目前可行基本解的目標是y^Tb，故達weak-duality等號。這證明有限最優primal必有同值dual；也得到讀取規則「y_i=−原slack變數的reduced cost」，若該slack基本則成本0。證明依完整字典恆等式，並非只在單一最優點比較係數。

同一數例的 Dual 與 Complementary Slackness

二維例dual是 $\min\ 8y_1+9y_2$，$y_1+3y_2\ge 3$、$2y_1+y_2\ge 2$、$y\ge 0$。終止目標$z=12-3x_3/5-4x_4/5$給$y=(3/5,4/5)$。兩個dual限制均等號，值24/5+36/5=12，與x=(2,3)吻合。

一般gap精確分解為

$b^Ty-c^Tx = y^T(b-Ax) + x^T(A^Ty-c)$。

對兩側可行，右側每項非負，所以gap=0當且僅當每項都0：$y_i(b_i-A_i\ x)=0$、$x_j((A^Ty)_j-c_j)=0$。這就是complementary slackness，必要與充分都來自此分解；有正slack則對應價格0，有正變數則對應dual限制緊束。

另例$\max\ 2x_1+x_2$，$x_1+x_2\le 4$，$x_1\le 2$，$x\ge 0$，dual可取y=(1,1)，上界6。x=(2,2)值6、全部互補乘積0；x'=(1,3)值5，唯一非零乘積為$y_2(2-x'_1)=1$。

同一數例的 Dual 與 Complementary Slackness （續）

$$\text{gap} = y^T(b - Ax) + x^T(A^Ty - c) \geq 0$$

$x=(2,2)$	$x'=(1,3)$
$y=(1,1)$，dual 值 6	$y=(1,1)$，dual 值 6
primal 6	primal 5
$y_2(2-x_1) = 0$	$y_2(2-x_1) = 1$
其餘三個乘積皆 0	其餘三個乘積皆 0

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

無界與不可行不能隨便互換

Primal無界時dual不可能可行，否則固定dual值由weak duality限制所有primal值；同理dual向下無界則primal不可行。逆向單靠這句不成立：primal不可行不代表dual無界，兩者可能都不可行。

例如 $\max\ -x \cdot 0 \cdot x \leq -1 \cdot x \geq 0$ 的primal不可行，dual $\min\ -y \cdot 0 \cdot y \geq -1 \cdot y \geq 0$ 可行但向下無界。改為 $\max\ x \cdot 0 \cdot x \leq -1 \cdot x \geq 0$，dual限制$0 \geq 1$也不可行，兩側皆不可行。

對一般符號可先轉standard再建dual；原primal等式限制的乘子可正可負，free primal變數對應dual等式。這些符號規則由拆成兩個非負變數或兩個相反不等式合併而來。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

29.5 The initial basic feasible solution

初始b有負項時，不能直接跑只維持可行性的ratio test。對$Ax\le b, x\ge 0$加入$a\ge 0$，解auxiliary：$\max -a$ · $Ax-a\cdot 1\le b$。它總可行：取x=0、a=max(0,−min_i b_i)。目標≤0，所以不會向上無界。

原LP可行⇒同一x配a=0可達目標0，已是auxiliary最優。反向auxiliary最優0⇒a=0⇒同一x滿足原限制。若最優<0，原LP不可行。不能以「目前人工變數仍正」提早判不可行，必先確定auxiliary已最優。

有m=0時直接取原點為可行基本解；若某c_j>0便有無界ray，否則原點最優，不必求空集合的min。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一次特殊 Pivot 建立 Auxiliary 可行基底

令$s_i=b_i-A_i x+a$，挑r使$b_r=\min_i b_i<0$。讓a入基、s_r離基；此步pivot係數−1，**不是**從可行基底選改善方向的ratio步。

解出 $a=-b_r+A_r\ x+s_r$，代其他列 $s_i=b_i-b_r+(A_r-A_i)x+s_r$。設新非基本$x=s_r=0$，得到$a=-b_r>0$、$s_i=b_i-b_r\ge 0$，故建立可行基底。現在才可使用前述simplex迴圈及lex防循環。

Lex防循環從這個可行字典開始：可只為迴圈加入符號RHS，再取常數項；reduced costs不依RHS，因此得到原auxiliary的真最優值及可行基本解，不會把ε目標誤判成原問題的不可行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

人工變數為零，仍要清理 Basis

Auxiliary最優0時，若a非基本，直接刪a欄。若a基本且值0，其列為 $a=-\Sigma\ a_{aj}\ x_j$；若存在非零係數，任選該x_j入基、a離基。這個θ=0的代數pivot不改目前點，係數可正可負，不是改善目標的ratio選步。

若整列係數皆0，則a=0是冗餘等式，刪此列與a即可。一般等式／多人工變數實作必處理此分支；對獨立的完整slack矩陣可另由row rank排除，但不應除以0。

最後把**原目標**中的基本變數逐一用目前列代入，得到只含非基本變數的目標；不可只改標題或把auxiliary目標−a當原答案。這時完成Phase I，才用新的可行basis跑Phase II。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Phase I 與原目標的完整操作例

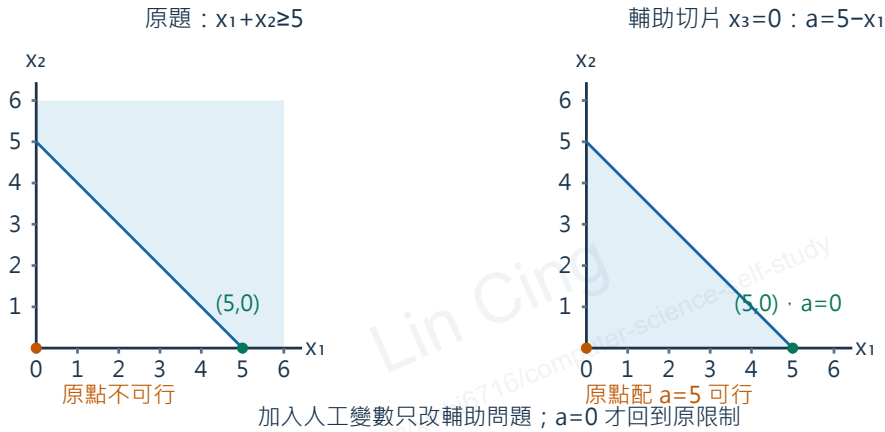
原題 $\min\ z=2x_1+x_2\ \cdot\ x_1+x_2\geq 5\ \cdot\ x\geq 0$。Surplus $x_3\geq 0$給$x_1+x_2-x_3=5$；引人工a：$a=5-x_1-x_2+x_3$，Phase I最小化w=a（等價最大化−a）。

狀態	基本列	非基本0時	目標
初始	$a=5-x_1-x_2+x_3$	$a=5$	$w=5$
x_1入、a離	$x_1=5-x_2+x_3-a$	$x_1=5$	$w=a$，最小0
刪a、代原目標	$x_1=5-x_2+x_3$	$x_1=5$	$z=10-x_2+2x_3$
x_2入、x_1離	$x_2=5-x_1+x_3$	$x_2=5$	$z=5+x_1+x_3$

最小化終止時係數≥0，故最優z=5。起始$(x_1,x_2)=(0,0)$對原題不可行；配a=5後只對輔助題可行。這個單等式例用一個人工欄；上一頁的共用a模型則直接適用所有Ax≤b列。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Phase I 與原目標的完整操作例 （續）



加入人工變數只改輔助問題；a=0 才回到原限制

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不可行也要有算完的證書

原題 $x\le 1$，$x\ge 2$，$x\ge 0$，加共用a後為$x-a\le 1$；$-x-a\le -2$。相加得$-2a\le -1$，故$a\ge 1/2$；取$x=3/2$、$a=1/2$兩式恰等號，所以auxiliary最大$-a=-1/2<0$，原題不可行。

完整字典：$s_1=1-x+a$、$s_2=-2+x+a$。先讓a入、s_2離，得$a=2-x+s_2$、$s_1=3-2x+s_2$、$w=-2+x-s_2$。x入基兩ratio為2、3/2，s_1離基，得

$x=3/2-s_1/2+s_2/2$、$a=1/2+s_1/2+s_2/2$、$w=-1/2-s_1/2-s_2/2$。

兩個reduced costs非正，確已最優，不是因尚有人工量就猜不可行。亦可直接把原兩個相反限制相加得到$0\le -1$作獨立矛盾證書。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

三種結果與基本定理的依賴關係

Phase I先有可行auxiliary基底，lex迴圈有限終止且auxiliary不向上無界；最優<0回infeasible，最優0清理人工欄並恢復原目標。

Phase II仍以可行字典啟動，lex規則保證有限終止。若沒有正分母，交付可行點與改善ray，證unbounded；否則終止時reduced costs≤0，交付可行基本解與同值dual，證finite optimum。

所以每個本章標準形式LP恰屬不可行、可行而目標無界、可行且達有限最優之一。非嚴格限制與有限維LP結構很關鍵；若改成$\max\ x\cdot 0\le x<1$，上確界1不被任何點達到，便不是這個三分結論的範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 29。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

27 多執行緒演算法

Part VII Selected Topics

本章定位

動態多執行緒以依賴DAG分開工作總量、關鍵路徑與排程；本章由spawn/sync與競爭修復，推導矩陣乘法和平行merge/sort的完整算法、一般證明及空間條件，避免把平均平行度當收益截止點。

前置知識

DAG 排程、並行成本模型與遞迴分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用 spawn、sync 描述動態多執行緒計算。
- 區分 work、span、parallelism。
- 分析矩陣乘法與合併排序的並行界限。
- 27.1 The basics of dynamic multithreading
- 27.2 Multithreaded matrix multiplication
- 27.3 Multithreaded merge sort

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

27.1 The basics of dynamic multithreading

動態多執行緒模型讓程式描述哪些計算可以並行，由排程器決定何時、在哪個處理器執行。邏輯task不是固定綁定的OS thread；spawn不承諾立刻取得另一個核心。

本章採相同速度的P個理想處理器與循序一致的共享記憶體：每條指令可看成某個符合各依賴的交錯順序，load/store各自是單一步驟；讀、加、寫的合成操作不自動具有原子性。單位成本算術、索引、spawn/sync為常數工作，排程器本身的時間、頻寬、cache及通訊成本不計入此理想界。

非空有限計算用DAG表示。節點可是一個strand（不含平行控制的一段順序指令），帶自己的正執行時間；空strand可刪除，要用單位節點證明時，可再拆成單位指令鏈。控制與資料依賴決定有向邊，而不是把畫面上同層的節點一律當成可並行。

Spawn、call、sync 的確切語義

`x = spawn F(...)`允許F與呼叫者後續程式並行；普通`y = G(...)`則呼叫者須等待G返回。`sync`等待本程序此前spawn的child全部完成，才執行後面的指令；它不是全系統的barrier。

spawn參數先在parent求值。child的區域變數屬該次呼叫，指向共享陣列的參數仍可能讓多task存取同一格；不能只看變數名稱不同就判獨立。讀spawn回傳值之前必須有相應依賴，通常明寫sync。

本章程序返回前隱含sync，但若先求`return x+y`的值才返回，隱含等待不足以允許提早讀x；因此先sync再算回傳式。刪掉spawn/sync並將parallel for改普通for得到serialization；它能解出正確答案，不代表所有平行交錯也正確。

Fibonacci 範例的程式與正確性

```
P-FIB(n):                // nonnegative integer n
    if n <= 1: return n
    x = spawn P-FIB(n-1)
    y = P-FIB(n-2)
    sync
    return x+y
```

兩個child只讀自己的n並回傳各自結果，沒有共享寫入。n下降使呼叫終止；基例F0=0、F1=1，加上歸納假設與sync後的x+y，得到正確Fn。若把兩個child都spawn，仍必等兩者後才能相加，漸近work/span不變，只改常數。

這個版本故意沒有memoization，同一參數會算多次。高平行度不等於work有效率：順序迭代法只需O(n)次算術，不能因遞迴版本可以占用更多核心，就忽略其指數工作量。

同一 strand 模型的 work 與 span

為了逐點驗算，將每個非基例呼叫分為三個單位strand：pre（到spawn）、cont（到普通第二呼叫）、join（sync後相加返回）；每個基例為一個單位strand。pre先於左child及cont，cont先於右child，兩個child結束都先於join。

令W為全部strand成本，D為最長依賴路徑成本，則

$$W_n = W_{n-1} + W_{n-2} + 3, \quad W_0 = W_1 = 1;$$

$$D_n = 2 + \max(D_{n-1}, 1 + D_{n-2}), \quad D_0 = D_1 = 1.$$

n	Fn	Wn	Dn
2	1	5	4
3	2	9	6
4	3	17	8

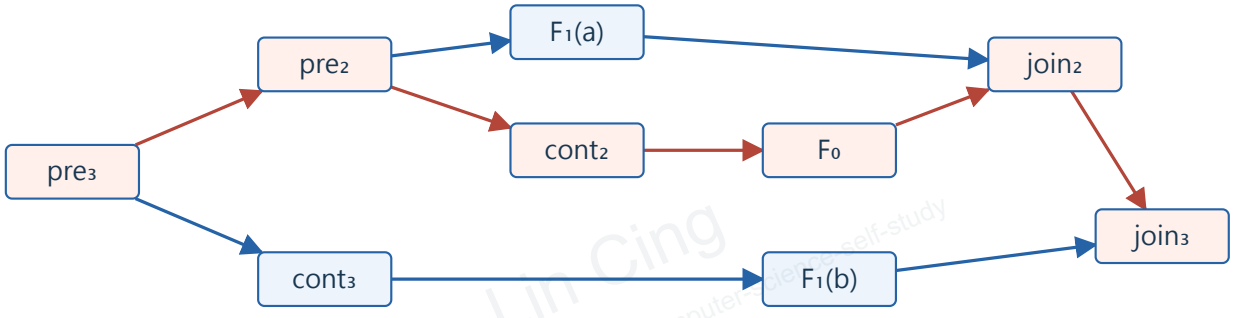
同一 strand 模型的 work 與 span （續）

Wn=4F(n+1)−3可直接代入歸納，Dn=2n對n≥2成立。令 $\varphi = (1+\sqrt{5})/2$；由ϕ²=ϕ+1，歸納得ϕ^(n−2)≤Fn≤ϕ^(n−1)（n≥2）：n=2、3可核對，兩界的相鄰和仍是下一個ϕ次方。因此W=Θ(ϕ^n)、D=Θ(n)。這是明列的單位strand模型，不把一個完整遞迴呼叫誤算成一個O(1)節點。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

九個 strand 的依賴可逐條追蹤

P-FIB(3) 的完整九個 strand：箭頭是先後依賴



每格恰為一個單位 strand；W=9。紅色最長路徑含6格，因此 D=6。

圖的pre3先分到pre2及cont3；cont3才呼叫右側F1。pre2也先分到其F1與cont2，cont2再呼叫F0。兩個基例都完成才可做join2，join2與右側F1都完成才可做join3。

紅路徑pre3→pre2→cont2→F0→join2→join3共6個節點，W=9、D=6。另一條pre3→cont3→F1→join3只有4個節點。圖上箭頭是依賴方向，不是把同一呼叫的所有節點排成一條長鏈。

Work law、span law 與可達的加速比

記 $W = T_1$、$D = T_\infty$、實際P處理器時間 T_P。每個時間單位最多完成P單位工作，且關鍵路徑不可並行，故

$$T_P \geq \max\{W/P, D\};$$

單位節點離散模型還有 $T_P \geq \lceil W/P \rceil$。加速比 $S_P = W/T_P \leq \min(P, W/D)$，效率 $S_P/P \leq 1$。平均平行度A=W/D限制加速比；P>A便無法有完美的P倍加速，不代表第P+1顆處理器必然沒有收益。

順序合成兩段時，W相加、D也相加；若兩段獨立地fork後join，W仍相加，D取兩段最大值，另加明列的fork/join成本。空計算W=D=0直接時間0，不把0/0稱為平行度。

Greedy 排程的完整上界證明

每個時間步，只要有至少P個ready節點便執行P個；不足P時全部執行，這是greedy。令完整步C、不完整步I，總時間T=C+I。計算尚未結束時，每個不完整步至少執行一個節點，故W≥PC+I。

不完整步會移除剩餘DAG的所有source。步後的最長路徑，可在該步開始前的DAG中向前延伸到至少一個被移除的source；若步後已空，步前span也至少1。因此每個不完整步使剩餘span至少減1，最多有D個不完整步，I≤D。代入得

$$T \leq \frac{W - I}{P} + I \leq \frac{W - D}{P} + D \leq W/P + D.$$

最後一步使用P≥1使I的係數1−1/P非負。單位節點證明可對整數長度strand拆鏈後套用；它不是無成本處理任意不可切割、長度未知task的另一個模型。

兩倍近似與 slackness 的量化

最優排程也受 $T_P^* \geq \max(W/P, D)$ 限制，故greedy的T≤W/P+D≤2T*。更精確地可得 $(2 - 1/P)T_P^*$ 上界，但無需預知未來DAG就有常數比保證。

定義slackness $\sigma = W/(PD) = A/P$。對D>0，上下界給

$$1 \leq \frac{T_P}{W/P} \leq 1 + \frac{1 - 1/P}{\sigma} \leq 1 + 1/\sigma.$$

所以σ很大時，理想greedy時間接近W/P；這是數學界，不是對真機頻寬或排程器開銷的實測。最優排程可以停用多餘處理器，所以其時間隨P不增；任意實作的測得時間另受開銷與排程選擇影響。

平行度30，第31顆仍有收益

697個互相獨立的單位工作全部完成後，才能開始23個單位工作的依賴鏈。W=697+23=720，D=1+23=24，A=30。

P	獨立階段的完整配置	階段時間	鏈時間	精確最短時間
30	23輪各30項，再7項	24	23	47
31	22輪各31項，再15項	23	23	46

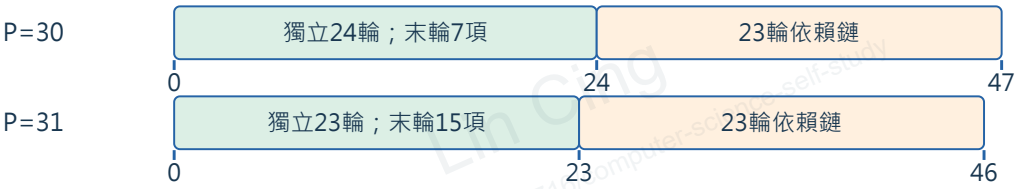
所有前驅都完成前鏈不能開始，所以ceil(697/P)+23同時是可行排程與下界。S30=720/47≈15.32、S31=720/46≈15.65，均≤30，但增加處理器確實仍有收益。P≥697才可把第一階段縮成1輪、達到D=24。

平行度30，第31顆仍有收益 （續）

相同 DAG：31個處理器仍比30個快

W=697+23=720
D=1+23=24

平均平行度 W/D=30；以下兩列使用相同時間比例尺。



$T_{30}=47$、$T_{31}=46$：加速比至多30，不代表第31個處理器沒有收益。

減 work 但增加 span，可能改變優劣

兩個正確版本分別有(W,D)=(1024,4)、(768,16)，都用理想greedy排程。

P=8時，第一版時間至少1024/8=128，第二版時間至多768/8+16=112，因此第二版一定較快。P=128時，第一版至多1024/128+4=12，第二版至少16，因此第一版反而一定較快。

這個判定用到不重疊的上下界，不把W/P+D的上界誤當精確時間。若兩版本的可行時間區間重疊，單靠W,D未必能判定哪個實際較快。少量核心上的單次時間，也不足以識別work與span各自的改變。

Parallel for 如何以 fork/join 實作

以下與合併算法均採0-based半開區間[l,r)，空區間l=r。

```
PAR-FOR(l,r,body):
    if l == r: return
    if r-l == 1: body(l); return
    m = floor((l+r)/2)
    spawn PAR-FOR(l,m,body)
    PAR-FOR(m,r,body)
    sync
```

n個非空迭代恰對應n葉、n−1內節點；每個內節點常數spawn/sync工作，所以額外work $O(n)$，不會憑空多出$n \lg n$。最長呼叫與join路徑深度$\Theta(\lg n)$，因此總span是$\Theta(\lg n)$+各迭代span最大值，前提是各迭代真的獨立。

若用普通for依序spawn n個task，parent本身就有$\Theta(n)$長的發出鏈，不能宣稱同樣對數span。平行迴圈結尾的sync也確保下一段讀取所有結果時已完成。

Matrix-vector：只平行不同 row 是安全的

```
MAT-VEC(A,x):
    n = A.rows
    allocate y[n]
    parallel for i = 0 to n-1:
        y[i] = 0
        for j = 0 to n-1:
            y[i] += A[i,j]*x[j]
    return y
```

每個i獨占y[i]，A,x只讀；內圈第j步後是前j+1個乘積的和，所以正確。work $\Theta(n^2)$，span $\Theta(\lg n)+\Theta(n)=\Theta(n)$，平均平行度$\Theta(n)$。n=0回空向量。

不能只把內圈也改parallel for：各j對同一y[i]做讀改寫，會產生determinacy race（未由依賴排序的衝突存取）。兩個邏輯並行存取同格、至少一個寫入便有此衝突；即使碰巧寫相同值，也不能用「跑過一次正確」證明獨立。

循序一致仍會遺失更新

遺失更新：逐指令讀值、暫存值與最後寫回

初始 x=0；每個 task 內部仍依 load → add → store。

時間步	Task 1	Task 2	共享 x
1	load $r_1 \leftarrow x = 0$		0
2		load $r_2 \leftarrow x = 0$	0
3	add $r_1 \leftarrow r_1+1 = 1$		0
4	store $x \leftarrow r_1$		1
5		add $r_2 \leftarrow r_2+1 = 1$	1
6		store $x \leftarrow r_2$	1

兩者結尾 sync 後讀到1：等待完成不能把三條指令變成不可分割操作。

圖的交錯尊重各task的load→add→store，最後x卻為1；如果第一個task全部完成才換第二個，會得到2。循序一致只限制單條指令的交錯，不把三條指令合成原子操作。

本章以互不衝突的分工或reduction消除此類競爭。原子加或鎖也可建立依賴，但其同格序列化、工作量與span必須另算；不能保留「全部加法同時完成」的分析。

用 reduction 正確地平行內積

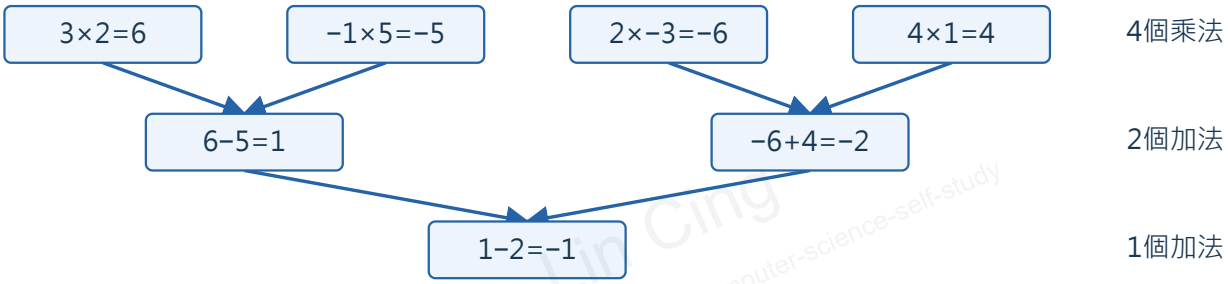
```
DOT(Arow,x,l,r):
    if l == r: return 0
    if r-l == 1: return Arow[l]*x[l]
    m = floor((l+r)/2)
    left = spawn DOT(Arow,x,l,m)
    right = DOT(Arow,x,m,r)
    sync
    return left+right
```

每個child只回傳自己的值，沒有共同累加格；結合律與歸納保證結果是區間內積。work為$\Theta(n)$，span $\Theta(\lg(n+1))$。把MAT-VEC每個row改成這個DOT，得到$\Theta(n^2)$work、$\Theta(\lg n)$span（$n \geq 2$）。

例row=[3,−1,2,4]、x=[2,5,−3,1]，乘積[6,−5,−6,4]，第一層和[1,−2]，最後−1。

用 reduction 正確地平行內積 （續）

內積不是共寫一個總和：四個私有乘積沿樹合併



算術 DAG：work=7、span=3；fork/join 另有 Θ(log n) 控制深度。

對浮點加法，固定的樹可能和順序累加有不同捨入結果；正確性證明先在精確、可結合算術下成立，不能順手承諾逐bit相同。

粒度與記憶體配置的模型

將n個常數成本迭代分成每批最多g個、以平衡PAR-FOR發出各批，1≤g≤n，work Θ(n)，span Θ(lg⌈n/g⌉+g)。較大g減少task常數開銷，也增加每葉序列成本；是否值得依P及slackness而定。

本章allocate取得未初始化、互不重疊儲存空間的描述符，抽象為常數span；若實作需要逐格初始化，必須計入相應工作並以parallel loop初始化才保留對數span。不是把一個串行清零n^2格的操作藏進O(1)。

矩陣區塊用(row起點、column起點、stride、階數)的view表示，不隱藏整塊複製。任務控制區、分配器、cache與記憶體頻寬在實作時另有成本；以下數學界皆在已明列的理想模型內。

27.2 Multithreaded matrix multiplication

```
P-MULTIPLY(A,B):
    n = A.rows                 // both n by n
    allocate C[n,n]
    parallel for i = 0 to n-1:
        parallel for j = 0 to n-1:
            C[i,j] = 0
            for k = 0 to n-1:
                C[i,j] += A[i,k]*B[k,j]
    return C
```

每個(i,j)獨占一格輸出，內圈仍依序累加。work $\Theta(n^3)$，span兩層發出樹$\Theta(\lg n)+\Theta(\lg n)$，再加一條內圈$\Theta(n)$，總$\Theta(n)$，平均平行度$\Theta(n^2)$。兩個parallel關鍵字沒有消除每格的n項依賴。

若每個Cij改由前述DOT類型的二分內積回傳，work仍$\Theta(n^3)$，span則$\Theta(\lg n)$，平均平行度$\Theta(n^3/\lg n)$。直接並行寫入Cij不是這個reduction算法。

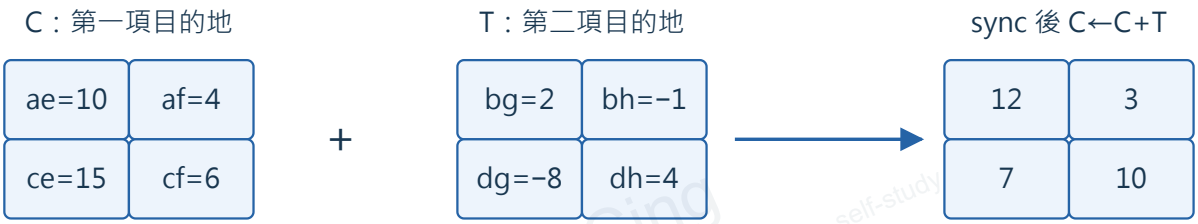
八次子乘法必須寫八個不同目的地

令A=[[a,b],[c,d]]、B=[[e,f],[g,h]]，此處字母是同階區塊。先計C=[[ae,af],[ce,cf]]與暫存T=[[bg,bh],[dg,dh]]，八個乘積各有獨占目的地。sync後才做C=C+T。

數例A=[[2,−1],[3,4]]、B=[[5,2],[−2,1]]：C暫時[[10,4],[15,6]]，T=[[2,−1],[−8,4]]，相加得[[12,3],[7,10]]。

八次子乘法必須寫八個不同目的地 （續）

八個獨立乘法：目的地分開，回收時才相加



八個乘法各寫 C 或 T 的一格；先等待全部乘法，再逐格相加。

A=[[2,−1],[3,4]]，B=[[5,2],[−2,1]]；同格的兩項沒有同時做 += 。

若把ae與bg兩個task都寫成C11+=product，兩者便衝突。分出T是在用額外空間解除寫入依賴，不是只加spawn就能平行。

遞迴矩陣乘法的完整程序

```
P-REC(C,A,B):
    n = A.rows
    if n == 1: C[0,0] = A[0,0]*B[0,0]; return
    allocate T[n,n]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    view C and T into four n/2 by n/2 blocks
    spawn P-REC(C11,a,e); spawn P-REC(C12,a,f)
    spawn P-REC(C21,c,e); spawn P-REC(C22,c,f)
    spawn P-REC(T11,b,g); spawn P-REC(T12,b,h)
    spawn P-REC(T21,d,g); P-REC(T22,d,h)
    sync
    parallel for i = 0 to n-1:
        parallel for j = 0 to n-1: C[i,j] += T[i,j]
```

前提為同階方陣、階數是2的冪，A與B只讀，C不與它們重疊。任意正階數先補零到下一個2冪再取回結果，n=0直接回空矩陣。T及每個child工作區也獨立；八個spawn/call數量是固定常數，不是n長的發出鏈。

八路矩陣乘法的歸納與遞迴成本

基例寫出正確單乘積。歸納假設各n/2子乘積正確，八個child因目的地不同而無競爭；sync確保全部乘積可讀，再逐格相加，依區塊乘法恆等式得到AB。只有共同讀A、B不構成衝突。

$$W_M(n) = 8W_M(n/2) + \Theta(n^2) = \Theta(n^3),$$
$$D_M(n) = D_M(n/2) + \Theta(\lg n) = \Theta(\lg^2 n).$$

span沿八個child中一條最長路徑，不是八者相加；每層回收的nested parallel for又有Θ(lg n)發出／join深度，不能當O(1)。n=2^L時Σr=1..L r=Θ(L^2)，同時給上、下界。n=4有8^2=64個純量乘法葉；平行化沒有少做乘法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-s

Strassen 平行化的三個同步階段

沿用第4章M編號，先以平行逐格加減準備(a+d)、(e+h)、(c+d)、(f−h)、(g−e)、(a+b)、(c−a)、(e+f)、(b−d)、(g+h)。輸入準備完成後，七乘積同時可執行：

$$M_1 = (a+d)(e+h), \quad M_2 = (c+d)e,$$
$$M_3 = a(f-h), \quad M_4 = d(g-e),$$
$$M_5 = (a+b)h, \quad M_6 = (c-a)(e+f),$$
$$M_7 = (b-d)(g+h).$$

七者完成後sync，再平行回收C11=M1+M4−M5+M7，C12=M3+M5，C21=M2+M4，C22=M1−M2+M3+M6。不能在輸入和差未完成前spawn消費它們，也不能在乘積未完成前回收。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strassen 的 work、span 與平行工作區

每層七個同階child，常數個矩陣加減：

$$W_S(n) = 7W_S(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}),$$
$$D_S(n) = D_S(n/2) + \Theta(\lg n) = \Theta(\lg^2 n).$$

正確性由第4章已展開的四個區塊恆等式及遞迴歸納，平行化另需上述三階段依賴。其平均平行度 Θ(n^(lg7)/lg²n)小於八乘法版本，卻具有較低work；平行度的比值不是單獨選算法的目標。

順序深度優先的O(n²)暫存不能直接移給所有child同時展開的版本。若每個呼叫分配自己的Θ(n²)工作區並保持到join，八路的完全展開空間滿足Q(n)=8Q(n/2)+Θ(n²)=Θ(n³)；七路則為Θ(n^(lg7))。有限P與工作區重用可降低峰值，但必同時核對排程與span，不能沿用一個未實作的空間界。

平行 Strassen 的可執行規格

定義PADD(X,Y,s)配置獨立R，以兩層parallel for寫R[i,j]=X[i,j]+s・Y[i,j]，等待迴圈完成再回R。取s=±1；其work Θ(n²)、span Θ(lg n)。以下A、B同階且先補到2冪，n=0由外層直接回空矩陣。

```
P-STRASSEN(A,B):
    n = A.rows
    if n == 1: return [[A[0,0]*B[0,0]]]
    view A as [[a,b],[c,d]], B as [[e,f],[g,h]]
    S1 = PADD(a,d,+1); S2 = PADD(e,h,+1)
    S3 = PADD(c,d,+1); S4 = PADD(f,h,-1)
    S5 = PADD(g,e,-1); S6 = PADD(a,b,+1)
    S7 = PADD(c,a,-1); S8 = PADD(e,f,+1)
    S9 = PADD(b,d,-1); S10 = PADD(g,h,+1)
    M1 = spawn P-STRASSEN(S1,S2)
    M2 = spawn P-STRASSEN(S3,e)
    M3 = spawn P-STRASSEN(a,S4)
    M4 = spawn P-STRASSEN(d,S5)
```

平行 Strassen 的可執行規格 （續）

```
    M5 = spawn P-STRASSEN(S6,h)
    M6 = spawn P-STRASSEN(S7,S8)
    M7 = P-STRASSEN(S9,S10)
    sync
    allocate C[n,n], viewed as four blocks
    parallel for i = 0 to n/2-1:
        parallel for j = 0 to n/2-1:
            C11[i,j] = M1[i,j]+M4[i,j]-M5[i,j]+M7[i,j]
            C12[i,j] = M3[i,j]+M5[i,j]
            C21[i,j] = M2[i,j]+M4[i,j]
            C22[i,j] = M1[i,j]-M2[i,j]+M3[i,j]+M6[i,j]
    return C
```

十個PADD在此依序呼叫，數量固定，故總span仍Θ(lg n)；先全部完成才spawn七乘法，與三階段依賴一致。沒有把任意數量的串行發出忽略掉。

27.3 Multithreaded merge sort

只把兩個遞迴sort並行、仍用順序MERGE，並不充分：

```
PAR-SORT-SERIAL-MERGE(A,l,r):
    if r-l <= 1: return
    m = floor((l+r)/2)
    spawn PAR-SORT-SERIAL-MERGE(A,l,m)
    PAR-SORT-SERIAL-MERGE(A,m,r)
    sync
    serial MERGE the two sorted halves using an auxiliary buffer
```

work為2W(n/2)+Θ(n)=Θ(n lg n)，span卻是D(n/2)+Θ(n)=Θ(n)，平均平行度僅Θ(lg n)。合併前的sync必要，否則會讀到未排序完成的run；要進一步縮短span，必須平行化merge本身。

二分切點先定義半開區間與平手規則

兩個輸入run位於只讀陣列T的[l1,r1)、[l2,r2)，輸出O[o,o+N)不得覆寫T，N為兩段長度和。為保證穩定性，每個record以(key,原始位置)作總序比較；原始位置唯一，相同key仍按原先順序。

```
LOWER-BOUND(T,l,r,x):
    while l < r:
        m = floor((l+r)/2)
        if T[m] < x: l = m+1
        else: r = m
    return l
```

不變量是原區間中小於x者的切點位於當前[l,r]；在原輸入run內，當前l左方都<x、r右方（含r）都≥x。每次依中點比較保留可能切點，長度嚴格縮小；空run直接回l，永遠不讀不存在的T[l]。工作及span O(1+lg(n2+1))，明列空run的常數處理。

可處理空 run 的完整 Parallel Merge

```
P-MERGE(T,l1,r1,l2,r2,O,o):
    n1 = r1-l1; n2 = r2-l2
    if n1 < n2: swap (l1,r1) with (l2,r2); swap n1,n2
    if n1 == 0: return
    m = floor((l1+r1)/2)
    j = LOWER-BOUND(T,l2,r2,T[m])
    q = o+(m-l1)+(j-l2)
    O[q] = T[m]
    spawn P-MERGE(T,l1,m,l2,j,O,o)
    P-MERGE(T,m+1,r1,j,r2,O,q+1)
    sync
```

先把較長run放第一描述符；交換的是view，不移動或重新標記record。若第一段空，第二段也空；若僅第二段空，仍從第一段取median逐步定位，不讀空段。每次拿走一個pivot，兩個child總長N−1，因此有限輸入必終止。

輸出位置、正確性與穩定性的一般證明

左側應有m−l1個第一run元素、j−l2個第二run元素，總長q−o。它們都在總序上小於pivot；其餘元素都大於pivot。故pivot的全域輸出位置是q，兩個child輸出區間恰為[o,q)與[q+1,o+N)，彼此及pivot均不重疊。

對N歸納：N=0空合併正確；兩child均較小，由歸納假設正確合併相應元素。左答案、pivot、右答案串接已排序且每個輸入record恰出現一次，得到完整merge。T只讀、O的寫入區間互斥，所以沒有determinacy race，sync後全部結果可用。

相同key按全域原始位置比較，輸出保持原先相對次序，因此穩定。若只比key且一律使用同一lower_bound規則，長度交換可能把相等的右run元素放到左run之前；普通排序正確不自動等於穩定。

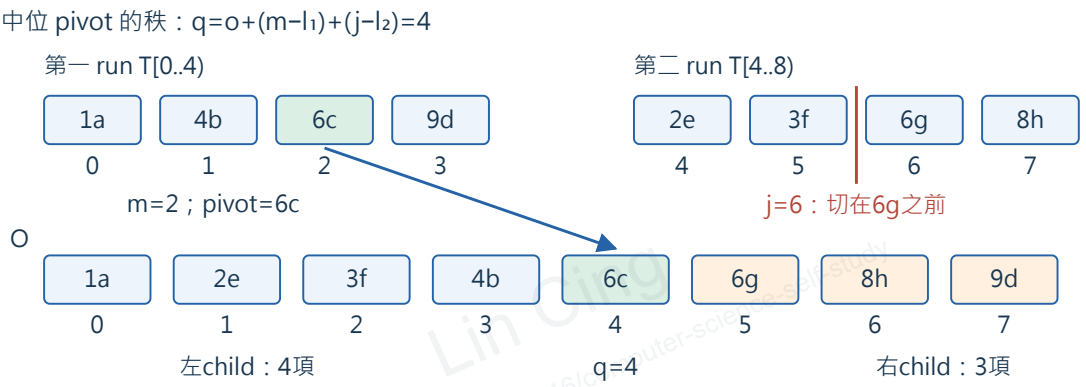
八個 record 的根切點與二分步驟

T=[1a,4b,6c,9d | 2e,3f,6g,8h]，字母依a..h表示原始位置0..7；比較先key再原始位置。
l1=0,r1=4,l2=4,r2=8,o=0。第一run長4，m=2、pivot=6c。

當前搜尋[l,r)	mid	T[mid]與6c比較	更新
[4,8)	6	6g>6c（key相等、位置較後）	r=6
[4,6)	5	3f<6c	l=6
[6,6)	—	空搜尋區間	回j=6

q=0+(2−0)+(6−4)=4；左child共4元素，右child共3元素。

八個 record 的根切點與二分步驟 (續)



比較 (key, 原始位置)：6c < 6g；左區、pivot、右區的輸出互不重疊。

同一數例的全部輸出寫入

以下為一個合法深度優先serialization的pivot寫入順序；平行執行可以改交錯，但目的格相同、各寫一次。

寫入序	pivot	q（0-based）	來源／輸出意義
1	6c	4	根的左右分界
2	4b	3	根左段的最後元素
3	3f	2	較長run交換後的median
4	2e	1	小run的切點
5	1a	0	另一run已空
6	8h	6	根右段交換後的median
7	6g	5	右段左child
8	9d	7	右段右child

同一數例的全部輸出寫入 （續）

最終O=[1a,2e,3f,4b,6c,6g,8h,9d]，兩個6仍c在g之前。即使改成先跑右child，只讀T及唯一輸出格也保證相同答案；不是靠整個陣列寫入順序維持排序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Merge 最壞 span 的上界與下界

較長run長n1≥n2，取median後任一child至多⌊n1/2⌋+n2≤(N+n2)/2≤3N/4。即使一側切點落在端點，這個上界仍成立。二分搜尋至多O(lg(N+1))，所以

$$D_M(N) \le D_M(3N/4) + O(\lg(N+1)) = O(\lg^2(N+1)).$$

沿路每次固定比例縮小，有O(lg N)層，第r層剩餘log大小O(lg N−cr)，等差和給平方對數。

下界用近等長交錯run，例如[0,2,⋯,2m−2]與[1,3,⋯,2m−1]，m是2冪。左child持續成為同型的兩個等長交錯run，長度每層減半，且搜尋另一run的median切點仍需Θ(lg m)比較。一路累加lg m+(lg m−1)+⋯給Ω(lg²m)，所以最壞span為Θ(lg²N)。若另一run始終空，搜尋只需常數、span則Θ(lg N)；不能把最壞界稱為每個輸入的精確成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性 work：一次寫出不等於已付清搜尋

每個非空呼叫挑走一個不同pivot，所以有N個非空呼叫；各生兩child，含空返回共2N+1個呼叫。N次輸出給Ω(N)，但O(N)上界還必須支付每個非空節點的二分搜尋，不能只說「每元素寫一次」。

大小m的節點非遞迴工作≤c lg(m+1)。把此成本平均分攤給它仍含有的m個輸入record，每個收 c・lg(m+1)/m。固定record x只在一條ancestor鏈上被收費，直到自己被選成pivot為止。

從最後一次含x的呼叫向根反數r層，因child≤3parent/4，該層大小至少(4/3)^r。下一頁將此一般不等式求和，連空child也可處理，不需要取lg0或聲稱小規模的兩個child都至少N/4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

每個 record 的搜尋費總和有常數界

$g(u)=\lg(u+1)/u$ 在u>0遞減，因導數分子的符號等於u/(u+1)−ln(u+1)<0；後者由積分 ∫ 0^u 1/(1+t)dt>u/(1+u)得到。因此反數r層的分攤費至多

$$c\frac{\lg((4/3)^r+1)}{(4/3)^r}\le c\,[1+r\lg(4/3)](3/4)^r.$$

幾何和Σq^r=1/(1−q)，以及由有限和求導／取極限所得Σr q^r=q/(1−q)²，對q=3/4都收斂。因此每個record總費O(1)，N個record共O(N)；再加N+1個空呼叫的常數費，總work O(N)。與輸出下界合併，W_M(N)=Θ(N)。

這是對任意合法切點的一般計費證明；空run與不平衡的小child已包含，不以有限例子取代上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩個 buffer 的完整 Parallel Merge Sort

先將輸入record附上原始位置，令X與Y兩個獨立長n陣列都含相同輸入（一次parallel copy），呼叫SORT(X,Y,0,n)，最後取Y。

```
SORT(src,dst,l,r):
    if l == r: return
    if r-l == 1: dst[l] = src[l]; return
    m = floor((l+r)/2)
    spawn SORT(dst,src,l,m)
    SORT(dst,src,m,r)
    sync
    P-MERGE(src,l,m,m,r,dst,l)
```

每個子呼叫交換buffer角色；parent的兩個已排序run會留在src，sync後merge到dst。P-MERGE輸入與輸出必須是不同實體buffer。這個雙buffer寫法避免每個sort呼叫再配置長度n的暫存，不把原地寫入與讀取尚未消費的輸入混用。

Buffer 角色交換的正確性與競爭檢查

每個SORT進入時，兩個buffer的當前區間都仍含該區間的原始輸入。根由copy建立；parent在啟動兩child前未改自身區間，兩個child又只碰各自互斥的半區，故每個child進入時也成立。

基例把唯一record複製到dst。由歸納假設，兩child結束後，parent的src左右兩run分別排序且元素不變；sync後的P-MERGE把兩者穩定合併到dst。歸納完成排序規格；parent無需承諾src最後仍與dst相同，只需在呼叫入口滿足條件。

兩child即使交換buffer角色，也仍在相同的一對全域陣列中寫互斥索引區間；parent只在sync後合併。總資料workspace $O(n)$，峰值task描述符$O(n)$、每條呼叫鏈$O(\lg n)$；初始化copy工作$\Theta(n)$、span $\Theta(\lg n)$，不改後述排序階數。

排序的緊界不能只把最壞成本相加

每層merge work Θ(n)，有Θ(lg n)層，故W_S(n)=Θ(n lg n)。span上界是

$$D_S(n) \le D_S(\lceil n/2 \rceil) + O(\lg^2 n) = O(\lg^3 n).$$

要證最壞下界，需同一輸入在沿途各層都產生昂貴merge。對n=2^L，按L位元反轉索引的順序放入鍵；n=8例[0,4,2,6,1,5,3,7]。根的兩半排序後是偶數與奇數交錯run；每個子問題的相對秩亦是相同構造。故每層都有前述Ω(lg²(size))的merge span，且parent merge在兩child都完成後才開始。

沿一條child鏈相加Σr=1..L r²=Θ(L³)，得到二冪規模的最壞D_S(n)=Θ(lg³n)。平均平行度為Θ(n/lg²n)，不是把merge的平方對數錯當sort的span。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

非二冪規模也能構造同時昂貴的各層

任意n可遞迴安排各區間的相對秩：n偶數時，前半放偶秩、後半放奇秩；n奇數時，較短的前半放奇秩、較長的後半放偶秩。再對兩半各自的有序鍵集合做相同安排。每個sort節點排序後的兩run都互相交錯，長度差至多1；二冪時就是前頁的位元反轉族。

對這種merge，較長run的median左方有m個本run元素，另一run左方有m或m+1個。左child仍是長度差至多1的交錯run，總長介於N/2−1與N/2+1；N≥8時，兩run的長度也都為Θ(N)。所以沿該路徑有Θ(lg N)層，每層二分搜尋須Θ(lg N)比較（規模隨層數固定比例下降），總span為Θ(lg²N)。

同一遞迴秩配置讓每個sort節點都滿足此條件。沿較大的child走，規模每層約減半，且須等它完成後才開始parent merge，故可串接各層關鍵路徑，得到Ω(lg³n)。這補足任意n的緊下界，不只在不同層各自選互不相容的最壞輸入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一模型下比較 work 與最壞 span

算法	Work	最壞 span	平均平行度
遞迴P-FIB	$\Theta(\varphi^n)$	$\Theta(n)$	$\Theta(\varphi^n/n)$
row平行matrix-vector	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$
reduction matrix-vector	$\Theta(n^2)$	$\Theta(\lg n)$	$\Theta(n^2/\lg n)$
雙外圈平行matrix product	$\Theta(n^3)$	$\Theta(n)$	$\Theta(n^2)$
八路遞迴matrix product	$\Theta(n^3)$	$\Theta(\lg^2 n)$	$\Theta(n^3/\lg^2 n)$
平行Strassen	$\Theta(n^{\lg 7})$	$\Theta(\lg^2 n)$	$\Theta(n^{\lg 7}/\lg^2 n)$
DOT reduction matrix product	$\Theta(n^3)$	$\Theta(\lg n)$	$\Theta(n^3/\lg n)$
Parallel merge	$\Theta(n)$	$\Theta(\lg^2 n)$	$\Theta(n/\lg^2 n)$
Parallel merge sort	$\Theta(n\lg n)$	$\Theta(\lg^3 n)$	$\Theta(n/\lg^2 n)$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一模型下比較 work 與最壞 span （續）

表中漸近式採n充分大；空與單元素依程序基例。Work有效率、span小、空間少是不同目標；對固定P，還要把W/P、D及記憶體需求一起比較，不以一個平行度比值代替整個分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 27。

Cormen et al., *Introduction to Algorithms*, 3rd ed., §§27.1-27.3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A 求和

Part VIII Appendix

本章定位

從有限恆等式推導等差、冪和、幾何、消去與乘積，再用固定常數歸納、尾比率、dyadic切分與積分雙界分析迴圈；明列無限級數及換序的收斂前提。

前置知識

等差、等比、對數與積分估計。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 使用求和公式與線性性。
- 以積分、分割與幾何級數建立界限。
- 避免把有限和與無窮級數混用。
- A.1 Summation formulas and properties
- A.2 Bounding summations

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.1 Summation formulas and properties

$\sum_{k=1}^{n} a_k$是有限加總，n=0時定義為0；$\prod_{k=1}^{n} a_k$是有限乘積，n=0時為1。空sum與空product是加法／乘法單位元素，使迴圈零次執行的公式也成立。有限sum可任意重排，並有$\sum(ca_k + b_k) = c\sum a_k + \sum b_k$。

無限級數$\sum_{k=1}^{\infty} a_k$定義為partial sums S_n的極限，不是「把∞代進有限公式」。若極限不存在或無有限值即發散。$a_k \to 0$是收斂必要條件（$a_k = S_k - S_{k-1}$），卻非充分；1/k趨0但harmonic sum發散。

若$\sum_{k=1}^{\infty} |a_k|$收斂，稱絕對收斂，可以任意重排而不改和。條件收斂如$\sum_{k=1}^{\infty} (-1)^{k+1}/k$不保此性：重排會改partial sums的抵消模式。以下無限sum的逐項微積分須有收斂區間，不能把有限線性性無條件擴成任何無限操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

等差和的配對與歸納

$S_n = 1 + 2 + \ldots + n$，倒序同和n+(n−1)+⋯+1；逐項相加有$2S_n = n(n+1)$，故$S_n = n(n+1)/2$，含n=0。

歸納也能核：S_0=0；若$S_n = n(n+1)/2$，則$S_{n+1} = S_n + (n+1) = (n+1)(n+2)/2$。這裡歸納假設的係數固定，不隨n悄悄變動。

一般a+(a+d)+⋯+(a+(n−1)d)=na+d n(n−1)/2。例迴圈j=3..7每次2j−1次工作，和為$(2 \cdot \sum_{j=1}^{7} j - 7) - (2 \cdot \sum_{j=1}^{2} j - 2) = 49 - 4 = 45$；直接5+7+9+11+13=45。改起點要扣掉正確prefix，不是把n=7誤當項數5。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

平方與立方和從消去式推導

由(k+1)³−k³=3k²+3k+1，k=0..n加總：

$(n+1)^3 = 3\sum_{k=0}^{n} k^2 + 3n(n+1)/2 + (n+1)$。整理得Σk²=n(n+1)(2n+1)/6。

由(k+1)⁴−k⁴=4k³+6k²+4k+1再加總，代入平方和與等差和，得到Σk³=n²(n+1)²/4=[n(n+1)/2]²。兩式n=0也成立。

例n=4：平方0+1+4+9+16=30；公式4・5・9/6=30。立方0+1+8+27+64=100；(4・5/2)²=100。公式是由整段telescoping恆等式推得，核小n只用來排除抄算失誤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

等比和：有限、無限、邊界分開

$G_n(x) = \sum_{k=0}^{n} x^k$。把$xG_n - G_n$展開，中間項消去，得$(x-1)G_n = x^{n+1} - 1$；x≠1時$G_n = (x^{n+1} - 1)/(x-1)$。x=1須獨立給$G_n = n+1$，不可除0。

|x|<1時$x^{n+1} \to 0$，$\sum_{k=0}^{\infty} x^k = 1/(1-x)$。x=1時partial sums無界；x=−1時partial sums在1與0間振盪；|x|>1時terms不趨0，所以無有限和。

例x=−1/2、n=3，G=1−1/2+1/4−1/8=5/8；有限公式同值。無限和則2/3，不能把四項和直接寫成無限和。截尾誤差$\sum_{k=n+1}^{\infty} x^k = x^{n+1}/(1-x)$，絕對值$\le |x|^{n+1}/(1-|x|)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

權重等比和與微積分的合法區間

有限多項式G_n可逐項微分。更直接，$W_n=\sum_{k=0}^{n}kx^k$，計W_n-xW_n，得

$(1-x)W_n=x+x^2+\ldots+x^n-nx^{n+1}$，所以$W_n=[x-(n+1)x^{n+1}+nx^{n+2}]/(1-x)^2$（x≠1）。x=1時$W_n=n(n+1)/2$。

|x|<1時$n|x|^n\to 0$（固定底數指數超過多項式），取極限得$\sum_{k=0}^{\infty}kx^k=x/(1-x)^2$。不必未證明就交換無限sum與微分。power series在收斂半徑內可逐項微分／積分；本例半徑1，端點須另查。

再次對收斂半徑內的式子作用x・d/dx，得$\sum_{k=0}^{\infty}k^2x^k=x(1+x)/(1-x)^3$。x=1/2時$\sum_{k=0}^{\infty}k/2^k=2$、$\sum_{k=0}^{\infty}k^2/2^k=6$；不是由「每項趨0」直接推出收斂。

Telescoping：只留下兩端

$\sum_{k=1}^{n}(a_k-a_{k-1})=a_n-a_0$，每個內部$a_k$恰加、減各一次。計算前先對齊索引及端點。

例如1/[k(k+2)]=(1/2)(1/k−1/(k+2))，故

$\sum_{k=1}^{n}1/[k(k+2)]=(1/2)(1+1/2-1/(n+1)-1/(n+2))$。

n=3時1/3+1/8+1/15=21/40；右式(1/2)(3/2−1/4−1/5)=21/40。取n→∞和為3/4，這一步還用了末端分母發散至∞。

乘積也可消去：$\prod_{k=2}^{n}(1-1/k^2)=\prod[(k-1)/k]\cdot\prod[(k+1)/k]=(1/n)\cdot((n+1)/2)=(n+1)/(2n)$，n≥2；空乘積n=1時公式也為1。

乘積轉sum與幾個不能省的前提

若每個$a_k > 0$，$\ln \prod a_k = \sum \ln a_k$；若有0，ln不定義；若有負值，也不能直接用實數ln。例$\prod_{k=1}^{n} 2^{3k+1} = 2^{\sum(3k+1)} = 2^{3n(n+1)/2+n}$。

對$0 \le u_k < 1$，$\ln(1-u_k)$介於$-u_k/(1-u_k)$與$-u_k$（由ln界），可把乘積變成和來控制，但u_k接近1時下界分母會變大，不能當固定常數。

若對所有k≥1有0<c₁f(k)≤h(k)≤c₂f(k)，加總立即得c₁Σf≤Σh≤c₂Σf。若只有尾段成立，須把有限prefix單獨保留；當Σf→∞且prefix有限時不改Θ階。常數必對整段k一致，不可每個k各挑一個再宣稱統一Θ。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.2 Bounding summations

先定要精確值、上界或雙邊階，再選方法。可用歸納、逐項界、固定比率界、切分區間、積分；一個粗上界不等於已找到緊界。

例$S_n = \sum_{k=0}^{n} 4^k$。欲證$S_n \le c4^n$，n0要求c≥1；若$S_n \le c4^n$，則$S_{n+1} \le c4^n + 4^{n+1} = (c/4+1)4^{n+1}$，要$\le c4^{n+1}$需c≥4/3。取固定c=4/3對全部n成立。末項$4^n$給下界，所以$S_n = \Theta(4^n)$。

錯誤歸納「$\sum_{k=1}^{n} k = O(n)$，加n+1仍O(n+1)」隱藏了會長大的常數。若假設Σk≤cn，下一步需cn+n+1≤c(n+1)，即n+1≤c，不可能對任意n成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最大項上界與尾段下界

對非負a_k，$\sum_{k=1}^{n} a_k \le n \cdot \max_{1\le k\le n} a_k$；對任何子集合J，$\sum a_k \ge \sum_{k\in J} a_k$。上界可用全體項，下界常只保留夠多的大項。

Σk≤n²。取k=ceil(n/2)..n，至少n/2筆，每項≥n/2，得Σk≥n²/4。對奇偶n同時成立，故Θ(n²)，不必先求精確公式。

固定實數r≥0，$\sum k^r \le n^{r+1}$；同一尾段給$\ge (n/2)(n/2)^r = 2^{-r-1}n^{r+1}$，故Θ$(n^{r+1})$。r固定很重要；若r=n，上下界常數$2^{-r-1}$不再與n無關，不能用此證明宣稱統一階。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

固定比例才有geometric尾界

假設$a_k \ge 0$且存在固定0<ρ<1，使$a_{k+1} \le \rho a_k$對所有k≥K成立。歸納得$a_{K+j} \le a_K \rho^j$，所以$\sum_{k=K}^{\infty} a_k \le a_K/(1-\rho)$。用乘法不等式寫可涵蓋$a_k = 0$，不需除0。

例$a_k = k^2/3^k$，k≥2時$a_{k+1}/a_k = (1/3)(1+1/k)^2 \le 3/4$。保留k0,1的prefix和1/3，從a₂=4/9起的尾和≤(4/9)/(1/4)=16/9，總和≤19/9；精確加權等比式給3/2，較小但同為有限界。

harmonic的比率k/(k+1)<1，卻趨1；不存在同一ρ<1控制全尾，不能套此法推收斂。若prefix本身依n，例如前兩項各n²，不能把「只有兩項」誤當O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

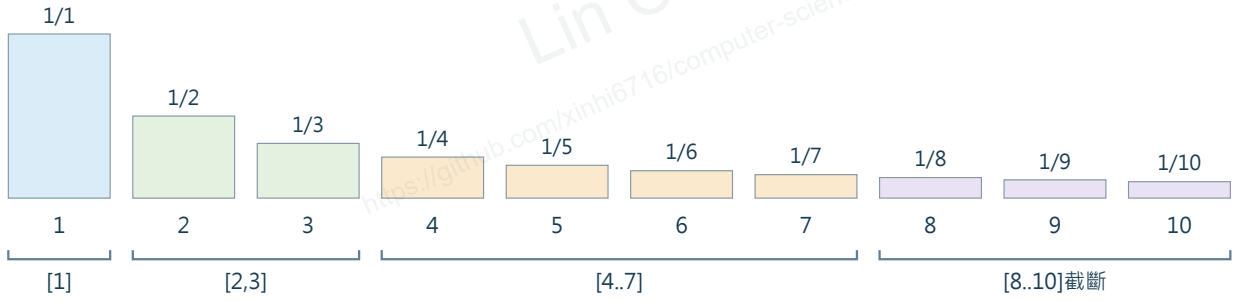
Dyadic分塊看出harmonic的log層數

$H_n = \sum_{k=1}^{n} 1/k$。第j塊$k = 2^j .. 2^{j+1} - 1$有2^j項，每項介於$1/2^{j+1}$與$1/2^j$，所以完整塊貢獻介於1/2與1。

上界補齊最後不完整塊：共有floor(lg n)+1塊，每塊≤1，$H_n \le \text{floor}(\lg n) + 1$。下界取前floor(lg n)個完整塊，$H_n \ge \text{floor}(\lg n)/2$，n≥2時為Ω(lg n)。兩界一起給Θ(lg n)，也證無限harmonic發散。

例n=10：區塊[1]、[2,3]、[4,5,6,7]、[8,9,10]，實際貢獻1、5/6、319/420、121/360；總H_{10}=7381/2520≈2.928968。最後一塊只有3項，不應誤列為8項。

n=10：每個完整dyadic塊的和介於1/2與1



積分界由每個單位區間證明

若f在[m−1,n+1]可積且單調不減，對每整數k有$\int_{k-1}^{k} f(x)dx \le f(k) \le \int_{k}^{k+1} f(x)dx$；加總得$\int_{m-1}^{n} f \le \sum_{k=m}^{n} f(k) \le \int_{m}^{n+1} f$。

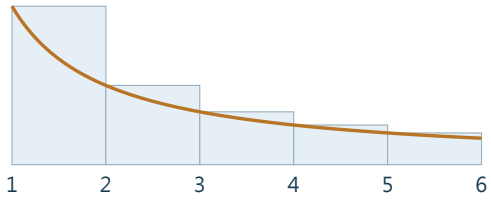
若f單調不增，各不等式反向：$\int_{m}^{n+1} f \le \sum f(k) \le \int_{m-1}^{n} f$。這裡不是把sum任意換成integral；左右端點的位移是控制誤差所需。

例如f=1/x在x>0遞減。下界$H_n \ge \int_{1}^{n+1} dx/x = \ln(n+1)$。上界不能從0積起，因1/x在0奇異；先取首項1，再對k2..n積分，得$H_n \le 1 + \int_{1}^{n} dx/x = 1 + \ln n$。

積分界由每個單位區間證明 （續）

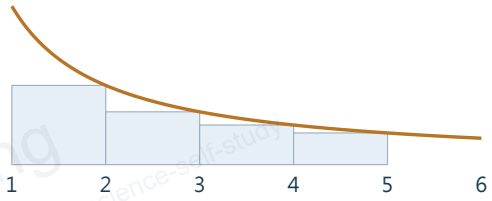
遞減f(x)=1/x：矩形高是1/k，兩邊的區間不同

下界：$\int_1^6 1/x\,dx \le H_5$



每個[k,k+1]：曲線≤1/k

上界：$H_5 - 1 \le \int_1^5 1/x\,dx$



每個[k−1,k]：曲線≥1/k

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

可重算的積分上下界

n=5時H_5=137/60≈2.283333，ln6≈1.791759≤H_5≤1+ln5≈2.609438。夾界有可量化寬度，不是宣稱sum等於ln n。一般$\ln(n+1) \le H_n \le \ln n + 1$，所以$H_n = \ln n + O(1)$。

對p>1，$\sum_{k=1}^{n} 1/k^p \le 1 + \int_1^n x^{-p}dx \le 1 + 1/(p-1)$，單調有界partial sums收斂。p=1發散如前；0≤p<1，積分下界隨n^{1-p}增長，和為$\Theta(n^{1-p})$，不是所有terms趨0都收斂。

對$f = x^r$、固定r≥0，積分給$n^{r+1}/(r+1) \le \sum_{k=1}^{n} k^r \le [(n+1)^{r+1} - 1]/(r+1)$，再次得$\Theta(n^{r+1})$，也說明模型中的常數可依固定r。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

雙重迴圈與改變加總次序

程式for i=1..n、for j=i..n，每次常數工作，次數$\sum_{i=1}^{n}(n-i+1)=n(n+1)/2$。也可依j計：每j有i=1..j，共$\sum_{j=1}^{n} j$。兩者數的是同一個索引集合{(i,j):1≤i≤j≤n}。

若每次工作j，則$\sum_{i=1}^{n}\sum_{j=i}^{n} j=\sum_{j=1}^{n}\sum_{i=1}^{j} j=\sum j^2=n(n+1)(2n+1)/6$。n=3各row和6、5、3，共14；按column1・1+2・2+3・3亦14。

這裡有限集合任意重排安全；無限雙重sum若含正負項，必先證絕對收斂，不能只因有限情況可換序就直接照搬。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix A。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B 集合、關係、函數、圖與樹

Part VIII Appendix

本章定位

以逐元素邏輯與一般證明串聯sets、equivalence partitions、functions、graphs與trees；明列圖型/路徑/方向/空集合及binary slots慣例，完整證tree六等價與二分圖判準。

前置知識

基本離散數學符號。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 正確使用集合、關係與函數的定義。
- 辨識 graph 與 tree 的結構性質。
- 連結笛卡兒積、等價關係與遞迴結構。
- B.1 Sets
- B.2 Relations
- B.3 Functions
- B.4 Graphs
- B.5 Trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.1 Sets

set是不重複、無順序的元素集合，{u,v,u}={v,u}；multiset另記multiplicity，sequence另記位置，三者不能互換。x∈A表示元素，A⊆B表示對每個x，x∈A⇒x∈B；A⊊B另要求不相等。∅⊆A對任何A成立，∅∈A則未必；{∅}有一個元素，並非空集合。

集合相等須A⊆B且B⊆A。自然數本章採N={0,1,⋯}，Z為整數，R為實數。寫{x∈Z:x可被3整除}是用性質限定既有集合，不是排序後的list。

集合運算的定義是逐元素邏輯：x∈A∪B iff (x∈A or x∈B)，交集用and，差A\B用(x∈A and x∉B)。complement必指定universe U，A^c=U\A；更換U會更換complement。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

集合恆等式是對任意元素的命題

性質	兩個對偶版本
單位／吸收	A∪∅=A；A∩∅=∅
冪等	A∪A=A；A∩A=A
交換／結合	A∪B=B∪A；A∪(B∪C)=(A∪B)∪C，交集同理
分配	A∩(B∪C)=(A∩B)∪(A∩C)；∪/∩互換亦成立
吸收	A∩(A∪B)=A；A∪(A∩B)=A
De Morgan	$(B\cap C)^c=B^c\cup C^c$；$(B\cup C)^c=B^c\cap C^c$

例如$x \in A \setminus (B \cap C)$ iff x∈A且非(x∈B且x∈C)，iff (x∈A且x∉B)或(x∈A且x∉C)，iff $x \in (A \setminus B) \cup (A \setminus C)$。這是任意x的雙向等價；只畫一個有限Venn例子不代替全稱證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partition、cardinality與重疊的計數

partition是非空blocks的集合：兩兩不交且聯集為S，使每個x∈S恰屬一塊。S=∅時允許空blocks集合，條件皆成立。例S={0,⋯,7}按mod3分為{0,3,6}、{1,4,7}、{2,5}。

對有限A,B，將A∪B拆成互斥的A\B、A∩B、B\A，可得|A∪B|=|A|+|B|−|A∩B|。例A={0,2,4,6}、B={2,3,6}，4+3−2=5；不能只把cardinalities相加而重複計交集。

三集合公式加三個單集合、減三個pair intersections、加triple intersection。一般inclusion-exclusion可逐元素證：一個落在r≥1個集合的元素係數為$\sum_{j=1}^{r}(-1)^{j+1}\binom{r}{j}=1$，因$(1-1)^r=0$。未落任何集合者係數0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Power set與Cartesian product的不同分母

power set P(S)={T:T⊆S}。若S有n個可區分元素，一個subset對每個元素只選入／不入，與n-bit字串一一對應，故$|P(S)|=2^n$；P(∅)={∅}仍有1筆。

ordered pair(a,b)保留位置，一般不同於(b,a)。A×B={(a,b):a∈A,b∈B}；固定a有|B|個pair，|A|個互斥rows給|A×B|=|A||B|。n-fold product計ordered n-tuples，有限集合時大小為各大小乘積；A^0含唯一空tuple，大小1。

例如A={p,q}、B={0,1,2}，A×B有6筆；P(A)只有∅,{p},{q},{p,q}四筆。sequence是由位置到值的function；相同值在不同位置可重複，不會像set自動合併。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

無限集合的大小要靠bijection

兩集合等cardinality指存在bijection，不是「兩個都無限就一樣多」。N到Z可設f(0)=0，f(2k−1)=k、f(2k)=−k（k≥1）；每個正負整數各有唯一原像，所以Z可數。N到偶數的2n也是bijection，無限集合可與自己的真子集等大。

R不可數可用diagonal argument：若(0,1)能列為$x_1,x_2,\cdots$，每個取不以無窮9結尾的十進位表示。建y的第i位：若x_i第i位不是1取1，否則取2。y∈(0,1)，只用1/2而無表示歧義，卻與每個x_i至少在第i位不同，矛盾。

有限集合的「注入且同大小即滿射」不能原封套到無限集合；f(n)=n+1在N上注入卻漏0。定理要清楚寫finite前提。

B.2 Relations

binary relation R⊆A×B；在單一A上則R⊆A×A。aRb表示(a,b)∈R，沒有先要求每個a都有對象，也沒有要求唯一對象。

性質	量詞：適用所有a,b,c∈A
reflexive	aRa
symmetric	aRb⇒bRa
antisymmetric	aRb且bRa⇒a=b
transitive	aRb且bRc⇒aRc

symmetric與antisymmetric不是互為否定：相等關係同時滿足兩者；含a↔b與b→c卻沒有c→b的relation可兩者都不滿足。空relation在非空A上對稱、反對稱、遞移皆真，卻不reflexive；不能憑「對稱且遞移」擅自假設aRb存在。

Equivalence relation如何產生partition

equivalence relation同時reflexive、symmetric、transitive。定[a]={b∈A:aRb}。由aRa，[a]非空且各a被覆蓋。若c∈[a]∩[b]，則aRc、bRc，對稱及遞移給aRb；任意x∈[a]有bRaRx，故x∈[b]；反向同理，所以兩類只可能相同或不交。去掉重複類後便是partition。

反向給partition，定aRb iff a,b在同一block。每a恰在一塊，故reflexive；同塊是symmetric；a,b同塊且b,c同塊，由block唯一性三者同塊，故transitive。a所在block正是[a]，兩個建構互相對應。

例如mod3等價：a−b為3的倍數。0=3・0證反身，負號證對稱，相加證遞移；classes是餘數0,1,2三類，限制到有限S時得到前述三blocks。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partial order、total order與極大／最大

partial order是reflexive、antisymmetric、transitive；total order再要求每對a,b可比較。total preorder只要求reflexive、transitive與任兩者可比較，允許不同元素互相等級相同，不必antisymmetric。

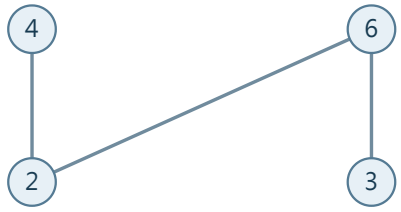
在poset內，maximum M滿足每個x≤M；maximal m只要求沒有嚴格更大的元素。minimum／minimal對偶。maximum若存在必唯一且maximal：若M,N都是maximum，M≤N且N≤M，由反對稱得相等；反向maximal不保證maximum。

S={2,3,4,6}以整除為≤。minimal={2,3}、maximal={4,6}，沒有minimum或maximum。Hasse圖只畫cover：x<y且沒有z使x<z<y；邊向上表示較大，省掉反身與傳遞邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Partial order、total order與極大／最大 （續）

S={2,3,4,6}：整除偏序的Hasse圖



向上：被整除的較大元素

covers：2–4、2–6、3–6

minimal：2、3

maximal：4、6

沒有minimum / maximum

線段只表示cover，不代表數值距離；反身與傳遞關係不另畫邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.3 Functions

function f:A→B是在relation中要求每a∈A**恰有一個**b∈B的特例。A是domain，B是codomain；range f(A)可能只是B的子集。partial function若有a未定義，就不符合這裡的total function定義。

injection要求f(a)=f(a′)⇒a=a′；surjection要求對每b∈B存在a使f(a)=b；bijection同時滿足。例 f:N→N、f(n)=2n，注入非滿射；把codomain改成非負偶數，便是bijection。f(n)=floor(n/2)在N→N滿射非注入（0,1同映0）。

有限A,B，注入的不同輸入占不同codomain元素，故|A|≤|B|；滿射須每b至少一個原像，故|A|≥|B|。若有限且同大小，一個條件便推出另一個；無限時前頁已有反例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Inverse、image與composition的條件

對bijection f:A→B，f^{-1}(b)是唯一a使f(a)=b，並有$f^{-1} \circ f = id_A$、$f \circ f^{-1} = id_B$。若只injection，inverse只能定義在range；若非injection，relational inverse一般不是function。

set preimage f^{-1}(T)={a:f(a)∈T}對任意function及T⊆B都可定義，不要求可逆。它與「inverse function」同符號但不是同一個物件。

f:A→B、g:B→C的composition為(g∘f)(a)=g(f(a))。若f,g均注入，g(f(a))=g(f(a′))先取消g再取消f，得a=a′；若均滿射，給c先找b再找a，得composition滿射。兩者均bijection時，$(g \circ f)^{-1} = f^{-1} \circ g^{-1}$，順序反轉。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B.4 Graphs

本章有限directed graph G=(V,E)，E⊆V×V，可有self-loop但不記parallel edges；undirected simple graph的E是不同頂點的unordered pairs，沒有self-loop。若用multigraph須另保每條edge identity及loop的度數計法。

directed edge(u,v)從u出、到v入；outdegree數出邊，indegree數入邊，self-loop各計一次。undirected degree數incident edges。每無向邊恰貢獻兩個端點，故Σdeg=2|E|；有向Σoutdeg=Σindeg=|E|。因此無向奇度頂點個數必偶數。

同一張圖換頂點位置或畫成曲線不改E。兩條線交叉處除非明畫頂點，並非可轉向的節點；edge方向、頂點identity與數值須由圖例或edge list確定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Walk、simple path與cycle的約定

walk是一串$v_0, \ldots, v_k$，每相鄰一對有edge，長度k數邊不數點；CLRS的path允許重複，對應這裡walk。simple path要求所有頂點不同；u到自己的長0路徑存在。

本章談acyclic時，undirected cycle指長≥3、除起終點外頂點不重複的封閉路徑；只沿同一邊u→v→u往返不是cycle。directed simple cycle可長1（self-loop）或2（雙向邊），方向須逐邊成立。一般closed walk可有重複，不能與simple cycle混為同一物件。

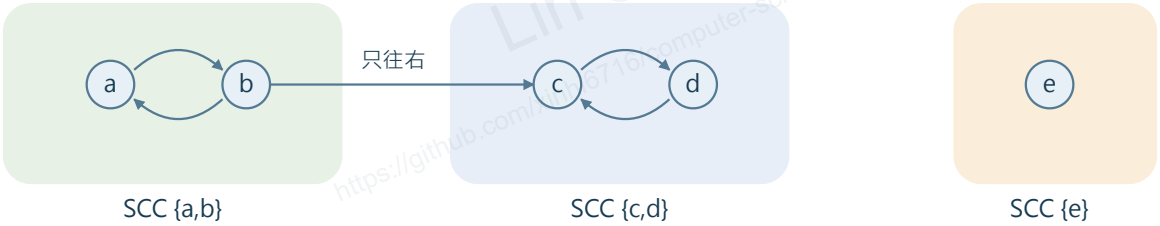
若u到v有walk，遇重複頂點$v_i = v_j$，刪去兩次出現之間的閉段仍連成walk且更短；反覆刪除，有限長度降到simple path。這個一般消去論證使「可達」不必靠無限列出walk判定。

連通分量、SCC與方向

無向可達關係reflexive（長0）、symmetric（倒走）、transitive（串接），所以components是equivalence classes。有向可達仍反身、遞移，但可能不對稱；mutual reachability則三性皆具，classes稱SCC。

原創digraph：a→b、b→a、b→c、c→d、d→c，另e孤立。SCC為{a,b}、{c,d}、{e}；存在a→d卻沒有d→a，故不能把兩個SCC合併。忽略方向的版本只有{a,b,c,d}與{e}兩個components。

可達不等於互相可達；每條箭頭都有指定方向



a可達d，d不可達a。忽略方向後a/b/c/d同一component，e仍孤立。

Subgraph、isomorphism與contraction

subgraph可取部分頂點與部分原edges，edges兩端仍須在所取頂點。induced subgraph G[W]則必保留所有兩端都在W的原edges，不能挑掉不喜歡的一條。例前圖W={a,b,c}的induced edges恰a→b,b→a,b→c。

isomorphism是bijection f:V→V′且(u,v)∈E iff (f(u),f(v))∈E′；只保一方向不足。頂點數、邊數、度數multiset相同是必要條件，不是充分：六頂點cycle C_6與兩個互斥三角形皆6邊、各degree2，但一個連通、一個不連通。

contract無向edge{u,v}時將端點合為新x，其他incident edges改接x，刪被縮edge；若保持simple graph表示，同一鄰居產生的重邊合一、自loop刪除。若演算法需保parallel edges（如某些隨機收縮），必明用multigraph，不能默默去重。

常見圖類與二分圖的一般判準

complete graph K_n每對不同頂點相鄰，|E|=n(n−1)/2。bipartite有V=L⊔R且每條edge跨兩側，等價可用兩色proper coloring。DAG是directed acyclic graph；tree／forest是無向圖概念，不能只看「箭頭沒有環」就叫tree。

二分圖不可能有奇cycle：沿cycle每跨一邊換側，奇次不能回到同一側。反向若無奇cycle，在每個component取根，依最短無向距離的奇偶分兩側。若edge(u,v)同奇偶，根到u、該edge、v回根構成奇長closed walk；任意奇closed walk若有重複頂點可拆為兩個closed walks，至少一個奇，反覆縮短會得到奇simple cycle，矛盾。

hypergraph的一條hyperedge可含多個頂點；可建incidence bipartite graph，一側原頂點、一側hyperedges，membership成為跨側edge。這個圖的節點種類與原圖不同，不能把一條hyperedge直接當普通pair。

B.5 Trees

本節free tree要求n=|V|≥1、simple undirected、connected且沒有simple cycle。forest只要求acyclic，可有多個components與isolates；空forest允許，空圖不作本節tree六等價條件的輸入。

一個n≥2的tree必有leaf：取最長simple path的一個端點v。若v另有不在路徑的鄰居，可延長；若另接路徑內非相鄰頂點，便成cycle，皆矛盾。故v只有路徑上的一個鄰居，degree1。

移除leaf及其唯一邊後仍connected（其他點間simple path不用leaf作內部點）且acyclic。歸納n1有0邊，tree n筆有n−1邊。forest若有c個components，每個component是tree，逐塊相加得$|E| = \sum(n_i - 1) = n - c$（包含空forest n=c=0）。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

六個條件為什麼等價：先看路徑與邊

對非空有限simple無向圖，下列六項等價：①tree；②每對頂點恰有一條simple path；③connected且刪任何edge都斷；④connected且m=n−1；⑤acyclic且m=n−1；⑥acyclic且加任何缺少的edge都成cycle。

①⇒②：連通給至少一條；若兩條不同simple paths，取第一次分歧與之後第一次會合的兩段，內部不交，合起來成cycle，矛盾。②⇒③：edge{u,v}本身就是唯一u-v路徑，刪後無路。

③⇒①：若有cycle，刪cycle一邊仍可沿其餘部分替代任何使用該邊的路徑，所以不斷，矛盾。①⇒④與⑤由前頁n−1邊證明。這些推論不把「有一條路」誤當「唯一一條路」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

六個條件的其餘方向與可重建構造

任何connected graph可建spanning tree：從一頂點S開始，只要S≠V，由連通性必有跨S與V\S的edge；加該edge與一個新頂點。每步只接一個新點，不成cycle；n−1步後得到n−1邊的tree，故一般connected graph m≥n−1。

④⇒①：上述spanning tree已用盡G的n−1邊，G無額外邊，故就是tree。⑤⇒①：由forest的m=n−c及m=n−1得c=1。①⇒⑥：任一新edge接u,v，與tree中唯一u-v路徑形成cycle。⑥⇒①：若有兩個不同components，加入跨component edge不可能成cycle，與條件矛盾；故connected。

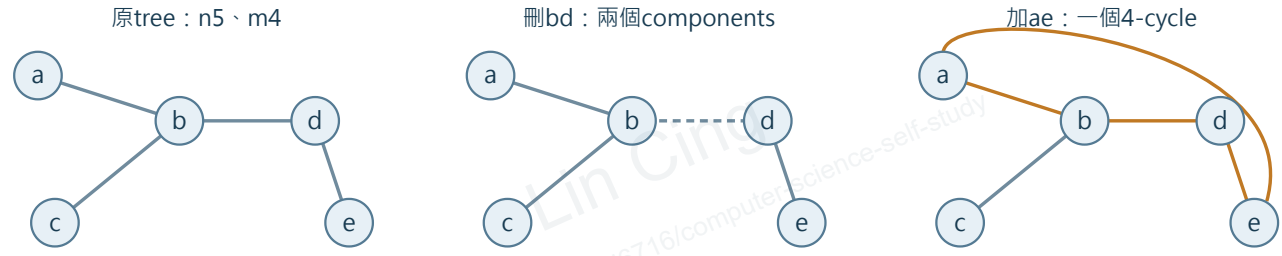
六個方向已封閉。注意「m=n−1」單獨不夠：三角形加一個isolated vertex有n4,m3卻不是tree。相反「acyclic」也不保connected。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一張圖的tree、非tree與新增cycle

原創tree V={a,b,c,d,e}，E={ab,bc,bd,de}，n5,m4。唯一a到e的simple path為a,b,d,e，長3。刪bd切成{a,b,c}與{d,e}；加入ae則形成a,b,d,e,a的4-cycle。

同一組頂點：刪tree邊必斷，加缺邊必成cycle



中圖虛線是被刪掉的邊；右圖橙色cycle沿a–b–d–e再走新邊e–a返回。

這裡有cycle時仍可能有spanning tree，只是圖本身不再是tree；minimum spanning tree的加權最佳化還需額外目標，並不是tree定義的一部分。

Rooted與ordered：高度、深度與degree要說慣例

從free tree指定root r後，每個非根x的parent是唯一r-x simple path中前一點；其餘向下鄰居是children。ancestor／descendant本章包含自己，proper才排除自己；subtree rooted at x取x全部descendants。

depth(x)=根到x的邊數，height(x)=x到最深leaf的邊數；leaf height0，root depth0，tree height=max depth。rooted degree數children；free-tree graph degree還包含parent，故非根兩者差1。

若前頁tree以b為root，children a,c,d，d另有child e；depth(a,c,d)=1、depth(e)=2，height(b)=2、height(d)=1，其他0。b graph degree與children均3，d graph degree2但children1。

ordered tree另外定每節點children先後；把b的(a,c,d)改成(c,a,d)，rooted關係同但ordered tree不同。

Binary／positional tree與空child

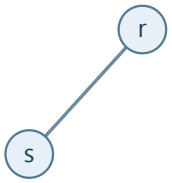
binary tree遞迴定義為空樹NIL，或一個root加互斥的left/right binary subtrees；只一個child時left與right仍不同。ordered tree只有一個child時沒有左slot／右slot之別，所以「degree≤2的ordered tree」不足以完全描述binary tree。

positional k-ary tree有固定slots1..k，空slot也帶位置資訊。將binary tree每個缺少child替成external NIL leaf，可得到每個internal node恰有兩個children的full binary tree，保留原left/right位置。

Binary／positional tree與空child （續）

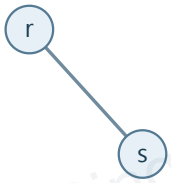
空slot也有位置：單一child不能省掉left/right資訊

只有left child



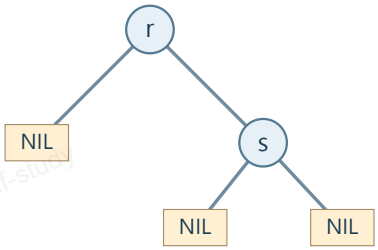
binary位置不同

只有right child



binary位置不同

右圖擴充：I2、L3



若原binary tree非空且有n≥1個實體nodes，總child slots2n，實體parent-child edges n−1，缺少slots=2n−(n−1)=n+1；擴充表示就有n個internal與n+1個external leaves。NIL sentinel若共用單一物件，邏輯空位置數仍是n+1，不等於實際配置n+1物件。空樹沒有實體父節點，故上述(parent,side) child slots為0；若另外表示空根的NIL位置，那是另一種計數。

Full、complete與height的精確計數

full k-ary tree每個internal恰k個children，leaves可不同深度。以I internal、L leaves計edge：kI=I+L−1，故L=(k−1)I+1。k2時L=I+1。

CLRS此附錄的complete k-ary另要求全部leaves同深h（常有文獻稱perfect）；對k≥2，第d層k^d個nodes，leaves數 $L = k^h$，internal數 $I = \sum_{d=0}^{h-1} k^d = (k^h - 1)/(k - 1)$。例k3,h2：levels1,3,9，I4,L9、總13，edges12=3I。k1是鏈，須另算I=h、L1，不可除k−1=0。

heap章節常稱complete binary為「末層可不滿、由左填滿」，與這裡all-leaves-same-depth慣例不同，使用時須看定義。一般非空binary tree高h至多$\sum_{d=0}^{h} 2^d = 2^{h+1} - 1$個nodes，所以h≥ceil(lg(n+1))−1；最斜鏈h=n−1，沒有平衡前提不能聲稱所有binary tree高Θ(lg n)。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix B。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C 計數與機率

Part VIII Appendix

本章定位

從sample object與統一分母建立計數、條件機率、Bayes及指標分析，再推完整geometric/binomial矩與尾界；明定獨立層級、零機率/支撐邊界及中心化r和總門檻t的區別。

前置知識

排列組合、機率公理與隨機變數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 用乘法、加法與補集計數。
- 計算期望與變異數，處理幾何與二項分布。
- 以 Chernoff bound 約束二項尾機率。
- C.1 Counting
- C.2 Probability
- C.3 Discrete random variables
- C.4 The geometric and binomial distributions
- C.5 The tails of the binomial distribution

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C.1 Counting

先定「一個結果」包含哪些資訊，再計數。sum rule要求互斥分類；product rule要求每個prefix後的選擇數已知。選擇集合可依先前結果改變，只要每層分支數固定，仍可相乘；若分支數不固定就逐branch加總，不能硬乘平均。

原創例：從6名學生選兩個不同職位，第一職6選、第二職5選，共30；若只是兩人committee、不分職位，每組被計兩個先後，除2得15。若允許同一人兼任，則變6^2=36。三個答案對應三種不同sample object。

長m、alphabet大小q的strings有q^m，允許重複且保位置。substring須連續，subsequence可跳位置；同一內容在不同起點是否算不同，必先約定。長n字串的長k≥1位置型substrings有max(0,n−k+1)個，不等於不同內容種類數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Permutations、combinations與不重複計數

相異n筆全部排列有n!；ordered選k筆無重複有P(n,k)=n!/(n−k)!，0≤k≤n。unordered k-subset每組恰對應k!個ordered結果，故$\binom{n}{k}$=n!/[k!(n−k)!]。k0唯一空選擇，$\binom{n}{0}$=1；k<0或k>n可擴充定為0。

若records可區分但keys重複，排列records仍n!；若只看key字串、各multiplicity為$m_1, \ldots, m_s$，則每key字串對應$\prod m_i!$種identity排列，故有$n!/\prod m_i!$個不同key字串。除法前須證每個結果被重複相同次數。

$\binom{n}{k}$=$\binom{n}{n-k}$由取補集bijection得。Pascal identity $\binom{n}{k}$=$\binom{n-1}{k}$+$\binom{n-1}{k-1}$，依某個指定元素不在／在subset分為兩個互斥且完備的類別。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二項式展開與entropy界的來源

展開$(x+y)^n$時，每個factor選x或y；恰k個位置選x共有$\binom{n}{k}$種，故$(x+y)^n=\sum_{k=0}^{n}\binom{n}{k}x^k y^{n-k}$。x=y=1得$\sum\binom{n}{k}=2^n$，這也是按subset大小分類的計數。

對1≤k≤n，$\binom{n}{k}=\prod_{j=0}^{k-1}(n-j)/(k-j)\geq(n/k)^k$，因n≥k使每個factor≥n/k。上界$\binom{n}{k}\leq n^k/k!\leq(en/k)^k$，最後用$k!\geq(k/e)^k$。

另令α=k/n∈(0,1)，展開$(\alpha+1-\alpha)^n=1$，其中一項非負且≤1，故$\binom{n}{k}\leq\alpha^{-k}(1-\alpha)^{-(n-k)}=2^{nH(\alpha)}$，H(α)=−α lgα−(1−α)lg(1−α)。端點以0lg0=0連續延伸，$\binom{n}{0}=\binom{n}{n}=1$。H′=lg((1−α)/α)，最大在1/2，值1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C.2 Probability

sample space S的elementary outcome是一個完整可區分結果，event是合法可測子集。有限／可數離散情況本章取所有subsets為events；不可數情況須指定對補集與可數聯集封閉的event族，不能無條件把每個subset都當event。

機率滿足P(A)≥0、P(S)=1，以及對兩兩互斥的有限／可數events，有$P(\cup A_i)=\sum P(A_i)$。因此P(∅)=0、P(A^c)=1−P(A)、A⊆B⇒P(A)≤P(B)。離散時$P(A)=\sum_{s\in A}P(s)$。

有限S只有在每個outcome等機率時才可P(A)=|A|/|S|。例如丟兩個公平且獨立的0/1 bits，四個ordered結果各1/4；把它們壓成「0、1、2個1」後三種計數不是等機率，而是1/4、1/2、1/4。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

聯集、union bound與不需要的獨立性

由互斥分解A∪B=(A\B)⊔(A∩B)⊔(B\A)，P(A∪B)=P(A)+P(B)−P(A∩B)≤P(A)+P(B)。多事件定B₁=A₁、$B_i = A_i \setminus (\bigcup_{j<i} A_j)$，$B_i$互斥、聯集相同且$B_i \subseteq A_i$，故$P(\cup A_i) = \sum P(B_i) \leq \sum P(A_i)$，可數版本也成立。

此union bound不需獨立；右側可大於1，實用時再取min(1,右側)。相反若要把$P(\cap A_i)$寫成$\prod P(A_i)$，必須有mutual independence，不能因為union bound不需獨立就混用。

例三個failure events各≤0.01，不論相關性，至少一個失敗機率≤0.03。若三個事件其實完全相同，聯集機率0.01而不是1−0.99³；後式需要獨立前提。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Conditional probability與chain rule

P(A|B)=P(A∩B)/P(B)，要求P(B)>0；這是在保留B內outcomes後重新normalize。P(A∩B)=P(B)P(A|B)。反覆應用得$P(\cap_{i=1}^{n} A_i) = P(A_1) \prod_{i=2}^{n} P(A_i | A_1 \cap \ldots \cap A_{i-1})$，各conditional使用時須分母正；若某prefix機率0，整交集已為0，不能再寫未定義的比值乘積。

原創抽樣：袋中3藍2紅，無放回連抽兩球，P(兩藍)=(3/5)(2/4)=3/10；第二次藍的邊際仍3/5，但在第一次藍的條件下只有1/2，故兩次不獨立。若放回並重新均勻抽取，才是(3/5)²。

知道「至少一事件發生」與知道「指定那一次發生」通常不同。描述觀測或篩選規則是機率模型的一部分，不可只看到剩兩種文字敘述就自動各給1/2。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Independence、互斥與pairwise的差異

A,B獨立指P(A∩B)=P(A)P(B)，這個定義也涵蓋機率0 events；若P(B)>0才可等價寫P(A|B)=P(A)。互斥指交集空，若兩者機率皆正則不獨立。空event則與任意event獨立。

mutual independence要求每個有限子集合的交集機率都等於邊際乘積；pairwise只核每一對，不足以乘三個以上。

取U,V為公平獨立bits，W=U XOR V，outcomes(0,0,0),(0,1,1),(1,0,1),(1,1,0)各1/4。U,V,W每對的四種組合都各1/4，故pairwise independent；但P(U=V=W=1)=0，而三邊際乘積=1/8，非mutual。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Independence、互斥與pairwise的差異 (續)

同一張完整表：pairwise independent，非mutually independent

U	V	W=U XOR V	P(outcome)
0	0	0	1/4
0	1	1	1/4
1	0	1	1/4
1	1	0	1/4

任一對：00/01/10/11各1/4

三個邊際P(=1)都為1/2

但111根本不在sample space

$0 \neq (1/2)^3 = 1/8$

每對核對成立，不代表三者交集可直接乘邊際；這裡variance可加，三重乘積不可分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Total probability與Bayes的分母

若B_i形成S的可數partition且$P(B_i)>0$，由$A=\sqcup(A\cap B_i)$，$P(A)=\sum P(B_i)P(A|B_i)$。Bayes再給$P(B_j|A)=P(B_j)P(A|B_j)/P(A)$，須P(A)>0。

原創監測器模型：故障prior P(F)=0.02；有故障時發alarm機率0.9，無故障時誤報0.1。則P(alarm)=0.02・0.9+0.98・0.1=0.116；posterior P(F|alarm)=0.018/0.116=9/58≈0.155172。

alarm的高sensitivity0.9不等於posterior0.9；大量正常樣本提供0.098誤報分母。兩次測試若只在給定F／F^c後獨立，雙alarm機率=0.02・0.9^2+0.98・0.1^2=0.026，posterior=0.0162/0.026=81/130≈0.623077；沒有先給條件獨立，不能平方likelihood。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Continuous uniform不給每點正機率

X在[a,b]連續uniform、a<b時，P(c≤X≤d)=(d−c)/(b−a)，a≤c≤d≤b。單點c=d機率0，但整區間機率1；可數可加性不要求對不可數個單點直接相加。

例[a,b]=[2,8]，P(3≤X≤5)=2/6=1/3，去掉兩端點結果相同。若Z=floor X，Z的值2..7各機率1/6，而Z=8只在單點X8出現，機率0；不得把七種整數標籤平均分配。

接下來主講離散random variables及pmf。離散機率質量P(X=x)與連續density不同：density可大於1，單點機率仍0。對連續分布應積分區間，不能把density當成每點機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C.3 Discrete random variables

random variable X:S→R是outcome的數值函數，event{X=x}是其preimage。pmf $p_X(x) = P(X = x)$，非負且$\sum_x p_X(x) = 1$；本章用「probability mass function」，避免把離散質量與連續density混淆。

joint pmf p(x,y)=P(X=x,Y=y)，marginals為$p_X(x) = \sum_y p(x, y)$、$p_Y(y) = \sum_x p(x, y)$。給$p_Y(y) > 0$，conditional pmf為$p(x, y)/p_Y(y)$。X,Y獨立當且僅當所有x,y都有$p(x, y) = p_X(x)p_Y(y)$。

原創：兩個獨立公平四面骰A,B∈{1,2,3,4}，令M=max(A,B)。P(M≤m)=$m^2/16$，P(M=m)=[m^2−$(m-1)^2$]/16=(2m−1)/16，m1..4質量1,3,5,7除16；不同M值不是等機率。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Expectation是加權和，不必是可能結果

$E[X] = \sum_x x p_X(x)$。有限支撐必可算；可數支撐要E|X|<∞才有有限且可重排的有號期望。非負X允許E[X]=∞；不能把∞−∞當合法期望運算。

對四面骰最大值M，E[M]=(1・1+2・3+3・5+4・7)/16=50/16=25/8；E[M^2]=(1+12+45+112)/16=170/16=85/8。期望3.125不在整數支撐，仍是正確平均。

任意g，只要E|g(X)|<∞，按每個X值的preimage分類得到$E[g(X)] = \sum_x g(x)p_X(x)$，不用先重新列g(X)的所有不同值。非線性時一般E[g(X)]≠g(E[X])；平方的差就是variance。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

線性期望的完整推導與indicator

若X,Y可積，從joint pmf有$E[X+Y]=\sum_x\sum_y(x+y)p(x,y)=\sum_x x\sum_y p(x,y)+\sum_y y\sum_x p(x,y)=E[X]+E[Y]$；絕對可積允許拆與換序，沒有使用獨立。E[aX+b]=aE[X]+b。有限多變數逐次套用。

indicator I_A在A發生為1，否則0，$E[I_A]=P(A)$。若X計某些events發生次數，$X=\sum I_i$，故$E[X]=\sum P(A_i)$，即使相依也成立。例uniform排列n個相異keys，inversion指標每對期望1/2，總n(n−1)/4。

若X,Y獨立且可積，joint分解後$E[XY]=(\sum xp_X)(\sum yp_Y)=E[X]E[Y]$。這次用到獨立；若X=Y是公平bit，E[XY]=E[X]=1/2≠1/4。多項乘積通常需mutual independence，而非只pairwise。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Tail-sum與Jensen的理由

非負整數X逐outcome有$X=\sum_{i\geq 1}IX\geq i$。有限截尾先求期望再單調取極限（每項非負），得$E[X]=\sum_{i\geq 1}P(X\geq i)$，可含∞。若X可以負值，不能直接用這個式子。

convex f滿足f(λx+(1−λ)y)≤λf(x)+(1−λ)f(y)，0≤λ≤1。有限支撐以此二點定義歸納：把前m−1個權重normalize成一點，與最後一點再套凸性，得到f(E[X])≤E[f(X)]。

一般可積情況，若f在包含支撐及μ=E[X]的區間凸，μ有supporting line f(x)≥f(μ)+a(x−μ)，取期望即E[f(X)]≥f(μ)；端點退化情況X必集中該端點。需兩側期望有定義；這不是任意函數都可把f移入／移出E的規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Variance、covariance與相依成本

若$E[X^2]<\infty$，$\mu=E[X]$，$Var(X)=E[(X-\mu)^2]=E[X^2]-\mu^2\geq 0$；$\sigma=\sqrt{Var}$為standard deviation。$Var(aX+b)=a^2Var(X)$，不是乘a。前例M的$Var=85/8-(25/8)^2=55/64$，$\sigma=\sqrt{55/8}$。

$Cov(X,Y)=E[XY]-E[X]E[Y]$，展開平方得$Var(X+Y)=VarX+VarY+2Cov$。n項時$Var(\sum X_i)=\sum VarX_i+2\sum_{i<j}Cov(X_i,X_j)$。pairwise independent且二次矩有限足以使covariances全0；不需mutual independent。

但只同分布不足。若$X_1=\ldots=X_n=$同一公平bit，sum只有0或n，各1/2，$variance=n^2/4$；逐項variance相加只有n/4。缺少covariance會把不確定性嚴重低估。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Markov與Chebyshev：各自的適用條件

若$X\geq 0$且$E[X]<\infty$，對$t>0$逐點有$X\geq t\,I\{X\geq t\}$，取期望得$P(X\geq t)\leq E[X]/t$（Markov）。不需獨立，但不能套負值變數；有正負抵消時平均小不表示正尾小。

對二次矩有限Y，取非負$X=(Y-\mu)^2$與$t=a^2$，$a>0$，得到$P(|Y-\mu|\geq a)\leq Var(Y)/a^2$（Chebyshev）。它控制雙尾；若只要一側也可用同上界，但可能寬鬆。

例如四面骰最大值M的μ25/8、variance55/64，a=2，雙尾$\leq 55/256\approx 0.214844$；真事件只有M1，機率1/16。上界不是宣稱真機率等於bound；超過1的界可取1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C.4 The geometric and binomial distributions

Bernoulli(p)取1（success）機率p、0機率q=1−p。$X^2=X$，故E[X]=p，Var=p−p^2=pq。說一串Bernoulli trials時，以下另要求mutual independence與相同p；只有各次邊際p不夠。

geometric本章與CLRS採**包含首次成功那次的trial數**G∈{1,2,⋯}，0<p≤1。G=k要求先k−1次失敗、再成功，$P(G=k)=q^{k-1}p$；p0沒有成功，G=∞，不是這個有限值geometric分布。

若別處採「成功前失敗次數」F=G−1，支撐從0開始、pmf為$q^f p$、E[F]=q/p；不能把兩種mean都寫1/p。p1時G≡1、F≡0，公式與variance0仍成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometric的和、期望與variance

$\sum_{k\ge1} q^{k-1}p = p/(1-q) = 1$。$P(G>t)=q^t$（整數t≥0），所以$P(G\ge i)=q^{i-1}$；tail-sum得E[G]=1/(1−q)=1/p。

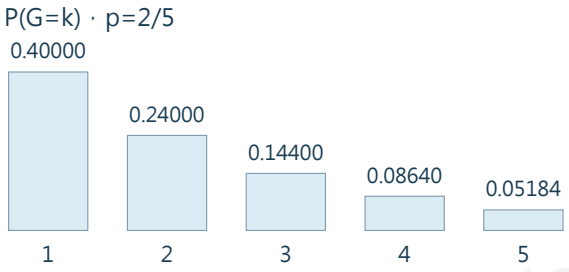
由加權等比公式可得$E[G^2]=(1+q)/p^2$：例如$\sum_{k\ge1} k^2q^{k-1} = (1+q)/(1-q)^3$，乘p即得。Var=$(1+q)/p^2-1/p^2=q/p^2$。q0情況直接由G1處理，不做除q。

原創p=2/5，k1..5的pmf依次2/5、6/25、18/125、54/625、162/3125；總2882/3125，剩尾$P(G>5)=(3/5)^5=243/3125$。E[G]=5/2、Var=15/4。期望不是「第2.5次真的成功」，而是多次重做此完整實驗的平均。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Geometric的和、期望與variance （續）

先固定計數慣例，再計pmf、尾端與期望



一個實現：F → F → F → S

G=4：包含最後成功trial

F=G−1=3：只數失敗

五次仍未成功：q^5=243/3125

此尾端沒有被前五根柱吃掉

橫軸是包含首次成功的trial index；E[G]=2.5，E[F]=1.5，兩種計數不可交換。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Memoryless與重試的模型邊界

對0<p<1、整數s,t≥0，$P(G > s+t | G > s) = q^{s+t}/q^s = q^t = P(G > t)$。已失敗s次不改未來的success率，因各trial獨立且同p；不是「已經失敗很久，下一次更容易成功」。

若retry成功率隨次數改、系統有同一持續故障、或抽樣無放回，都不符合相同獨立trials。此時尾端可能是$\prod_{i=1}^{t}(1-p_i)$（獨立但不同p），或需要conditional probabilities，不能硬寫q^t。

若每trial固定成本c，總成本cG期望c/p；若成本依成功／失敗不同，總成本是$c_{\text{fail}}(G-1)+c_{\text{success}}$，期望$c_{\text{fail}}q/p + c_{\text{success}}$。先定成本與trial的關係，再套線性期望。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Binomial的pmf、mean與variance

互相獨立同p的n次trial，$X = \sum_{i=1}^{n} I_i \in 0, \ldots, n$。固定一組k個成功位置，其機率$p^k q^{n-k}$；$\binom{n}{k}$個互斥位置集合給$b(k; n, p) = \binom{n}{k} p^k q^{n-k}$。總和$(p+q)^n = 1$。

線性期望給μ=np，不需獨立；variance要各對covariance0才可Σpq=npq。完整binomial pmf則用了mutual independence，pairwise只夠mean／variance，不保整個分布。n0退化X0；p0退化0、p1退化n，0⁰在pmf端點依空乘積視為1。

不同p_i但mutually independent時是Poisson-binomial，mean為$\sum_i p_i$、variance為$\sum_i p_i(1-p_i)$，不應把平均p塞進binomial pmf就說相同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一張能逐項核對的binomial表

n=8,p=1/4,q=3/4，各pmf的共同分母4⁸=65536；分子$\binom{8}{k}3^{8-k}$如下。

k	0	1	2	3	4	5	6	7	8
分子	6561	17496	20412	13608	5670	1512	252	24	1

分子總65536，Σk・分子=131072給E[X]=2；Σk²・分子=360448給E[X²]=11/2，所以Var=11/2−4=3/2。

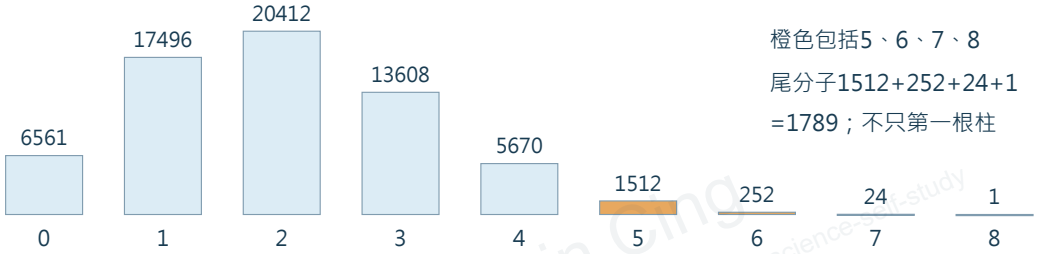
精確尾P(X≥5)=(1512+252+24+1)/65536=1789/65536≈0.027298。不是只算b(5)=1512/65536，也不是把「五次以上」漏掉k8。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一張能逐項核對的binomial表 （續）

單一機率柱與尾端總和不同：門檻5含5

每柱高度 = 分子 / 65536；n8、p1/4



橙色包括5、6、7、8

尾分子1512+252+24+1

=1789；不只第一根柱

Σ分子=65536；mean2、variance1.5；P(X≥5)=1789/65536≈0.027298。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Mode由相鄰比率定，不能直接當mean

對0<p<1、1≤k≤n，有b(k)/b(k−1)=[(n−k+1)p]/(kq)。比率>1 iff k<(n+1)p；等於1 iff k=(n+1)p。

若(n+1)p不是整數，唯一mode=floor((n+1)p)；若是整數m，m−1與m都是modes。n8,p1/4時(n+1)p=2.25，mode2，mean2恰巧相同；n4,p1/2時m=2.5，mode2；n3,p1/2時m2，modes1,2，mean1.5沒有整數機率柱。

對0≤k≤n，前述entropy/binomial coefficient界給$b(k) \le (np/k)^k(nq/(n-k))^{n-k}$，端點以連續或0⁰慣例處理。這是單一pmf上界，不是整個tail上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C.5 The tails of the binomial distribution

先對齊嚴格／非嚴格尾：整數X的P(X≥a)從ceil(a)加到n；P(X>a)從floor(a)+1起。P(X<k)（整數k）只加0..k−1，別和P(X≤k)混用。

對X~Bin(n,p)，事件X≥k意味存在一組k個指定trials全部success。每組機率p^k，共$\binom{n}{k}$組；用union bound（組與組不必互斥）得$P(X \geq k) \leq \binom{n}{k} p^k$。k0界1，k>n真事件空。左尾用failures Y=n−X~Bin(n,q)：$P(X \leq k) \leq \binom{n}{k} q^{n-k}$。

數例n8,p1/4,k5，右尾≤56/1024=7/128≈0.054688，真尾1789/65536≈0.027298。重複計入同一次出現6個以上success的outcomes，正是上界通常不相等的原因。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

左尾的geometric比率界：推導而非背公式

假設0<p<1、整數0<k<np，q=1−p。對1≤i≤k，b(i−1)/b(i)=iq/[(n−i+1)p]<kq/[(n−k)p]=ρ；因k<np，ρ<1。

由後往前，$b(k-j) < \rho^j b(k)$，j1..k。因此$P(X < k) < b(k) \sum_{j \geq 1} \rho^j = [\rho/(1-\rho)]b(k) = [kq/(np-k)]b(k)$。分母為正是前提，不可在k=np直接代入。

若0<k≤np/2，係數kq/(np−k)≤q≤1，故$L_k = P(X < k) < b(k)$，於是$L_k / L_{k+1} = L_k/(L_k + b(k)) < 1/2$。這說遠左尾再往左縮一個整數門檻時，機率至少減半。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

右尾由failures換元，門檻也要換

令Y=n−X~Bin(n,q)。當np<k<n，n−k<nq，將左尾界套到整數門檻n−k：

P(X>k)=P(Y<n−k)<[(n−k)p/(nq−(n−k))]b(n−k;n,q)=[(n−k)p/(k−np)]b(k;n,p)。

若k>(np+n)/2，則n−k<nq/2，左尾減半性給P(X>k)<(1/2)P(X>k−1)。success與failure的角色互換，不能只把原公式中的<改成>。

n8,p1/4,k4：係數(8−4)(1/4)/(4−2)=1/2，故P(X>4)<2835/65536≈0.043259；真尾1789/65536。它比本例union界7/128小，但不同參數時不必總是較緊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exponential moment把尾事件轉成非負量

令I_i為mutually independent Bernoulli trials（參數p_i），$X=\sum I_i$，$\mu=\sum p_i$。對任意λ>0及t，exp單調與Markov給$P(X\ge t)\le e^{-\lambda t}E[e^{\lambda X}]$。獨立性在下一步不可省：

$E[e^{\lambda X}]=\prod E[e^{\lambda I_i}]=\prod[1+p_i(e^{\lambda}-1)]\le \exp[\mu(e^{\lambda}-1)]$，因$1+z\le e^{z}$。故$P(X\ge t)\le \exp[\mu(e^{\lambda}-1)-\lambda t]$。

若μ>0、t>μ，對λ微分$\mu e^{\lambda}-t=0$，最小在λ=ln(t/μ)>0，二階$\mu e^{\lambda}>0$。代入得$P(X\ge t)\le e^{-\mu}(e\mu/t)^{t}$。此界允許各$p_i$不同；不需同分布，但需mutual independence。μ0時全部$I_i=0$幾乎必然，正尾直接0，不代ln(t/0)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

原書中心化r界與較精細的t界不能混用

CLRS所列較鬆形式：對r>μ>0，$P(X-\mu \geq r) \leq (e\mu/r)^r$。可直接證：$E[e^{\lambda(I_i-p_i)}] = p_i e^{\lambda(1-p_i)} + (1-p_i)e^{-\lambda p_i} \leq p_i e^{\lambda} + 1 \leq \exp(p_i e^{\lambda})$。相乘再用Markov，得$P(X-\mu \geq r) \leq \exp(\mu e^{\lambda} - \lambda r)$；取λ=ln(r/μ)即得。

本式r是**超過mean的幅度**，事件門檻是t=μ+r；前頁t則是**總success門檻**。若μ10,r40，中心化事件X≥50，較鬆bound為$(e/4)^{40} \approx 1.947 \times 10^{-7}$；精細t界則$e^{-10}(e/5)^{50} \approx 2.650 \times 10^{-18}$。兩者都是上界，不能把40與50交換後仍稱同一推導。

r只略大μ時$(e\mu/r)^r$可超1，仍合法卻無資訊，需取min(1,bound)。若是整數binomial，真sum起點為ceil(μ+r)；不把非整數門檻向下捨。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一binomial尾端比較不同保證

原創X~Bin(1000,0.01)，μ10，關心X≥50：Markov≤10/50=0.2；Chebyshev以Var9.9及偏差40給≤9.9/1600=0.0061875；中心化r40界約1.947e−7；精細t50界約2.650e−18。

精確值仍是$\sum_{k=50}^{1000} \binom{1000}{k} 0.01^k 0.99^{1000-k}$，可用相鄰比率遞推或log-pmf避免直接算大factorial後浮點溢出。這些界用到的條件不同：Markov只需非負與mean；Chebyshev需variance；最後兩個用mutual independent indicators。

若所有I_i共用一個Bernoulli(.01)，X只可能0或1000，P(X≥50)=.01，仍符合Markov，但原variance與exponential-moment乘積均不適用。相同marginals不是使用binomial／Chernoff的充分理由。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix C。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D 矩陣

Part VIII Appendix

本章定位

以entry、尺寸及field前提重建矩陣運算，完整追蹤消去、rank因式分解與inverse；證明row/column rank、det運算律與Gram正定條件，明確區分實數、複數與有限field。

前置知識

向量空間、線性映射與矩陣乘法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 使用矩陣運算與轉置。
- 辨識 rank、determinant、inverse 的關係。
- 理解 SPD 與特徵值的必要條件。
- D.1 Matrices and matrix operations
- D.2 Basic matrix properties

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D.1 Matrices and matrix operations

本附錄除另註外取輸入維度為正整數、entries在R，vector預設column；rank0的factorization允許0個中間columns／rows，空加總定為0。$A \in \mathbb{R}^{m \times n}$有m個rows、n個columns，$a_{ij}$先row再column。矩陣相等須尺寸相同且每個entry相同；矩陣加法與scalar multiplication逐entry定義。

例$A = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{bmatrix}$是2×3，$A^T = \begin{bmatrix} 1 & 0 \\ 2 & 1 \\ 0 & 1 \end{bmatrix}$是3×2，$(A^T)_{ji} = a_{ij}$。轉置不是把同一串記憶體不加定義就改形狀；row-major／column-major是儲存慣例，數學transpose由索引對應定義。

向量$x = (1, -1, 2)^T$是3×1，x^T是1×3。e_i是第i筆1、其他0的standard basis vector，維度由使用處指定；0矩陣亦須尺寸相符。scalar0與2×3零矩陣不是相同型別。

常見square結構的精確零位置

類型	n×n矩陣的entry條件
diagonal	$i \neq j \Rightarrow a_{ij} = 0$
identity I_n	對角1、其他0
tridiagonal	$\lvert i - j \rvert > 1 \Rightarrow a_{ij} = 0$
upper triangular	$i > j \Rightarrow a_{ij} = 0$
lower triangular	$i < j \Rightarrow a_{ij} = 0$
unit triangular	對應triangle且全部對角1
permutation	每個row及每個column恰一個1、其餘0
real symmetric	$A = A^T$

常見square結構的精確零位置 (續)

這些類別可重疊：identity同時diagonal、上下triangular、symmetric、permutation。tridiagonal不保symmetric，因上下對角線數值可不同。unit triangular不表示整張矩陣都是1；unit vector也不是全部entries1的向量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

乘法先核維度，再核每個收縮索引

若$A \in \mathbb{R}^{m\times n}$、$B \in \mathbb{R}^{n\times p}$，才有$C = AB \in \mathbb{R}^{m\times p}$，$c_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj}$。中間索引k被加總，外側i,j留下；不是逐entry相乘。

沿用A，取$B = \begin{bmatrix} 2 & 1 \\ -1 & 3 \\ 4 & 0 \end{bmatrix}$，則$AB = \begin{bmatrix} 0 & 7 \\ 3 & 3 \end{bmatrix}$。四筆計算依序：1・2+2・(−1)+0・4=0；1・1+2・3+0・0=7；0・2+1・(−1)+1・4=3；0・1+1・3+1・0=3。

反向BA也有定義但為3×3：$BA = \begin{bmatrix} 2 & 5 & 1 \\ -1 & 1 & 3 \\ 4 & 8 & 0 \end{bmatrix}$，與AB連尺寸都不同。即使兩者同為square也通常AB≠BA；交換律不能由scalar算術照搬。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

乘法先核維度，再核每個收縮索引 (續)

先看尺寸，再沿同一個k對齊的三項相加

A：2 rows × 3 columns

1	2	0
0	1	1

×

B：3 rows × 2 columns

2	1
-1	3
4	0

=

C=AB：2×2

0	7
3	3

C[1,2] = A的row1 · B的column2 = 1×1 + 2×3 + 0×0 = 7

黃色兩段各長3，收縮同一個k；輸出保留row1、column2，不能逐entry相乘。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

矩陣乘法的可實作版本與成本

```
MULTIPLY(A[m,n], B[n,p])
    allocate independent C[m,p]
    for i=1..m
        for j=1..p
            s = A[i,1] * B[1,j]
            for k=2..n
                s = s + A[i,k] * B[k,j]
            C[i,j] = s
    return C
```

n≥1時共mnp次scalar multiplies、mp(n−1)次scalar additions，另有mp次輸出寫入及迴圈控制。單次scalar算術O(1)的RAM模型下為Θ(mnp)；n×n為Θ(n^3)。若先s=0並做n次s+=，則多做mp次加法，是另一個精確計數但同階。

輸出C不得alias尚待讀的A/B；否則寫C可能破壞後續輸入。任意精度整數或field元素的bit成本須另計；「矩陣乘法做n^3次乘法」不是「任意bit長皆n^3時間」的保證。

結合、分配、轉置：每個等式都要尺寸相容

令A為m×n、B為n×p、C為p×q。第(i,j)筆的$(AB)C$為$\sum_{k=1}^{p}\sum_{\ell=1}^{n}a_{i\ell}b_{\ell k}c_{kj}$；有限sum換序得$\sum_{\ell=1}^{n}a_{i\ell}\sum_{k=1}^{p}b_{\ell k}c_{kj}=(A(BC))_{ij}$，故結合成立。分配律由scalar分配逐entry得；單位元須寫$I_mA=AI_n=A$。

$(AB)^T=B^TA^T$：左邊第(j,i)筆是$\sum_k a_{ik}b_{kj}$；右邊同一位置$\sum_k(B^T)_{jk}(A^T)_{ki}=\sum_k b_{kj}a_{ik}$，由scalar交換相等。矩陣乘法本身不交換，轉置後順序反轉不可漏。

若A,B都real symmetric，A+B與A−B仍對稱，但AB對稱 iff AB=BA，因$(AB)^T=BA$。不能把「兩個對稱矩陣」直接當成「其積也對稱」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ax、inner product與outer product的三個輸出型別

將A的columns記$a_1,\dots,a_n$，$Ax=\sum_j x_ja_j$是m-vector。沿用A與$x=(1,-1,2)^T$，$Ax=(1,0)^T-(2,1)^T+2(0,1)^T=(-1,1)^T$，與逐row點積相同。

同長real vectors u,v的u^Tv是scalar；uv^T是matrix。例$u=(1,2)^T$、$v=(3,-1)^T$，$u^Tv=1$，$uv^T=\begin{bmatrix}3&-1\\6&-2\end{bmatrix}$。outer product每個column為$v_ju$，故rank≤1；u,v都非零時至少一個column為非零倍u，rank=1。任一為0則rank0。

$\|x\|_2=\sqrt{x^Tx}$只對real x直接如此寫；complex需$x^*x=\sum_i|x_i|^2$，其中*是conjugate transpose，不是普通transpose。例x=[i]，$x^Tx=-1$，不是長度平方。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Permutation左乘換rows，右乘換columns

取$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$，則$Px = (x_3, x_1, x_2)^T$。若A有3個rows，PA把rows改為舊row3、row1、row2；因$(PA)_{ij} = \sum_k p_{ik} a_{kj}$，每row恰選一筆。

若M有3個columns，MP的columns則是舊column2、column3、column1，因P的三columns依序e_2, e_3, e_1。左乘與右乘的重新排列方向要從entry式或basis columns讀取，不能只記「都照3,1,2」。

不同columns of P是不同basis vectors，$P^T P = I = P P^T$，故$P^{-1} = P^T$。兩個permutation matrices相乘仍是permutation matrix，因它們的basis重排composition仍恰是一個bijection。

D.2 Basic matrix properties

vectors $v_1, \ldots, v_n$線性獨立指$\sum_j c_j v_j = 0$只可能所有$c_j = 0$；若存在非全0係數就是dependent。A的column rank是最大獨立column數，row rank類似，zero matrix rank0。

因$Ax = \sum_j x_j a_j$，columns獨立 iff Ax=0只有x=0；若有非零x使Ax=0，本文稱nonzero null vector。整個null space含0，不能把「沒有非零null vector」誤寫成「null space沒有任何向量」。

rank不是非零entries個數，也不是只數原始主對角非零數。例$M = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 0 & 1 & 1 \end{bmatrix}$有8個非零entries，但column3=column1+column2，且前兩column獨立，所以rank2；null vector為$(-1, -1, 1)^T$。

消去法的操作與完整中間狀態

三種elementary row operations都可逆：交換兩rows；一row乘非零scalar；一row加另一row的倍數。每種可表示左乘invertible elementary matrix，所以不改column之間的linear dependence；rows本身的span也不改。

對M：$R_2 \leftarrow R_2 - 2R_1$，得 $\begin{bmatrix}1&2&3\\0&0&0\\0&1&1\end{bmatrix}$；交換$R_2, R_3$，得 $\begin{bmatrix}1&2&3\\0&1&1\\0&0&0\end{bmatrix}$；$R_1 \leftarrow R_1 - 2R_2$，得$R = \begin{bmatrix}1&0&1\\0&1&1\\0&0&0\end{bmatrix}$。

R有pivots columns1,2，第三變數free，解$Rx=0$為$x_1=-x_3$、$x_2=-x_3$，所以null space為$\{t(-1,-1,1)^T : t \in \mathbb{R}\}$。非零row2個，彼此leading positions不同而獨立；非pivot column是pivot columns之和。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

消去法的操作與完整中間狀態 （續）

M=BC：中間維度2，從消去結果追回原columns

原M：3×3、rank2

1	2	3
2	4	6
0	1	1

=

B：原pivot columns

1	2
2	4
0	1

×

C：各column的座標

1	0	1
0	1	1

M的column3 = column1 + column2，所以C的column3=$(1,1)^T$。

B保留原向量；消去後的unit columns只用於讀取座標，不能直接取代B。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不預設主對角可用的Gauss–Jordan

```
R = copy(A[m,n]); r = 0; pivots = empty list
for j=1..n
    find t in r+1..m with R[t,j] != 0
    if none: continue
    r = r+1; swap rows r,t
    divide row r by R[r,j]
    for i=1..m with i != r
        a = R[i,j]
        row i = row i - a * row r
    append j to pivots
return R, pivots
```

每步pivot前，前r個pivot columns已是distinct unit columns；未用rows在它們上皆0。新pivot取剩餘row、normalization後消去其他rows，不破壞舊pivot columns，因新row在舊pivot位置0。若剩餘rows該column全0才跳過。迴圈有限，得到r個pivots與zero rows，field operations O(mn・min(m,n))。

精確算術中!=0依field判定；浮點則需pivoting與尺度／容許值，不能把固定|pivot|<10^{-12}當成對所有矩陣普遍正確的rank定義。

為什麼row rank等於column rank

設R=EA為row-reduced形式，E是invertible。E乘在每個column上是同一可逆線性映射，所以columns的相依係數在A與R相同；R恰r個pivot columns獨立，其他column是它們線性組合，故column rank(A)=r。

row operations每步以可逆組合改rows，因此row span(A)=row span(R)。R非零rows r個，依pivot位置可逐係數消去證獨立，故row rank(A)=r。於是可統稱rank(A)，介於0與min(m,n)。

nullity=n−r：pivot variables由n−r個free variables唯一線性決定；每次只令一個free variable1、其餘0得到一個null basis。它們獨立（看free coordinates），也生成所有null solutions，完成dimension計數，不只在M一例成立。

Rank factorization與乘積rank界

取A原本的r個pivot columns組成B（m×r），每個原column在B下的coordinates排成C（r×n），則A=BC。B columns獨立；C含r×r identity位置。反向若A=UV且U只有s個columns，A的columns全在span(U)，rank(A)≤s，所以r也是最小可因式分解的中間維度。

M的原pivot columns給$B=\begin{bmatrix}1&2\\2&4\\0&1\end{bmatrix}$，$C=\begin{bmatrix}1&0&1\\0&1&1\end{bmatrix}$，逐column乘回恰M。不可拿R的pivot columns當成原M的columns，因row operation改變了向量本身。

對相容A,D，若$A=B_rC_r$，則$AD=B_r(C_rD)$，rank(AD)≤rank(A)；若$D=U_sV_s$，$AD=(AU_s)V_s$，亦≤rank(D)。故rank(AD)≤min(rank A,rank D)。若左因子P可逆，配合A=P⁻¹(PA)反向界得到rank(PA)=rank(A)；右側可逆同理。

Inverse存在與唯一性的全部條件

n×n的inverse B要求AB=I=BA。若B,C皆inverse，B=B(AC)=(BA)C=C，故唯一。可逆A,B有(AB)⁻¹=B⁻¹A⁻¹；左右各乘檢查都回I。$(A^{-1})^T=(A^T)^{-1}$亦由轉置兩個inverse等式得。

square A full rank iff無非零null vector，等價linear map x↦Ax injective。n個獨立columns在n維空間成basis，故每e_j有唯一解$Ab_j=e_j$；把b_j排成B得AB=I。再由A(BA−I)=0及A injective，逐column得BA=I，故full rank iff invertible。反向若Ax=0，乘inverse即x=0。

rectangular時只一側inverse不保另一側：A=[1,0]、$B=(1,0)^T$有AB=[1]，但BA=diag(1,0)≠I₂。不能把square定理不改尺寸套用。

一個可逐步核對的inverse

取$H=\begin{bmatrix}2&1\\3&2\end{bmatrix}$，從augmented matrix[H|I]做同一組row operations：

$$\left[\begin{array}{cc|cc}2&1&1&0\\3&2&0&1\end{array}\right]\longrightarrow\left[\begin{array}{cc|cc}2&1&1&0\\0&1/2&-3/2&1\end{array}\right]\longrightarrow\left[\begin{array}{cc|cc}2&1&1&0\\0&1&-3&2\end{array}\right].$$

依序為$R_2-(3/2)R_1$、R_2乘2；再R_1-R_2，最後R_1除2，得到$[I\mid H^{-1}]$，$H^{-1}=\begin{bmatrix}2&-1\\-3&2\end{bmatrix}$。檢查$HH^{-1}=\begin{bmatrix}4-3&-2+2\\6-6&-3+4\end{bmatrix}=I$，反向乘也為I。

消去法對左半做E，右半同時變E；當EH=I，E就是inverse。若左半無法產生n個pivots，不能假裝最後右半就是inverse。

Determinant、minor與cofactor

對square A，minor matrix A[ij]刪row i及column j；cofactor為$C_{ij}=(-1)^{i+j}\det A[ij]$。n1時det[a]=a；若採0×0 convention，det(empty)=1以統一cofactor式。沿row i展開$\det A=\sum_{j=1}^{n}a_{ij}C_{ij}$。

原創$N=\begin{bmatrix}2&1&0\\1&3&2\\0&-1&1\end{bmatrix}$，第一row signs為+−+，所以detN=2・(3・1−2・(−1))

−1・(1・1−2・0)+0=10−1=9。minor是矩陣；其det是scalar；cofactor再加正負號，三者不混同。

一般permutation式為$\det A=\sum_{\sigma\in S_n}\operatorname{sgn}(\sigma)\prod_{i=1}^{n}a_{i,\sigma(i)}$。每項在每row／column各選一筆，sign由permutation parity決定；按固定row選到的column分組便得到cofactor展開。

Determinant運算律的原因

permutation式對每個column各自linear；兩columns相同時，把交換這兩column的permutations配對，項互消，故det0；交換兩columns使sign反號。對rows同理，且σ↦σ⁻¹使$\det(A^T) = \det(A)$。因此一row乘α，det乘α；一row加另一row的倍數，multilinearity加出的重複row項為0，det不變。

任一對columns交替multilinear的scalar函數F，把每column依basis e_i展開；重複basis index的項為0，留下各permutation，且$F(e_{\sigma(1)}, \ldots, e_{\sigma(n)}) = sgn(\sigma)F(I)$，所以F(B)=F(I)det(B)。

固定A，令F(B)=det(AB)；AB每column為A乘該column，故F仍交替multilinear，F(I)=det(A)。由唯一展開得det(AB)=det(A)det(B)。這證明不要求A預先可逆；若det A=0，F亦處處為0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Determinant與singularity，不只背2×2公式

upper triangular的非零permutation項須σ(i)≥i對每i成立；但Σσ(i)=Σi，所以只能全部σ(i)=i，故det是對角乘積。lower triangular同理，或轉置化成upper。row echelon若有zero row，det0；若全部n個pivots非零，triangular對角乘積非零。

消去過程的row swap、非零scaling只乘非零因子，row-add不改det，故原det為0 iff echelon缺pivot iff rank<n iff singular。於是det≠0是可逆判準，不代表數值上well-conditioned。

例diag(1,ε)，ε≠0時det ε且可逆；ε很小仍是精確full rank，inverse含1/ε，誤差可能放大。實際解Ax=b通常以穩定factorization求解，不必先顯式形成inverse；數學存在性與數值可靠性是不同問題。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Real symmetric正定與Gram的一般證明

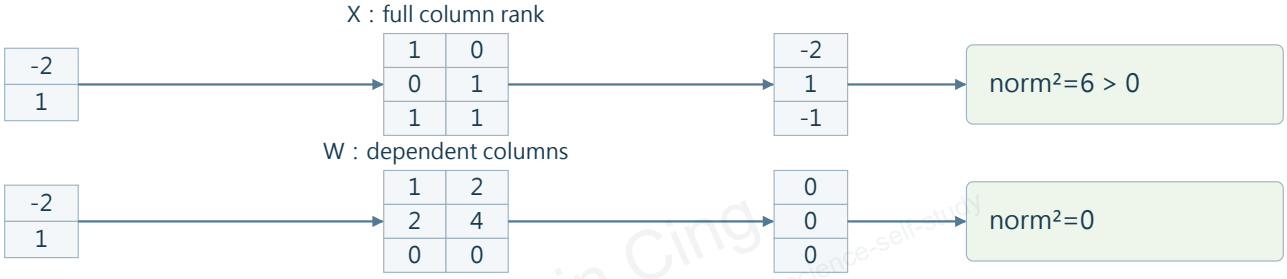
本附錄正定採**real symmetric** S且對每個非零real x有$x^T S x > 0$；semidefinite把>改≥。若不要求對稱，quadratic form只看$(S + S^T)/2$，不能由$x^T S x > 0$自動推出$S = S^T$。

對任意real m×n A，$G = A^T A$為symmetric，且$x^T G x = (Ax)^T (Ax) = \|Ax\|_2^2 \geq 0$。等於0 iff Ax=0。因此G正定 iff A無非零null vector iff A full column rank；若n>m就不可能。只說「A非零」不足。

原創$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$，$X^T X = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$；對$x = (s,t)^T$，$x^T G x = s^2 + t^2 + (s+t)^2 > 0$除非s=t=0。對$W = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 0 & 0 \end{bmatrix}$，$W^T W = \begin{bmatrix} 5 & 10 \\ 10 & 20 \end{bmatrix}$，取$x = (-2,1)^T$使Wx=0，只有semidefinite。

Real symmetric正定與Gram的一般證明 (續)

比較$x^T A^T Ax = \text{norm}(Ax)^2$：同一輸入，兩種rank



同一非零x=(−2,1)ᵀ。一般正定結論要證全部非零x；W只需一個null vector即可反證。

複數與有限field：哪些定理仍可用

前述加乘、inverse、rank及det的field論證可改用C或prime field F_p；但R的正負及euclidean內積不能照搬。composite模數的ring不一定每非零元素可除，例如mod6的2非零卻無inverse，所以pivot normalization與field rank定理須重查。

complex版本以A^*為conjugate transpose，Hermitian要求$S^* = S$。Gram A^*A semidefinite，full column rank時positive definite，因$x^*A^*Ax = \sum_i |(Ax)_i|^2$。普通$A^TA$不保此結論：A=[i]時$A^TA = [-1]$，$A^*A = [1]$。

對real或complex square A，分別有$\det(A^TA) = (\det A)^2$，與$\det(A^*A) = |\det A|^2 \geq 0$；第一式只在real情況可推≥0。代數中的transpose恆等式和內積中的conjugation須依field清楚區分。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Appendix D。

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study